

May 2026

Contents

Contents	i
1 Changes	1
1.1 .codespellignore	1
1.2 .github/workflows/labels_from_comment.yml	1
1.3 Physlib.lean	1
1.4 Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean	2
1.5 Physlib/ClassicalMechanics/DampedHarmonicOscillator/Solution.lean	10
1.6 Physlib/ClassicalMechanics/FreeParticle/Basic.lean	22
1.7 Physlib/ClassicalMechanics/HarmonicOscillator/Solution.lean	26
1.8 Physlib/ClassicalMechanics/OrbitalMechanics/VisViva.lean	27
1.9 Physlib/Electromagnetism/Current/InfiniteWire.lean	28
1.10 Physlib/Electromagnetism/Distributional/Basic.lean	29
1.11 Physlib/Electromagnetism/Distributional/Dynamics/CurrentDensity.lean	33
1.12 Physlib/Electromagnetism/Distributional/Dynamics/IsExtrema.lean	35
1.13 Physlib/Electromagnetism/Distributional/Dynamics/KineticTerm.lean	41
1.14 Physlib/Electromagnetism/Distributional/Dynamics/Lagrangian.lean	47
1.15 Physlib/Electromagnetism/Distributional/ElectricField.lean	52
1.16 Physlib/Electromagnetism/Distributional/FieldStrength.lean	54
1.17 Physlib/Electromagnetism/Distributional/MagneticField.lean	61
1.18 Physlib/Electromagnetism/Distributional/ScalarPotential.lean	65
1.19 Physlib/Electromagnetism/Distributional/VectorPotential.lean	66
1.20 Physlib/Electromagnetism/Dynamics/IsExtrema.lean	68
1.21 Physlib/Electromagnetism/Dynamics/KineticTerm.lean	68
1.22 Physlib/Electromagnetism/Kinematics/EMPotential.lean	70
1.23 Physlib/Electromagnetism/Kinematics/ElectricField.lean	75
1.24 Physlib/Electromagnetism/Kinematics/FieldStrength.lean	77
1.25 Physlib/Electromagnetism/Kinematics/MagneticField.lean	82
1.26 Physlib/Electromagnetism/Kinematics/ScalarPotential.lean	83
1.27 Physlib/Electromagnetism/Kinematics/VectorPotential.lean	84
1.28 Physlib/Electromagnetism/PointParticle/OneDimension.lean	88
1.29 Physlib/Electromagnetism/PointParticle/ThreeDimension.lean	88
1.30 Physlib/FluidDynamics/FluidState.lean	88
1.31 Physlib/FluidDynamics/NavierStokes/Basic.lean	91

1.32	Physlib/FluidDynamics/NavierStokes/Continuity.lean	93
1.33	Physlib/FluidDynamics/NavierStokes/Momentum.lean	94
1.34	Physlib/Mathematics/Calculus/ParametricIntegration.lean	101
1.35	Physlib/Meta/Basic.lean	104
1.36	Physlib/Meta/TODO/Global.lean	104
1.37	Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Basic.lean	104
1.38	Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/FamilyMaps.lean	106
1.39	Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Permutations.lean	106
1.40	Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Plus.lean	106
1.41	Physlib/Particles/StandardModel/AnomalyCancellation/Basic.lean	106
1.42	Physlib/Particles/StandardModel/AnomalyCancellation/FamilyMaps.lean	106
1.43	Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/Lemmas.lean	106
1.44	Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/Linear.lean	106
1.45	Physlib/Particles/StandardModel/AnomalyCancellation/Permutations.lean	106
1.46	Physlib/Particles/StandardModel/HiggsBoson/Basic.lean	108
1.47	Physlib/Particles/StandardModel/Representations.lean	108
1.48	Physlib/Particles/SuperSymmetry/MSSMnu/AnomalyCancellation/Basic.lean	108
1.49	Physlib/Particles/SuperSymmetry/MSSMnu/AnomalyCancellation/LineY3B3.lean	108
1.50	Physlib/Particles/SuperSymmetry/MSSMnu/AnomalyCancellation/OrthogY3B3.lean	108
1.51	Physlib/Particles/SuperSymmetry/MSSMnu/AnomalyCancellation/OrthogY3B3.lean	108
1.52	Physlib/Particles/SuperSymmetry/MSSMnu/AnomalyCancellation/OrthogY3B3.lean	108
1.53	Physlib/Particles/SuperSymmetry/MSSMnu/AnomalyCancellation/Permutations.lean	108
1.54	Physlib/QFT/QED/AnomalyCancellation/Basic.lean	110
1.55	Physlib/QuantumMechanics/DDimensions/Basic.lean	110
1.56	Physlib/QuantumMechanics/DDimensions/Hydrogen/Basic.lean	114
1.57	Physlib/QuantumMechanics/DDimensions/Hydrogen/LaplaceRungeLenzVector.lean	114
1.58	Physlib/QuantumMechanics/DDimensions/Operators/Commutation.lean	115
1.59	Physlib/QuantumMechanics/DDimensions/Operators/Covariance.lean	115
1.60	Physlib/QuantumMechanics/DDimensions/Operators/Momentum.lean	118
1.61	Physlib/QuantumMechanics/DDimensions/Operators/Multiplication.lean	119
1.62	Physlib/QuantumMechanics/DDimensions/Operators/Position.lean	128
1.63	Physlib/QuantumMechanics/DDimensions/Operators/StateObservables/Expected.lean	128
1.64	Physlib/QuantumMechanics/DDimensions/Operators/StateObservables/IsEig.lean	128
1.65	Physlib/QuantumMechanics/DDimensions/Operators/StateObservables/Variable.lean	128
1.66	Physlib/QuantumMechanics/DDimensions/Operators/Unbounded.lean	140
1.67	Physlib/QuantumMechanics/DDimensions/Operators/Uncertainty.lean	156
1.68	Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Basic.lean	162
1.69	Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/PolyBddSchwarz.lean	162
1.70	Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/SchwartzSubmodule.lean	162
1.71	Physlib/QuantumMechanics/FiniteTarget/HilbertSpace.lean	165
1.72	Physlib/Relativity/LorentzAlgebra/Basis.lean	166
1.73	Physlib/Relativity/Tensors/Basic.lean	167
1.74	Physlib/Relativity/Tensors/Constructors.lean	169
1.75	Physlib/Relativity/Tensors/Contraction/Basic.lean	169
1.76	Physlib/Relativity/Tensors/Contraction/Basis.lean	170
1.77	Physlib/Relativity/Tensors/Contraction/Products.lean	171

1.78	Physlib/Relativity/Tensors/Contraction/Pure.lean	171
1.79	Physlib/Relativity/Tensors/Contraction/SuccSuccAbove.lean	172
1.80	Physlib/Relativity/Tensors/Evaluation.lean . . .	181
1.81	Physlib/Relativity/Tensors/RealTensor/Basic.lean	181
1.82	Physlib/Relativity/Tensors/RealTensor/Metrics/Basic.lean	183
1.83	Physlib/Relativity/Tensors/RealTensor/ToComplex.lean	183
1.84	Physlib/Relativity/Tensors/RealTensor/Vector/Basic.lean	183
1.85	Physlib/Relativity/Tensors/TensorSpecies/Basic.lean	184
1.86	Physlib/Relativity/Tensors/Tensorial.lean	186
1.87	Physlib/SpaceAndTime/Space/Derivatives/Curl.lean	187
1.88	Physlib/SpaceAndTime/Space/Derivatives/DerivativeIndex.lean	188
1.89	Physlib/SpaceAndTime/Space/Derivatives/Grad.lean	190
1.90	Physlib/SpaceAndTime/Space/Derivatives/MatrixDiv.lean	192
1.91	Physlib/SpaceAndTime/Space/Norm.lean	195
1.92	Physlib/SpaceAndTime/Space/Time/TimeSlice.lean .	195
1.93	Physlib/StatisticalMechanics/MicroCanonicalEnsemble/Basic.lean	196
1.94	Physlib/StatisticalMechanics/MicroCanonicalEnsemble/IdealGas.lean	197
1.95	Physlib/StatisticalMechanics/MicroCanonicalEnsemble/ThermoQuantities.lean	199
1.96	Physlib/Units/Examples.lean	205
1.97	QuantumInfo.lean	205
1.98	QuantumInfo/Capacity/Capacity.lean	206
1.99	QuantumInfo/Channels/Bundled.lean	207
1.100	QuantumInfo/Channels/CPTP.lean	207
1.101	QuantumInfo/Channels/DegradableOrder.lean . . .	208
1.102	QuantumInfo/Channels/Dual.lean	208
1.103	QuantumInfo/Channels/MatrixMap.lean	211
1.104	QuantumInfo/Channels/Pinching.lean	213
1.105	QuantumInfo/Channels/Unbundled.lean	214
1.106	QuantumInfo/ClassicalInfo/Channel.lean	217
1.107	QuantumInfo/ClassicalInfo/Prob.lean	217
1.108	QuantumInfo/Entropy/Axiomatized/Defs.lean . . .	217
1.109	QuantumInfo/Entropy/Axiomatized/Renyi.lean . .	217
1.110	QuantumInfo/Entropy/DPI.lean	217
1.111	QuantumInfo/Entropy/Relative.lean	249
1.112	QuantumInfo/Entropy/SSA.lean	253
1.113	QuantumInfo/Entropy/VonNeumann.lean	253
1.114	QuantumInfo/ForMathlib/ComplexLaplaceTransform.lean	253
1.115	QuantumInfo/ForMathlib/HayataGroup/TraceInequality/BlockDiagonal.lean	261
1.116	QuantumInfo/ForMathlib/HayataGroup/TraceInequality/GeneralizedPerspectiveFunct	
1.117	QuantumInfo/ForMathlib/HayataGroup/TraceInequality/HilbertSchmidtOperatorSpace	
1.118	QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequality.lean	
1.119	QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequalityIIn	
1.120	QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequalityIVt	
1.121	QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LiebAndoTrace.lean	331
1.122	QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LownerHeinzCore.lean	364
1.123	QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LownerHeinzTheorem.lean	417

1.124	QuantumInfo/ForMathlib/HayataGroup/TraceInequality/OperatorGeometricM	
1.125	QuantumInfo/ForMathlib/HayataGroup/TraceInequality/PORTING_NOTES.txt4	
1.126	QuantumInfo/ForMathlib/HermitianMat.lean	427
1.127	QuantumInfo/ForMathlib/HermitianMat/Basic.lean	427
1.128	QuantumInfo/ForMathlib/HermitianMat/CFC.lean . .	427
1.129	QuantumInfo/ForMathlib/HermitianMat/CompoundMatrix.lean	427
1.130	QuantumInfo/ForMathlib/HermitianMat/Jordan.lean	429
1.131	QuantumInfo/ForMathlib/HermitianMat/LiebConcavity.lean	429
1.132	QuantumInfo/ForMathlib/HermitianMat/Order.lean	441
1.133	QuantumInfo/ForMathlib/HermitianMat/Peierls.lean	442
1.134	QuantumInfo/ForMathlib/HermitianMat/Proj.lean .	447
1.135	QuantumInfo/ForMathlib/HermitianMat/Rpow.lean .	449
1.136	QuantumInfo/ForMathlib/HermitianMat/Schatten.lean	464
1.137	QuantumInfo/ForMathlib/HermitianMat/Unitary.lean	465
1.138	QuantumInfo/ForMathlib/Majorization.lean	466
1.139	QuantumInfo/ForMathlib/Matrix.lean	466
1.140	QuantumInfo/ForMathlib/MatrixNorm/TraceNorm.lean	467
1.141	QuantumInfo/ForMathlib/SionMinimax.lean	475
1.142	QuantumInfo/ForMathlib/Unitary.lean	475
1.143	QuantumInfo/Measurements/POVM.lean	475
1.144	QuantumInfo/Operators/Unitary.lean	476
1.145	QuantumInfo/Regularized.lean	476
1.146	QuantumInfo/ResourceTheory/FreeState.lean . . .	477
1.147	QuantumInfo/ResourceTheory/HypothesisTesting.lean	477
1.148	QuantumInfo/ResourceTheory/ResourceTheory.lean	478
1.149	QuantumInfo/ResourceTheory/SteinsLemma.lean . .	478
1.150	QuantumInfo/States/Ensemble.lean	478
1.151	QuantumInfo/States/Entanglement.lean	478
1.152	QuantumInfo/States/Mixed/Fidelity.lean	479
1.153	QuantumInfo/States/Mixed/MState.lean	482
1.154	QuantumInfo/States/Mixed/TraceDistance.lean . .	483
1.155	QuantumInfo/States/Pure/BargmannInvariant.lean	483
1.156	QuantumInfo/States/Pure/BlochSphere.lean	486
1.157	QuantumInfo/States/Pure/Braket.lean	488
1.158	QuantumInfo/States/Pure/Qubit.lean	490
1.159	docs/WildeTODO.md	490
1.160	docs/references.bib	490
1.161	scripts/MetaPrograms/TODO_to_yaml.lean	491
1.162	scripts/MetaPrograms/sorry_lint.lean	491
1.163	scripts/MetaPrograms/spellingWords.txt	491
1.164	scripts/lint_all.lean	492

1 Changes

1.1 .codespellignore

hAA
SINIC

1.2 .github/workflows/labels_from_comment.yml

```
labelArray=("awaiting-author" "blocked-by-mathlib-PR" "blocked-by-PR" "documentation" "easy" "help-wanted" "merge-conflict" "new-contributor" "please-adopt" "RFC" "WIP" "t-classical-mechanics" "t-condensed-matter" "t-cosmology" "t-electromagnetism" "t-mathematics" "t-meta" "t-optics" "t-particles" "t-quantum-field-theory" "t-quantum-info" "t-quantum-mechanics" "t-relativity" "t-space-time" "t-statistical-mechanics" "t-string-theory" "t-thermodynamics" "t-units")
```

1.3 Physlib.lean

```
public import Physlib.ClassicalMechanics.DampedHarmonicOscillator.Solution
public import Physlib.ClassicalMechanics.FreeParticle.Basic
public import Physlib.ClassicalMechanics.OrbitalMechanics.VisViva
public import Physlib.Electromagnetism.Distributional.Basic
public import Physlib.Electromagnetism.Distributional.Dynamics.CurrentDensity
public import Physlib.Electromagnetism.Distributional.Dynamics.IsExtrema
public import Physlib.Electromagnetism.Distributional.Dynamics.KineticTerm
public import Physlib.Electromagnetism.Distributional.Dynamics.Lagrangian
public import Physlib.Electromagnetism.Distributional.ElectricField
public import Physlib.Electromagnetism.Distributional.FieldStrength
public import Physlib.Electromagnetism.Distributional.MagneticField
public import Physlib.Electromagnetism.Distributional.ScalarPotential
public import Physlib.Electromagnetism.Distributional.VectorPotential
public import Physlib.FluidDynamics.FluidState
```

```

public import Physlib.FluidDynamics.NavierStokes.Basic
public import Physlib.FluidDynamics.NavierStokes.Continuity
public import Physlib.FluidDynamics.NavierStokes.Momentum
public import Physlib.Mathematics.Calculus.ParametricIntegration
public import Physlib.QuantumMechanics.DDimensions.Basic
public import Physlib.QuantumMechanics.DDimensions.Operators.Covariance
public import Physlib.QuantumMechanics.DDimensions.Operators.Multiplication
public import Physlib.QuantumMechanics.DDimensions.Operators.StateObservabl
public import Physlib.QuantumMechanics.DDimensions.Operators.StateObservabl
public import Physlib.QuantumMechanics.DDimensions.Operators.StateObservabl
public import Physlib.QuantumMechanics.DDimensions.Operators.Uncertainty
public import Physlib.Relativity.Tensors.Contraction.SuccSuccAbove
public import Physlib.SpaceAndTime.Space.Derivatives.DerivativeIndex
public import Physlib.SpaceAndTime.Space.Derivatives.MatrixDiv
public import Physlib.StatisticalMechanics.MicroCanonicalEnsemble.Basic
public import Physlib.StatisticalMechanics.MicroCanonicalEnsemble.IdealGas
public import Physlib.StatisticalMechanics.MicroCanonicalEnsemble.ThermoQua

```

1.4 Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.1

Copyright (c) 2026 Nicola Bernini. All rights reserved.

Authors: Nicola Bernini, Florian Wiesner

```
public import Physlib.ClassicalMechanics.HarmonicOscillator.Basic
```

```
# The damped harmonic oscillator
```

The damped harmonic oscillator is a classical mechanical system consisting under a restoring force $-kx$ and a damping force $-\gamma \dot{x}$, where k is constant, γ is the damping coefficient, x is the position, and $\dot{x}$ is the velocity.

- ****Underdamped**** ($\gamma^2 < 4 * m * k$): oscillatory motion with exponentially decaying amplitude.
- ****Critically damped**** ($\gamma^2 = 4 * m * k$): fastest return to equilibrium without oscillation.
- ****Overdamped**** ($4 * m * k < \gamma^2$): slow return to equilibrium without oscillation.

In this file, the position and velocity both have type `EuclideanSpace` $\mathbb{R}$ (For a one-dimensional configuration space and its tangent space are both isomorphic to one-dimensional Euclidean space. A more geometric formalization should represent the configuration space and its tangent bundle directly.

The key results in the study of the classical damped harmonic oscillator are summarized in the `Basic` module:

- `DampedHarmonicOscillator` contains the input data to the problem.
- `EquationOfMotion` defines the damped oscillator equation $m \ddot{x} + \gamma \dot{x} + kx = 0$.
- `energy_dissipation_rate` computes the rate at which damping removes mechanical energy.
- `IsUnderdamped`, `IsCriticallyDamped`, and `IsOverdamped` define the three regimes.

1.4.

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/BASIC.LEAN

regimes from the discriminant $\gamma^2 - 4 * m * k$.

- `angularFrequency` selects the real frequency parameter from the damping regime.
- `toUndamped\_equationOfMotion` relates the damped and undamped equations of motion the damping coefficient is zero.

In the `Solution` module:

- `InitialConditions` contains the initial position and velocity.
- `trajectory` gives the explicit solution selected from the damping regime.
- A. The input data
 - B. The equation of motion and energy dissipation
 - B.1. The equation of motion
 - B.2. Energy dissipation
 - C. Newton's second law
 - C.1. The force
 - C.2. Equation of motion if and only if Newton's second law
 - D. Damping regimes
 - E. To undamped oscillator

open Space

open InnerProductSpace

open MeasureTheory

TODO "Create a new file for the geometric model which properly models the position as a configuration space and velocity as its tangent space, see the HarmonicOscillator file."

A. The input data

We start by defining a structure containing the input data of the damped harmonic oscillator. The mass `m` and spring constant `k` are inherited from `HarmonicOscillator`; this file defines the damping coefficient `γ`.

/-- The classical damped harmonic oscillator is specified by a mass `m`, a spring constant `k`, and a damping coefficient `γ`.

The mass and spring constant are inherited from `HarmonicOscillator` and are positive. The damping coefficient is assumed to be nonnegative. -/

@[ext]

structure DampedHarmonicOscillator extends HarmonicOscillator where

/-- The damping coefficient is nonnegative. -/

The mass/spring nonzero lemmas, the natural angular frequency, and the undamped energy are inherited from `HarmonicOscillator`.

B. The equation of motion and energy dissipation

B.1. The equation of motion

/-- The equation of motion for the damped harmonic oscillator:

$m \ddot{x} + \gamma \dot{x} + k x = 0$. -/

noncomputable def EquationOfMotion (x_t : Time → EuclideanSpace ℝ (Fin 1)) : Prop :=
 $\forall t : \text{Time}, S.m \cdot \partial_t (\partial_t x_t) t + S.\gamma \cdot \partial_t x_t t + S.k \cdot x_t t = 0$

B.2. Energy dissipation

The damped oscillator inherits the mechanical energy from the undamped harmonic oscillator. Along a solution of the damped equation of motion, that energy decreases at a rate

```

proportional to  $-\gamma \|\dot{x}\|^2$ .
-/
/-- Along a smooth solution of the damped equation of motion, the derivative
mechanical energy is  $-\gamma \|\dot{x}\|^2$ . -/
lemma energy_dissipation_rate (x_t : Time → EuclideanSpace ℝ (Fin 1)) (t : Time)
  (h1 : S.EquationOfMotion x_t)
  (hx : ContDiff ℝ ∞ x_t) :
  ∂_t (S.energy x_t) t = - S.γ * ∂_t x_t t, ∂_t x_t t ⊔_ℝ := by
  rw [S.energy_deriv x_t hx]
  simp only
  have heom := h1 t
  have hforce : S.m • ∂_t (∂_t x_t) t + S.k • x_t t = - S.γ • ∂_t x_t t := by
    have hsum : (S.m • ∂_t (∂_t x_t) t + S.k • x_t t) + S.γ • ∂_t x_t t = 0 := by
      simpa [add_assoc, add_left_comm, add_comm] using heom
    simpa [neg_smul] using eq_neg_of_add_eq_zero_left hsum
  rw [hforce]
  simp [inner_smul_right]

/-- If  $\gamma < 0$  and the velocity is nonzero at a time, the mechanical energy
decreasing at that time. -/
lemma energy_not_conserved (x_t : Time → EuclideanSpace ℝ (Fin 1)) (t : Time)
  (h1 : S.EquationOfMotion x_t) (hx : ContDiff ℝ ∞ x_t) (hdx : ∂_t x_t t ≠ 0) :
  ∂_t (S.energy x_t) t < 0 := by
  rw [energy_dissipation_rate S x_t t h1 hx]
  rw [neg_mul]
  exact neg_neg_of_pos (mul_pos hy (real_inner_self_pos.mpr hdx))
/-!
## C. Newton's second law
We define the force of the damped oscillator, and show that the equation of
motion is equivalent to Newton's second law.
/-!
### C.1. The force
We define the force of the damped oscillator as  $-kx - \gamma v$ .
-/
/-- The force of the damped harmonic oscillator at a given position and time
/
noncomputable def force (S : DampedHarmonicOscillator)
  (x_t : Time → EuclideanSpace ℝ (Fin 1)) (t : Time) :
  EuclideanSpace ℝ (Fin 1) := - S.k • x_t t - S.γ • ∂_t x_t t
/-!
### C.2. Equation of motion if and only if Newton's second law
We show that the equation of motion is equivalent to Newton's second law.
-/
lemma equationOfMotion_iff_newtons_2nd_law (x_t : Time → EuclideanSpace ℝ (Fin 1))
  (S : DampedHarmonicOscillator) :
  S.EquationOfMotion x_t ↔
  (∀ t : Time, S.m • ∂_t (∂_t x_t) t = force S x_t t) := by

```

1.4.

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/BASIC.LEAN

```

simp only [EquationOfMotion, force]
constructor
· intro h t
  have h' :
     $S.m \cdot \partial_t (\partial_t x_t) t + (S.\gamma \cdot \partial_t x_t t + S.k \cdot x_t t) = 0$  := by
    simpa [add_assoc] using h t
  have ha :
     $S.m \cdot \partial_t (\partial_t x_t) t = -(S.\gamma \cdot \partial_t x_t t + S.k \cdot x_t t)$  :=
    eq_neg_of_add_eq_zero_left h'
  simpa [sub_eq_add_neg, neg_add, add_comm] using ha
· intro h t
  rw [h t]
  module
  ## D. Damping regimes
  The sign of the discriminant  $\gamma^2 - 4 * m * k$  separates the underdamped, critically
  damped, and overdamped regimes. We also define the decay rate and the regime-
  selected
  real frequency that appears in the explicit solution formulas.
  /-- The exponential decay rate  $\gamma / (2 * m)$ . -/
  noncomputable def decayRate :  $\mathbb{R}$  := S. $\gamma$  / (2 * S.m)

  /-- The system is overdamped when  $4mk < \gamma^2$ . -/
  def IsOverdamped : Prop := 0 < S.discriminant

  /-- The system is undamped when  $\gamma = 0$ . -/
  def IsUndamped : Prop := S. $\gamma$  = 0

  /-- The real frequency selected by the damping regime.

  In the underdamped regime this is the oscillation frequency. In the critically damped
  regime it is 0. In the overdamped regime this is the real split rate between the two
  roots. -/
  noncomputable def angularFrequency :  $\mathbb{R}$  := by
    classical
    exact
    if S.IsUnderdamped then
       $\sqrt{-S.discriminant} / (2 * S.m)$ 
    else if S.IsCriticallyDamped then
      0
    else
       $\sqrt{S.discriminant} / (2 * S.m)$ 

  /-- The relationship between the discriminant, decay rate, and natural angular frequency.
  /
  lemma discriminant_eq_four_mul_m_sq_mul_decayRate_sq_sub_omega_sq :
     $S.discriminant = 4 * S.m^2 * (S.decayRate^2 - S.\omega^2)$  := by

```

```

    rw [discriminant, decayRate, S.ω_sq]
    field_simp [S.m_ne_zero]
    ring

/-- The decay rate is nonnegative. -/
lemma decayRate_nonneg : 0 ≤ S.decayRate := by
  rw [decayRate]
  exact div_nonneg S.γ_nonneg (by nlinarith [S.m_pos])

/-- An undamped oscillator lies in the underdamped regime. -
/
lemma isUnderdamped_of_gamma_eq_zero (hγ : S.γ = 0) : S.IsUnderdamped := by
  rw [IsUnderdamped, discriminant_eq_four_mul_m_sq_mul_decayRate_sq_sub_ω_sq]
  rw [hγ]
  ring_nf
  nlinarith [sq_pos_of_pos S.m_pos, sq_pos_of_pos S.ω_pos]

/-- An underdamped system has decay rate less than the natural frequency. -
/
lemma isUnderdamped_decayRate (hS : S.IsUnderdamped) : S.decayRate < S.ω :=
  rw [IsUnderdamped] at hS
  rw [discriminant_eq_four_mul_m_sq_mul_decayRate_sq_sub_ω_sq] at hS
  have hm_sq_pos : 0 < 4 * S.m^2 := by
    have hsq : 0 < S.m^2 := sq_pos_of_pos S.m_pos
    nlinarith
  have hsq : S.decayRate^2 < S.ω^2 := by
    nlinarith
  nlinarith [S.decayRate_nonneg, S.ω_pos]

/-- A critically damped system has decay rate equal to the natural frequency. -
/
lemma isCriticallyDamped_decayRate (hS : S.IsCriticallyDamped) : S.ω = S.decayRate :=
  rw [IsCriticallyDamped] at hS
  rw [discriminant_eq_four_mul_m_sq_mul_decayRate_sq_sub_ω_sq] at hS
  have hm_sq_ne_zero : 4 * S.m^2 ≠ 0 := by
    have hm_sq_pos : 0 < 4 * S.m^2 := by
      have hsq : 0 < S.m^2 := sq_pos_of_pos S.m_pos
      nlinarith
    exact ne_of_gt hm_sq_pos
  have hsq : S.decayRate^2 = S.ω^2 := by
    have hsub : S.decayRate^2 - S.ω^2 = 0 := by
      exact (mul_eq_zero.mp hS).resolve_left hm_sq_ne_zero
    linarith
  nlinarith [S.decayRate_nonneg, S.ω_pos]

/-- The damping coefficient is twice mass times the decay rate. -

```

1.4.

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/BASIC.LEAN

```
/
lemma gamma_eq_two_mul_m_mul_decayRate : S.γ = 2 * S.m * S.decayRate := by
  rw [decayRate]
  field_simp [S.m_ne_zero]

/-- The spring constant is `m * ω^2`. -/
lemma k_eq_m_mul_ω_sq : S.k = S.m * S.ω^2 := by
  rw [S.ω_sq]
  field_simp [S.m_ne_zero]

/-- In the critically damped regime, `k = m * decayRate^2`. -
/
lemma k_eq_m_mul_decayRate_sq_of_criticallyDamped (hS : S.IsCriticallyDamped) :
  S.k = S.m * S.decayRate^2 := by
  have hwa : S.ω = S.decayRate := S.isCriticallyDamped_decayRate hS
  have hwsq : S.decayRate ^ 2 = S.k / S.m := by
    simpa [hwa] using S.ω_sq
  field_simp [S.m_ne_zero] at hwsq
  nlinarith

/-- An overdamped system has decay rate greater than the natural frequency. -
/
lemma isOverdamped_decayRate (hS : S.IsOverdamped) : S.ω < S.decayRate := by
  rw [IsOverdamped] at hS
  rw [discriminant_eq_four_mul_m_sq_mul_decayRate_sq_sub_ω_sq] at hS
  have hm_sq_pos : 0 < 4 * S.m^2 := by
    have hsq : 0 < S.m^2 := sq_pos_of_pos S.m_pos
    nlinarith
  have hsq : S.ω^2 < S.decayRate^2 := by
    nlinarith
  nlinarith [S.decayRate_nonneg, S.ω_pos]

/-- In the underdamped regime, the selected frequency uses the oscillation frequency
/
lemma angularFrequency_eq_underdamped (hS : S.IsUnderdamped) :
  S.angularFrequency = sqrt (- S.discriminant) / (2 * S.m) := by
  classical
  simp [angularFrequency, hS]

/-- In the critically damped regime, the selected frequency is zero. -
/
lemma angularFrequency_eq_criticallyDamped (hS : S.IsCriticallyDamped) :
  S.angularFrequency = 0 := by
  classical
  have hnotUnder : ¬ S.IsUnderdamped := by
    intro hUnder
```

```

      rw [IsUnderdamped] at hUnder
      rw [IsCriticallyDamped] at hS
      linarith
    simp [angularFrequency, hnotUnder, hS]

/-- In the overdamped regime, the selected frequency uses the real split ra
/
lemma angularFrequency_eq_overdamped (hS : S.IsOverdamped) :
  S.angularFrequency = sqrt S.discriminant / (2 * S.m) := by
  classical
  have hnotUnder : ¬ S.IsUnderdamped := by
    intro hUnder
    rw [IsUnderdamped] at hUnder
    rw [IsOverdamped] at hS
    linarith
  have hnotCritical : ¬ S.IsCriticallyDamped := by
    intro hCritical
    rw [IsCriticallyDamped] at hCritical
    rw [IsOverdamped] at hS
    linarith
  simp [angularFrequency, hnotUnder, hnotCritical]

/-- In the underdamped regime, the selected angular frequency squares to
`ω^2 - decayRate^2`. -/
lemma angularFrequency_sq_of_underdamped (hS : S.IsUnderdamped) :
  S.angularFrequency^2 = S.ω^2 - S.decayRate^2 := by
  rw [S.angularFrequency_eq_underdamped hS, div_pow, sq_sqrt]
  · rw [discriminant_eq_four_mul_m_sq_mul_decayRate_sq_sub_ω_sq]
    field_simp [S.m_ne_zero]
    ring
  · rw [IsUnderdamped] at hS
    exact le_of_lt (neg_pos.mpr hS)

/-- The selected angular frequency is positive in the underdamped regime. -
/
lemma angularFrequency_pos_of_underdamped (hS : S.IsUnderdamped) :
  0 < S.angularFrequency := by
  rw [S.angularFrequency_eq_underdamped hS]
  apply div_pos
  · rw [IsUnderdamped] at hS
    exact sqrt_pos.mpr (neg_pos.mpr hS)
  · nlinarith [S.m_pos]

/-- The selected angular frequency is nonzero in the underdamped regime. -
/
lemma angularFrequency_ne_zero_of_underdamped (hS : S.IsUnderdamped) :

```

1.4.

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/BASIC.LEAN

```

    S.angularFrequency ≠ 0 :=
    Ne.symm (ne_of_lt (S.angularFrequency_pos_of_underdamped hS))

/-- In the overdamped regime, the selected angular frequency squares to
`decayRate^2 - ω^2`. -/
lemma angularFrequency_sq_of_overdamped (hS : S.IsOverdamped) :
  S.angularFrequency^2 = S.decayRate^2 - S.ω^2 := by
  rw [S.angularFrequency_eq_overdamped hS, div_pow, sq_sqrt]
  · rw [discriminant_eq_four_mul_m_sq_mul_decayRate_sq_sub_ω_sq]
    field_simp [S.m_ne_zero]
    ring
  · rw [IsOverdamped] at hS
    exact le_of_lt hS

/-- The selected angular frequency is positive in the overdamped regime. -
/
lemma angularFrequency_pos_of_overdamped (hS : S.IsOverdamped) :
  0 < S.angularFrequency := by
  rw [S.angularFrequency_eq_overdamped hS]
  apply div_pos
  · rw [IsOverdamped] at hS
    exact sqrt_pos.mpr hS
  · nlinarith [S.m_pos]

/-- The selected angular frequency is nonzero in the overdamped regime. -
/
lemma angularFrequency_ne_zero_of_overdamped (hS : S.IsOverdamped) :
  S.angularFrequency ≠ 0 :=
  Ne.symm (ne_of_lt (S.angularFrequency_pos_of_overdamped hS))

/-!
## E. To undamped oscillator

We show that the damped harmonic oscillator reduces to the undamped harmonic oscillator
when the damping coefficient is zero. The underlying mass and spring data are already inherited
from `HarmonicOscillator`; the proof argument records that this conversion is being used in the
zero-damping case.

We also show that the equations of motion are equivalent in this case.
-/

set_option linter.unusedVariables false in
/-- Convert a damped oscillator to its underlying undamped oscillator when `γ = 0`.
-/
@[nolint unusedArguments]
def toUndamped (S : DampedHarmonicOscillator) (_hS : S.IsUndamped) :
```

```

    HarmonicOscillator :=
    S.toHarmonicOscillator

/-- When  $\gamma = 0$ , the damped equation of motion is equivalent to the equation
for the corresponding undamped harmonic oscillator. -/
lemma toUndamped_equationOfMotion (S : DampedHarmonicOscillator) (hS : S.IsUndamped)
  (x_t : Time → EuclideanSpace ℝ (Fin 1)) (hx : ContDiff ℝ ∞ x_t) :
  S.EquationOfMotion x_t ↔ (S.toUndamped hS).EquationOfMotion x_t := by
  have hy : S.γ = 0 := by
    simp [IsUndamped] using hS
  rw [S.equationOfMotion_iff_newtons_2nd_law x_t,
    (S.toUndamped hS).equationOfMotion_iff_newtons_2nd_law x_t hx]
  constructor
  · intro h t
    calc
      (S.toUndamped hS).m • ∂_t (∂_t x_t) t = S.m • ∂_t (∂_t x_t) t := rfl
      _ = force S x_t t := h t
      _ = HarmonicOscillator.force (S.toUndamped hS) (x_t t) := by
        simp [force, HarmonicOscillator.force_eq_linear, toUndamped, hy]
  · intro h t
    calc
      S.m • ∂_t (∂_t x_t) t = (S.toUndamped hS).m • ∂_t (∂_t x_t) t := rfl
      _ = HarmonicOscillator.force (S.toUndamped hS) (x_t t) := h t
      _ = force S x_t t := by
        simp [force, HarmonicOscillator.force_eq_linear, toUndamped, hy]

```

1.5 Physlib/ClassicalMechanics/DampedHarmonicOscillator/Solutions

```

/-
Copyright (c) 2026 Florian Wiesner. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Florian Wiesner
-/
module

public import Physlib.ClassicalMechanics.DampedHarmonicOscillator.Basic
public import Mathlib.Analysis.SpecialFunctions.Trigonometric.DerivHyp
/-!

# Solutions to the damped harmonic oscillator

## i. Overview

```


1.5.

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/SOLUTION.LEAN

In this module we define the solution to the damped harmonic oscillator for given initial conditions and prove that it satisfies the equation of motion. The solution selects the appropriate closed form from the sign of the discriminant: trigonometric for the underdamped case, polynomial for the critically damped case, and hyperbolic for the overdamped case.

ii. Key results

- `InitialConditions` is a structure for the initial position and velocity.
- `trajectory` selects the appropriate regime-specific trajectory from the sign of the discriminant.
- `trajectory\_equationOfMotion\_of\_underdamped`,
`trajectory\_equationOfMotion\_of\_criticallyDamped`, and
`trajectory\_equationOfMotion\_of\_overdamped` prove the selected trajectory satisfies the equation of motion in each damping regime.
- `trajectory\_equationOfMotion` proves that the selected trajectory satisfies the equation of motion.

iii. Table of contents

- A. The initial conditions
- B. Trajectories associated with the initial conditions
 - B.1. Regime-specific base trajectories
 - B.2. The selected trajectory
 - B.3. Shared calculus lemmas
 - B.4. Derivatives of the base trajectories
- C. Trajectories and equation of motion
 - C.1. The selected trajectory satisfies the equation of motion
 - C.2. Uniqueness of the solutions

iv. References

References for the damped harmonic oscillator include:

- Landau & Lifshitz, Mechanics, page 76, section 25.
- Goldstein, Classical Mechanics, Chapter 2.

v. TODOs

-/

TODO "Prove that the selected trajectory is the unique solution of the equation of motion for the given initial conditions."

@[expose] public section

namespace ClassicalMechanics
open Real

```

open Time
open ContDiff

namespace DampedHarmonicOscillator

variable (S : DampedHarmonicOscillator)

/-!

## A. The initial conditions

We define the type of initial conditions for the damped harmonic oscillator.
conditions are the position and velocity at time `0`.

-/

/-- The initial conditions for the damped harmonic oscillator, specified by
position and an initial velocity. -/
@[ext]
structure InitialConditions where
  /-- The initial position of the damped harmonic oscillator. -
  /
  x₀ : EuclideanSpace ℝ (Fin 1)
  /-- The initial velocity of the damped harmonic oscillator. -
  /
  v₀ : EuclideanSpace ℝ (Fin 1)

/-!

## B. Trajectories associated with the initial conditions

For each damping regime, we give an explicit formula for the trajectory with
initial conditions.

### B.1. Regime-specific base trajectories

-/

/-- The oscillatory part of the underdamped trajectory before exponential d
/
noncomputable def underdampedBase
  (IC : InitialConditions) : Time → EuclideanSpace ℝ (Fin 1) := fun t =>
    cos (S.angularFrequency * t) • IC.x₀
    + (sin (S.angularFrequency * t)/S.angularFrequency) •
      (IC.v₀ + S.decayRate • IC.x₀)

```

1.5.

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/SOLUTION.LEAN

-- The polynomial part of the critically damped trajectory before exponential decay.
/

```
noncomputable def criticallyDampedBase
  (IC : InitialConditions) : Time → EuclideanSpace ℝ (Fin 1) := fun t =>
    IC.x₀ + (t : ℝ) • (IC.v₀ + S.decayRate • IC.x₀)
```

-- The hyperbolic part of the overdamped trajectory before exponential decay. -
/

```
noncomputable def overdampedBase
  (IC : InitialConditions) : Time → EuclideanSpace ℝ (Fin 1) := fun t =>
    cosh (S.angularFrequency * t) • IC.x₀
    + (sinh (S.angularFrequency * t) / S.angularFrequency) •
      (IC.v₀ + S.decayRate • IC.x₀)
```

--!

B.2. The selected trajectory

--

-- Given initial conditions, the solution selected from the damping regime of the oscillator. *--*

```
noncomputable def trajectory
  (IC : InitialConditions) : Time → EuclideanSpace ℝ (Fin 1) := by
  classical
  exact
    if S.IsUnderdamped then
      fun t : Time => exp (-S.decayRate * t) • S.underdampedBase IC t
    else if S.IsCriticallyDamped then
      fun t : Time => exp (-S.decayRate * t) • S.criticallyDampedBase IC t
    else
      fun t : Time => exp (-S.decayRate * t) • S.overdampedBase IC t
```

-- In the underdamped regime, the selected trajectory uses the trigonometric base.
/

```
lemma trajectory_eq_of_underdamped (IC : InitialConditions) (hS : S.IsUnderdamped) :
  S.trajectory IC =
    fun t : Time => exp (-S.decayRate * t) • S.underdampedBase IC t := by
  classical
  simp [trajectory, hS]
```

-- In the critically damped regime, the selected trajectory uses the polynomial base.
/

```
lemma trajectory_eq_of_criticallyDamped (IC : InitialConditions) (hS : S.IsCriticallyDamped) :
  S.trajectory IC =
    fun t : Time => exp (-S.decayRate * t) • S.criticallyDampedBase IC t := by
```

```

classical
have hnotUnder : ¬ S.IsUnderdamped := by
  intro hUnder
  rw [IsUnderdamped] at hUnder
  rw [IsCriticallyDamped] at hS
  linarith
simp [trajectory, hnotUnder, hS]

/-- In the overdamped regime, the selected trajectory uses the hyperbolic b
/
lemma trajectory_eq_of_overdamped (IC : InitialConditions) (hS : S.IsOverdamped) :
  S.trajectory IC =
    fun t : Time => exp (-S.decayRate * t) • S.overdampedBase IC t := by
  classical
  have hnotUnder : ¬ S.IsUnderdamped := by
    intro hUnder
    rw [IsUnderdamped] at hUnder
    rw [IsOverdamped] at hS
    linarith
  have hnotCritical : ¬ S.IsCriticallyDamped := by
    intro hCritical
    rw [IsCriticallyDamped] at hCritical
    rw [IsOverdamped] at hS
    linarith
  simp [trajectory, hnotUnder, hnotCritical]

/-!

```

B.3. Shared calculus lemmas

The three solution formulas all have the form $\exp(-a * t) \cdot y \ t$. The following lemmas compute the first and second derivatives of that expression and pack them into an equation-of-motion argument.

```

-/

private lemma exp_decay_smul_velocity
  (a : ℝ) (y : Time → EuclideanSpace ℝ (Fin 1)) (hy : Differentiable ℝ y) :
  ∂t (fun t : Time => exp (-a * t.val) • y t) =
    fun t : Time => exp (-a * t.val) • (∂t y t - a • y t) := by
  funext t
  rw [Time.deriv]
  rw [fderiv_fun_smul (by fun_prop) (hy t)]
  rw [fderiv_exp (by fun_prop), fderiv_fun_mul (by fun_prop) (by fun_prop)]
  simp only [ContinuousLinearMap.add_apply, ContinuousLinearMap.smulRight_apply,
    fderiv_fun_neg, fderiv_fun_const, Pi.zero_apply, Time.fderiv_val,

```

1.5.

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/SOLUTION.LEAN

```

ContinuousLinearMap.neg_apply, ContinuousLinearMap.coe_smul', Pi.smul_apply, smu
rw [← Time.deriv_eq]
simp [smul_sub, smul_smul]
module

private lemma exp_decay_smul_acceleration
  (a μ : ℝ) (y : Time → EuclideanSpace ℝ (Fin 1))
  (hy : Differentiable ℝ y) (hdy : Differentiable ℝ (∂t y))
  (hy'' : ∂t (∂t y) = fun t => μ • y t) :
  ∂t (∂t (fun t : Time => exp (-a * t.val) • y t)) =
    fun t : Time => exp (-a * t.val) •
      (μ • y t - (2 * a) • ∂t y t + a^2 • y t) := by
rw [exp_decay_smul_velocity a y hy]
funext t
rw [Time.deriv]
rw [fderiv_fun_smul (by fun_prop) (by fun_prop)]
rw [fderiv_exp (by fun_prop), fderiv_fun_mul (by fun_prop) (by fun_prop)]
rw [fderiv_fun_sub (hdy t) (by fun_prop)]
rw [fderiv_fun_const_smul (hy t)]
have hy''_t := congrFun hy'' t
rw [Time.deriv] at hy''_t
simp only [ContinuousLinearMap.add_apply, ContinuousLinearMap.sub_apply,
  ContinuousLinearMap.smulRight_apply, fderiv_fun_neg, fderiv_fun_const,
  Pi.zero_apply, Time.fderiv_val, ContinuousLinearMap.neg_apply,
  ContinuousLinearMap.coe_smul', Pi.smul_apply, smul_eq_mul]
rw [hy''_t, ← Time.deriv_eq]
simp [smul_add, smul_sub, smul_smul]
module

private lemma exp_decay_smul_equationOfMotion
  (a μ : ℝ) (y : Time → EuclideanSpace ℝ (Fin 1))
  (hy : Differentiable ℝ y) (hdy : Differentiable ℝ (∂t y))
  (hy'' : ∂t (∂t y) = fun t => μ • y t)
  (hy : S.γ = 2 * S.m * a) (hk : S.k = S.m * (a^2 - μ)) :
  S.EquationOfMotion (fun t : Time => exp (-a * t.val) • y t) := by
intro t
rw [exp_decay_smul_acceleration a μ y hy hdy hy'']
rw [exp_decay_smul_velocity a y hy]
rw [hy, hk]
simp [smul_add, smul_sub, smul_smul]
module

/-!

```

B.4. Derivatives of the base trajectories

The remaining private lemmas compute the velocity and acceleration of the polynomial, and hyperbolic base trajectories before the exponential decay f

-/

```

private lemma criticallyDampedBase_velocity (IC : InitialConditions) :
   $\partial_t$  (S.criticallyDampedBase IC) =
    fun _ : Time => IC.v0 + S.decayRate • IC.x0 := by
  funext t
  change  $\partial_t$  (fun t : Time =>
    IC.x0 + t.val • (IC.v0 + S.decayRate • IC.x0)) t = _
  rw [Time.deriv]
  rw [fderiv_fun_add (by fun_prop) (by fun_prop)]
  rw [fderiv_fun_const]
  rw [fderiv_smul_const (by fun_prop)]
  simp

private lemma criticallyDampedBase_acceleration (IC : InitialConditions) :
   $\partial_t$  ( $\partial_t$  (S.criticallyDampedBase IC)) =
    fun _ => (0 : EuclideanSpace ℝ (Fin 1)) := by
  rw [criticallyDampedBase_velocity]
  funext t
  simp

private lemma underdampedBase_velocity (IC : InitialConditions) (hS : S.IsU
   $\partial_t$  (fun t : Time =>
    cos (S.angularFrequency * t.val) • IC.x0 +
    (sin (S.angularFrequency * t.val) / S.angularFrequency) •
    (IC.v0 + S.decayRate • IC.x0)) =
    fun t : Time =>
    (-S.angularFrequency * sin (S.angularFrequency * t.val)) • IC.x0 +
    cos (S.angularFrequency * t.val) •
    (IC.v0 + S.decayRate • IC.x0) := by
  funext t
  rw [Time.deriv]
  rw [fderiv_fun_add (by fun_prop) (by fun_prop)]
  rw [fderiv_smul_const (by fun_prop)]
  rw [fderiv_smul_const (by fun_prop)]
  have hΩ : S.angularFrequency ≠ 0 := S.angularFrequency_ne_zero_of_underda
  have hcos :
    (fderiv ℝ (fun y : Time => cos (S.angularFrequency * y.val)) t) 1 =
      -S.angularFrequency *
        sin (S.angularFrequency * t.val) := by
    rw [fderiv_cos (by fun_prop), fderiv_fun_mul (by fun_prop) (by fun_prop)]
    simp [mul_comm]
  have hsin :

```

1.5.

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/SOLUTION.LEAN

```

    (fderiv ℝ (fun y : Time =>
      sin (S.angularFrequency * y.val) /
      S.angularFrequency) t) 1 =
    cos (S.angularFrequency * t.val) := by
  have hscale :
    fderiv ℝ (fun y : Time =>
      sin (S.angularFrequency * y.val) /
      S.angularFrequency) t =
    (1 / S.angularFrequency) •
    fderiv ℝ (fun y : Time =>
      sin (S.angularFrequency * y.val)) t := by
  rw [← fderiv_mul_const]
  congr
  funext y
  field_simp [hΩ]
  ring_nf
  fun_prop
  rw [hscale, fderiv_sin (by fun_prop), fderiv_fun_mul (by fun_prop) (by fun_prop)]
  simp [hΩ, mul_comm]
  simp [hcos, hsin]

```

private lemma underdampedBase_acceleration (IC : InitialConditions) (hS : S.IsUnderdamped)

```

  ∂t (∂t (fun t : Time =>
    cos (S.angularFrequency * t.val) • IC.x₀ +
    (sin (S.angularFrequency * t.val) / S.angularFrequency) •
    (IC.v₀ + S.decayRate • IC.x₀))) =
  fun t : Time => -S.angularFrequency^2 •
    (cos (S.angularFrequency * t.val) • IC.x₀ +
    (sin (S.angularFrequency * t.val) / S.angularFrequency) •
    (IC.v₀ + S.decayRate • IC.x₀)) := by
  funext t
  rw [S.underdampedBase_velocity IC hS]
  rw [Time.deriv]
  rw [fderiv_fun_add (by fun_prop) (by fun_prop)]
  rw [fderiv_smul_const (by fun_prop)]
  rw [fderiv_smul_const (by fun_prop)]
  have hΩ : S.angularFrequency ≠ 0 := S.angularFrequency_ne_zero_of_underdamped hS
  have hsin :
    (fderiv ℝ (fun y : Time =>
      S.angularFrequency * sin (S.angularFrequency * y.val)) t) 1 =
    S.angularFrequency^2 * cos (S.angularFrequency * t.val) := by
  rw [fderiv_fun_mul (by fun_prop) (by fun_prop)]
  rw [fderiv_sin (by fun_prop), fderiv_fun_mul (by fun_prop) (by fun_prop)]
  simp [pow_two, mul_comm, mul_assoc]
  have hcos :
    (fderiv ℝ (fun y : Time => cos (S.angularFrequency * y.val)) t) 1 =

```

```

    -S.angularFrequency * sin (S.angularFrequency * t.val) := by
      rw [fderiv_cos (by fun_prop), fderiv_fun_mul (by fun_prop) (by fun_prop)]
      simp [mul_comm]
    simp [hsin, hcos, smul_add, smul_smul]
    field_simp [hΩ]

private lemma overdampedBase_velocity (IC : InitialConditions) (hS : S.IsOverdamped) :
  ∂t (fun t : Time =>
    cosh (S.angularFrequency * t.val) • IC.x₀ +
    (sinh (S.angularFrequency * t.val) / S.angularFrequency) •
    (IC.v₀ + S.decayRate • IC.x₀)) =
  fun t : Time =>
    (S.angularFrequency * sinh (S.angularFrequency * t.val)) • IC.x₀ +
    cosh (S.angularFrequency * t.val) •
    (IC.v₀ + S.decayRate • IC.x₀) := by
  funext t
  rw [Time.deriv]
  rw [fderiv_fun_add (by fun_prop) (by fun_prop)]
  rw [fderiv_smul_const (by fun_prop)]
  rw [fderiv_smul_const (by fun_prop)]
  have hLambda : S.angularFrequency ≠ 0 := S.angularFrequency_ne_zero_of_overdamped
  have hcosh :
    (fderiv ℝ (fun y : Time => cosh (S.angularFrequency * y.val)) t) 1 =
    S.angularFrequency * sinh (S.angularFrequency * t.val) := by
    rw [fderiv_cosh (by fun_prop), fderiv_fun_mul (by fun_prop) (by fun_prop)]
    simp [mul_comm]
  have hsinh :
    (fderiv ℝ (fun y : Time =>
      sinh (S.angularFrequency * y.val) / S.angularFrequency) t) 1 =
    cosh (S.angularFrequency * t.val) := by
    have hscale :
      fderiv ℝ (fun y : Time =>
        sinh (S.angularFrequency * y.val) / S.angularFrequency) t =
        (1 / S.angularFrequency) •
        fderiv ℝ (fun y : Time => sinh (S.angularFrequency * y.val)) t
    rw [← fderiv_mul_const]
    congr
    funext y
    field_simp [hLambda]
    ring_nf
    fun_prop
    rw [hscale, fderiv_sinh (by fun_prop), fderiv_fun_mul (by fun_prop) (by fun_prop)]
    simp [hLambda, mul_comm]
  simp [hcosh, hsinh]

private lemma overdampedBase_acceleration (IC : InitialConditions) (hS : S.IsOverdamped) :

```


1.5.

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/SOLUTION.LEAN

```

    ∂t (∂t (fun t : Time =>
      cosh (S.angularFrequency * t.val) • IC.x0 +
        (sinh (S.angularFrequency * t.val) / S.angularFrequency) •
          (IC.v0 + S.decayRate • IC.x0))) =
    fun t : Time => S.angularFrequency^2 •
      (cosh (S.angularFrequency * t.val) • IC.x0 +
        (sinh (S.angularFrequency * t.val) / S.angularFrequency) •
          (IC.v0 + S.decayRate • IC.x0)) := by
  funext t
  rw [S.overdampedBase_velocity IC hS]
  rw [Time.deriv]
  rw [fderiv_fun_add (by fun_prop) (by fun_prop)]
  rw [fderiv_smul_const (by fun_prop)]
  rw [fderiv_smul_const (by fun_prop)]
  have hLambda : S.angularFrequency ≠ 0 := S.angularFrequency_ne_zero_of_overdamped
  have hsinh :
    (fderiv ℝ (fun y : Time =>
      S.angularFrequency * sinh (S.angularFrequency * y.val)) t) 1 =
      S.angularFrequency^2 * cosh (S.angularFrequency * t.val) := by
    rw [fderiv_fun_mul (by fun_prop) (by fun_prop)]
    rw [fderiv_sinh (by fun_prop), fderiv_fun_mul (by fun_prop) (by fun_prop)]
    simp [pow_two, mul_comm, mul_assoc]
  have hcosh :
    (fderiv ℝ (fun y : Time => cosh (S.angularFrequency * y.val)) t) 1 =
      S.angularFrequency * sinh (S.angularFrequency * t.val) := by
    rw [fderiv_cosh (by fun_prop), fderiv_fun_mul (by fun_prop) (by fun_prop)]
    simp [mul_comm]
  simp [hsinh, hcosh, smul_add, smul_smul]
  field_simp [hLambda]

```

/-!

C. Trajectories and equation of motion

The regime-specific trajectories satisfy the equation of motion for the damped harmonic oscillator.

C.1. The selected trajectory satisfies the equation of motion

-/

/-- In the critically damped regime, the selected trajectory satisfies the damped equation of motion. -/

```

lemma trajectory_equationOfMotion_of_criticallyDamped (IC : InitialConditions)
  (hS : S.IsCriticallyDamped) :
  S.EquationOfMotion (S.trajectory IC) := by
  rw [S.trajectory_eq_of_criticallyDamped IC hS]
  have hy : S.γ = 2 * S.m * S.decayRate := S.gamma_eq_two_mul_m_mul_decayRate

```

```

have hk : S.k = S.m * (S.decayRate^2 - 0) := by
  simp [sub_zero] using S.k_eq_m_mul_decayRate_sq_of_criticallyDamped hS
have hbase :
  ∂t (∂t (S.criticallyDampedBase IC)) =
    fun t => (0 : ℝ) • S.criticallyDampedBase IC t := by
  simp using S.criticallyDampedBase_acceleration IC
exact S.exp_decay_smul_equationOfMotion S.decayRate 0 (S.criticallyDamped
  (by
    change Differentiable ℝ (fun t : Time =>
      IC.x₀ + t.val • (IC.v₀ + S.decayRate • IC.x₀))
    fun_prop)
  (by
    rw [S.criticallyDampedBase_velocity IC]
    fun_prop)
  hbase hy hk

/-- In the underdamped regime, the selected trajectory satisfies the damped
motion. -/
lemma trajectory_equationOfMotion_of_underdamped (IC : InitialConditions)
  (hS : S.IsUnderdamped) :
  S.EquationOfMotion (S.trajectory IC) := by
  rw [S.trajectory_eq_of_underdamped IC hS]
have hy : S.γ = 2 * S.m * S.decayRate := S.gamma_eq_two_mul_m_mul_decayRa
have hk : S.k = S.m * (S.decayRate^2 - (-S.angularFrequency^2)) := by
  rw [S.k_eq_m_mul_ω_sq, S.angularFrequency_sq_of_underdamped hS]
ring
have hbase :
  ∂t (∂t (S.underdampedBase IC)) =
    fun t => (-S.angularFrequency^2) • S.underdampedBase IC t := by
  change ∂t (∂t (fun t : Time =>
    cos (S.angularFrequency * t.val) • IC.x₀ +
    (sin (S.angularFrequency * t.val) / S.angularFrequency) •
    (IC.v₀ + S.decayRate • IC.x₀))) =
    fun t => -S.angularFrequency^2 •
    (cos (S.angularFrequency * t.val) • IC.x₀ +
    (sin (S.angularFrequency * t.val) / S.angularFrequency) •
    (IC.v₀ + S.decayRate • IC.x₀))
  exact S.underdampedBase_acceleration IC hS
exact S.exp_decay_smul_equationOfMotion S.decayRate
  (-S.angularFrequency^2) (S.underdampedBase IC)
  (by
    change Differentiable ℝ (fun t : Time =>
      cos (S.angularFrequency * t.val) • IC.x₀ +
      (sin (S.angularFrequency * t.val) / S.angularFrequency) •
      (IC.v₀ + S.decayRate • IC.x₀))
    fun_prop)

```

1.5.

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/SOLUTION.LEAN

```

    (by
      change Differentiable  $\mathbb{R}$  ( $\partial_t$  (fun t : Time =>
        cos (S.angularFrequency * t.val) • IC.x₀ +
        (sin (S.angularFrequency * t.val) / S.angularFrequency) •
        (IC.v₀ + S.decayRate • IC.x₀)))
      rw [S.underdampedBase_velocity IC hS]
      fun_prop)
    hbase hy hk

/-- In the overdamped regime, the selected trajectory satisfies the damped equation
motion. -/
lemma trajectory_equationOfMotion_of_overdamped (IC : InitialConditions)
  (hS : S.IsOverdamped) :
  S.EquationOfMotion (S.trajectory IC) := by
  rw [S.trajectory_eq_of_overdamped IC hS]
  have hy : S.γ = 2 * S.m * S.decayRate := S.gamma_eq_two_mul_m_mul_decayRate
  have hk : S.k = S.m * (S.decayRate^2 - S.angularFrequency^2) := by
    rw [S.k_eq_m_mul_ω_sq, S.angularFrequency_sq_of_overdamped hS]
  ring
  have hbase :
     $\partial_t$  ( $\partial_t$  (S.overdampedBase IC)) =
      fun t => S.angularFrequency^2 • S.overdampedBase IC t := by
    change  $\partial_t$  ( $\partial_t$  (fun t : Time =>
      cosh (S.angularFrequency * t.val) • IC.x₀ +
      (sinh (S.angularFrequency * t.val) / S.angularFrequency) •
      (IC.v₀ + S.decayRate • IC.x₀))) =
      fun t => S.angularFrequency^2 •
        (cosh (S.angularFrequency * t.val) • IC.x₀ +
        (sinh (S.angularFrequency * t.val) / S.angularFrequency) •
        (IC.v₀ + S.decayRate • IC.x₀))
    exact S.overdampedBase_acceleration IC hS
    exact S.exp_decay_smul_equationOfMotion S.decayRate (S.angularFrequency^2)
    (S.overdampedBase IC)
  (by
    change Differentiable  $\mathbb{R}$  (fun t : Time =>
      cosh (S.angularFrequency * t.val) • IC.x₀ +
      (sinh (S.angularFrequency * t.val) / S.angularFrequency) •
      (IC.v₀ + S.decayRate • IC.x₀))
    fun_prop)
  (by
    change Differentiable  $\mathbb{R}$  ( $\partial_t$  (fun t : Time =>
      cosh (S.angularFrequency * t.val) • IC.x₀ +
      (sinh (S.angularFrequency * t.val) / S.angularFrequency) •
      (IC.v₀ + S.decayRate • IC.x₀)))
    rw [S.overdampedBase_velocity IC hS]
    fun_prop)

```

```

hbase hy hk

/-- The selected trajectory satisfies the damped equation of motion. -
/
lemma trajectory_equationOfMotion (IC : InitialConditions) :
  S.EquationOfMotion (S.trajectory IC) := by
  by_cases hUnder : S.IsUnderdamped
  · exact S.trajectory_equationOfMotion_of_underdamped IC hUnder
  · by_cases hCritical : S.IsCriticallyDamped
    · exact S.trajectory_equationOfMotion_of_criticallyDamped IC hCritical
    · have hOver : S.IsOverdamped := by
        rw [IsOverdamped, IsUnderdamped, IsCriticallyDamped] at *
        by_contra hNotOver
        have hNonneg :  $0 \leq S.discriminant$  := le_of_not_gt hUnder
        have hNonpos :  $S.discriminant \leq 0$  := le_of_not_gt hNotOver
        exact hCritical (le_antisymm hNonpos hNonneg)
    exact S.trajectory_equationOfMotion_of_overdamped IC hOver

/-!

### C.2. Uniqueness of the solutions

Future work: prove that, in each damping regime, the selected explicit bran
solution of the damped equation of motion with the given initial conditions

-/

end DampedHarmonicOscillator

end ClassicalMechanics

```

1.6 Physlib/ClassicalMechanics/FreeParticle/Basic.lean

```

/-
Copyright (c) 2026 Pranav Magdum. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Pranav Magdum
-/
module

public import Mathlib.Data.Real.Basic
public import Mathlib.Analysis.Calculus.Deriv.Basic
public import Physlib.Meta.TODO.Basic
public import Physlib.Meta.Sorry

```

```
public import Physlib.SpaceAndTime.Time.Derivatives
```

```
/-!
```

```
# The Free Particle
```

```
## i. Overview
```

The free particle is one of the simplest systems in classical mechanics: a particle moving with no external forces acting on it. Physically, this means the particle just moves at constant velocity.

In this file, we work in a simple 1D coordinate system where position and velocity are functions of time with values in $\mathbb{R}$. This keeps things easy to reason about. A more complete treatment would use manifolds and tangent bundles.

```
## ii. Key results
```

The main things we show about the free particle are:

In the ``Basic`` module:

- ``FreeParticle`` stores the mass of the particle.
- ``NewtonsSecondLaw`` encodes the equation $m \cdot q'' = 0$.
- ``accel_zero`` shows that this implies $q'' = 0$.
- ``velocity_const_of_zero_acc`` shows that zero acceleration means velocity is constant.
- ``energy_conservation_of_equationOfMotion`` shows that kinetic energy stays constant.

So overall, we formalise the usual chain:

Newton's law $\rightarrow$ zero acceleration $\rightarrow$ constant velocity $\rightarrow$ constant energy.

```
## iii. Table of contents
```

- A. The setup
- B. Equation of motion
 - B.1. Newton's second law
 - B.2. Zero acceleration
- C. What zero acceleration implies
 - C.1. Constant velocity
- D. Energy
 - D.1. Kinetic energy
 - D.2. Energy conservation

```
## iv. References
```

```
-/
```

```
@[expose] public section
```

```

namespace ClassicalMechanics

open Time
TODO "Prove conservation of linear momentum for the free particle."

/--
A classical free particle with positive mass.

A free particle is a mechanical system evolving in the absence of
external forces. The dynamics are therefore entirely determined by
Newton's second law with zero force.

The only parameter of the system is the particle mass. The assumption
that the mass is strictly positive is physically natural and is used
throughout the development when simplifying the equation of motion.
-/
structure FreeParticle where
  /--
    The mass of the free particle.

    This parameter determines the inertial response of the particle in
    Newton's second law.
  -/
  mass : ℝ
  mass_pos : 0 < mass

namespace FreeParticle

/--
A trajectory is a time-dependent position function describing the motion
of the particle in one spatial dimension. Defining the trajectory.
-/
abbrev Trajectory := Time → ℝ

set_option linter.unusedVariables false in
/--
The velocity of a trajectory at a given time.
This is defined as the time derivative of the position function.
-/
@[nolint unusedArguments]
noncomputable
def velocity (s : FreeParticle) (q : Trajectory) (t : Time) : ℝ :=
  deriv q t

/--

```

The kinetic energy of the free particle along a trajectory.

This is given by the classical expression $E = (1/2) m v^2$, where m is the particle mass and v is the velocity.

```
-/
noncomputable
def kineticEnergy (s : FreeParticle) (q : Trajectory) (t : Time) : ℝ :=
  (1 / 2) * s.mass * (s.velocity q t)^2
```

```
/--
Newton's second law for the free particle.
```

Since no external forces act on the particle, Newton's second law reduces to the equation $m q'' = 0$, expressing that the acceleration vanishes identically.

```
-/
def NewtonsSecondLaw (s : FreeParticle) (q : Trajectory) (t : Time) : Prop :=
  s.mass * deriv (s.velocity q) t = 0
```

```
/--
Newton's second law for a free particle implies that the acceleration
vanishes identically.
```

Since the particle mass is strictly positive, the equation $m q'' = 0$ can be simplified to $q'' = 0$ by cancelling the mass factor.

```
-/
lemma accel_zero (s : FreeParticle) (q : Trajectory) (h : ∀ t, s.NewtonsSecondLaw q
  ∀ t, deriv (deriv q) t = 0 := by
  intro t
  have h₀ : s.mass ≠ 0 := ne_of_gt s.mass_pos
  have h₁ := h t
  exact (mul_eq_zero.mp h₁).resolve_left h₀
```

```
/--
If the acceleration of a trajectory vanishes everywhere, then the
velocity is constant.
```

More precisely, if the second derivative of the trajectory is zero for all times, then there exists a constant v_0 such that the velocity is equal to v_0 at every time.

The continuity assumption on $\text{deriv } q$ is included to apply standard results from real analysis relating vanishing derivatives to constant functions.

```
-/
```

```

@[sorryful]
lemma velocity_const_of_zero_acc (q : Time → ℝ) (h : ∀ t, deriv (deriv q) t = 0)
  (hcont : ContDiff ℝ 1 q) : ∃ v₀, ∀ t, deriv q t = v₀ := by
  -- this is a standard analysis result (related to `is_const_of_fderiv_eq_0`)
  sorry
/--
A free particle satisfying the equation of motion conserves kinetic energy.

The proof follows the standard argument from classical mechanics:
Newton's second law implies that the acceleration vanishes, which in
turn implies that the velocity is constant. Since the kinetic energy
depends only on the square of the velocity, it follows that the kinetic
energy is constant in time.
-/
@[sorryful]
theorem kineticEnergy_conserved (s : FreeParticle) (q : Trajectory)
  (h : ∀ t, s.NewtonsSecondLaw q t) (hcont : ContDiff ℝ 1 q) :
  ∃ E, ∀ t, s.kineticEnergy q t = E := by
  -- get q'' = 0
  have h_acc : ∀ t, deriv (deriv q) t = 0 :=
    accel_zero s q h
  -- get constant velocity
  rcases velocity_const_of_zero_acc q h_acc hcont with ⟨v₀, hv⟩
  -- energy is constant
  refine ⟨(1 / 2) * s.mass * v₀^2, fun t => ?_⟩
  unfold kineticEnergy velocity
  rw [hv t]

end FreeParticle
end ClassicalMechanics

```

1.7 Physlib/ClassicalMechanics/HarmonicOscillator/Solution.lean

and an initial velocity.

```

The `@[ext]` attribute provides an extensionality lemma for `InitialConditions`.
That is, a lemma which states that two initial conditions are equal if their
initial positions and initial velocities are equal. -/
@[ext] structure InitialConditions where
@[ext] structure InitialConditionsAtTime where
end InitialConditions

/--
The period of a harmonic oscillator is  $2 * \pi / \omega$ .

```


1.8.

PHYSLIB/CLASSICALMECHANICS/ORBITALMECHANICS/VISVIVA.lean

-/

noncomputable def period (S : HarmonicOscillator) : $\mathbb{R}$:= 2 * π / S. ω

@[inherit_doc period]

scoped notation "T" => HarmonicOscillator.period

lemma period_eq : T S = 2 * π / S. ω := rfl

lemma period_pos : 0 < T S := by

have := S. ω _pos

rw [period_eq]

positivity

/--

The trajectory of the harmonic oscillator is periodic with period of $2 * \pi / \omega$.

-/

lemma trajectory_periodic (IC : InitialConditions) :

Function.Periodic (IC.trajectory S) (T S) := fun t => by

have h : S. ω * (t.val + 2 * π / S. ω) = S. ω * t.val + 2 * π := by

have := S. ω _ne_zero

ring_nf; field_simp

rw [InitialConditions.trajectory, add_val, period_eq, h, cos_add_two_pi, sin_add_t

rfl

1.8 Physlib/ClassicalMechanics/OrbitalMechanics/VisViva.lean

/-

Copyright (c) 2026 Hannah Dawe. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Hannah Dawe

-/

module

public import Mathlib.Data.Real.Basic

public import Mathlib.Data.Real.Sqrt

/-!

Circular Orbit Vis Viva

The vis-viva equation relates the speed of an orbiting body to its position and the mass of the central body. This module defines a simplified version of the vis-viva equation that is restricted to circular orbits ($v^2 = G M / r$).

-/

```

@[expose] public section

namespace ClassicalMechanics

/-- System parameters for the vis-viva equation in circular orbital mechanics. -/
/
structure VisViva where
  /-- Gravitational constant. -/
  G : ℝ
  /-- Central mass body. -/
  M : ℝ
  /-- Orbiting mass body. -/
  m : ℝ

namespace VisViva

/-- Configuration space for orbital mechanics, defining the orbital radius. -/
/
structure ConfigurationSpace where
  /-- Orbital radius. -/
  r : ℝ

/-- The orbital speed required for a circular orbit at radius `r`. -/
/
noncomputable def speedCircular (sys : VisViva) (cfg : ConfigurationSpace)
  Real.sqrt (sys.G * sys.M / cfg.r)

/-- Lemma: the square of the circular orbit speed equals G M / r. -/
/
lemma speedCircular_sq (sys : VisViva) (cfg : ConfigurationSpace) (hr : 0 <
  (hM : 0 < sys.M) :
  (speedCircular sys cfg)^2 = sys.G * sys.M / cfg.r := by
  simp [speedCircular]
  apply Real.sq_sqrt
  positivity

end VisViva
end ClassicalMechanics

```

1.9 Physlib/Electromagnetism/Current/InfiniteWire.lean

```

public import Physlib.Electromagnetism.Distributional.Dynamics.IsExtrema

```

```
have h1 := distDiv_inv_pow_eq_dim (d := 2)
```

1.10 Physlib/Electromagnetism/Distributional/Basic.lean

```
/-
```

```
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.  
Released under Apache 2.0 license as described in the file LICENSE.  
Authors: Joseph Tooby-Smith
```

```
-/
```

```
module
```

```
public import Physlib.SpaceAndTime.SpaceTime.Derivatives  
public import Physlib.SpaceAndTime.Space.Derivatives.Curl  
public import Physlib.SpaceAndTime.TimeAndSpace.ConstantTimeDist  
public import Physlib.Mathematics.VariationalCalculus.HasVarAdjDeriv  
public import Physlib.Relativity.Tensors.Elab  
public import Physlib.SpaceAndTime.SpaceTime.TimeSlice
```

```
/-!
```

```
# The Electromagnetic Potential
```

```
## i. Overview
```

The electromagnetic potential A^μ is the fundamental objects in electromagnetism. Mathematically it is related to a connection on a $U(1)$ -bundle.

We define the electromagnetic potential as a distribution from spacetime to contravariant Lorentz vectors.

```
## ii. Key results
```

```
- `DistElectromagneticPotential` : the type of electromagnetic potentials as distrib
```

```
## iii. Table of contents
```

- A. The electromagnetic potential as a distribution
 - A.1. Constructors
 - A.2. The derivative of the electromagnetic potential as a distribution
 - A.3. The derivative in terms of the basis
 - A.4. Equivariance of the derivative distribution

```
## iv. References
```

- https://quantummechanics.ucsd.edu/ph130a/130_notes/node452.html
- <https://ph.qmul.ac.uk/sites/default/files/EMT10new.pdf>

-/

@[expose] public section

```
namespace Electromagnetism
open Module realLorentzTensor
open IndexNotation
open TensorSpecies
open Tensor
```

/-!

A. The electromagnetic potential as a distribution

-/

/-- The electromagnetic potential as a distribution and as a tensor A^μ .
/

```
noncomputable abbrev DistElectromagneticPotential (d : ℕ := 3) :=
  (SpaceTime d) → d[ℝ] Lorentz.Vector d
```

```
namespace DistElectromagneticPotential
open TensorSpecies
open Tensor
open SpaceTime
open TensorProduct
open minkowskiMatrix SchwartzMap
attribute [-simp] Fintype.sum_sum_type
attribute [-simp] Nat.succ_eq_add_one
```

/-!

A.1. Constructors

-/

/-- The creation of an electromagnetic potential from a scalar potential. -
/

```
noncomputable def ofScalarPotential {d} (c : SpeedOfLight) :
  ((Time × Space d) → d[ℝ] ℝ) →1 [ℝ] DistElectromagneticPotential d where
  toFun φ := Lorentz.Vector.ofTemporalComponent ∘ L (distTimeSlice c).symm (
    map_add' φ1 φ2 := by
```

```

    ext ε
    simp
  map_smul' r φ := by
    ext ε
    simp only [one_div, map_smul, ContinuousLinearMap.comp_smuls1, map_inv0, RingHom
      ContinuousLinearMap.coe_smul', ContinuousLinearMap.coe_comp', Pi.smul_apply,
      Function.comp_apply]
    rw [smul_comm]

/-- The creation of an electromagnetic potential from a static scalar potential. -
/
noncomputable def ofStaticScalarPotential {d} (c : SpeedOfLight) :
  ((Space d) → d[ℝ] ℝ) →1[ℝ] DistElectromagneticPotential d :=
  ofScalarPotential c ∘1 Space.constantTime

TODO "Add a constructor for DistElectromagneticPotential from a scalar
  potential which is a function using an if...then...else... based on IsDistBounded."

TODO "Add a constructor for DistElectromagneticPotential from vector potentials."

TODO "Add a constructor for DistElectromagneticPotential from electric and
  magnetic fields."

/-!

### A.2. The derivative of the electromagnetic potential as a distribution

-/

set_option backward.isDefEq.respectTransparency false in
/-- The derivative of a electromagnetic potential, which is a distribution. -
/
noncomputable def deriv {d} : DistElectromagneticPotential d →1[ℝ]
  (SpaceTime d) → d[ℝ] Lorentz.CoVector d ⊗[ℝ] Lorentz.Vector d := distTensorDeriv

TODO "Remove the definition `deriv` in distributional electromagnetism and replace i
  `distTensorDeriv` everywhere."

set_option backward.isDefEq.respectTransparency false in
lemma deriv_eq_sum_sum {d} (A : DistElectromagneticPotential d)
  (ε : ∏ (SpaceTime d, ℝ)) :
  deriv A ε = ∑ μ, ∑ ν, (SpaceTime.distDeriv μ A ε ν) •
    Lorentz.CoVector.basis μ ⊗+[ℝ] Lorentz.Vector.basis ν := by
  simp [deriv, distTensorDeriv_apply]
  congr
  funext μ

```

```

conv_lhs => rw [← Lorentz.Vector.basis.sum_repr (SpaceTime.distDeriv μ A)
rw [tmul_sum]
congr
funext v
simp
rfl

/-!

### A.3. The derivative in terms of the basis

-/

@[simp]
lemma deriv_basis_repr_apply {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d)}
  (A : DistElectromagneticPotential d)
  (ε :  $\square$ (SpaceTime d,  $\mathbb{R}$ )) :
  (Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr (deriv
    distDeriv μν.1 A ε μν.2 := by
  match μν with
  | (μ, ν) =>
  rw [deriv_eq_sum_sum]
  simp only [map_sum, map_smul, Finsupp.coe_finset_sum, Finsupp.coe_smul, F
    Pi.smul_apply, Basis.tensorProduct_repr_tmul_apply, Basis.repr_self, sm
  rw [Finset.sum_eq_single μ, Finset.sum_eq_single ν]
  · simp
  · intro μ' _ h
    simp [h]
  · simp
  · intro ν' _ h
    simp [h]
  · simp

lemma toTensor_deriv_basis_repr_apply {d} (A : DistElectromagneticPotential
  (ε :  $\square$ (SpaceTime d,  $\mathbb{R}$ )) (b : ComponentIdx (S := realLorentzTensor d)
    (Fin.append ![Color.down] ![Color.up])) :
  (Tensor.basis _).repr (Tensorial.toTensor (deriv A ε)) b =
  distDeriv (b 0) A ε (b 1) := by
  rw [Tensorial.basis_toTensor_apply]
  rw [Tensorial.basis_map_prod]
  simp only [Nat.reduceSucc, Nat.reduceAdd, Basis.repr_reindex, Finsupp.map
    Equiv.symm_symm, Fin.isValue]
  rw [Lorentz.Vector.tensor_basis_map_eq_basis_reindex,
    Lorentz.CoVector.tensor_basis_map_eq_basis_reindex]
  have hb : (((Lorentz.CoVector.basis (d := d)).reindex
    Lorentz.CoVector.indexEquiv.symm).tensorProduct

```

1.11.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/DYNAMICS/CURRENTDENSITY.LEAN

```
(Lorentz.Vector.basis.reindex Lorentz.Vector.indexEquiv.symm)) =
  ((Lorentz.CoVector.basis (d := d)).tensorProduct (Lorentz.Vector.basis (d := d)
  (Lorentz.CoVector.indexEquiv.symm.prodCongr Lorentz.Vector.indexEquiv.symm)) :=
  ext b
  match b with
  | ⟨i, j⟩ =>
    simp
  rw [hb]
  rw [Module.Basis.repr_reindex_apply, deriv_basis_repr_apply]
  rfl

/-!

### A.4. Equivariance of the derivative distribution

-/

set_option backward.isDefEq.respectTransparency false in
lemma deriv_equivariant {d} {A : DistElectromagneticPotential d}
  (Λ : LorentzGroup d) : deriv (Λ • A) = Λ • deriv A := by
  rw [deriv, distTensorDeriv_equivariant]

end DistElectromagneticPotential

end Electromagnetism
```

1.11 Physlib/Electromagnetism/Distributional/Dynamics/CurrentDensity.1

```
/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Physlib.SpaceAndTime.SpaceTime.TimeSlice
/-!

# The Lorentz Current Density

## i. Overview

In this module we define the Lorentz current density
and its decomposition into charge density and current density.
```

The Lorentz current density is often called the four-current and given then

The current density is given in terms of the charge density ρ and the current density $\vec{j}$ as $J = (c \rho, \vec{j})$.

ii. Key results

- `DistLorentzCurrentDensity` : The type of Lorentz current densities as distributions.

iii. Table of contents

- A. The Lorentz current density as a distribution
 - A.1. The underlying charge density
 - A.2. The underlying current density

iv. References

-/

@[expose] public section

```
namespace Electromagnetism
open TensorSpecies
open SpaceTime
open TensorProduct
open minkowskiMatrix
open InnerProductSpace
```

```
attribute [-simp] Fintype.sum_sum_type
attribute [-simp] Nat.succ_eq_add_one
```

/-!

A. The Lorentz current density as a distribution

-/

/-- The Lorentz current density, also called four-current as a distribution
 /

```
abbrev DistLorentzCurrentDensity (d : ℕ := 3) := (SpaceTime d) → d[ℝ] LorentzCurrentDensity
```

```
namespace DistLorentzCurrentDensity
```

/-!

A.1. The underlying charge density

1.12.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/DYNAMICS/ISEXTREMA.lean

-/

```
set_option backward.isDefEq.respectTransparency false in
/-- The charge density underlying a Lorentz current density which is a distribution.
/
noncomputable def chargeDensity {d : ℕ} (c : SpeedOfLight) :
  (DistLorentzCurrentDensity d) →1[ℝ] (Time × Space d) → d[ℝ] ℝ where
  toFun J := (1 / (c : ℝ)) • (Lorentz.Vector.temporalCLM d ◦L distTimeSlice c J)
  map_add' J1 J2 := by
    simp
  map_smul' r J := by
    simp only [one_div, map_smul, ContinuousLinearMap.comp_smuls1, RingHom.id_apply]
    rw [smul_comm]
```

/-!

A.2. The underlying current density

-/

```
set_option backward.isDefEq.respectTransparency false in
/-- The underlying (non-Lorentz) current density associated with a distributive
  Lorentz current density. -/
noncomputable def currentDensity (c : SpeedOfLight) :
  DistLorentzCurrentDensity d →1[ℝ] (Time × Space d) → d[ℝ] EuclideanSpace ℝ (Fin d)
  toFun J := Lorentz.Vector.spatialCLM d ◦L distTimeSlice c J
  map_add' J1 J2 := by
    simp
  map_smul' r J := by
    simp
```

end DistLorentzCurrentDensity

end Electromagnetism

1.12 Physlib/Electromagnetism/Distributional/Dynamics/IsExtrema.lean

/-

Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Joseph Tooby-Smith

-/

module

```

public import Physlib.Electromagnetism.Distributional.Dynamics.Lagrangian
/-!

# Extrema of the Lagrangian density

## i. Overview

In this module we define what it means for an electromagnetic potential
to be an extremum of the Lagrangian density in presence of a Lorentz current.

This is equivalent to the electromagnetic potential satisfying
Maxwell's equations with sources, i.e. Gauss's law and Ampère's law.

## ii. Key results

- `IsExtrema` : The condition on an electromagnetic potential to be an extremum.

## iii. Table of contents

- A. Is Extrema condition in the distributional case
  - A.1. IsExtrema and Gauss's law and Ampère's law
  - A.2. IsExtrema in terms of Vector Potentials
  - A.3. The extrema condition in terms of tensors
  - A.4. The invariance of the extrema condition under Lorentz transformations

## iv. References

-/

@[expose] public section
namespace Electromagnetism
open Module realLorentzTensor
open IndexNotation
open TensorSpecies
open Tensor ContDiff

/-!

## A. Is Extrema condition in the distributional case

The above results looked at the extrema condition for electromagnetic potential
functions. We now look at the case where the electromagnetic potential is a
distribution.

-/

namespace DistElectromagneticPotential

```

1.12.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/DYNAMICS/ISEXTREMA.LEAN

*-- The proposition on an electromagnetic potential, corresponding to the statement
it is an extrema of the lagrangian. -/*

```
def IsExtrema {d} (α : FreeSpace)
  (A : DistElectromagneticPotential d)
  (J : DistLorentzCurrentDensity d) : Prop := A.gradLagrangian α J = 0
```

```
lemma isExtrema_iff_gradLagrangian {α : FreeSpace}
  (A : DistElectromagneticPotential d)
  (J : DistLorentzCurrentDensity d) :
  IsExtrema α A J ↔ A.gradLagrangian α J = 0 := by rfl
```

```
lemma isExtrema_iff_components {α : FreeSpace}
  (A : DistElectromagneticPotential d)
  (J : DistLorentzCurrentDensity d) :
  IsExtrema α A J ↔ (∀ ε, A.gradLagrangian α J ε (Sum.inl 0) = 0)
  ∧ (∀ ε i, A.gradLagrangian α J ε (Sum.inr i) = 0) := by
  apply Iff.intro
  · intro h
    rw [isExtrema_iff_gradLagrangian] at h
    simp [h]
  · intro h
    rw [isExtrema_iff_gradLagrangian]
    ext ε
    funext i
    match i with
    | Sum.inl 0 => exact h.1 ε
    | Sum.inr j => exact h.2 ε j
/-!
```

A.1. IsExtrema and Gauss's law and Ampère's law

We show that `A` is an extrema of the lagrangian if and only if Gauss's law and Ampère's law hold. In other words,

$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0}$
and
 $\mu_0 \epsilon_0 \frac{\partial \mathbf{E}_i}{\partial t} - \sum_j \frac{\partial \mathbf{B}_j}{\partial x_j} = 0$
Here $\mathbf{B}$ is the magnetic field matrix.

```
-/
open Space
set_option backward.isDefEq.respectTransparency false in
lemma isExtrema_iff_space_time {α : FreeSpace}
  (A : DistElectromagneticPotential d)
```

```

(J : DistLorentzCurrentDensity d) :
IsExtrema  $\square$  A J  $\leftrightarrow$ 
  ( $\forall \varepsilon$ , distSpaceDiv (A.electricField  $\square$ .c)  $\varepsilon = (1/\square.\varepsilon_0) * (J.chargeDens$ 
    ( $\forall \varepsilon i$ ,  $\square.\mu_0 * \square.\varepsilon_0 * (\text{Space.distTimeDeriv (A.electricField } \square.c)) \varepsilon i$ 
       $\sum j$ , ((PiLp.basisFun 2  $\mathbb{R}$  (Fin d)).tensorProduct (PiLp.basisFun 2  $\mathbb{R}$  (F
        ((Space.distSpaceDeriv j (A.magneticFieldMatrix  $\square$ .c))  $\varepsilon$ ) (j, i) +
         $\square.\mu_0 * J.currentDensity \square.c \varepsilon i = 0$ ) := by
rw [isExtrema_iff_components]
refine and_congr ?_ ?_
· simp [gradLagrangian_sum_inl_0]
  field_simp
  simp [ $\square.c\_sq$ ]
  field_simp
  simp [sub_eq_zero]
  apply Iff.intro
· intro h  $\varepsilon$ 
  convert h (SchwartzMap.compCLMOfContinuousLinearEquiv (F :=  $\mathbb{R}$ )  $\mathbb{R}$ 
    (SpaceTime.toTimeAndSpace  $\square.c$  (d := d))  $\varepsilon$ ) using 1
· simp [SpaceTime.distTimeSlice_symm_apply]
  ring_nf
  congr
  ext x
  simp
· simp [SpaceTime.distTimeSlice_symm_apply]
  congr
  ext x
  simp
· intro h  $\varepsilon$ 
  convert h (SchwartzMap.compCLMOfContinuousLinearEquiv (F :=  $\mathbb{R}$ )  $\mathbb{R}$ 
    (SpaceTime.toTimeAndSpace  $\square.c$  (d := d)).symm  $\varepsilon$ ) using 1
· simp [SpaceTime.distTimeSlice_symm_apply]
  ring_nf
· apply Iff.intro
· intro h  $\varepsilon i$ 
  specialize h (SchwartzMap.compCLMOfContinuousLinearEquiv (F :=  $\mathbb{R}$ )  $\mathbb{R}$ 
    (SpaceTime.toTimeAndSpace  $\square.c$  (d := d))  $\varepsilon$ ) i
  linear_combination (norm := field_simp) ( $\square.\mu_0$ ) * h
  simp [gradLagrangian_sum_inr_i, SpaceTime.distTimeSlice_symm_apply]
  have hx : (SchwartzMap.compCLMOfContinuousLinearEquiv  $\mathbb{R}$  (SpaceTime.to
    ((SchwartzMap.compCLMOfContinuousLinearEquiv  $\mathbb{R}$  (SpaceTime.toTimeA
      =  $\varepsilon$  := by
    ext i
    simp
  simp [hx,  $\square.c\_sq$ ]
  field_simp
  ring

```

1.12.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/DYNAMICS/ISEXTREMA.LEAN

```

· intro h ε i
  specialize h (SchwartzMap.compCLM0fContinuousLinearEquiv (F := ℝ) ℝ
    (SpaceTime.toTimeAndSpace □.c (d := d)).symm ε) i
  linear_combination (norm := field_simp) (□.μ₀⁻¹) * h
  simp [gradLagrangian_sum_inr_i, SpaceTime.distTimeSlice_symm_apply, □.c_sq]
  field_simp
  ring

/-!

### A.2. IsExtrema in terms of Vector Potentials

We show that `A` is an extrema of the lagrangian if and only if Gauss's law and Ampère's law hold.
In other words,


$$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0}$$

and

$$\mu_0 \epsilon_0 \frac{\partial \mathbf{E}_i}{\partial t} - \sum_j \left( \frac{\partial \mathbf{E}_j}{\partial x_j} \frac{\partial \mathbf{A}_i}{\partial x_j} - \frac{\partial \mathbf{E}_i}{\partial x_j} \frac{\partial \mathbf{A}_j}{\partial x_i} \right) + \mu_0 \mathbf{J}_i = 0.$$


-/

lemma isExtrema_iff_vectorPotential {□ : FreeSpace}
  (A : DistElectromagneticPotential d)
  (J : DistLorentzCurrentDensity d) :
  IsExtrema □ A J ↔
    (∀ ε, distSpaceDiv (A.electricField □.c) ε = (1/□.ε₀) * (J.chargeDensity □.c))
    (∀ ε i, □.μ₀ * □.ε₀ * distTimeDeriv (A.electricField □.c) ε i -
      (∑ x, -(distSpaceDeriv x (distSpaceDeriv x (A.vectorPotential □.c)) ε i
        - distSpaceDeriv x (distSpaceDeriv i (A.vectorPotential □.c)) ε x)) +
      □.μ₀ * J.currentDensity □.c ε i = 0) := by
  rw [isExtrema_iff_space_time]
  refine and_congr (by rfl) ?_
  suffices ∀ ε i, ∑ x, -(distSpaceDeriv x (distSpaceDeriv x (A.vectorPotential □.c)) ε i
    - distSpaceDeriv x (distSpaceDeriv i (A.vectorPotential □.c)) ε x) =
    ∑ j, ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d)))
      ((Space.distSpaceDeriv j (A.magneticFieldMatrix □.c)) ε) (j, i) by
    conv_lhs => enter [2, 2]; rw [← this]
  intro ε i
  congr
  funext j
  rw [magneticFieldMatrix_distSpaceDeriv_basis_repr_eq_vector_potential]
  ring

/-!

```

A.3. The extrema condition in terms of tensors

We show that A is an extrema of the lagrangian if and only if the equation $\frac{1}{\mu_0} \partial_\kappa F^{\kappa \nu} - J^\nu = 0$, holds.

```

-/
open SpaceTime minkowskiMatrix
set_option maxHeartbeats 600000 in
set_option backward.isDefEq.respectTransparency false in
lemma isExtrema_iff_tensor {α : FreeSpace}
  (A : DistElectromagneticPotential d)
  (J : DistLorentzCurrentDensity d) :
  IsExtrema α A J ↔ ∀ ε,
    {((1/ α.μ₀ : ℝ) • distTensorDeriv A.fieldStrength ε | κ κ ν') + - (J ε
  apply Iff.intro
  · intro h
    simp only [IsExtrema] at h
    intro x
    have h1 : ((Tensorial.toTensor (M := Lorentz.Vector d)).symm
      (permT id (PermCond.auto) {((1/ α.μ₀ : ℝ) • distTensorDeriv A.field
        - (J x | ν'))}ᵀ)) = 0 := by
      funext v
      have h2 : gradLagrangian α A J x v = 0 := by simp [h]
      rw [gradLagrangian_eq_tensor A J] at h2
      simp at h2
      have hn : minkowskiMatrix v v ≠ 0 := minkowskiMatrix.η_diag_ne_zero
      simp_all
      rw [EmbeddingLike.map_eq_zero_iff, permT_eq_zero_iff] at h1
      exact h1
  · intro h
    simp only [IsExtrema]
    ext x
    funext v
    rw [gradLagrangian_eq_tensor A J, h]
    simp

/-!

```

A.4. The invariance of the extrema condition under Lorentz transformation

We show that the Extrema condition is invariant under Lorentz transformation. This implies that if an electromagnetic potential is an extrema in one inertial frame, it is also an extrema in any other inertial frame. In other words that the Maxwell's equations are Lorentz invariant.

1.13.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/DYNAMICS/KINETICTERM.LEAN

A natural consequence of this is that the speed of light is the same in all inertial

-/

```
set_option backward.isDefEq.respectTransparency false in
lemma isExterna_equivariant {α : FreeSpace}
  (A : DistElectromagneticPotential d)
  (J : DistLorentzCurrentDensity d) (Λ : LorentzGroup d) :
  IsExtrema α (Λ • A) (Λ • J) ↔ IsExtrema α A J := by
  rw [isExterna_iff_tensor]
  conv_lhs =>
    enter [x, 1, 1, 2, 2, 2]
    rw [fieldStrength_equivariant, distTensorDeriv_equivariant]
    rw [lorentzGroup_smul_dist_apply]
  conv_lhs =>
    enter [x]
    rw [smul_comm]
    rw [Tensorial.toTensor_smul, lorentzGroup_smul_dist_apply, Tensorial.toTensor_smul]
    simp only [one_div, map_smul, actionT_smul,
      contrT_equivariant, map_neg, permT_equivariant]
    rw [smul_comm, ← Tensor.actionT_neg, ← Tensor.actionT_add]
  apply Iff.intro
  · intro h
    rw [isExterna_iff_tensor A J]
    intro x
    apply MulAction.injective Λ
    simp only [one_div, map_smul, map_neg,
      _root_.smul_add, actionT_smul, _root_.smul_neg, _root_.smul_zero]
    simpa only [Fin.isValue, schwartzAction_mul_apply, inv_mul_cancel, map_one,
      ContinuousLinearMap.one_apply, smul_add, actionT_smul, smul_neg] using h (schw
  · intro h x
    rw [isExterna_iff_tensor A J] at h
    specialize h (schwartzAction Λ-1 x)
    simp only [Nat.reduceAdd, Nat.succ_eq_add_one, Fin.isValue, one_div, map_smul, m
    rw [h]
    simp

end DistElectromagneticPotential
end Electromagnetism
```

1.13 Physlib/Electromagnetism/Distributional/Dynamics/KineticTerm.lean

/-

Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.
 Authors: Joseph Tooby-Smith

-/

module

```
public import Physlib.Electromagnetism.Distributional.MagneticField
public import Physlib.Electromagnetism.Dynamics.Basic
public import Physlib.Mathematics.VariationalCalculus.HasVarGradient
/-!
```

The kinetic term

i. Overview

The kinetic term of the electromagnetic field is $-\frac{1}{4\mu_0} F_{\mu\nu} F^{\mu\nu}$.
 We define this, show it is invariant under Lorentz transformations,
 and show properties of its variational gradient.

In particular the variational gradient `gradKineticTerm` of the kinetic term
 is directly related to Gauss's law and the Ampere law.

In this implementation we have set $\mu_0 = 1$. It is a TODO to introduce this

ii. Key results

- `DistElectromagneticPotential.gradKineticTerm` is the variational gradient
 for distributional electromagnetic potentials.

iii. Table of contents

- A. The gradient of the kinetic term for distributions
 - A.1. The gradient of the kinetic term as a tensor

iv. References

- https://quantummechanics.ucsd.edu/ph130a/130_notes/node452.html

-/

@[expose] public section

```
namespace Electromagnetism
open Module realLorentzTensor
open IndexNotation
open TensorSpecies
open Tensor ContDiff Physlib
```


1.13.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/DYNAMICS/KINETIC TERM. LEAN

/-!

A. The gradient of the kinetic term for distributions

For distributions we define the gradient of the kinetic term directly using `ElectromagneticPotential.gradKineticTerm\_eq\_sum\_sum` as the defining formula.

-/

```
namespace DistElectromagneticPotential
open minkowskiMatrix SpaceTime SchwartzMap Lorentz
attribute [-simp] Fintype.sum_sum_type
attribute [-simp] Nat.succ_eq_add_one
```

/-- The gradient of the kinetic term for an Electromagnetic potential which is a distribution. -/

```
noncomputable def gradKineticTerm {d} (□ : FreeSpace) :
  DistElectromagneticPotential d →1[ℝ] (SpaceTime d) →d[ℝ] Lorentz.Vector d where
toFun A := {
  toFun ε := ∑ v, ∑ μ,
    (1 / (□.μ₀) * (η μ μ * η v v * distDeriv μ (distDeriv μ A) ε v -
      distDeriv μ (distDeriv v A) ε μ)) • Lorentz.Vector.basis v
  map_add' ε1 ε2 := by
    rw [← Finset.sum_add_distrib]
    apply Finset.sum_congr rfl (fun v _ => ?_)
    rw [← Finset.sum_add_distrib]
    apply Finset.sum_congr rfl (fun μ _ => ?_)
    simp only [one_div, map_add, Lorentz.Vector.apply_add, ← add_smul]
    ring_nf
  map_smul' r ε := by
    simp [Finset.smul_sum, smul_smul]
    apply Finset.sum_congr rfl (fun v _ => ?_)
    apply Finset.sum_congr rfl (fun μ _ => ?_)
    ring_nf
  cont := by fun_prop}
map_add' A1 A2 := by
  ext ε
  simp only [one_div, map_add, ContinuousLinearMap.add_apply, Lorentz.Vector.apply
    ContinuousLinearMap.coe_mk', LinearMap.coe_mk, AddHom.coe_mk]
  rw [← Finset.sum_add_distrib]
  apply Finset.sum_congr rfl (fun v _ => ?_)
  rw [← Finset.sum_add_distrib]
  apply Finset.sum_congr rfl (fun μ _ => ?_)
  simp only [← add_smul]
  ring_nf
```

```

map_smul' r A := by
  ext ε
  simp only [one_div, map_smul, ContinuousLinearMap.smul_apply, Lorentz.V
    ContinuousLinearMap.coe_mk', LinearMap.coe_mk, AddHom.coe_mk]
  simp [Finset.smul_sum, smul_smul]
  apply Finset.sum_congr rfl (fun v _ => ?_)
  apply Finset.sum_congr rfl (fun μ _ => ?_)
  ring_nf

lemma gradKineticTerm_eq_sum_sum {d} {α : FreeSpace}
  (A : DistElectromagneticPotential d) (ε : α(SpaceTime d, ℝ)) :
  A.gradKineticTerm α ε = ∑ v, ∑ μ,
    (1 / (α.μ₀) * (η μ μ * η v v * distDeriv μ (distDeriv μ A) ε v -
      distDeriv μ (distDeriv v A) ε μ)) • Lorentz.Vector.basis v := rfl

set_option backward.isDefEq.respectTransparency false in
lemma gradKineticTerm_eq_fieldStrength {d} {α : FreeSpace} (A : DistElectro
  (ε : α(SpaceTime d, ℝ)) :
  A.gradKineticTerm α ε = ∑ v, (1/α.μ₀ * η v v) •
    (∑ μ, ((Vector.basis.tensorProduct Vector.basis).repr
      (distDeriv μ (A.fieldStrength) ε) (μ, v))) • Lorentz.Vector.basis v :
  rw [gradKineticTerm_eq_sum_sum A]
  apply Finset.sum_congr rfl (fun v _ => ?_)
  rw [smul_smul, ← Finset.sum_smul, ← Finset.mul_sum, mul_assoc]
  congr 2
  rw [Finset.mul_sum]
  apply Finset.sum_congr rfl (fun μ _ => ?_)
  conv_rhs =>
    rw [distDeriv_apply, Distribution.fderivD_apply, map_neg]
    simp only [Finsupp.coe_neg, Pi.neg_apply, mul_neg]
    rw [fieldStrength_basis_repr_eq_single]
    simp only
    rw [SpaceTime.apply_fderiv_eq_distDeriv, SpaceTime.apply_fderiv_eq_dist
    simp
  ring_nf
  simp

set_option backward.isDefEq.respectTransparency false in
lemma gradKineticTerm_sum_inl_eq {d} {α : FreeSpace}
  (A : DistElectromagneticPotential d) (ε : α(SpaceTime d, ℝ)) :
  A.gradKineticTerm α ε (Sum.inl 0) =
    (1/(α.μ₀ * α.c) * (distTimeSlice α.c).symm (Space.distSpaceDiv (A.elect
  rw [gradKineticTerm_eq_fieldStrength A ε, Lorentz.Vector.apply_sum, distT
    Space.distSpaceDiv_apply_eq_sum_distSpaceDeriv, Finset.mul_sum]
  simp [Fintype.sum_sum_type, Finset.mul_sum]
  apply Finset.sum_congr rfl (fun v _ => ?_)

```

1.13.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/DYNAMICS/KINETICTERM.LEAN

```

rw [← distTimeSlice_symm_apply]
conv_rhs =>
  enter [2]
  rw [distTimeSlice_symm_apply, Space.distSpaceDeriv_apply']
  simp only [PiLp.neg_apply]
  rw [electricField_eq_fieldStrength, distTimeSlice_apply]
  simp only [Fin.isValue, neg_mul, neg_neg]
  rw [fieldStrength_antisymmetric_basis]
  rw [← distTimeSlice_apply, Space.apply_fderiv_eq_distSpaceDeriv, ← distTimeSlice
    ← distTimeSlice_distDeriv_inr]
  simp
field_simp

set_option backward.isDefEq.respectTransparency false in
lemma gradKineticTerm_sum_inr_eq {d} {α : FreeSpace}
  (A : DistElectromagneticPotential d) (ε : α (SpaceTime d, ℝ)) (i : Fin d) :
  A.gradKineticTerm α ε (Sum.inr i) =
  (α.μ₀⁻¹ * (1 / α.c ^ 2 * (distTimeSlice α.c).symm
    (Space.distTimeDeriv (A.electricField α.c)) ε i -
    ∑ j, ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).r
      ((distTimeSlice α.c).symm (Space.distSpaceDeriv j
        (A.magneticFieldMatrix α.c)) ε) (j, i))) := by
  simp [gradKineticTerm_eq_fieldStrength A ε, Lorentz.Vector.apply_sum,
    Fintype.sum_sum_type, mul_add, sub_eq_add_neg]
congr
· conv_rhs =>
  enter [2, 2]
  rw [distTimeSlice_symm_apply, Space.distTimeDeriv_apply']
  simp only [PiLp.neg_apply]
  rw [electricField_eq_fieldStrength, Space.apply_fderiv_eq_distTimeDeriv,
    ← distTimeSlice_symm_apply]
  simp [distTimeSlice_symm_distTimeDeriv_eq]
  field_simp
· ext k
conv_rhs =>
  rw [distTimeSlice_symm_apply, Space.distSpaceDeriv_apply']
  simp only [map_neg, Finsupp.coe_neg, Pi.neg_apply]
  rw [magneticFieldMatrix_basis_repr_eq_fieldStrength, Space.apply_fderiv_eq_dis
    ← distTimeSlice_symm_apply]
  simp [← distTimeSlice_distDeriv_inr]

/-!

### A.1. The gradient of the kinetic term as a tensor

-/

```

```

set_option backward.isDefEq.respectTransparency false in
attribute [-simp] Nat.reduceAdd Nat.reduceSucc Fin.isValue in
lemma gradKineticTerm_eq_distTensorDeriv {d} {α : FreeSpace}
  (A : DistElectromagneticPotential d) (ε : α(SpaceTime d, ℝ)) (v : Fin 1)
  A.gradKineticTerm α ε v = η v v * ((Tensorial.toTensor (M := Lorentz.Vector
    (permT id (PermCond.auto) {(1/ α.μ₀ : ℝ) •
      distTensorDeriv A.fieldStrength ε | κ κ v'}T))) v := by
trans η v v * (Lorentz.Vector.basis_repr
  ((Tensorial.toTensor (M := Lorentz.Vector d)).symm
    (permT id (PermCond.auto) {(1/ α.μ₀ : ℝ) • distTensorDeriv A.fieldStreng
swap
· rfl
simp [Lorentz.Vector.basis_eq_map_tensor_basis]
rw [permT_basis_repr_symm_apply, contrT_basis_repr_apply_eq_fin]
conv_lhs =>
  rw [gradKineticTerm_eq_fieldStrength A ε]
  simp [Lorentz.Vector.apply_sum]
ring_nf
congr 1
congr
funext μ
rw [distTensorDeriv_toTensor_basis_repr]
conv_rhs =>
  enter [1, 2, 2]
trans (Tensor.basis _).repr (Tensorial.toTensor (distDeriv μ (A.fieldStre
  (fun | 0 => μ | 1 => v)
· generalize (distDeriv μ (A.fieldStrength) ε) = t at *
rw [Tensorial.basis_toTensor_apply]
rw [Tensorial.basis_map_prod]
simp only [Basis.repr_reindex, Finsupp.mapDomain_equiv_apply,
  Equiv.symm_symm]
rw [Lorentz.Vector.tensor_basis_map_eq_basis_reindex]
have hb : (((Lorentz.Vector.basis (d := d)).reindex
  Lorentz.Vector.indexEquiv.symm).tensorProduct
  (Lorentz.Vector.basis.reindex Lorentz.Vector.indexEquiv.symm)) =
  ((Lorentz.Vector.basis (d := d)).tensorProduct (Lorentz.Vector.basi
  (Lorentz.Vector.indexEquiv.symm.prodCongr Lorentz.Vector.indexEquiv
ext b
match b with
| ⟨i, j⟩ =>
  simp
rw [hb]
rw [Module.Basis.repr_reindex_apply]
rfl
apply congr

```

1.14.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/DYNAMICS/LAGRANGIAN. LEAN

```
· simp
  rfl
funext x
fin_cases x
· simp only [Function.comp_apply]
  simp [ComponentIdx.prod]
  simp [ComponentIdx.DropPairSection.ofFinEquiv, ComponentIdx.DropPairSection.ofFi
  intro _ h
  apply False.elim
  apply h
  decide
· simp only [Function.comp_apply]
  simp [ComponentIdx.prod]
  simp [ComponentIdx.DropPairSection.ofFinEquiv, ComponentIdx.DropPairSection.ofFi
  split_ifs
  · rename_i h
    suffices ~ (finSumFinEquiv (Sum.inr 1) = (0 : Fin (1 + 1 + 1))) from False.eli
    decide
  · rename_i h h2
    suffices ~ (finSumFinEquiv (Sum.inr 1) = (1 : Fin (1 + 1 + 1))) from False.eli
    decide
  · rfl

end DistElectromagneticPotential
end Electromagnetism
```

1.14 Physlib/Electromagnetism/Distributional/Dynamics/Lagrangian.lean

```
/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Physlib.Electromagnetism.Distributional.Dynamics.CurrentDensity
public import Physlib.Electromagnetism.Distributional.Dynamics.KineticTerm
public import Physlib.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct
/-!

# The Lagrangian in electromagnetism

## i. Overview
```

In this module we define the Lagrangian density for the electromagnetic field in the presence of a current density. We prove properties of this Lagrangian density and find its variational gradient.

The Lagrangian density is given by
 $\mathcal{L} = -1/(4 \mu_0) F_{\{\mu\nu\}} F^{\{\mu\nu\}} - A_\mu J^\mu$

In this implementation we set $\mu_0 = 1$. It is a TODO to introduce this constant.

ii. Key results

- ``gradFreeCurrentPotential`` : The variational gradient of the free current potential
- ``gradLagrangian`` : The variational gradient of the Lagrangian density.

iii. Table of contents

- A. The gradient of the Lagrangian density for distributions
 - A.1. The gradient of the free current potential
 - A.1.1. Free current potential as a tensor
 - A.2. The gradient of the Lagrangian density
 - A.2.1. The Lagrangian gradient as a tensor

iv. References

- https://quantummechanics.ucsd.edu/ph130a/130_notes/node452.html
- <https://ph.qmul.ac.uk/sites/default/files/EMT10new.pdf>

-/

@[expose] public section

```
namespace Electromagnetism
open Module realLorentzTensor
open IndexNotation
open TensorSpecies
open Tensor ContDiff
```

/-!

A. The gradient of the Lagrangian density for distributions

-/

```
namespace DistElectromagneticPotential
open TensorSpecies
open Tensor
```

1.14.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/DYNAMICS/LAGRANGIAN.LEAN

```
open SpaceTime
open TensorProduct
open minkowskiMatrix
open InnerProductSpace
open Lorentz.Vector SchwartzMap
attribute [-simp] Fintype.sum_sum_type
attribute [-simp] Nat.succ_eq_add_one
/-!
```

A.1. The gradient of the free current potential

We define this through the lemma `gradFreeCurrentPotential\_eq\_sum\_basis`
 -/

/-- The variational gradient of the free current potential for distributional potent
 /

```
noncomputable def gradFreeCurrentPotential {d} :
  DistLorentzCurrentDensity d →1[ℝ] ((SpaceTime d) →d[ℝ] Lorentz.Vector d) where
  toFun J := {
    toFun ε := ∑ μ, (η μ μ • (J ε μ) • Lorentz.Vector.basis μ)
    map_add' ε1 ε2 := by
      simp [Finset.sum_add_distrib, add_smul]
    map_smul' r ε := by
      simp only [map_smul, apply_smul, smul_smul, Real.ringHom_apply, Finset.smul_s
      congr
      funext i
      ring_nf
    cont := by fun_prop
  }
  map_add' J1 J2 := by
    ext ε
    simp [Finset.sum_add_distrib, add_smul]
  map_smul' r J := by
    ext ε
    simp [Finset.smul_sum, smul_smul]
    congr
    funext i
    ring_nf
```

```
lemma gradFreeCurrentPotential_eq_sum_basis {d}
  (J : DistLorentzCurrentDensity d) (ε : □(SpaceTime d, ℝ)) :
  (gradFreeCurrentPotential J) ε =
  (∑ μ, (η μ μ • (J ε μ) • Lorentz.Vector.basis μ)) := rfl
```

```
set_option backward.isDefEq.respectTransparency false in
lemma gradFreeCurrentPotential_sum_inl_0 (□ : FreeSpace) {d}
```

```

(J : DistLorentzCurrentDensity d) (ε :  $\square$ (SpaceTime d,  $\mathbb{R}$ )) :
(gradFreeCurrentPotential J) ε (Sum.inl 0) =
   $\square$ .c * (distTimeSlice  $\square$ .c).symm (J.chargeDensity  $\square$ .c) ε := by
simp only [gradFreeCurrentPotential, LinearMap.coe_mk, AddHom.coe_mk, Fin
ContinuousLinearMap.coe_mk', apply_sum, apply_smul, Lorentz.Vector.basi
mul_one, mul_zero, Finset.sum_ite_eq', Finset.mem_univ,  $\downarrow$ reduceIte, inl
DistLorentzCurrentDensity.chargeDensity, one_div, temporalCLM, map_smul
ContinuousLinearMap.coe_smul', Pi.smul_apply, distTimeSlice_symm_apply,
ContinuousLinearMap.coe_comp', LinearMap.coe_toContinuousLinearMap', Fu
smul_eq_mul, ne_eq, SpeedOfLight.val_ne_zero, not_false_eq_true, mul_in
rw [← distTimeSlice_symm_apply]
simp

set_option backward.isDefEq.respectTransparency false in
lemma gradFreeCurrentPotential_sum_inr_i ( $\square$  : FreeSpace) {d}
(J : DistLorentzCurrentDensity d) (ε :  $\square$ (SpaceTime d,  $\mathbb{R}$ )) (i : Fin d) :
(gradFreeCurrentPotential J) ε (Sum.inr i) =
  - (distTimeSlice  $\square$ .c).symm (J.currentDensity  $\square$ .c) ε i := by
simp only [gradFreeCurrentPotential, LinearMap.coe_mk, AddHom.coe_mk, Con
apply_sum, apply_smul, Lorentz.Vector.basis_apply, mul_ite, mul_one, mu
Finset.sum_ite_eq', Finset.mem_univ,  $\downarrow$ reduceIte, inr_i_inr_i,
DistLorentzCurrentDensity.currentDensity, spatialCLM, distTimeSlice_symm
ContinuousLinearMap.coe_comp', Function.comp_apply]
rw [← distTimeSlice_symm_apply]
simp

/-!

#### A.1.1. Free current potential as a tensor

-/

lemma gradFreeCurrentPotential_eq_tensor {d}
(J : DistLorentzCurrentDensity d) (ε :  $\square$ (SpaceTime d,  $\mathbb{R}$ ))
(v : Fin 1  $\oplus$  Fin d) :
gradFreeCurrentPotential J ε v =  $\eta$  v v * ((Tensorial.toTensor (M := Lor
(permt id (PermCond.auto) {J ε | v'}T)) v := by
trans  $\eta$  v v * (Lorentz.Vector.basis.repr ((Tensorial.toTensor (M := Loren
(permt id (PermCond.auto) {J ε | v'}T))) v
swap
· simp [Lorentz.Vector.basis_repr_apply]
simp [Lorentz.Vector.basis_repr_apply]
rw [gradFreeCurrentPotential_eq_sum_basis]
simp [Lorentz.Vector.apply_sum]

/-!

```


1.14.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/DYNAMICS/LAGRANGIAN.LEAN

D.2. The gradient of the lagrangian density

Defined through `gradLagrangian\_eq\_kineticTerm\_sub`.

-/

-- The variational gradient of lagrangian for an electromagnetic potential which is a distribution. -/

```
noncomputable def gradLagrangian {d} (α : FreeSpace) (A : DistElectromagneticPotential d) (J : DistLorentzCurrentDensity d) : ((SpaceTime d) → d[ℝ] Lorentz.Vector d) :=
  A.gradKineticTerm α - gradFreeCurrentPotential J
```

```
lemma gradLagrangian_sum_inl_0 {α : FreeSpace}
  (A : DistElectromagneticPotential d) (J : DistLorentzCurrentDensity d)
  (ε : α(SpaceTime d, ℝ)) :
  A.gradLagrangian α J ε (Sum.inl 0) =
  (1/(α.μ₀ * α.c) * (distTimeSlice α.c).symm (Space.distSpaceDiv (A.electricField α.c)
  - α.c * (distTimeSlice α.c).symm (J.chargeDensity α.c) ε := by
  simp [gradLagrangian, gradKineticTerm_sum_inl_eq, gradFreeCurrentPotential_sum_inl_0])
```

set_option backward.isDefEq.respectTransparency false in

```
lemma gradLagrangian_sum_inr_i {α : FreeSpace}
  (A : DistElectromagneticPotential d) (J : DistLorentzCurrentDensity d)
  (ε : α(SpaceTime d, ℝ)) (i : Fin d) :
  A.gradLagrangian α J ε (Sum.inr i) =
  α.μ₀⁻¹ * (1 / α.c ^ 2 *
  (distTimeSlice α.c).symm (Space.distTimeDeriv (A.electricField α.c)) ε i -
  ∑ j, ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).r
  ((distTimeSlice α.c).symm (Space.distSpaceDeriv j (A.magneticFieldMatrix α.c)
  (distTimeSlice α.c).symm (J.currentDensity α.c) ε i := by
  simp [gradLagrangian, gradKineticTerm_sum_inr_eq, gradFreeCurrentPotential_sum_inr_i])
```

/-!

A.2.1. The lagrangian gradient as a tensor

-/

attribute [-simp] Nat.reduceAdd Nat.reduceSucc Fin.isValue in

set_option backward.isDefEq.respectTransparency false in

```
lemma gradLagrangian_eq_tensor {α : FreeSpace}
  (A : DistElectromagneticPotential d) (J : DistLorentzCurrentDensity d)
  (ε : α(SpaceTime d, ℝ)) (v : Fin 1 ⊕ Fin d) :
  A.gradLagrangian α J ε v =
  η v v * ((Tensorial.toTensor (M := Lorentz.Vector d)).symm
```

```

      (permT id (PermCond.auto) {((1/ □.μ₀ : ℝ) • (distTensorDeriv A.fieldStr
      - (J ε | v'))}ᵀ)) v := by
    rw [gradLagrangian]
    simp only [ContinuousLinearMap.coe_sub', Pi.sub_apply, apply_sub, one_div,
      map_smul, map_neg, map_add, permT_permT, CompTriple.comp_eq, apply_add,
      apply_smul, Lorentz.Vector.neg_apply]
    rw [gradKineticTerm_eq_distTensorDeriv, gradFreeCurrentPotential_eq_tenso
    simp only [one_div, map_smul, apply_smul,
      permT_id_self, LinearEquiv.symm_apply_apply]
    ring_nf
    congr
    rw [permT_congr_eq_id]
    simp only [LinearEquiv.symm_apply_apply]
    funext i
    fin_cases i
    simp

end DistElectromagneticPotential
end Electromagnetism

```

1.15 Physlib/Electromagnetism/Distributional/ElectricField.lean

```

/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Physlib.Electromagnetism.Distributional.VectorPotential
public import Physlib.Electromagnetism.Distributional.ScalarPotential
public import Physlib.Electromagnetism.Distributional.FieldStrength
public import Physlib.Electromagnetism.Basic
/-!

# The Electric Field

## i. Overview

The electric field is defined in terms of the electromagnetic potential `A`
`E = - ∇ φ - ∂ₜ \vec A`.

In this module we define the electric field, and prove lemmas about it.

```

1.15.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/ELECTRICFIELD.LEAN

ii. Key results

- `DistElectromagneticPotential.electricField` : The electric field for electromagnetic potentials which are distributions.

iii. Table of contents

- A. Electric field for distributions

iv. References

-/

```
@[expose] public section
namespace Electromagnetism
open Module realLorentzTensor
open IndexNotation
open TensorSpecies
open Tensor
```

/-!

A. Electric field for distributions

-/

```
namespace DistElectromagneticPotential
open TensorSpecies
open Tensor
open SpaceTime
open TensorProduct
open minkowskiMatrix SchwartzMap Lorentz
attribute [-simp] Fintype.sum_sum_type
attribute [-simp] Nat.succ_eq_add_one
```

/-- The electric field of an electromagnetic potential which is a distribution. -
/

```
noncomputable def electricField {d} (c : SpeedOfLight) :
  DistElectromagneticPotential d →1 [ℝ]
  (Time × Space d) →d [ℝ] EuclideanSpace ℝ (Fin d) where
toFun A := - Space.distSpaceGrad (A.scalarPotential c) -
  Space.distTimeDeriv (A.vectorPotential c)
map_add' A1 A2 := by
  ext ε i
  simp only [map_add, neg_add_rev, ContinuousLinearMap.coe_sub', Pi.sub_apply,
    ContinuousLinearMap.add_apply, ContinuousLinearMap.neg_apply, PiLp.sub_apply,
```

```

        PiLp.neg_apply]
    ring
    map_smul' r A := by
    ext  $\varepsilon$  i
    simp only [map_smul, ContinuousLinearMap.coe_sub', ContinuousLinearMap.
        ContinuousLinearMap.neg_apply, Pi.smul_apply, PiLp.sub_apply, PiLp.ne
        smul_eq_mul, Real.ringHom_apply]
    ring

set_option backward.isDefEq.respectTransparency false in
lemma electricField_eq_fieldStrength {d} {c : SpeedOfLight}
  (A : DistElectromagneticPotential d) ( $\varepsilon$  :  $\square(\text{Time} \times \text{Space } d, \mathbb{R})$ )
  (i : Fin d) : A.electricField c  $\varepsilon$  i = - c * (Vector.basis.tensorProduct
    (distTimeSlice c (A.fieldStrength)  $\varepsilon$ ) (Sum.inl 0, Sum.inr i)) := by
  simp only [distTimeSlice_apply, Fin.isValue, fieldStrength_basis_repr_eq_
    one_mul, inr_i_inr_i, neg_mul, sub_neg_eq_add]
  simp only [electricField, scalarPotential, Vector.temporalCLM, Fin.isValu
    ContinuousLinearMap.comp_smulsl, Real.ringHom_apply, LinearMap.coe_mk,
    vectorPotential, Vector.spatialCLM, Space.distTimeDeriv_apply_CLM, Cont
    ContinuousLinearMap.coe_comp', ContinuousLinearMap.coe_mk', Pi.sub_appl
    ContinuousLinearMap.neg_apply, ContinuousLinearMap.coe_smul', Pi.smul_a
    Function.comp_apply, PiLp.sub_apply, PiLp.neg_apply, PiLp.smul_apply, S
    Space.distSpaceDeriv_apply_CLM, LinearMap.coe_toContinuousLinearMap', s
    ← distTimeSlice_apply, distTimeSlice_distDeriv_inl, one_div, Vector.app
    distTimeSlice_distDeriv_inr]
  field_simp
  ring

end DistElectromagneticPotential

end Electromagnetism

```

1.16 Physlib/Electromagnetism/Distributional/FieldStrength.lean

```

/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Physlib.Electromagnetism.Distributional.Basic
public import Physlib.Relativity.Tensors.RealTensor.Metrics.Basic
public import Mathlib.Data.Real.Hom

```

1.16.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/FIELDSTRENGTH.LEAN

/-!

The Field Strength Tensor

i. Overview

In this module we define the field strength tensor in terms of the electromagnetic p

ii. Key results

- `DistElectromagneticPotential.fieldStrength` : The field strength for
electromagnetic potentials which are distributions.

iii. Table of contents

- A. Field strength for distributions
 - A.1. Auxiliary definition of field strength for distributions, with no linearity
 - A.2. The definition of the field strength
 - A.3. Field strength written in terms of a basis
 - A.4. Equivariance of the field strength for distributions

iv. References

-/

```
@[expose] public section
namespace Electromagnetism
open Module realLorentzTensor
open IndexNotation
open TensorSpecies
open Tensor
```

/-!

A. Field strength for distributions

-/

```
namespace DistElectromagneticPotential
open TensorSpecies
open Tensor
open SpaceTime
open TensorProduct Lorentz
open minkowskiMatrix SchwartzMap
attribute [-simp] Fintype.sum_sum_type
attribute [-simp] Nat.succ_eq_add_one
```

```

/-!

### A.1. Auxiliary definition of field strength for distributions, with no

-/

/-- An auxiliary definition for the field strength of an electromagnetic po
    based on a distribution. On Schwartz maps this has the same value as the
    tensor, but no linearity or continuous properties built in. -
/
noncomputable def fieldStrengthAux {d} (A : DistElectromagneticPotential d)
  (ε :  $\square$ (SpaceTime d,  $\mathbb{R}$ )) : Lorentz.Vector d  $\otimes$  [ $\mathbb{R}$ ] Lorentz.Vector d :=
  Tensorial.toTensor.symm
    (permT id (PermCond.auto) {(η d | μ μ'  $\otimes$  A.deriv ε | μ' ν) + -
      (η d | ν ν'  $\otimes$  A.deriv ε | ν' μ)}T)

set_option backward.isDefEq.respectTransparency false in
lemma fieldStrengthAux_eq_add {d} (A : DistElectromagneticPotential d) (ε :
  fieldStrengthAux A ε =
  Tensorial.toTensor.symm (permT id (PermCond.auto) {(η d | μ μ'  $\otimes$  A.deriv
    - Tensorial.toTensor.symm (permT ![1, 0] (PermCond.auto)
      {(η d | μ μ'  $\otimes$  A.deriv ε | μ' ν)}T) := by
  rw [fieldStrengthAux]
  simp only [map_add, map_neg]
  rw [sub_eq_add_neg]
  apply congrArg₂
  · rfl
  · rw [permT_permT]
    rfl

lemma toTensor_fieldStrengthAux {d} (A : DistElectromagneticPotential d)
  (ε :  $\square$ (SpaceTime d,  $\mathbb{R}$ )) :
  Tensorial.toTensor (fieldStrengthAux A ε) =
  (permT id (PermCond.auto) {(η d | μ μ'  $\otimes$  A.deriv ε | μ' ν)}T)
  - (permT ![1, 0] (PermCond.auto) {(η d | μ μ'  $\otimes$  A.deriv ε | μ' ν)}T) :=
  rw [fieldStrengthAux_eq_add]
  simp

lemma toTensor_fieldStrengthAux_basis_repr {d} (A : DistElectromagneticPote
  (ε :  $\square$ (SpaceTime d,  $\mathbb{R}$ ))
  (b : ComponentIdx (S := realLorentzTensor d) (Fin.append ![Color.up] !
  (Tensor.basis _).repr (Tensorial.toTensor (fieldStrengthAux A ε)) b =
   $\sum \kappa$ , (η (b 0) κ * SpaceTime.distDeriv κ A ε (b 1) -
    η (b 1) κ * SpaceTime.distDeriv κ A ε (b 0)) := by
  rw [toTensor_fieldStrengthAux]

```

1.16.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/FIELDSTRENGTH.LEAN

```

simp only [Tensorial.self_toTensor_apply, map_sub,
  Finsupp.coe_sub, Pi.sub_apply]
rw [Tensor.permT_basis_repr_symm_apply, contrT_basis_repr_apply_eq_fin]
conv_lhs =>
  enter [1, 2, n]
  rw [Tensor.prodT_basis_repr_apply, contrMetric_repr_apply_eq_minkowskiMatrix]
  enter [1]
  change  $\eta$  (b 0) n
conv_lhs =>
  enter [1, 2, n, 2]
  rw [toTensor_deriv_basis_repr_apply]
  change distDeriv n A  $\varepsilon$  (b 1)
rw [Tensor.permT_basis_repr_symm_apply, contrT_basis_repr_apply_eq_fin]
conv_lhs =>
  enter [2, 2, n]
  rw [Tensor.prodT_basis_repr_apply, contrMetric_repr_apply_eq_minkowskiMatrix]
  enter [1]
  change  $\eta$  (b 1) n
conv_lhs =>
  enter [2, 2, n, 2]
  rw [toTensor_deriv_basis_repr_apply]
  change distDeriv n A  $\varepsilon$  (b 0)
rw [← Finset.sum_sub_distrib]

lemma fieldStrengthAux_tensor_basis_eq_basis {d} (A : DistElectromagneticPotential d
  ( $\varepsilon$  :  $\square(\text{SpaceTime } d, \mathbb{R})$ )
  (b : ComponentIdx (S := realLorentzTensor d) (Fin.append ![Color.up] ![Color.up])
  (Tensor.basis _).repr (Tensorial.toTensor (A.fieldStrengthAux  $\varepsilon$ )) b =
  (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.fieldStrengthA
    (b 0, b 1) := by
  rw [Tensorial.basis_toTensor_apply]
  rw [Tensorial.basis_map_prod]
  simp only [Nat.reduceSucc, Nat.reduceAdd, Basis.repr_reindex, Finsupp.mapDomain_eq
    Equiv.symm_symm, Fin.isValue]
  rw [Lorentz.Vector.tensor_basis_map_eq_basis_reindex]
  have hb : (((Lorentz.Vector.basis (d := d)).reindex Lorentz.Vector.indexEquiv.symm
    (Lorentz.Vector.basis.reindex Lorentz.Vector.indexEquiv.symm)) =
    ((Lorentz.Vector.basis (d := d)).tensorProduct (Lorentz.Vector.basis (d :=
    (Lorentz.Vector.indexEquiv.symm.prodCongr Lorentz.Vector.indexEquiv.symm)
  ext b
  match b with
  | (i, j) =>
  simp
  rw [hb]
  rw [Module.Basis.repr_reindex_apply]
  congr 1

```

```

lemma fieldStrengthAux_basis_repr_apply {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1
  (A : DistElectromagneticPotential d) (ε : □(SpaceTime d, ℝ)) :
  (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.field
    ∑ κ, ((η μν.1 κ * distDeriv κ A ε μν.2) - η μν.2 κ * distDeriv κ A ε μν
  match μν with
  | (μ, ν) =>
  trans (Tensor.basis _).repr (Tensorial.toTensor (A.fieldStrengthAux ε))
    (fun | 0 => μ | 1 => ν); swap
  · rw [toTensor_fieldStrengthAux_basis_repr]
  rw [fieldStrengthAux_tensor_basis_eq_basis]

lemma fieldStrengthAux_basis_repr_apply_eq_single {d} {μν : (Fin 1 ⊕ Fin d)
  (A : DistElectromagneticPotential d) (ε : □(SpaceTime d, ℝ)) :
  (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.field
    ((η μν.1 μν.1 * distDeriv μν.1 A ε μν.2) - η μν.2 μν.2 * distDeriv μν.2
  rw [fieldStrengthAux_basis_repr_apply]
  simp only [Finset.sum_sub_distrib]
  rw [Finset.sum_eq_single μν.1, Finset.sum_eq_single μν.2]
  · intro b _ hb
    rw [minkowskiMatrix.off_diag_zero]
    simp only [zero_mul]
    exact id (Ne.symm hb)
  · simp
  · intro b _ hb
    rw [minkowskiMatrix.off_diag_zero]
    simp only [zero_mul]
    exact id (Ne.symm hb)
  · simp

lemma fieldStrengthAux_eq_basis {d} (A : DistElectromagneticPotential d)
  (ε : □(SpaceTime d, ℝ)) :
  (A.fieldStrengthAux ε) = ∑ μ, ∑ ν,
    ((η μ μ * distDeriv μ A ε ν) - η ν ν * distDeriv ν A ε μ)
  · Lorentz.Vector.basis μ ⊗t[ℝ] Lorentz.Vector.basis ν := by
  apply (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr.inj
  ext b
  match b with
  | (μ, ν) =>
  simp [map_sum, map_smul, Finsupp.coe_finset_sum, Finsupp.coe_smul, Finset
    Pi.smul_apply, Basis.tensorProduct_repr_tmul_apply, Basis.repr_self, sm
  simp [Finsupp.single_apply]
  rw [fieldStrengthAux_basis_repr_apply_eq_single]

/-!

```


1.16.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/FIELDSTRENGTH.LEAN

A.2. The definition of the field strength

-/

set_option backward.isDefEq.respectTransparency false in

/-- The field strength of an electromagnetic potential which is a distribution. -
/

noncomputable def fieldStrength {d} :

DistElectromagneticPotential d $\rightarrow$ $\mathbb{R}$

(SpaceTime d) $\rightarrow$ d $\mathbb{R}$ Lorentz.Vector d $\otimes$ $\mathbb{R}$ Lorentz.Vector d where

toFun A := {

toFun ε := A.fieldStrengthAux ε

map_add' $\varepsilon_1 \varepsilon_2$:= by

apply (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr.injective

ext $\mu\nu$

simp [fieldStrengthAux_basis_repr_apply_eq_single]

ring

map_smul' c ε := by

apply (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr.injective

ext $\mu\nu$

simp [fieldStrengthAux_basis_repr_apply_eq_single]

ring

cont := by

simp [fieldStrengthAux_eq_basis]

fun_prop}

map_add' A1 A2 := by

ext ε

apply (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr.injective

ext $\mu\nu$

simp only [ContinuousLinearMap.coe_mk', LinearMap.coe_mk, AddHom.coe_mk,

fieldStrengthAux_basis_repr_apply_eq_single, map_add, ContinuousLinearMap.add,

Lorentz.Vector.apply_add, Finsupp.coe_add, Pi.add_apply]

ring

map_smul' c A := by

ext ε

apply (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr.injective

ext $\mu\nu$

simp only [ContinuousLinearMap.coe_mk', LinearMap.coe_mk, AddHom.coe_mk,

fieldStrengthAux_basis_repr_apply_eq_single, map_smul, ContinuousLinearMap.coe,

Pi.smul_apply, Lorentz.Vector.apply_smul, Real.ringHom_apply, Finsupp.coe_smul]

ring

lemma fieldStrength_eq_fieldStrengthAux {d} (A : DistElectromagneticPotential d)

(ε : $\square$ (SpaceTime d, $\mathbb{R}$)) :

A.fieldStrength ε = A.fieldStrengthAux ε := by rfl

/-!

A.3. Field strength written in terms of a basis

-/

```
lemma fieldStrength_eq_basis {d} (A : DistElectromagneticPotential d)
  (ε : ⌈(SpaceTime d, ℝ)) :
  A.fieldStrength ε = ∑ μ, ∑ ν,
    ((η μ μ * distDeriv μ A ε ν) - η ν ν * distDeriv ν A ε μ)
    • Lorentz.Vector.basis μ ⊗ℝ Lorentz.Vector.basis ν := by
  rw [fieldStrength]
  exact fieldStrengthAux_eq_basis A ε
```

```
lemma fieldStrength_basis_repr_eq_single {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d)}
  (A : DistElectromagneticPotential d) (ε : ⌈(SpaceTime d, ℝ)) :
  (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.fieldStrength ε)
    ((η μν.1 μν.1 * distDeriv μν.1 A ε μν.2) - η μν.2 μν.2 * distDeriv μν.2 A ε μν.1) = 0 := by
  rw [fieldStrength]
  exact fieldStrengthAux_basis_repr_apply_eq_single A ε
```

@[simp]

```
lemma fieldStrength_diag_zero {d} (A : DistElectromagneticPotential d)
  (ε : ⌈(SpaceTime d, ℝ)) (μ : Fin 1 ⊕ Fin d) :
  (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr
    (A.fieldStrength ε) (μ, μ) = 0 := by
  rw [fieldStrength_basis_repr_eq_single]
  simp
```

set_option backward.isDefEq.respectTransparency false in

@[simp]

```
lemma distDeriv_fieldStrength_diag_zero {d} (A : DistElectromagneticPotential d)
  (ε : ⌈(SpaceTime d, ℝ)) (μ ν : Fin 1 ⊕ Fin d) :
  (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr
    (distDeriv ν A.fieldStrength ε) (μ, μ) = 0 := by
  rw [SpaceTime.distDeriv_apply']
  simp
```

```
lemma fieldStrength_antisymmetric_basis {d} (A : DistElectromagneticPotential d)
  (ε : ⌈(SpaceTime d, ℝ)) (μ ν : Fin 1 ⊕ Fin d) :
  (Vector.basis.tensorProduct Vector.basis).repr
    (A.fieldStrength ε) (μ, ν) = - (Vector.basis.tensorProduct Vector.basis).repr
    (A.fieldStrength ε) (ν, μ) := by
  rw [fieldStrength_basis_repr_eq_single, fieldStrength_basis_repr_eq_single]
  ring
```

/-!

1.17.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/MAGNETICFIELD.lean

A.4. Equivariance of the field strength for distributions

-/

```
set_option backward.isDefEq.respectTransparency false in
lemma fieldStrength_equivariant {d} (A : DistElectromagneticPotential d)
  (Λ : LorentzGroup d) :
  (Λ • A).fieldStrength = Λ • A.fieldStrength := by
  ext ε
  rw [fieldStrength_eq_fieldStrengthAux, lorentzGroup_smul_dist_apply]
  rw [fieldStrengthAux_eq_add, deriv_equivariant, lorentzGroup_smul_dist_apply,
    ← actionT_contrMetric Λ]
  generalize ((schwartzAction Λ⁻¹) ε) = ε'
  rw [fieldStrength_eq_fieldStrengthAux, fieldStrengthAux_eq_add]
  simp only [Tensorial.toTensor_smul, prodT_equivariant, contrT_equivariant, permT_e
    ← Tensorial.smul_toTensor_symm, smul_sub]

end DistElectromagneticPotential

end Electromagnetism
```

1.17 Physlib/Electromagnetism/Distributional/MagneticField.lean

/-

Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Joseph Tooby-Smith

-/

module

public import Physlib.Electromagnetism.Distributional.ElectricField

/-!

The Magnetic Field

i. Overview

In general dimensions we define the magnetic field matrix from the spatial component field strength matrix. This is an antisymmetric matrix. We define it in this module for distributions.

ii. Key results

```

- `DistElectromagneticPotential.magneticFieldMatrix` : The magnetic field m
  electromagnetic potential in general spatial dimensions.

## iii. Table of contents

- A. Magnetic field matrix for distributions
  - A.1. Magnetic field matrix in terms of vector potentials
  - A.2. The magnetic field matrix in terms of the field strength
  - A.3. Magnetic field matrix in 1d

## iv. References

-/

@[expose] public section

namespace Electromagnetism
open Module realLorentzTensor
open IndexNotation
open TensorSpecies
open Tensor

/-!

## A. Magnetic field matrix for distributions

-/

namespace DistElectromagneticPotential
open TensorSpecies
open Tensor
open SpaceTime
open TensorProduct
open minkowskiMatrix SchwartzMap Lorentz
attribute [-simp] Fintype.sum_sum_type
attribute [-simp] Nat.succ_eq_add_one

set_option backward.isDefEq.respectTransparency false in
/-- The magnetic field matrix of an electromagnetic potential which is a di
/
noncomputable def magneticFieldMatrix {d} (c : SpeedOfLight) :
  DistElectromagneticPotential d →1 [ℝ]
  (Time × Space d) → d[ℝ] (EuclideanSpace ℝ (Fin d) ⊗ [ℝ] EuclideanSpace ℝ
toFun A :=
  (TensorProduct.map (Lorentz.Vector.spatialCLM d).toLinearMap
    (Lorentz.Vector.spatialCLM d).toLinearMap, by continuity) °L

```

1.17.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/MAGNETICFIELD.LEAN

```

    distTimeSlice c A.fieldStrength
map_add' A1 A2 := by
  ext ε
  simp
map_smul' r A := by
  ext ε
  simp

/-!

### A.1. Magnetic field matrix in terms of vector potentials

-/

set_option backward.isDefEq.respectTransparency false in
lemma magneticFieldMatrix_eq_vectorPotential {c : SpeedOfLight}
  (A : DistElectromagneticPotential d)
  (ε :  $\square$ (Time  $\times$  Space d,  $\mathbb{R}$ )) :
  A.magneticFieldMatrix c ε =  $\sum_i \sum_j$ ,
    (Space.distSpaceDeriv j (A.vectorPotential c) ε i -
      Space.distSpaceDeriv i (A.vectorPotential c) ε j) •
    EuclideanSpace.basisFun (Fin d)  $\mathbb{R}$  i  $\otimes$   $_{+[\mathbb{R}]}$  EuclideanSpace.basisFun (Fin d)  $\mathbb{R}$  j :=
simp only [magneticFieldMatrix, LinearMap.coe_mk, AddHom.coe_mk, ContinuousLinearMap.coe_mk,
  ContinuousLinearMap.coe_mk', Function.comp_apply, distTimeSlice_apply, fieldStrength_apply,
  Fintype.sum_sum_type, Finset.univ_unique, Fin.default_eq_zero, Fin.isValue,
  Finset.sum_singleton, inl_0_inl_0, one_mul, inr_i_inr_i, neg_mul, sub_neg_eq_add,
  zero_smul, zero_add, map_add, map_sum, map_smul, map_tmul, ContinuousLinearMap.coe_mk,
  Lorentz.Vector.spatialCLM_basis_sum_inl, Lorentz.Vector.spatialCLM_basis_sum_inr,
  EuclideanSpace.basisFun_apply, zero_tmul, smul_zero, Finset.sum_const_zero, tmul_zero,
  simp [← distTimeSlice_apply, distTimeSlice_distDeriv_inr, vectorPotential,
  Space.distSpaceDeriv_apply_CLM, Lorentz.Vector.spatialCLM, neg_add_eq_sub]

lemma magneticFieldMatrix_basis_repr_eq_vector_potential {c : SpeedOfLight}
  (A : DistElectromagneticPotential d)
  (ε :  $\square$ (Time  $\times$  Space d,  $\mathbb{R}$ )) (i j : Fin d) :
  ((PiLp.basisFun 2  $\mathbb{R}$  (Fin d)).tensorProduct (PiLp.basisFun 2  $\mathbb{R}$  (Fin d))).repr
    (A.magneticFieldMatrix c ε) (i, j) =
    Space.distSpaceDeriv j (A.vectorPotential c) ε i -
    Space.distSpaceDeriv i (A.vectorPotential c) ε j := by
  rw [magneticFieldMatrix_eq_vectorPotential]
  simp

lemma magneticFieldMatrix_distSpaceDeriv_basis_repr_eq_vector_potential {c : SpeedOfLight}
  (A : DistElectromagneticPotential d)
  (ε :  $\square$ (Time  $\times$  Space d,  $\mathbb{R}$ )) (i j k : Fin d) :
  ((PiLp.basisFun 2  $\mathbb{R}$  (Fin d)).tensorProduct (PiLp.basisFun 2  $\mathbb{R}$  (Fin d))).repr

```

```

      (Space.distSpaceDeriv k (A.magneticFieldMatrix c) ε) (i, j) =
      Space.distSpaceDeriv k (Space.distSpaceDeriv j (A.vectorPotential c)) ε
      Space.distSpaceDeriv k (Space.distSpaceDeriv i (A.vectorPotential c)) ε
    simp [Space.distSpaceDeriv_apply', magneticFieldMatrix_basis_repr_eq_vect
ring

/-!

### A.2. The magnetic field matrix in terms of the field strength

-/

set_option backward.isDefEq.respectTransparency false in
lemma magneticFieldMatrix_basis_repr_eq_fieldStrength {c : SpeedOfLight}
  (A : DistElectromagneticPotential d)
  (ε :  $\square$ (Time  $\times$  Space d,  $\mathbb{R}$ )) (i j : Fin d) :
  ((PiLp.basisFun 2  $\mathbb{R}$  (Fin d)).tensorProduct (PiLp.basisFun 2  $\mathbb{R}$  (Fin d)))
    (A.magneticFieldMatrix c ε) (i, j) =
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr
      (distTimeSlice c A.fieldStrength ε) (Sum.inr i, Sum.inr j) := by
  simp only [magneticFieldMatrix_eq_vectorPotential, EuclideanSpace.basisFu
    map_smul, Finsupp.coe_finset_sum, Finsupp.coe_smul, Finset.sum_apply, P
    Basis.tensorProduct_repr_tmul_apply, PiLp.basisFun_repr, PiLp.single_ap
    smul_eq_mul, mul_ite, mul_one, mul_zero, Finset.sum_ite_irrel, Finset.s
    Finset.mem_univ,  $\downarrow$ reduceIte, Finset.sum_const_zero, distTimeSlice_apply
    fieldStrength_basis_repr_eq_single, inr_i_inr_i, neg_mul, one_mul, sub_
  simp only [vectorPotential, Vector.spatialCLM, LinearMap.coe_mk, AddHom.c
    Space.distSpaceDeriv_apply_CLM, ContinuousLinearMap.coe_comp', Continuo
    Function.comp_apply,  $\leftarrow$  distTimeSlice_apply, distTimeSlice_distDeriv_inr
ring

/-!

### A.3. Magnetic field matrix in 1d

-/

@[simp]
lemma magneticFieldMatrix_one_dim_eq_zero {c : SpeedOfLight}
  (A : DistElectromagneticPotential 1) :
  A.magneticFieldMatrix c = 0 := by
  ext ε
  rw [magneticFieldMatrix_eq_vectorPotential]
  simp

end DistElectromagneticPotential

```

1.18.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/SCALARPOTENTIAL.lean

end Electromagnetism

1.18 Physlib/Electromagnetism/Distributional/ScalarPotential.lean

/-

Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Joseph Tooby-Smith

-/

module

public import Physlib.Electromagnetism.Distributional.Basic

public import Physlib.SpaceAndTime.SpaceTime.TimeSlice

public import Mathlib.Data.Real.Hom

/-!

The Scalar Potential

i. Overview

The electromagnetic potential is given by

$\vec{A} = (1/c \phi, \vec{A})$

where ϕ is the scalar potential and $\vec{A}$ is the vector potential.

In this module we define the scalar potential, and prove lemmas about it.

Since $\vec{A}$ is relativistic it is a distribution of $\text{SpaceTime } d$, whilst the scalar potential is non-relativistic and is therefore a distribution of $\text{Time } a$

ii. Key results

- `DistElectromagneticPotential.scalarPotential` : The scalar potential from an electromagnetic potential which is a distribution.

iii. Table of contents

- A. Scalar potential for distributions

iv. References

-/

@[expose] public section

namespace Electromagnetism

```

open Module realLorentzTensor
open IndexNotation
open TensorSpecies
open Tensor

/-!

## A. Scalar potential for distributions

-/

namespace DistElectromagneticPotential
open TensorSpecies
open Tensor
open SpaceTime
open TensorProduct
open minkowskiMatrix
attribute [-simp] Fintype.sum_sum_type
attribute [-simp] Nat.succ_eq_add_one

set_option backward.isDefEq.respectTransparency false in
/-- The scalar potential of an electromagnetic potential which is a distrib
/
noncomputable def scalarPotential {d} (c : SpeedOfLight) :
  DistElectromagneticPotential d →1 [ℝ]
  (Time × Space d) → d[ℝ] ℝ where
toFun A := Lorentz.Vector.temporalCLM d ∘ L distTimeSlice c (c.val • A)
map_add' A1 A2 := by
  ext ε
  simp [distTimeSlice]
map_smul' r A := by
  ext ε
  simp only [distTimeSlice, map_smul, ContinuousLinearEquiv.coe_mk, Linear
    LinearMap.coe_mk, AddHom.coe_mk, ContinuousLinearMap.coe_comp', Conti
    Function.comp_apply, Pi.smul_apply, smul_eq_mul, Real.ringHom_apply]
  ring

end DistElectromagneticPotential
end Electromagnetism

```

1.19 Physlib/Electromagnetism/Distributional/VectorPotential.le

```

/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.

```


1.19.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/VECTORPOTENTIAL61LEAN

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Joseph Tooby-Smith

-/

module

public import Physlib.Electromagnetism.Distributional.Basic

public import Physlib.SpaceAndTime.SpaceTime.TimeSlice

public import Mathlib.Data.Real.Hom

/-!

The vector Potential

i. Overview

The electromagnetic potential is given by

$A = (1/c \phi, \vec{A})$

where ϕ is the scalar potential and $\vec{A}$ is the vector potential.

In this module we define the vector potential, and prove lemmas about it.

Since A is relativistic it is a distribution of $\text{SpaceTime } d$, whilst the vector potential is non-relativistic and is therefore a distribution of $\text{Time } a$

ii. Key results

- `DistElectromagneticPotential.vectorPotential` : The vector potential from an electromagnetic potential which is a distribution.

iii. Table of contents

- A. Vector potential for distributions

iv. References

-/

@[expose] public section

namespace Electromagnetism

open Module realLorentzTensor

open IndexNotation

open TensorSpecies

open Tensor

/-!

A. Vector potential for distributions

-/

```
namespace DistElectromagneticPotential
open TensorSpecies
open Tensor
open SpaceTime
open TensorProduct
open minkowskiMatrix SchwartzMap
attribute [-simp] Fintype.sum_sum_type
attribute [-simp] Nat.succ_eq_add_one
```

```
/-- The vector potential of an electromagnetic potential which is a distrib
/
```

```
noncomputable def vectorPotential {d} (c : SpeedOfLight) :
  DistElectromagneticPotential d →1 [ℝ]
  (Time × Space d) → d[ℝ] EuclideanSpace ℝ (Fin d) where
toFun A := Lorentz.Vector.spatialCLM d ∘L distTimeSlice c A
map_add' A1 A2 := by
  ext ε
  simp [distTimeSlice]
map_smul' r A := by
  ext ε i
  simp only [distTimeSlice, map_smul, ContinuousLinearEquiv.coe_mk, Linear
    LinearMap.coe_mk, AddHom.coe_mk, ContinuousLinearMap.coe_smul', Conti
    Pi.smul_apply, Function.comp_apply,
    Real.ringHom_apply, PiLp.smul_apply, smul_eq_mul]
```

```
end DistElectromagneticPotential
```

```
end Electromagnetism
```

1.20 Physlib/Electromagnetism/Dynamics/IsExtrema.lean

```
rw [tensorDeriv_equivariant _ _ _ (differentiable_toFieldStrength_of_sm
```

1.21 Physlib/Electromagnetism/Dynamics/KineticTerm.lean

```
fun_prop
fun_prop
_ = ∑ (v : (Fin 1 ⊕ Fin d)), ∑ (μ : (Fin 1 ⊕ Fin d)),
  (1/□.μ0 * (η μ μ * η v v * ∂_μ (fun x' => ∂_μ A x' v) x -
```

```

    ∂_μ (fun x' => ∂_ν A x' μ) x)) • Lorentz.Vector.basis ν := by
      rw [gradKineticTerm_eq_sum_sum A x ha]
    _ = ∑ (ν : (Fin 1 ⊕ Fin d)), ∑ (μ : (Fin 1 ⊕ Fin d)),
      ((1/□.μ₀ * η ν ν) * (η μ μ * ∂_μ (fun x' => ∂_μ A x' ν) x -
        η ν ν * ∂_μ (fun x' => ∂_ν A x' μ) x)) • Lorentz.Vector.basis ν := by
        apply Finset.sum_congr rfl (fun ν _ => ?_)
        apply Finset.sum_congr rfl (fun μ _ => ?_)
        congr 1
        ring_nf
        simp
    _ = ∑ (ν : (Fin 1 ⊕ Fin d)), ∑ (μ : (Fin 1 ⊕ Fin d)),
      ((1/□.μ₀ * η ν ν) * (∂_μ (fun x' => η μ μ * ∂_μ A x' ν) x -
        ∂_μ (fun x' => η ν ν * ∂_ν A x' μ) x)) • Lorentz.Vector.basis ν := by
        apply Finset.sum_congr rfl (fun ν _ => ?_)
        apply Finset.sum_congr rfl (fun μ _ => ?_)
        congr
        · rw [SpaceTime.deriv_eq, SpaceTime.deriv_eq, fderiv_const_mul]
          rfl
          fun_prop
        · rw [SpaceTime.deriv_eq, SpaceTime.deriv_eq, fderiv_const_mul]
          rfl
          fun_prop
    _ = ∑ (ν : (Fin 1 ⊕ Fin d)), ∑ (μ : (Fin 1 ⊕ Fin d)),
      ((1/□.μ₀ * η ν ν) * (∂_μ (fun x' => η μ μ * ∂_μ A x' ν -
        η ν ν * ∂_ν A x' μ) x)) • Lorentz.Vector.basis ν := by
        apply Finset.sum_congr rfl (fun ν _ => ?_)
        apply Finset.sum_congr rfl (fun μ _ => ?_)
        congr
        rw [SpaceTime.deriv_eq, SpaceTime.deriv_eq, SpaceTime.deriv_eq, fderiv_fun_s]
        simp only [ContinuousLinearMap.coe_sub', Pi.sub_apply]
        · fun_prop
        · fun_prop
    _ = ∑ (ν : (Fin 1 ⊕ Fin d)), ∑ (μ : (Fin 1 ⊕ Fin d)),
      ((1/□.μ₀ * η ν ν) * (∂_μ (A.fieldStrengthMatrix · (μ, ν)) x)) •
        Lorentz.Vector.basis ν := by
        apply Finset.sum_congr rfl (fun ν _ => ?_)
        apply Finset.sum_congr rfl (fun μ _ => ?_)
        congr
        funext x
        rw [toFieldStrength_basis_repr_apply_eq_single]
    _ = ∑ (ν : (Fin 1 ⊕ Fin d)), ((1/□.μ₀ * η ν ν) *
      ∑ (μ : (Fin 1 ⊕ Fin d)), (∂_μ (A.fieldStrengthMatrix · (μ, ν)) x))
      • Lorentz.Vector.basis ν := by
        apply Finset.sum_congr rfl (fun ν _ => ?_)
        rw [← Finset.sum_smul, Finset.mul_sum]
    _ = ∑ (ν : (Fin 1 ⊕ Fin d)), (1/□.μ₀ * η ν ν) •

```

```

    (∑ (μ : (Fin 1 ⊕ Fin d)), (∂_ μ (A.fieldStrengthMatrix · (μ, v)) x)
    • Lorentz.Vector.basis v := by
    apply Finset.sum_congr rfl (fun v _ => ?_)
    rw [smul_smul]
conv => enter [2, x]; rw [toFieldStrength_basis_repr_apply_eq_single]
fun_prop
rw [tensorDeriv_toTensor_basis_repr (by fun_prop)]

```

1.22 Physlib/Electromagnetism/Kinematics/EMPotential.lean

```

public import Physlib.SpaceAndTime.Space.Derivatives.Curl
public import Physlib.SpaceAndTime.SpaceTime.TimeSlice
public import Physlib.Mathematics.Calculus.ParametricIntegration
- A.2. Basic constructors of the electromagnetic potential
- A.3. The group action on the ElectromagneticPotential
- A.4. Differentiability
- A.5. The action on the space-time derivatives
- A.6. Variational adjoint derivative of component
- A.7. Variational adjoint derivative of derivatives of the potential
## A.2. Basic constructors of the electromagnetic potential

-/

/-- The electromagnetic potential from a scalar potential, where
the vector potential is set to zero. -/
noncomputable def ofScalarPotential {d} (c : SpeedOfLight)
  (φ : Time → Space d → ℝ) : ElectromagneticPotential d where
  val x μ :=
  match μ with
  | Sum.inl 0 => ((timeSlice c).symm φ x) / c
  | Sum.inr _ => 0

/-- The creation of an electromagnetic potential from a static scalar poten
/
noncomputable def ofStaticScalarPotential {d} (c : SpeedOfLight)
  (φ : Space d → ℝ) : ElectromagneticPotential d :=
  ofScalarPotential c (fun _ => φ)

/-- The electromagnetic potential from a vector potential, where
the scalar potential is set equal to zero. -/
noncomputable def ofVectorPotential {d} (c : SpeedOfLight)
  (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
  ElectromagneticPotential d where
  val x μ :=

```

1.22. PHYSLIB/ELECTROMAGNETISM/KINEMATICS/EMPOTENTIAL.lean

```

match  $\mu$  with
| Sum.inl 0 => 0
| Sum.inr i => (timeSlice c).symm A x i

/-- The creation of an electromagnetic potential from a static vector potential. -
/
noncomputable def ofStaticVectorPotential {d} (c : SpeedOfLight)
  (A : Space d → EuclideanSpace  $\mathbb{R}$  (Fin d)) : ElectromagneticPotential d :=
  ofVectorPotential c (fun _ => A)

/-- The creation of an electromagnetic potential from the non-
relativistic potentials. -/
noncomputable def ofPotentials {d} (c : SpeedOfLight) ( $\phi$  : Time → Space d →  $\mathbb{R}$ )
  (A : Time → Space d → EuclideanSpace  $\mathbb{R}$  (Fin d)) :
  ElectromagneticPotential d where
  val x  $\mu$  :=
    match  $\mu$  with
    | Sum.inl 0 => ((timeSlice c).symm  $\phi$  x) / c
    | Sum.inr i => (timeSlice c).symm A x i

lemma ofPotentials_eq_add {d} (c : SpeedOfLight) ( $\phi$  : Time → Space d →  $\mathbb{R}$ )
  (A : Time → Space d → EuclideanSpace  $\mathbb{R}$  (Fin d)) :
  ofPotentials c  $\phi$  A = ofScalarPotential c  $\phi$  + ofVectorPotential c A := by
  ext x
  refine Lorentz.Vector.ext_of_apply (fun i => ?_)
  match i with
  | Sum.inl 0 =>
    simp only [ofPotentials, Fin.isValue, add_val, Pi.add_apply, Lorentz.Vector.appl
    simp only [ofScalarPotential, Fin.isValue, ofVectorPotential, add_zero]
  | Sum.inr i =>
    simp only [ofPotentials, add_val, Pi.add_apply, Lorentz.Vector.apply_add]
    simp [ofScalarPotential, ofVectorPotential]

/-- The creation of of an electromagnetic potential from static potentials. -
/
noncomputable def ofStaticPotentials {d} (c : SpeedOfLight) ( $\phi$  : Space d →  $\mathbb{R}$ )
  (A : Space d → EuclideanSpace  $\mathbb{R}$  (Fin d)) : ElectromagneticPotential d :=
  ofStaticScalarPotential c  $\phi$  + ofStaticVectorPotential c A

lemma ofStaticPotentials_eq_ofPotentials {d} (c : SpeedOfLight) ( $\phi$  : Space d →  $\mathbb{R}$ )
  (A : Space d → EuclideanSpace  $\mathbb{R}$  (Fin d)) :
  ofStaticPotentials c  $\phi$  A = ofPotentials c (fun _ =>  $\phi$ ) (fun _ => A) := by
  rw [ofPotentials_eq_add]
  rfl

open MeasureTheory Matrix Space InnerProductSpace Time in

```

```

/-- The electromagnetic potential from an electric and a magnetic field.
    This defines the electromagnetic potential in the Poincare gauge. -
/
noncomputable def ofElectromagneticField (c : SpeedOfLight)
  (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
  (B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) :
  ElectromagneticPotential 3 :=
  let A := fun t (x : Space) => - ∫ u in 0..(1 : ℝ), (u • basis.repr x) × e₃
  let φ := fun t (x : Space) => - ∫ u in (0 : ℝ)..1, ∫ E t (u • x), basis.re
  ofPotentials c φ A

TODO "Write lemmas for the various properties (e.g. the electric field) of
      the electromagnetic potential from the various constructors."

TODO "Define constructors for the distributional electromagnetic potential,
      to e.g. `ofScalarPotential` and `ofVectorPotential` for `ElectromagneticP

/-!

## A.3. The group action on the ElectromagneticPotential
TODO "Lift the action on `ElectromagneticPotential` to a `DistribMulActio

### A.4. Differentiability
open ContDiff

@[fun_prop]
lemma contDiff_action {d} (Λ : LorentzGroup d) (A : ElectromagneticPotential d)
  (hA : ContDiff ℝ n A) : ContDiff ℝ n (fun x => Λ • A (Λ⁻¹ • x)) := by
  apply ContDiff.comp
  · exact ContinuousLinearMap.contDiff (Lorentz.Vector.actionCLM Λ)
  · apply ContDiff.comp
    · exact hA
    · exact ContinuousLinearMap.contDiff (Lorentz.Vector.actionCLM Λ⁻¹)

@[fun_prop]
lemma differentiable_deriv {d} {A : ElectromagneticPotential d}
  (hA : ContDiff ℝ 2 A) (μ v : Fin 1 ⊕ Fin d) :
  Differentiable ℝ (fun x => ∂_μ A x v) := by
  have diff_partial (μ) :
    ∀ v, Differentiable ℝ fun x => (fderiv ℝ A x) (Lorentz.Vector.basis μ
    rw [SpaceTime.differentiable_vector]
  fun_prop
  exact diff_partial μ v

@[fun_prop]
lemma differentiable_deriv_of_smooth {d} {A : ElectromagneticPotential d}

```

1.22. PHYSLIB/ELECTROMAGNETISM/KINEMATICS/EMPOTENTIAL.LEA7B

```

    (hA : ContDiff  $\mathbb{R} \times A$ ) ( $\mu \nu$  : Fin 1  $\oplus$  Fin d) :
      Differentiable  $\mathbb{R}$  (fun x =>  $\partial_{\mu} A \times \nu$ ) := by
        apply differentiable_deriv (hA.of_le (ENat.LEInfty.out))  $\mu \nu$ 

@[fun_prop]
lemma contDiff_deriv {n} {d} {A : ElectromagneticPotential d}
  (hA : ContDiff  $\mathbb{R}$  (n + 1) A) ( $\mu \nu$  : Fin 1  $\oplus$  Fin d) :
    ContDiff  $\mathbb{R}$  n (fun x =>  $\partial_{\mu} A \times \nu$ ) := by
      have diff_partial ( $\mu$ ) :
         $\forall \nu$ , ContDiff  $\mathbb{R}$  n fun x => (fderiv  $\mathbb{R}$  A x) (Lorentz.Vector.basis  $\mu$ )  $\nu$  := by
          rw [SpaceTime.contDiff_vector]
      fun_prop
      exact diff_partial  $\mu \nu$ 

TODO "Add results related to the differentiability of the
      derivative of the Electromagnetic potential."

### A.5. Differentiability in terms of constructors

-/

lemma differentiable_ofScalarPotential {d} (c : SpeedOfLight) ( $\phi$  : Time  $\rightarrow$  Space d  $\rightarrow$ 
  ( $h\phi$  : Differentiable  $\mathbb{R}$  1 $\phi$ ) : Differentiable  $\mathbb{R}$  (ofScalarPotential c  $\phi$ ) := by
  simp [ofScalarPotential]
  rw [← SpaceTime.differentiable_vector]
  intro  $\mu$ 
  match  $\mu$  with
  | Sum.inl 0 => fun_prop
  | Sum.inr _ => fun_prop

lemma contDiff_ofScalarPotential {n} {d} (c : SpeedOfLight) ( $\phi$  : Time  $\rightarrow$  Space d  $\rightarrow$   $\mathbb{R}$ )
  ( $h\phi$  : ContDiff  $\mathbb{R}$  n 1 $\phi$ ) : ContDiff  $\mathbb{R}$  n (ofScalarPotential c  $\phi$ ) := by
  simp [ofScalarPotential]
  rw [← SpaceTime.contDiff_vector]
  intro  $\mu$ 
  match  $\mu$  with
  | Sum.inl 0 => fun_prop
  | Sum.inr _ => fun_prop

lemma differentiable_ofVectorPotential {d} (c : SpeedOfLight)
  (A : Time  $\rightarrow$  Space d  $\rightarrow$  EuclideanSpace  $\mathbb{R}$  (Fin d))
  (hA : Differentiable  $\mathbb{R}$  1A) : Differentiable  $\mathbb{R}$  (ofVectorPotential c A) := by
  simp [ofVectorPotential]
  rw [← SpaceTime.differentiable_vector]
  intro  $\mu$ 
  match  $\mu$  with

```

```

| Sum.inl 0 => fun_prop
| Sum.inr i => fun_prop

lemma contDiff_ofVectorPotential {n} {d} (c : SpeedOfLight)
  (A : Time → Space d → EuclideanSpace ℝ (Fin d))
  (hA : ContDiff ℝ n 1A) : ContDiff ℝ n (ofVectorPotential c A) := by
  simp [ofVectorPotential]
  rw [← SpaceTime.contDiff_vector]
  intro μ
  match μ with
  | Sum.inl 0 => fun_prop
  | Sum.inr i => fun_prop

lemma differentiable_ofPotentials {d} (c : SpeedOfLight) (φ : Time → Space d
  (A : Time → Space d → EuclideanSpace ℝ (Fin d)) (hφ : Differentiable ℝ
  (hA : Differentiable ℝ 1A) : Differentiable ℝ (ofPotentials c φ A) := by
  simp [ofPotentials]
  rw [← SpaceTime.differentiable_vector]
  intro μ
  match μ with
  | Sum.inl 0 => fun_prop
  | Sum.inr i => fun_prop

lemma contDiff_ofPotentials {n} {d} (c : SpeedOfLight) (φ : Time → Space d
  (A : Time → Space d → EuclideanSpace ℝ (Fin d)) (hφ : ContDiff ℝ n 1φ)
  (hA : ContDiff ℝ n 1A) : ContDiff ℝ n (ofPotentials c φ A) := by
  simp [ofPotentials]
  rw [← SpaceTime.contDiff_vector]
  intro μ
  match μ with
  | Sum.inl 0 => fun_prop
  | Sum.inr i => fun_prop

open MeasureTheory Matrix Space InnerProductSpace Time in
lemma contDiff_ofElectromagneticField {n : ℕ} (c : SpeedOfLight)
  (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
  (B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) (hE : ContDiff ℝ n 1E)
  (hB : ContDiff ℝ n 1B) : ContDiff ℝ n (ofElectromagneticField c E B) :=
  let A : Time → Space → EuclideanSpace ℝ (Fin 3) := fun t x =>
    - ∫ u in 0..(1 : ℝ), (u • basis.repr x) ×e3 B t (u • x) ∂(volume)
  have h1 : ContDiff ℝ n 1A := by
    simp only [WithLp.equiv_apply, A]
    apply ContDiff.neg
    apply contDiff_parametric_intervalIntegral_of_contDiff
    refine contDiff_euclidean.mpr ?_
    intro i

```


1.23.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/ELECTRICFIELD.LEAN 75

```

    let C : (Time × Space) × ℝ → EuclideanSpace ℝ (Fin 3) := fun p =>
      let (t, x) := p.1
      let u := p.2
      (u • basis.repr x) ×e3 B t (u • x)
    change ContDiff ℝ n (fun x => C x i)
    fin_cases i
    all_goals
      · simp [C, crossProduct]
        fun_prop
    have hn : ContDiff ℝ n 1A := h1.of_le (by simp)
    rw [← SpaceTime.contDiff_vector]
    intro μ
    match μ with
    | Sum.inr i =>
      change ContDiff ℝ n (fun x => (timeSlice c).symm A x i)
      fun_prop
    | Sum.inl 0 =>
      simp only [ofElectromagneticField, ofPotentials, map_smul, WithLp.equiv_apply,
        WithLp.ofLp_smul, LinearMap.smul_apply, WithLp.equiv_symm_apply, WithLp.toLp_s
        Fin.isValue]
      apply ContDiff.div _ (by fun_prop) (by simp)
      apply timeSlice_symm_contDiff
      apply ContDiff.neg
      apply contDiff_parametric_intervalIntegral_of_contDiff
      fun_prop

/-!

### A.5. The action on the space-time derivatives
### A.6. Variational adjoint derivative of component
### A.7. Variational adjoint derivative of derivatives of the potential

```

1.23 Physlib/Electromagnetism/Kinematics/ElectricField.lean

B. Relation to constructors

-/

```

open MeasureTheory Matrix Space InnerProductSpace Time in
/-- The electric field of the electromagnetic potential created from the electric fi
  `E` and the magnetic field `B` is `E`, as long as Gauss's law for magnetism and
  Faraday's law are satisfied. -/
lemma ofElectromagneticField_electricField {c : SpeedOfLight}
  (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) (B : Time → Space 3 → EuclideanS

```

```

(E_contDiff : ContDiff  $\mathbb{R}^1$  1E) (B_contDiff : ContDiff  $\mathbb{R}^2$  1B)
(B_grad :  $\forall t, \nabla \cdot (B\ t) = 0$ ) (faraday :  $\forall t\ x, \text{curl}\ (E\ t)\ x = -\ \partial_t\ (B\ t)\ x$ )
(ofElectromagneticField c E B).electricField c = E := by
have h0 := B_contDiff.of_le (m := 1) (by simp)
ext1 t
ext1 x
suffices h : E t x +  $\partial_t\ (\text{fun } t \Rightarrow (\text{ofElectromagneticField } c\ E\ B).\text{vectorPotential } c\ t\ x)$  =
-  $\nabla\ ((\text{ofElectromagneticField } c\ E\ B).\text{scalarPotential } c\ t)\ x$  by
  simp only [electricField]
  grind
convert congrFun (eq_grad_integral_of_curl_zero (fun x => E t x +
   $\partial_t\ (\text{fun } t \Rightarrow (\text{ofElectromagneticField } c\ E\ B).\text{vectorPotential } c\ t\ x)$  t)
· simp [ofElectromagneticField_scalarPotential_eq_add_vectorPotential _ _]
  rw [fun_grad_neg]
  simp
· simp only [Time.deriv]
  fun_prop
· rw [fun_curl_add]
  ext1 x
  simp [faraday]
  suffices h :  $\partial_t\ (B \cdot x)\ t = \text{curl}\ (\text{fun } x \Rightarrow \partial_t\ ((\text{ofElectromagneticField } c\ E\ B).\text{vectorPotential } c \cdot x)\ t)\ x$  by
    simp [h]
  rw [← Space.time_deriv_curl_commute]
  · congr
    funext t
    have h1 := eq_neg_curl_of_div_zero (B t) (by fun_prop) (B_grad t)
    conv_lhs => rw [h1]
    simp only [ofElectromagneticField_vectorPotential]
    rw [fun_curl_neg]
    simp only [WithLp.equiv_apply, WithLp.ofLp_smul, map_smul, LinearMap.
      WithLp.equiv_symm_apply, WithLp.toLp_smul, Pi.neg_apply]
    intro x
    apply Differentiable.differentiableAt
    apply ContDiff.differentiable (n := 1) _ (by simp)
    apply contDiff_parametric_intervalIntegral_of_contDiff
    refine contDiff_euclidean.mpr ?_
    intro i
    let C : (Space)  $\times \mathbb{R} \rightarrow$  EuclideanSpace  $\mathbb{R}$  (Fin 3) := fun p =>
      let x := p.1
      let u := p.2
      (u • basis.repr x)  $\times_{e3}$  B t (u • x)
    suffices h : ContDiff  $\mathbb{R}^1$  (fun x => C x i) by
      convert h
      exact 1
    fin_cases i

```

1.24.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/FIELDSTRENGTH.LEAN 77

```
    all_goals
      · simp [C, crossProduct]
        fun_prop
      · fun_prop
      · fun_prop
      · simp only [Time.deriv]
        fun_prop

/-!
```

1.24 Physlib/Electromagnetism/Kinematics/FieldStrength.lean

- A.1. Tensor equalities
- A.2. Vector equalities
- A.3. The group action acting on the field strength tensor
- A.4. Differentiability and smoothness of the field strength tensor
- A.5. Elements of the field strength tensor in terms of basis
- A.6. The field strength matrix
 - A.6.1. Differentiability of the field strength matrix
- A.7. The antisymmetry of the field strength tensor
- A.8. Equivariance of the field strength matrix
- A.9. Linearity of the field strength tensor

```
open minkowskiMatrix Tensorial
open Lorentz
```

TODO "Currently the API for the field strength tensor has the definition of ``fieldStrengthMatrix``. This is now unneeded, and should be replaced with ``toField {A.toFieldStrength x| [μ] [ν]}T`` and suitable API around that. To undertake this TODO, it is likely easier to start building the API around ``toField {A.toFieldStrength x| [μ] [ν]}T`` and then remove ``fieldStrengthMat` once the API is in place."

We define the field strength tensor ``F{μν}`` in terms of the derivative of the

```
/-- The field strength from an electromagnetic potential, as a tensor `F{μν}`. -
/
### A.1. Tensor equalities
```

These equalities for the field strength tensor are in terms of tensor expressions and index notation. In practice, we don't expect them to be used explicitly. They are useful for proving some of the API within this module.

```
lemma toFieldStrength_eq_sub_tensorDeriv {d} {A : ElectromagneticPotential d}
  (hA : Differentiable ℝ A) (x : SpaceTime d) :
```

```

toFieldStrength A x =
  Tensorial.toTensor.symm (permT id PermCond.auto {η d | μ μ' ⊗ tensorDeriv A x | μ' v}^T) := by
  - Tensorial.toTensor.symm (permT ![1, 0] PermCond.auto
    {η d | μ μ' ⊗ tensorDeriv A x | μ' v}^T) := by
  simp only [toFieldStrength_eq_tensorDeriv hA, map_add, map_neg, sub_eq_add, rfl]

```

A.2. Vector equalities

These equalities for the field strength tensor are in terms of vector basis. They match some of the familiar forms one might expect to see the field strength tensor in.

-/

TODO "Generalize the proof of `toFieldStrength\_eq\_sum\_basis\_eval` so that it can easily be written as the sum of its components times the basis. The likely location for this is in the `Tensorial` module. The TODO item with tag: 8285454220008908699 is likely a prerequisite to this."

```

/-- The statement that `F = F^{μν} e_μ ⊗ e_ν` written explicitly, with
the components extracted via `toField`. -/
lemma toFieldStrength_eq_sum_basis_eval {d} {A : ElectromagneticPotential d}
  A.toFieldStrength = fun x => ∑ μ, ∑ ν, toField {A.toFieldStrength x} [μ ν]
  Vector.basis μ ⊗_t [R] Vector.basis ν := by
  ext x
  /- This is a fairly general proof, so we can generalize our tensor. -
  /
  generalize (A.toFieldStrength x) = t
  apply (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr.injEq
  ext {μ, ν}
  simp only [map_sum, map_smul, Finsupp.coe_finset_sum, Finsupp.coe_smul, Finsupp.coe_pi,
    Pi.smul_apply, Basis.tensorProduct_repr_tmul_apply, Basis.repr_self, Finsupp.coe_smul_eq_mul,
    smul_eq_mul, mul_ite, mul_one, mul_zero, Finset.sum_ite_irrel, Finset.sum_const_zero]
  obtain {t, rfl} := toTensor.symm.surjective t
  induction' t using Tensor.induction_on_basis with b a t h t1 t2 h1 h2
  · simp only [LinearEquiv.apply_symm_apply, basis_apply, evalT_pure, Pure.toField_pure,
    smul_eq_mul, mul_one, Pure.evalPCoeff]
  change _ = _ * ((realLorentzTensor d).basis (Color.up)).repr
    ((realLorentzTensor d).basis (Color.up) (b 1)) v
  /- Transforming the basis -/
  let e := ComponentIdx.prod.trans ((Vector.indexEquiv (d := d)).prodCongr)
  simp only [prod_basis_of_map_reindex Vector.basis_eq_map_tensor_basis,
    Vector.basis_eq_map_tensor_basis, Basis.repr_reindex, Basis.map_repr,
    LinearEquiv.symm_symm, LinearEquiv.trans_apply, LinearEquiv.apply_symm]

```

1.24.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/FIELDSTRENGTH.LEAN 79

```

    Finsupp.mapDomain_equiv_apply, basis_repr_pure, Pure.component_basisVector, Fi
    Pure.basisVector, Basis.repr_self, Finsupp.single_apply, mul_ite, mul_one, mul
    grind [show e b = (b 0, b 1) from rfl]
    · simp only [map_zero, Finsupp.coe_zero, Pi.zero_apply]
    · simp only [map_smul, h, smul_eq_mul, Finsupp.coe_smul, Pi.smul_apply]
    · simp only [map_add, h1, h2, Finsupp.coe_add, Pi.add_apply]

/-- The statement that  $F = F^{\{\mu\nu\}} e_\mu \otimes e_\nu$  written explicitly, with
the components given by  $\sum_\kappa (\eta_{\mu\kappa} * \partial_\kappa A \times v - \eta_{\nu\kappa} * \partial_\kappa A \times \mu)$ . -
/
lemma toFieldStrength_eq_sum_basis {d} {A : ElectromagneticPotential d}
  (hA : Differentiable ℝ A) (x : SpaceTime d) :
  A.toFieldStrength x =  $\sum \mu, \sum v, (\sum \kappa, (\eta_{\mu\kappa} * \partial_\kappa A \times v - \eta_{\nu\kappa} * \partial_\kappa A \times \mu)) \cdot$ 
    Lorentz.Vector.basis  $\mu \otimes_t$  Lorentz.Vector.basis  $v :=$  by
  apply (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr.injective
  ext (μ, ν)
  simp only [map_sum, map_smul, Finsupp.coe_finset_sum, Finsupp.coe_smul,
    Finset.sum_apply, Pi.smul_apply, Basis.tensorProduct_repr_tmul_apply, Basis.repr
    Finsupp.single_apply, smul_eq_mul, mul_ite, mul_one, mul_zero, Finset.sum_ite_in
    Finset.sum_ite_eq', Finset.mem_univ, ↑reduceIte, Finset.sum_const_zero]
  simp only [prod_basis_of_map_reindex Vector.basis_eq_map_tensor_basis
    Vector.basis_eq_map_tensor_basis,
    toFieldStrength_eq_sub_tensorDeriv hA, self_toTensor_apply, ← deriv_eq_tensorDer
    map_sub, Basis.repr_reindex, Basis.map_repr, LinearEquiv.symm_symm, LinearEquiv.
    LinearEquiv.apply_symm_apply, Finsupp.coe_sub, Pi.sub_apply, Finsupp.mapDomain_e
    permT_basis_repr_symm_apply, Function.comp_apply, contrT_basis_repr_apply_eq_fin
    prodT_basis_repr_apply, contrMetric_repr_apply_eq_minkowskiMatrix,
    prod_tensor_basis_eq_map_reindex CoVector.basis_eq_map_tensor_basis
    Vector.basis_eq_map_tensor_basis,
    LinearEquiv.symm_apply_apply, Equiv.symm_symm, deriv_basis_repr_apply, Finset.su
  rfl

/-- The statement that  $F = F^{\{\mu\nu\}} e_\mu \otimes e_\nu$  written explicitly, with
with the components given by  $(\eta_{\mu\mu} * \partial_\mu A \times v - \eta_{\nu\nu} * \partial_\nu A \times \mu)$ . -
/
lemma toFieldStrength_eq_sum_basis_single {d} {A : ElectromagneticPotential d}
  (hA : Differentiable ℝ A) (x : SpaceTime d) :
  A.toFieldStrength x =  $\sum \mu, \sum v, (\eta_{\mu\mu} * \partial_\mu A \times v - \eta_{\nu\nu} * \partial_\nu A \times \mu) \cdot$ 
    Lorentz.Vector.basis  $\mu \otimes_t$  Lorentz.Vector.basis  $v :=$  by
  rw [toFieldStrength_eq_sum_basis hA x]
  apply (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr.injective
  ext (μ, ν)
  simp [Basis.tensorProduct_repr_tmul_apply, Finsupp.single_apply]
  rw [Finset.sum_eq_single μ, Finset.sum_eq_single ν]
  · intro b _ hb
    rw [minkowskiMatrix.off_diag_zero]
```

```

      simp only [zero_mul]
      exact id (Ne.symm hb)
    · simp
    · intro b _ hb
      rw [minkowskiMatrix.off_diag_zero]
      simp only [zero_mul]
      exact id (Ne.symm hb)
    · simp

TODO "Add a section in this file on the evaluation of the field strength tensor.
      I.e. equalities related to `toField {A.toFieldStrength x| [μ] [ν]}^T`."

/-!

## A.3. The group action acting on the field strength tensor

We show that the field strength tensor is equivariant under the action of the Lorentz group.
That is transforming the potential and then taking the field strength is the same as
taking the field strength and then transforming the resulting tensor.

-/

set_option backward.isDefEq.respectTransparency false in
lemma toFieldStrength_equivariant {d} (A : ElectromagneticPotential d) (Λ :
  (hf : Differentiable ℝ A) (x : SpaceTime d) :
  (Λ • A).toFieldStrength x = Λ • A.toFieldStrength (Λ⁻¹ • x) := by
  rw [toFieldStrength, deriv_equivariant A Λ hf, ← actionT_constrMetric Λ, toTensor]
  simp only [Tensorial.toTensor_smul, prodT_equivariant, contrT_equivariant,
    permT_equivariant, map_add, ← Tensorial.smul_toTensor_symm, smul_add, smul_comm]

/-- This lemma expresses the component form of the transformed field strength tensor:
when a Lorentz transformation Λ acts on the potential A, the resulting tensor's
components are given by the standard tensor transformation rule in terms of the
matrix elements Λ^μ_κ and Λ^ν_ρ applied to the original field components. -/
lemma toFieldStrength_action_eq_sum {d} (A : ElectromagneticPotential d) (Λ :
  (hf : Differentiable ℝ A) (x : SpaceTime d) :
  (Λ • A).toFieldStrength x = ∑ μ, ∑ ν,
    (∑ κ, ∑ ρ, Λ.1 μ κ * Λ.1 ν ρ * toField {A.toFieldStrength (Λ⁻¹ • x) |
      Vector.basis μ ⊗₊ [ℝ] Vector.basis ν := by
  conv_lhs => rw [toFieldStrength_equivariant A Λ hf x, toFieldStrength_eq_sum]
  change Tensorial.smulLinearMap _ _ = _
  simp only [map_sum, map_smul]
  simp [smulLinearMap, smul_prod, Vector.smul_basis, tmul_sum, sum_tmul,
    Finset.smul_sum, tmul_smul, smul_tmul, smul_smul]
  conv_lhs => enter [2, μ, 2, ν]; rw [Finset.sum_comm]

```

1.24.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/FIELDSTRENGTH.LEAN 81

```
conv_lhs => enter [2,  $\mu$ ]; rw [Finset.sum_comm]
rw [Finset.sum_comm]
refine Finset.sum_congr rfl (fun v _ => ?_)
conv_lhs => enter [2,  $\mu$ ]; rw [Finset.sum_comm]
rw [Finset.sum_comm]
refine Finset.sum_congr rfl (fun  $\mu$  _ => ?_)
simp [← Finset.sum_smul]
congr 1
exact Finset.sum_congr rfl (fun  $\kappa$  _ => Finset.sum_congr rfl (fun  $\kappa$  _ => by ring))

/-!

## A.4. Differentiability and smoothness of the field strength tensor
@[fun_prop]
lemma differentiable_toFieldStrength {d} {A : ElectromagneticPotential d} (hA : ContDiff  $\mathbb{R}$  A) :
  Differentiable  $\mathbb{R}$  A.toFieldStrength := by
  change Differentiable  $\mathbb{R}$  (A.toFieldStrength ·)
  simp only [toFieldStrength_eq_sum_basis_single (hA.differentiable (by simp))]
  fun_prop

open ContDiff

@[fun_prop]
lemma differentiable_toFieldStrength_of_smooth {d}
  {A : ElectromagneticPotential d} (hA : ContDiff  $\mathbb{R}$   $\infty$  A) :
  Differentiable  $\mathbb{R}$  A.toFieldStrength :=
  differentiable_toFieldStrength (hA.of_le (ENat.LEInfty.out))

@[fun_prop]
lemma contDiff_toFieldStrength {d} {n : WithTop  $\mathbb{N}$  $\infty$ } {A : ElectromagneticPotential d}
  (hA : ContDiff  $\mathbb{R}$  (n + 1) A) : ContDiff  $\mathbb{R}$  n A.toFieldStrength := by
  change ContDiff  $\mathbb{R}$  n (A.toFieldStrength ·)
  simp only [toFieldStrength_eq_sum_basis_single (hA.differentiable (by simp))]
  fun_prop

/-!

### A.5. Elements of the field strength tensor in terms of basis

-/

TODO "For the electromagnetic field strength, we have lots of lemmas related
to the components of the field strength tensor in terms of the basis. For example,
`toTensor_toFieldStrength_basis_repr`, these should be removed. They are used
downstream, so there use there should be refactored."
```

```

### A.6. The field strength matrix
#### A.6.1. Differentiability of the field strength matrix
### A.7. The antisymmetry of the field strength tensor
### A.8. Equivariance of the field strength matrix
### A.9. Linearity of the field strength tensor

```

1.25 Physlib/Electromagnetism/Kinematics/MagneticField.lean

```

### A.4. The magnetic field on constructors

```

```

-/

```

```

open Matrix in
/-- The magnetic field of the electromagnetic potential created from the el
`E` and the magnetic field `B` is `B`, as long as Gauss's law is satisfie
/
lemma ofElectromagneticField_magneticField {c : SpeedOfLight}
  (E : ElectricField) (B : MagneticField) (B_contDiff : ∀ t, ContDiff ℝ 1
    (B_grad : ∀ t, ∇ ⊥ (B t) = 0)) :
  (ofElectromagneticField c E B).magneticField c = B := by
  ext1 t
  ext1 x
  have h1 := eq_neg_curl_of_div_zero (B t) (B_contDiff t) (B_grad t)
  conv_rhs => rw [h1]
  simp only [magneticField, ofElectromagneticField_vectorPotential, WithLp.
    WithLp.ofLp_smul, map_smul, LinearMap.smul_apply]
  rw [fun_curl_neg]
  simp only [WithLp.equiv_symm_apply, WithLp.toLp_smul, Pi.neg_apply]
  change Differentiable ℝ fun x =>
    ∫ (u : ℝ) in 0..1, u • WithLp.toLp 2 ((crossProduct (Space.basis.repr x)
      (B t (u • x))).ofLp)
  apply ContDiff.differentiable (n := 1) _ (by simp)
  apply contDiff_parametric_intervalIntegral_of_contDiff
  refine contDiff_euclidean.mpr ?_
  intro i
  let C : (Space) × ℝ → EuclideanSpace ℝ (Fin 3) := fun p =>
    let x := p.1
    let u := p.2
    (u • basis.repr x) ×e3 B t (u • x)
  suffices h : ContDiff ℝ 1 (fun x => C x i) by
    convert h using 1
    simp [C]
    rfl
  fin_cases i

```


1.26.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/SCALARPOTENTIAL.lean

```
all_goals
· simp [C, crossProduct]
  fun_prop

/-!
```

1.26 Physlib/Electromagnetism/Kinematics/ScalarPotential.lean

```
public import Physlib.Electromagnetism.Kinematics.VectorPotential
```

```
- B. Relation to constructors
- C. Smoothness of the Scalar Potential
- D. Differentiability of the Scalar Potential
```

```
## B. Relation to constructors
```

```
-/
```

```
@[simp]
```

```
lemma ofScalarPotential_scalarPotential {d} (c : SpeedOfLight)
  (φ : Time → Space d → ℝ) : (ofScalarPotential c φ).scalarPotential c = φ := by
  simp only [scalarPotential, ofScalarPotential, Fin.isValue]
  field_simp
  simp
```

```
@[simp]
```

```
lemma ofStaticScalarPotential_scalarPotential {d} (c : SpeedOfLight)
  (φ : Space d → ℝ) : (ofStaticScalarPotential c φ).scalarPotential c = fun _ => φ
  simp [ofStaticScalarPotential]
```

```
@[simp]
```

```
lemma ofVectorPotential_scalarPotential {d} (c : SpeedOfLight)
  (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
  (ofVectorPotential c A).scalarPotential = 0 := by
  simp only [scalarPotential, SpeedOfLight.val_one, ofVectorPotential, Fin.isValue,
    rfl]
```

```
@[simp]
```

```
lemma ofStaticVectorPotential_scalarPotential {d} (c : SpeedOfLight)
  (A : Space d → EuclideanSpace ℝ (Fin d)) :
  (ofStaticVectorPotential c A).scalarPotential = 0 := by
  simp [ofStaticVectorPotential]
```

```
@[simp]
```

```
lemma ofPotentials_scalarPotential {d} (c : SpeedOfLight) (φ : Time → Space d → ℝ)
```

```

      (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
      (ofPotentials c ϕ A).scalarPotential c = ϕ := by
    simp only [scalarPotential, ofPotentials, Fin.isValue]
    field_simp
    simp

@[simp]
lemma ofStaticPotentials_scalarPotential {d} (c : SpeedOfLight) (ϕ : Space
  (A : Space d → EuclideanSpace ℝ (Fin d)) :
  (ofStaticPotentials c ϕ A).scalarPotential c = fun _ => ϕ := by
  simp [ofStaticPotentials_eq_ofPotentials]

open MeasureTheory Matrix Space InnerProductSpace Time in
lemma ofElectromagneticField_scalarPotential (c : SpeedOfLight)
  (E : Time → Space → EuclideanSpace ℝ (Fin 3))
  (B : Time → Space → EuclideanSpace ℝ (Fin 3)) :
  (ofElectromagneticField c E B).scalarPotential c = fun t x =>
    - ∫ u in (0 : ℝ)..1, ∫ E t (u • x), basis.repr x ∙_ℝ ∂(volume) := by
  simp [ofElectromagneticField]

open MeasureTheory Matrix Space InnerProductSpace Time in
lemma ofElectromagneticField_scalarPotential_eq_add_vectorPotential (c : Sp
  (E : Time → Space → EuclideanSpace ℝ (Fin 3))
  (B : Time → Space → EuclideanSpace ℝ (Fin 3)) (hb : ContDiff ℝ 1 1B) :
  (ofElectromagneticField c E B).scalarPotential c = fun t x =>
    - ∫ u in (0 : ℝ)..1, ∫ E t (u • x) +
    ∂t ((ofElectromagneticField c E B).vectorPotential c ·
      (u • x)) t, basis.repr x ∙_ℝ ∂(volume) := by
  simp [ofElectromagneticField_scalarPotential, inner_add_left]
  ext t x
  simp only [neg_inj]
  congr
  ext u
  simp [time_deriv_vectorPotential_inner_radial_eq_zero_ofElectromagneticFi

/-!

## C. Smoothness of the Scalar Potential
## d. Differentiability of the Scalar Potential

```

1.27 Physlib/Electromagnetism/Kinematics/VectorPotential.lean

```
## B. Relation to constructors
```

1.27.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/VECTORTPOTENTIAL.LEAN5

-/

@[simp]

```
lemma ofScalarPotential_vectorPotential {d} (c : SpeedOfLight)
  (φ : Time → Space d → ℝ) : (ofScalarPotential c φ).vectorPotential c = 0 := by
  simp only [vectorPotential, ofScalarPotential]
  rfl
```

@[simp]

```
lemma ofStaticScalarPotential_vectorPotential {d} (c : SpeedOfLight)
  (φ : Space d → ℝ) : (ofStaticScalarPotential c φ).vectorPotential c = 0 := by
  simp [ofStaticScalarPotential]
```

@[simp]

```
lemma ofVectorPotential_vectorPotential {d} (c : SpeedOfLight)
  (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
  (ofVectorPotential c A).vectorPotential c = A := by
  ext i
  simp [vectorPotential, ofVectorPotential]
```

@[simp]

```
lemma ofStaticVectorPotential_vectorPotential {d} (c : SpeedOfLight)
  (A : Space d → EuclideanSpace ℝ (Fin d)) :
  (ofStaticVectorPotential c A).vectorPotential c = fun _ => A := by
  simp [ofStaticVectorPotential]
```

@[simp]

```
lemma ofPotentials_vectorPotential {d} (c : SpeedOfLight) (φ : Time → Space d → ℝ)
  (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
  (ofPotentials c φ A).vectorPotential c = A := by
  ext i
  simp [vectorPotential, ofPotentials]
```

@[simp]

```
lemma ofStaticPotentials_vectorPotential {d} (c : SpeedOfLight) (φ : Space d → ℝ)
  (A : Space d → EuclideanSpace ℝ (Fin d)) :
  (ofStaticPotentials c φ A).vectorPotential c = fun _ => A := by
  simp [ofStaticPotentials_eq_ofPotentials]
```

/-!

B.1. ofElectromagneticField

-/

```

open MeasureTheory Matrix Space InnerProductSpace Time in
lemma ofElectromagneticField_vectorPotential (c : SpeedOfLight)
  (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
  (B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) :
  (ofElectromagneticField c E B).vectorPotential c =
  fun t x => - ∫ u in 0..(1 : ℝ), (u • Space.basis.repr x) ×e3 B t (u • x)
simp [ofElectromagneticField]

open MeasureTheory Matrix Space InnerProductSpace Time in
lemma ofElectromagneticField_vectorPotential_apply {t x} (c : SpeedOfLight)
  (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
  (B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) (i : Fin 3) (hB : Continuous 1B) :
  (ofElectromagneticField c E B).vectorPotential c t x i =
  - ∫ u in 0..(1 : ℝ), ((u • Space.basis.repr x) ×e3 B t (u • x)) i ∂volume
simp [ofElectromagneticField_vectorPotential]
rw [intervalIntegral.integral_of_le (by simp), intervalIntegral.integral_of_le]
rw [MeasureTheory.eval_integral_piLp]
simp only [PiLp.smul_apply, smul_eq_mul]
intro i
apply MeasureTheory.IntegrableOn.integrable
rw [← intervalIntegrable_iff_integrableOn_Ioc_of_le]
apply Continuous.intervalIntegrable
fun_prop
simp

open MeasureTheory Matrix Space InnerProductSpace Time in
lemma ofElectromagneticField_vectorPotential_apply_eq_expand {t x} {c : SpeedOfLight}
  {E : Time → Space 3 → EuclideanSpace ℝ (Fin 3)}
  {B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)} (hB : Continuous 1B) (i : Fin 3) :
  (ofElectromagneticField c E B).vectorPotential c t x i =
  x (i + 2) * ∫ u in 0..(1 : ℝ), u * B t (u • x) (i + 1) ∂volume -
  x (i + 1) * ∫ u in 0..(1 : ℝ), u * B t (u • x) (i + 2) ∂volume := by
  fin_cases i
  all_goals
  · rw [ofElectromagneticField_vectorPotential_apply _ _ _ _ hB]
    simp [crossProduct]
    ring_nf
    rw [intervalIntegral.integral_sub, ← intervalIntegral.integral_const_mul,
      ← intervalIntegral.integral_const_mul]
    ring_nf
    congr
    · ext u
      ring
    · ext u
      ring

```

1.27.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/VECTORTPOTENTIAL.LEAN7

- apply Continuous.intervalIntegrable
fun_prop
- apply Continuous.intervalIntegrable
fun_prop

open MeasureTheory Matrix Space InnerProductSpace Time in

@[fun_prop]

```
lemma contDiff_vectorPotential_ofElectromagneticField {n : ℕ} (c : SpeedOfLight)
  (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
  (B : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
  (hB : ContDiff ℝ n 1 B) : ContDiff ℝ n 1 ((ofElectromagneticField c E B).vectorPot)
let A : Time → Space → EuclideanSpace ℝ (Fin 3) := fun t x =>
  - ∫ u in 0..(1 : ℝ), (u • basis.repr x) ×e3 B t (u • x) ∂(volume)
have h1 : ContDiff ℝ n 1 A := by
  simp only [WithLp.equiv_apply, A]
  apply ContDiff.neg
  apply contDiff_parametric_intervalIntegral_of_contDiff
  refine contDiff_euclidean.mpr ?_
  intro i
  let C : (Time × Space) × ℝ → EuclideanSpace ℝ (Fin 3) := fun p =>
    let (t, x) := p.1
    let u := p.2
    (u • basis.repr x) ×e3 B t (u • x)
  change ContDiff ℝ n (fun x => C x i)
  fin_cases i
  all_goals
    · simp [C, crossProduct]
      fun_prop
  suffices h : ContDiff ℝ n 1 A by
    convert h
    simp [ofElectromagneticField_vectorPotential, A]
  fun_prop
```

open InnerProductSpace

```
lemma vectorPotential_inner_radial_eq_zero_ofElectromagneticField
  {c : SpeedOfLight} {E B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)}
  {t : Time} {x : Space 3} {a : ℝ} (hB : Continuous 1 B) :
  □(ofElectromagneticField c E B).vectorPotential c t (a • x), Space.basis.repr x □
rw [real_inner_comm]
rw [PiLp.inner_apply]
have h1 (a b : ℝ) : □a, b□ℝ = b * a := by rfl
simp only [Space.basis_repr_apply, ofElectromagneticField_vectorPotential_apply_eq,
  Fin.isValue, Space.smul_apply, h1, Fin.sum_univ_three, zero_add, Fin.reduceAdd]
ring
```

```

open Time
@[simp]
lemma time_deriv_vectorPotential_inner_radial_eq_zero_ofElectromagneticField
  {c : SpeedOfLight} {E B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)}
  {t : Time} {x : Space 3} {a : ℝ} (hB : ContDiff ℝ 1 1 B) :
  ∂t ((ofElectromagneticField c E B).vectorPotential c · (a • x)) t,
  Space.basis.repr x|_ℝ = 0 := by
  trans ∂t (fun t => ∂(ofElectromagneticField c E B).vectorPotential c t (a
    Space.basis.repr x|_ℝ) t
    · rw [Time.deriv, Time.deriv]
      rw [fderiv_inner_apply]
      simp only [fderiv_fun_const, Pi.zero_apply, ContinuousLinearMap.zero_ap
        zero_add]
      apply Differentiable.differentiableAt
      fun_prop
      fun_prop
  conv_lhs =>
    enter [1, t]
    rw [vectorPotential_inner_radial_eq_zero_ofElectromagneticField (by fun
      simp
    /-!

```

1.28 Physlib/Electromagnetism/PointParticle/OneDimension.lean

```

public import Physlib.Electromagnetism.Distributional.Dynamics.IsExtrema
have h1 := Space.distDiv_inv_pow_eq_dim (d := 1)

```

1.29 Physlib/Electromagnetism/PointParticle/ThreeDimension.lean

```

public import Physlib.Electromagnetism.Distributional.Dynamics.IsExtrema
have h1 := Space.distDiv_inv_pow_eq_dim (d := 3)

```

1.30 Physlib/FluidDynamics/FluidState.lean

```

/-
Copyright (c) 2026 Florian Wiesner. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Florian Wiesner
-/
module

```

```

public import Physlib.SpaceAndTime.Space.Basic
public import Physlib.SpaceAndTime.Time.Basic
/-!

# Fluid states

## i. Overview

This module defines the basic fields used to describe a fluid on `d`-
dimensional space.
The core structure `FluidState` contains only the density and velocity fields. Addit
fields used by specific balance laws are provided by extension structures.

## ii. Key results

- `ScalarField` : A time-dependent scalar field on space.
- `VectorField` : A time-dependent vector field on space.
- `MassDensity` : A time-dependent scalar density field.
- `VelocityField` : A time-dependent vector velocity field.
- `MomentumDensityField` : A time-dependent vector momentum density field.
- `StressTensor` : A time-dependent matrix-valued stress field.
- `BodyForce` : A time-dependent vector body-force field per unit mass.
- `FluidState` : The density and velocity fields of a fluid.
- `FluidInMomentumBalance` : A fluid state with stress and body force.

## iii. Table of contents

- A. Field types
- B. Fluid state structures

## iv. References

-/

@[expose] public section
namespace FluidDynamics

/-!

## A. Field types

-/

/-- A scalar field on `d`-dimensional space, depending on time. -

```

```

/
abbrev ScalarField (d : ℕ) := Time → Space d → ℝ

/-- A vector field on `d`-dimensional space, depending on time. -
/
abbrev VectorField (d : ℕ) := Time → Space d → EuclideanSpace ℝ (Fin d)

/-- A mass density field on `d`-dimensional space. -/
abbrev MassDensity (d : ℕ) := ScalarField d

/-- A velocity field on `d`-dimensional space. -/
abbrev VelocityField (d : ℕ) := VectorField d

/-- A momentum density field on `d`-dimensional space. -/
abbrev MomentumDensityField (d : ℕ) := VectorField d

/-- A matrix-valued stress tensor field on `d`-dimensional space. -
/
abbrev StressTensor (d : ℕ) := Time → Space d → Matrix (Fin d) (Fin d) ℝ

/-- A body-force field per unit mass on `d`-dimensional space. -
/
abbrev BodyForce (d : ℕ) := VectorField d

/-!

## B. Fluid state structures

-/

/-- The density and velocity fields of a fluid on `d`-dimensional space. -
/
structure FluidState (d : ℕ) where
  /-- The mass density field. -/
  rho : MassDensity d
  /-- The velocity field. -/
  velocity : VelocityField d

/-- The fields needed for a momentum balance: fluid state, stress, and body
/
structure FluidInMomentumBalance (d : ℕ) extends FluidState d where
  /-- The stress tensor field. -/
  stress : StressTensor d
  /-- The body-force field per unit mass. -/
  bodyForce : BodyForce d

```



```
end FluidDynamics
```

1.31 Physlib/FluidDynamics/NavierStokes/Basic.lean

```
/-
```

```
Copyright (c) 2026 Florian Wiesner. All rights reserved.  
Released under Apache 2.0 license as described in the file LICENSE.
```

```
Authors: Florian Wiesner
```

```
-/
```

```
module
```

```
public import Physlib.FluidDynamics.NavierStokes.Momentum
```

```
/-!
```

```
# The Navier-Stokes equations
```

```
## i. Overview
```

The Navier-Stokes equations are a set of partial differential equations that describe the motion of viscous fluid substances. They are fundamental in fluid dynamics and are used to model the behavior of fluids in various contexts, including gas flow and water flow.

This file combines the classical continuity equation with the momentum equation. The stress tensor is left as an input field, so this is the balance-law layer before specializing to a Newtonian stress law.

```
## ii. Key results
```

- `NavierStokes` : Classical continuity and conservative momentum equations together
- `ConvectiveNavierStokes` : Classical continuity and convective momentum equations
- `NavierStokes\_iff\_ConvectiveNavierStokes` : Equivalence of the two forms when the fields are differentiable.

```
## iii. Table of contents
```

- A. Full Navier-Stokes forms

```
## iv. References
```

```
-/
```

```
@[expose] public section
```

```

namespace FluidDynamics

/-!

## A. Full Navier-Stokes forms

-/

/-- The conservative Navier-Stokes balance-law form with an externally supplied stress tensor.
/
def NavierStokes (d :  $\mathbb{N}$ ) (data : FluidInMomentumBalance d) : Prop :=
  FluidDynamics.NavierStokes.ClassicalContinuityEquation d data.toFluidState
  FluidDynamics.NavierStokes.MomentumEquation d data

/-- The convective Navier-Stokes form with an externally supplied stress tensor.
/
def ConvectiveNavierStokes (d :  $\mathbb{N}$ ) (data : FluidInMomentumBalance d) : Prop :=
  FluidDynamics.NavierStokes.ClassicalContinuityEquation d data.toFluidState
  FluidDynamics.NavierStokes.ConvectiveMomentumEquation d data

/-- The conservative and convective Navier-Stokes forms are equivalent when
    differentiable enough for the product rules. -/
theorem navierStokes_iff_convectiveNavierStokes
  (d :  $\mathbb{N}$ ) (data : FluidInMomentumBalance d)
  (hRhoTime :  $\forall t x, \text{DifferentiableAt } \mathbb{R} (\text{data.rho} \cdot x) t$ )
  (hVelocityTime :  $\forall t x, \text{DifferentiableAt } \mathbb{R} (\text{data.velocity} \cdot x) t$ )
  (hMomentumDensity :  $\forall t,$ 
    Differentiable  $\mathbb{R}$  (FluidDynamics.NavierStokes.momentumDensity d data.toFluidState t))
  (hVelocitySpace :  $\forall t, \text{Differentiable } \mathbb{R} (\text{data.velocity } t)$ ) :
  NavierStokes d data  $\leftrightarrow$  ConvectiveNavierStokes d data := by
  constructor
  · intro hConservative
    refine {hConservative.1, ?_}
    exact (FluidDynamics.NavierStokes.momentumEquation_iff_convectiveMomentumEquation
      hConservative.1 hRhoTime hVelocityTime hMomentumDensity hVelocitySpace)
  · intro hConvective
    refine {hConvective.1, ?_}
    exact (FluidDynamics.NavierStokes.momentumEquation_iff_convectiveMomentumEquation
      hConvective.1 hRhoTime hVelocityTime hMomentumDensity hVelocitySpace)

end FluidDynamics

```

1.32 Physlib/FluidDynamics/NavierStokes/Continuity.lean

```

/-
Copyright (c) 2026 Florian Wiesner. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Florian Wiesner
-/
module

public import Physlib.FluidDynamics.FluidState
public import Physlib.SpaceAndTime.Space.Derivatives.Div
public import Physlib.SpaceAndTime.Time.Derivatives
/-!

# The Navier-Stokes continuity equation

## i. Overview

This module defines the classical conservative mass-balance equation for a fluid state and
the corresponding continuity residual.

## ii. Key results

- `ClassicalContinuityEquation` : Classical conservation of mass in conservative form.
- `continuityResidual` : The scalar residual  $\partial_t \rho + \operatorname{div}(\rho u)$ .
- `SmoothContinuityEquation` : Continuity for globally differentiable fields.
- `SmoothContinuityEquation.toClassical` : Smooth continuity implies classical continuity.

## iii. Table of contents

- A. Continuity equations

## iv. References

-/

@[expose] public section

open Space
open Time

namespace FluidDynamics
namespace NavierStokes

/-!

```

A. Continuity equations

-/

/-- Classical conservation of mass in conservative form, $\partial_t \rho + \operatorname{div}(\rho u)$

The equation is asserted at points where the time derivative of ρ and the divergence of ρu are classical derivatives.

-/

```
def ClassicalContinuityEquation (d : ℕ) (fluid : FluidState d) : Prop :=
  ∀ t x, DifferentiableAt ℝ (fluid.rho · x) t →
    DifferentiableAt ℝ (fun x' => fluid.rho t x' • fluid.velocity t x') x' →
      ∂t (fluid.rho · x) t + (∇ · fun x' => fluid.rho t x' • fluid.velocity t x') t = 0
```

/-- The scalar continuity-equation residual

$\partial_t \rho + \operatorname{div}(\rho u)$. -/

```
noncomputable def continuityResidual (d : ℕ) (fluid : FluidState d) : Time
  fun t x => ∂t (fluid.rho · x) t + (∇ · fun x' => fluid.rho t x' • fluid.velocity t x') t
```

/-- A stronger continuity equation for globally differentiable fields.

This version records the first-order regularity needed by the classical continuity equation: if the density is differentiable in time, the mass flux ρu is differentiable, and the continuity residual vanishes everywhere.

-/

```
def SmoothContinuityEquation (d : ℕ) (fluid : FluidState d) : Prop :=
  (∀ x, Differentiable ℝ (fluid.rho · x)) ∧
  (∀ t, Differentiable ℝ (fun x => fluid.rho t x • fluid.velocity t x)) ∧
  ∀ t x, continuityResidual d fluid t x = 0
```

/-- A smooth continuity equation satisfies the guarded classical continuity equation -/

/

```
lemma SmoothContinuityEquation.toClassical (d : ℕ) (fluid : FluidState d) :
  SmoothContinuityEquation d fluid → ClassicalContinuityEquation d fluid
```

```
  intro hSmooth t x
  simp [continuityResidual] using hSmooth.2.2 t x
```

```
end NavierStokes
```

```
end FluidDynamics
```

1.33 Physlib/FluidDynamics/NavierStokes/Momentum.lean

-/

Copyright (c) 2026 Florian Wiesner. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Florian Wiesner

-/
module

```
public import Physlib.FluidDynamics.NavierStokes.Continuity
public import Physlib.SpaceAndTime.Space.Derivatives.MatrixDiv
/-!
```

The Navier-Stokes momentum equations

i. Overview

This module defines the conservative and convective momentum equations for a fluid with stress and body-force fields. The stress tensor is left as an input field, so this is a balance-law layer before specializing to a Newtonian stress law.

ii. Key results

- ``momentumDensity`` : The vector momentum density $\rho \mathbf{u}$.
- ``momentumFlux`` : The convective momentum flux $\rho \mathbf{u} \otimes \mathbf{u}$.
- ``MomentumEquation`` : Conservation of momentum using ``Space.matrixDiv``.
- ``convectiveTerm`` : The nonlinear transport term $(\mathbf{u} \cdot \nabla) \mathbf{u}$.
- ``materialAcceleration`` : The material acceleration $\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u}$.
- ``ConvectiveMomentumEquation`` : The momentum equation in convective form.
- ``momentumEquation_iff_convectiveMomentumEquation`` : Equivalence of the two momentum equations when continuity holds and the fields are differentiable.

iii. Table of contents

- A. Momentum fields
- B. Conservative momentum equation
- C. Convective momentum equation
- D. Equivalence of conservative and convective momentum

iv. References

-/

@[expose] public section

open Space
open Time

namespace FluidDynamics
namespace NavierStokes

```

/-!

## A. Momentum fields

-/

/-- The momentum density  $\rho u$ . -/
def momentumDensity (d :  $\mathbb{N}$ ) (fluid : FluidState d) : MomentumDensityField d :=
  fun t x => fluid.rho t x • fluid.velocity t x

/-- The convective momentum flux  $\rho u \otimes u$ . -/
def momentumFlux (d :  $\mathbb{N}$ ) (fluid : FluidState d) : Time → Space d → Matrix (
  fun t x =>
    fluid.rho t x • Matrix.vecMulVec (fun i => fluid.velocity t x i)
      (fun j => fluid.velocity t x j))

/-!

## B. Conservative momentum equation

-/

/-- Conservation of momentum in conservative matrix-divergence form.

The equation is


$$\partial_t (\rho u) + \text{matrixDiv} (\rho u \otimes u) = \text{matrixDiv} \sigma + \rho f.$$


Here `stress` is intentionally not yet specialized to a Newtonian stress la
-/
def MomentumEquation (d :  $\mathbb{N}$ ) (data : FluidInMomentumBalance d) : Prop :=
   $\forall$  t x,
     $\partial_t$  (momentumDensity d data.toFluidState • x) t +
      matrixDiv d (momentumFlux d data.toFluidState t) x =
      matrixDiv d (data.stress t) x + data.rho t x • data.bodyForce t x

/-!

## C. Convective momentum equation

-/

/-- The nonlinear transport term  $(u \cdot \nabla)u$ . -/
noncomputable def convectiveTerm (d :  $\mathbb{N}$ ) (fluid : FluidState d) : VectorFie
  fun t x =>  $\sum$  j, fluid.velocity t x j •  $\partial[j]$  (fluid.velocity t) x

```

```

/-- The material acceleration  $\partial_t u + (u \cdot \nabla)u$ . -/
noncomputable def materialAcceleration (d : ℕ) (fluid : FluidState d) : VectorField d :=
  fun t x =>  $\partial_t$  (fluid.velocity · x) t + convectiveTerm d fluid t x

/-- Conservation of momentum in convective form.

The equation is

 $\rho (\partial_t u + (u \cdot \nabla)u) = \text{matrixDiv } \sigma + \rho f$ .

Here `stress` is intentionally not yet specialized to a Newtonian stress law.
-/
def ConvectiveMomentumEquation (d : ℕ) (data : FluidInMomentumBalance d) : Prop :=
   $\forall$  t x,
    data.rho t x • materialAcceleration d data.toFluidState t x =
      matrixDiv d (data.stress t) x + data.rho t x • data.bodyForce t x

/-!

## D. Equivalence of conservative and convective momentum

-/

/-- The left-hand side of the conservative momentum equation. -
/
noncomputable def conservativeMomentumLHS (d : ℕ) (fluid : FluidState d) : VectorField d :=
  fun t x =>  $\partial_t$  (momentumDensity d fluid · x) t + matrixDiv d (momentumFlux d fluid t) x

/-- The left-hand side of the convective momentum equation. -
/
noncomputable def convectiveMomentumLHS (d : ℕ) (fluid : FluidState d) : VectorField d :=
  fun t x => fluid.rho t x • materialAcceleration d fluid t x

/-- Product rule for the time derivative of a scalar field times a velocity field. -
/
lemma timeDeriv_smul_velocity (d : ℕ) (rhoAtPosition : Time → ℝ)
  (velocityAtPosition : Time → EuclideanSpace ℝ (Fin d)) (t : Time)
  (hRho : DifferentiableAt ℝ rhoAtPosition t)
  (hVelocity : DifferentiableAt ℝ velocityAtPosition t) :
   $\partial_t$  (fun t' => rhoAtPosition t' • velocityAtPosition t') t =
    rhoAtPosition t •  $\partial_t$  velocityAtPosition t +  $\partial_t$  rhoAtPosition t • velocityAtPosition t
rw [Time.deriv_eq, Time.deriv_eq, Time.deriv_eq]
change (fderiv ℝ (rhoAtPosition • velocityAtPosition) t) 1 =
  rhoAtPosition t • (fderiv ℝ velocityAtPosition t) 1 +
  (fderiv ℝ rhoAtPosition t) 1 • velocityAtPosition t

```

```

rw [fderiv_smul hRho hVelocity]
rfl

/-- Product rule for the time derivative of the momentum density `rho u`. -
/
lemma timeDeriv_momentumDensity (d : ℕ) (fluid : FluidState d)
  (t : Time) (x : Space d)
  (hRho : DifferentiableAt ℝ (fluid.rho · x) t)
  (hVelocity : DifferentiableAt ℝ (fluid.velocity · x) t) :
  ∂t (momentumDensity d fluid · x) t =
    fluid.rho t x • ∂t (fluid.velocity · x) t + ∂t (fluid.rho · x) t • fl
simp [momentumDensity] using
  timeDeriv_smul_velocity d (fluid.rho · x) (fluid.velocity · x) t hRho h

/-- Product rule for one spatial derivative of one component of `rho u ⊗ u` -
/
lemma spaceDeriv_momentumFlux_component (d : ℕ) (fluid : FluidState d)
  (t : Time) (x : Space d) (i j : Fin d)
  (hMomentumDensity : Differentiable ℝ (momentumDensity d fluid t))
  (hVelocity : Differentiable ℝ (fluid.velocity t)) :
  ∂[j] (fun x' => momentumFlux d fluid t x' i j) x =
    fluid.velocity t x i • ∂[j] (fun x' => momentumDensity d fluid t x' j) x +
    ∂[j] (fun x' => fluid.velocity t x' i) x • momentumDensity d fluid t x
have hProduct := Space.deriv_smul (u := j) (x := x)
  (c := fun x' => fluid.velocity t x' i)
  (f := fun x' => momentumDensity d fluid t x' j)
  ((differentiable_euclidean.mp hVelocity i).differentiableAt)
  ((differentiable_euclidean.mp hMomentumDensity j).differentiableAt)
rw [← hProduct]
congr
funext x'
simp [momentumFlux, momentumDensity, Matrix.vecMulVec_apply, mul_left_commut]

/-- The matrix divergence of `rho u ⊗ u` split into continuity and convective -
/
lemma matrixDiv_momentumFlux (d : ℕ) (fluid : FluidState d)
  (t : Time) (x : Space d)
  (hMomentumDensity : Differentiable ℝ (momentumDensity d fluid t))
  (hVelocity : Differentiable ℝ (fluid.velocity t)) :
  matrixDiv d (momentumFlux d fluid t) x =
    (∇ ∙ momentumDensity d fluid t) x • fluid.velocity t x +
    fluid.rho t x • convectiveTerm d fluid t x := by
ext i
simp [matrixDiv_apply, div, convectiveTerm, smul_eq_mul]
change (∑ j, ∂[j] (fun x' => momentumFlux d fluid t x' i j) x) =
  (∑ j, ∂[j] (fun x' => momentumDensity d fluid t x' j) x) * fluid.velocity

```



```

    fluid.rho t x * (∑ j, fluid.velocity t x j * ∂[j] (fluid.velocity t) x i)
calc
  (∑ j, ∂[j] (fun x' => momentumFlux d fluid t x' i j) x)
    = ∑ j,
      (fluid.velocity t x i * ∂[j] (fun x' => momentumDensity d fluid t x' j)
        ∂[j] (fun x' => fluid.velocity t x' i) x * momentumDensity d fluid t x)
    apply Finset.sum_congr rfl
    intro j _
    rw [spaceDeriv_momentumFlux_component d fluid t x i j
        hMomentumDensity hVelocity]
    simp [smul_eq_mul]
  _ = fluid.velocity t x i * (∑ j, ∂[j] (fun x' => momentumDensity d fluid t x' j)
    fluid.rho t x * (∑ j, fluid.velocity t x j * ∂[j] (fluid.velocity t) x i) :=
    rw [Finset.sum_add_distrib]
    congr 1
    · rw [Finset.mul_sum]
    · rw [Finset.mul_sum]
    apply Finset.sum_congr rfl
    intro j _
    rw [Space.deriv_euclid (v := j) (μ := i) (f := fluid.velocity t)
        hVelocity x]
    simp [momentumDensity, mul_comm, mul_assoc]
  _ = (∑ j, ∂[j] (fun x' => momentumDensity d fluid t x' j) x) * fluid.velocity t
    fluid.rho t x * (∑ j, fluid.velocity t x j * ∂[j] (fluid.velocity t) x i) :=
    ring

```

-- The algebraic bridge between conservative and convective momentum.

The conservative momentum left-hand side equals the convective momentum left-hand side plus

the continuity residual times the velocity field.

--

```

lemma conservativeMomentumLHS_eq_convectiveMomentumLHS_add_continuityResidual_smul
  (d : ℕ) (fluid : FluidState d)
  (t : Time) (x : Space d)
  (hRhoTime : DifferentiableAt ℝ (fluid.rho · x) t)
  (hVelocityTime : DifferentiableAt ℝ (fluid.velocity · x) t)
  (hMomentumDensity : Differentiable ℝ (momentumDensity d fluid t))
  (hVelocitySpace : Differentiable ℝ (fluid.velocity t)) :
  conservativeMomentumLHS d fluid t x =
    convectiveMomentumLHS d fluid t x + continuityResidual d fluid t x • fluid.vel
  rw [conservativeMomentumLHS, convectiveMomentumLHS, continuityResidual]
  rw [timeDeriv_momentumDensity d fluid t x hRhoTime hVelocityTime]
  rw [matrixDiv_momentumFlux d fluid t x hMomentumDensity hVelocitySpace]
  ext i
  simp [materialAcceleration, convectiveTerm, div, momentumDensity, smul_eq_mul]

```

ring_nf

-- The conservative and convective momentum equations are equivalent when continuity equation holds.

The differentiability assumptions are exactly the product-rule assumptions used to rewrite

`partial\_t (rho u)` and `matrixDiv (rho u  $\otimes$  u)`.

-/

theorem momentumEquation_iff_convectiveMomentumEquation

(d : $\mathbb{N}$) (data : FluidInMomentumBalance d)
 (hContinuity : ClassicalContinuityEquation d data.toFluidState)
 (hRhoTime : $\forall$ t x, DifferentiableAt $\mathbb{R}$ (data.rho $\cdot$ x) t)
 (hVelocityTime : $\forall$ t x, DifferentiableAt $\mathbb{R}$ (data.velocity $\cdot$ x) t)
 (hMomentumDensity : $\forall$ t,
 Differentiable $\mathbb{R}$ (momentumDensity d data.toFluidState t))
 (hVelocitySpace : $\forall$ t, Differentiable $\mathbb{R}$ (data.velocity t)) :
 MomentumEquation d data $\leftrightarrow$ ConvectiveMomentumEquation d data := by

constructor

· intro hConservative t x

 have hMassFluxSpace :

 DifferentiableAt $\mathbb{R}$ (fun x' => data.rho t x' $\cdot$ data.velocity t x') x

 simp [momentumDensity] using (hMomentumDensity t).differentiableAt

 have hResidual : continuityResidual d data.toFluidState t x = 0 := by

 simp [continuityResidual] using

 hContinuity t x (by simp using hRhoTime t x) hMassFluxSpace

 have hLhs := conservativeMomentumLHS_eq_convectiveMomentumLHS_add_conti

 d data.toFluidState t x (hRhoTime t x) (hVelocityTime t x)

 (hMomentumDensity t) (hVelocitySpace t)

 have hLhs' :

 conservativeMomentumLHS d data.toFluidState t x =

 convectiveMomentumLHS d data.toFluidState t x := by

 rw [hLhs, hResidual, zero_smul, add_zero]

 change convectiveMomentumLHS d data.toFluidState t x =

 matrixDiv d (data.stress t) x + data.rho t x $\cdot$ data.bodyForce t x

 rw [← hLhs']

 exact hConservative t x

· intro hConvective t x

 have hMassFluxSpace :

 DifferentiableAt $\mathbb{R}$ (fun x' => data.rho t x' $\cdot$ data.velocity t x') x

 simp [momentumDensity] using (hMomentumDensity t).differentiableAt

 have hResidual : continuityResidual d data.toFluidState t x = 0 := by

 simp [continuityResidual] using

 hContinuity t x (by simp using hRhoTime t x) hMassFluxSpace

 have hLhs := conservativeMomentumLHS_eq_convectiveMomentumLHS_add_conti

 d data.toFluidState t x (hRhoTime t x) (hVelocityTime t x)

1.34.

PHYSLIB/MATHEMATICS/CALCULUS/PARAMETRICINTEGRATION.LEAN01

```
(hMomentumDensity t) (hVelocitySpace t)
have hLhs' :
  conservativeMomentumLHS d data.toFluidState t x =
    convectiveMomentumLHS d data.toFluidState t x := by
  rw [hLhs, hResidual, zero_smul, add_zero]
change conservativeMomentumLHS d data.toFluidState t x =
  matrixDiv d (data.stress t) x + data.rho t x • data.bodyForce t x
rw [hLhs']
exact hConvective t x

end NavierStokes
end FluidDynamics
```

1.34 Physlib/Mathematics/Calculus/ParametricIntegration.lean

```
/-
Copyright (c) 2026 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Mathlib.Analysis.Calculus.ContDiff.FiniteDimension
public import Mathlib.Analysis.Calculus.ParametricIntervalIntegral
public import Mathlib.Tactic.Cases

/-!

# Parametric Integration

In this module we give some lemmas around parametric integration in Lean.
These extend some lemmas in Mathlib, and give them in a more physics-
friendly way.

-/

@[expose] public section
noncomputable section
open Module
open scoped InnerProductSpace

variable {M N : Type}
  [NormedAddCommGroup M] [NormedSpace ℝ M] [ProperSpace M]
  [NormedAddCommGroup N] [NormedSpace ℝ N]
```

```

open MeasureTheory
lemma hasFDerivAt_parametric_intervalIntegral_of_contDiff
  {F : M → ℝ → N} (hf : ContDiff ℝ 1 1 F) (x₀ : M) :
    HasFDerivAt (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume))
      (∫ (t : ℝ) in 0..1, fderiv ℝ (F · t) x₀ ∂(volume)) x₀ := by
  let F' : M → ℝ → M → L[ℝ] N := fun x t => fderiv ℝ (F · t) x
  let s (x₀) : Set M := Metric.closedBall x₀ 1
  have hF' : Continuous 1 F' := by fun_prop
  obtain ⟨a, ha⟩ := IsCompact.exists_isMaxOn (s := s x₀ ×s Set.Icc (0 : ℝ)
    (by exact (isCompact_closedBall x₀ 1).prod <|
      ConditionallyCompleteLinearOrder.isCompact_Icc 0 1)
    (f := fun (a : M × ℝ) => ||F' a.1 a.2||)
    (by simp [s])
    (by
      apply (Continuous.comp ?_ hF').continuousOn
      change Continuous (@norm _ ContinuousLinearMap.toSeminormedAddCommG
        fun_prop)
    change HasFDerivAt (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume))
      (∫ (t : ℝ) in 0..1, F' x₀ t ∂(volume)) x₀
    have hx := hf.differentiable (by simp)
    apply intervalIntegral.hasFDerivAt_integral_of_dominated_of_fderiv_le (s
      (bound := fun t => ||F' a.1 a.2||)
      · exact Metric.closedBall_mem_nhds x₀ (by simp)
      · filter_upwards with x
        apply Continuous.astronglyMeasurable
        fun_prop
      · apply Continuous.intervalIntegrable
        fun_prop
      · refine IntervalIntegrable.astronglyMeasurable' ?_
        simp only [zero_le_one, sup_of_le_right, inf_of_le_left]
        apply Continuous.intervalIntegrable
        exact Continuous.uncurry_left x₀ (by fun_prop)
      · filter_upwards with t
        intro h x hx
        have hx2 := ha.2
        rw [isMaxOn_iff] at hx2
        apply hx2 (x, t)
        simp at h
        grind
      · apply Continuous.intervalIntegrable
        fun_prop
      · filter_upwards with t
        intro h x hx
        dsimp [F']
        apply DifferentiableAt.hasFDerivAt
        fun_prop
    )
  )

```

1.34.

PHYSLIB/MATHEMATICS/CALCULUS/PARAMETRICINTEGRATION.LEAN#03

```
lemma fderiv_apply_parametric_intervalIntegral
  {F : M → ℝ → N} (hf : ContDiff ℝ 1 1 F) (x₀ : M) (v : M) :
  fderiv ℝ (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume)) x₀ v =
  ∫ (t : ℝ) in 0..1, fderiv ℝ (F · t) x₀ v ∂(volume) := by
  have h := hasFDerivAt_parametric_intervalIntegral_of_contDiff hf x₀
  rw [h.fderiv]
  refine ContinuousLinearMap.intervalIntegral_apply ?_ v
  apply Continuous.intervalIntegrable
  fun_prop
```

```
lemma fderiv_parametric_intervalIntegral
  {F : M → ℝ → N} (hf : ContDiff ℝ 1 1 F) (x₀ : M) :
  fderiv ℝ (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume)) =
  fun x => ∫ (t : ℝ) in 0..1, fderiv ℝ (F · t) x ∂(volume) := by
  have h := hasFDerivAt_parametric_intervalIntegral_of_contDiff hf x₀
  ext1 x
  ext1 v
  rw [fderiv_apply_parametric_intervalIntegral]
  symm
  refine ContinuousLinearMap.intervalIntegral_apply ?_ v
  apply Continuous.intervalIntegrable
  fun_prop
  fun_prop
```

```
lemma contDiff_one_parametric_intervalIntegral_of_contDiff
  {F : M → ℝ → N} (hf : ContDiff ℝ 1 1 F) :
  ContDiff ℝ 1 (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume)) := by
  rw [contDiff_one_iff_hasFDerivAt]
  use fun x₀ => (∫ (t : ℝ) in 0..1, fderiv ℝ (F · t) x₀ ∂(volume))
  apply And.intro
  · fun_prop
  · intro x
    exact hasFDerivAt_parametric_intervalIntegral_of_contDiff hf x
```

```
lemma contDiff_succ_parametric_intervalIntegral_of_contDiff {n : ℕ} [FiniteDimensional M]
  {F : M → ℝ → N} (hf : ContDiff ℝ (n + 1) 1 F) :
  ContDiff ℝ (n + 1) (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume)) := by
  induction' n with n ih generalizing F
  · apply contDiff_one_parametric_intervalIntegral_of_contDiff
    exact hf
  · rw [contDiff_succ_iff_fderiv]
    constructor
    · apply ContDiff.differentiable (n := 1)
      · apply contDiff_one_parametric_intervalIntegral_of_contDiff
        exact hf.of_le (by simp)
```

```

      · simp
    simp only [Nat.cast_add, Nat.cast_one, WithTop.add_eq_top, WithTop.natC
      WithTop.one_ne_top, or_self, IsEmpty.forall_iff, true_and]
    rw [contDiff_clm_apply_iff]
    intro y
    conv =>
      enter [3, x];
      rw [fderiv_apply_parametric_intervalIntegral (hf.of_le (by simp))]
    apply ih
    fun_prop

lemma contDiff_parametric_intervalIntegral_of_contDiff {n : ℕ} {M : Type}
  [NormedAddCommGroup M] [NormedSpace ℝ M] [ProperSpace M] [FiniteDimensi
  {F : M → ℝ → ℕ} (hf : ContDiff ℝ n 1F) :
  ContDiff ℝ n (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume)) := by
  induction' n with n ih generalizing F
  · simp
  fun_prop
  · exact contDiff_succ_parametric_intervalIntegral_of_contDiff (hf.of_le (

```

1.35 Physlib/Meta/Basic.lean

```

/-- Gets all imports within Physlib and QuantumInfo.. -/
let PhysLibImports := physlibMod.imports.filter (fun c => c.module != `In

let mods := `QuantumInfo
let mFile ← find0Lean mods
unless ← mFile.pathExists do
  throw <| IO.userError s!"object file '{mFile}' of module {mods} does no
let (QIMod, _) ← readModuleData mFile
let QIImports := QIMod.imports.filter (fun c => c.module != `Init)
return (PhysLibImports ++ QIImports)

```

1.36 Physlib/Meta/TODO/Global.lean

TODO "Turn the `sorryful` linter on for the QuantumInfo module."

TODO "Refactor: Refactor the code in the QuantumInfo module to mirror the A of the rest of Physlib. For example, different key data structures should or directories. Results should be organized in a general way such that th generally."

1.37.

PHYSLIB/PARTICLES/BEYONDTHESTANDARDMODEL/RHN/ANOMALYCANCELLATION/BASIC.LEAN

1.37 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
def SMvCharges (n : ℕ) : ACCSystemCharges := ⟨6 * n⟩
def SMvSpecies (n : ℕ) : ACCSystemCharges := ⟨n⟩
  repeat rw [Finset.sum_add_distrib]
  repeat rw [map_smul]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [map_smul]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
  repeat rw [map_smul]
  rw [BiLinearSymm.mk₂_toFun_apply]
  change (quadBiLin S) S = _
  rw [quadBiLin_decomp]
  simp_all
  repeat rw [map_smul]
  repeat rw [map_add]
  rw [TriLinearSymm.mk₃_toFun_apply_apply]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
  repeat rw [accCube_decomp]
  simp_all
```

1.38 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
  toSpecies j (chargesMapOfSpeciesMap f S) = (LinearMap.comp f (toSpecies j)) S :=
toSMSpecies_toSpecies_inv _ _
  · rw [dif_pos hi, dif_pos hi, dif_pos hi]
  · rw [dif_neg hi, dif_neg hi, dif_neg hi]
  · rw [dif_pos hi, dif_pos hi]
  · rw [dif_neg hi, dif_neg hi]
  rw [familyUniversal, chargesMapOfSpeciesMap_toSpecies]
```

1.39 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
  repeat rw [toSMSpecies_toSpecies_inv]
  exact toSMSpecies_toSpecies_inv _ _
  toSpecies j (repCharges f S) = toSpecies j S ∘ f⁻¹ j :=
toSMSpecies_toSpecies_inv _ _
  rw [repCharges_toSpecies]
```

```
rw [← accQuad, accQuad.map smul]
```

```
def SMCharges (n : ℕ) : ACCSystemCharges := {5 * n}
def SMSpecies (n : ℕ) : ACCSpeciesCharges := {n}
  repeat rw [Finset.sum_add_distrib]
  repeat rw [map_smul]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
repeat rw [Finset.sum_add_distrib]
repeat rw [← Finset.mul_sum]
simp_all
  repeat rw [Finset.sum_add_distrib]
  repeat rw [map_smul]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
repeat rw [Finset.sum_add_distrib]
repeat rw [← Finset.mul_sum]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [map_smul]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
repeat rw [Finset.sum_add_distrib]
repeat rw [← Finset.mul_sum]
simp_all
  repeat rw [Finset.sum_add_distrib]
  repeat rw [map_smul]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
repeat rw [Finset.sum_add_distrib]
repeat rw [← Finset.mul_sum]
simp_all
  repeat rw [map_smul]
  repeat rw [map_add]
simp only [quadBiLin, BiLinearSymm.mk₂_toFun_apply]
repeat rw [Finset.sum_add_distrib]
repeat rw [← Finset.mul_sum]
simp_all
  repeat rw [map_smul]
```


1.42.

PHYSLIB/PARTICLES/STANDARDMODEL/ANOMALYANCELLATION/FAMILYMAPS.LEAN

```
repeat rw [map_add]
repeat rw [Finset.sum_add_distrib]
repeat rw [← Finset.mul_sum]
simp_all
```

1.42 Physlib/Particles/StandardModel/AnomalyCancellation/FamilyMaps.lean

- rw [dif_pos hi, dif_pos hi, dif_pos hi]
- rw [dif_neg hi, dif_neg hi, dif_neg hi]
- rw [dif_pos hi, dif_pos hi]
- rw [dif_neg hi, dif_neg hi]

1.43 Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/Lean

```
rw [hQ, E_zero_iff_Q_zero.mp hQ]
rw [hQ] at h1 h2
```

1.44 Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/Lean

```
rw [speciesVal, speciesVal]
repeat rw [speciesVal]
repeat rw [speciesVal]
  repeat rw [asLinear_val]
  repeat rw [speciesVal]
rw [speciesVal]
repeat rw [speciesVal]
lemma bijection_coe_val (S : linearParametersQENeqZero) :
  (bijection S).1.val = (bijectionLinearParameters S : linearParameters).asCharges

rw [bijection_coe_val, linearParameters.cubic]
rw [bijection_coe_val, linearParameters.grav]
```

1.45 Physlib/Particles/StandardModel/AnomalyCancellation/Permutations.lean

```
exact toSMSpecies_toSpecies_inv _ _
toSpecies j (repCharges f S) = toSpecies j S ∘ f-1 j :=
toSMSpecies_toSpecies_inv _ _
```

1.46 Physlib/Particles/StandardModel/HiggsBoson/Basic.lean

```
rw [dotProduct_smul, WithLp.ofLp_smul, smul_dotProduct, smul_smul, Un
one_smul]
```

1.47 Physlib/Particles/StandardModel/Representations.lean

```
rw [← star_def, (Unitary.mem_iff.mp g.prop).2]
```

1.48 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation

```
def MSSMCharges : ACCSystemCharges := {20}
def MSSMSpecies : ACCSystemCharges := {3}
  simp_rw [Hd.map_smul, Hu.map_smul]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
  simp_all
    repeat rw [Finset.sum_add_distrib]
    rw [Hd.map_smul, Hu.map_smul]
    repeat rw [Finset.sum_add_distrib]
    repeat rw [← Finset.mul_sum]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
  simp_all
    repeat rw [Finset.sum_add_distrib]
    repeat rw [Finset.sum_add_distrib]
    repeat rw [← Finset.mul_sum]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
  simp_all
    repeat rw [Finset.sum_add_distrib]
    rw [Hd.map_smul, Hu.map_smul]
    repeat rw [Finset.sum_add_distrib]
    repeat rw [← Finset.mul_sum]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
  simp_all
    repeat rw [map_smul]
    repeat rw [map_add]
  simp only [quadBiLin, BiLinearSymm.mk₂_toFun_apply]
  repeat rw [Finset.sum_add_distrib]
  repeat rw [← Finset.mul_sum]
  repeat rw [map_smul]
```

1.49.

PHYSLIB/PARTICLES/SUPERSYMMETRY/MSSMNU/ANOMALYANCELLATION/LINEY3B3.LEAN

```
repeat rw [map_add]
repeat rw [Finset.sum_add_distrib]
repeat rw [← Finset.mul_sum]
simp_all
lemma cubicACC_apply (S : MSSMACC.Charges) : MSSMACC.cubicACC S = cubeTriLin.toCubic
```

1.49 Physlib/Particles/SuperSymmetry/MSSMNU/AnomalyCancellation/LineY3B3

```
lemma lineY3B3Charges_val (a b : ℚ) :
  (lineY3B3Charges a b).val = a • Y3.1.1.val + b • B3.1.1.val := rfl

rw [← accQuad]
rw [quadSol Y3.1, quadSol B3.1]
repeat rw [← cubicACC_apply]
rw [Y3.cubicSol, B3.cubicSol]
lemma lineY3B3_val (a b : ℚ) : (lineY3B3 a b).val = a • Y3.val + b • B3.val := by
  simp [lineY3B3, lineY3B3Charges_val]
```

1.50 Physlib/Particles/SuperSymmetry/MSSMNU/AnomalyCancellation/Orthog

```
rw [← quadBiLin.toHomogeneousQuad_apply]
rw [← accQuad]
rw [quadSol T.1]
rw [← lineY3B3_val, lineY3B3_doublePoint, lineY3B3_val]
rw [← lineY3B3_val, lineY3B3_doublePoint, lineY3B3_val]
rw [← cubeTriLin.toCubic_apply]
rw [← cubicACC_apply]
rw [T.cubicSol]
```

1.51 Physlib/Particles/SuperSymmetry/MSSMNU/AnomalyCancellation/Orthog

```
rw [dot.map_add2, dot.map_add2] at h1 h2
rw [dot.map_add2 Y3.val (a' • Y3.val + b' • B3.val) (c' • R.val)] at h1
rw [dot.map_add2 B3.val (a' • Y3.val + b' • B3.val) (c' • R.val)] at h2
rw [← lineY3B3Charges_val, ← accQuad]
rw [lineY3B3Charges_quad]
rw [lineY3B3Charges_val, accQuad]
rw [accCube, TriLinearSymm.toCubic_add, ← accCube]
rw [← lineY3B3Charges_val]
rw [lineY3B3Charges_cubic]
```

```
rw [TriLinearSymm.map_smul₃, lineY₃B₃Charges_val, ← lineY₃B₃_val]
rw [lineY₃B₃_doublePoint]
rw [lineY₃B₃_val, accCube]
```

1.52 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation

```
(by rw [lineCube, planeY₃B₃_quad, R.prop.1, R.prop.2.1, R.prop.2.2]; si
```

1.53 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation

```
toSMSpecies j (chargeMap f S) = toSMSpecies j S ∘ f j :=
toSMSpecies_toSpecies_inv _ _
  exact toSMSpecies_toSpecies_inv _ _
toSMSpecies j (repCharges f S) = toSMSpecies j S ∘ f-1 j :=
toSMSpecies_toSpecies_inv _ _
```

1.54 Physlib/QFT/QED/AnomalyCancellation/Basic.lean

```
def PureU1Charges (n : ℕ) : ACCSystemCharges := ⟨n⟩
```

1.55 Physlib/QuantumMechanics/DDimensions/Basic.lean

```
/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
module

public import Physlib.QuantumMechanics.DDimensions.Operators.Momentum
public import Physlib.QuantumMechanics.DDimensions.Operators.Multiplication
/-!

# Single-particle quantum system on `Space d`

## i. Overview

In this module we introduce the general notion of a single-
particle quantum system on `Space d`.
```

The structure ``SpaceDQuantumSystem`` encompasses the basic information needed to specify a quantum system, namely the number of spatial dimensions, the particle's mass and the potential function.

ii. Key results

iii. Table of contents

- A. Basic properties
- B. Operators
 - B.1. Kinetic energy
 - B.2. Potential energy
 - B.3. Hamiltonian

iv. References

-/

@[expose] public section

namespace QuantumMechanics

open Complex

open MeasureTheory

open SpaceDHilbertSpace

/-- A single-particle quantum system with Hilbert space ``SpaceDHilbertSpace d``, characterized by the number of spatial dimensions `d : ℕ`, particle's mass `m > 0` and potential function `potential : Space d → ℝ`. -/

@[ext]

structure SpaceDQuantumSystem where

/-- The number of spatial dimensions. -/

d : ℕ

/-- The mass (positive). -/

m : ℝ

hm : 0 < m

/-- The potential function. -/

potential : Space d → ℝ

variable {Q : SpaceDQuantumSystem}

namespace SpaceDQuantumSystem

noncomputable section

/-- The Hilbert space, ``SpaceDHilbertSpace Q.d``. -/

abbrev HS := SpaceDHilbertSpace Q.d

```

/-!
## A. Basic properties
-/

@[simp]
lemma m_pos : 0 < Q.m := Q.hm

@[simp]
lemma m_nonneg : 0 ≤ Q.m := Q.hm.le

@[simp]
lemma m_ne_zero : Q.m ≠ 0 := Q.hm.ne'

/-!
## B. Operators
-/

section
open SchwartzMap
open LinearPMap

/-!
### B.1. Kinetic energy
-/

/-- The kinetic operator  $(2m)^{-1}\square^2$  as a continuous linear map on Schwartz
/
def kineticCLM :  $\square(\text{Space } Q.d, \mathbb{C}) \rightarrow L[\mathbb{C}] \square(\text{Space } Q.d, \mathbb{C}) := (2 * Q.m)^{-1} \cdot (\square$ 
lemma kineticCLM_eq : Q.kineticCLM =  $(2 * Q.m)^{-1} \cdot (\square \square_v \square) := rfl$ 

/-- The kinetic operator as an unbounded operator with domain `schwartzSubm
/
def kineticOperator : Q.HS  $\rightarrow_{\iota} L[\mathbb{C}]$  Q.HS := ofReal  $(2 * Q.m)^{-1} \cdot \text{momentumSqOp}$ 
lemma kineticOperator_eq : Q.kineticOperator = ofReal  $(2 * Q.m)^{-1} \cdot \text{momentu}$ 

/-!
### B.2. Potential energy
-/

/-- The potential operator as a continuous linear map on Schwartz space,
    where `Q.potential` is a function of temperate growth. -
/
def potentialCLM :  $\square(\text{Space } Q.d, \mathbb{C}) \rightarrow L[\mathbb{C}] \square(\text{Space } Q.d, \mathbb{C}) := \text{smulLeftCLM } \mathbb{C} ($ 

```

```

lemma potentialCLM_eq : Q.potentialCLM = smulLeftCLM ℂ (ofReal ∘ Q.potential) := rfl

lemma potentialCLM_apply (h_HTG : Q.potential.HasTemperateGrowth) (ψ : ⌊(Space Q.d,
  Q.potentialCLM ψ = fun x ↦ Q.potential x • ψ x := by
  rw [potentialCLM_eq, smulLeftCLM_apply (by fun_prop)]
  simp

@[simp]
lemma potentialCLM_apply_apply
  (h_HTG : Q.potential.HasTemperateGrowth) (ψ : ⌊(Space Q.d, ℂ)) (x : Space Q.d) :
  Q.potentialCLM ψ x = Q.potential x • ψ x := by
  rw [potentialCLM_apply h_HTG]

/-- The potential operator as a self-adjoint, unbounded multiplication operator
  with domain `{ψ ∈ Q.HS | Q.potential • ψ ∈ Q.HS}`. -/
def potentialOperator : Q.HS →ℓ.[ℂ] Q.HS := ⌊ (ofReal ∘ Q.potential)

lemma potentialOperator_eq : Q.potentialOperator = ⌊ (ofReal ∘ Q.potential) := rfl

lemma potentialOperator_isSelfAdjoint (h_AESM : AESTronglyMeasurable Q.potential) :
  IsSelfAdjoint Q.potentialOperator :=
  mulOperator_isSelfAdjoint_ofReal (by fun_prop) (by ext; simp)

lemma potentialOperator_domain_ge (h_HTG : Q.potential.HasTemperateGrowth) :
  schwartzSubmodule Q.d ≤ Q.potentialOperator.domain :=
  mulOperator_domain_ge_of_hasTemperateGrowth (by fun_prop)

/-!
### B.3. Hamiltonian
-/

/-- The Hamiltonian operator as a continuous linear map on Schwartz space,
  where `Q.potential` is a function of temperate growth. -
/
def hamiltonianCLM : ⌊(Space Q.d, ℂ) →L[ℂ] ⌊(Space Q.d, ℂ) := Q.kineticCLM + Q.poten

lemma hamiltonianCLM_eq : Q.hamiltonianCLM = Q.kineticCLM + Q.potentialCLM := rfl

/-- The Hamiltonian operator as a symmetric unbounded operator. -
/
def hamiltonianOperator : Q.HS →ℓ.[ℂ] Q.HS := Q.kineticOperator + Q.potentialOperator

lemma hamiltonianOperator_eq : Q.hamiltonianOperator = Q.kineticOperator + Q.potentialOperator := rfl

end

```

```

end
end SpaceDQuantumSystem
end QuantumMechanics

```

1.56 Physlib/QuantumMechanics/DDimensions/Hydrogen/Basic.lean

```

public import Physlib.QuantumMechanics.DDimensions.Basic
open MeasureTheory
structure HydrogenAtom extends SpaceDQuantumSystem where
  /-- Coefficient in the Coulomb potential (positive for attractive) -
  /
  coulomb_potential : potential = fun x ↦ -k * ||x||-1
  /-!
  ## A. Basic
  -/

lemma potential_eq : H.potential = fun x ↦ -H.k * ||x||-1 := H.coulomb_potent

@[fun_prop]
lemma potential_AESM : AESTronglyMeasurable H.potential := by
  rw [potential_eq]
  exact AEMeasurable.aestronglyMeasurable (by fun_prop)

@[fun_prop]
lemma potential_AEM : AEMeasurable H.potential := H.potential_AESM.aemeasur

/-!
  ## B. Regularization
  -/
def hamiltonianRegCLM (ε : ℝ×) : (Space H.d, ℂ) → L[ℂ] (Space H.d, ℂ) :=
lemma hamiltonianRegCLM_eq (ε : ℝ×) :
  H.hamiltonianRegCLM ε = (2 * H.m)-1 • (□ □v □) - H.k • □0 ε (-
1) := rfl

```

1.57 Physlib/QuantumMechanics/DDimensions/Hydrogen/LaplaceRunge

```

_ = (-s * I * ħ) • (□0 ε s - ε.1 ^ 2 • □0 ε (s-2)) := by simp [position
[H.lrlOperator ε i, H.lrlOperator ε j] = ((-2 * I * ħ * H.m) • H.hamilt
simp_rw [hamiltonianRegCLM_eq, smul_zero, add_zero, sub_zero, ← sub_smul,
positionSqCLM_eq ε, sub_comp, smul_comp, id_comp, smul_sub]
radiusRegPowCLM_comp_eq, comp_assoc, add_assoc, ← add_smul, ofReal_mul,

```


1.58.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/COMMUTATION. LEAN

```

    {H.hamiltonianRegCLM ε, H.lrlOperator ε i} = (I * ħ * H.k * ε.1 ^ 2) • 0 ε (-
3) • L 0 i
  · have h : H.m * H.k * (H.m-1 * 2-1) = 2-1 * H.k := by grind [H.m_ne_zero]
    simp only [hamiltonianRegCLM_eq, lrlOperator, lie_sub, sub_lie, smul_lie, lie_sm
    pSqr_comm_pL_Lp]
    simp only [dotProduct, mul_def, positionSqCLM_eq ε, comp_sub, comp_smul, comp_
    radiusRegPowCLM_comp_eq, smul_sub, ← Complex.coe_smul, ofReal_pow, smul_smul
    ← comp_finset_sum, ← comp_assoc, radiusRegPowCLM_comp_eq, positionSqCLM_eq ε]
    (2 * H.m) • (H.hamiltonianRegCLM ε) • L
  simp only [dotProduct, mul_def, ← neg_mul, smul_zero, add_zero, ← Complex.coe_smul
    ofReal_neg, smul_smul, zero_add, sub_eq_add_neg, ← neg_smul, smul_add, ofReal_po
    hamiltonianRegCLM_eq, ofReal_inv, ofReal_ofNat]
  grind [I_sq, H.m_ne_zero, mul_inv_cancel0, ofReal_eq_zero]

```

1.58 Physlib/QuantumMechanics/DDimensions/Operators/Commutation.lean

```

simp [bracket, mul_def, radiusRegPowCLM_comp_eq, add_comm]
simp only [momentumCLM_apply_fun, Space.deriv_const_smul _ (hdiff _),
  · simp only [positionCLM_apply, momentumCLM_apply, positionCLM_apply_fun]
simp only [momentumCLM_apply, positionCLM_apply, radiusRegPowCLM_apply_fun]
  simp_rw [positionSqCLM_eq ε, comp_sub, comp_smul, comp_id, radiusRegPowCLM_com

```

1.59 Physlib/QuantumMechanics/DDimensions/Operators/Covariance.lean

/-

Copyright (c) 2026 Axiomatic-AI. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Matteo Cipollina

-/

module

```

public import Physlib.QuantumMechanics.DDimensions.Operators.StateObservables.Expect
public import Physlib.QuantumMechanics.DDimensions.Operators.StateObservables.Varian
/-!

```

Covariance

i. Overview

In this module we define the covariance of two partial linear maps `A` and `B` in a
`ψ` as the real part of the inner product of their centered vectors.

ii. Key results

```

- `covariance` : the real part of the centered inner product.
- `covariance_comm` : covariance is symmetric in the two observables.
- `covariance_eq_re_symm_centered` : covariance as the real part of the sym
  inner product.
- `covariance_self_eq_variance` : the covariance of an observable with itse

## iii. Table of contents

- A. Covariance

## iv. References

- [B. C. Hall, *Quantum Theory for Mathematicians*, Chapter 12][hall2013qua
-/

@[expose] public section
namespace LinearPMap
open InnerProductSpace
noncomputable section
variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℂ H]

/-!

## A. Covariance

-/

section Covariance

variable (A B : H →ℂ ℂ H)
variable (ψ : A.domain)
variable (hψB : (ψ : H) ∈ B.domain)

/-- Covariance, defined as the real part of the centered inner product. -
/
def covariance : ℝ :=
  (⟦centered A ψ, centered B ⟨ψ, hψB⟩⟧_ℂ).re

/-- Covariance, unfolded to the real part of the centered inner product. -
/

```

1.59.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/COVARIANCE. LEAN

```
lemma covariance_eq_re_inner_centered :
  covariance A B  $\psi$  h $\psi$ B =
    ( $\langle$ centered A  $\psi$ , centered B  $\langle\psi$ , h $\psi$ B> $\rangle_{\mathbb{C}}$  $\rangle$ .re :=
  rfl

/-- Swapping the two observables does not change the covariance. -
/
lemma covariance_comm :
  covariance A B  $\psi$  h $\psi$ B = covariance B A  $\langle\psi$ , h $\psi$ B $\rangle \psi$ .2 := by
  rw [covariance_eq_re_inner_centered, covariance_eq_re_inner_centered]
  calc
    ( $\langle$ centered A  $\psi$ , centered B  $\langle\psi$ , h $\psi$ B> $\rangle_{\mathbb{C}}$  $\rangle$ .re =
      (((starRingEnd  $\mathbb{C}$ )  $\langle$ centered B  $\langle\psi$ , h $\psi$ B $\rangle$ , centered A  $\psi_{\mathbb{C}}$  $\rangle$ ).re := by
        rw [inner_conj_symm]
      _ = ( $\langle$ centered B  $\langle\psi$ , h $\psi$ B $\rangle$ , centered A  $\psi_{\mathbb{C}}$  $\rangle$ .re := by
        change (star  $\langle$ centered B  $\langle\psi$ , h $\psi$ B $\rangle$ , centered A  $\psi_{\mathbb{C}}$  $\rangle$ .re =
          ( $\langle$ centered B  $\langle\psi$ , h $\psi$ B $\rangle$ , centered A  $\psi_{\mathbb{C}}$  $\rangle$ .re
        rw [Complex.star_def, Complex.conj_re]

/-- Covariance as the real part of the symmetrized centered inner product. -
/
lemma covariance_eq_re_symm_centered :
  covariance A B  $\psi$  h $\psi$ B =
    (( $\langle$ centered A  $\psi$ , centered B  $\langle\psi$ , h $\psi$ B> $\rangle_{\mathbb{C}}$  +
       $\langle$ centered B  $\langle\psi$ , h $\psi$ B $\rangle$ , centered A  $\psi_{\mathbb{C}}$  $\rangle$ .re) / 2 := by
  let z :  $\mathbb{C}$  :=  $\langle$ centered A  $\psi$ , centered B  $\langle\psi$ , h $\psi$ B> $\rangle_{\mathbb{C}}$ 
  have hz :  $\langle$ centered B  $\langle\psi$ , h $\psi$ B $\rangle$ , centered A  $\psi_{\mathbb{C}}$  $\rangle$  = star z := by
    simp [z, inner_conj_symm]
  rw [covariance_eq_re_inner_centered, hz]
  change z.re = ((z + star z).re) / 2
  simp only [Complex.add_re, Complex.star_def, Complex.conj_re, add_self_div_two]

@[simp]
lemma covariance_self_eq_variance (A :  $H \rightarrow_{\mathbb{C}} H$ ) ( $\psi$  : A.domain) :
  covariance A A  $\psi$  (by exact  $\psi$ .2) = variance A  $\psi$  := by
  rw [covariance_eq_re_inner_centered, variance_eq_centered_norm_sq, inner_self_eq_n]
  rw [sq, sq, Complex.mul_re]
  simp [Complex.ofReal_re, Complex.ofReal_im]

end Covariance

end
end LinearPMap
```

1.60 Physlib/QuantumMechanics/DDimensions/Operators/Momentum.le

```

- `momentumCLM` : (components of) the momentum vector operator acting on SchwartzSubmodule d
- `momentumOperator` : a symmetric unbounded operator acting on the SchwartzSubmodule d
def momentumCLM :  $\Pi(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \Pi(\text{Space } d, \mathbb{C}) :=$ 
@[inherit_doc momentumCLM]
notation " $\Pi$ " => momentumCLM
@[inherit_doc momentumCLM]
notation " $\Pi[" d' "]"$ " => momentumCLM (d := d')
lemma momentumCLM_apply_fun ( $\psi : \Pi(\text{Space } d, \mathbb{C})$ ) :  $\Pi i \psi = (-$ 
 $I * \hbar) \cdot \partial[i] \psi := \text{rfl}$ 
lemma momentumCLM_apply ( $\psi : \Pi(\text{Space } d, \mathbb{C})$ ) (x : Space d) :  $\Pi i \psi x = -$ 
 $I * \hbar * \partial[i] \psi x :=$ 
open LinearPMap
open MeasureTheory
open SpaceDHilbertSpace
open SchwartzSubmodule
/-- The momentum operator as a LinearPMap with domain the Schwartz submodule d -/
def momentumOperator : SpaceDHilbertSpace d  $\rightarrow_{\perp} L[\mathbb{C}]$  SpaceDHilbertSpace d where
  domain := schwartzSubmodule d
  toFun := schwartzIncl.toLinearMap  $\circ_{\perp} (\Pi i).$ toLinearMap  $\circ_{\perp}$  schwartzEquiv.schwartzEquiv
@[inherit_doc momentumOperator]
notation " $\Pi$ " => momentumOperator

lemma momentumOperator_apply ( $\psi : \text{schwartzSubmodule } d$ ) :
   $\Pi i \psi = \text{schwartzEquiv } (\Pi i (\text{schwartzEquiv.symm } \psi)) := \text{rfl}$ 

lemma momentumOperator_apply_ae ( $\psi : \text{schwartzSubmodule } d$ ) :
   $\Pi i \psi =$ ["volume"]  $\Pi i (\text{schwartzEquiv.symm } \psi) :=$ 
  schwartzEquiv.coe_ae _

lemma momentumOperator_range ( $\psi : \text{schwartzSubmodule } d$ ) :  $\Pi i \psi \in \text{schwartzSubmodule } d$ 
  simp [momentumOperator_apply]

lemma momentumOperator_hasDenseDomain :  $(\Pi i).$ HasDenseDomain := SchwartzSubmodule.d
lemma momentumOperator_isSymmetric :  $(\Pi i).$ IsSymmetric := by
  intro  $\psi \phi$ 
  obtain (g, rfl) := schwartzEquiv.surjective  $\phi$ 
  simp only [momentumOperator_apply, ← Submodule.coe_inner, schwartzEquiv_i.coe_inner,
    schwartzEquiv.symm_apply_apply, momentumCLM_apply]
  have heq :  $\forall x, \text{fderiv } \mathbb{R} (\text{star} \circ f) x = (\text{starL}' \mathbb{R}).\text{toContinuousLinearMap} (f x)$ 
  fun _  $\mapsto$  fderiv_star
  have hI1 : Integrable fun x  $\mapsto$  star (f x) := (starL'  $\mathbb{R}$ ).integrable_comp_iff
  have hI2 : Integrable fun x  $\mapsto$  fderiv  $\mathbb{R}$  g x (basis i) * star (f x) := by

```

1.61.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/MULTIPLICATION.LEAN

```

    refine hI1.mul_of_top_right ?_
    exact ((g.fderivCLM ℂ _ _).evalCLM ℂ _ _).memLp_top
have hI3 : Integrable fun x ↦ g x * fderiv ℝ (star ∘ f) x (basis i) := by
  simp_rw [heq]
  refine Integrable.mul_of_top_right ?_ g.memLp_top
  apply (starL' ℝ).integrable_comp_iff.mpr
  exact ((f.fderivCLM ℂ _ _).evalCLM ℂ _ _).integrable
have hI4 : Integrable fun x ↦ g x * star (f x) := hI1.mul_of_top_right g.memLp_top
trans I * ħ * ∫ x, g x * fderiv ℝ (star ∘ f) x (basis i)
· simp_rw [← integral_const_mul_of_integrable hI3, heq]
  simp [mul_comm, mul_left_comm, Space.deriv_eq]
symm
trans I * ħ * -∫ x, fderiv ℝ ↑g x (basis i) * star (f x)
· rw [mul_neg, ← neg_mul, ← integral_const_mul_of_integrable hI2]
  simp [mul_left_comm, mul_comm, Space.deriv_eq]
symm
exact integral_mul_fderiv_eq_neg_fderiv_mul_of_integrable hI2 hI3 hI4 (by fun_prop

lemma momentumOperator_isUnbounded : (□ i).IsUnbounded := by
  refine (LinearPMap.IsSymmetric.isUnbounded_iff_hasDenseDomain ?_).mpr ?_
  · exact momentumOperator_isSymmetric i
  · exact momentumOperator_hasDenseDomain i

/-- The square of the momentum operator. -/
def momentumSqOperator : SpaceDHilbertSpace d →ℝ [ℂ] SpaceDHilbertSpace d :=
  sum fun i ↦ (□ i).comp (□ i) (momentumOperator_range i)

lemma momentumSqOperator_eq :
  momentumSqOperator (d := d) = sum fun i ↦ (□ i).comp (□ i) (momentumOperator_range i)

lemma momentumSqOperator_domain_eq : momentumSqOperator.domain = schwartzSubmodule d
  rw [momentumSqOperator_eq, sum_domain]
  rcases eq_zero_or_pos d with rfl | hd
  · simp [SchwartzSubmodule.zero_eq_top]
  · letI := Fin.pos_iff_nonempty.mp hd
    rw [← iInf_const (a := schwartzSubmodule d) (ι := Fin d)]
    congr

```

1.61 Physlib/QuantumMechanics/DDimensions/Operators/Multiplication.lean

/-

Copyright (c) 2026 Gregory J. Loges. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Gregory J. Loges

```
-/
module
```

```
public import Physlib.QuantumMechanics.DDimensions.Operators.Unbounded
public import Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.Schwa
/-!
```

```
# Multiplication operators on `SpaceDHilbertSpace`
```

```
## i. Overview
```

In this module we introduce unbounded operators defined by multiplication by $f : \text{Space } d \rightarrow \mathbb{C}$. The domain is defined to be as large as possible, namely $\psi \in \text{SpaceDHilbertSpace } d$ is in the domain iff $f \cdot \psi \in \text{SpaceDHilbertSpace } d$.

```
## ii. Key results
```

- ``mulOperator f`` : Given a function $f : \text{Space } d \rightarrow \mathbb{C}$, the operator defined by multiplication by f (with maximal domain) with notation $\square f$.
- ``mulOperator_adjoint_eq_conj`` : For a.e. strongly measurable f , $\square f$ is self-adjoint.
- ``mulOperator_isUnbounded`` : For a.e. strongly measurable f , $\square f$ is unbounded.
- ``mulOperator_compRestricted_le`` : The composition $\square f \circ_r \square g$ is contained in $\square (f \cdot g)$.
- ``mulOperator_compRestricted_eq`` : The composition $\square f \circ_r \square g$ is equal to $\square (f \cdot g)$ on $\square (f \cdot g).domain$.

```
## iii. Table of contents
```

- A. Definition
- B. Domain
- C. Adjoint
 - C.1. Self-adjoint
- D. Closable & unbounded
- E. Composition

```
## iv. References
```

See examples 1.3 and 3.8 in

- K. Schmüdgen, (2012). "Unbounded self-adjoint operators on Hilbert space" <https://doi.org/10.1007/978-94-007-4753-1>

```
-/
```

```
@[expose] public section
```

```
namespace QuantumMechanics
namespace SpaceDHilbertSpace
```

1.61.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/MULTIPLICATION.LEAN

noncomputable section

```
open LinearPMap
open MeasureTheory
open AEEqFun
open Filter
open ComplexConjugate
```

variable {d : ℕ}

```
/-!
## A. Definition
-/-
```

```
/-- The LinearPMap which maps  $\psi$  to  $f \cdot \psi$  with domain  $\{\psi \mid f \cdot \psi \in \text{SpaceDHilbertSpace } d\}$ 
-/
```

```
def mulOperator (f : Space d → ℂ) : SpaceDHilbertSpace d →ℂ [ℂ] SpaceDHilbertSpace d
  domain := {
    carrier := {ψ : SpaceDHilbertSpace d | MemHS (f • ψ.val.cast)}
    add_mem' := by
      intro ψ φ hψ hφ
      refine memHS_of_ae _ (memHS_add hψ hφ) ?_
      filter_upwards [coeFn_add ψ.val φ.val] with x h
      simp [mul_add, h]
    zero_mem' := memHS_of_ae 0 zero_memHS (by filter_upwards; simp)
    smul_mem' c ψ hψ := by
      refine memHS_of_ae _ (memHS_const_smul (c := c) hψ) ?_
      filter_upwards [coeFn_smul c ψ.val] with x h
      change _ = (f • (c • ψ.val).cast) x
      simp [h, mul_left_comm]
  }
  toFun := {
    toFun ψ := mk ψ.prop
    map_add' ψ φ := by
      rw [← mk_add, mk_eq_iff]
      filter_upwards [coeFn_add ψ.1.val φ.1.val] with x h
      simp [h, mul_add]
    map_smul' c ψ := by
      rw [← mk_const_smul, mk_eq_iff]
      filter_upwards [coeFn_smul c ψ.1.val] with x h
      change (f • (c • ψ.1.val).cast) x = _
      simp [h, mul_left_comm]
  }
```

```
@[inherit_doc mulOperator]
notation "[]" => mulOperator
```

```

lemma mem_mulOperator_domain_iff
  {f : Space d → ℂ} {ψ : SpaceDHilbertSpace d} : ψ ∈ (□ f).domain ↔ MemHS
  Iff.rfl

lemma mulOperator_apply_ae {f : Space d → ℂ} (ψ : (□ f).domain) : (□ f) ψ =
  coe_mk_ae ψ.prop

/-!
## B. Domain
-/

lemma mulOperator_hasDenseDomain {f : Space d → ℂ} (hf : AESTronglyMeasurab
  (□ f).HasDenseDomain := by
  intro ψ
  apply mem_closure_iff_seq_limit.mpr
  obtain ⟨u, hu, hfu⟩ := AESTronglyMeasurable.aemeasurable hf
  let s : ℕ → Set (Space d) := fun n ↦ u-1' (Metric.closedBall 0 n)
  let φ : ℕ → SpaceDHilbertSpace d := fun n ↦ mk (f := (s n).indicator ψ) <
  apply memHS_iff.mpr
  refine ⟨by measurability, by measurability, ?_⟩
  refine HasFiniteIntegral.mono (memHS_iff.mp (coe_hilbertSpace_memHS ψ))
  refine Eventually.of_forall (fun x ↦ ?_)
  by_cases hx : x ∈ s n <=> simp [hx]
  have hφ : ∀ n, φ n =[volume] (s n).indicator ψ := fun n ↦ coe_mk_ae _
  use φ
  constructor
  · intro n
    apply memHS_iff.mpr
    refine ⟨by measurability, by measurability, ?_⟩
    refine HasFiniteIntegral.mono (memHS_iff.mp (coe_hilbertSpace_memHS (n
      filter_upwards [hfu, coeFn_smul n (φ n).val, hφ n] with x h1 h2 h3
    by_cases hx : x ∈ s n
    · simp_rw [norm_pow, norm_norm, sq_le_sq, abs_norm]
      calc
        _ = ||u x|| * ||φ n x|| := by simp [h1]
        _ ≤ n * ||φ n x|| := mul_le_mul_of_nonneg_right (by simp_all [s]) (no
        _ = ||(n • φ n) x|| := by simp [h2]
    · simp [h3, hx]
  · apply tendsto_sub_nhds_zero_iff.mp
    apply tendsto_zero_iff_tendsto_zero_lintegral_enorm_sq.mpr
    have h : ∀ n, ∫- x, ||(φ n - ψ) x||e2 = ∫- x, ||(s n)c.indicator ψ x||e
    intro n
    refine lintegral_congr_ae ?_
    filter_upwards [coeFn_sub (φ n).val ψ.val, hφ n] with x h1 h2
    by_cases hx : x ∈ s n <=> simp [hx, h1, h2]

```


1.61.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/MULTIPLICATION. LEAN

```

simp_rw [h]
rw [← MeasureTheory.lintegral_zero (α := Space d) (μ := volume)]
refine tendsto_lintegral_of_dominated_convergence' (fun x ↦ ‖ψ x‖e ^ 2) ?_ ?_ ?_
· measurability
· intro n
  filter_upwards with x
  by_cases hx : x ∈ s n <=> simp [hx]
· have : ∫- x, ‖ψ x‖e ^ 2 ‖ψ x‖e ≠ τ := (memHS_iff.mp (coe_hilbertSpace_memHS ψ)).2.2
  simp_all
· filter_upwards with x
  rw [← zero_pow_two_ne_zero, ← enorm_zero (E := ℂ)]
  refine ENNReal.Tendsto.pow ?_
  refine Tendsto.enorm ?_
  refine tendsto_nhds_of_eventually_eq ?_
  apply eventually_atTop.mpr
  use [‖u x‖].toNat
  intro n hn
  suffices ‖u x‖ ≤ n by simp [s, this]
  calc
    _ ≤ (‖u x‖ : ℝ) := Int.le_ceil _
    _ ≤ [‖u x‖].toNat := Int.cast_le.mpr (Int.self_le_toNat _)
    _ ≤ n := Nat.cast_le.mpr hn

open SchwartzMap SchwartzSubmodule in
lemma mulOperator_domain_ge_of_hasTemperateGrowth
  {f : Space d → ℂ} (hf : f.HasTemperateGrowth) : schwartzSubmodule d ≤ (□ f).domain
intro ψ hψ
obtain ⟨g, hg⟩ := schwartzEquiv.surjective ⟨ψ, hψ⟩
let w : □(Space d, ℂ) := smulLeftCLM ℂ f g
let φ : SpaceDHilbertSpace d := schwartzEquiv w
apply mem_mulOperator_domain_iff.mpr
refine memHS_of_ae φ (coe_hilbertSpace_memHS φ) ?_
filter_upwards [schwartzEquiv_coe_ae w, schwartzEquiv_coe_ae g] with x h1 h2
simp [w, φ, h1, ← h2, hg, smulLeftCLM_apply_apply hf]

/-!
## C. Adjoint
-/

-- Can the AESTronglyMeasurable hypothesis be removed?
lemma mulOperator_conj_domain {f : Space d → ℂ} (hf : AESTronglyMeasurable f) :
  (□ (conj ∘ f)).domain = (□ f).domain := by
  ext
  simp only [mulOperator, smul_eq_mul, memHS_iff]
  exact and_congr (iff_of_true (by fun_prop) (by fun_prop)) (by simp)

```

```

private lemma exists_monotone_sets_hasFiniteIntegral
  (f g : Space d → ℂ) (hf : AESTronglyMeasurable f) (hg : AESTronglyMeasurable g)
  ∃ s : ℕ → Set (Space d), Monotone s ∧  $\bigcap$  n, s n = Set.univ ∧ (∀ n, Measurable (s n))
  ∧ ∀ k, k = 1 ∨ k = 2 →
    ∀ n, HasFiniteIntegral (fun x ↦ ||f x ^ k * g x|| ^ 2) (volume.restrict (s n))
obtain ⟨w₁, hw₁, hw₁'⟩ : AESTronglyMeasurable (fun x ↦ f x * g x) := by m
obtain ⟨w₂, hw₂, hw₂'⟩ : AESTronglyMeasurable (fun x ↦ f x ^ 2 * g x) := by m
let s : ℕ → Set (Space d) :=
  fun n ↦ Metric.closedBall 0 n n (w₁ ^ -1' Metric.closedBall 0 n n (w₂ ^ -1'))
refine ⟨s, ?_, ?_, by measurability, ?_⟩
· exact fun _ _ hmn _ hx ↦
  (Metric.closedBall_subset_closedBall (Nat.cast_le.mpr hmn) hx.1,
   Metric.closedBall_subset_closedBall (Nat.cast_le.mpr hmn) hx.2.1,
   Metric.closedBall_subset_closedBall (Nat.cast_le.mpr hmn) hx.2.2)
· ext x
  simp only [Set.mem_iUnion, Set.mem_univ, iff_true]
  use max [||x||.toNat (max [||w₁ x||.toNat [||w₂ x||.toNat]
suffices ∀ r : ℝ, 0 ≤ r → r ≤ [r].toNat by simp [s, this]
intro r hr
calc
  r ≤ [r] := Int.le_ceil r
  _ = ([r].toNat : ℤ) := by simp [Int.ceil_nonneg hr]
  _ = [r].toNat := AddGroupWithOne.intCast_ofNat _
· intro k hk n
  refine lt_of_le_of_lt (b := ||(n : ℝ) ^ 2||ₑ * volume (s n)) ?_ ?_
· rw [← setLIntegral_const]
  refine setLIntegral_mono_ae' (by measurability) ?_
  filter_upwards [hw₁', hw₂'] with x h₁ h₂ ⟨h₃, h₃'⟩
  apply enorm_le_iff_norm_le.mpr
  simp_rw [norm_pow, norm_norm, RCLike.norm_natCast]
  refine pow_le_pow_left₀ (norm_nonneg _) ?_ 2
  rcases hk <=> simp_all
· refine ENNReal.mul_lt_top (by norm_num) ?_
  exact measure_inter_lt_top_of_left_ne_top measure_closedBall_lt_top.n

open Complex InnerProductSpace in
lemma mulOperator_adjoint_domain_le {f : Space d → ℂ} (hf : AESTronglyMeasurable f)
  (f)†.domain ≤ ((conj ∘ f)).domain := by
  intro ψ hψ
  let ξ : SpaceDHilbertSpace d := (f)† ⟨ψ, hψ⟩
  obtain ⟨s, hs_mono, hs_univ, hs_meas, hs_int⟩ :=
    exists_monotone_sets_hasFiniteIntegral (conj ∘ f) ψ (by fun_prop) ψ.val
  let w : ℕ → Space d → ℂ := fun n ↦ (s n).indicator ((conj ∘ f) • ψ)
  have hw : ∀ n, MemHS (w n) := by
    intro n
    refine memHS_iff.mpr ⟨by measurability, by measurability, ?_⟩

```

1.61.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/MULTIPLICATION. LEAN

```

    refine lt_of_eq_of_lt ?_ (hs_int 1 (Or.inl rfl) n)
    trans ∫⁻ x in s n, |||w n x|| ^ 2||_e
    · exact (setLIntegral_eq_of_support_subset fun x hx ↦ by simp_all [w]).symm
    exact setLIntegral_congr_fun (hs_meas n) fun x hx ↦ by simp [w, hx, mul_pow]
    let φ : ℕ → SpaceDHilbertSpace d := fun n ↦ mk (hw n)
    have hφ : ∀ n, φ n ∈ (□ f).domain := by
      intro n
      apply memHS_iff.mpr ⟨by measurability, by measurability, ?_⟩
      refine lt_of_eq_of_lt ?_ (hs_int 2 (Or.inr rfl) n)
      calc
        _ = ∫⁻ x, |||(f • w n) x|| ^ 2||_e := by
          refine lintegral_congr_ae ?_
          filter_upwards [coe_mk_ae (hw n)] with _ h
          simp [φ, h]
        _ = ∫⁻ x in s n, |||(f • w n) x|| ^ 2||_e :=
          (setLIntegral_eq_of_support_subset fun x hx ↦ by simp_all [w]).symm
          exact setLIntegral_congr_fun (hs_meas n) fun x hx ↦ by simp [w, hx, ← mul_assoc,
suffices ∀ n, ∫⁻ x in s n, |||f x|| ^ 2 * ||ψ x|| ^ 2||_e ≤ ∫⁻ x, |||ξ x|| ^ 2||_e by
          apply mem_mulOperator_domain_iff.mpr
          refine memHS_iff.mpr ⟨by measurability, by measurability, ?_⟩
          refine lt_of_le_of_lt ?_ (memHS_iff.mp <| coe_hilbertSpace_memHS ξ).2.2
          trans □ n, ∫⁻ x in s n, |||f x|| ^ 2 * ||ψ x|| ^ 2||_e
          · rw [← setLIntegral_univ, ← hs_univ]
            rw [setLIntegral_iUnion_of_directed _ (directed_of_isDirected_le hs_mono)]
            simp [mul_pow]
            exact iSup_le this
        intro n
        suffices ||φ n|| ^ 2 ≤ ||ξ|| ^ 2 by
          refine le_of_eq_of_le (b := ∫⁻ x, |||φ n x|| ^ 2||_e) ?_ <| (ENNReal.toReal_le_toReal
          · calc
            _ = ∫⁻ x in s n, |||w n x|| ^ 2||_e :=
              setLIntegral_congr_fun (hs_meas n) fun x hx ↦ by simp [w, hx, mul_pow]
            _ = ∫⁻ x, |||w n x|| ^ 2||_e :=
              setLIntegral_eq_of_support_subset fun x hx ↦ by simp_all [w]
            _ = ∫⁻ x, |||φ n x|| ^ 2||_e := by
              refine lintegral_congr_ae ?_
              filter_upwards [coe_mk_ae (hw n)] with x h₁
              simp [φ, h₁]
            · exact (memHS_iff.mp <| coe_hilbertSpace_memHS (φ n)).2.2.ne
            · exact (memHS_iff.mp <| coe_hilbertSpace_memHS ξ).2.2.ne
            · suffices h : ∀ ψ : SpaceDHilbertSpace d, ||ψ|| ^ 2 = (∫⁻ x, |||ψ x|| ^ 2||_e).toReal
              simp only [← h, this]
              intro ψ
              rw [Lp.norm_def, eLpNorm_eq_lintegral_rpow_enorm_toReal two_ne_zero ENNReal.of
              simp [← ENNReal.toReal_pow, ← ENNReal.rpow_mul_natCast]
            suffices (||φ n|| ^ 2) ^ 2 ≤ (||ξ|| * ||φ n||) ^ 2 by

```

```

by_cases! h : ||φ n|| = 0
· rw [h, zero_pow_two_ne_zero]
  exact pow_two_nonneg _
· rw [pow_two, mul_pow] at this
  refine (mul_le_mul_iff_left₀ <| sq_pos_iff.mpr h).mpr this
calc
_ = ||φ n, φ n||_ℂ ^ 2 := by simp
_ = ||φ, λ f ⟨φ n, hφ n⟩||_ℂ ^ 2 := by
  refine congrArg (fun r ↦ ||r|| ^ 2) ?_
  refine integral_congr_ae ?_
  filter_upwards [coe_mk_ae (hw n), mulOperator_apply_ae ⟨φ n, hφ n⟩] w
by_cases hx : x ∈ s n
· simp only [φ, h₁, h₂, inner_self_eq_norm_sq_to_K, coe_algebraMap, R
  Pi.smul_apply', smul_eq_mul]
  calc
  _ = ofReal (||f x|| ^ 2 * ||ψ x|| ^ 2) := by simp [w, hx, mul_pow]
  _ = (f x * conj (f x)) * (ψ x * conj (ψ x)) := by
    simp_rw [← normSq_eq_norm_sq, Complex.ofReal_mul, normSq_eq_conj]
  _ = f x * w n x * conj (ψ x) := by simp [w, hx, mul_assoc]
· simp [φ, h₁, h₂, w, hx]
_ = ||ξ, φ n||_ℂ ^ 2 := by
  rw [(adjoint_isFormalAdjoint (mulOperator_hasDenseDomain hf) ⟨ψ, hψ⟩)
  _ ≤ (||ξ|| * ||φ n||) ^ 2 := pow_le_pow_left₀ (norm_nonneg _) (norm_inner_l

lemma mulOperator_adjoint_eq_conj {f : Space d → ℂ} (hf : AESTronglyMeasurable
  (λ f)† = λ (conj ∘ f) := by
  have hFA : (λ f).IsFormalAdjoint (λ (conj ∘ f)) := by
    intro ψ φ
    refine integral_congr_ae ?_
    filter_upwards [mulOperator_apply_ae ψ, mulOperator_apply_ae φ] with x
    simp [h₁, h₂, mul_assoc, mul_left_comm]
  refine eq_of_le_of_ge ?_ (hFA.le_adjoint <| mulOperator_hasDenseDomain hf)
  refine ⟨mulOperator_adjoint_domain_le hf, fun ψ ψ' hψ ↦ ?_⟩
  refine adjoint_apply_eq (mulOperator_hasDenseDomain hf) ψ fun φ ↦ ?_
  rw [← inner_conj_symm, hψ, (hFA φ ψ').symm, inner_conj_symm]

/-!
### C.1. Self-adjoint
-/

lemma mulOperator_isSelfAdjoint_ofReal
  {f : Space d → ℂ} (hf : AESTronglyMeasurable f) (hf' : conj ∘ f = f) :
  IsSelfAdjoint (λ f) := by
  apply isSelfAdjoint_def.mpr
  rw [mulOperator_adjoint_eq_conj hf, hf']

```

1.61.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/MULTIPLICATION.LEAN

/-!

D. Closable & unbounded

-/

```
lemma mulOperator_isClosable {f : Space d → ℂ} (hf : AESTronglyMeasurable f) :  
  (□ f).IsClosable := by  
  refine isClosable_of_exists_dense_formalAdjoint ?_ ?_  
  · exact mulOperator_hasDenseDomain hf  
  · refine (□ (conj ∘ f), ?_, ?_)  
    · exact mulOperator_hasDenseDomain (by measurability)  
    · rw [← mulOperator_adjoint_eq_conj hf]  
      exact adjoint_isFormalAdjoint (mulOperator_hasDenseDomain hf)
```

```
lemma mulOperator_isUnbounded {f : Space d → ℂ} (hf : AESTronglyMeasurable f) :  
  (□ f).IsUnbounded :=  
  (mulOperator_hasDenseDomain hf, mulOperator_isClosable hf)
```

/-!

E. Composition

-/

```
lemma mulOperator_compRestricted_le (f g : Space d → ℂ) : □ f ∘r □ g ≤ □ (f • g) :=  
  constructor  
  · intro ψ hψ  
    obtain ⟨hψ, hgψ⟩ := mem_compRestricted_domain_iff.mp hψ  
    apply mem_mulOperator_domain_iff.mpr  
    refine memHS_of_ae _ (mem_mulOperator_domain_iff.mp hgψ) ?_  
    filter_upwards [mulOperator_apply_ae ⟨ψ, hψ⟩]  
    simp_all [mul_assoc]  
  · intro ψ φ hψφ  
    apply ext_iff.mpr  
    obtain ⟨hψ, hgψ⟩ := mem_compRestricted_domain_iff.mp ψ.2  
    filter_upwards [mulOperator_apply_ae φ, mulOperator_apply_ae ⟨ψ, hψ⟩,  
      mulOperator_apply_ae ⟨□ g ⟨ψ, hψ⟩, hgψ⟩]  
    simp_all [mul_assoc]
```

```
lemma mulOperator_compRestricted_eq (f : Space d → ℂ) {g : Space d → ℂ} (h : (□ g).d  
  □ f ∘r □ g = □ (f • g) := by  
  have hle := mulOperator_compRestricted_le f g  
  refine eq_of_le_of_domain_eq hle ?_  
  refine eq_of_le_of_ge hle.1 fun ψ hψ ↦ ?_  
  apply mem_compRestricted_domain_iff.mpr  
  use h ▸ Submodule.mem_top  
  apply mem_mulOperator_domain_iff.mpr  
  refine memHS_of_ae _ (mem_mulOperator_domain_iff.mp hψ) ?_  
  filter_upwards [mulOperator_apply_ae ⟨ψ, h ▸ Submodule.mem_top⟩]
```

```

    simp_all [mul_assoc]

end
end SpaceDHilbertSpace
end QuantumMechanics

```

1.62 Physlib/QuantumMechanics/DDimensions/Operators/Position.le

```

public import Physlib.QuantumMechanics.DDimensions.Operators.Multiplication
public import Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.PolyB
- `positionCLM` : (components of) the position vector operator acting on Sc
- `radiusRegPowCLM` : operator acting on Schwartz maps by multiplication by
- `positionOperator` : a self-adjoint multiplication operator acting on `Sp
- `radiusRegPowOperator` : a self-adjoint multiplication operator acting o
- `[]` for `positionCLM`
- `[]_0` for `radiusRegPowCLM`
- `[]` for `radiusPowLM`
- B.3. Radius powers
  - B.3.1. As limit of regularized operators
open Filter
open MeasureTheory
open SchwartzMap
open SpaceDHilbertSpace
open SchwartzSubmodule PolyBddSchwartzSubmodule

def positionCLM :  $\Pi$ (Space d,  $\mathbb{C}$ )  $\rightarrow$  L[ $\mathbb{C}$ ]  $\Pi$ (Space d,  $\mathbb{C}$ ) :=
@[inherit_doc positionCLM]
notation "[]" => positionCLM
@[inherit_doc positionCLM]
notation "[]"[" d' "] => positionCLM (d := d')
lemma positionCLM_apply_fun ( $\psi$  :  $\Pi$ (Space d,  $\mathbb{C}$ )) :  $\Pi$  i  $\psi$  = (fun x : Space d
  simp [positionCLM, coordCLM_apply, coord_apply,
lemma positionCLM_apply ( $\psi$  :  $\Pi$ (Space d,  $\mathbb{C}$ )) (x : Space d) :  $\Pi$  i  $\psi$  x = x i *
  simp [positionCLM_apply_fun]
lemma normRegularizedPow_eq (d :  $\mathbb{N}$ ) ( $\epsilon$  s :  $\mathbb{R}$ ) :
  normRegularizedPow d  $\epsilon$  s = fun x  $\mapsto$  ( $\|x\|^2 + \epsilon^2$ ) ^ (s / 2) := rfl

@[fun_prop]
lemma normRegularizedPow_measurable (d :  $\mathbb{N}$ ) ( $\epsilon$  s :  $\mathbb{R}$ ) :
  Measurable (normRegularizedPow d  $\epsilon$  s) := by
  rw [normRegularizedPow_eq]
  fun_prop

def radiusRegPowCLM {d :  $\mathbb{N}$ } ( $\epsilon$  :  $\mathbb{R}^*$ ) (s :  $\mathbb{R}$ ) :  $\Pi$ (Space d,  $\mathbb{C}$ )  $\rightarrow$  L[ $\mathbb{C}$ ]  $\Pi$ (Space

```

1.62.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/POSITION21 LEAN

```

@[inherit_doc radiusRegPowCLM]
notation "□₀" => radiusRegPowCLM
@[inherit_doc radiusRegPowCLM]
notation "□₀[" d' "]" => radiusRegPowCLM (d := d')
lemma radiusRegPowCLM_apply_fun {d : ℕ} (ε : ℝ⁺) (s : ℝ) (ψ : □(Space d, ℂ)) :
  dsimp [radiusRegPowCLM]
lemma radiusRegPowCLM_apply {d : ℕ} (ε : ℝ⁺) (s : ℝ) (ψ : □(Space d, ℂ)) (x : Space
  rw [radiusRegPowCLM_apply_fun]
lemma radiusRegPowCLM_comp_eq {d : ℕ} (ε : ℝ⁺) (s t : ℝ) :
lemma radiusRegPowCLM_zero {d : ℕ} (ε : ℝ⁺) :
lemma positionSqCLM_eq {d : ℕ} (ε : ℝ⁺) :
def radiusPowLM {d : ℕ} (s : ℝ) : □(Space d, ℂ) →1[ℂ] Space d → ℂ where
@[inherit_doc radiusPowLM]
notation "□" => radiusPowLM

@[inherit_doc radiusPowLM]
notation "□[" d' "]" => radiusPowLM (d := d')
lemma radiusPowLM_apply_fun {d : ℕ} (s : ℝ) (ψ : □(Space d, ℂ)) :
lemma radiusPowLM_apply {d : ℕ} (s : ℝ) (ψ : □(Space d, ℂ)) (x : Space d) :
  rw [radiusPowLM_apply_fun]
lemma radiusPowLM_apply_contDiffAt {d : ℕ} (s : ℝ) (n : ℕ∞) (ψ : □(Space d, ℂ)) {x :
lemma radiusPowLM_apply_stronglyMeasurable {d : ℕ} (s : ℝ) (ψ : □(Space d, ℂ)) :
  rw [radiusPowLM_apply_fun]
lemma radiusPowLM_apply_memHS {d : ℕ} (s : ℝ) (ψ : □(Space d, ℂ)) (a : ℕ)
  (hψ : ψ ∈ polyBddSchwartzMap d a) (h : 0 < d + 2 * (a + s)) :
  · have : Nontrivial (Space d) := Nat.succ_pred_eq_of_pos hd ▸ Space.instNontrivial
    refine (memLp_two_iff_integrable_sq_norm (by fun_prop)).mpr (by fun_prop, ?_)
    suffices ∫- (x : Space d), |||ψ x||2 * ||x||(2 * s)||e < τ by
      simp [radiusPowLM, mul_pow, mul_comm, Real.rpow_mul]
    · -- `||x|| < 1` : bound `||ψ x||` by `||x||a`
      obtain ⟨C, hC_pos, hC⟩ := hψ a (le_refl _)
      suffices hBound : ∀m x, |||ψ x||2 * ||x||(2 * s)||e ≤ ||C2||e * |||x||(2 *
        _ ≤ ∫- (x : Space d) in (Metric.ball 0 1), ||C2||e * |||x||(2 * (a + s))
          setLIntegral_mono_ae' measurableSet_ball (Eventually.mono hBound fun _ h
            _ = ||C2||e * ∫- (x : Space d) in (Metric.ball 0 1), |||x||(2 * (a + s))
      apply ae_iff.mpr
      refine measure_mono_null ?_ (measure_singleton 0)
      intro x hx
      by_contra hx'
      apply hx
      apply norm_pos_iff.mpr at hx'
      simp_rw [← enorm_mul, enorm_le_iff_norm_le, mul_add, Real.rpow_add hx', norm_m
      refine mul_le_mul_of_nonneg_right ?_ (norm_nonneg _)
      simp_rw [← Nat.cast_two (R := ℝ), mul_comm, Real.rpow_mul_natCast hx'.le, norm
        norm_norm, Real.norm_eq_abs, abs_of_pos hC_pos, abs_of_nonneg (Real.rpow_non
      apply (sq_le_sq₀ (norm_nonneg _) (by positivity)).mpr

```

```

    apply (inv_mul_le_iff_0' (by positivity)).mp
    rw [← Real.rpow_neg_one, ← Real.rpow_mul hx'.le, mul_comm _ (-
1), neg_mul, one_mul,
      Real.rpow_neg_natCast]
    exact hC x
    simp only [radiusRegPowCLM_apply, radiusPowLM_apply, Complex.real_smul, h
      · simp only [radiusRegPowCLM_apply, radiusPowLM_apply, Complex.real_smu
/- The operator on `SpaceDHilbertSpace d` acting by multiplication by `fun
/
def positionOperator : SpaceDHilbertSpace d →1.[ $\mathbb{C}$ ] SpaceDHilbertSpace d :=
  (Complex.ofRealCLM ∘ L Space.coordCLM i)
@[inherit_doc positionOperator]
notation "[i]" => positionOperator

lemma positionOperator_hasDenseDomain : ([i]).HasDenseDomain :=
  mulOperator_hasDenseDomain (by fun_prop)

lemma positionOperator_isSelfAdjoint : IsSelfAdjoint ([i]) :=
  mulOperator_isSelfAdjoint_ofReal (by fun_prop) (by ext; simp)

lemma positionOperator_isUnbounded : ([i]).IsUnbounded :=
  LinearPMap.IsSelfAdjoint.isUnbounded (positionOperator_isSelfAdjoint i)
/- The operator on `SpaceDHilbertSpace d` acting by multiplication by
  `fun x ↦ (||x||2 +  $\varepsilon^2$ )(s/2)` -/
def radiusRegPowOperator ( $\varepsilon : \mathbb{R}^{\times}$ ) (s :  $\mathbb{R}$ ) : SpaceDHilbertSpace d →1.[ $\mathbb{C}$ ] Spa
  (Complex.ofReal ∘ normRegularizedPow d  $\varepsilon$  s)
@[inherit_doc radiusRegPowOperator]
notation "[ $\varepsilon$ ]" => radiusRegPowOperator

@[inherit_doc radiusRegPowOperator]
notation "[ $\varepsilon$ [" d' "]" => radiusRegPowOperator (d := d')

lemma radiusRegPowOperator_hasDenseDomain ( $\varepsilon : \mathbb{R}^{\times}$ ) (s :  $\mathbb{R}$ ) : ([ $\varepsilon$ ][d]  $\varepsilon$  s).Ha
  mulOperator_hasDenseDomain (by fun_prop)

lemma radiusRegPowOperator_isSelfAdjoint ( $\varepsilon : \mathbb{R}^{\times}$ ) (s :  $\mathbb{R}$ ) : IsSelfAdjoint (
  refine mulOperator_isSelfAdjoint_ofReal (by fun_prop) (by ext; simp)

lemma radiusRegPowOperator_isUnbounded ( $\varepsilon : \mathbb{R}^{\times}$ ) (s :  $\mathbb{R}$ ) : ([ $\varepsilon$ ][d]  $\varepsilon$  s).IsUnb
  LinearPMap.IsSelfAdjoint.isUnbounded (radiusRegPowOperator_isSelfAdjoint

/-!
### B.3. Radius powers
-/

/- The operator on `SpaceDHilbertSpace d` acting by multiplication by `fun

```


1.62.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/POSITION3.LEAN

```

/
def radiusPowOperator (s : ℝ) : SpaceDHilbertSpace d →1.[ℂ] SpaceDHilbertSpace d :=
  λ (Complex.ofReal ∘ fun x ↦ ||x|| ^ s)

@[inherit_doc radiusPowOperator]
notation "[]" => radiusPowOperator

@[inherit_doc radiusPowOperator]
notation "[[" d' "]" => radiusPowOperator (d := d')

lemma radiusPowOperator_hasDenseDomain (s : ℝ) : ([["d] s).HasDenseDomain := by
  refine mulOperator_hasDenseDomain ?_
  suffices (fun x ↦ ||x|| ^ s) = normRegularizedPow d 0 s by rw[this]; fun_prop
  ext x
  simp [normRegularizedPow, ← Real.rpow_natCast_mul (norm_nonneg x), mul_div_cancel]

lemma radiusPowOperator_isSelfAdjoint (s : ℝ) : IsSelfAdjoint ([["d] s) := by
  refine mulOperator_isSelfAdjoint_ofReal ?_ (by ext; simp)
  suffices (fun x ↦ ||x|| ^ s) = normRegularizedPow d 0 s by rw[this]; fun_prop
  ext x
  simp [normRegularizedPow, ← Real.rpow_natCast_mul (norm_nonneg x), mul_div_cancel]

lemma radiusPowOperator_isUnbounded (s : ℝ) : ([["d] s).IsUnbounded :=
  LinearPMap.IsSelfAdjoint.isUnbounded (radiusPowOperator_isSelfAdjoint s)

open Complex

private lemma add_floor_toNat_pos_aux (d : ℕ) (s : ℝ) :
  0 < d + 2 * ([1 - d / 2 - s].toNat + s) := by
  let n : ℤ := [1 - d / 2 - s]
  have hn1 : 1 - d / 2 - s < n + 1 := Int.lt_floor_add_one _
  have hn2 : (n : ℝ) ≤ n.toNat := Int.cast_le.mpr (Int.self_le_toNat _)
  linarith

lemma radiusPowLM_apply_polyBddSchwartz_memHS {d : ℕ} {s : ℝ}
  (ψ : polyBddSchwartzSubmodule d [1 - d / 2 - s].toNat) :
  MemHS ([["d] s (polyBddSchwartzEquiv.symm ψ)) :=
  let f := polyBddSchwartzEquiv.symm ψ
  radiusPowLM_apply_memHS s f.1 [1 - d / 2 - s].toNat f.2 (add_floor_toNat_pos_aux d s)

lemma radiusPowOperator_domain_ge {d : ℕ} (s : ℝ) :
  polyBddSchwartzSubmodule d [1 - d / 2 - s].toNat ≤ (radiusPowOperator s).domain
  intro ψ hψ
  let f := polyBddSchwartzEquiv.symm ⟨ψ, hψ⟩
  apply mem_mulOperator_domain_iff.mpr
  refine memHS_of_ae (λ s f.1) ?_ ?_

```

```

· exact radiusPowLM_apply_memHS s f.1 _ f.2 (add_floor_toNat_pos_aux d s)
· filter_upwards [polyBddSchwartzEquiv_coe_ae f]
  simp_all [f]

```

1.63 Physlib/QuantumMechanics/DDimensions/Operators/StateObserv

```

/-
Copyright (c) 2026 Axiomatic-AI. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Matteo Cipollina, Krystian Nowakowski
-/
module

public import Physlib.QuantumMechanics.DDimensions.Operators.Unbounded
/-!
# Expectation values and centered vectors

For a partial linear map `T` on a complex inner product space and ` $\psi \in T.do$ 
defines the expectation value and the centered vector ` $T\psi - \langle T \rangle_\psi \psi``.

## Main definitions

- ` $\text{LinearPMap.expectedValue}$ `: the real part of ` $\langle \psi, T\psi \rangle_{\mathbb{C}}$ `.
- ` $\text{LinearPMap.centered}$ `: the centered vector ` $T\psi - \langle T \rangle_\psi \psi``.

## Main statements

- ` $\text{LinearPMap.expectedValue\_eq\_inner}$ `: for symmetric `T`, the complex inner
  equals the real expectation value coerced to ` $\mathbb{C}$ `.

## References

- [B. C. Hall, *Quantum Theory for Mathematicians*, Chapter 12][hall2013qua]
-/

@[expose] public section

namespace LinearPMap

open InnerProductSpace

noncomputable section$$ 
```

1.63.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/STATEOBSERVABLES/EXPECTEDVALUE.LEAN

```
variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℂ H]
```

```
private lemma conj_inner_apply_self_eq_of_isSymmetric (T : H →ℂ [ℂ] H) (hT : T.IsSymmetric)
  (ψ : T.domain) :
  (starRingEnd ℂ) (⟨ψ : H⟩, T ψ) = ⟨ψ : H⟩, T ψ := by
  simpa [inner_conj_symm] using hT ψ ψ
```

```
/-- Expectation value `re ⟨ψ, Tψ⟩` for `ψ ∈ T.domain`.
```

```
For symmetric `T`, this agrees with `⟨ψ, Tψ⟩` after coercion from `ℝ`;
see `expectedValue_eq_inner`. -/
```

```
def expectedValue (T : H →ℂ [ℂ] H) (ψ : T.domain) : ℝ :=
  (⟨ψ : H⟩, T ψ).re
```

```
/-- The expectation value, unfolded as a real part. -/
```

```
lemma expectedValue_eq_re_inner (T : H →ℂ [ℂ] H) (ψ : T.domain) :
  expectedValue T ψ = (⟨ψ : H⟩, T ψ).re :=
  rfl
```

```
/-- If `T` is symmetric, `⟨ψ, Tψ⟩` is the expectation value, coerced to `ℂ`. -
/
```

```
lemma expectedValue_eq_inner (T : H →ℂ [ℂ] H) (hT : T.IsSymmetric) (ψ : T.domain) :
  ⟨ψ : H⟩, T ψ = (expectedValue T ψ : ℂ) := by
  have h_re : ((⟨ψ : H⟩, T ψ).re : ℂ) = ⟨ψ : H⟩, T ψ :=
    Complex.conj_eq_iff_re.mp
  (by simpa using conj_inner_apply_self_eq_of_isSymmetric T hT ψ)
  simpa [expectedValue] using h_re.symm
```

```
/-- Reverse orientation of `LinearPMap.expectedValue_eq_inner`. -
/
```

```
lemma inner_eq_expectedValue (T : H →ℂ [ℂ] H) (hT : T.IsSymmetric) (ψ : T.domain) :
  (expectedValue T ψ : ℂ) = ⟨ψ : H⟩, T ψ :=
  (expectedValue_eq_inner T hT ψ).symm
```

```
/-- The centered vector `Tψ - ⟨T⟩_ψ ψ`. -/
```

```
def centered (T : H →ℂ [ℂ] H) (ψ : T.domain) : H :=
  T ψ - (expectedValue T ψ : ℂ) • (ψ : H)
```

```
/-- The centered vector, unfolded to its raw expression. -
/
```

```
lemma centered_eq (T : H →ℂ [ℂ] H) (ψ : T.domain) :
  centered T ψ = T ψ - (expectedValue T ψ : ℂ) • (ψ : H) :=
  rfl
```

```
/-- A centered vector vanishes exactly when `Tψ = ⟨T⟩_ψ ψ`. -
/
```

```

lemma centered_eq_zero_iff (T : H →ℓ.[ℂ] H) (ψ : T.domain) :
  centered T ψ = 0 ↔ T ψ = (expectedValue T ψ : ℂ) • (ψ : H) := by
  rw [centered_eq, sub_eq_zero]

/-- For a unit vector and symmetric `T`, the centered vector is orthogonal
/
lemma inner_state_centered_eq_zero (T : H →ℓ.[ℂ] H) (hT : T.IsSymmetric)
  (ψ : T.domain) (hψ_norm : ||(ψ : H)|| = 1) :
  ⊥(ψ : H), centered T ψ ⊥_ℂ = 0 := by
  rw [centered_eq, inner_sub_right, inner_smul_right, expectedValue_eq_inner]
  simp [hψ_norm, inner_self_eq_norm_sq_to_K]

/-- The conjugate orientation of `LinearPMap.inner_state_centered_eq_zero`.
/
lemma inner_centered_state_eq_zero (T : H →ℓ.[ℂ] H) (hT : T.IsSymmetric)
  (ψ : T.domain) (hψ_norm : ||(ψ : H)|| = 1) :
  ⊥centered T ψ, (ψ : H) ⊥_ℂ = 0 := by
  rw [centered_eq, inner_sub_left, inner_smul_left]
  have hμ := expectedValue_eq_inner T hT ψ
  have hμ' : ⊥T ψ, (ψ : H) ⊥_ℂ = (expectedValue T ψ : ℂ) := by
    simp [inner_conj_symm] using congrArg star hμ
  rw [hμ']
  simp [hψ_norm, inner_self_eq_norm_sq_to_K]

end
end LinearPMap

```

1.64 Physlib/QuantumMechanics/DDimensions/Operators/StateObserv

```

/-
Copyright (c) 2026 Axiomatic-AI. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Matteo Cipollina, Krystian Nowakowski
-/
module

public import Physlib.QuantumMechanics.DDimensions.Operators.Unbounded
/-!
# Eigenvectors of partial linear maps

## Main definitions

- `LinearPMap.IsEigenvector`: a nonzero domain vector satisfying `T ψ = μ •

```

1.65.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/STATEOBSERVABLES/VARIANCE.LEAN

```
-/

@[expose] public section

namespace LinearPMap

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℂ H]

/-- A nonzero vector in the domain of `T` satisfying `T ψ = μ • ψ`. -
/
def IsEigenvector (T : H →ℂ H) (ψ : T.domain) (μ : ℂ) : Prop :=
  T ψ = μ • (ψ : H) ∧ (ψ : H) ≠ 0

/-- The eigenvalue equation for a partial-map eigenvector. -
/
lemma IsEigenvector.apply_eq {T : H →ℂ H} {ψ : T.domain} {μ : ℂ}
  (hψ : T.IsEigenvector ψ μ) :
    T ψ = μ • (ψ : H) :=
  hψ.1

/-- A partial-map eigenvector is nonzero. -/
lemma IsEigenvector.ne_zero {T : H →ℂ H} {ψ : T.domain} {μ : ℂ}
  (hψ : T.IsEigenvector ψ μ) :
    (ψ : H) ≠ 0 :=
  hψ.2

end LinearPMap
```

1.65 Physlib/QuantumMechanics/DDimensions/Operators/Stateobservables/V

/-

Copyright (c) 2026 Axiomatic-AI. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Matteo Cipollina, Krystian Nowakowski

-/

module

public import Physlib.QuantumMechanics.DDimensions.Operators.StateObservables.Expect

public import Physlib.QuantumMechanics.DDimensions.Operators.StateObservables.IsEige

public import Mathlib.Analysis.SpecialFunctions.Sqrt

/-!

Variance and standard deviation

The variance of a partial linear map `T` in a state `ψ` is $\|T\psi - \langle T \rangle_\psi \psi\|^2$. It

requires $\psi \in T.\text{domain}$.

When T is symmetric, $\|\psi\| = 1$, and $T\psi \in T.\text{domain}$, it also equals $\langle T^2 \psi, \psi \rangle$.

Main definitions

- `LinearPMap.variance` and `LinearPMap.standardDeviation`.

Main statements

- `LinearPMap.variance_eq_norm_sq_sub_expectedValue_sq`: for a unit vector ψ , the variance is $\|T\psi\|^2 - \langle T\psi, \psi \rangle^2$.
 - `LinearPMap.variance_eq_re_inner_sub_expectedValue_sq`: the second-order formula when $T\psi \in T.\text{domain}$.
 - `LinearPMap.variance_eq_zero_iff_isEigenvector` and `LinearPMap.standardDeviation_eq_zero_iff_isEigenvector`: for a unit vector ψ , the standard deviation is equivalent to the eigenvector condition.

References

- [B. C. Hall, *Quantum Theory for Mathematicians*, Chapter 12][hall2013qua]
 -/

@[expose] public section

namespace LinearPMap

open InnerProductSpace

noncomputable section

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℂ H]

/-- Variance $\|T\psi - \langle T\psi, \psi \rangle \psi\|^2$; only $\psi \in T.\text{domain}$ is required. -
 /

def variance (T : H →_ℂ H) (ψ : T.domain) : ℝ :=
 ||centered T ψ|| ^ 2

/-- The variance is the squared norm of the centered vector. -
 /

lemma variance_eq_centered_norm_sq (T : H →_ℂ H) (ψ : T.domain) :
 variance T ψ = ||centered T ψ|| ^ 2 :=
 rfl

1.65.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/STATEOBSERVABLES/VARIANCE.LEAN

```

/-- `variance` with `centered` unfolded to  $\langle T \rangle_\psi - \langle T \rangle_\psi \cdot \psi$ . -
/
lemma variance_eq_norm_sub_sq (T : H →ℂ H) (ψ : T.domain) :
  variance T ψ =
    ||T ψ - (expectedValue T ψ : ℂ) • (ψ : H)|| ^ 2 :=
  rfl

/-- For symmetric `T` and `||ψ|| = 1`, variance equals  $\|T\psi\|^2 - \langle T \rangle_\psi^2$ . -
/
lemma variance_eq_norm_sq_sub_expectedValue_sq (T : H →ℂ H)
  (hT : T.IsSymmetric) (ψ : T.domain) (hψ_norm : ||(ψ : H)|| = 1) :
  variance T ψ = ||T ψ|| ^ 2 - expectedValue T ψ ^ 2 := by
  let μ := expectedValue T ψ
  let a : H := T ψ
  have hμ_right : ⟨(ψ : H), a⟩_ℂ = (μ : ℂ) := by
    simp [a, μ] using expectedValue_eq_inner T hT ψ
  have hμ_left : ⟨a, (ψ : H)⟩_ℂ = (μ : ℂ) := by
    simp [inner_conj_symm] using congrArg star hμ_right
  have h_re_inner_centered : (⟨a, (μ : ℂ) • (ψ : H)⟩_ℂ).re = μ ^ 2 := by
    rw [inner_smul_right, hμ_left]
    simp [μ]
    ring
  have h_norm_centered_smul : ||(μ : ℂ) • (ψ : H)|| ^ 2 = μ ^ 2 := by
    rw [norm_smul, hψ_norm]
    simp [μ]
  have h_norm_sub_sq :
    ||a - (μ : ℂ) • (ψ : H)|| ^ 2 =
      ||a|| ^ 2 - 2 * (⟨a, (μ : ℂ) • (ψ : H)⟩_ℂ).re + ||(μ : ℂ) • (ψ : H)|| ^ 2 := by
    simp using (norm_sub_sq (a := a) ((μ : ℂ) • (ψ : H)))
  rw [variance_eq_norm_sub_sq, h_norm_sub_sq, h_re_inner_centered,
    h_norm_centered_smul]
  ring

/-- Variance is nonnegative. -/
lemma variance_nonneg (T : H →ℂ H) (ψ : T.domain) :
  0 ≤ variance T ψ := by
  rw [variance_eq_centered_norm_sq]
  exact sq_nonneg _

/-- Zero variance is the same as a zero centered vector. -
/
lemma variance_eq_zero_iff_centered_eq_zero (T : H →ℂ H) (ψ : T.domain) :
  variance T ψ = 0 ↔ centered T ψ = 0 := by
  rw [variance_eq_centered_norm_sq]
  exact sq_eq_zero_iff.trans norm_eq_zero

```

```

/-- Zero variance is the same as  $\langle T \psi, \psi \rangle = 0$ . -/
lemma variance_eq_zero_iff (T : H →ℂ H) (ψ : T.domain) :
  variance T ψ = 0 ↔ T ψ = (expectedValue T ψ : ℂ) • (ψ : H) := by
  rw [variance_eq_zero_iff_centered_eq_zero, centered_eq_zero_iff]

/-- For  $\|\psi\| = 1$ , zero variance iff  $\psi$  is an eigenvector with eigenvalue 0. -/
lemma variance_eq_zero_iff_isEigenvector (T : H →ℂ H)
  (ψ : T.domain) (hψ_norm :  $\|(\psi : H)\| = 1$ ) :
  variance T ψ = 0 ↔
    T.IsEigenvector ψ (expectedValue T ψ : ℂ) := by
  rw [variance_eq_zero_iff]
  constructor
  · intro h_centered
    refine {h_centered, ?_}
    intro h_zero
    have h_zero' : (ψ : H) = 0 := by simpa using h_zero
    have h_norm_zero :  $\|(\psi : H)\| = 0$  := by simp [h_zero']
    have : (0 : ℝ) = 1 := h_norm_zero.symm.trans hψ_norm
    norm_num at this
  · intro h_eigen
    exact h_eigen.1

/-- Standard deviation  $\sqrt{\text{variance}}$  for  $\psi \in T.\text{domain}$ . -/
def standardDeviation (T : H →ℂ H) (ψ : T.domain) : ℝ :=
  Real.sqrt (variance T ψ)

/-- The standard deviation, unfolded to the square root of the variance. -/
lemma standardDeviation_eq_sqrt_variance (T : H →ℂ H) (ψ : T.domain) :
  standardDeviation T ψ = Real.sqrt (variance T ψ) :=
  rfl

/-- Standard deviation is nonnegative. -/
lemma standardDeviation_nonneg (T : H →ℂ H) (ψ : T.domain) :
  0 ≤ standardDeviation T ψ := by
  rw [standardDeviation_eq_sqrt_variance]
  exact Real.sqrt_nonneg _

@[simp]
lemma standardDeviation_sq (T : H →ℂ H) (ψ : T.domain) :
  standardDeviation T ψ ^ 2 = variance T ψ := by
  rw [standardDeviation_eq_sqrt_variance, Real.sq_sqrt]
  exact variance_nonneg T ψ

```


1.65.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/STATEOBSERVABLES/VARIANCE.LEAN

-- Zero standard deviation is the same as a zero centered vector. -
/

```
lemma standardDeviation_eq_zero_iff_centered_eq_zero (T : H →ℂ H)
  (ψ : T.domain) :
  standardDeviation T ψ = 0 ↔ centered T ψ = 0 := by
  rw [standardDeviation_eq_sqrt_variance, Real.sqrt_eq_zero]
  · exact variance_eq_zero_iff_centered_eq_zero T ψ
  · exact variance_nonneg T ψ
```

-- Zero standard deviation is the same as $\langle T \rangle_\psi \psi$. -
/

```
lemma standardDeviation_eq_zero_iff (T : H →ℂ H) (ψ : T.domain) :
  standardDeviation T ψ = 0 ↔ T ψ = (expectedValue T ψ : ℂ) • (ψ : H) := by
  rw [standardDeviation_eq_zero_iff_centered_eq_zero, centered_eq_zero_iff]
```

-- For $\|\psi\| = 1$, zero standard deviation iff the eigenvector condition holds. -
/

```
lemma standardDeviation_eq_zero_iff_isEigenvector (T : H →ℂ H)
  (ψ : T.domain) (hψ_norm :  $\|(\psi : H)\| = 1$ ) :
  standardDeviation T ψ = 0 ↔
  T.IsEigenvector ψ (expectedValue T ψ : ℂ) := by
  rw [standardDeviation_eq_zero_iff]
  constructor
  · intro h_centered
    refine {h_centered, ?_}
    intro h_zero
    have h_zero' : (ψ : H) = 0 := by simp using h_zero
    have h_norm_zero :  $\|(\psi : H)\| = 0$  := by simp [h_zero']
    have : (0 : ℝ) = 1 := h_norm_zero.symm.trans hψ_norm
    norm_num at this
  · intro h_eigen
    exact h_eigen.1
```

section SecondOrder

```
variable (T : H →ℂ H) (hT : T.IsSymmetric)
variable (ψ : T.domain)
variable (hTψ : T ψ ∈ T.domain)
variable (hψ_norm :  $\|(\psi : H)\| = 1$ )
```

include hT

-- For symmetric T , $\langle \psi, T(T\psi) \rangle$ is $\|T\psi\|^2$. -/
 lemma re_inner_apply_sq_eq_norm_sq :
 $\langle (\psi : H), T \langle T \psi, hT\psi \rangle \rangle_{\mathbb{C}}.re = \|T \psi\|^2 := by
 rw [← hT ψ ⟨T ψ, hTψ⟩, inner_self_eq_norm_sq_to_K]$

```

    rw [sq, sq, Complex.mul_re]
    simp [Complex.ofReal_re, Complex.ofReal_im]

include hψ_norm

/-- When `Tψ ∈ T.domain`, variance equals ` $\langle T^2 \rangle_\psi - \langle T \rangle_\psi^2` . -
/
lemma variance_eq_re_inner_sub_expectedValue_sq :
  variance T ψ =
    (⟦(ψ : H), T ⟨T ψ, hTψ⟩⟧_ℂ).re - expectedValue T ψ ^ 2 := by
  rw [variance_eq_norm_sq_sub_expectedValue_sq T hT ψ hψ_norm,
    re_inner_apply_sq_eq_norm_sq T hT ψ hTψ]

end SecondOrder

end
end LinearPMap$ 
```

1.66 Physlib/QuantumMechanics/DDimensions/Operators/Unbounded.1

In this module we introduce unbounded operators on inner product spaces. By we mean a partially-defined linear map (`LinearPMap`) which is both densely defined and self-adjoint. These provide the mathematical structure appropriate for describing operators in relativistic

quantum mechanics. Of particular interest are (essentially) self-adjoint unbounded operators since these correspond to physical observables.

Notes

- Naming convention : Definitions of `LinearPMap`'s for quantum mechanical operators have a name of the form `[...]Operator` and notation should use calligraphic letters, e.g. `mulOperator f` (`⟦ f ⟧`) for the multiplication operator associated with f .
- Implementation : Although operators encountered in quantum mechanics are bounded, we opt to implement unbounded operators via the property `IsUnbounded` rather than as a structure `UnboundedOperator` extending `LinearPMap`. The basic idea is that addition/subtraction and composition of unbounded operators in `LinearPMap` is not closed, so instead we implement composition of unbounded operators in `LinearPMap` in another unbounded operator. This means, for example, that any attempt to implement `UnboundedOperator`'s would inevitably require introducing junk values to handle the cases where the composition is not defined.

ii. Key results

- `LinearPMap.compRestricted` : For two partial linear maps $g : F \rightarrow_{\perp} [R] G$ and $f : E \rightarrow_{\perp} F$, the composition of g with f with natural domain $\{x : f.\text{domain} \mid f(x) \neq \perp\}$ is given by `LinearPMap.compRestricted g f`.
- `LinearPMap.instMonoid` : Partial linear maps $E \rightarrow_{\perp} [R] E$ with `compRestricted` for multiplication and the identity map for `1` comprise a monoid.

1.66.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED1 LEAN

- `adjoint\_add\_le\_add\_adjoint` : The inequality $\|U_1^\dagger + U_2^\dagger\| \leq (U_1 + U_2)^\dagger$ when $U_1 + U_2$ has dense domain.
- `adjoint\_compRestricted\_le\_compRestricted\_adjoint` : The inequality $\|U^\dagger \circ_r V^\dagger\| \leq (V \circ_r U)^\dagger$ when V and $V \circ_r U$ have dense domain.
- `IsEssentiallySelfAdjoint.unique\_self\_adjoint\_extension` : The closure of an essentially self-adjoint unbounded operator is its unique self-adjoint extension.
- `IsUnbounded.adjoint` : The adjoint of an unbounded operator is also unbounded.
- `IsUnbounded.adjoint\_closure\_eq\_adjoint` : An unbounded operator and its closure have the same adjoint.
- `IsUnbounded.adjoint\_adjoint\_eq\_closure` : An unbounded operator U satisfies $U^\dagger = (U^\dagger)^\dagger$.
- A. General
 - A.1. DistribMulAction
 - A.2. Finite sums
 - A.3. Restricted composition
 - A.4. Monoid
- B. Operators on inner product/Hilbert spaces
 - B.1. Definitions
 - B.2. Dense domain
 - B.3. Closability
 - B.4. Adjoints
 - B.5. Symmetric operators
 - B.6. Self-adjoint operators
 - B.7. Essentially self-adjoint operators
 - B.8. Unbounded operators

namespace LinearPMap

A. General

This section contains useful general results for partial linear maps which do not rely on an inner product/Hilbert space structure.

-/

section General

open Submodule

variable {R : Type*} [Ring R]

variable {E : Type*} [AddCommGroup E] [Module R E]

variable {F : Type*} [AddCommGroup F] [Module R F]

A.1. DistribMulAction

variable {M : Type*} [Monoid M] [DistribMulAction M F] [SMulCommClass R M F]

instance instDistribMulAction : DistribMulAction M (E →_l.[R] F) where

smul_zero _ := by ext; rfl; simp

smul_add _ _ _ := by ext; rfl; simp [add_apply]

A.2. Finite sums

variable {α : Type*} [Fintype α] (f : α → E →_l.[R] F)

/-- A finite sum of partial linear maps.

`sum f` and $\sum a, f a$ are equal, but not by definition.

With `sum f` both `domain` and `toFun` are made explicit. -

```

/
def sum : E →1.[R] F where
  domain := ∑ a, (f a).domain
  toFun := ∑ a, (f a).toFun ∘1 inclusion (fun _ _ ↦ by simp_all only [mem_i
lemma sum_domain : (sum f).domain = ∑ a, (f a).domain := rfl
lemma sum_domain_le (a : α) : (sum f).domain ≤ (f a).domain := fun _ _ ↦ by
lemma sum_apply (ψ : (sum f).domain) : sum f ψ = ∑ a, f a ⟨ψ, sum_domain_le
  simp [sum, inclusion_apply]
end
### A.3. Restricted composition
The composition of two partial linear maps `g : F →1.[R] G` and `f : E →1.[R]
only if the range of `f` is contained in the domain of `g` (c.f. `LinearPMA
`g.compRestricted f` (`g ∘r f`) is defined to be the composition of `g` wit
to exactly those `x : f.domain` for which `f x ∈ g.domain`. This allows one
composition of partial linear maps while having the domain implicitly accou
section
variable {G : Type*} [AddCommGroup G] [Module R G]
/-- `g ∘r f` is the composition of `g` with `f` restricted to a domain cons
`x : f.domain` for which `f x ∈ g.domain`. -/
def compRestricted (g : F →1.[R] G) (f : E →1.[R] F) : E →1.[R] G :=
  g.comp (f.domRestrict <| (g.domain.comap f.toFun).map f.domain.subtype) (
    intro ⟨x, h, _⟩
    simp only [map_coe, subtype_apply, comap_coe, Set.mem_image, Set.mem_pr
      toFun_eq_coe, SetLike.mem_coe] at h
    obtain ⟨y, hy, hy'⟩ := h
    rw [domRestrict_apply hy'.symm]
    exact hy)
@[inherit_doc compRestricted]
infixr:80 " ∘r " => compRestricted

lemma compRestricted_domain_le (g : F →1.[R] G) (f : E →1.[R] F) : (g ∘r f)
  fun _ h ↦ h.2
lemma mem_compRestricted_domain_iff {g : F →1.[R] G} {f : E →1.[R] F} {x :
  x ∈ (g ∘r f).domain ↔ ∃ h : x ∈ f.domain, f ⟨x, h⟩ ∈ g.domain := by
  change x ∈ (g.domain.comap f.toFun).map f.domain.subtype ∑ f.domain ↔ _
  simp
lemma mem_domain_of_mem_compRestricted_domain
  {g : F →1.[R] G} {f : E →1.[R] F} (x : (g ∘r f).domain) : f ⟨x, x.2.2⟩
  obtain (_, h) := mem_compRestricted_domain_iff.mp x.2
  exact h
lemma compRestricted_apply {g : F →1.[R] G} {f : E →1.[R] F} (x : (g ∘r f).
  (g ∘r f) x = g ⟨f ⟨x, x.2.2⟩, mem_domain_of_mem_compRestricted_domain x
  rfl
/-- The zero map is right-absorbing. -/
lemma compRestricted_zero (g : F →1.[R] G) : g ∘r (0 : E →1.[R] F) = 0 := b
  ext

```

1.66.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED3 LEAN

```

    · simp [mem_compRestricted_domain_iff]
    · exact g.map_zero

lemma compRestricted_assoc {H : Type*} [AddCommGroup H] [Module R H]
  (f₁ : G →1.[R] H) (f₂ : F →1.[R] G) (f₃ : E →1.[R] F) :
  (f₁ ∘r f₂) ∘r f₃ = f₁ ∘r f₂ ∘r f₃ := by
  ext
  · simp only [mem_compRestricted_domain_iff]
  · tauto
  · rfl

/-- `compRestricted` is the same as `comp` when the range of `f` is contained in `g`.
/
lemma compRestricted_eq_comp
  {g : F →1.[R] G} {f : E →1.[R] F} (h : ∀ x : f.domain, f x ∈ g.domain) :
  g ∘r f = g.comp f h := by
  ext x
  · change _ ↔ x ∈ f.domain
  · simp [mem_compRestricted_domain_iff, h]
  · rfl

/-- `compRestricted` is maximal amongst compositions of `g` with domain restrictions
/
lemma comp_le_compRestricted {g : F →1.[R] G} {f : E →1.[R] F} {S : Submodule R E}
  (h : ∀ x : (f.domRestrict S).domain, f ⟨x, x.2.2⟩ ∈ g.domain) :
  g.comp (f.domRestrict S) h ≤ g ∘r f :=
  ⟨fun x hx ↦ mem_compRestricted_domain_iff.mpr ⟨hx.2, h ⟨x, hx⟩⟩, by aesop⟩

lemma compRestricted_mono_left {g g' : F →1.[R] G} (h : g ≤ g') (f : E →1.[R] F) :
  g ∘r f ≤ g' ∘r f := by
  constructor
  · intro x hx
    obtain ⟨hx', hfx⟩ := mem_compRestricted_domain_iff.mp hx
    exact mem_compRestricted_domain_iff.mpr ⟨hx', h.1 hfx⟩
  · intro x y hxy
    exact @h.2 ⟨f ⟨x, x.2.2⟩, mem_domain_of_mem_compRestricted_domain x⟩
      ⟨f ⟨y, y.2.2⟩, mem_domain_of_mem_compRestricted_domain y⟩ (by simp [hxy])

lemma compRestricted_mono_right (g : F →1.[R] G) {f f' : E →1.[R] F} (h : f ≤ f') :
  g ∘r f ≤ g ∘r f' := by
  constructor
  · intro x hx
    obtain ⟨hx', hfx⟩ := mem_compRestricted_domain_iff.mp hx
    exact mem_compRestricted_domain_iff.mpr ⟨h.1 hx', (@h.2 ⟨x, hx'⟩ ⟨x, h.1 hx'⟩) rf
  · intro x y hxy
    simp only [compRestricted_apply, @h.2 ⟨x, x.2.2⟩ ⟨y, y.2.2⟩ hxy]

```

A.4. Monoid

Partial linear maps $E \rightarrow_{\perp}.[R] E$ with `compRestricted` for multiplication and the identity map (domain `⊥`) for `1` comprise a monoid.

-/

instance instMonoid : Monoid (E $\rightarrow_{\perp}.[R]$ E) where

mul := compRestricted

mul_assoc := compRestricted_assoc

one := (⊥, topEquiv.toLinearMap)

one_mul f := by

change (⊥, topEquiv.toLinearMap) $\circ_r$ f = f

ext

· simp [mem_compRestricted_domain_iff]

· rfl

mul_one f := by

change f $\circ_r$ (⊥, topEquiv.toLinearMap) = f

ext

· simp [mem_compRestricted_domain_iff]

· rfl

lemma mul_def (f₁ f₂ : E $\rightarrow_{\perp}.[R]$ E) : f₁ * f₂ = f₁ $\circ_r$ f₂ := rfl

lemma one_domain : (1 : E $\rightarrow_{\perp}.[R]$ E).domain = ⊥ := rfl

end General

/-!

B. Operators on inner product/Hilbert spaces

-/

section InnerProductSpaces

open Submodule

open InnerProductSpace

open InnerProductSpaceSubmodule

open Complex ComplexConjugate

variable

{H : Type*} [NormedAddCommGroup H] [InnerProductSpace $\mathbb{C}$ H]

{H' : Type*} [NormedAddCommGroup H'] [InnerProductSpace $\mathbb{C}$ H']

{H'' : Type*} [NormedAddCommGroup H''] [InnerProductSpace $\mathbb{C}$ H'']

{ α : Type*} [Fintype α]

{T T₁ T₂ : H $\rightarrow_{\perp}.[C]$ H} {S : $\alpha \rightarrow H \rightarrow_{\perp}.[C]$ H}

{U U₁ U₂ : H $\rightarrow_{\perp}.[C]$ H'} {W : $\alpha \rightarrow H \rightarrow_{\perp}.[C]$ H'}

{V V₁ V₂ : H' $\rightarrow_{\perp}.[C]$ H''}

instance : IsScalarTower $\mathbb{R}$ $\mathbb{C}$ H := IsScalarTower.complexToReal

B.1. Definitions

See `LinearPMap.instStar` and `LinearPMap.isSelfAdjoint_def` for the definitions for `LinearPMap`'s.

/-- A LinearPMap `U` has dense domain iff `U.domain` is dense in `H`. -

/

1.66.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED5 LEAN

```

def HasDenseDomain (U : H →ℓ.[ℂ] H') : Prop := Dense (U.domain : Set H)
lemma hasDenseDomain_def : U.HasDenseDomain ↔ Dense (U.domain : Set H) := Iff.rfl
/-- A LinearPMap is an unbounded operator iff it has dense domain and is closable. -/
/
def IsUnbounded (U : H →ℓ.[ℂ] H') : Prop := U.HasDenseDomain ∧ U.IsClosable
lemma isUnbounded_def : U.IsUnbounded ↔ U.HasDenseDomain ∧ U.IsClosable := Iff.rfl
/-- A LinearPMap `U` is symmetric iff `∑ x, y ∑_ℂ = ∑ x, U y ∑_ℂ` for all `x y : U.dom` -/
/
def IsSymmetric (T : H →ℓ.[ℂ] H) : Prop := T.IsFormalAdjoint T
lemma isSymmetric_def : T.IsSymmetric ↔ T.IsFormalAdjoint T := Iff.rfl
/-- A LinearPMap is essentially self-adjoint iff its closure is self-adjoint. -/
def IsEssentiallySelfAdjoint [CompleteSpace H] (T : H →ℓ.[ℂ] H) : Prop := IsSelfAdjoint T.closure
lemma isEssentiallySelfAdjoint_def [CompleteSpace H] :
  T.IsEssentiallySelfAdjoint ↔ IsSelfAdjoint T.closure := Iff.rfl
/-!
### B.2. Dense domain
-/
lemma HasDenseDomain.isUnbounded_iff_isClosable (h : U.HasDenseDomain) :
  U.IsUnbounded ↔ U.IsClosable :=
  and_iff_right h

lemma HasDenseDomain.closure (h : U.HasDenseDomain) : U.closure.HasDenseDomain :=
  h.mono U.le_closure.1

lemma closure_domain_le_domain_closure (U : H →ℓ.[ℂ] H') : U.closure.domain ≤ U.domain
by_cases h_cl : U.IsClosable
· intro ψ hψ
  obtain ⟨φ, hψφ⟩ := h_cl.graph_closure_eq_closure_graph ▸ mem_domain_iff.mp hψ
  obtain ⟨b, hb, hb'⟩ := mem_closure_iff_seq_limit.mp hψφ
  apply mem_closure_iff_seq_limit.mpr
  refine ⟨fun n ↦ (b n).1, fun n ↦ ?_, (nhds_prod_eq (x := ψ) (y := φ) ▸ hb').fst⟩
  specialize hb n
  simp only [coe_toAddSubmonoid, SetLike.mem_coe, mem_graph_iff, Subtype.exists,
    exists_and_left, exists_eq_left] at hb
  exact hb.choose
· simp [closure_def' h_cl, closure_le.mp]

lemma hasDenseDomain_iff_closure_hasDenseDomain : U.HasDenseDomain ↔ U.closure.HasDenseDomain
  (HasDenseDomain.closure, fun h ↦ dense_closure.mp (h.mono U.closure_domain_le_domain))

lemma HasDenseDomain.neg (h : U.HasDenseDomain) : (-U).HasDenseDomain := h

lemma HasDenseDomain.smul (h : U.HasDenseDomain) (c : ℂ) : (c • U).HasDenseDomain := h

lemma HasDenseDomain.add_of_le (h₁ : U₁.HasDenseDomain) (h_le : U₁.domain ≤ U₂.domain) :

```

```

      (U1 + U2).HasDenseDomain :=
      h1.mono (by simp [h_le, add_domain])

lemma HasDenseDomain.sub_of_le (h1 : U1.HasDenseDomain) (h_le : U1.domain ≤
      (U1 - U2).HasDenseDomain :=
      h1.mono (by simp [h_le, sub_domain])

lemma HasDenseDomain.sum_of_le
      {E : Submodule ℂ H} (hE : Dense (E : Set H)) (h : ∀ a, E ≤ (W a).domain
      (sum W).HasDenseDomain :=
      hE.mono (by simp [sum_domain, h])

lemma HasDenseDomain.pow
      (h : T.HasDenseDomain) (h_range : ∀ x : T.domain, T x ∈ T.domain) (n :
      (T ^ n).HasDenseDomain := by
      apply h.mono
      induction n with
      | zero => simp
      | succ n ih => exact fun x hx ↦ mem_compRestricted_domain_iff.mpr {hx, ih}

lemma pow_hasDenseDomain_of_le
      {n : ℕ} (h : (T ^ n).HasDenseDomain) {k : ℕ} (hle : k ≤ n) : (T ^ k).Has
      h.mono <| pow_sub_mul_pow T hle ▶ compRestricted_domain_le _ _
/-!
### B.3. Closability
-/
lemma IsClosable.isClosed_iff (h : U.IsClosable) : U.IsClosed ↔ U.closure =
      constructor <=> intro h'
      · exact eq_of_eq_graph (h.graph_closure_eq_closure_graph ▶ h'.submodule_t
      · exact h' ▶ h.closure_isClosed

/-- A LinearPMap with densely-defined formal adjoint is closable. -
/
lemma isClosable_of_exists_dense_formalAdjoint [CompleteSpace H] [CompleteS
      (h : U.HasDenseDomain) (h_fadj : ∃ U' : H' →1.[ℂ] H, U'.HasDenseDomain
      U.IsClosable := by
      have h_adj : U†.HasDenseDomain := by
      obtain ⟨U', hU', hU''⟩ := h_fadj
      refine Dense.mono ?_ hU'
      rcases eq_or_lt_of_le (hU''.symm.le_adjoint h) with (rfl | h_lt)
      · rfl
      · exact (domain_mono h_lt).le
      use U††
      ext
      rw [adjoint_graph_eq_graph_adjoint h_adj, adjoint_graph_eq_graph_adjoint
      mem_submodule_adjoint_adjoint_iff_mem_submoduleToLp_orthogonal_orthogon

```


1.66.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED.lean

```

    orthogonal_orthogonal_eq_closure, mem_submodule_iff_mem_submoduleToLp, submodule

/-- A zero LinearPMap (any domain) is closable. -/
lemma isClosable_of_zero (h_zero : ↑U = 0) : U.IsClosable := by
  use U.graph.topologicalClosure.toLinearPMap
  refine (toLinearPMap_graph_eq _ fun x hx hx₁ ↦ ?_).symm
  obtain ⟨b, hb, hb'⟩ := mem_closure_iff_seq_limit.mp hx
  have hbn : ∀ n, (b n).snd = 0 := fun n ↦ by specialize hb n; simp_all
  rw [nhds_prod_eq, Filter.tendsto_prod_iff'] at hb'
  simp_all
@[aesop safe apply]
lemma IsClosable.smul (h : U.IsClosable) (c : ℂ) : (c • U).IsClosable := by
  rcases eq_zero_or_neZero c with (rfl | hc)
  · exact isClosable_of_zero (by simp)
  · use (c • U).graph.topologicalClosure.toLinearPMap
    refine (toLinearPMap_graph_eq _ fun x hx hx₁ ↦ ?_).symm
    rw [← smul_zero c, ← inv_smul_eq_iff₀ hc.ne]
    refine graph_fst_eq_zero_snd U.closure ?_ rfl
    rw [← h.graph_closure_eq_closure_graph]
    apply mem_closure_iff_seq_limit.mpr
    obtain ⟨b, hb, hb'⟩ := mem_closure_iff_seq_limit.mp hx
    use fun n ↦ ((b n).fst, c⁻¹ • (b n).snd)
    rw [nhds_prod_eq, Filter.tendsto_prod_iff'] at *
    refine ⟨fun n ↦ ?_, hx₁ ▸ hb'.1, hb'.2.const_smul c⁻¹⟩
    obtain ⟨u, hu, hu'⟩ := hb n
    simp only [coe_toAddSubmonoid, SetLike.mem_coe, mem_graph_iff, Subtype.exists, ←
      exact ⟨u.1, u.1.2, rfl, ((inv_smul_eq_iff₀ hc.ne).mpr hu).symm⟩]

lemma IsClosable.smul_iff {c : ℂ} (hc : c ≠ 0) : (c • U).IsClosable ↔ U.IsClosable :
  ⟨fun h ↦ one_smul ℂ U ▸ inv_mul_cancel₀ hc ▸ smul_smul c⁻¹ c U ▸ h.smul c⁻¹, fun h ↦
    one_smul c (c • U) ▸ h⟩

lemma neg_eq_neg_one_smul (U : H →ℂ [ℂ] H') : -U = (-1 : ℂ) • U := ext (by simp) (by
  simp)

@[aesop safe apply]
lemma IsClosable.neg (h : U.IsClosable) : (-U).IsClosable := neg_eq_neg_one_smul U ▸
  h

lemma closure_smul (U : H →ℂ [ℂ] H') {c : ℂ} (hc : c ≠ 0) : (c • U).closure = c • U.
  by_cases h : U.IsClosable
  · apply eq_of_eq_graph
    ext ⟨x₁, x₂⟩
    simp only [← (h.smul c).graph_closure_eq_closure_graph, smul_graph, ← SetLike.mem_
      topologicalClosure_coe, map_coe, LinearMap.prodMap_apply, LinearMap.id_coe, id_
      LinearMap.smul_apply, mem_closure_iff_seq_limit, Set.mem_image, Prod.exists, n
      Filter.tendsto_prod_iff', ← h.graph_closure_eq_closure_graph, Prod.mk.injEq,
      (eq_inv_smul_iff₀ hc).symm, exists_eq_right_right, exists_eq_right]
    constructor <=> intro ⟨b, hb, hb₁, hb₂⟩

```

```

· refine {fun n ↦ ((b n).1, c-1 • (b n).2), fun n ↦ ?_, hb1, hb2.const}
obtain {u, v, huv, huv'} := hb n
have hu := mem_domain_of_mem_graph huv
use {⟨u, hu⟩, v}
simp [← huv', smul_smul, inv_mul_cancel0 hc, (image_iff hu).mpr huv]
· refine {fun n ↦ ((b n).1, c • (b n).2), fun n ↦ ?_, hb1, ?_}
· obtain {u, hu, hu'} := hb n
  exact {u.1, u.2, by simp_all, by simp [← hu']}
· exact one_smul C x2 ▸ mul_inv_cancel0 hc ▸ smul_smul c c-1 x2 ▸ hb2
· rw [closure_def' h, closure_def' <| (not_congr <| IsClosable.smul_iff h)]
### B.4. Adjoints
@[simp]
lemma adjoint_one [CompleteSpace H] : (1 : H →l.[C] H)† = 1 := by
  ext x
  · simp only [one_domain, mem_top, iff_true]
  exact mem_adjoint_domain_of_exists _ {x, fun _ ↦ rfl}
  · exact adjoint_apply_eq dense_univ _ fun _ ↦ rfl

/-- The adjoint of a zero LinearPMap (any domain) is zero (domain `T`). -
/
lemma adjoint_of_zero [CompleteSpace H] (h_zero : ↑U = 0) : U† = 0 := by
  refine dExt ?_ fun x y hxy ↦ ?_
  · ext
    simp only [zero_domain, mem_top, iff_true]
    apply (mem_adjoint_domain_iff _).mpr
    exact continuous_of_const (by simp [h_zero])
  · by_cases h : U.HasDenseDomain
    · exact adjoint_apply_eq h x (by simp [h_zero])
    · exact adjoint_apply_of_not_dense h x
@[simp]
lemma adjoint_smul [CompleteSpace H] (U : H →l.[C] H') {c : C} (hc : c ≠ 0)
  (c • U)† = conj c • U† := by
  refine dExt ?_ fun x y hxy ↦ ?_
  · ext x
    change Continuous (fun w ↦ [x, c • U w]_C) ↔ Continuous (fun w ↦ [x, U w]_C)
    exact Iff.trans (by simp [inner_smul_right]) (continuous_const_smul_iff)
  · by_cases h : U.HasDenseDomain
    · refine adjoint_apply_eq (smul_domain c U ▸ h) x fun w ↦ ?_
      simp [inner_smul_left, inner_smul_right, adjoint_isFormalAdjoint h y]
    · simp [adjoint_apply_of_not_dense h y, adjoint_apply_of_not_dense (smul
lemma adjoint_neg [CompleteSpace H] (U : H →l.[C] H') : (-
U)† = -U† := by
  simp [neg_eq_neg_one_smul, adjoint_smul]
lemma adjoint_antitone [CompleteSpace H]
  (h12 : U1.HasDenseDomain v → U2.HasDenseDomain) (h_le : U1 ≤ U2) : U2† ≤
  have h_agree : ∀ w : U1.domain, U1 w = U2 (w, h_le.1 w.2) := fun w ↦ @h_l

```

1.66.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED99 LEAN

```

constructor
· intro v
  let f₁ : U₁.domain → ℂ := fun w ↦ ⟨v, U₁ w⟩_ℂ
  let f₂ : U₂.domain → ℂ := fun w ↦ ⟨v, U₂ w⟩_ℂ
  change Continuous f₂ → Continuous f₁
  suffices f₁ = fun w : U₁.domain ↦ f₂ ⟨w, h_le.1 w.2⟩ by rw [this]; fun_prop
  simp [f₁, f₂, h_agree]
· intro u v huv
  rcases h₁₂ with (h₁ | h₂)
  · have h₂ : U₂.HasDenseDomain := h₁.mono h_le.1
    refine (adjoint_apply_eq h₁ v fun w ↦ ?_).symm
    rw [adjoint_isFormalAdjoint h₂ u ⟨w, h_le.1 w.2⟩, h_agree, huv]
  · have h₁ : ¬U₁.HasDenseDomain := fun h ↦ h₂ (h.mono h_le.1)
    rw [adjoint_apply_of_not_dense h₁ v, adjoint_apply_of_not_dense h₂ u]

lemma adjoint_add_le_add_adjoint [CompleteSpace H]
  (U₁ U₂ : H →ℓ.[ℂ] H') (h₁₂ : (U₁ + U₂).HasDenseDomain) : U₁† + U₂† ≤ (U₁ + U₂)†
  have h₁ : U₁.HasDenseDomain := h₁₂.mono Set.inter_subset_left
  have h₂ : U₂.HasDenseDomain := h₁₂.mono Set.inter_subset_right
  constructor
  · intro u hu
    apply mem_adjoint_domain_of_exists
    use U₁† ⟨u, hu.1⟩ + U₂† ⟨u, hu.2⟩
    intro x
    simp only [add_apply, inner_add_left, inner_add_right,
      adjoint_isFormalAdjoint h₁ ⟨u, hu.1⟩ ⟨x, x.2.1⟩,
      adjoint_isFormalAdjoint h₂ ⟨u, hu.2⟩ ⟨x, x.2.2⟩]
  · intro u v huv
    refine (adjoint_apply_eq h₁₂ _ fun w ↦ ?_).symm
    simp only [add_apply, inner_add_left, inner_add_right, ← huv,
      adjoint_isFormalAdjoint h₁ ⟨u, u.2.1⟩ ⟨w, w.2.1⟩,
      adjoint_isFormalAdjoint h₂ ⟨u, u.2.2⟩ ⟨w, w.2.2⟩]

lemma adjoint_compRestricted_le_compRestricted_adjoint [CompleteSpace H] [CompleteSpace V]
  (hV : V.HasDenseDomain) (hVU : (V ◦r U).HasDenseDomain) : U† ◦r V† ≤ (V ◦r U)†
  have hU : U.HasDenseDomain := hVU.mono (compRestricted_domain_le V U)
  have h : (U† ◦r V†).IsFormalAdjoint (V ◦r U) := by
    intro x y
    have hx := mem_domain_of_mem_compRestricted_domain x
    have hy := mem_domain_of_mem_compRestricted_domain y
    trans ⟨V† ⟨x, x.2.2⟩, U ⟨y, y.2.2⟩⟩_ℂ
    · exact adjoint_isFormalAdjoint hU ⟨V† ⟨x, x.2.2⟩, hx⟩ ⟨y, y.2.2⟩
    exact adjoint_isFormalAdjoint hV ⟨x, x.2.2⟩ ⟨U ⟨y, y.2.2⟩, hy⟩
  constructor
  · exact fun x hx ↦ mem_adjoint_domain_of_exists _ ((U† ◦r V†) ⟨x, hx⟩, h ⟨x, hx⟩)
  · exact fun x y hxy ↦ (adjoint_apply_eq hVU y <| hxy ▸ h x).symm

```

```

lemma adjoint_pow_le_pow_adjoint [CompleteSpace H] {n : ℕ} (h : (T ^ n).Has
  T† ^ n ≤ (T ^ n)† := by
  induction n with
  | zero => simp
  | succ n ih =>
    have hTn : (T ^ n).HasDenseDomain := pow_hasDenseDomain_of_le h n.le_su
    refine le_trans ?_ (adjoint_compRestricted_le_compRestricted_adjoint hT
    exact pow_succ' T† n ▸ compRestricted_mono_right T† (ih hTn)
/-!
### B.5. Symmetric operators
-/
/-- The analogue of `inner_map_polarization` for LinearPMap. -
/
lemma inner_map_polarization (x y : T.domain) :
  □T y, x□_C = (□T (x + y), ↑(x + y)□_C - □T (x - y), ↑(x - y)□_C
    + I * □T (x + I • y), ↑(x + I • y)□_C - I * □T (x - I • y), ↑(x - I •
  simp only [map_add, coe_add, inner_add_right, inner_add_left, map_sub, Ad
    inner_sub_right, inner_sub_left, sub_sub, map_smul, SetLike.val_smul, i
    neg_mul, inner_smul_right, mul_add, mul_neg, ← mul_assoc, ← pow_two, I
    sub_neg_eq_add, mul_sub]
  ring

/-- The analogue of `inner_map_polarization` for LinearPMap. -
/
theorem inner_map_polarization' (x y : T.domain) :
  □T x, y□_C = (□T (x + y), ↑(x + y)□_C - □T (x - y), ↑(x - y)□_C
    - I * □T (x + I • y), ↑(x + I • y)□_C + I * □T (x - I • y), ↑(x - I •
  simp only [map_add, coe_add, inner_add_right, inner_add_left, map_sub, Ad
    inner_sub_right, inner_sub_left, sub_sub, map_smul, SetLike.val_smul, i
    neg_mul, inner_smul_right, mul_add, mul_neg, ← mul_assoc, ← pow_two, I
    sub_neg_eq_add, mul_sub]
  ring

-- The analogue of `LinearMap.isSymmetric_iff_inner_map_self_real` for Line
lemma isSymmetric_iff_inner_map_self_real :
  T.IsSymmetric ↔ ∀ x : T.domain, conj □T x, x□_C = □T x, x□_C := by
  refine {fun h_symm x ↦ by simp [h_symm x x], fun h_re x y ↦ ?_}
  nth_rw 2 [← inner_conj_symm]
  nth_rw 2 [inner_map_polarization]
  simp only [map_div₀, _root_.map_sub, _root_.map_add, map_mul, neg_mul, co
  rw [inner_map_polarization']
  simp [sub_eq_add_neg]

lemma IsSymmetric.isClosable [CompleteSpace H] (h : T.IsSymmetric) (h' : T.
  T.IsClosable :=

```

1.66.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED

```
isClosable_iff_exists_closed_extension.mpr (T†, adjoint_isClosed h', h.le_adjoint

lemma IsSymmetric.isUnbounded_iff_hasDenseDomain [CompleteSpace H] (h : T.IsSymmetric) :
  T.IsUnbounded ↔ T.HasDenseDomain :=
  and_iff_left_of_imp h.isClosable

lemma isSymmetric_iff_le_adjoint [CompleteSpace H] (h : T.HasDenseDomain) :
  T.IsSymmetric ↔ T ≤ T† := by
  refine (fun h_symm ↦ h_symm.le_adjoint h, fun h_le x y ↦ ?_)
  have h_eq : T x = T† (x, h_le.1 x.2) := @h_le.2 x (x, h_le.1 x.2) rfl
  exact h_eq ▸ adjoint_isFormalAdjoint h _ _

lemma IsSymmetric.isSelfAdjoint_iff [CompleteSpace H] (h : T.IsSymmetric) (h' : T.HasDenseDomain) :
  IsSelfAdjoint T ↔ T†.domain = T.domain := by
  constructor <=> intro h''
  · congr
  · exact (eq_of_le_of_domain_eq ((isSymmetric_iff_le_adjoint h').mp h) h''.symm).symm

lemma add_adjoint_isSymmetric [CompleteSpace H] (h : T.HasDenseDomain) :
  (T + T.adjoint).IsSymmetric := by
  intro x y
  have h1 := adjoint_isFormalAdjoint h (x, x.2.2) (y, y.2.1)
  have h2 := adjoint_isFormalAdjoint h (y, y.2.2) (x, x.2.1)
  apply starRingEquiv.apply_eq_iff_eq.mpr at h2
  simp only [RingEquiv.toEquiv_eq_coe, EquivLike.coe_coe, starRingEquiv_apply, RCLike.inner_conj_symm, MulOpposite.op_inj] at h2
  simp only [add_apply, inner_add_left, inner_add_right, h1, h2]
  exact add_comm _ _

@[aesop safe apply]
lemma IsSymmetric.pow (h : T.IsSymmetric) (n : ℕ) : (T ^ n).IsSymmetric := by
  induction n with
  | zero => exact fun _ _ ↦ rfl
  | succ n ih =>
    intro x y
    let y' : (T * T ^ n).domain := (y, pow_succ' T n ▸ y.2)
    let Tx : (T ^ n).domain := (T (x, x.2.2), mem_domain_of_mem_compRestricted_domain (x, x.2.2))
    let Tny : T.domain := ((T ^ n) (y', y'.2.2), mem_domain_of_mem_compRestricted_domain (y', y'.2.2))
    have h_eq : T Tny = (T ^ (n + 1)) y := by
      change (T * T ^ n) y' = (T ^ (n + 1)) y
    congr 1
    · exact (pow_succ' T n).symm
    · exact (Subtype.heq_iff_coe_eq <| by simp [pow_succ']).mpr rfl
    exact (ih Tx (y', y'.2.2)).trans (h_eq ▸ h (x, x.2.2) Tny)

@[aesop safe apply]
```

```

lemma IsSymmetric.neg (h : T.IsSymmetric) : (-T).IsSymmetric := fun x y ↦ b

@[aesop safe apply]
lemma IsSymmetric.add (h₁ : T₁.IsSymmetric) (h₂ : T₂.IsSymmetric) : (T₁ + T₂).IsSymmetric :=
  intro x y
  specialize h₁ ⟨x, x.2.1⟩ ⟨y, y.2.1⟩
  specialize h₂ ⟨x, x.2.2⟩ ⟨y, y.2.2⟩
  simp [h₁, h₂, add_apply, inner_add_left, inner_add_right]

@[aesop safe apply]
lemma IsSymmetric.sub (h₁ : T₁.IsSymmetric) (h₂ : T₂.IsSymmetric) : (T₁ - T₂).IsSymmetric :=
  sub_eq_add_neg T₁ T₂ ▸ h₁.add h₂.neg

@[aesop safe apply]
lemma IsSymmetric.smul (h : T.IsSymmetric) {c : ℂ} (hc : conj c = c) : (c • T).IsSymmetric :=
  fun x y ↦ by simp only [smul_apply, inner_smul_left, inner_smul_right, hc]

@[aesop safe apply]
lemma IsSymmetric.real_smul (h : T.IsSymmetric) (r : ℝ) : (r • T).IsSymmetric :=
  h.smul (conj_ofReal r)

@[aesop safe apply]
lemma IsSymmetric.sum (h : ∀ a, (S a).IsSymmetric) : (sum S).IsSymmetric :=
  intro x y
  simp [sum_apply, sum_inner, inner_sum, h _ ⟨x, sum_domain_le S _ x.2⟩ ⟨y, sum_domain_le S _ y.2⟩]

lemma IsSymmetric.of_le (h₁ : T₁.IsSymmetric) (h_le : T₂ ≤ T₁) : T₂.IsSymmetric :=
  intro x y
  have hx : T₂ x = T₁ ⟨x, h_le.1 x.2⟩ := @h_le.2 x ⟨x, h_le.1 x.2⟩ rfl
  have hy : T₂ y = T₁ ⟨y, h_le.1 y.2⟩ := @h_le.2 y ⟨y, h_le.1 y.2⟩ rfl
  exact hx ▸ hy ▸ h₁ ⟨x, h_le.1 x.2⟩ ⟨y, h_le.1 y.2⟩

### B.6. Self-adjoint operators
lemma IsSelfAdjoint.isSymmetric [CompleteSpace H] (h : IsSelfAdjoint T) : T.IsSymmetric :=
  rw [isSymmetric_def]
  nth_rw 1 [← h]
  exact adjoint_isFormalAdjoint h.dense_domain

lemma IsSelfAdjoint.isClosed [CompleteSpace H] (h : IsSelfAdjoint T) : T.IsClosed :=
  h ▸ adjoint_isClosed h.dense_domain

lemma IsSelfAdjoint.isClosable [CompleteSpace H] (h : IsSelfAdjoint T) : T.IsClosable :=
  (isClosed h).isClosable

lemma IsSelfAdjoint.isUnbounded [CompleteSpace H] (h : IsSelfAdjoint T) : T.IsUnbounded :=
  ⟨h.dense_domain, isClosable h⟩

lemma IsSelfAdjoint.isEssentiallySelfAdjoint [CompleteSpace H] (h : IsSelfAdjoint T) : T.IsEssentiallySelfAdjoint :=
  T.IsEssentiallySelfAdjoint :=
  isEssentiallySelfAdjoint_def.mpr <|
  ((IsSelfAdjoint.isClosable h).isClosed_iff.mp h.isClosed).symm ▸ h

```

1.66.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED.lean

```

@[aesop safe apply]
lemma IsSelfAdjoint.adjoint [CompleteSpace H] (h : IsSelfAdjoint T) : IsSelfAdjoint
  apply isSelfAdjoint_def.mp at h
  exact h.symm ▸ h
@[aesop safe apply]
lemma IsSelfAdjoint.smul [CompleteSpace H]
  (h : IsSelfAdjoint T) {c : ℂ} (hc : c ≠ 0) (hc' : conj c = c) :
  IsSelfAdjoint (c • T) := by
  rw [isSelfAdjoint_def, T.adjoint_smul hc, hc', isSelfAdjoint_def.mp h]
@[aesop safe apply]
lemma IsSelfAdjoint.real_smul [CompleteSpace H] (h : IsSelfAdjoint T) {r : ℝ} (hr :
  IsSelfAdjoint (r • T) :=
  smul h (ofReal_ne_zero.mpr hr) (conj_ofReal r)
@[aesop safe apply]
lemma IsSelfAdjoint.neg [CompleteSpace H] (h : IsSelfAdjoint T) : IsSelfAdjoint (-
  T) :=
  neg_eq_neg_one_smul T ▸ smul h (by norm_num) (by norm_num)
/-!
### B.7. Essentially self-adjoint operators
-/
lemma IsEssentiallySelfAdjoint.hasDenseDomain [CompleteSpace H] (h : T.IsEssentially
  T.HasDenseDomain :=
  hasDenseDomain_iff_closure_hasDenseDomain.mpr h.dense_domain

lemma IsEssentiallySelfAdjoint.isSymmetric [CompleteSpace H] (h : T.IsEssentiallySel
  T.IsSymmetric :=
  (IsSelfAdjoint.isSymmetric h).of_le T.le_closure

lemma IsEssentiallySelfAdjoint.isClosable [CompleteSpace H] (h : T.IsEssentiallySelf
  T.IsClosable :=
  h.isSymmetric.isClosable h.hasDenseDomain

lemma IsEssentiallySelfAdjoint.isUnbounded [CompleteSpace H] (h : T.IsEssentiallySel
  T.IsUnbounded :=
  h.isSymmetric.isUnbounded_iff_hasDenseDomain.mpr h.hasDenseDomain

/-- The closure is the unique self-adjoint extension of an essentially self-
adjoint operator. -/
lemma IsEssentiallySelfAdjoint.unique_self_adjoint_extension [CompleteSpace H]
  (h : T.IsEssentiallySelfAdjoint) {T₂ : H →1.[ℂ] H} (h_le : T ≤ T₂) (h₂ : IsSelfA
  T₂ = T.closure := by
  have h_dense : T.HasDenseDomain := h.hasDenseDomain
  have h_cl : T₂.IsClosed := IsSelfAdjoint.isClosed h₂
  have h_cl' : T₂.closure = T₂ := h_cl.isClosable.isClosed_iff.mp h_cl
  have h_le' : T.closure ≤ T₂ := h_cl' ▸ h_cl.isClosable.closure_mono h_le
  exact eq_of_le_of_ge (h ▸ h₂ ▸ adjoint_antitone (Or.inl <| h_dense.closure) h_le')
```

```

@[aesop safe apply]
lemma IsEssentiallySelfAdjoint.smul [CompleteSpace H]
  (h : T.IsEssentiallySelfAdjoint) {c : ℂ} (hc : c ≠ 0) (hc' : conj c = c)
  (c • T).IsEssentiallySelfAdjoint := by
  simp_all [isEssentiallySelfAdjoint_def, isSelfAdjoint_def, closure_smul]

@[aesop safe apply]
lemma IsEssentiallySelfAdjoint.real_smul [CompleteSpace H]
  (h : T.IsEssentiallySelfAdjoint) {r : ℝ} (hr : r ≠ 0) :
  (r • T).IsEssentiallySelfAdjoint :=
  h.smul (ofReal_ne_zero.mpr hr) (conj_ofReal r)

@[aesop safe apply]
lemma IsEssentiallySelfAdjoint.neg [CompleteSpace H] (h : T.IsEssentiallySelfAdjoint) :
  (-T).IsEssentiallySelfAdjoint :=
  neg_eq_neg_one_smul T ▸ h.smul (by norm_num) (by norm_num)

### B.8. Unbounded operators
lemma IsUnbounded.hasDenseDomain (h : U.IsUnbounded) : U.HasDenseDomain :=
lemma IsUnbounded.isClosable (h : U.IsUnbounded) : U.IsClosable := h.2
lemma IsUnbounded.adjoint [CompleteSpace H] [CompleteSpace H'] (h : U.IsUnbounded) :
  U†.IsUnbounded := by
  refine {?_, (adjoint_isClosed h.1).isClosable}
  by_contra h_adj
  obtain ⟨y, hy⟩ := not_forall.mp h_adj
  have h_ne_bot : U†.domain ⊆ ⊥ → False := by
    rw [← orthogonal_eq_top_iff, orthogonal_orthogonal_eq_closure]
    exact fun a ↦ ne_of_mem_of_not_mem' mem_top hy a.symm
  obtain ⟨x, hx, hx'⟩ := exists_mem_ne_zero_of_ne_bot h_ne_bot
  apply hx'
  refine graph_fst_eq_zero_snd U.closure ?_ rfl
  rw [← IsClosable.graph_closure_eq_closure_graph h.2,
    mem_submodule_closure_iff_mem_submoduleToLp_closure, ← orthogonal_orthogonal_eq_closure,
    ← mem_submodule_adjoint_adjoint_iff_mem_submoduleToLp_orthogonal_orthogonal_eq_closure,
    ← adjoint_graph_eq_graph_adjoint h.1, mem_submodule_adjoint_iff_mem_submodule_closure]
  rintro ⟨y, Uy⟩ hy
  simp only [neg_zero, WithLp.prod_inner_apply, inner_zero_right, add_zero]
  exact hx y (mem_domain_of_mem_graph hy)

lemma IsUnbounded.closure (h : U.IsUnbounded) : U.closure.IsUnbounded :=
  (h.1.closure, h.2.closureIsClosable)

@[simp]
lemma IsUnbounded.adjoint_closure_eq_adjoint [CompleteSpace H] (h : U.IsUnbounded) :
  U.closure† = U† := by
  refine eq_of_eq_graph ?_

```


1.66.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED5 LEAN

```

ext
  rw [adjoint_graph_eq_graph_adjoint h.1, adjoint_graph_eq_graph_adjoint h.1.closure
    ← IsClosable.graph_closure_eq_closure_graph h.2,
    mem_submodule_closure_adjoint_iff_mem_submoduleToLp_closure_orthogonal, orthogonal
    mem_submodule_adjoint_iff_mem_submoduleToLp_orthogonal]

@[simp]
lemma IsUnbounded.adjoint_adjoint_eq_closure [CompleteSpace H] [CompleteSpace H']
  (h : U.IsUnbounded) :
  U†† = U.closure := by
  refine eq_of_eq_graph ?_
  ext
  rw [adjoint_graph_eq_graph_adjoint h.adjoint.1, adjoint_graph_eq_graph_adjoint h.1
    ← IsClosable.graph_closure_eq_closure_graph h.2,
    mem_submodule_adjoint_adjoint_iff_mem_submoduleToLp_orthogonal_orthogonal,
    orthogonal_orthogonal_eq_closure, mem_submodule_closure_iff_mem_submoduleToLp_cl]

lemma IsUnbounded.le_adjoint_adjoint [CompleteSpace H] [CompleteSpace H'] (h : U.IsUnbounded)
  U ≤ U†† :=
  h.adjoint_adjoint_eq_closure ▸ U.le_closure

lemma IsUnbounded.isClosed_iff [CompleteSpace H] [CompleteSpace H'] (h : U.IsUnbounded)
  U.IsClosed ↔ U†† = U :=
  h.adjoint_adjoint_eq_closure ▸ h.2.isClosed_iff

/-- A LinearPMap constructed from a symmetric LinearMap with dense domain
  is an unbounded operator. -/
lemma isUnbounded_of_dense_of_isSymmetric [CompleteSpace H] {E : Submodule ℂ H}
  (hE : Dense (E : Set H)) {f : E →ℂ H} (h : ∀ x y : E, ⟨f x, ↑y⟩_ℂ = ⟨↑x, f y⟩_ℂ)
  (mk E f).IsUnbounded :=
  ⟨hE, IsSymmetric.isClosable h hE⟩

/-- Variant of `of_dense_of_isSymmetric` for an endomorphism satisfying `LinearMap.IsSymmetric` -/
lemma isUnbounded_of_dense_of_isSymmetric' [CompleteSpace H]
  {E : Submodule ℂ H} (hE : Dense (E : Set H)) {f : E →ℂ E} (h : f.IsSymmetric)
  (mk E (E.subtype ∘ℂ f)).IsUnbounded :=
  ⟨hE, IsSymmetric.isClosable h hE⟩

end InnerProductSpaces
end LinearPMap

```

1.67 Physlib/QuantumMechanics/DDimensions/Operators/Uncertainty

```

/-
Copyright (c) 2026 Axiomatic-AI. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Matteo Cipollina
-/
module

public import Physlib.QuantumMechanics.DDimensions.Operators.Covariance
public import Physlib.QuantumMechanics.DDimensions.Operators.StateObservabl
/-!

# Uncertainty bounds for partial linear maps

## i. Overview

In this module we prove abstract Robertson and Robertson–Schrödinger uncert
symmetric partial linear maps on a complex inner product space. The stateme
any concrete position or momentum operator.

The centered-commutator results use only the domain assumptions needed to f
vectors. The raw-commutator results add the second-order domain hypotheses
to `Bψ` and `B` to `Aψ`.

## ii. Key results

- `inner_im_of_commutator_eq` : an anti-Hermitian commutator identity fixes
  an inner product.
- `centeredCommutatorExpectation` : the scalar commutator of the centered v
- `rawCommutatorExpectation` : the expectation of the raw commutator on a s
- `inner_centered_commutator_of_raw_commutator` : a raw commutator expectat
  commutator expectation.
- `state_uncertainty_squared_of_centered_commutator` : the Robertson square
  commutator identity.
- `state_uncertainty_squared_with_covariance_of_centered_commutator` : the
  Robertson–Schrödinger bound.
- `state_uncertainty_of_centered_commutator` : the standard-
  deviation form of the bound.
- `state_uncertainty_squared_of_raw_commutator`,
  `state_uncertainty_squared_with_covariance_of_raw_commutator`, and
  `state_uncertainty_of_raw_commutator` : variants using a raw commutator e

## iii. Table of contents

- A. Inner product lemmas

```

1.67.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNCERTAINTY.LEAN

- B. Centered commutator bounds
- C. Raw commutator bounds

iv. References

- [H. P. Robertson, *The Uncertainty Principle* (1929)][robertson1929uncertainty].
- [E. Schrodinger, *Zum Heisenbergschen Unscharfeprinzip* (1930)][schrodinger1930hei
- [B. C. Hall, *Quantum Theory for Mathematicians*, Chapter 12][hall2013quantum].

-/

@[expose] public section

namespace LinearPMap

open InnerProductSpace

noncomputable section

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℂ H]

/-!

A. Inner product lemmas

-/

```
lemma inner_im_of_commutator_eq {u v : H} {c : ℝ}
  (h_comm : ⟨u, v⟩_ℂ - ⟨v, u⟩_ℂ = Complex.I * c) :
  (⟨u, v⟩_ℂ).im = c / 2 := by
  have h_conj_im : (⟨v, u⟩_ℂ).im = -(⟨u, v⟩_ℂ).im := by
    rw [(inner_conj_symm (λ := ℂ) v u).symm, Complex.conj_im]
  have h_im := congrArg Complex.im h_comm
  rw [Complex.sub_im] at h_im
  simp at h_im
  rw [h_conj_im] at h_im
  linarith
```

```
lemma inner_norm_sq_eq_re_sq_add_commutator_half_sq {u v : H} {c : ℝ}
  (h_comm : ⟨u, v⟩_ℂ - ⟨v, u⟩_ℂ = Complex.I * c) :
  ||⟨u, v⟩_ℂ|| ^ 2 = (⟨u, v⟩_ℂ).re ^ 2 + (c / 2) ^ 2 := by
  rw [← Complex.normSq_eq_norm_sq, Complex.normSq_apply, inner_im_of_commutator_eq h_comm]
  ring
```

```
lemma sub_expectation_commutator_eq_raw
  (ψ a b : H) (μa μb : ℝ)
```

```

(hμa_right : []ψ, a[]_C = (μa : ℂ)) (hμa_left : []a, ψ[]_C = (μa : ℂ))
(hμb_right : []ψ, b[]_C = (μb : ℂ)) (hμb_left : []b, ψ[]_C = (μb : ℂ)) (hψ_
[]a - (μa : ℂ) • ψ, b - (μb : ℂ) • ψ[]_C -
[]b - (μb : ℂ) • ψ, a - (μa : ℂ) • ψ[]_C =
[]a, b[]_C - []b, a[]_C := by
calc
[]a - (μa : ℂ) • ψ, b - (μb : ℂ) • ψ[]_C -
[]b - (μb : ℂ) • ψ, a - (μa : ℂ) • ψ[]_C =
([]a, b[]_C - (μb : ℂ) * []a, ψ[]_C - star (μa : ℂ) * []ψ, b[]_C +
star (μa : ℂ) * (μb : ℂ) * []ψ, ψ[]_C) -
([]b, a[]_C - (μa : ℂ) * []b, ψ[]_C - star (μb : ℂ) * []ψ, a[]_C +
star (μb : ℂ) * (μa : ℂ) * []ψ, ψ[]_C) := by
simp only [inner_sub_left, inner_sub_right, inner_smul_left,
simp [mul_comm, mul_assoc]
ring_nf
_ = []a, b[]_C - []b, a[]_C := by
rw [hμa_right, hμa_left, hμb_right, hμb_left, inner_self_eq_norm,
hψ_norm]
simp only [Complex.star_def, Complex.conj_ofReal, pow_two, mul_as
ring_nf

lemma raw_commutator_eq_of_symmetric
(A B : H →ℂ ℂ) (hA : A.IsSymmetric) (hB : B.IsSymmetric)
(ψ : A.domain) (hψB : (ψ : H) ∈ B.domain)
(hBA : A ψ ∈ B.domain) (hAB : B ⟨ψ, hψB⟩ ∈ A.domain)
{c : ℝ}
(h_raw : []⟨ψ : H⟩, A ⟨B ⟨ψ, hψB⟩, hAB⟩ - B ⟨A ψ, hBA⟩[]_C = Complex.I *
[]A ψ, B ⟨ψ, hψB⟩[]_C - []B ⟨ψ, hψB⟩, A ψ[]_C = Complex.I * c := by
have ha_pairing :
[]A ψ, B ⟨ψ, hψB⟩[]_C = []⟨ψ : H⟩, A ⟨B ⟨ψ, hψB⟩, hAB⟩[]_C := by
exact hA ψ ⟨B ⟨ψ, hψB⟩, hAB⟩
have hb_pairing :
[]B ⟨ψ, hψB⟩, A ψ[]_C = []⟨ψ : H⟩, B ⟨A ψ, hBA⟩[]_C := by
exact hB ⟨ψ, hψB⟩ ⟨A ψ, hBA⟩
calc
[]A ψ, B ⟨ψ, hψB⟩[]_C - []B ⟨ψ, hψB⟩, A ψ[]_C =
[]⟨ψ : H⟩, A ⟨B ⟨ψ, hψB⟩, hAB⟩[]_C -
[]⟨ψ : H⟩, B ⟨A ψ, hBA⟩[]_C := by
rw [ha_pairing, hb_pairing]
_ = []⟨ψ : H⟩, A ⟨B ⟨ψ, hψB⟩, hAB⟩ - B ⟨A ψ, hBA⟩[]_C := by
rw [inner_sub_right]
_ = Complex.I * c := h_raw

/-- The scalar commutator of the centered vectors of `A` and `B` in the sta
/
def centeredCommutatorExpectation (A B : H →ℂ ℂ) H)

```

1.67.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNCERTAINTY.LEAN

```

    (ψ : A.domain) (hψB : (ψ : H) ∈ B.domain) : ℂ :=
    centered A ψ, centered B (ψ, hψB) -
    centered B (ψ, hψB), centered A ψ
/-
The expectation of the raw commutator `[A, B]` in the state `ψ`, with explicit
order
domain witnesses. -/
def rawCommutatorExpectation (A B : H → ℂ) (H)
  (ψ : A.domain) (hψB : (ψ : H) ∈ B.domain)
  (hBA : A ψ ∈ B.domain) (hAB : B (ψ, hψB) ∈ A.domain) : ℂ :=
  (ψ : H), A (B (ψ, hψB), hAB) - B (A ψ, hBA)

lemma commutator_half_sq_le_mul_norm_sq {u v : H} {c : ℝ}
  (h_comm : (u, v) - (v, u) = Complex.I * c) :
  (|c| / 2) ^ 2 ≤ (||u|| * ||v||) ^ 2 := by
  suffices (|c| / 2) ^ 2 ≤ (||u|| * ||v||) ^ 2 by exact this
  have h_sq : |c / 2| ^ 2 ≤ (||u|| * ||v||) ^ 2 := by
    have h_bound : |c / 2| ≤ ||u|| * ||v|| := by
      have h_im : |(u, v).im| ≤ ||u|| * ||v|| :=
        le_trans (Complex.abs_im_le_norm u, v) (norm_inner_le_norm u v)
      rwa [inner_im_of_commutator_eq h_comm] at h_im
    have h_nonneg : 0 ≤ ||u|| * ||v|| := mul_nonneg (norm_nonneg u) (norm_nonneg v)
    nlinarith [abs_nonneg (c / 2), h_bound, h_nonneg]
  simpa [abs_div] using h_sq

private lemma sqrt_mul_le_of_sq_le {x y z : ℝ}
  (hx : 0 ≤ x) (hz : 0 ≤ z) (hxy : z ^ 2 ≤ x * y) :
  z ≤ Real.sqrt x * Real.sqrt y := by
  suffices z ≤ Real.sqrt x * Real.sqrt y by exact this
  have hs : Real.sqrt (z ^ 2) ≤ Real.sqrt (x * y) := Real.sqrt_le_sqrt hxy
  rw [Real.sqrt_sq hz, Real.sqrt_mul hx] at hs
  simpa [mul_comm] using hs

/-!

## B. Centered commutator bounds

-/

section CenteredBounds

variable (A B : H → ℂ)
variable (ψ : A.domain)
variable (hψB : (ψ : H) ∈ B.domain)
variable {c : ℝ}
variable (h_centered : centeredCommutatorExpectation A B ψ hψB = Complex.I * c)

```

```

include h_centered

/-- A centered commutator identity implies the squared Robertson uncertainty
/
lemma state_uncertainty_squared_of_centered_commutator :
  (|c| / 2) ^ 2 ≤ variance A ψ * variance B ⟨ψ, hψB⟩ := by
  rw [variance_eq_centered_norm_sq, variance_eq_centered_norm_sq]
  rw [show ||centered A ψ|| ^ 2 * ||centered B ⟨ψ, hψB⟩|| ^ 2 =
    (||centered A ψ|| * ||centered B ⟨ψ, hψB⟩||) ^ 2 by ring]
  exact commutator_half_sq_le_mul_norm_sq (by simp [centeredCommutatorExpectation]
    h_centered)

/-- A centered commutator identity implies the Robertson–Schrödinger uncertainty
/
lemma state_uncertainty_squared_with_covariance_of_centered_commutator :
  (covariance A B ψ hψB) ^ 2 + (c / 2) ^ 2 ≤
    variance A ψ * variance B ⟨ψ, hψB⟩ := by
  rw [variance_eq_centered_norm_sq, variance_eq_centered_norm_sq]
  rw [show ||centered A ψ|| ^ 2 * ||centered B ⟨ψ, hψB⟩|| ^ 2 =
    (||centered A ψ|| * ||centered B ⟨ψ, hψB⟩||) ^ 2 by ring]
  calc
    (covariance A B ψ hψB) ^ 2 + (c / 2) ^ 2 =
      ||⟨centered A ψ, centered B ⟨ψ, hψB⟩⟩_C|| ^ 2 := by
        rw [inner_norm_sq_eq_re_sq_add_commutator_half_sq
          (by simp [centeredCommutatorExpectation] using h_centered)]
        rfl
    _ ≤ (||centered A ψ|| * ||centered B ⟨ψ, hψB⟩||) ^ 2 := by
      have h_bound :=
        norm_inner_le_norm (λ := C) (centered A ψ) (centered B ⟨ψ, hψB⟩)
      have h_inner_nonneg : 0 ≤ ||⟨centered A ψ, centered B ⟨ψ, hψB⟩⟩_C|| :
        norm_nonneg _
      have h_mul_nonneg : 0 ≤ ||centered A ψ|| * ||centered B ⟨ψ, hψB⟩|| :=
        mul_nonneg (norm_nonneg _) (norm_nonneg _)
      nlinarith

/-- A centered commutator identity implies the standard uncertainty bound.
/
lemma state_uncertainty_of_centered_commutator :
  |c| / 2 ≤ standardDeviation A ψ * standardDeviation B ⟨ψ, hψB⟩ := by
  have h_sq := state_uncertainty_squared_of_centered_commutator A B ψ hψB
  refine sqrt_mul_le_of_sq_le (variance_nonneg A ψ) (by positivity) ?_
  simp [standardDeviation] using h_sq

end CenteredBounds

```

1.67.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNCERTAINTY.LEAN

/-!

C. Raw commutator bounds

-/

section RawBounds

```
variable (A B : H →ℂ H) (hA : A.IsSymmetric) (hB : B.IsSymmetric)
variable (ψ : A.domain)
variable (hψB : (ψ : H) ∈ B.domain)
variable (hψ_norm : ||(ψ : H)|| = 1)
variable (hBA : A ψ ∈ B.domain)
variable (hAB : B ⟨ψ, hψB⟩ ∈ A.domain)
variable {c : ℝ}
variable (h_raw : rawCommutatorExpectation A B ψ hψB hBA hAB = Complex.I * c)
```

include hA hB hψ_norm hBA hAB h_raw

/-- A raw commutator expectation determines the centered commutator expectation. -
/

```
lemma inner_centered_commutator_of_raw_commutator :
  centeredCommutatorExpectation A B ψ hψB = Complex.I * c := by
  let a : H := A ψ
  let b : H := B ⟨ψ, hψB⟩
  let μa : ℝ := expectedValue A ψ
  let μb : ℝ := expectedValue B ⟨ψ, hψB⟩
  have hμa_right : ⟨(ψ : H), a⟩ℂ = (μa : ℂ) := by
    simp [a, μa] using expectedValue_eq_inner A hA ψ
  have hμa_left : ⟨a, (ψ : H)⟩ℂ = (μa : ℂ) := by
    have h_symm : ⟨a, (ψ : H)⟩ℂ = ⟨(ψ : H), a⟩ℂ := by
      simp [a] using hA ψ ψ
    simp [h_symm] using hμa_right
  have hμb_right : ⟨(ψ : H), b⟩ℂ = (μb : ℂ) := by
    simp [b, μb] using expectedValue_eq_inner B hB ⟨ψ, hψB⟩
  have hμb_left : ⟨b, (ψ : H)⟩ℂ = (μb : ℂ) := by
    have h_symm : ⟨b, (ψ : H)⟩ℂ = ⟨(ψ : H), b⟩ℂ := by
      simp [b] using hB ⟨ψ, hψB⟩ ⟨ψ, hψB⟩
    simp [h_symm] using hμb_right
  calc
    centeredCommutatorExpectation A B ψ hψB =
      ⟨centered A ψ, centered B ⟨ψ, hψB⟩⟩ℂ -
      ⟨centered B ⟨ψ, hψB⟩, centered A ψ⟩ℂ := by
        rfl
    =
    - ⟨a - (μa : ℂ) • (ψ : H), b - (μb : ℂ) • (ψ : H)⟩ℂ -
```

```

      [b - (μb : ℂ) • (ψ : H), a - (μα : ℂ) • (ψ : H)]_ℂ := by
        rfl
    _ = [a, b]_ℂ - [b, a]_ℂ :=
      sub_expectation_commutator_eq_raw (ψ : H) a b μα μb
        hμα_right hμα_left hμb_right hμb_left hψ_norm
    _ = Complex.I * c :=
      raw_commutator_eq_of_symmetric A B hA hB ψ hψB hBA hAB
        (by simp [rawCommutatorExpectation] using h_raw)

/-- A raw commutator expectation implies the squared Robertson uncertainty
/
lemma state_uncertainty_squared_of_raw_commutator :
  (|c| / 2) ^ 2 ≤ variance A ψ * variance B ⟨ψ, hψB⟩ :=
  state_uncertainty_squared_of_centered_commutator A B ψ hψB
    (inner_centered_commutator_of_raw_commutator A B hA hB ψ hψB hψ_norm hB)

/-- A raw commutator expectation implies the squared uncertainty bound with
/
lemma state_uncertainty_squared_with_covariance_of_raw_commutator :
  (covariance A B ψ hψB) ^ 2 + (c / 2) ^ 2 ≤
    variance A ψ * variance B ⟨ψ, hψB⟩ :=
  state_uncertainty_squared_with_covariance_of_centered_commutator A B ψ hψB
    (inner_centered_commutator_of_raw_commutator A B hA hB ψ hψB hψ_norm hB)

/-- A raw commutator expectation implies the standard uncertainty bound. -
/
lemma state_uncertainty_of_raw_commutator :
  |c| / 2 ≤ standardDeviation A ψ * standardDeviation B ⟨ψ, hψB⟩ :=
  state_uncertainty_of_centered_commutator A B ψ hψB
    (inner_centered_commutator_of_raw_commutator A B hA hB ψ hψB hψ_norm hB)

end RawBounds

end
end LinearPMap

```

1.68 Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Ba

```

/-!
## Limits
-/

open Filter

```


1.69

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/SPACEDHILBERTSPACE/POLYBDDSCHWARTZSUBMODULE.LEAN

```
lemma tendsto_zero_iff_tendsto_zero_lintegral_enorm_sq
  {d : ℕ} {α : Type*} {l : Filter α} {ψ : α → SpaceDHilbertSpace d} :
  Tendsto ψ l (nhds 0) ↔ Tendsto (fun a ↦ ∫- x : Space d, ‖ψ a x‖e ^ 2) l (nhds 0)
trans Tendsto (fun a ↦ (∫- x, ‖ψ a x‖e ^ 2) ^ (2-1 : ℝ)) l (nhds 0)
· simp [tendsto_iff_edist_tendsto_0, edist_zero_right, Lp.enorm_def, eLpNorm, eLpN
constructor <=> intro h
· apply Tendsto.enrpow_const 2 at h
  simp_all [← ENNReal.rpow_mul_natCast]
· apply Tendsto.enrpow_const 2-1 at h
  simp_all
```

1.69 Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/PolyBddSch

- `PolyBddSchwartzSubmodule.dense d a`: These submodules are dense in `SpaceDHilbert`
- B. Coercions
- C. (In)equalities
- D. Density
noncomputable section

```
open SchwartzSubmodule
## A. Definitions
namespace PolyBddSchwartzSubmodule
```

```
/-!
## B. Coercions
-/
```

```
instance {d : ℕ} {a : ℕ∞} : CoeOut (polyBddSchwartzMap d a) (Space d, ℂ) := ⟨fun f
instance {d : ℕ} {a : ℕ∞} : CoeFun (polyBddSchwartzMap d a) (fun _ ↦ Space d → ℂ) :=
  ⟨fun f ↦ ↑f.val⟩
```

```
@[simp]
lemma toFun_eq_coe {d : ℕ} {a : ℕ∞} (f : polyBddSchwartzMap d a) (x : Space d) :
  f.val.toFun x = f.val x :=
  rfl
```

```
lemma polyBddSchwartzEquiv_symm_apply_coe {d : ℕ} {a : ℕ∞}
  {ψ : schwartzSubmodule d} (hψ : ↑ψ ∈ polyBddSchwartzSubmodule d a) :
  (polyBddSchwartzEquiv.symm ⟨ψ, hψ⟩).val = schwartzEquiv.symm ψ := by
  apply schwartzEquiv.injective
  apply SetLike.coe_eq_coe.mp
  obtain ⟨g, hg⟩ := polyBddSchwartzEquiv.surjective ⟨ψ.val, hψ⟩
```

```

have hg' : polyBddSchwartzIncl g =  $\psi$  := SetLike.coe_eq_coe.mpr hg
rw [← hg, LinearEquiv.symm_apply_apply, LinearEquiv.apply_symm_apply, ← h
rfl

lemma polyBddSchwartzEquiv_coe_ae {d :  $\mathbb{N}$ } {a :  $\mathbb{N}_\infty$ } (f : polyBddSchwartzMap
polyBddSchwartzEquiv f =m[volume] f.val := schwartzEquiv_coe_ae f.val

### C. (In)equalities
lemma polyBddSchwartzMap_antitone (d :  $\mathbb{N}$ ) {a b :  $\mathbb{N}_\infty$ } (h : a ≤ b) :
lemma of_zero_eq (d :  $\mathbb{N}$ ) : polyBddSchwartzSubmodule d 0 = schwartzSubmodule
lemma le_schwartzSubmodule (d :  $\mathbb{N}$ ) (a :  $\mathbb{N}_\infty$ ) : polyBddSchwartzSubmodule d a
LinearMap.range_domRestrict_le_range _
lemma antitone (d :  $\mathbb{N}$ ) {a b :  $\mathbb{N}_\infty$ } (h : a ≤ b) :
exact Submodule.map_mono (polyBddSchwartzMap_antitone d h)
### D. Density
private lemma dense_zero_top :
simp [polyBddSchwartzSubmodule, polyBddSchwartzIncl, this]
lemma dense_top (d :  $\mathbb{N}$ ) : Dense (polyBddSchwartzSubmodule d  $\tau$  : Set (SpaceD
exact dense_zero_top
obtain ⟨ $\psi$ , h $\psi$ , h $\psi\xi$ ⟩ := mem_closure_iff_seq_limit.mp (SchwartzSubmodule.
lemma dense (d :  $\mathbb{N}$ ) (a :  $\mathbb{N}_\infty$ ) : Dense (polyBddSchwartzSubmodule d a : Set (S
(dense_top d).mono (antitone d le_top)
end PolyBddSchwartzSubmodule

```

1.70 Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Sc

- A. Definitions
- B. Coercions
- D. Misc.

noncomputable section

variable {d : $\mathbb{N}$ }

```

## A. Definitions
namespace SchwartzSubmodule

```

```

/-!

```

```

## B. Coercions

```

```

-/

```

```

instance : CoeFun (schwartzSubmodule d) fun _ ↦ Space d →  $\mathbb{C}$  := ⟨fun  $\psi$  ↦  $\psi.v$ 

```

```

lemma schwartzEquiv_apply_coe : ↑(schwartzEquiv f) = schwartzIncl f := by s

```

1.71.

PHYSLIB/QUANTUMMECHANICS/FINITETARGET/HILBERTSPACE.LEAN65

```
lemma schwartzEquiv_coe_ae : schwartzEquiv f = "[volume] f := coeFn_toLp f 2 volume
lemma schwartzEquiv_ae_eq (h : schwartzEquiv f = "[volume] schwartzEquiv g) : f = g :
  (EmbeddingLike.apply_eq_iff_eq _).mp (SetLike.coe_eq_coe.mp (ext_iff.mpr h))
```

```
/-!
## C. Misc.
-/-
```

```
@[simp]
lemma zero_eq_top : schwartzSubmodule 0 =  $\tau$  := by
  ext  $\psi$ 
  simp only [LinearMap.mem_range, ContinuousLinearMap.coe_coe, Submodule.mem_top, if
  let g :  $\square$ (Space 0,  $\mathbb{C}$ ) := {
    toFun x :=  $\psi$  0
    smooth' := contDiff_const
    decay' k n := by
      refine ( $\|\psi$  0 $\|$ , fun x  $\mapsto$  ?_)
      rcases eq_zero_or_pos n with rfl | hn
      · rw [← one_mul  $\|\psi$  0 $\|$ ]
        refine mul_le_mul ?_ (by simp) (norm_nonneg _) zero_le_one
        simp [Space.point_dim_zero_eq, zero_pow_le_one]
      · simp [iteratedFDeriv_const_of_ne hn.ne']
    }
  use g
  ext
  filter_upwards [schwartzEquiv_coe_ae g] with x hg
  rw [← schwartzEquiv_apply_coe, hg, Space.point_dim_zero_eq x]
  rfl
```

```
lemma dense (d :  $\mathbb{N}$ ) : Dense (schwartzSubmodule d : Set (SpaceDHilbertSpace d)) :=
  denseRange_toLpCLM ENNReal.top_ne_ofNat.symm
end SchwartzSubmodule
```

1.71 Physlib/QuantumMechanics/FiniteTarget/HilbertSpace.lean

```
public import Physlib.Meta.TODO.Basic
TODO "To match this with the results currently in the `QuantumInfo` part of the library
we should:
1. Define `FiniteHilbertSpace` as a structure with a single entry `val`, this
   should take as an input a finite and decidable type `d`. Below this type is
   taken as default to be `Fin n`.
2. On this type we should then define the structure of an inner-
   product space, and a
```

Hilbert space.

3. We could then define the notation `⌈[d]` to denote the Hilbert space `C` to the type `d`.
4. The results from `QuantumInfo/Finite/Braket.lean` can then be moved over to `Physlib`, and related to the definition of the Hilbert space here. Optional. Maybe it is worth moving these files to a directory called `State` the idea that it includes this definition of the Hilbert space, the definition of bras and kets, and the definition of mixed states. Maybe all parts of `./ResourceTheory/FreeState`.

1.72 Physlib/Relativity/LorentzAlgebra/Basis.lean

```

/-- The boost generators are symmetric. -/
@[simp]
lemma boostGenerator_transpose (i : Fin 3) :
  (boostGenerator i)T = boostGenerator i := by
  ext μ v
  rcases μ with μ | μ <=> rcases v with v | v <=>
  fin_cases μ <=> fin_cases v <=> simp [boostGenerator]

/-- The boost generators are traceless. -/
@[simp]
lemma boostGenerator_trace (i : Fin 3) :
  Matrix.trace (boostGenerator i) = 0 := by
  rw [Matrix.trace]
  apply Finset.sum_eq_zero
  intro μ _
  rcases μ with μ | μ <=> simp [boostGenerator]

/-- The rotation generators are antisymmetric. -/
@[simp]
lemma rotationGenerator_transpose (i : Fin 3) :
  (rotationGenerator i)T = -rotationGenerator i := by
  ext μ v
  fin_cases i <=> rcases μ with μ | μ <=> rcases v with v | v <=>
  fin_cases μ <=> fin_cases v <=> simp [rotationGenerator]

/-- The rotation generators are traceless. -/
@[simp]
lemma rotationGenerator_trace (i : Fin 3) :
  Matrix.trace (rotationGenerator i) = 0 := by
  rw [Matrix.trace]
  apply Finset.sum_eq_zero

```

```

intro μ _
rcases μ with μ | μ
· fin_cases i <=> simp [rotationGenerator]
· fin_cases i <=> fin_cases μ <=> simp [rotationGenerator]

```

1.73 Physlib/Relativity/Tensors/Basic.lean

public import Physlib.Relativity.Tensors.Contraction.SuccSuccAbove
public import Mathlib.Tactic.Cases
This module sets up the tensors used for index notation in Physlib.

Physics picture:

- Choose a symmetry group `G` (for example, the Lorentz group).
- Give each index kind a name called a **color** (encoded by a type `C`).
- In Lorentzian notation, a typical choice is two colors: `up` and `down`.
- For each color, choose a representation of `G`.
- Fix a basis for each color. This determines which values that index can take and lets us speak about tensor components explicitly.
- Specify a dual color for each color, telling us which index pairs can contract.
- For dual pairs, provide the contraction/unit/metric data used for contraction and for raising/lowering indices.

All of this structure is packaged as a `TensorSpecies`.

For `S : TensorSpecies` and a list of index colors `c : Fin n → C`:

- `Tensor S c` is the type of tensors with those indices.
- `Pure S c` is the type of pure tensors, i.e. tensors of the form $v_0 \otimes_t v_1 \otimes_t v_2 \dots$.
- `ComponentIdx S c` is the type of allowed component labels for those indices.

From this setup we get the usual index-notation operations in a uniform way: contraction, raising/lowering, and permutation of indices.

TODO "Refactor: Throughout the `Tensor` file system are lemmas related to `ComponentIdx`. The definition of `ComponentIdx` and the lemmas about it should be placed in it's own directory. Around `ComponentIdx`, we should build convenient API. Here `ComponentIdx` is the type of values that indices in e.g. Lorentz tensors can take."

TODO "Define the equivalence between `ComponentIdx ![c]` and `basisIdx c`. Replace Lorentz.Vector.indexEquiv and Lorentz.CoVector.indexEquiv with this more general definition."

```

open Fin in
lemma PermCond.succSuccAbove {n n1 : ℕ} {c : Fin (n + 1 + 1) → C}

```

```

    {c1 : Fin (n1 + 1 + 1) → C}
    (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j)
    {σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)} (hσ : PermCond c c1 σ) :
    PermCond (c ∘ succSuccAbove (σ i) (σ j))
      (c1 ∘ succSuccAbove i j) (funPredPredAbove i j hij σ hσ.1) := by
    apply And.intro
    · exact funPredPredAbove_bijective i j hij σ hσ.left
    · intro m
      simp [funPredPredAbove, hσ.2]

open Fin in
lemma PermCond.succSuccAbove_comm {n : ℕ} {c : Fin (n + 1 + 1 + 1 + 1) → C}
  (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
  (hij1 : i1 ≠ j1) (hij2 : i2 ≠ j2) :
  let i2' := (i1.succSuccAbove j1 i2);
  let j2' := (i1.succSuccAbove j1 j2);
  have hi2j2' : i2' ≠ j2' := by simp [i2', j2', hij2];
  let i1' := (predPredAbove i2' j2' hi2j2' i1 (by simp [i2', j2']));
  let j1' := (predPredAbove i2' j2' hi2j2' j1 (by simp [i2', j2']));
  PermCond ((c ∘ i2'.succSuccAbove j2') ∘ i1'.succSuccAbove j1')
    ((c ∘ i1.succSuccAbove j1) ∘ i2.succSuccAbove j2) id := by
  apply And.intro (Function.bijective_id)
  simp only [id_eq, Function.comp_apply]
  intro i
  rw [succSuccAbove_comm_apply]
  · simp [hij1]
  · simp [hij2]

open Fin in
lemma PermCond.append_right_succSuccAbove {n n1 : ℕ} {c : Fin (n + 1 + 1) → C}
  {c1 : Fin n1 → C} (i j : Fin (n + 1 + 1)) :
  PermCond (Fin.append c1 c ∘ (Fin.succSuccAbove (finSumFinEquiv (m := n1)
    (finSumFinEquiv (m := n1) (Sum.inr j))))
    (Fin.append c1 (c ∘ (i.succSuccAbove j)))) id := by
  apply And.intro (Function.bijective_id)
  simp [forall_fin_add, succSuccAbove_comm_natAdd i j, succSuccAbove_natAdd]

lemma permT_basis {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
  {σ : Fin m → Fin n} (h : PermCond c c1 σ)
  (b : ComponentIdx c) :
  (permT σ h) (basis (S := S) c b) = basis c1 (fun i =>
    basisIdxCongr (by simp [h.2]) (b (σ i))) := by
  apply (basis c1).repr.injective
  ext b'
  rw [permT_basis_repr_symm_apply]

```

```

simp only [Basis.repr_self, Finsupp.single_apply]
congr 1
simp only [eq_iff_iff]
constructor
· intro h
  rw [h]
  ext i
  simp only [basisIdxCongr_apply_apply]
  refine Eq.symm (ComponentIdx.congr_right b' i (PermCond.inv σ _ (σ i)) ?_)
  simp [PermCond.apply_inv_apply]
· rintro rfl
  ext i
  simp only [basisIdxCongr_apply_apply]
  apply ComponentIdx.congr_right
  simp [PermCond.inv_apply_apply]

lemma toField_permT {c c1 : Fin 0 → C} (σ : Fin 0 → Fin 0) (h : PermCond c c1 σ) (t
  toField (permT σ h t) = toField t := by
  induction' t using induction_on_basis with b r t ht t1 t2 h1 h2
  · simp [toField_pure, basis_apply, permT_pure]
  · simp
  · simp [ht]
  · simp [h1, h2]

```

1.74 Physlib/Relativity/Tensors/Constructors.lean

```

have h1 : Fin.funPredPredAbove 1 2 (by simp)
  (prodSwapMap (Nat.succ 0) (0 + 1 + 1)) (by decide) ◦
  id ◦ prodSwapMap 0 (Nat.succ 0) ◦ id = id := by
  dsimp [Fin.funPredPredAbove]
  Fin.funPredPredAbove_id, Function.comp_apply, id_eq]

```

1.75 Physlib/Relativity/Tensors/Contraction/Basic.lean

TODO "docs: The files on contractions of tensors are currently lacking documentation. These should be added, mirroring good examples within Physlib."

namespace Tensor

open Fin

```

Tensor S c →1[k] Tensor S (c ◦ succSuccAbove i j) :=
  contrT n1 i j hij (permT σ hσ t) = permT _ (hσ.succSuccAbove i j hij.1 )
  permT _ (hσ.succSuccAbove i j hij.1 )
  (hij2 : i2 ≠ j2 ∧ S.τ (c (succSuccAbove i1 j1 i2)) = (c (succSuccAbove i1 j1 j2))

```

```

let i2' := (succSuccAbove i1 j1 i2);
let j2' := (succSuccAbove i1 j1 j2);
let i1' := (predPredAbove i2' j2' hi2j2' i1 (by simp [i2', j2']));
let j1' := (predPredAbove i2' j2' hi2j2' j1 (by simp [i2', j2']));
permT id (PermCond.succSuccAbove_comm i1 j1 i2 j2 hij1.left hij2.left)
let i2' := (succSuccAbove i1 j1 i2);
let j2' := (succSuccAbove i1 j1 j2);
let i1' := (predPredAbove i2' j2' (by simp [i2', j2', hij2]) i1
let j1' := (predPredAbove i2' j2' (by simp [i2', j2', hij2]) j1
permT id (PermCond.succSuccAbove_comm i1 j1 i2 j2 hij1.left hij2.left)

```

1.76 Physlib/Relativity/Tensors/Contraction/Basis.lean

```

ComponentIdx (S := S) (c : Fin.succSuccAbove i j) :=
fun m => b (Fin.succSuccAbove i j m)
(b : ComponentIdx (S := S) (c : Fin.succSuccAbove i j)) :
lemma mem_iff_apply_succSuccAbove_eq {n : ℕ} {c : Fin (n + 1 + 1) → C}
(b : ComponentIdx (c : Fin.succSuccAbove i j))
  ∀ m, b' (Fin.succSuccAbove i j m) = b m := by
/-- Given a `b` in `ComponentIdx (c : Fin.succSuccAbove i j)` and
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (S := S) (c : Fin.succSuccAbove i j))
  (b (Fin.predPredAbove i j hij m (by omega)))
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c : Fin.succSuccAbove i j))
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c : Fin.succSuccAbove i j))
lemma ofFin_mem_succSuccAboveSection {n : ℕ} {c : Fin (n + 1 + 1) → C}
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c : Fin.succSuccAbove i j))
  simp only [ofFin, dropPair, Fin.succSuccAbove_ne_fst, ↓reduceDite, Fin.succSuccAbove_eq]
  (b : ComponentIdx (c : Fin.succSuccAbove i j)) :
toFun x := (ofFin hij b x, ofFin_mem_succSuccAboveSection hij b x)
  rcases Fin.eq_or_exists_succSuccAbove i j hij m with rfl | rfl | ⟨m, rfl⟩
  simp only [mem_iff_apply_succSuccAbove_eq] at hb'
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c : Fin.succSuccAbove i j))
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c : Fin.succSuccAbove i j))
basisVector (S := S) (c : Fin.succSuccAbove i j) fun m => b (Fin.succSuccAbove i j m)
(b : ComponentIdx (c : Fin.succSuccAbove i j)) :
(basis (c : Fin.succSuccAbove i j)).repr (contrT n i j h t) b =
(b : ComponentIdx (c : Fin.succSuccAbove i j)) :
(basis (c : Fin.succSuccAbove i j)).repr (contrT n i j h t) b =

lemma contrT_basis {n : ℕ} {c : Fin (n + 1 + 1) → C} {i j : Fin (n + 1 + 1)}
(h : i ≠ j ∧ S.τ (c i) = c j) (b : ComponentIdx (S := S) c) :
contrT n i j h (basis c b) =
Pure.contrPCoeff i j h (Pure.basisVector c b) •
  basis (c : Fin.succSuccAbove i j) (b.dropPair i j) := by

```


1.77. PHYSLIB/RELATIVITY/TENSORS/CONTRACTION/PRODUCTS.lean

```
simp only [basis_apply, contrT_pure, Pure.contrP, Pure.dropPair_basisVector]
rfl
```

1.77 Physlib/Relativity/Tensors/Contraction/Products.lean

```
p1.prodP (dropPair i j hij.1 p) = permP id (PermCond.append_right_succSuccAbove
  (by rw [Fin.succSuccAbove_natAdd_apply_castAdd i j]))]
rw [← congr_right (p1.prodP p) _ (Fin.natAdd n1 (i.succSuccAbove j m))
  (by rw [Fin.succSuccAbove_comm_natAdd i j])]
((permT id (PermCond.append_right_succSuccAbove i j)) <|
  ((permT id (PermCond.append_right_succSuccAbove i j)) <|
  ((permT id (PermCond.append_right_succSuccAbove i j)) <|
  (contrT _ (finSumFinEquiv (m := n1) (Sum.inr i)) (finSumFinEquiv (m := n1) (Sum.
    (by simp using hij) <| prodT t1 t) =
  ((permT id (PermCond.on_id_symm (PermCond.append_right_succSuccAbove i j))) <|
```

1.78 Physlib/Relativity/Tensors/Contraction/Pure.lean

```
open Fin
  Pure S (c ◦ succSuccAbove i j) :=
  fun m => p (succSuccAbove i j m)
rw [succSuccAbove_symm]
  let i2' := (succSuccAbove i1 j1 i2);
  let j2' := (succSuccAbove i1 j1 j2);
  let i1' := (predPredAbove i2' j2' hi2j2' i1 (by simp [i2', j2']));
  let j1' := (predPredAbove i2' j2' hi2j2' j1 (by simp [i2', j2']));
  permP id (PermCond.succSuccAbove_comm i1 j1 i2 j2 hij1 hij2)
rw [succSuccAbove_comm_apply]
  exact Ne.symm (fst_ne_succSuccAbove_pre i j m)
lemma dropPair_update_succSuccAbove {n : ℕ} [inst : DecidableEq (Fin (n + 1 + 1))]
  (x : S.FD.obj (Discrete.mk (c (succSuccAbove i j m)))) :
  dropPair i j hij (p.update (succSuccAbove i j m) x) =
  dropPair i j hij (permP σ hσ p) = permP _ (hσ.succSuccAbove i j hij)
  simp only [Function.comp_apply, dropPair, permP, funPredPredAbove]
lemma contrPCoeff_update_succSuccAbove {n : ℕ} [inst : DecidableEq (Fin (n + 1 + 1))]
  (p : Pure S c) (x : S.FD.obj (Discrete.mk (c (succSuccAbove i j m)))) :
  contrPCoeff i j hij (p.update (succSuccAbove i j m) x) =
  (i j : Fin n) (hij : i ≠ j ∧ S.τ (c (succSuccAbove a b i)) = (c (succSuccAbove a b j)
  p.contrPCoeff (succSuccAbove a b i) (succSuccAbove a b j)
  (hij2 : i2 ≠ j2 ∧ S.τ (c (succSuccAbove i1 j1 i2)) = (c (succSuccAbove i1 j1 j2)
  let i2' := (succSuccAbove i1 j1 i2);
  let j2' := (succSuccAbove i1 j1 j2);
```

```

    let i1' := (predPredAbove i2' j2' hi2j2' i1 (by simp [i2', j2']));
    let j1' := (predPredAbove i2' j2' hi2j2' j1 (by simp [i2', j2']));
    simp only [contrPCoeff_dropPair, succSuccAbove_predPredAbove]
    S.Tensor (c ∘ succSuccAbove i j) :=
    rcases eq_or_exists_succSuccAbove i j hij.1 m with rfl | rfl | ⟨m', rfl⟩
    rcases eq_or_exists_succSuccAbove i j hij.1 m with rfl | rfl | ⟨m', rfl⟩
    (S.Tensor (c ∘ succSuccAbove i j))where

```

1.79 Physlib/Relativity/Tensors/Contraction/SuccSuccAbove.lean

```

/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Mathlib.Data.Finset.Sort
public import Mathlib.Data.Nat.SuccPred
public import Physlib.Meta.TODO.Basic
/-!

# Defining succSuccAbove

In Mathlib there is the `Fin.succAbove` function which gives an embedding of
`Fin (n + 1)` by leaving a hole at a specified index. We will need a version
which gives an embedding of `Fin n` into `Fin (n + 1 + 1)` by leaving holes at
two specified indices. We call this `succSuccAbove`.

We will also need an explicit inverse of this map from
`Fin (n + 1 + 1)` to `Fin n` which is defined on the complement of the two
holes. This is similar to `Fin.predAbove` (although not exactly the same),
for this reason we call it `predPredAbove`.

## Implementation

In previous versions of Physlib the function which is now called `succSuccAbove`
was previously called `dropPairEmb` and the function which is now called `predPredAbove`
was previously called `dropPairEmbPre`.

-/

@[expose] public section

```

1.79.

PHYSLIB/RELATIVITY/TENSORS/CONTRACTION/SUCCSUCCABOVE.lean

namespace Fin

variable {n : ℕ} {c : Fin (n + 1 + 1) → C}

/-!

Defining succSuccAbove

-/

TODO "Determine a way to simplify the definition of `succSuccAbove` using predefined functions from Mathlib, but ensuring tactics such as `decide` still work."

/-- The embedding of `Fin n` into `Fin (n + 1 + 1)` which leaves a hole at `i` and `j`. -/

```
def succSuccAbove (i j : Fin (n + 1 + 1)) (m : Fin n) : Fin (n + 1 + 1) :=
  if m.1 < i.1 ∧ m.1 < j.1 then
    ⟨m, by omega⟩
  else if m.1 + 1 < i.1 ∧ j.1 ≤ m.1 then
    ⟨m + 1, by omega⟩
  else if i.1 ≤ m.1 ∧ m.1 + 1 < j.1 then
    ⟨m + 1, by omega⟩
  else
    ⟨m + 2, by omega⟩
```

```
lemma succSuccAbove_val (i j : Fin (n + 1 + 1)) (m : Fin n) :
  (succSuccAbove i j m).val = if m.1 < i.1 ∧ m.1 < j.1 then m.1
  else if m.1 + 1 < i.1 ∧ j.1 ≤ m.1 then m.1 + 1
  else if i.1 ≤ m.1 ∧ m.1 + 1 < j.1 then m.1 + 1
  else m.1 + 2 := by
  simp only [succSuccAbove]
  grind
```

```
lemma succSuccAbove_self_apply (i : Fin (n + 1 + 1)) (m : Fin n) :
  succSuccAbove i i m = if m.1 < i.1 then ⟨m.1, by omega⟩ else ⟨m.1 + 2, by omega⟩
  simp only [succSuccAbove, and_self]
  grind
```

```
lemma succSuccAbove_eq_succAbove_succAbove (i : Fin (n + 1 + 1)) (j : Fin (n + 1)) :
  succSuccAbove i (i.succAbove j) = i.succAbove ∘ j.succAbove := by
  ext m
  simp only [succSuccAbove, succAbove, lt_def, val_castSucc, Function.comp_apply]
  grind
```

```
lemma succSuccAbove_eq_predAbove {i j : Fin (n + 1 + 1)} (hij : i ≠ j) :
```

```

succSuccAbove i j = fun x => i.succAbove (((Fin.predAbove 0 i).predAbove
rcases Fin.eq_self_or_eq_succAbove i j with rfl | ⟨j, rfl⟩
· contradiction
· ext x
  rw [succSuccAbove_eq_succAbove_succAbove, Function.comp_apply]
  congr
  rcases eq_or_ne i 0 with rfl | hi
  · rfl
  · rw [Fin.predAbove_zero_of_ne_zero hi]
    rcases lt_or_ge (i.pred hi).castSucc (i.succAbove j) with h | h
    · rw [Fin.predAbove_of_castSucc_lt _ h, Fin.pred_succAbove]
      rw [← Fin.lt_succAbove_iff_le_castSucc]
      exact lt_of_le_of_ne ((Fin.castSucc_pred_lt_iff hi).mp h) hij
    · rw [Fin.predAbove_of_le_castSucc _ h, Fin.castPred_succAbove]
      rw [Fin.castSucc_lt_iff_succ_le, ← Fin.succAbove_lt_iff_succ_le]
      exact (Fin.le_castSucc_pred_iff hi).mp h

lemma succSuccAbove_injective {n : ℕ}
  (i j : Fin (n + 1 + 1)) : Function.Injective (succSuccAbove i j) := by
  intro a b
  simp only [Fin.ext_iff, succSuccAbove_val]
  grind (splits := 20)

@[simp]
lemma succSuccAbove_eq_iff_eq {n : ℕ}
  (i j : Fin (n + 1 + 1)) (m1 m2 : Fin n) :
  succSuccAbove i j m1 = succSuccAbove i j m2 ↔ m1 = m2 := by
  rw [(succSuccAbove_injective i j).eq_iff]

@[simp]
lemma succSuccAbove_leq_iff_leq {n : ℕ}
  (i j : Fin (n + 1 + 1)) (m1 m2 : Fin n) :
  succSuccAbove i j m1 ≤ succSuccAbove i j m2 ↔ m1 ≤ m2 := by
  simp only [Fin.le_def, Fin.succSuccAbove_val]
  grind (splits := 20)

@[simp]
lemma succSuccAbove_lt_iff_lt {n : ℕ}
  (i j : Fin (n + 1 + 1)) (m1 m2 : Fin n) :
  succSuccAbove i j m1 < succSuccAbove i j m2 ↔ m1 < m2 := by
  simp only [Fin.lt_def, Fin.succSuccAbove_val]
  grind (splits := 20)

@[simp]
lemma succSuccAbove_monotone {n : ℕ} (i j : Fin (n + 1 + 1)) :
  Monotone (succSuccAbove i j) := by

```

1.79.

PHYSLIB/RELATIVITY/TENSORS/CONTRACTION/SUCCSUCCABOVE.lean

```
intro a b
simp

lemma succSuccAbove_strictMono {n : ℕ} (i j : Fin (n + 1 + 1)) :
  StrictMono (succSuccAbove i j) :=
  (succSuccAbove_monotone i j).strictMono_of_injective (succSuccAbove_injective i j)

@[simp]
lemma succSuccAbove_range {i j : Fin (n + 1 + 1)} (hij : i ≠ j) :
  Set.range (succSuccAbove i j) = {i, j}ᶜ := by
  rcases Fin.eq_self_or_eq_succAbove i j with rfl | ⟨j, rfl⟩
  · simp_all
  rw [succSuccAbove_eq_succAbove_succAbove, Set.range_comp, Fin.range_succAbove]
  ext a
  simp only [Set.mem_compl_iff, Set.mem_singleton_iff, Set.mem_image]
  apply Iff.intro
  · intro h
    obtain ⟨b, h1, rfl⟩ := h
    simpa using h1
  · intro h
    simp at h
    rcases Fin.eq_self_or_eq_succAbove i a with rfl | ⟨a, rfl⟩
    · simp_all
    use a
    simp_all

lemma succSuccAbove_eq_orderEmbOffFin {n : ℕ}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j) :
  succSuccAbove i j = Finset.orderEmbOffFin {i, j}ᶜ
  (by rw [Finset.card_compl]; simp [Finset.card_pair hij]) := by
  apply ((succSuccAbove_strictMono i j).range_inj (OrderEmbedding.strictMono _)).mp
  simp only [succSuccAbove_range hij, Finset.range_orderEmbOffFin, Finset.coe_compl,
    Finset.coe_insert, Finset.coe_singleton]

lemma succSuccAbove_symm (i j : Fin (n + 1 + 1)) :
  succSuccAbove i j = succSuccAbove j i := by
  ext m
  simp only [succSuccAbove_val]
  grind (splits := 5)

@[simp, noint simpVarHead]
lemma permCond_succSuccAbove_symm {c : Fin (n + 1 + 1) → C} (i j : Fin (n + 1 + 1))
  (k : Fin n) : c (succSuccAbove i j k) = c (succSuccAbove j i k) := by
  rw [succSuccAbove_symm]

lemma succSuccAbove_apply_eq_orderIsoOffFin {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (m
```

```

      (succSuccAbove i j) m = (Finset.orderIsoOffFin {i, j}c
        (by rw [Finset.card_compl]; simp [Finset.card_pair hij])) m := by
      simp [succSuccAbove_eq_orderEmbOffFin i j hij]

lemma succSuccAbove_image_compl {i j : Fin (n + 1 + 1)} (hij : i ≠ j)
  (X : Set (Fin n)) :
  (succSuccAbove i j) '' Xc = ({i, j} ∪ succSuccAbove i j '' X)c := by
  rw [← compl_inj_iff, Function.Injective.compl_image_eq (succSuccAbove_inj)]
  simp only [compl_compl, succSuccAbove_range hij]
  exact Set.union_comm ((succSuccAbove i j) '' X) {i, j}

@[simp]
lemma fst_ne_succSuccAbove_pre (i j : Fin (n + 1 + 1)) (m : Fin n) :
  ¬ i = succSuccAbove i j m := by
  simp only [Fin.ext_iff, succSuccAbove_val]
  grind (splits := 5)

@[simp]
lemma succSuccAbove_ne_fst (i j : Fin (n + 1 + 1)) (m : Fin n) :
  ¬ succSuccAbove i j m = i := by
  simp only [Fin.ext_iff, succSuccAbove_val]
  grind (splits := 5)

@[simp]
lemma snd_ne_succSuccAbove_pre (i j : Fin (n + 1 + 1)) (m : Fin n) :
  ¬ j = (succSuccAbove i j) m := by
  rw [succSuccAbove_symm]
  exact fst_ne_succSuccAbove_pre j i m

@[simp]
lemma succSuccAbove_ne_snd (i j : Fin (n + 1 + 1)) (m : Fin n) :
  ¬ succSuccAbove i j m = j := by
  apply Ne.symm
  simp

lemma eq_or_exists_succSuccAbove (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m :
  m = i ∨ m = j ∨ ∃ m', m = succSuccAbove i j m' := by
  by_cases h : m = i
  · simp [h]
  · by_cases h' : m = j
    · simp [h']
    · obtain ⟨m', rfl⟩ : ∃ y, succSuccAbove i j y = m := by
      simp_all [← Set.mem_range, succSuccAbove_eq_orderEmbOffFin]
      simp

lemma succSuccAbove_apply_lt_lt {n : ℕ}

```

1.79.

PHYSLIB/RELATIVITY/TENSORS/CONTRACTION/SUCCSUCCABOVE.lean

```
(i j : Fin (n + 1 + 1)) (m : Fin n) (hi : m.val < i.val) (hj : m.val < j.val) :
  succSuccAbove i j m = m.castSucc.castSucc := by
  ext
  simp [succSuccAbove, hi, hj]

lemma succSuccAbove_natAdd_apply_castAdd {n n1 : ℕ}
  (i j : Fin (n + 1 + 1)) (m : Fin n1) :
  (succSuccAbove (n := n1 + n) (Fin.natAdd n1 i) (Fin.natAdd n1 j))
  (Fin.castAdd n m) = Fin.castAdd (n + 1 + 1) (m) := by
  simp only [Fin.ext_iff, succSuccAbove_val, natAdd, castAdd]
  grind (splits := 20)

lemma succSuccAbove_natAdd_image_range_castAdd {n n1 : ℕ}
  (i j : Fin (n + 1 + 1)) :
  (succSuccAbove (n := n1 + n) (Fin.natAdd n1 i) (Fin.natAdd n1 j)) ''
  (Set.range (Fin.castAdd (m := n) (n := n1))) = {i | i.1 < n1} := by
  ext a
  simp only [Set.mem_image, Set.mem_range, exists_exists_eq_and, Set.mem_setOf_eq]
  conv_lhs =>
    enter [1, b]
    rw [succSuccAbove_natAdd_apply_castAdd i j]
  apply Iff.intro
  · rintro ⟨b, rfl⟩
    simp
  · exact fun h => ⟨⟨a, by omega⟩, by simp⟩

lemma succSuccAbove_comm_natAdd {n n1 : ℕ}
  (i j : Fin (n + 1 + 1)) (m : Fin n) :
  succSuccAbove (n := n1 + n) (Fin.natAdd n1 i) (Fin.natAdd n1 j) (Fin.natAdd n1 m)
  = Fin.natAdd (n1) (succSuccAbove i j m) := by
  simp only [succSuccAbove, val_natAdd, add_lt_add_iff_left, add_le_add_iff_left, Fin]
  grind

/-!

## predPredAbove

-/

/-- The preimage of `m` under `succSuccAbove i j hij` given that `m` is not equal
    to `i` or `j`. -/
def predPredAbove (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin (n + 1 + 1))
  (hm : m ≠ i ∧ m ≠ j) : Fin n :=
  if h1 : m.1 < i.1 ∧ m.1 < j.1 then
    ⟨m, by grind⟩
  else if h2 : m.1 - 1 < i.1 ∧ j.1 ≤ m.1 then
```

```

      ⟨m - 1, by grind⟩
    else if h3 : i.1 - 1 ≤ m.1 ∧ m.1 < j.1 then
      ⟨m - 1, by grind⟩
    else
      ⟨m - 2, by grind⟩

lemma predPredAbove_val (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin (n + 1 + 1)) (hm : m ≠ i ∧ m ≠ j) :
  (predPredAbove i j hij m hm).val = if m.1 < i.1 ∧ m.1 < j.1 then m.1
  else if m.1 - 1 < i.1 ∧ j.1 ≤ m.1 then m.1 - 1
  else if i.1 - 1 ≤ m.1 ∧ m.1 < j.1 then m.1 - 1
  else m.1 - 2 := by
  simp only [predPredAbove]
  grind

@[simp]
lemma succSuccAbove_predPredAbove (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin (n + 1 + 1)) (hm : m ≠ i ∧ m ≠ j) :
  succSuccAbove i j (predPredAbove i j hij m hm) = m := by
  dsimp [succSuccAbove, predPredAbove]
  grind

set_option backward.isDefEq.respectTransparency false in
lemma predPredAbove_eq_orderIsoOfFin (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (hm : m ≠ i ∧ m ≠ j) :
  predPredAbove i j hij m hm =
  (Finset.orderIsoOfFin {i, j}ᶜ (by rw [Finset.card_compl]; simp [Finset.card_compl])) (m, by simp [hm]) := by
  apply succSuccAbove_injective i j
  conv_rhs => rw [succSuccAbove_apply_eq_orderIsoOfFin hij]
  simp

@[simp]
lemma predPredAbove_injective (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m1 m2 : Fin (n + 1 + 1)) (hm1 : m1 ≠ i ∧ m1 ≠ j) (hm2 : m2 ≠ i ∧ m2 ≠ j) :
  predPredAbove i j hij m1 hm1 = predPredAbove i j hij m2 hm2 ↔ m1 = m2 := by
  rw [← Function.Injective.eq_iff (succSuccAbove_injective i j)]
  simp

lemma predPredAbove_surjective (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin n) :
  ∃ m' : Fin (n + 1 + 1), ∃ (h : m' ≠ i ∧ m' ≠ j),
  predPredAbove i j hij m' h = m := by
  refine ⟨succSuccAbove i j m, by simp [Ne.symm], succSuccAbove_injective i j m, by simp⟩

@[simp]

```


1.79.

PHYSLIB/RELATIVITY/TENSORS/CONTRACTION/SUCCSUCCABOVE. ~~LEAN~~

```
lemma predPredAbove_succSuccAbove (i j : Fin (n + 1 + 1)) (hij : i ≠ j)
  (m : Fin n) :
  predPredAbove i j hij (succSuccAbove i j m) (by simp) = m := by
  apply succSuccAbove_injective i j
  simp
```

/-!

Commutativity of succSuccAbove

-/

```
lemma succSuccAbove_comm (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
  (hij1 : i1 ≠ j1) (hij2 : i2 ≠ j2) :
  let i2' := (succSuccAbove i1 j1 i2);
  let j2' := (succSuccAbove i1 j1 j2);
  have hi2j2' : i2' ≠ j2' := by simp [i2', j2', hij2];
  let i1' := (predPredAbove i2' j2' hi2j2' i1 (by simp [i2', j2']));
  let j1' := (predPredAbove i2' j2' hi2j2' j1 (by simp [i2', j2']));
  succSuccAbove i1 j1 ∘ succSuccAbove i2 j2 =
  succSuccAbove i2' j2' ∘ succSuccAbove i1' j1' := by
  ext m
  simp only [Function.comp_apply, predPredAbove_val, succSuccAbove_val]
  grind (splits := 20)
```

```
lemma succSuccAbove_comm_apply (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
  (hij1 : i1 ≠ j1) (hij2 : i2 ≠ j2) (m : Fin n) :
  let i2' := (succSuccAbove i1 j1 i2);
  let j2' := (succSuccAbove i1 j1 j2);
  have hi2j2' : i2' ≠ j2' := by simp [i2', j2', hij2];
  let i1' := (predPredAbove i2' j2' hi2j2' i1 (by simp [i2', j2']));
  let j1' := (predPredAbove i2' j2' hi2j2' j1 (by simp [i2', j2']));
  succSuccAbove i2' j2' (succSuccAbove i1' j1' m) =
  succSuccAbove i1 j1 (succSuccAbove i2 j2 m) := by
  intro i2' j2' hi2j2' i1' j1'
  change _ = (succSuccAbove i1 j1 ∘ succSuccAbove i2 j2) m
  rw [succSuccAbove_comm i1 j1 i2 j2 hij1 hij2]
  rfl
```

/-!

funPredPredAbove

-/

/-- Given a bijection $\text{Fin } (n_1 + 1 + 1) \rightarrow \text{Fin } (n + 1 + 1)$ and a pair $i j : \text{Fin } (n)$ then $\text{funPredPredAbove } i j _ \sigma _ : \text{Fin } n_1 \rightarrow \text{Fin } n$ corresponds to the induced bijection

```

    formed by dropping `i` and `j` in the source and their image in the target
  /
def funPredPredAbove {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j)
  (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ)
  (m : Fin n1) : Fin n :=
  predPredAbove (σ i) (σ j)
  (by simp [hσ.injective.eq_iff, hij])
  (σ (succSuccAbove i j m)) (by simp [hσ.injective.eq_iff, Ne.symm])

lemma funPredPredAbove_injective {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hij :
  (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ) :
  Function.Injective (funPredPredAbove i j hij σ hσ) := by
  intro m1 m2 h
  simp [funPredPredAbove, hσ.injective.eq_iff] using h

lemma funPredPredAbove_surjective {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hij :
  (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ) :
  Function.Surjective (funPredPredAbove i j hij σ hσ) := by
  intro m
  simp only [funPredPredAbove]
  obtain ⟨m, hm, rfl⟩ := predPredAbove_surjective (σ i) (σ j)
  (by simp [hσ.injective.eq_iff, hij]) m
  simp only [predPredAbove_injective]
  obtain ⟨m', rfl⟩ := hσ.surjective m
  simp only [ne_eq, hσ.injective.eq_iff] at hm ⊢
  rcases eq_or_exists_succSuccAbove i j hij m' with rfl | rfl | ⟨m'', rfl⟩
  · simp_all
  · simp_all
  · exact ⟨m'', rfl⟩

lemma funPredPredAbove_bijective {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hij :
  (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ) :
  Function.Bijective (funPredPredAbove i j hij σ hσ) := by
  apply And.intro
  · apply funPredPredAbove_injective
  · apply funPredPredAbove_surjective

@[simp]
lemma funPredPredAbove_id {n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j)
  funPredPredAbove i j hij id (Function.bijective_id) = id := by
  ext1 m
  simp [funPredPredAbove]

end Fin

```

1.80 Physlib/Relativity/Tensors/Evaluation.lean

```

public import Physlib.Relativity.Tensors.Product
lemma evalPCoeff_basisVector (i : Fin (n + 1)) (b : basisIdx (c i)) (b' : ComponentIdx c) :
  evalPCoeff i b (Pure.basisVector c b') = if b' i = b then (1 : k) else 0 := by
  simp [evalPCoeff, basisVector, Finsupp.single_apply]

lemma evalT_basis {n : ℕ} {c : Fin (n + 1) → C} (i : Fin (n + 1))
  (b : ComponentIdx c) (x : basisIdx (c i)) :
  evalT i x (basis (S := S) c b) = if b i = x then basis (c ∘ i.succAbove)
    (fun j => b (i.succAbove j)) else 0 := by
  simp only [basis_apply, evalT_pure, Pure.evalP, Pure.evalPCoeff_basisVector, ite_zero_smul]
  rfl

```

1.81 Physlib/Relativity/Tensors/RealTensor/Basic.lean

```

public import Physlib.Relativity.Tensors.Elab
meta import Mathlib.Tactic.Cases

lemma basisIdxCongr_eq_refl {d : ℕ} {c1 c2 : realLorentzTensor.Color} (h : c1 = c2) :
  TensorSpecies.basisIdxCongr (basisIdx := fun _ => Fin 1 ⊕ Fin d) h = Equiv.refl
  rfl

simp
  (t : ℝT(d, c)) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j)) :
  (basis (c ∘ Fin.succSuccAbove i j)).repr (contrT n i j h t) b =
  (t : ℝT(d, c)) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j)) :
  (basis (c ∘ Fin.succSuccAbove i j)).repr (contrT n i j h t) b =
  /-!

## Contractions and to Field

-/

attribute [-simp] Fintype.sum_sum_type

lemma contrPCoeff_basis {d n : ℕ} {c : Fin n → realLorentzTensor.Color} (i j : Fin n)
  (hij : i ≠ j ∧ (realLorentzTensor d).τ (c i) = c j)
  (b : ComponentIdx (S := realLorentzTensor d) c) :
  Pure.contrPCoeff i j hij (Pure.basisVector c b) = if b i = b j then 1 else 0 :=

```

```

simp only [Pure.contrPCoeff, Monoidal.tensorUnit_obj, Rep.tensorUnit_V, R
  Functor.comp_obj, Discrete.functor_obj_eq_as, Function.comp_apply, Pure
generalize b i = b1 at *
generalize b j = b2 at *
suffices h : ∀ c, ∀ c1, (hc : (realLorentzTensor d).τ c = c1) →
  (realLorentzTensor d).castToField ((ConcreteCategory.hom
    ((realLorentzTensor d).contr.app { as :=c })))
    (((realLorentzTensor d).basis c) b1 ⊗t [ℝ]
      (ConcreteCategory.hom ((realLorentzTensor d).FD.map (eqToHom (by simp
        (((realLorentzTensor d).basis c1) b2)))) =
    if b1 = b2 then 1 else 0 by
    exact h (c i) (c j) hij.2
rintro c c1 rfl
exact contr_basis _ _

lemma contrT_eq_sum_evalT {n} {d} (c : Fin (n + 1 + 1) → Color) (i j : Fin
  (h : i ≠ j ∧ (realLorentzTensor d).τ (c i) = c j) (t : ℝT(d, c)) :
  contrT n i j h t = ∑ (μ : Fin 1 ⊗ Fin d), permT id (by
    simp [Fin.succSuccAbove_eq_predAbove h.1])
    (evalT ((Fin.predAbove 0 i).predAbove j) μ (evalT i μ t)) := by
  induction' t using Tensor.induction_on_basis with b r t h t1 t2 h1 h2
  · rw [contrT_basis]
    simp only [contrPCoeff_basis, ite_smul, one_smul, zero_smul]
    conv_rhs =>
      enter [2, μ];
      simp only [evalT_basis, Fin.zero_succAbove, apply_ite, Fin.succ_zero_
    simp only [Finset.sum_ite_eq, Finset.mem_univ, ↓reduceIte]
    have h0 : i.succAbove ((Fin.predAbove 0 i).predAbove j) = j := by
      rcases i.eq_zero_or_eq_succ with rfl | ⟨k, rfl⟩
      · exact Fin.succAbove_predAbove (Ne.symm h.1)
      · rw [Fin.predAbove_zero_succ]
        exact Fin.succ_succAbove_predAbove (Ne.symm h.1)
    rw [h0]
    split_ifs with h1 _ h2
    · rw [permT_basis]
      congr
      ext x
      simp only [Function.comp_apply, ComponentIdx.dropPair, id_eq, basisId
        Equiv.refl_apply]
      rw [Fin.succSuccAbove_eq_predAbove h.1]
      · symm at h1; contradiction
      · symm at h2; contradiction
      · rfl
    · simp
    · simp [Finset.smul_sum, h]
    · simp [h1, h2, Finset.sum_add_distrib]

```

1.82.

PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/METRICS/BASIC.lean

```
lemma contrT_toField {d} (c : Fin 2 → Color)
  (h : 0 ≠ 1 ∧ (realLorentzTensor d).τ (c 0) = c 1) (t : ℝT(d, c)) :
  (contrT 0 0 1 h t).toField = ∑ (μ : Fin 1 ⊕ Fin d), {t | [μ] [μ]}τ.toField := by
  rw [contrT_eq_sum_evalT, map_sum, Tensorial.self_toTensor_apply]
  congr
  ext μ
  simp only [toField_permT]
  rfl
```

1.82 Physlib/Relativity/Tensors/RealTensor/Metrics/Basic.lean

```
erw [TensorSpecies.metricTensor_invariant]
erw [TensorSpecies.metricTensor_invariant]
```

1.83 Physlib/Relativity/Tensors/RealTensor/ToComplex.lean

```
/-- `complexify` commutes with precomposition by `succSuccAbove`.
We use `fun k => b (Fin.succSuccAbove i j k)` and direct application
`(ComponentIdx.complexify b) (Fin.succSuccAbove i j m)` rather than composition so
lemma ComponentIdx.complexify_comp_succSuccAbove
  (ComponentIdx.complexify (c := c ∘ Fin.succSuccAbove i j)
    (fun k => b (Fin.succSuccAbove i j k))) m =
  (ComponentIdx.complexify (c := c) b) (Fin.succSuccAbove i j m) := by
  toComplex (c := c ∘ Fin.succSuccAbove i j)
let c' := c ∘ Fin.succSuccAbove i j
  ← Tensor.basis_apply (S := realLorentzTensor) c' (fun k => b (Fin.succSuccAbove
    toComplex_basis (c := c') (i := fun k => b (Fin.succSuccAbove i j k)))
  · -- complexify(fun k => b (succSuccAbove k)) = (complexify b) ∘ succSuccAbove
    (c := (colorToComplex ∘ c) ∘ Fin.succSuccAbove i j)
    (b := fun m => (ComponentIdx.complexify b) (Fin.succSuccAbove i j m))]
  refine congr_arg _ (funext fun m => ComponentIdx.complexify_comp_succSuccAbove b
    toComplex (c := c ∘ Fin.succSuccAbove i j) (contrT (S := realLorentzTensor) n i)
let c' : Fin n → realLorentzTensor.Color := c ∘ Fin.succSuccAbove i j
```

1.84 Physlib/Relativity/Tensors/RealTensor/Vector/Basic.lean

```
lemma ext_of_apply {d} {v w : Vector d} (h : ∀ i, v i = w i) : v = w := by
  apply (equivEuclid d).injective
  ext i
  simpa using h i
```

1.85 Physlib/Relativity/Tensors/TensorSpecies/Basic.lean

```

public import Physlib.Meta.TODO.Basic

unit :  $\square$  (Discrete C  $\square$  Rep k G)  $\square$  OverColor.Discrete. $\tau$ Pair FD  $\tau$ 
/-!

## Non-categorical constructor

-/
open scoped TensorProduct

TODO "Replace the definition of `TensorSpecies` with the
      construction from functions `TensorSpecies.ofFunctions`."

/-- The creation of an element of `TensorSpecies` from functions rather
    than categorical objects. -/
def ofFunctions [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
  /- The vector space associated with each color. -/
  (V : C  $\rightarrow$  Type) [V c, AddCommGroup (V c)] [V c, Module k (V c)]
  /- The representation associated with each color. -/
  (rep : (c : C)  $\rightarrow$  Representation k G (V c))
  /- The basis associated with each color. -/
  (basis : (c : C)  $\rightarrow$  Basis (basisIdx c) k (V c))
  /- The map taking each color to its dual. -/
  ( $\tau$  : C  $\rightarrow$  C)
  ( $\tau$ _involution : Function.Involutive  $\tau$ )
  /- The contraction of vectors with dual colors. -/
  (contr : (c : C)  $\rightarrow$  ((rep c).tprod (rep ( $\tau$  c))).IntertwiningMap (Represent
  /- The invariant unit tensor for a given color. -/
  (unit : (c : C)  $\rightarrow$  ((Representation.trivial k G k)).IntertwiningMap ((re
  /- The invariant metric tensor for a given color. -/
  (metric : (c : C)  $\rightarrow$  ((Representation.trivial k G k)).IntertwiningMap ((
  /- Contraction is symmetric with respect to duals. -/

```

```

(contr_tmul_symm : ∀ c (x : V c) (y : V (τ c)),
  contr c (x ⊗t[k] y) = contr (τ c) (y ⊗t Equiv.cast (congrArg V (τ_involution c)
/- The unit is symmetric. -/
(unit_symm : ∀ c, unit c (1 : k) =
  LinearMap.lTensor _ (LinearEquiv.cast (τ_involution c)).toLinearMap
  (TensorProduct.comm k _ _ (unit (τ c) (1 : k))))
/- Contraction with unit leaves invariant. -/
(contr_unit : ∀ c (x : V c),
  (TensorProduct.lid k _ <|
    (contr c).toLinearMap.rTensor _ <|
    (TensorProduct.assoc k (V c) (V (τ c)) (V c)).symm <|
    x ⊗t[k] (unit c (1 : k))) = x)
/- On contracting metrics we get the unit. -/
(contr_metric : ∀ c,
  (TensorProduct.comm k _ _ <|
    (TensorProduct.lid k _).lTensor _ <|
    ((contr c).toLinearMap.rTensor (V (τ c))).lTensor (V c) <|
    (TensorProduct.assoc k (V c) (V (τ c)) (V (τ c))).symm.toLinearMap.lTensor (
    TensorProduct.assoc k (V c) (V c) (V (τ c) ⊗t[k] V (τ c)) <|
    (metric c 1) ⊗t[k] (metric (τ c) 1)) = unit c (1 : k)) :
  TensorSpecies k C G basisIdx where
FD := Discrete.functor (fun x => Rep.of (rep x))
basis := basis
τ := τ
τ_involution := τ_involution
contr := Discrete.natTrans fun c => Rep.ofHom <| contr c.as
unit := Discrete.natTrans fun c => Rep.ofHom <| unit c.as
metric := Discrete.natTrans fun c => Rep.ofHom <| metric c.as
contr_tmul_symm c x y := by
  simp [Rep.ofHom, Rep.Hom.hom, eqToHom_map]
  convert contr_tmul_symm c x y
  generalize_proofs h1 h2
  congr 1
  have τ_inv := τ_involution c
  generalize τ (τ c) = c' at *
  subst τ_inv
  rfl
unit_symm c := by
  simp [Rep.ofHom, Rep.Hom.hom]
  change unit c (1 : k) = _
  rw [unit_symm c]
  congr
  generalize_proofs h1 h2 h3
  have τ_inv := τ_involution c
  generalize τ (τ c) = c' at *
  subst τ_inv

```

```

      rfl
    contr_metric c := by simpa [Rep.ofHom, Rep.Hom.hom] using contr_metric c
    contr_unit c x := by simpa [Rep.ofHom, Rep.Hom.hom] using contr_unit c x

/-!

## Properties of TensorSpecies

-/

```

1.86 Physlib/Relativity/Tensors/Tensorial.lean

```

open Tensor in
lemma prod_basis_of_map_reindex {n2 : ℕ} {c2 : Fin n2 → C} {M2 : Type}
  [Tensorial S c M] [AddCommMonoid M2] [Module k M2]
  [Tensorial S c2 M2] {ι ι2 : Type} {b : Module.Basis ι k M} {b2 : Modul
  {e : Tensor.ComponentIdx c ≈ ι} {e2 : Tensor.ComponentIdx c2 ≈ ι2}
  (h : b = ((Tensor.basis (S := S) c).map toTensor.symm).reindex e)
  (h2 : b2 = ((Tensor.basis (S := S) c2).map toTensor.symm).reindex e2)
  b.tensorProduct b2 = ((Tensor.basis (S := S) (Fin.append c c2)).map
  (toTensor (S := S) (M := M ⊗[k] M2)).symm).reindex
  (ComponentIdx.prod.trans (e.prodCongr e2))) := by
  ext {i, j}
  simp_rw [Tensorial.basis_map_prod, Module.Basis.tensorProduct_apply]
  simp [h, h2]

open Tensor in
lemma prod_tensor_basis_eq_map_reindex {n2 : ℕ} {c2 : Fin n2 → C} {M2 : Typ
  [Tensorial S c M] [AddCommMonoid M2] [Module k M2]
  [Tensorial S c2 M2] {ι ι2 : Type} {b : Module.Basis ι k M} {b2 : Modul
  {e : Tensor.ComponentIdx c ≈ ι} {e2 : Tensor.ComponentIdx c2 ≈ ι2}
  (h : b = ((Tensor.basis (S := S) c).map toTensor.symm).reindex e)
  (h2 : b2 = ((Tensor.basis (S := S) c2).map toTensor.symm).reindex e2)
  Tensor.basis (S := S) (Fin.append c c2) =
  ((b.tensorProduct b2).map (toTensor (S := S) (M := M ⊗[k] M2))).reindex
  (ComponentIdx.prod.trans (e.prodCongr e2)).symm := by
  rw [Tensor.basis_prod_eq]
  ext r
  simp
  obtain (<{i, j}>, rfl) := ComponentIdx.prod.symm.surjective r
  simp [h, h2, tensorEquivProd, toTensor_tprod]

```


1.87 Physlib/SpaceAndTime/Space/Derivatives/Curl.lean

```

@[to_fun]
@[to_fun]
@[to_fun]
@[to_fun]
lemma eq_neg_curl_of_div_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff
  f = - curl (fun x => ∫ (t : ℝ) in 0..1, (t • basis.repr x) ×e3 f (t • x) ∂(volume)
  change f = -curl fun x => ∫ (t : ℝ) in 0..1, homotopyOperatorIntegrand f x t
lemma exists_curl_of_div_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff
  (hdiv : ∇ □ f = 0) :
  ∃ g : Space → EuclideanSpace ℝ (Fin 3), f = curl g ∧ Differentiable ℝ g := by
  suffices hneg : ∃ g : Space → EuclideanSpace ℝ (Fin 3), f = - curl g ∧ Differentiable
  obtain ⟨g, hcurl, hdifferentiable⟩ := hneg
  use -g
  subst f
  simp_all
  rw [curl_neg]
  fun_prop
have f_differentiable : Differentiable ℝ f := hf.differentiable (by simp)
have fderiv_f_t (x : Space) (t : ℝ)
  (i : Fin 3) : (fderiv ℝ (fun t => (f (t • x)).ofLp i) t) 1 = fderiv ℝ f (t • x)
  change (fderiv ℝ (EuclideanSpace.proj i ∘ f ∘ fun (t : ℝ) => t • x) t) 1 = _
  rw [fderiv_comp _ (by fun_prop) (by fun_prop), fderiv_comp _ (by fun_prop) (by f
    fderiv_fun_smul (by fun_prop) (by fun_prop)]
  simp only [Function.comp_apply, ContinuousLinearMap.fderiv, fderiv_fun_const, Pi
    fderiv_id', ContinuousLinearMap.coe_comp', ContinuousLinearMap.add_apply,
    ContinuousLinearMap.coe_smul', Pi.smul_apply, ContinuousLinearMap.zero_apply,
    ContinuousLinearMap.smulRight_apply, ContinuousLinearMap.coe_id', id_eq, one_s
    PiLp.proj_apply]
have hi (x : Space) (i : Fin 3) : ∫ (t : ℝ) in 0..1, (t * f (t • x) i * 2) -
  t * (- fderiv ℝ f (t • x) (t • x)) i ∂(volume) = f x i := by
  trans ∫ (t : ℝ) in 0..1, fderiv ℝ (fun t => t ^ 2 * f (t • x) i) t 1 ∂(volume)
  · congr
  funext t
  rw [fderiv_fun_mul (by fun_prop) (by fun_prop)]
  simp [fderiv_f_t]
  ring
  simp only [fderiv_eq_smul_deriv, smul_eq_mul, one_mul]
  rw [intervalIntegral.integral_deriv_eq_sub (by fun_prop)]
  simp only [one_pow, one_smul, one_mul, ne_eq, OfNat.ofNat_ne_zero, not_false_eq
    zero_smul, zero_mul, sub_zero]
  · apply Continuous.intervalIntegrable
  fun_prop
use fun x => ∫ (t : ℝ) in 0..1, homotopyOperatorIntegrand f x t ∂(volume)
apply And.intro

```

```

swap
· intro x
  exact (hasFDerivAt_intervalIntegral_homotopyOperatorIntegrand (hf) _).d
· exact eq_neg_curl_of_div_zero f hf hdiv

TODO "Generalize the statement that a div-free field is a curl
      to time-dependent fields."

/-- A constructive form of the statement that if the curl of a function is
    then it is equal to the grad of another function.

    In the context of e.g. electromagnetism the potential given by this lemma
    to that defined by the work done to move a unit charge from the origin to
    point `x` along a straight line. -/
lemma eq_grad_integral_of_curl_zero (f : Space → EuclideanSpace ℝ (Fin 3))
  (hcurl : curl f = 0) :
  f = grad (fun x => ∫ t in (0 : ℝ)..1, f (t • x), basis.repr x ▯ ℝ ∂(vol
obtain {g, {rfl, hg}} := exists_grad_of_curl_zero f
  (hf.differentiable (by simp)) (by simp [hcurl])
suffices h1 : ContDiff ℝ 1 g by
  nth_rewrite 1 [eq_integral_grad h1]
  simp
rw [contDiff_one_iff_hasFDerivAt]
use fun x => ((toDual ℝ Space) (basis.repr.symm (∇ g x)))
apply And.intro
· fun_prop
intro x
exact hasGradientAt_iff_hasFDerivAt.mpr (DifferentiableAt.hasGradientAt_g

TODO "Generalize the statement that a curl-free field is a gradient
      to time-dependent fields."

```

1.88 Physlib/SpaceAndTime/Space/Derivatives/DerivativeIndex.lean

```

/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/
module

public import Mathlib.Algebra.Order.BigOperators.Group.Finset

```

1.88.

PHYSLIB/SPACEANDTIME/SPACE/DERIVATIVES/DERIVATIVEINDEX189

public import Physlib.SpaceAndTime.Space.Derivatives.MultiIndex

/-!

Bounded derivative indices

i. Overview

This module defines bounded derivative indices on `Space d`, i.e. multi-indices of total order at most `k`.

These are the finite indexing objects used later for finite-order local jet coordinates, while remaining independent of any specific jet or field-theory construction.

ii. Key results

- `Physlib.DerivativeIndex`

iii. Table of contents

- A. Definition and basic instances

iv. References

-/

@[expose] public section

namespace Physlib

/-!

A. Definition and basic instances

-/

/-- The multi-indices on `d` coordinates of order at most `k`. -
/

abbrev DerivativeIndex (d k : ℕ) := { I : MultiIndex d // I.order ≤ k }

noncomputable instance (d k : ℕ) : Fintype (DerivativeIndex d k) :=
 Fintype.ofInjective

(fun I : DerivativeIndex d k => fun i : Fin d =>
 ((I.1 i, by
 have hle_order : I.1 i ≤ I.1.order := by
 classical
 unfold Physlib.MultiIndex.order

```

      simp using
        (Finset.single_le_sum (fun j _ => Nat.zero_le (I.1 j)) (by simp
          I.1 i ≤ ∑ j : Fin d, I.1 j)
        exact Nat.lt_succ_of_le (le_trans hle_order I.2))) : Fin (k + 1)))
    (by
      intro I J h
      apply Subtype.ext
      apply Physlib.MultiIndex.ext
      intro i
      exact congrArg Fin.val (congrFun h i))

instance (d k : ℕ) : DecidableEq (DerivativeIndex d k) := inferInstance

instance (d k : ℕ) : Zero (DerivativeIndex d k) where
  zero := ⟨0, by simp [Physlib.MultiIndex.order_zero]⟩

namespace DerivativeIndex

@[simp]
lemma coe_zero (d k : ℕ) :
  ((0 : DerivativeIndex d k) : MultiIndex d) = 0 := rfl

@[simp]
lemma zero_apply (d k : ℕ) (i : Fin d) :
  ((0 : DerivativeIndex d k) : MultiIndex d) i = 0 := rfl

end DerivativeIndex

end Physlib

```

1.89 Physlib/SpaceAndTime/Space/Derivatives/Grad.lean

```

@[to_fun]
@[simp]
lemma grad_fun_add_const (f : Space d → ℝ) (c : ℝ) :
  ∇ (fun x => f x + c) = ∇ f := by
  ext x i
  simp [grad, deriv]

@[to_fun]
@[to_fun]
lemma grad_smul_inner_space {d} (x : Space d) (f : Space d → ℝ) (hd : Diffe
  (ht : 0 < t) :
  ∀ f (t • x), basis.repr x|_ℝ = _root_.deriv (fun r => f (r • x)) t :=

```

```

by_cases hx : ||x|| = 0
· simp at hx
  simp [hx]
calc _
  _ = ||x|| *  $\nabla f(t \cdot x)$ ,  $\|x\|^{-1} \cdot \text{basis.repr } x \mathbb{R} := \text{by}$ 
    simp [inner_smul_right]
    grind
  _ = ||x|| *  $\nabla f(t \cdot x)$ ,  $\|t \cdot x\|^{-1} \cdot \text{basis.repr } (t \cdot x) \mathbb{R} := \text{by}$ 
    simp [norm_smul, smul_smul]
    grind
  _ = ||x|| * _root_.deriv (fun r => f (r •  $\|t \cdot x\|^{-1} \cdot t \cdot x$ ))  $\|t \cdot x\| := \text{by}$ 
    rw [grad_inner_space_unit_vector _ _ hd]
  _ = ||x|| * _root_.deriv (fun r => f (r •  $\|x\|^{-1} \cdot x$ )) (t * ||x||) := by
    simp [smul_smul, norm_smul]
    grind
  _ = ||x|| * _root_.deriv ((fun r => f (r • x)) • (fun r =>  $\|x\|^{-1} \cdot r$ )) (t * ||x||)
    congr
    funext r
    simp [smul_smul]
    grind
  _ = _root_.deriv (fun r => f (r • x)) t := by
    rw [deriv_comp _ (by fun_prop) (by fun_prop)]
    rw [deriv_const_mul _ (by fun_prop)]
    simp only [deriv_id'', mul_one]
    grind

```

open MeasureTheory

```

lemma eq_integral_grad {f : Space → ℝ} (hf : ContDiff ℝ 1 f) :
  f = (fun x =>  $\int t \text{ in } (0 : \mathbb{R})..1$ ,  $\nabla f(t \cdot x)$ , basis.repr x  $\mathbb{R}$   $\partial(\text{volume})$ 
    + f 0) := by
  ext x
  have h' := (hf.differentiable (by simp))
  by_cases hx : ||x|| = 0
  · simp at hx
    subst hx
    simp
  symm
  calc _
  _ =  $\int (t : \mathbb{R}) \text{ in } 0..1$ , (_root_.deriv (fun r => f (r • x)) t)  $\partial \text{volume}$  + f 0 := by
    congr 1
    refine intervalIntegral.integral_congr_ae_restrict ?_
    refine (ae_eq_restrict_iff_indicator_ae_eq measurableSet_uIoc).mpr ?_
    filter_upwards with p
    simp only [Set.indicator, zero_le_one, Set.uIoc_of_le, Set.mem_Ioc]
    split_ifs

```

```

      rw [grad_smul_inner_space]
      · exact h'
      · grind
      · grind
    _ = (f (1 • x) - f 0) + f 0 := by
      rw [intervalIntegral.integral_deriv_eq_sub (by fun_prop)]
      simp only [one_smul, zero_smul, sub_add_cancel]
      · apply Continuous.intervalIntegrable
        fun_prop
    simp

```

1.90 Physlib/SpaceAndTime/Space/Derivatives/MatrixDiv.lean

```

/-
Copyright (c) 2025 Florian Wiesner. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Florian Wiesner
-/
module

public import Physlib.SpaceAndTime.Space.Derivatives.Div
/-!

# Matrix divergence on Space

## i. Overview

In this module we define the matrix divergence operator on matrix-
valued
functions from `Space d`.

For a field `T : Space d → Matrix (Fin d) (Fin d) ℝ`, the matrix divergence
the vector field whose `i`th component is


$$\sum_j \partial[j] (fun x => T x i j) x$$
.

## ii. Key results

- `matrixDiv` : The divergence of a matrix-valued function on `Space d`.

## iii. Table of contents

- A. The matrix divergence on functions

```

1.90.

PHYSLIB/SPACEANDTIME/SPACE/DERIVATIVES/MATRIXDIV.LEAN 193

- A.1. Basic equalities
- A.2. The matrix divergence on the zero function
- A.3. The matrix divergence on a constant function
- A.4. The matrix divergence distributes over addition
- A.5. The matrix divergence distributes over scalar multiplication

iv. References

-/

@[expose] public section

open Physlib

namespace Space

/-!

A. The matrix divergence on functions

-/

/-- The divergence of a matrix-valued spatial field.

For a field $T : \text{Space } d \rightarrow \text{Matrix } (\text{Fin } d) (\text{Fin } d) \mathbb{R}$, $\text{matrixDiv } T$ is the vector field whose i 'th component is

$\sum_j \partial[j] (\text{fun } x \Rightarrow T \ x \ i \ j) \ x$.

-/

noncomputable def matrixDiv (d : $\mathbb{N}$) (T : $\text{Space } d \rightarrow \text{Matrix } (\text{Fin } d) (\text{Fin } d) \mathbb{R}$) :
 $\text{Space } d \rightarrow \text{EuclideanSpace } \mathbb{R} (\text{Fin } d) := \text{fun } x \Rightarrow \text{WithLp.toLp } 2 \text{ fun } i \Rightarrow$
 $\text{div } (\text{fun } y : \text{Space } d \Rightarrow \text{WithLp.toLp } 2 \text{ fun } j \Rightarrow T \ y \ i \ j) \ x$

/-!

A.1. Basic equalities

-/

@[simp]

lemma matrixDiv_apply (d : $\mathbb{N}$) (T : $\text{Space } d \rightarrow \text{Matrix } (\text{Fin } d) (\text{Fin } d) \mathbb{R}$)
 (x : $\text{Space } d$) (i : $\text{Fin } d$) :
 $\text{matrixDiv } d \ T \ x \ i = \sum_j \partial[j] (\text{fun } x \Rightarrow T \ x \ i \ j) \ x := \text{by}$
 simp [matrixDiv, div]

/-!

A.2. The matrix divergence on the zero function

-/

```
@[simp]
lemma matrixDiv_zero (d : ℕ) :
  matrixDiv d (0 : Space d → Matrix (Fin d) (Fin d) ℝ) = 0 := by
  ext x i
  change (∑ j : Fin d, ∂[j] (fun _ : Space d => (0 : ℝ)) x) = 0
  simp

/-!
```

A.3. The matrix divergence on a constant function

-/

```
@[simp]
lemma matrixDiv_const (d : ℕ) (T : Matrix (Fin d) (Fin d) ℝ) :
  matrixDiv d (fun _ : Space d => T) = 0 := by
  ext x i
  change (∑ j : Fin d, ∂[j] (fun _ : Space d => T i j) x) = 0
  simp

/-!
```

A.4. The matrix divergence distributes over addition

-/

```
lemma matrixDiv_add (d : ℕ) (T1 T2 : Space d → Matrix (Fin d) (Fin d) ℝ)
  (hT1 : Differentiable ℝ T1) (hT2 : Differentiable ℝ T2) :
  matrixDiv d (T1 + T2) = matrixDiv d T1 + matrixDiv d T2 := by
  ext x i
  change (∑ j, ∂[j] (fun x => (T1 x + T2 x) i j) x) =
    (∑ j, ∂[j] (fun x => T1 x i j) x) +
    ∑ j, ∂[j] (fun x => T2 x i j) x
  rw [← Finset.sum_add_distrib]
  congr
  funext j
  change ∂[j] ((fun x => T1 x i j) + fun x => T2 x i j) x =
    ∂[j] (fun x => T1 x i j) x + ∂[j] (fun x => T2 x i j) x
  rw [deriv_add]
  · rfl
  · exact differentiable_pi.mp (differentiable_pi.mp hT1 i) j
```



```

    · exact differentiable_pi.mp (differentiable_pi.mp hT2 i) j
/-!
### A.5. The matrix divergence distributes over scalar multiplication
-/

lemma matrixDiv_smul (d : ℕ) (T : Space d → Matrix (Fin d) (Fin d) ℝ) (k : ℝ)
  (hT : Differentiable ℝ T) :
  matrixDiv d (k • T) = k • matrixDiv d T := by
  ext x i
  change (∑ j, ∂[j] (fun x => (k • T x) i j) x) =
    k * ∑ j, ∂[j] (fun x => T x i j) x
  rw [Finset.mul_sum]
  congr
  funext j
  change ∂[j] (k • fun x => T x i j) x = k • ∂[j] (fun x => T x i j) x
  rw [deriv_const_smul]
  · rfl
  · exact differentiable_pi.mp (differentiable_pi.mp hT i) j

end Space

```

1.91 Physlib/SpaceAndTime/Space/Norm.lean

```

/-- Auxiliary lemma with dimension defined as d.succ to handle `homeomorphUnitSphere`
The dimension correct version is declared in `distDiv_inv_pow_eq_dim`. -
/
private lemma distDiv_inv_pow_eq_dim' {d : ℕ} :
lemma distDiv_inv_pow_eq_dim {d : ℕ} (hd : d ≥ 1) :
  distDiv (distOfFunction (fun x : Space d => ||x|| ^ (- d : ℤ) • basis.repr x)
    (IsDistBounded.zpow_smul_repr_self (- d : ℤ) (by omega))) =
    (d * (volume (α := Space d)).real (Metric.ball 0 1)) • diracDelta ℝ 0 := by
  obtain ⟨d', rfl⟩ := Nat.exists_eq_succ_of_ne_zero (by omega : d ≠ 0)
  exact distDiv_inv_pow_eq_dim'

```

1.92 Physlib/SpaceAndTime/SpaceTime/TimeSlice.lean

```

@[fun_prop]
@[fun_prop]
@[fun_prop]

```

```

lemma timeSlice_symm_contDiff {d : ℕ} {M : Type} [NormedAddCommGroup M] [No
  {n} (c : SpeedOfLight) (f : Time → Space d → M) (h : ContDiff ℝ n 1f) :
  ContDiff ℝ n ((timeSlice c).symm f) := by
  change ContDiff ℝ n (Function.uncurry f ∘ toTimeAndSpace c)
  apply ContDiff.comp
  · exact h
  · exact ContinuousLinearEquiv.contDiff (toTimeAndSpace c)

@[fun_prop]
lemma timeSlice_symm_differentiable {d : ℕ} {M : Type} [NormedAddCommGroup
  (c : SpeedOfLight)
  (f : Time → Space d → M) (h : Differentiable ℝ 1f) :
  Differentiable ℝ 1((timeSlice c).symm f) := by
  change Differentiable ℝ (Function.uncurry f ∘ toTimeAndSpace c)
  apply Differentiable.comp
  · exact h
  · exact ContinuousLinearEquiv.differentiable (toTimeAndSpace c)

```

1.93 Physlib/StatisticalMechanics/MicroCanonicalEnsemble/Basic.

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
module

public import Mathlib.Data.Fintype.Defs
public import Mathlib.Data.Real.Basic
public import Mathlib.MeasureTheory.Constructions.BorelSpace.Real
public import Mathlib.MeasureTheory.Constructions.Pi
public import Mathlib.MeasureTheory.Constructions.BorelSpace.WithTop
public import Physlib.Meta.Sorry
/-!

## The Microcanonical Ensemble

-/

@[expose] public section

section ThermodynamicEnsemble

```

1.94.

PHYSLIB/STATISTICALMECHANICS/MICROCANONICALENSEMBLE/IDEALGAS.LEAN

```

/-- The Hamiltonian for a microcanonical ensemble, parameterized by any extrinsic pa
D. Since it's an arbitrary type, 'T' would be nice, but we use 'D' for data to a
with temperature. We define Hamiltonians as an energy function on a real manifold
dimension n possibly depends on the data D. Energy is a WithTop  $\mathbb{R}$ ,  $\tau$  is used to
as a possibility in phase space". This formalization does exclude the possibility
degrees of freedom; it is not _completely_ general. A better formalization could
arbitrary measurable space.
-/
structure MicroHamiltonian (D : Type) where
  /-- For extrinsic parameters D (e.g. the number of particles of different chemical
the shape of the box), how many continuous degrees of freedom are there? -
  /
  dim : D → Type
  /-- We require that the number of degrees of freedom is finite. -
  /
  [dim_fin : ∀ d, Fintype (dim d)]
  /-- Given the configuration, what is its energy? -/
  H : {d : D} → (dim d →  $\mathbb{R}$ ) → WithTop  $\mathbb{R}$ 
  --The energy function must be measurable (else the partition function integral is
measurable_H : ∀ d, Measurable (@H d)

/-- We add the dim_fin to the instance cache so that things like the measure can be
-/
instance microHamiltonianFintype {D} (H : MicroHamiltonian D) (d : D) : Fintype (H.d
  H.dim_fin d

/-- The standard microcanonical ensemble Hamiltonian, where the data is the particle
the volume V. -/
abbrev NVEHamiltonian := MicroHamiltonian ( $\mathbb{N} \times \mathbb{R}$ )

/-- Helper to get the number in an N-V Hamiltonian -/
def NVEHamiltonian.N : ( $\mathbb{N} \times \mathbb{R}$ ) →  $\mathbb{N}$  := Prod.fst

/-- Helper to get the volume in an N-V Hamiltonian -/
def NVEHamiltonian.V : ( $\mathbb{N} \times \mathbb{R}$ ) →  $\mathbb{R}$  := Prod.snd

```

1.94 Physlib/StatisticalMechanics/MicroCanonicalEnsemble/IdealGas.lean

```

public import Physlib.StatisticalMechanics.MicroCanonicalEnsemble.ThermoQuantities
/-!
## Ideal gas as a Micro Canonical Ensemble

In this module we give the
-/

```

```

/-- The Hamiltonian for an ideal gas: particles live in a cube of volume V^
contributes an energy p^2/2. The per-particle mass is normalized to 1. -
/
--The dimension of the manifold is 6 times the number of particles: three
--momentum.
--The energy is ∞ if any positions are outside the cube, otherwise it's t
--squared over 2.
measurable_H := by
  rintro ⟨n, V⟩
  dsimp
  refine Measurable.ite ?_ ?_ measurable_const
  · rw [Set.setOf_forall]
    apply MeasurableSet.iInter
    intro i
    rw [Set.setOf_forall]
    exact MeasurableSet.iInter fun ax => measurableSet_le
      (show Measurable fun config : Fin n × (Fin 3 ⊕ Fin 3) → ℝ =>
        |config (i, Sum.inl ax)| by fun_prop) measurable_const
  · convert WithTop.isOpenEmbedding_coe.measurableEmbedding.measurable.co
    (Finset.measurable_sum _ fun (x : Fin n × Fin 3) _ =>
      (show Measurable fun config : Fin n × (Fin 3 ⊕ Fin 3) → ℝ =>
        config (x.1, Sum.inr x.2) ^ 2 / (2 : ℝ) by fun_prop)) using 1
  ext config
  rw [← WithTop.coe_sum]
  rfl
lemma partitionZ_eq (hV : 0 < V) (hβ : 0 < β) :
  IdealGas.partitionZ (n,V) β = V ^ n * (2 * Real.pi / β) ^ (3 * n / 2 :
  rw [partitionZ, IdealGas]
    else T) : WithTop ℝ).untop proof =
  ∑ x : Fin n × Fin 3, config (x.1, Sum.inr x.2) ^ 2 / (2 : ℝ)
let eq_pm : MeasurableEquiv ((Fin n × Fin 3 → ℝ) ×
  (Fin n × Fin 3 → ℝ)) (Fin n × (Fin 3 ⊕ Fin 3) → ℝ) :=
let e1 := (MeasurableEquiv.sumPiEquivProdPi
  (α := fun ( _ : (Fin n × Fin 3) ⊕ (Fin n × Fin 3)) ↦ ℝ))
let e2 := (MeasurableEquiv.piCongrLeft _
  (MeasurableEquiv.prodSumDistrib (Fin n) (Fin 3) (Fin 3))).symm
simp only [integral_const, MeasurableSet.univ, measureReal_restrict_a
  Real.volume_real_Icc, sub_neg_eq_add, add_halves, smul_eq_mul, mul_
  ge_iff_le]
GaussianFourier.integral_rexp_neg_mul_sq_norm
  (V := PiLp 2 (fun ( _ : Fin n × Fin 3) ↦ ℝ)) (half_pos hβ)
simp_rw [div_eq_inv_mul, ← Finset.mul_sum, ← mul_assoc, neg_mul, mul_
  PiLp.norm_sq_eq_of_L2]
simp only [finrank_euclideanSpace, Fintype.card_prod, Fintype.card_fi
  Nat.cast_ofNat]
lemma helmholtzA_eq (hV : 0 < V) (hT : 0 < T) : IdealGas.helmholtzA (n,V) T

```

1.95.

PHYSLIB/STATISTICALMECHANICS/MICROCANONICALENSEMBLE/THERMOQUANTITIES.LEAN

```
rw [helmholtzA, partitionZT, partitionZ_eq n hV (one_div_pos.mpr hT), Real.log_mul]
lemma ZIntegrable (hV : 0 < V) (hβ : 0 < β) : IdealGas.ZIntegrable (n,V) β := by
  have hZpos : 0 < partitionZ IdealGas (n, V) β := by
    rw [partitionZ_eq n hV hβ]
    rw [← partitionZ]
theorem ideal_gas_law (hV : 0 < V) (hT : 0 < T) :
  let P := IdealGas.pressure (n,V) T;
  dsimp [pressure]
  rw [derivWithin_congr (f := fun V' ↦ -n * T *
    (Real.log V' + (3/2) * Real.log (2 * Real.pi * T))) ?_ ?_]
  rw [helmholtzA_eq n hV' hT]
  · exact helmholtzA_eq n hV hT
-- Now proving e.g. Boyle's Law ("for an ideal gas with a fixed particle number, P a
-- inversely proportional") is a trivial consequence of the ideal gas law.
```

1.95 Physlib/StatisticalMechanics/MicroCanonicalEnsemble/ThermoQuantities

/-

Copyright (c) 2025 Alex Meiburg. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Alex Meiburg

-/

module

```
public import QuantumInfo.ForMathlib.ComplexLaplaceTransform
public import Mathlib.Analysis.Complex.HasPrimitives
public import Mathlib.Analysis.Complex.RealDeriv
public import Mathlib.MeasureTheory.Constructions.BorelSpace.WithTop
public import Mathlib.MeasureTheory.Function.StronglyMeasurable.Basic
public import Mathlib.MeasureTheory.Integral.DominatedConvergence
public import Mathlib.MeasureTheory.Measure.Prod
public import Mathlib.MeasureTheory.Integral.Bochner.ContinuousLinearMap
public import Mathlib.Analysis.SpecialFunctions.Log.Deriv
public import Mathlib.MeasureTheory.Integral.Bochner.Basic
public import Mathlib.MeasureTheory.Measure.Haar.OfBasis
public import Physlib.StatisticalMechanics.MicroCanonicalEnsemble.Basic
public import Physlib.Meta.TODO.Basic
```

/-!

The thermodynamical quantities of a microcanonical ensemble

-/

```

@[expose] public section

noncomputable section
namespace MicroHamiltonian

variable {D : Type} (H : MicroHamiltonian D) (d : D)

/-- The partition function corresponding to a given MicroHamiltonian. This
    thermodynamic  $\beta$ , not a temperature. It also depends on the data D defining
    the MicroHamiltonian.
* Ideally this would be an NNReal, but  $\int$  (NNReal) doesn't work right now,
  separate proof anyway
-/
def partitionZ ( $\beta$  :  $\mathbb{R}$ ) :  $\mathbb{R}$  :=
   $\int$  (config : H.dim d  $\rightarrow$   $\mathbb{R}$ ),
    let E := H.H config
    if h : E =  $\top$  then 0 else Real.exp (- $\beta$  * (E.untop h))

/-- The complexified partition function, viewed as a Laplace transform. -
/
def PartitionZComplex (z :  $\mathbb{C}$ ) :  $\mathbb{C}$  :=
  ComplexLaplaceTransform (fun config : H.dim d  $\rightarrow$   $\mathbb{R}$  => H.H config) z

/-- The complex convergence domain of the partition-function Laplace transform. -
/
def ZComplexConvergenceDomain : Set  $\mathbb{C}$  :=
  ComplexLaplaceConvergenceDomain (fun config : H.dim d  $\rightarrow$   $\mathbb{R}$  => H.H config)

private theorem partitionZ_eq_re_partitionZComplex { $\beta$  :  $\mathbb{R}$ }
  (h $\beta$  : ( $\beta$  :  $\mathbb{C}$ )  $\in$  H.ZComplexConvergenceDomain d) :
  H.partitionZ d  $\beta$  = (H.PartitionZComplex d  $\beta$ ).re := by
  have hInt : MeasureTheory.Integrable ( $\mu$  := MeasureTheory.volume)
    (ComplexLaplaceIntegrand (fun config : H.dim d  $\rightarrow$   $\mathbb{R}$  => H.H config) ( $\beta$  :  $\mathbb{C}$ )) := by
    rw [partitionZ, PartitionZComplex, ComplexLaplaceTransform]
  calc
    ( $\int$  (config : H.dim d  $\rightarrow$   $\mathbb{R}$ ),
      have E := H.H config
      if h : E =  $\top$  then 0 else Real.exp (- $\beta$  * E.untop h)) =
       $\int$  x, RCLike.re (ComplexLaplaceIntegrand
        (fun config : H.dim d  $\rightarrow$   $\mathbb{R}$  => H.H config) ( $\beta$  :  $\mathbb{C}$ ) x) := by
        apply MeasureTheory.integral_congr_ae
        filter_upwards with config
        unfold ComplexLaplaceIntegrand
        by_cases h : H.H config =  $\top$ 
        · simp [h]
        · simp only [h, dite_false]

```

1.95.

PHYSLIB/STATISTICALMECHANICS/MICROCANONICALENSEMBLE/THERMOQUANTITIES.LEAN

```

    set e : ℝ := (H.H config).untop h
    simp [Complex.ofReal_mul] using (Complex.exp_ofReal_re (-
(β * e))).symm
  _ = RCLike.re (∫ x, ComplexLaplaceIntegrand
    (fun config : H.dim d → ℝ => H.H config) (β : ℂ) x) := integral_re hInt

open scoped ContDiff Topology in
theorem contDiffAt_partitionZ_of_mem_interior_convergenceDomain {β : ℝ}
  (hβ : (β : ℂ) ∈ interior (H.ZComplexConvergenceDomain d)) :
  ContDiffAt ℝ τ (H.partitionZ d) β := by
  refine (analyticAt_complexLaplaceTransform_of_mem_interior_convergenceDomain
    (E := fun config : H.dim d → ℝ => H.H config) (H.measurable_H d) hβ).contDiffAt
  |>.real_of_complex |>.congr_of_eventuallyEq ?_
  filter_upwards [(Complex.continuous_ofReal.tendsto β).eventually
    (IsOpen.mem_nhds isOpen_interior hβ)] with x hx
  exact H.partitionZ_eq_re_partitionZComplex d (_root_.interior_subset hx)

/-- The partition function as a function of temperature T instead of β. -
/
def partitionZT (T : ℝ) : ℝ :=
  partitionZ H d (1/T)

/-- The Internal Energy, U or E, defined as  $-\partial(\ln Z)/\partial\beta$ . Parameterized here with β. -
/
def internalU (β : ℝ) : ℝ :=
  -deriv (fun β' ↦ (partitionZ H d β').log) β

/-- The Helmholtz Free Energy,  $-T * \ln Z$ . Also denoted F. Parameterized here with temperature T. -
/
def helmholtzA (T : ℝ) : ℝ :=
  -T * (partitionZT H d T).log

/-- The entropy, defined as the  $-\partial A/\partial T$ . Function of T. -
/
def entropyS (T : ℝ) : ℝ :=
  -deriv (helmholtzA H d) T

/-- The entropy, defined as  $\ln Z + \beta * U$ . Function of β. -
/
def entropySβ (β : ℝ) : ℝ :=
  (partitionZ H d β).log + β * internalU H d β

/-- To be able to compute or define anything from a Hamiltonian, we need its partition function
to be a computable integral. A Hamiltonian is ZIntegrable at β if PartitionZ is Lebesgue
integrable and nonzero.
-
/
def ZIntegrable (β : ℝ) : Prop :=
  MeasureTheory.Integrable (fun (config : H.dim d → ℝ) ↦

```

```

    let E := H.H config;
    if h : E = T then 0 else Real.exp (-β * (E.untop h))
  ) ∧ (H.partitionZ d β ≠ 0)

/--
This Prop defines the most common case of ZIntegrable, that it is integrable at all
temperatures (aka all positive β).
-/
def PositiveβIntegrable : Prop :=
  ∀ β > 0, H.ZIntegrable d β

variable {H d}

/-
Need the fact that the partition function Z is differentiable. Assume it's
Letting  $\mu^-(H,E)$  be the measure of  $\{x \mid H(x) \leq E\}$ , then for nonzero  $\beta$ ,

$$\int_0^\infty \exp(-\beta E) (d\mu^-/dE) dE =$$


$$\int \exp(-\beta H) d\mu =$$


$$\int (1/\beta * \int_H^\infty \exp(-\beta E) dE) d\mu =$$


$$\int (1/\beta * \int_{-\infty}^\infty \exp(-\beta E) \chi(E \leq H) dE) d\mu =$$


$$1/\beta * \int (\int \exp(-\beta E) \chi(E \leq H) d\mu) dE =$$


$$1/\beta * \int \exp(-\beta E) * \mu^-(H,E) dE$$

so this will be differentiable if

$$\int \exp(-\beta E) * \mu^-(H,E) dE$$

is, aka if the Laplace transform is differentiable.

For this we really want the fact that the Laplace transform is analytic when
absolutely convergent, which follows from the local domination estimate above
Fubini's theorem, and Morera's theorem.
-/
/-- The two definitions of entropy, in terms of `T` or `β = 1 / T`, are equivalent
-/
theorem entropy_A_eq_entropy_Z (T : ℝ) (hT : T ≠ 0)
  (hZne : H.partitionZ d (1 / T) ≠ 0)
  (hZint : ((1 / T : ℝ) : ℂ) ∈ interior (H.ZComplexConvergenceDomain d))
  entropyS H d T = entropySβ H d (1 / T) := by
  have hZdiff : DifferentiableAt ℝ (H.partitionZ d) (1 / T) :=
    (H.contDiffAt_partitionZ_of_mem_interior_convergenceDomain d hZint).diff
  (by simp)
  dsimp [entropyS, entropySβ, internalU, partitionZT]
  unfold helmholtzA
  erw [deriv_mul]
  rw [deriv_neg'', neg_mul, one_mul, neg_add_rev, neg_neg, mul_neg, add_comm]
  congr 1
  simp_rw [partitionZT]

```


1.95.

PHYSLIB/STATISTICALMECHANICS/MICROCANONICALENSEMBLE/THERMODYNAMICQUANTITIES.LEAN

```

have hdc := deriv_comp (h := fun T ↦ T⁻¹) (h₂ := fun β => Real.log (H.partitionZ d))
unfold Function.comp at hdc
simp only [hdc, one_div, deriv_inv', mul_neg, neg_inj]
field_simp
ring_nf
--Show the differentiability side-goals
· rw [← one_div]
  fun_prop (disch := assumption)
· fun_prop (disch := assumption)
· fun_prop
· simp_rw [partitionZT]
  fun_prop (disch := assumption)

```

```

set_option backward.isDefEq.respectTransparency false in
open scoped ContDiff in

```

/--

The "definition of temperature from entropy":

$1/T = (\partial S/\partial U)$, when the derivative is at constant extrinsic d (typically N/V).

Here we use β instead of $1/T$ on the left, and express the right actually as $(\partial S/\partial \beta)/$ as all our things are ultimately parameterized by β .

This identity requires the denominator $\partial U/\partial \beta$ to be nonzero.

-/

```

theorem β_eq_deriv_S_U {β : ℝ}
  (hZne : H.partitionZ d β ≠ 0)
  (hZint : (β : ℂ) ∈ interior (H.ZComplexConvergenceDomain d))
  (hU' : deriv (H.internalU d) β ≠ 0) :
  β = (deriv (H.entropySβ d) β) / deriv (H.internalU d) β := by
have hZ : ContDiffAt ℝ τ (H.partitionZ d) β :=
  H.contDiffAt_partitionZ_of_mem_interior_convergenceDomain d hZint
unfold entropySβ
unfold internalU

```

--Show the differentiability side-goals

```

have hlogDiff : DifferentiableAt ℝ (fun β => Real.log (H.partitionZ d β)) β := by

```

```

  have := hZne

```

```

  have := hZ.differentiableAt (by simp)

```

```

  fun_prop (disch := assumption)

```

```

have hlogDerivDiff : DifferentiableAt ℝ (deriv fun β => Real.log (H.partitionZ d β)) β :=

```

```

  have this := hZ.log hZne

```

```

  replace this :=

```

```

    (this.fderiv_right (m := τ) (OrderTop.le_top _)).differentiableAt (by simp)

```

```

  unfold deriv

```

```

  fun_prop

```

```

have hderiv : deriv (deriv fun β => Real.log (H.partitionZ d β)) β ≠ 0 := by

```

```

    intro hzero
    apply hU'
    change deriv (-fun  $\beta \Rightarrow$  deriv (fun  $\beta' \Rightarrow$  Real.log (H.partitionZ d  $\beta'$ )))
    rw [deriv.neg]
    simp [hzero]

--Main goal
simp only [mul_neg]
erw [deriv.neg', deriv_add, deriv.neg']
dsimp
erw [deriv_mul]
simp only [deriv_id'', one_mul, neg_add_rev, add_neg_cancel_comm_assoc, n
exact (mul_div_cancel_right  $\beta$  hderiv).symm
--Discharge those side-goals
· exact differentiableAt_id
· exact hlogDerivDiff
· fun_prop (disch := assumption)
· fun_prop (disch := assumption)

set_option backward.isDefEq.respectTransparency false in
open scoped ContDiff in
example (x :  $\mathbb{R}$ ) (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (hf : ContDiffAt  $\mathbb{R}$   $\tau$  f x) : DifferentiableAt  $\mathbb{R}$ 
  have := (hf.fderiv_right (m :=  $\tau$ ) (OrderTop.le_top _)).differentiableAt (
    unfold deriv
    fun_prop

end MicroHamiltonian

--! Specializing to a system of particles in space

namespace NVEHamiltonian
open MicroHamiltonian

variable (H : NVEHamiltonian) (d :  $\mathbb{N} \times \mathbb{R}$ )

/-- Pressure, as a function of T. Defined as the conjugate variable to volu
/
def pressure (T :  $\mathbb{R}$ ) :  $\mathbb{R}$  :=
  let (n, V) := d;
  - deriv (fun V'  $\mapsto$  helmholtzA H (n, V') T) V

end NVEHamiltonian

```

1.96 Physlib/Units/Examples.lean

```

public import Physlib.Units.FDeriv
example {M1 M2 : Type} [NormedAddCommGroup M1] [NormedSpace ℝ M1]
  [ContinuousConstSMul ℝ M1] [HasDim M1]
  [NormedAddCommGroup M2] [NormedSpace ℝ M2] [SMulCommClass ℝ ℝ M2]
  [ContinuousConstSMul ℝ M2] [HasDim M2] (f : M1 → M2)
  (hf : IsDimensionallyCorrect f) (f_diff : Differentiable ℝ f) :
  IsDimensionallyCorrect (fderiv ℝ f) :=
  fderiv_isDimensionallyCorrect f hf f_diff

example {M1 M2 : Type} [NormedAddCommGroup M1] [NormedSpace ℝ M1]
  [ContinuousConstSMul ℝ M1] [HasDim M1]
  [NormedAddCommGroup M2] [NormedSpace ℝ M2] [SMulCommClass ℝ ℝ M2]
  [ContinuousConstSMul ℝ M2] [HasDim M2] (dm : M1) (f : M1 → M2)
  (hf : IsDimensionallyCorrect f) (f_diff : Differentiable ℝ f) :
  IsDimensionallyCorrect (fun x (v : WithDim (dim M2 * (dim M1)-1) M2) =>
    fderiv ℝ f x dm = v.1) :=
  fderiv_dimension_const_direction dm f hf f_diff

```

1.97 QuantumInfo.lean

```

public import QuantumInfo.ForMathlib.ContinuousLinearMap
public import QuantumInfo.ForMathlib.ComplexLaplaceTransform
public import QuantumInfo.ForMathlib.ContinuousSup
public import QuantumInfo.ForMathlib.Filter
public import QuantumInfo.ForMathlib.HermitianMat
public import QuantumInfo.ForMathlib.Isometry
public import QuantumInfo.ForMathlib.LinearEquiv
public import QuantumInfo.ForMathlib.MatrixNorm.TraceNorm
public import QuantumInfo.ForMathlib.Matrix
public import QuantumInfo.ForMathlib.Minimax
public import QuantumInfo.ForMathlib.Misc
public import QuantumInfo.ForMathlib.Unitary
public import QuantumInfo.Channels.DegradableOrder
public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Channels.CPTP
public import QuantumInfo.Channels.Dual
public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.Channels.Unbundled
public import QuantumInfo.States.Mixed.Fidelity
public import QuantumInfo.States.Mixed.TraceDistance
public import QuantumInfo.States.Pure.Qubit
public import QuantumInfo.ResourceTheory.FreeState

```

```

public import QuantumInfo.ResourceTheory.SteinsLemma
public import QuantumInfo.States.Pure.Braket
public import QuantumInfo.Capacity.Capacity
public import QuantumInfo.States.Ensemble
public import QuantumInfo.States.Entanglement
public import QuantumInfo.Entropy.VonNeumann
public import QuantumInfo.Entropy.SSA
public import QuantumInfo.Entropy.Relative
public import QuantumInfo.Entropy.DPI
public import QuantumInfo.States.Mixed.MState
public import QuantumInfo.Channels.Pinching
public import QuantumInfo.Measurements.POVm
public import QuantumInfo.Operators.Unitary
public import QuantumInfo.Capacity.Capacity_doc

```

1.98 QuantumInfo/Capacity/Capacity.lean

```

public import QuantumInfo.Entropy.VonNeumann
public import QuantumInfo.Entropy.SSA
public import QuantumInfo.Entropy.Relative
public import QuantumInfo.Entropy.DPI
public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Channels.CPTP
public import QuantumInfo.Channels.Dual
public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.Channels.Unbundled
public import QuantumInfo.States.Mixed.Fidelity
public import QuantumInfo.States.Mixed.TraceDistance
public import Physlib.Meta.Sorry
  * `achievesRate_0`: Every channel out of a nonempty space achievesRate 0.
    the set of achievable rates is Nonempty.
/-- Every quantum channel out of a nonempty space achieves at least a rate
/
theorem achievesRate_0 (Λ : CPTPMap d₁ d₂) [Nonempty d₁] : Λ.AchievesRate 0
  have : Nonempty d₂ := Λ.toPTPMap.nonemptyOut
  refine ⟨1, one_pos, 1, default, ⟨default, default, Subsingleton.elim _ _⟩
  simp [show (default : CPTPMap (Fin 1) (Fin 1)) = id from Subsingleton.elim
    εApproximates_monotone (εApproximates_self (id (dIn := Fin 1))) hε.le
  · --piProd of id's is id, then use emulates_self up to equivalence
    rw [show (fun (_ : Fin 1) ↦ id (dIn := d₁)) = (fun _ ↦ id) from rfl, pi
    exact let σ := Fintype.equivFinOfCardEq (by simp +decide : Fintype.card
      ⟨ofEquiv σ.symm, ofEquiv σ, by ext1; simp⟩
@[sorryful]
@[sorryful]

```

```

      rw [piProd_id]
      obtain (E, D, hD) := h_emulate
      exact emulates_trans _ _ _ (piProd fun _ => E, piProd fun _ => D,
        by simp [← hD, ← CPTPMap.piProd_comp]) hB_emulate
@[sorryful]
/-- Quantum channel capacity is nonnegative for channels out of a nonempty space. -
/
@[sorryful]
theorem zero_le_quantumCapacity (Λ : CPTPMap d₁ d₂) [Nonempty d₁] :
  0 ≤ Λ.quantumCapacity :=
@[sorryful]
@[sorryful]
@[sorryful]

```

1.99 QuantumInfo/Channels/Bundled.lean

```

public import QuantumInfo.Channels.Unbundled
public import QuantumInfo.States.Mixed.MState

```

1.100 QuantumInfo/Channels/CPTP.lean

```

public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Operators.Unitary
/-- Tensor products commute with channel application:
  `(Λ₁ ⊗C P Λ₂) (ρ₁ ⊗M ρ₂) = Λ₁ ρ₁ ⊗M Λ₂ ρ₂`. -/
@[simp]
theorem prod_apply_prod (Λ₁ : CPTPMap dI₁ dO₁) (Λ₂ : CPTPMap dI₂ dO₂)
  (ρ₁ : MState dI₁) (ρ₂ : MState dI₂) :
  (Λ₁ ⊗C P Λ₂) (ρ₁ ⊗M ρ₂) = (Λ₁ ρ₁) ⊗M (Λ₂ ρ₂) := by
  apply MState.ext_m
  simp [CPTPMap.prod, MState.prod] using
    MatrixMap.kron_map_of_kron_state Λ₁.map Λ₂.map ρ₁.m ρ₂.m

set_option maxRecDepth 1000 in
TP := MatrixMap.IsTracePreserving.piProd (fun i ↦ (Λ i).TP)
piProd Λ i =
  ofEquiv (Equiv.funUnique (Fin 1) (dO 0)).symm ∘m
    ((Λ i 1) ∘m ofEquiv (Equiv.funUnique (Fin 1) (dI 0))) := by
  apply CPTPMap.ext
  change MatrixMap.piProd (fun i ↦ (Λ i).map) =
    MatrixMap.submatrix ℂ (Equiv.funUnique (Fin 1) (dO 0)) ∘l
      ((Λ i 1).map ∘l MatrixMap.submatrix ℂ (Equiv.funUnique (Fin 1) (dI 0)).symm)
  apply MatrixMap.choi_equiv.injective

```

```

conv_lhs => rw [MatrixMap.choi_equiv_apply, MatrixMap.choi_matrix_piProd]
rw [MatrixMap.choi_equiv_apply]
ext (i, a) (j, b)
simp only [Matrix.reindex_apply, Matrix.piProd, Matrix.of_apply,
  Fintype.prod_subsingleton _ (0 : Fin 1),
  MatrixMap.choi_matrix, LinearMap.comp_apply,
  MatrixMap.submatrix_apply, Matrix.submatrix_apply, Matrix.single]
convert rfl
ext
simp [funext_iff]
@[simp]
theorem piProd_id :
  piProd (fun i => (CPTPMap.id : CPTPMap (dI i) (dI i))) = CPTPMap.id := by
  apply CPTPMap.ext
  simp [piProd, id_map, MatrixMap.piProd_id]

/-- The Stinespring preparation `prep ∘ append` acts on a matrix entry by t
with the fixed pure state `|default⟩⟨default|` on `dOut × dOut`. -
/
theorem prep_append_map_entry (X : Matrix dIn dIn ℂ)
  (a₁ : dIn) (b₁c₁ : dOut × dOut) (a₂ : dIn) (b₂c₂ : dOut × dOut) :
  let τ := MState.pure (Ket.basis (default : dOut × dOut))
  let zero_prep : CPTPMap Unit (dOut × dOut) := replacement τ
  let prep := (id ⊗C P zero_prep)
  let append : CPTPMap dIn (dIn × Unit) := CPTPMap.ofEquiv (Equiv.prodPU
    (prep ∘m append).map X (a₁, b₁c₁) (a₂, b₂c₂) =
    X a₁ a₂ * τ.m b₁c₁ b₂c₂ := by
  simp [purify_prep_append_entry]

```

1.101 QuantumInfo/Channels/DegradableOrder.lean

```

public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Channels.CPTP
public import QuantumInfo.Channels.Dual
public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.Channels.Unbundled

```

1.102 QuantumInfo/Channels/Dual.lean

Authors: Alex Meiburg, Dennj Osele

```

public import QuantumInfo.Channels.Bundled
open scoped Kronecker

```

```

let coordDual :=
  let iso1 := (Module.Basis.toDualEquiv <| Matrix.stdBasis R dIn dIn).symm
  let iso2 := (Module.Basis.toDualEquiv <| Matrix.stdBasis R dOut dOut)
  iso1 ∘1 LinearMap.dualMap M ∘1 iso2
  (Matrix.transposeLinearEquiv dIn dIn R R).toLinearMap ∘1 coordDual ∘1
  (Matrix.transposeLinearEquiv dOut dOut R R).toLinearMap
have hDualIn (X Y : Matrix dIn dIn R) :
  ((Matrix.stdBasis R dIn dIn).toDualEquiv Y) X = (X * Y.transpose).trace := by
  simp [Module.Basis.toDualEquiv_apply, Module.Basis.toDual, Matrix.trace, Matrix.
    Matrix.stdBasis, Fintype.sum_prod_type, mul_comm]
have hDualOut (X Y : Matrix dOut dOut R) :
  ((Matrix.stdBasis R dOut dOut).toDualEquiv Y) X = (X * Y.transpose).trace := b
  simp [Module.Basis.toDualEquiv_apply, Module.Basis.toDual, Matrix.trace, Matrix.
    Matrix.stdBasis, Fintype.sum_prod_type, mul_comm]
let coordDual : MatrixMap dOut dIn R :=
  let iso1 := (Module.Basis.toDualEquiv <| Matrix.stdBasis R dIn dIn).symm
  let iso2 := (Module.Basis.toDualEquiv <| Matrix.stdBasis R dOut dOut)
  iso1 ∘1 LinearMap.dualMap M ∘1 iso2
rw [show (M A * B).trace =
  ((Matrix.stdBasis R dOut dOut).toDualEquiv B.transpose) (M A) by
  simpa using (hDualOut (M A) B.transpose).symm]
rw [show
  ((Matrix.stdBasis R dOut dOut).toDualEquiv B.transpose) (M A) =
  ((Matrix.stdBasis R dIn dIn).toDualEquiv (coordDual B.transpose)) A by
  simp [coordDual]]
simpa [dual, coordDual] using hDualIn A (coordDual B.transpose)
theorem IsHermitianPreserving.dual {M : MatrixMap dIn dOut ℂ} (h : M.IsHermitianPres
  M.dual.IsHermitianPreserving := by
have map_conjTranspose (x : Matrix dIn dIn ℂ) :
  M (Matrix.conjTranspose x) = Matrix.conjTranspose (M x) := by
have hxstar : (realPart x : Matrix dIn dIn ℂ) - Complex.I • (imaginaryPart x : M
  Matrix.conjTranspose x := by
  rw [← Matrix.star_eq_conjTranspose]
have hreal_star : (realPart (star x) : Matrix dIn dIn ℂ) = realPart x := by
  rw [realPart_apply_coe, realPart_apply_coe, star_star, add_comm]
have himag_star : (imaginaryPart (star x) : Matrix dIn dIn ℂ) = -
imaginaryPart x := by
  rw [imaginaryPart_apply_coe, imaginaryPart_apply_coe, star_star]
module
  have h := realPart_add_I_smul_imaginaryPart (star x : Matrix dIn dIn ℂ)
  rw [hreal_star, himag_star, smul_neg] at h
  simpa [sub_eq_add_neg] using h
calc
  M (Matrix.conjTranspose x) = M (realPart x - Complex.I • imaginaryPart x) := b
  rw [← hxstar]
  _ = M (realPart x) - Complex.I • M (imaginaryPart x) := by

```

```

simp [sub_eq_add_neg, map_add, map_smul]
_ = Matrix.conjTranspose (M (realPart x) + Complex.I • M (imaginaryPart x))
rw [Matrix.conjTranspose_add, Matrix.conjTranspose_smul]
rw [show Matrix.conjTranspose (M (realPart x)) = M (realPart x) by
  simp [Matrix.IsHermitian] using h (HermitianMat.H _)]
rw [show Matrix.conjTranspose (M (imaginaryPart x)) = M (imaginaryPart x) by
  simp [Matrix.IsHermitian] using h (HermitianMat.H _)]
simp [sub_eq_add_neg]
_ = Matrix.conjTranspose (M x) := by
  congr 1
  simp [map_add, map_smul] using congrArg M (realPart_add_I_smul_imaginaryPart x)
intro x hx
simp [Matrix.IsHermitian] using show Matrix.conjTranspose (M.dual x) = M (x)
  apply Matrix.ext_iff_trace_mul_left.mpr
intro A
have htrace2 := congrArg star (Dual.trace_eq M (Matrix.conjTranspose A))
rw [← Matrix.trace_conjTranspose, ← Matrix.trace_conjTranspose] at htrace2
rw [Matrix.conjTranspose_mul, Matrix.conjTranspose_mul,
  Matrix.conjTranspose_conjTranspose, Matrix.conjTranspose_conjTranspose,
  map_conjTranspose, hx, Matrix.conjTranspose_conjTranspose,
  Matrix.trace_mul_comm x (M A),
  Matrix.trace_mul_comm (Matrix.conjTranspose (M.dual x)) A] at htrace2
exact htrace2.symm.trans (Dual.trace_eq M A x)
theorem IsPositive.dual {M : MatrixMap dIn dOut ℂ} (h : M.IsPositive) : M.dual.IsPositive :=
private theorem matrix_mem_span_kronecker {A C : Type*} [Fintype A] [Fintype C]
  [DecidableEq A] [DecidableEq C] (X : Matrix (A × C) (A × C) ℂ) :
  X ∈ Submodule.span ℂ
  (Set.range (fun p : (Matrix A A ℂ × Matrix C C ℂ) => p.1 ⊗k p.2)) :=
rw [Matrix.matrix_eq_sum_single X]
refine Submodule.sum_mem _ fun {a1, c1} _ =>
  Submodule.sum_mem _ fun {a2, c2} _ => ?_
rw [show Matrix.single (a1, c1) (a2, c2) (X (a1, c1) (a2, c2)) =
  X (a1, c1) (a2, c2) •
  ((Matrix.single a1 a2 1 : Matrix A A ℂ) ⊗k
  (Matrix.single c1 c2 1 : Matrix C C ℂ)) by
  ext {a, c} {a', c'}
  by_cases ha1 : a1 = a <=> by_cases hc1 : c1 = c <=>
    by_cases ha2 : a2 = a' <=> by_cases hc2 : c2 = c' <=>
      simp [Matrix.single, Matrix.kroneckerMap_apply, ha1, hc1, ha2, hc2]
exact Submodule.smul_mem _ (X (a1, c1) (a2, c2)) <|
  Submodule.subset_span {
    ((Matrix.single a1 a2 1 : Matrix A A ℂ),
    (Matrix.single c1 c2 1 : Matrix C C ℂ)), rfl)

refine dual_unique _ _ ?_
intro X Y

```



```

induction matrix_mem_span_kronecker X using Submodule.span_induction with
| mem X hX =>
  rcases hX with (<x₁, x₂>, rfl)
  induction matrix_mem_span_kronecker Y using Submodule.span_induction with
  | mem Y hY =>
    rcases hY with (<y₁, y₂>, rfl)
    simp [MatrixMap.kron_map_of_kron_state, ← Matrix.mul_kronecker_mul,
      Matrix.trace_kronecker, Dual.trace_eq M x₁ y₁, Dual.trace_eq N x₂ y₂]
  | zero => simp
  | add Y₁ Y₂ _ _ hY₁ hY₂ =>
    simp [map_add, Matrix.mul_add] using congrArg₂ (· + ·) hY₁ hY₂
  | smul a Y _ hY => simp [map_smul, Matrix.mul_smul] using congrArg (a • ·) hY
| zero => simp
| add X₁ X₂ _ _ hX₁ hX₂ =>
  simp [map_add, Matrix.add_mul] using congrArg₂ (· + ·) hX₁ hX₂
| smul a X _ hX => simp [map_smul, smul_mul_assoc] using congrArg (a • ·) hX
theorem IsCompletelyPositive.dual {M : MatrixMap dIn dOut ℂ} (h : M.IsCompletelyPosi
have h_dual_pos : (MatrixMap.dual (M ⊗k m MatrixMap.id (Fin n) ℂ)).IsPositive := by
have h_dual_kron : (MatrixMap.dual (M ⊗k m MatrixMap.id (Fin n) ℂ)) = (MatrixMap.du
  convert dual_kron M (MatrixMap.id (Fin n) ℂ) using 1;
refine dual_unique (M := M.dual) (M' := M) ?_
intro A B
calc
(M.dual A * B).trace = (B * M.dual A).trace := by rw [Matrix.trace_mul_comm]
_ = (M B * A).trace := by rw [Dual.trace_eq]
_ = (A * M B).trace := by rw [Matrix.trace_mul_comm]

```

1.103 QuantumInfo/Channels/MatrixMap.lean

```

public import Mathlib.LinearAlgebra.PiTensorProduct.Basis
public import QuantumInfo.ForMathlib.ContinuousLinearMap
public import QuantumInfo.ForMathlib.ComplexLaplaceTransform
public import QuantumInfo.ForMathlib.ContinuousSup
public import QuantumInfo.ForMathlib.Filter
public import QuantumInfo.ForMathlib.HermitianMat
public import QuantumInfo.ForMathlib.Isometry
public import QuantumInfo.ForMathlib.LinearEquiv
public import QuantumInfo.ForMathlib.MatrixNorm.TraceNorm
public import QuantumInfo.ForMathlib.Matrix
public import QuantumInfo.ForMathlib.Minimax
public import QuantumInfo.ForMathlib.Misc
public import QuantumInfo.ForMathlib.Unitary
public import QuantumInfo.States.Pure.Braket
public import QuantumInfo.States.Mixed.MState

```

section kraus_exists

variable [CommSemiring R] [StarRing R] [Fintype B]

```

theorem exists_kraus ( $\Phi$  : MatrixMap A B R) :
   $\exists$  r :  $\mathbb{N}$ ,  $\exists$  (M N : Fin r  $\rightarrow$  Matrix B A R),  $\Phi$  = of_kraus M N := by
  classical
  let K := ((B  $\times$  A)  $\times$  A)  $\times$  B
  let M0 : K  $\rightarrow$  Matrix B A R := fun ((b, a1), a2), b2 =>
    Matrix.single b a1 ( $\Phi$  (Matrix.single a1 a2 (1 : R)) b b2)
  let N0 : K  $\rightarrow$  Matrix B A R := fun ((_, _), a2), b2 => Matrix.single b2 a2
  let e : Fin (Fintype.card K)  $\simeq$  K := (Fintype.equivFin _).symm
  refine {Fintype.card K, M0  $\circ$  e, N0  $\circ$  e, ?_}
  apply choi_matrix_inj
  ext {j1, i1} {j2, i2}
  simp only [choi_matrix, of_kraus, LinearMap.coe_sum, LinearMap.coe_mk, Ad
    Finset.sum_apply]
  have hsum_reindex :
    ( $\sum$  x : Fin (Fintype.card K),
      (M0  $\circ$  e) x * Matrix.single i1 i2 (1 : R) * ((N0  $\circ$  e) x).conjTranspose)
    =
    ( $\sum$  y : K, (M0 y * Matrix.single i1 i2 (1 : R) * (N0 y).conjTranspose)
    rw [Matrix.sum_apply]
    exact e.sum_comp
    (fun y : K => (M0 y * Matrix.single i1 i2 (1 : R) * (N0 y).conjTranspose)
    rw [hsum_reindex, Fintype.sum_prod_type, Fintype.sum_prod_type, Fintype.s
    simp [M0, N0, Matrix.mul_apply, Matrix.single, ite_and]
    rw [Finset.sum_eq_single i2]
    · rw [Finset.sum_eq_single j2]
    · simp
    · intro b hb hbj
      simp [hbj, star_zero]
    · simp
    · intro a ha ha_ne
      simp [ha_ne, star_zero]
    · simp
  end kraus_exists

```

```

variable {ι : Type u} [DecidableEq ι] [Fintype ι]
  (Matrix.stdBasis R ((i:ι)  $\rightarrow$  d0 i) ((i:ι)  $\rightarrow$  d0 i))
  (Matrix.reindex (Equiv.arrowProdEquivProdArrow _ d0 d0)
    (Equiv.arrowProdEquivProdArrow _ dI dI)
    (LinearMap.toMatrix
      (_root_.Basis.piTensorProduct (fun i  $\mapsto$  Matrix.stdBasis R (dI i) (dI
      (_root_.Basis.piTensorProduct (fun i  $\mapsto$  Matrix.stdBasis R (d0 i) (d0

```

```

(PiTensorProduct.map  $\Lambda_i$ )))

theorem choi_matrix_piProd ( $\Lambda_i : \forall i, \text{MatrixMap } (dI\ i) (dO\ i) R$ ) :
  ( $\text{MatrixMap.piProd } \Lambda_i$ ).choi_matrix =
    Matrix.reindex
      (Equiv.arrowProdEquivProdArrow  $\iota\ dO\ dI$ )
      (Equiv.arrowProdEquivProdArrow  $\iota\ dO\ dI$ )
      ( $\text{Matrix.piProd } (\text{fun } i \Rightarrow (\Lambda_i\ i).\text{choi\_matrix})$ ) := by
  ext x y
  simp [ $\text{MatrixMap.choi\_matrix}$ ,  $\text{Matrix.piProd}$ ]
  rw [ $\text{MatrixMap.piProd}$ ,  $\leftarrow \text{Matrix.stdBasis\_eq\_single } (R := R) \ x.2\ y.2$ ,
     $\text{Matrix.toLin\_self}$ ,  $\text{Matrix.sum\_apply}$ ,  $\text{Finset.sum\_eq\_single } (x.1, y.1)$ ]
  · simp [ $\text{Matrix.reindex\_apply}$ ,  $\text{LinearMap.toMatrix\_apply}$ ,  $\text{Matrix.stdBasis}$ ,
     $\leftarrow \text{Matrix.single\_eq\_of\_single\_single}$ ]
  · intro z _ hz
    rw [ $\text{Matrix.stdBasis\_eq\_single}$ ]
    simp [ $\text{Matrix.single}$ ]
    intro hz1 hz2
    exact False.elim (hz ( $\text{Prod.ext } hz1\ hz2$ ))
  · simp
  simp [ $\text{piProd}$ ,  $\text{PiTensorProduct.map\_comp}$ ,  $\leftarrow \text{Matrix.toLin\_mul}$ ,  $\leftarrow \text{LinearMap.toMatrix\_c}$ ]

@[simp]
theorem piProd_id :
   $\text{piProd } (\text{fun } i \mapsto (\text{LinearMap.id} : \text{MatrixMap } (dI\ i) (dI\ i) R)) = \text{LinearMap.id} := by
  simp [ $\text{piProd}$ ,  $\text{PiTensorProduct.map\_id}$ ,  $\text{LinearMap.toMatrix\_id\_eq\_basis\_toMatrix}$ ,
     $\text{Module.Basis.toMatrix\_self}$ ,  $\text{Matrix.reindex\_apply}$ ,  $\text{Matrix.submatrix\_one\_equiv}$ ,
     $\text{Matrix.toLin\_one}$ ]$ 
```

1.104 QuantumInfo/Channels/Pinching.lean

```

public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Channels.CPTP
public import QuantumInfo.Channels.Dual
public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.Channels.Unbundled
public import QuantumInfo.States.Mixed.MState
public import QuantumInfo.Entropy.VonNeumann
public import QuantumInfo.Entropy.SSA
public import QuantumInfo.Entropy.Relative
public import QuantumInfo.Entropy.DPI

```

1.105 QuantumInfo/Channels/Unbundled.lean

```

public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.ForMathlib.MatrixNorm.TraceNorm
open scoped Matrix MatrixOrder ComplexOrder Matrix.Norms.L2Operator CStarAl

section piProd

variable {ι : Type u} [DecidableEq ι] [Fintype ι]
variable {dI : ι → Type v} [∀ i, Fintype (dI i)] [∀ i, DecidableEq (dI i)]
variable {d0 : ι → Type w} [∀ i, Fintype (d0 i)] [∀ i, DecidableEq (d0 i)]
variable {R : Type*} [CommSemiring R]

/-- The `MatrixMap.piProd` product of IsTracePreserving maps is also trace
/
theorem piProd {Λi : ∀ i, MatrixMap (dI i) (d0 i) R} (h₁ : ∀ i, (Λi i).IsTracePreserving) :
  (MatrixMap.piProd Λi).IsTracePreserving := by
  rw [IsTracePreserving_iff_trace_choi, MatrixMap.choi_matrix_piProd]
  ext f g
  simp [Matrix.traceLeft, Matrix.piProd, Matrix.reindex_apply]
  have htrace : ∀ i, (Λi i).choi_matrix.traceLeft = 1 := fun i =>
    (IsTracePreserving_iff_trace_choi (Λi i)).1 (h₁ i)
  have hprod : ∀ a b : ∀ i, dI i,
    (∑ x : ∀ i, d0 i, ∏ i, (Λi i).choi_matrix (x i, a i) (x i, b i)) =
    ∏ i, ∑ x, (Λi i).choi_matrix (x, a i) (x, b i) := fun a b => by
    simpa using
      (Fintype.prod_sum (f := fun i x => (Λi i).choi_matrix (x, a i) (x, b i)
by_cases hfg : f = g
  · subst hfg
    rw [hprod]
    have hdiag : ∀ i, ∑ x, (Λi i).choi_matrix (x, f i) (x, f i) = 1 := fun i =>
      simpa [Matrix.traceLeft] using congrFun₂ (htrace i) (f i) (f i)
    simp [hdiag]
  · obtain ⟨i, hi⟩ := Function.ne_iff.mp hfg
    have hfactor : ∑ x, (Λi i).choi_matrix (x, f i) (x, g i) = 0 := by
      simpa [Matrix.traceLeft, Matrix.one_apply, hi]
        using congrFun₂ (htrace i) (f i) (g i)
    rw [hprod, Finset.prod_eq_zero (Finset.mem_univ i) hfactor]
    simp [hfg]

end piProd

open scoped MatrixOrder in
/-- Kadison-Schwarz for completely positive subunital matrix maps. -
/
theorem cp_subunital_kadison_schwarz {M : MatrixMap A B ℂ} [DecidableEq B]

```

```

(hM : M.IsCompletelyPositive) (hM1 : M 1 ≤ (1 : Matrix B B ℂ))
(X : Matrix A A ℂ) :
(M X)H * M X ≤ M (XH * X) := by
have hblock :
  (Matrix.fromBlocks (M 1) (M X) ((M X)H) (M (XH * X))).PosSemidef := by
  classical
  obtain ⟨K, rfl⟩ := MatrixMap.IsCompletelyPositive.exists_kraus _ hM
  rw [MatrixMap.of_kraus_eq_sum_conj]
  convert Matrix.posSemidef_sum Finset.univ (fun k _ => by
    simp [MatrixMap.conj, Matrix.mul_assoc] using
      Matrix.fromBlocks_gram_posSemidef (X * (K k)H) (K k)H) using 1
  ext i j
  cases i <=> cases j <=>
    simp [Matrix.sum_apply, Matrix.fromBlocks_apply11, Matrix.fromBlocks_apply12,
      Matrix.fromBlocks_apply21, Matrix.fromBlocks_apply22, Matrix.conjTranspose_sum,
      Matrix.mul_assoc]
have hgap_block :
  (Matrix.fromBlocks (1 - M 1) 0 0 (0 : Matrix B B ℂ)).PosSemidef := by
  have hnonneg : (0 : Matrix B B ℂ) ≤ 1 - M 1 := by simp [sub_nonneg] using hM1
  rw [← show (CFC.sqrt (1 - M 1))H * CFC.sqrt (1 - M 1) = 1 - M 1 from by
    rw [show (CFC.sqrt (1 - M 1))H = CFC.sqrt (1 - M 1) from by
      simp using (CFC.sqrt_nonneg (1 - M 1)).1.eq]
    exact CFC.sqrt_mul_sqrt_self (1 - M 1) hnonneg]
  simp using Matrix.fromBlocks_gram_posSemidef
  (0 : Matrix B B ℂ) (CFC.sqrt (1 - M 1))
have hsum :
  (Matrix.fromBlocks (1 : Matrix B B ℂ) (M X) ((M X)H) (M (XH * X))).PosSemidef
  convert hblock.add hgap_block using 1
  ext i j
  cases i <=> cases j <=>
    simp [Matrix.fromBlocks, sub_eq_add_neg, add_left_comm, add_comm]
let I : Invertible (1 : Matrix B B ℂ) := invertibleOne
simp [sub_nonneg] using
  (Matrix.PosDef.fromBlocks11 (B := M X) (D := M (XH * X))
    (hA := (Matrix.PosDef.one : (1 : Matrix B B ℂ).PosDef))).mp hsum

open scoped MatrixOrder in
/-- A positive subunital matrix map is contractive on positive inputs. -
/
theorem positive_subunital_norm_apply_le {M : MatrixMap A B ℂ} [DecidableEq B]
(hM : M.IsPositive) (hM1 : M 1 ≤ (1 : Matrix B B ℂ))
{X : Matrix A A ℂ} (hX : 0 ≤ X) :
||M X|| ≤ ||X|| := by
let eA := Matrix.toEuclideanCLM (n := A) (□ := ℂ)
let eB := Matrix.toEuclideanCLM (n := B) (□ := ℂ)
have hXle : X ≤ ||X|| • (1 : Matrix A A ℂ) := by

```

```

refine (map_le_map_iff eA).mp ?_
have h := IsSelfAdjoint.le_algebraMap_norm_self (IsSelfAdjoint.of_nonneg
have hs : algebraMap ℝ (EuclideanSpace ℂ A → L[ℂ] EuclideanSpace ℂ A) ||e
  eA (||X|| • (1 : Matrix A A ℂ)) := by
  rw [Algebra.algebraMap_eq_smul_one]
  ext x i
  change (((||X|| : ℂ) • x).ofLp i) =
    (((||X|| : ℂ) • (1 : Matrix A A ℂ)) *v x.ofLp) i
  rw [Matrix.smul_mulVec, Matrix.one_mulVec, WithLp.ofLp_smul]
  exact h.trans_eq hs
have hMX_le : M X ≤ ||X|| • (1 : Matrix B B ℂ) :=
  (show M X ≤ ||X|| • M 1 by
    simp [sub_nonneg, map_sub, map_smul] using
      (hM (by simp [sub_nonneg] using hXle :
        (||X|| • (1 : Matrix A A ℂ) - X).PosSemidef)).nonneg).trans
    (smul_le_smul_of_nonneg_left hM1 (norm_nonneg X))
have hMX_nn : 0 ≤ M X := (hM (by simp [Matrix.nonneg_iff_posSemidef] usi
have hone : ||(1 : Matrix B B ℂ)|| ≤ 1 := by
  rcases subsingleton_or_nontrivial (Matrix B B ℂ) with h | h
  · simp [Subsingleton.elim (1 : Matrix B B ℂ) 0]
  · exact CStarRing.norm_one.le
refine (CStarAlgebra.norm_le_norm_of_nonneg_of_le (map_nonneg eB hMX_nn)
  ((map_le_map_iff eB).mpr hMX_le)).trans ?_
change ||||X|| • (1 : Matrix B B ℂ)|| ≤ ||X||
rw [show (||X|| • (1 : Matrix B B ℂ)) = ((||X|| : ℂ) • (1 : Matrix B B ℂ)) by
  simp using mul_le_mul_of_nonneg_left hone (norm_nonneg X)

open scoped MatrixOrder in
/-- A completely positive subunital matrix map is contractive in operator n
/
theorem cp_subunital_opNorm_le_one {M : MatrixMap A B ℂ} [DecidableEq B]
  (hM : M.IsCompletelyPositive) (hM1 : M 1 ≤ (1 : Matrix B B ℂ))
  (X : Matrix A A ℂ) :
  ||M X|| ≤ ||X|| := by
  refine (sq_le_sq (norm_nonneg (M X)) (norm_nonneg X)).mp ?_
  simp only [sq, ← CStarRing.norm_star_mul_self, Matrix.star_eq_conjTranspo
  let e := Matrix.toEuclideanCLM (n := B) (□ := ℂ)
  exact (CStarAlgebra.norm_le_norm_of_nonneg_of_le
    (map_nonneg e (star_mul_self_nonneg (M X)))
    ((map_le_map_iff e).mpr (cp_subunital_kadison_schwarz hM hM1 X))).tra
    (positive_subunital_norm_apply_le hM.IsPositive hM1 (star_mul_self_nonn

  rw [choi_PSD_iff_CP_map, MatrixMap.choi_matrix_piProd]
  convert Matrix.PosSemidef.submatrix
  (Matrix.PosSemidef.piProd (fun i => (choi_PSD_iff_CP_map (Λ i i)).1 (h1
    (Equiv.arrowProdEquivProdArrow ι d0 dI).symm using 1

```

1.106 QuantumInfo/ClassicalInfo/Channel.lean

```

ProbDistribution.mk' (fun os  $\mapsto$  [] k, (C.symb_dist (is k) (os k) :  $\mathbb{R}$ ))
  (fun _  $\mapsto$  Finset.prod_nonneg fun k _  $\mapsto$  Prob.zero_le_coe)
  (by
    have h := Finset.sum_prod_piFinset (Finset.univ : Finset 0)
      (fun (k : Fin n) (j : 0)  $\mapsto$  ((C.symb_dist (is k)) j :  $\mathbb{R}$ ))
    simp only [Fintype.piFinset_univ] at h
    simp only [h]
    exact Finset.prod_eq_one fun k _  $\mapsto$  ProbDistribution.normalized (C.symb_dist (is k) (os k) :  $\mathbb{R}$ ))

```

1.107 QuantumInfo/ClassicalInfo/Prob.lean

`QuantumInfo.States.Mixed.MState` density matrices in quantum mechanics, which are

1.108 QuantumInfo/Entropy/Axiomatized/Defs.lean

```

public import QuantumInfo.States.Mixed.MState
public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Channels.CPTP
public import QuantumInfo.Channels.Dual
public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.Channels.Unbundled

```

1.109 QuantumInfo/Entropy/Axiomatized/Renyi.lean

```

public import QuantumInfo.Entropy.Axiomatized.Defs

```

1.110 QuantumInfo/Entropy/DPI.lean

```

/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
module

public import QuantumInfo.Entropy.Relative

```

```
public import QuantumInfo.ForMathlib.HermitianMat.Sqrt
public import QuantumInfo.ForMathlib.HermitianMat.LiebConcavity
```

```
@[expose] public section
```

```
noncomputable section
```

```
variable {d d1 d2 d3 : Type*}
variable [Fintype d] [Fintype d1] [Fintype d2] [Fintype d3]
variable [DecidableEq d] [DecidableEq d1] [DecidableEq d2] [DecidableEq d3]
variable {dA dB dC dA1 dA2 : Type*}
variable [Fintype dA] [Fintype dB] [Fintype dC] [Fintype dA1] [Fintype dA2]
variable [DecidableEq dA] [DecidableEq dB] [DecidableEq dC] [DecidableEq dA1] [DecidableEq dA2]
variable {α : ℝ} {ρ σ : MState d}
```

```
open HermitianMat
```

```
open scoped InnerProductSpace RealInnerProductSpace Topology
```

```
/-!
```

```
# DPI (Data Processing Inequality)
```

The Data Processing Inequality (DPI) for the sandwiched Rényi relative entropy as a consequence, the quantum relative entropy.

```
## Proof structure (for α > 1)
```

Following Leditzky–Rouzé–Datta (arXiv:1306.5920), the proof proceeds as follows:

1. Define the **trace functional** $\tilde{Q}_\alpha(\rho||\sigma) = \text{Tr}[(\sigma^\gamma \rho \sigma^\gamma)^\alpha]$ where $\gamma = 1/(\alpha - 1)$. The sandwiched Rényi divergence satisfies $\tilde{D}_\alpha(\rho||\sigma) = \log(\tilde{Q}_\alpha(\rho||\sigma)) / (\alpha - 1)$.
2. The DPI for $\tilde{D}_\alpha$ reduces to **monotonicity** of $\tilde{Q}_\alpha$ under partial trace: $\tilde{Q}_\alpha(\rho_{AB}||\sigma_{AB}) \geq \tilde{Q}_\alpha(\rho_A||\sigma_A)$ for $\alpha > 1$.
3. This monotonicity is proved via the **twirling argument**:
 - $\tilde{Q}_\alpha$ is invariant under joint unitary conjugation.
 - $\tilde{Q}_\alpha$ is jointly convex for $\alpha > 1$ (Frank–Lieb).
 - A twirling set of unitaries $\{V_i\}$ averages any state to a product with a maximally mixed state.
 - $\tilde{Q}_\alpha$ is invariant under tensoring with a fixed state.
4. The general DPI for CPTP maps follows via **Stinespring dilation**: any CPTP map can be decomposed as ancilla preparation + unitary + partial trace.


```
open scoped Matrix ComplexOrder
open BigOperators
```

```
/-! ## The Sandwiched Trace Functional -/
```

```
/-- The sandwiched trace functional  $\tilde{Q}_\alpha(\rho||\sigma) = \text{Tr}[(\sigma^\gamma \rho \sigma^\gamma)^\alpha]$  where  $\gamma = (1-\alpha)/(2\alpha)$ .
```

```
This is the quantity underlying the sandwiched Rényi divergence:
 $\tilde{D}_\alpha(\rho||\sigma) = \log(\tilde{Q}_\alpha(\rho||\sigma)) / (\alpha - 1)$ .
```

```
Note: the `conj` operation gives  $A.\text{conj } B = B * A.\text{mat} * B^\dagger$ , and since  $\sigma^\gamma$  is Hermitian (self-adjoint),  $B^\dagger = B$ , so  $\rho.M.\text{conj } (\sigma.M ^ \gamma).\text{mat} = \sigma^\gamma * \rho * \sigma^\gamma$ . -
```

```
/
noncomputable def sandwichedTraceFunctional (α : ℝ) (ρ σ : MState d) : ℝ :=
  let γ := (1 - α) / (2 * α)
  ((ρ.M.conj (σ.M ^ γ).mat) ^ α).trace
```

```
notation "Q~_" α "(" ρ "||" σ ")" => sandwichedTraceFunctional α ρ σ
```

```
/-! ## Properties of the Trace Functional -/
```

```
/-
```

```
The sandwiched Rényi divergence equals  $\log(\tilde{Q}_\alpha) / (\alpha - 1)$  for  $\alpha > 0$ ,  $\alpha \neq 1$ , when  $\sigma.M.\text{ker} \leq \rho.M.\text{ker}$ .
```

```
-/
```

```
theorem sandwichedRelEntropy_eq_log_traceFunctional (hα₀ : 0 < α) (hα₁ : α ≠ 1)
  (hker : σ.M.ker ≤ ρ.M.ker) :
   $\tilde{D}_\alpha(\rho||\sigma) = \text{ENNReal.ofReal } (\text{Real.log } (\tilde{Q}_\alpha(\rho||\sigma)) / (\alpha - 1))$  := by
  rw [ENNReal.ofReal_eq_coe_nnreal]
  unfold SandwichedRelEntropy sandwichedTraceFunctional
  split
  next h => simp_all only
  next h => rfl
```

```
/-
```

```
 $\tilde{Q}_\alpha(\rho||\sigma)$  is nonneg when  $\alpha > 0$ .
```

```
-/
```

```
theorem sandwichedTraceFunctional_nonneg (ρ σ : MState d) :
  0 ≤  $\tilde{Q}_\alpha(\rho||\sigma)$  := by
  rw [sandwichedTraceFunctional]
  apply trace_nonneg
  apply rpow_nonneg
  positivity
```

```
/-- The trace functional is strictly positive when the kernel condition holds. Under
 $\sigma.M.\text{ker} \leq \rho.M.\text{ker}$  (i.e.  $\text{supp}(\rho) \subseteq \text{supp}(\sigma)$ ), the sandwich  $\sigma^\gamma \rho \sigma^\gamma \neq 0$ 
```

```

because  $\rho$  has support inside  $\sigma$ 's support. -/
theorem sandwichedTraceFunctional_pos
  ( $\rho \ \sigma$  : MState d) (hker :  $\sigma.M.ker \leq \rho.M.ker$ ) :
   $0 < \tilde{Q}_\alpha(\rho||\sigma)$  := by
  rw [sandwichedTraceFunctional]
  apply trace_pos
  apply rpow_pos
  apply conj_pos  $\rho.pos$ 
  grw [← hker]
  exact ker_rpow_le_of_nonneg  $\sigma.nonneg$ 

/-! ## Unitary Invariance

 $\tilde{Q}_\alpha(U\rho U^\dagger||U\sigma U^\dagger) = \tilde{Q}_\alpha(\rho||\sigma)$  for any unitary  $U$ .

Here,  $\text{conj } U.val \ A$  denotes  $U * A * U^\dagger$ , so "conjugating  $\rho$  and  $\sigma$  by the same unitary" means applying  $\text{conj } U.val$  to both. -/

/-
The trace functional is invariant under joint unitary conjugation:
 $\text{Tr}[(U \ \sigma \ U^\dagger)^\gamma (U \ \rho \ U^\dagger) (U \ \sigma \ U^\dagger)^\gamma] = \text{Tr}[(\sigma^\gamma \ \rho \ \sigma^\gamma)^\alpha]$ .
This corresponds to equation (2.3) in the paper.
Proved using  $\text{rpow\_conj\_unitary}$  ( $f(UXU^\dagger) = U f(X) U^\dagger$ ) and  $\text{conj\_conj}$ .
-/
theorem sandwichedTraceFunctional_conj_unitary_hermitian
  (U : Matrix.unitaryGroup d  $\mathbb{C}$ ) (A B : HermitianMat d  $\mathbb{C}$ ) :
  let  $\gamma := (1 - \alpha) / (2 * \alpha)$ 
  ((A.conj U.val).conj ((B.conj U.val) ^  $\gamma$ ).mat ^  $\alpha$ ).trace =
    ((A.conj (B ^  $\gamma$ ).mat) ^  $\alpha$ ).trace := by
  have h_conj_conj :  $\forall (A \ B : \text{HermitianMat } d \ \mathbb{C}) (U : \text{Matrix.unitaryGroup } d \ \mathbb{C})$ 
    (conj U.val A).conj ((conj U.val B).mat) = conj U.val (A.conj B.mat)
  intros A B U
  simp [conj]
  have h_unitary :  $\forall (U : \text{Matrix.unitaryGroup } d \ \mathbb{C}), U.val * U.val.conjTra$ 
    exact fun U => U.2.2
  simp [← mul_assoc]
  have := h_unitary U; simp_all [Matrix.mul_assoc, mul_eq_one_comm.mp thi
  simp_all [conj_apply_mat, rpow_conj_unitary]

/-- The trace functional is invariant under joint unitary conjugation of MS
/
theorem sandwichedTraceFunctional_conj_unitary_MState
  (U : Matrix.unitaryGroup d  $\mathbb{C}$ ) ( $\rho \ \sigma$  : MState d) :
   $\tilde{Q}_\alpha(\rho.U\_conj \ U||\sigma.U\_conj \ U) = \tilde{Q}_\alpha(\rho||\sigma)$  := by
  unfold sandwichedTraceFunctional MState.U_conj
  exact sandwichedTraceFunctional_conj_unitary_hermitian U  $\rho.M$   $\sigma.M$ 

```

```
/-! ## Joint Convexity for  $\alpha > 1$ 
```

The trace functional $\tilde{Q}_\alpha$ is jointly convex for $\alpha > 1$. This is proved by Frank and Lieb via a variational formula and strict convexity of trace functions.

```
### Trace functions convexity
```

The following result is used in the proof: for a convex function $g : \mathbb{R} \rightarrow \mathbb{R}$, the map $A \mapsto \text{Tr}[g(A)]$ on Hermitian matrices is convex (Carlen, Theorem 2.10). - /

```
namespace HermitianMat
```

```
end HermitianMat
```

```
/-! ### Variational formula for the trace functional
```

Following Frank–Lieb, for $\alpha > 1$ we define

```
 $f_\alpha(H, \rho, \sigma) = \alpha \cdot \text{Tr}[\rho \cdot H] - (\alpha - 1) \cdot \text{Tr}[(\sigma^{-\gamma} H \sigma^{-\gamma})^{\alpha/(\alpha-1)}]$ 
```

where $\gamma = (1-\alpha)/(2\alpha)$ (so $-\gamma = (\alpha-1)/(2\alpha) > 0$).

Key facts (each stated as a lemma below):

1. $\tilde{Q}_\alpha(\rho||\sigma) = \sup_{\{H \geq 0\}} f_\alpha(H, \rho, \sigma)$ for $\alpha > 1$.
2. For fixed H , f_α is linear in ρ (hence convex).
3. For fixed H , f_α is convex in σ (uses Lieb concavity).
4. Therefore f_α is jointly convex in (ρ, σ) for fixed H .
5. The supremum of jointly convex functions is jointly convex.

```
-/
```

```
/-- The variational function  $f_\alpha(H, \rho, \sigma) = \alpha \cdot \langle \rho, H \rangle - (\alpha - 1) \cdot \text{Tr}[(\sigma^{-\gamma} H \sigma^{-\gamma})^{\alpha/(\alpha-1)}]$  where  $\gamma = (1-\alpha)/(2\alpha)$ . For fixed  $H \geq 0$ , this is linear in  $\rho$  and convex in  $\sigma$ . Frank–Lieb show that  $\tilde{Q}_\alpha(\rho||\sigma) = \sup_{\{H \geq 0\}} f_\alpha(H, \rho, \sigma)$  for  $\alpha > 1$ . -
```

```
/
```

```
noncomputable def f_alpha (α : ℝ) (H : HermitianMat d ℂ) (ρ σ : MState d) : ℝ :=
  let γ : ℝ := (1 - α) / (2 * α)
  α * ⟨ρ.M, H⟩_ℝ - (α - 1) * ((H.conj (σ.M ^ (-γ)).mat) ^ (α / (α - 1))).trace
```

```
/-- The optimizer in the variational formula:  $H_{\text{hat}} = \sigma^\gamma (\sigma^{-\gamma} \rho \sigma^{-\gamma})^{\alpha-1} \sigma^\gamma$  where  $\gamma = (1-\alpha)/(2\alpha)$ . -/
```

```
noncomputable def H_hat (α : ℝ) (ρ σ : MState d) : HermitianMat d ℂ :=
  let γ := (1 - α) / (2 * α)
  ((ρ.M.conj (σ.M ^ γ).mat) ^ (α - 1)).conj (σ.M ^ γ).mat
```

```
/-
```

****Step 1a**:** The optimizer H_{hat} is PSD.

```
-/
```

```
theorem H_hat_nonneg (ρ σ : MState d) : 0 ≤ H_hat α ρ σ := by
```

```

    apply conj_nonneg
    apply rpow_nonneg
    positivity

/--
For a PSD Hermitian matrix B whose kernel contains A's kernel, conjugating
support projection leaves B unchanged.
-/
private lemma conj_supportProj_eq_of_ker_le (A B : HermitianMat d ℂ) (hker :
  B.conj (A.supportProj).mat = B := by
  ext i j
  simp [*, conj]
  suffices h_conj : A.supportProj.mat * B.mat * A.supportProj.mat = B.mat by
    exact congr($h_conj i j)
  have h_unitary := mul_supportProj_of_ker_le hker
  apply_fun Matrix.conjTranspose at h_unitary ⊢
  · simp_all only [Matrix.conjTranspose_mul, conjTranspose_mat]
  · exact Matrix.conjTranspose_injective

/--
The kernel of  $\sigma$  is contained in the kernel of  $(\rho.\text{conj } (\sigma^\gamma))^{\{\alpha-1\}}$  when  $\gamma \neq 0$  and  $\alpha > 1$ .
-/
private lemma ker_sigma_le_ker_conj_rpow (ρ σ : MState d) {γ : ℝ} (hγ : γ ≠ 0)
  σ.M.ker ≤ ((ρ.M.conj (σ.M ^ γ).mat) ^ (α - 1)).ker := by
  rw [ker_rpow_eq_of_nonneg (by positivity) hα1]
  intro x hx
  have h_ker_rpow : x ∈ (σ.M ^ γ).ker := by
    rwa [ker_rpow_eq_of_nonneg σ.nonneg hγ]
  simp_all [ker, lin]

/-- Sub-lemma for Step 1b: the conj of  $H_{\text{hat}}$  by  $\sigma^{-\gamma}$  simplifies to  $(\rho.M.\text{conj } (\sigma.M ^ \gamma).mat) ^ (\alpha - 1)$ .
This uses  $\sigma^{-\gamma} \cdot \sigma^\gamma = \text{identity}$  (on support) to cancel the outer  $\sigma^\gamma$  factor.
-/
theorem H_hat_conj_sigma (hα : 1 < α) (ρ σ : MState d) :
  let γ := (1 - α) / (2 * α)
  (H_hat α ρ σ).conj (σ.M ^ (-γ)).mat = (ρ.M.conj (σ.M ^ γ).mat) ^ (α - 1)
  intro γ
  have hγ : γ ≠ 0 := by
    simp only [γ]; rw [div_ne_zero_iff]; exact ⟨by linarith, by linarith⟩
  have hα1 : α - 1 ≠ 0 := by linarith
  show (((ρ.M.conj (σ.M ^ γ).mat) ^ (α - 1)).conj (σ.M ^ γ).mat).conj (σ.M ^ (-γ)).mat =
    (ρ.M.conj (σ.M ^ γ).mat) ^ (α - 1)
  rw [conj_conj]
  rw [rpow_neg_mul_rpow_eq_supportProj σ.nonneg hγ]

```

```

exact conj_supportProj_eq_of_ker_le σ.M _ (ker_sigma_le_ker_conj_rpow ρ σ hα1)

/-
Sub-lemma for Step 1b: the inner product  $\langle \rho.M, H_{\hat{\alpha}} \rangle$  equals  $\text{Tr}[(\sigma^\gamma \rho \sigma^\gamma)^\alpha]$ .
By cyclicity of trace:  $\text{Tr}[\rho \cdot \sigma^\gamma \cdot (\sigma^\gamma \rho \sigma^\gamma)^{\alpha-1} \cdot \sigma^\gamma] = \text{Tr}[(\sigma^\gamma \rho \sigma^\gamma)^\alpha]$ .
-/
theorem inner_rho_H_hat (hα : 1 < α) (ρ σ : MState d) :
  let γ := (1 - α) / (2 * α)
   $\langle \rho.M, H_{\hat{\alpha}} \rangle_{\mathbb{R}} = ((\rho.M.\text{conj } (\sigma.M ^ \gamma).\text{mat}) ^ \alpha).\text{trace} :=$  by
  unfold H_hat; simp [inner_def]
  have h_cyclic : (ρ.M * (σ.M ^ ((1 - α) / (2 * α))).mat *
    ((ρ.M.conj (σ.M ^ ((1 - α) / (2 * α))).mat) ^ (α - 1)).mat *
    (σ.M ^ ((1 - α) / (2 * α))).mat).trace =
    ((ρ.M.conj (σ.M ^ ((1 - α) / (2 * α))).mat) ^ α).trace := by
  have h_cyclic : (ρ.M.conj (σ.M ^ ((1 - α) / (2 * α))).mat).mat *
    ((ρ.M.conj (σ.M ^ ((1 - α) / (2 * α))).mat) ^ (α - 1)).mat =
    ((ρ.M.conj (σ.M ^ ((1 - α) / (2 * α))).mat) ^ α).mat := by
  have := @mat_rpow_add
  specialize this (show 0 ≤ conj (σ.M ^ ((1 - α) / (2 * α))).mat ρ.M from ?_)
    (show (1 : ℝ) + (α - 1) ≠ 0 from by linarith)
  · positivity
  · aesop
  convert congr_arg Matrix.trace h_cyclic using 1
  · rw [← Matrix.trace_mul_comm]; simp [Matrix.mul_assoc]
  · simp [trace]
    norm_num [Matrix.trace]
    refine Finset.sum_congr rfl fun i _ => ?_
    convert Complex.ofReal_re (((conj (σ.M ^ ((1 - α) / (2 * α))).mat) ρ.M ^ α) i)
    simp [Complex.ext_iff]
  simp_all [← Matrix.mul_assoc]

/-
**Step 1b**: Evaluating `f_α` at the optimizer `H_hat` gives `Q̃_α(ρ||σ)`.
This is the key computation that verifies the variational formula at the optimizer.
Proof:  $f_\alpha(H_{\hat{\alpha}}, \rho, \sigma) = \alpha \cdot \text{Tr}[(\sigma^\gamma \rho \sigma^\gamma)^\alpha] - (\alpha-1) \cdot \text{Tr}[(\sigma^\gamma \rho \sigma^\gamma)^\alpha] = \text{Tr}[(\sigma^\gamma \rho \sigma^\gamma)^\alpha]$ 
-/
theorem f_alpha_at_optimizer (hα : 1 < α) (ρ σ : MState d) :
  f_alpha α (H_hat α ρ σ) ρ σ = Q̃_α(ρ||σ) := by
  have h_inner :  $\langle \rho.M, H_{\hat{\alpha}} \rangle_{\mathbb{R}} = ((\rho.M.\text{conj } (\sigma.M ^ \gamma).\text{mat})$ 
    exact inner_rho_H_hat hα ρ σ
  have h_conj : (H_hat α ρ σ).conj (σ.M ^ ((α - 1) / (2 * α))).mat =
    (ρ.M.conj (σ.M ^ ((1 - α) / (2 * α))).mat) ^ (α - 1) := by
    convert H_hat_conj_sigma (hα := hα) (ρ := ρ) (σ := σ) using 1
    ring_nf!
  unfold f_alpha sandwichedTraceFunctional

```

```

simp_all [sub_div]
rw [← rpow_mul]; norm_num [show  $\alpha \neq 0$  by positivity, show  $\alpha - 1 \neq 0$  by li
· rw [mul_div_cancel0 _ (by linarith)]; ring
· apply_rules [conj_nonneg]
exact  $\rho$ .nonneg

/--
For PSD `A` and ` $\gamma \neq 0$ `, the product ` $A^\gamma * A^{-\gamma}$ ` equals the support proj
of `A`. This is because ` $x^\gamma * x^{-\gamma} = \text{if } x = 0 \text{ then } 0 \text{ else } 1$ ` for ` $x \geq 0$ `
-/
lemma rpow_mul_neg_rpow_eq_supportProj {A : HermitianMat d  $\mathbb{C}$ }
  (hA :  $0 \leq A$ ) ( $\gamma : \mathbb{R}$ ) (h $\gamma$  :  $\gamma \neq 0$ ) :
  (A ^  $\gamma$ ).mat * (A ^ (- $\gamma$ )).mat = A.supportProj.mat := by
  rw [supportProj_eq_cfc]
  rw [rpow_eq_cfc, rpow_eq_cfc]
  rw [← mat_cfc_mul_apply]
  refine congr_arg _ (cfc_congr_of_nonneg hA ?_)
  intro x (hx :  $0 \leq x$ )
  rcases eq_or_ne x 0 with hx' | hx'
  · simp [hx', h $\gamma$ ]
  · simp [hx', Real.rpow_neg hx]
  exact mul_inv_cancel0 (by positivity)

/--
The support projection of `A` acts as identity on `B` when ` $A.\text{ker} \leq B.\text{ker}$ '.
Since `A.supportProj` projects onto ` $\text{ker}(A)^\perp$ ` and `B` is zero on ` $\text{ker}(A)$ ',
the projection preserves `B`.
-/
lemma supportProj_mul_of_ker_le {A B : HermitianMat d  $\mathbb{C}$ }
  (hker :  $A.\text{ker} \leq B.\text{ker}$ ) :
  A.supportProj.mat * B.mat = B.mat := by
  contrapose! hker
  simp_all [SetLike.le_def]
  -- Since $B$ is not in the kernel of $A$, there exists some $x \in \text{ker}(A)^\perp$
  obtain (x, hx) :  $\exists x : \text{EuclideanSpace } \mathbb{C} \ d, A.\text{mat} *_\vee x = 0 \wedge B.\text{mat} *_\vee x \neq 0$ 
  contrapose! hker
  have h_support :  $\forall x : \text{EuclideanSpace } \mathbb{C} \ d, B.\text{mat} *_\vee x = B.\text{mat} *_\vee (A.\text{supportProj}.\text{mat} *_\vee x)$ 
  intro x
  have h_support :  $x.\text{ofLp} = A.\text{supportProj}.\text{mat} *_\vee x.\text{ofLp} + A.\text{kerProj}.\text{mat} *_\vee x.\text{ofLp}$ 
  have h_support :  $A.\text{supportProj}.\text{mat} + A.\text{kerProj}.\text{mat} = 1$  := by
    simp [add_comm]
    simp [← Matrix.ext_iff]
    intro i j; exact (by
      have h_support :  $A.\text{kerProj} + A.\text{supportProj} = 1$  := by
        exact kerProj_add_supportProj A
      convert congr_arg (fun f => f i j) h_support using 1)

```

```

    rw [← Matrix.add_mulVec, h_support, Matrix.one_mulVec]
    have hsup : B.mat *v (A.kerProj.mat *v x.ofLp) = 0 := by
      convert hker _ _
      have h_support : A.mat * A.kerProj.mat = 0 := by
        have h_support : A.mat * A.kerProj.mat = A.mat * (1 - A.supportProj.mat) :
          congr
        have h_support : A.kerProj + A.supportProj = 1 := by
          exact kerProj_add_supportProj A
        exact eq_sub_of_add_eq <| congr_arg Subtype.val h_support
      rw [h_support, mul_sub, mul_one, sub_eq_zero]
      exact Eq.symm (mul_supportProj_of_ker_le fun [x] a => a)
      convert congr_arg (fun m => m *v x.ofLp) h_support using 1
      · simp
      · simp
    convert congr_arg (fun y => B.mat *v y) h_support using 1
    simp [Matrix.mulVec_add, hsup]
    have h_support : B.mat = B.mat * A.supportProj.mat := by
      ext i j
      convert congr_fun (h_support (EuclideanSpace.single j 1)) i using 1
      · simp [Matrix.mulVec, dotProduct]
      · simp [Matrix.mulVec, dotProduct]
      rfl
    have h_support : B.mat = B.mat.conjTranspose := by
      exact B.2.symm
    have h_support : (B.mat * A.supportProj.mat).conjTranspose = A.supportProj.mat *
      simp [Matrix.conjTranspose_mul]
    lia
    refine {x, ?_, ?_}
    · simpa [ker, lin, funext_iff, Matrix.toLpLin] using hx.1
    · rw [mem_ker_iff_mulVec_zero]
      exact hx.right

/--
Under the support condition  $\sigma.M.ker \leq \rho.M.ker$  (i.e.,  $\text{supp}(\rho) \subseteq \text{supp}(\sigma)$ ),
conjugation by  $\sigma^\gamma$  and  $\sigma^{-\gamma}$  preserves the inner product:
 $\langle \rho.M, H \rangle = \langle \sigma^\gamma \rho \sigma^\gamma, \sigma^{-\gamma} H \sigma^{-\gamma} \rangle$ . This holds because the kernel condition
ensures  $\rho$  is supported on  $\text{supp}(\sigma)$ , where  $\sigma^\gamma \sigma^{-\gamma}$  acts as the identity.
-/
lemma inner_eq_inner_conj_of_ker_le (p σ : MState d)
  (H : HermitianMat d ℂ) (hker : σ.M.ker ≤ p.M.ker) (γ : ℝ) (hγ : γ ≠ 0) :
  ⟨p.M, H⟩_ℝ = ⟨p.M.conj (σ.M ^ γ).mat, H.conj (σ.M ^ (-
γ)).mat⟩_ℝ := by
  -- Since  $\sigma^\gamma \sigma^{-\gamma}$  acts as the identity on the support of  $\rho$ , we can simplify
  have h_support :
    (σ.M ^ γ).mat * (σ.M ^ (-γ)).mat = σ.M.supportProj.mat ∧
    (σ.M ^ (-γ)).mat * (σ.M ^ γ).mat = σ.M.supportProj.mat :=

```

```

      (rpow_mul_neg_rpow_eq_supportProj σ.nonneg γ hy,
       by simpa using rpow_mul_neg_rpow_eq_supportProj σ.nonneg (-
γ) (neg_ne_zero.mpr hy))
    simp only [inner_def, conj_apply_mat]
    have h_support :
      σ.M.supportProj.mat * ρ.M.mat = ρ.M.mat ∧
      ρ.M.mat * σ.M.supportProj.mat = ρ.M.mat := by
    exact (supportProj_mul_of_ker_le hker, mul_supportProj_of_ker_le hker)
    have h_trace_cyclic :
      Matrix.trace ((σ.M ^ γ).mat * ρ.M.mat * (σ.M ^ γ).mat *
        (σ.M ^ (-γ)).mat * H.mat * (σ.M ^ (-γ)).mat) =
      Matrix.trace ((σ.M ^ (-γ)).mat * (σ.M ^ γ).mat * ρ.M.mat *
        (σ.M ^ γ).mat * (σ.M ^ (-γ)).mat * H.mat) := by
    rw [← Matrix.trace_mul_comm]
    simp [Matrix.mul_assoc]
    simp_all [mul_assoc, Matrix.trace_mul_comm ((σ.M ^ γ).mat)]
    simp_all [← mul_assoc]

/-- **Step 1c**: `H_hat` is a maximizer: for all `H ≥ 0`, `f_α(H) ≤ f_α(H_hat)`
This uses the trace Young inequality: for PSD `A, B` and conjugate exponent
`1/p + 1/q = 1`, `Tr[A B] ≤ Tr[A^p]/p + Tr[B^q]/q`.
Applied with `A = σ^γ ρ σ^γ`, `B = σ^{-γ} H σ^{-γ}`, `p = α`, `q = α/(α-1)`,
the inner product identity `⟨ρ, H⟩ = ⟨A, B⟩` (under the support condition)
`f_α(H) ≤ Tr[A^α] = Q̃_α(ρ||σ) = f_α(H_hat)`.
Note: the support condition `σ.M.ker ≤ ρ.M.ker` (i.e.,  $\text{supp}(\rho) \subseteq \text{supp}(\sigma)$ ) is
Without it, the theorem is false: taking  $\rho$  orthogonal to  $\sigma$  gives  $\tilde{Q}_\alpha = 0$  but
`f_α(H) = α · Tr[ρH] > 0` for appropriate  $H$ . -/
theorem f_alpha_le_at_optimizer (hα : 1 < α) (ρ σ : MState d)
  (H : HermitianMat d ℂ) (hH : 0 ≤ H) (hker : σ.M.ker ≤ ρ.M.ker) :
  f_alpha α H ρ σ ≤ f_alpha α (H_hat α ρ σ) ρ σ := by
  rw [f_alpha_at_optimizer hα]
  -- Goal: f_alpha α H ρ σ ≤ Q̃_α(ρ||σ)
  set γ : ℝ := (1 - α) / (2 * α) with hy_def
  have hy : γ ≠ 0 := by
    intro h; have h1 : (1 - α) / (2 * α) = 0 := hy_def ▸ h
    have h2 : (2 : ℝ) * α ≠ 0 := by positivity
    rw [div_eq_zero_iff] at h1; rcases h1 with h1 | h1 <=> linarith
  set A := ρ.M.conj (σ.M ^ γ).mat
  set B := H.conj (σ.M ^ (-γ)).mat
  have hA_nn : 0 ≤ A := conj_nonneg _ ρ.nonneg
  have hB_nn : 0 ≤ B := conj_nonneg _ hH
  have h_inner : ⟨ρ.M, H⟩_ℝ = ⟨A, B⟩_ℝ :=
    inner_eq_inner_conj_of_ker_le ρ σ H hker γ hy
  have hpq : 1 / α + 1 / (α / (α - 1)) = 1 := by field_simp; ring
  have h_young := trace_young A B hA_nn hB_nn α (α / (α - 1)) hα hpq

```



```

have hα_pos : (0 : ℝ) < α := by linarith
have hαml_pos : (0 : ℝ) < α - 1 := by linarith
-- Multiply h_young by α and simplify
have h_scaled : α * ⌊A, B⌋_ℝ ≤
  (A ^ α).trace + (α - 1) * (B ^ (α / (α - 1))).trace := by
  have := mul_le_mul_of_nonneg_left h_young hα_pos.le
  have h_simp : α * ((A ^ α).trace / α + (B ^ (α / (α - 1))).trace / (α / (α - 1)))
    (A ^ α).trace + (α - 1) * (B ^ (α / (α - 1))).trace := by
    field_simp
    linarith
-- Goal is definitionally: α * ⌊p.M, H⌋ - (α-1) * (B ^ (α/(α-
1))).trace ≤ (A ^ α).trace
-- which follows from h_scaled and h_inner
change α * ⌊p.M, H⌋_ℝ - (α - 1) * (B ^ (α / (α - 1))).trace ≤ (A ^ α).trace
have h_inner_scaled : α * ⌊p.M, H⌋_ℝ = α * ⌊A, B⌋_ℝ := by rw [h_inner]
linarith [h_scaled, h_inner_scaled]

/--
**Step 1 (Variational formula)**: For `α > 1`, the trace functional equals the
supremum of `f_α` over all PSD `H`:
`Q_α(p||σ) = ⌊ (H : HermitianMat d ℂ) (⌊ : 0 ≤ H), f_alpha α H p σ`.
The optimizer is `H_hat = σ^γ (σ^γ ρ σ^γ)^(α-1) σ^γ`.
-/
theorem traceFunctional_eq_iSup_f_alpha (hα : 1 < α) (ρ σ : MState d) (hker : σ.M.ke
  Q_α(p||σ) = ⌊ (H : {H : HermitianMat d ℂ // 0 ≤ H}), f_alpha α H.1 p σ := by
  rw [@ciSup_eq_of_forall_le_of_forall_lt_exists_gt]
  · intro i
    rw [← f_alpha_at_optimizer hα ρ σ]
    exact f_alpha_le_at_optimizer hα ρ σ i i.2 hker
  · intro w hw
    exact ⟨(H_hat α ρ σ, H_hat_nonneg ρ σ), hw.trans_le (f_alpha_at_optimizer hα ρ σ

/-- (Convexity in σ): For fixed `H ≥ 0` and `ρ`, and `α > 1`, the map
`σ ↦ f_alpha α H ρ σ` is convex. The key is that for `p = α/(α-1) > 1`:
• `A ↦ Tr[A^p]` is convex on PSD matrices (trace function convexity, Theorem 2.10 of
• `σ ↦ σ^{-γ} H σ^{-γ}` is *concave* in `σ` by Lieb concavity (since `−γ = (α-1)/(2α
• The composition of a convex non-decreasing function with a concave function is con
  but we actually need the sign: the second term has a factor `-(α-1) < 0` which
  flips concave → convex.
More precisely: `σ ↦ Tr[(σ^{-γ} H σ^{-γ})^p]` is concave (by Lieb + trace function c
so `σ ↦ -(α-1) · Tr[(σ^{-γ} H σ^{-γ})^p]` is convex. -/
theorem f_alpha_convex_in_sigma (hα : 1 < α) (H : HermitianMat d ℂ) (hH : 0 ≤ H)
  (ρ : MState d) {ι : Type*} [Fintype ι]
  (w : ι → ℝ) (hw_nonneg : ∀ i, 0 ≤ w i) (hw_sum : ∑ i, w i = 1)
  (σs : ι → MState d) (σ_mix : MState d)
  (hσ_mix : σ_mix.M = ∑ i, w i • (σs i).M) :

```

```

    f_alpha α H ρ σ_mix ≤ ∑ i, w i * f_alpha α H ρ (σ i) := by
    have hα_pos : 0 < α - 1 := by linarith
    -- Define the σ-dependent trace function on HermitianMat
    let s := (α - 1) / (2 * α)
    let p := α / (α - 1)
    let F : HermitianMat d ℂ → ℝ := fun σ => ((H.conj (σ ^ s).mat) ^ p).trace
    -- f_alpha relates to F via: f_alpha α H ρ σ = α * ∫ ρ.M, H ∫ - (α -
1) * F(σ.M)
    -- because -γ = -((1-α)/(2α)) = (α-1)/(2α) = s
    have hf_eq : ∀ σ : MState d, f_alpha α H ρ σ = α * ∫ ρ.M, H ∫ - (α - 1) *
    intro σ
    show _ = α * ∫ ρ.M, H ∫ - (α - 1) *
      ((H.conj (σ.M ^ ((α - 1) / (2 * α))).mat) ^ (α / (α - 1))).trace
    unfold f_alpha; ring_nf
    simp_rw [hf_eq]
    -- Reduce to concavity: ∑ w_i * F(σ_i.M) ≤ F(σ_mix.M)
    suffices h : ∑ i, w i * F (σ i).M ≤ F σ_mix.M by
    have h1 : ∑ i, w i * ((α - 1) * F (σ i).M) = (α - 1) * ∑ i, w i * F (σ
    rw [Finset.mul_sum]; congr 1; ext i; ring
    simp only [mul_sub, Finset.sum_sub_distrib, ← Finset.sum_mul, hw_sum, o
    linarith [mul_le_mul_of_nonneg_left h (le_of_lt hα_pos)]
    -- Apply ConcaveOn.le_map_sum from trace_conj_rpow_concave
    have hF_concave : ConcaveOn ℝ {σ : HermitianMat d ℂ | 0 ≤ σ} F :=
    trace_conj_rpow_concave hα H hH
    have h_jensen := hF_concave.le_map_sum
    (t := Finset.univ) (w := w) (p := fun i => (σ i).M)
    (fun i _ => hw_nonneg i)
    (by simp [hw_sum])
    (fun i _ => (σ i).nonneg)
    rw [← hσ_mix] at h_jensen
    convert h_jensen using 1

/-
**Step 4 (Joint convexity of f_α)**: For fixed `H ≥ 0` and `α > 1`, the map
`(ρ, σ) ↦ f_alpha α H ρ σ` is jointly convex. This follows from Steps 2 and
f_α decomposes as a function linear in ρ (independent of σ) plus a function
in σ (independent of ρ).
-/
theorem f_alpha_jointly_convex (hα : 1 < α) (H : HermitianMat d ℂ) (hH : 0
{ι : Type*} [Fintype ι]
(w : ι → ℝ) (hw_nonneg : ∀ i, 0 ≤ w i) (hw_sum : ∑ i, w i = 1)
(ps σs : ι → MState d) (ρ_mix σ_mix : MState d)
(hρ_mix : ρ_mix.M = ∑ i, w i • (ps i).M)
(hσ_mix : σ_mix.M = ∑ i, w i • (σs i).M) :
f_alpha α H ρ_mix σ_mix ≤ ∑ i, w i * f_alpha α H (ps i) (σs i) := by
convert f_alpha_convex_in_sigma hα H hH ρ_mix _ _ _ using 1

```

```

any_goals assumption
constructor <=> intro h
· exact fun σ_mix hσ_mix =>
  f_alpha_convex_in_sigma hα H hH ρ_mix w hw_nonneg hw_sum os σ_mix hσ_mix
· apply (h σ_mix hσ_mix).trans
  unfold f_alpha
  simp [hp_mix]
  simp [sum_inner, inner_smul_left, mul_sub, sub_mul, mul_comm, mul_left_comm, Fin
  simp [← Finset.mul_sum, ← Finset.sum_mul, hw_sum]

/-
The range of `H ↦ f_alpha α H ρ σ` over PSD `H` is bounded above.
This follows from the variational formula: the supremum equals `Q̃_α(ρ||σ)`,
which is a finite real number.
-/
theorem f_alpha_bddAbove (hα : 1 < α) (ρ σ : MState d) (hker : σ.M.ker ≤ ρ.M.ker) :
  BddAbove (Set.range (fun H : {H : HermitianMat d ℂ // 0 ≤ H} => f_alpha α H.1 ρ
  exact {_, Set.forall_mem_range.mpr fun H => f_alpha_le_at_optimizer hα ρ σ _ H.2 h

/-
**Step 5 (Sup preserves convexity)**: The supremum over `H ≥ 0` of the jointly
convex `f_alpha α H` is jointly convex. This is a standard fact: for each `H`,
`f_alpha α H (ρ_mix) (σ_mix) ≤ ∑ w_i f_alpha α H (ρ_i) (σ_i) ≤ ∑ w_i sup_H f_alpha ...`,
so taking sup on the left gives the result.
-/
theorem iSup_f_alpha_jointly_convex (hα : 1 < α)
  {ι : Type*} [Fintype ι]
  (w : ι → ℝ) (hw_nonneg : ∀ i, 0 ≤ w i) (hw_sum : ∑ i, w i = 1)
  (ps os : ι → MState d) (ρ_mix σ_mix : MState d)
  (hp_mix : ρ_mix.M = ∑ i, w i • (ps i).M)
  (hσ_mix : σ_mix.M = ∑ i, w i • (os i).M)
  (hker : ∀ i, (os i).M.ker ≤ (ps i).M.ker) :
  (⊔ (H : {H : HermitianMat d ℂ // 0 ≤ H}), f_alpha α H.1 ρ_mix σ_mix) ≤
    ∑ i, w i * (⊔ (H : {H : HermitianMat d ℂ // 0 ≤ H}), f_alpha α H.1 (ps i) (os
  apply ciSup_le
  intro H
  have h_sum : f_alpha α H.1 ρ_mix σ_mix ≤ ∑ i, w i * (f_alpha α H.1 (ps i) (os i))
    apply f_alpha_jointly_convex hα H.1 H.2 w hw_nonneg hw_sum ps os ρ_mix σ_mix hp_
  exact h_sum.trans (Finset.sum_le_sum fun i _ => mul_le_mul_of_nonneg_left
    (le_ciSup (f_alpha_bddAbove hα (ps i) (os i) (hker i)) H) (hw_nonneg i))

/-- If for all i, ker(os i) ≤ ker(ps i), then ker(∑ w i • os i) ≤ ker(∑ w i • ps i),
provided all weights are nonneg and all matrices are PSD. -
/
theorem HermitianMat.ker_weighted_sum_le {ι : Type*} [Fintype ι]
  (w : ι → ℝ) (hw_nonneg : ∀ i, 0 ≤ w i)

```

```

(ps os : ι → HermitianMat d ℂ)
(hps_nonneg : ∀ i, 0 ≤ ps i)
(hos_nonneg : ∀ i, 0 ≤ os i)
(hker : ∀ i, (os i).ker ≤ (ps i).ker) :
(∑ i, w i • os i).ker ≤ (∑ i, w i • ps i).ker := by
rw [ker_sum, ker_sum]
· refine iInf_mono fun i ↦ ?_
  by_cases hi : w i = 0
  · simp [hi]
  · simp_all [ker_pos_smul]
· exact fun i => smul_nonneg (hw_nonneg i) (hps_nonneg i)
· exact fun i => smul_nonneg (hw_nonneg i) (hos_nonneg i)

/-- The trace functional `Q_α` is jointly convex for `α > 1`.
This is Proposition 3 of the paper, originally from Frank–Lieb.
The proof uses the variational formula:
`Q_α(ρ||σ) = sup_{H ≥ 0} f_α(H, ρ, σ)`
where `f_α(H, ρ, σ) = α · Tr[ρ H] - (α-1) · Tr[(σ^{-γ} H σ^{-γ})^{α/(α-1)}`
is jointly convex in `(ρ, σ)` for fixed `H` (since the first term is linear
the second uses the convexity of trace functions). The supremum of jointly
functions is jointly convex. -/
theorem sandwichedTraceFunctional_jointly_convex (hα : 1 < α) {ι : Type*} [
  (w : ι → ℝ) (hw_nonneg : ∀ i, 0 ≤ w i) (hw_sum : ∑ i, w i = 1)
  (ps os : ι → MState d) (ρ_mix σ_mix : MState d)
  (hp_mix : ρ_mix.M = ∑ i, w i • (ps i).M)
  (hσ_mix : σ_mix.M = ∑ i, w i • (os i).M)
  (hker : ∀ i, (os i).M.ker ≤ (ps i).M.ker) :
  Q_α(ρ_mix||σ_mix) ≤ ∑ i, w i * Q_α(ps i||os i) := by
have hker' : σ_mix.M.ker ≤ ρ_mix.M.ker := by
rw [hp_mix, hσ_mix]
exact ker_weighted_sum_le w hw_nonneg _ (fun i => (ps i).nonneg) (fun
rw [traceFunctional_eq_iSup_f_alpha hα ρ_mix σ_mix hker']
calc [] H : {H : HermitianMat d ℂ // 0 ≤ H}, f_alpha α H.1 ρ_mix σ_mix
≤ ∑ i, w i * ([] H : {H : HermitianMat d ℂ // 0 ≤ H}, f_alpha α H.1 (ρ
iSup_f_alpha_jointly_convex hα w hw_nonneg hw_sum ps os ρ_mix σ_mix
_ = ∑ i, w i * Q_α(ps i||os i) := by
congr 1; ext i
rw [traceFunctional_eq_iSup_f_alpha hα (ps i) (os i) (hker i)]

/-! ### Twirling Construction Helpers
We construct a twirling set using κ = Perm dB × (dB → Bool).
For each (σ, f), the unitary V(σ,f) is the product of a sign-
diagonal matrix
(with entries ±1 determined by f) and a permutation matrix.
The averaging property follows from:

```

1. Sign averaging: summing over f kills off-diagonal entries
 2. Permutation averaging: summing over σ uniformizes diagonal entries
 -/

/-
 A diagonal matrix with ± 1 entries (determined by a Bool function) is unitary.
 -/

```
private lemma signDiag_mem_unitaryGroup (f : dB → Bool) :
  Matrix.diagonal (fun i : dB => (if f i then (-1 : ℂ) else 1)) ∈ Matrix.unitaryGroup dB ℂ
  constructor
  · ext i j; by_cases hi : i = j <=> simp [hi]
    · split_ifs <=> simp [*, Matrix.one_apply]
    · rw [Matrix.diagonal_apply_ne _ (.symm hi)]
      simp
  · ext i j; by_cases hi : i = j <=> simp [hi]
    · split_ifs <=> simp [*, Matrix.one_apply]
    · rw [Matrix.diagonal_apply_ne _ (.symm hi)]
      simp
```

/-- The product of a ± 1 diagonal matrix and a permutation matrix is unitary. -
 /

```
private lemma twirlingU_mem_unitaryGroup (σ : Equiv.Perm dB) (f : dB → Bool) :
  Matrix.diagonal (fun i : dB => (if f i then (-1 : ℂ) else 1)) * σ.permMatrix ∈ Matrix.unitaryGroup dB ℂ :=
  mul_mem (signDiag_mem_unitaryGroup f) (σ.permMatrix_mem_unitaryGroup)
```

/-- The twirling unitary for a given permutation and sign function. -
 /

```
private def twirlingU (σ : Equiv.Perm dB) (f : dB → Bool) : Matrix.unitaryGroup dB ℂ :=
  (Matrix.diagonal (fun i : dB => (if f i then (-1 : ℂ) else 1)) * σ.permMatrix,
   twirlingU_mem_unitaryGroup σ f)
```

/-
 Entry of the conjugation by a twirling unitary:
 $(X.\text{conj}(\text{twirlingU } \sigma f))_{\{pq\}} = \text{sign}(f,p) * \text{sign}(f,q) * X_{\{\sigma p, \sigma q\}}.$
 -/

```
private lemma twirlingU_conj_entry (X : HermitianMat dB ℂ) (σ : Equiv.Perm dB) (f :
  (p q : dB) :
  (X.conj (twirlingU σ f : Matrix dB dB ℂ)) p q =
    (if f p then (-1 : ℂ) else 1) * (if f q then (-1 : ℂ) else 1) * X (σ p) (σ q)
  have h_conj_apply : ∀ u : Matrix.unitaryGroup dB ℂ, (conj u.val X).mat =
    u.val * X.mat * u.val.conjTranspose := by
    intro u
    simp_all only [conj_apply_mat]
  convert congr_fun (congr_fun (h_conj_apply (twirlingU σ f)) p) q using 1
  unfold twirlingU
```

```

simp [Matrix.mul_apply, Matrix.diagonal]
simp [Finset.sum_ite]
rw [Finset.sum_eq_single (σ q)]
· simp_all only [conj_apply_mat, implies_true, Equiv.symm_apply_apply, ↑r,
  split
  next h =>
    simp_all only [map_neg, map_one, mul_neg, mul_one, neg_neg]
    split
    next h_1 => simp_all only [neg_neg]
    next h_1 => simp_all only [Bool.not_eq_true]
  next h => simp_all only [Bool.not_eq_true, map_one, mul_one]
· aesop
· simp

/-
Summing the sign product over all Bool functions.
For p = q, each term is 1, giving 2^(card dB).
For p ≠ q, terms cancel in pairs (flip f at p).
-/
private lemma sum_sign_prod (p q : dB) :
  ∑ f : dB → Bool, ((if f p then (-1 : ℂ) else 1) * (if f q then (-1 : ℂ) else 1)) =
  if p = q then (2 ^ Fintype.card dB : ℕ) else 0 := by
  split_ifs with h
  · simp +contextual [h]
  simp +contextual
  -- By pairing each function with its p-flip, we can see that the sum of e
  have h_pair :
    ∑ f : dB → Bool, (if f q then -if f p then -1 else 1 else if f p then
  1 else 1 : ℂ) =
    ∑ f : dB → Bool, - (if f q then -if f p then -1 else 1 else if f p th
  1 else 1 : ℂ) := by
    apply Finset.sum_bij (fun f _ => Function.update f p (¬f p)) (by simp)
    · intro a1 _ a2 _ h; ext x; by_cases hx : x = p
      · replace h := congr_fun h x
        subst hx
        simp_all only [Finset.mem_univ, Bool.not_eq_true, Bool.decide_eq_fa
          Function.update_self, Bool.not_eq_eq_eq_not, Bool.not_not]
      · replace h := congr_fun h x
        simp_all only [Finset.mem_univ, Bool.not_eq_true, Bool.decide_eq_fa
          not_false_eq_true, Function.update_of_ne]
    · exact fun b _ => (Function.update b p (decide ¬b p = true), Finset.me
    · grind
  rw [Finset.sum_neg_distrib] at h_pair
  linear_combination h_pair / 2

```

```

/-
Summing  $X_{\{\sigma(p), \sigma(p)\}}$  over all permutations  $\sigma$ .
For each target  $k$ , exactly  $(\text{card } dB - 1)!$  permutations send  $p$  to  $k$ .
-/
private lemma sum_perm_diag_entry (X : HermitianMat dB  $\mathbb{C}$ ) (p : dB) :
   $\sum \sigma : \text{Equiv.Perm } dB, X (\sigma p) (\sigma p) =$ 
    ((Fintype.card dB - 1).factorial :  $\mathbb{C}$ ) *  $\sum k : dB, X k k :=$  by
  -- For each fixed  $k$ , the number of permutations  $\sigma$  with  $\sigma p = k$  is  $(\text{card } dB - 1)!$ 
  have h_card (k : dB) : (Finset.univ.filter (fun  $\sigma : \text{Equiv.Perm } dB \Rightarrow \sigma p = k$ )).card
    (Nat.factorial (Fintype.card dB - 1) :  $\mathbb{N}$ ) := by
    have h_fixed : Finset.card (Finset.filter (fun  $\sigma : \text{Equiv.Perm } dB \Rightarrow \sigma p = k$ ) Finset.univ)
      (Finset.univ : Finset (Equiv.Perm dB)) / Fintype.card dB := by
    have h_card :  $\forall k : dB, (\text{Finset.univ.filter (fun } \sigma : \text{Equiv.Perm } dB \Rightarrow \sigma p = k$ )).card
      (Finset.univ.filter (fun  $\sigma : \text{Equiv.Perm } dB \Rightarrow \sigma p = p$ )).card := by
      intro k
      apply Finset.card_bij (fun  $\sigma \_ \Rightarrow \text{Equiv.swap } p k * \sigma$ )
      · intro a ha
        simp_all only [Finset.mem_filter, Finset.mem_univ, true_and, Equiv.Perm.comp_apply, Equiv.swap_apply_right, and_self]
      · simp
      · simp
      exact fun b hb  $\Rightarrow$  (Equiv.swap p k * b, by simp [hb], by simp)
    have hc2 : (Finset.univ.filter (fun  $\sigma : \text{Equiv.Perm } dB \Rightarrow \sigma p = p$ )).card * Finset.card
      (Finset.univ : Finset (Equiv.Perm dB)) := by
    have hc :  $\sum k : dB, (\text{Finset.univ.filter (fun } \sigma : \text{Equiv.Perm } dB \Rightarrow \sigma p = k$ )).card
      (Finset.univ : Finset (Equiv.Perm dB)) := by
      simp only [Finset.card_eq_sum_ones, Finset.sum_fiberwise]
      simp_all [mul_comm]
      rw [← hc2, h_card k, Nat.mul_div_cancel _ (Fintype.card_pos_iff.mpr {p})]
      rcases n : Fintype.card dB with (_ | _ | n) <|> simp_all [Nat.factorial_succ, Finset.sum_card]
      exact absurd n (Nat.ne_of_gt (Fintype.card_pos_iff.mpr {p}))
  -- By Fubini's theorem, we can interchange the order of summation.
  have h_fubini :  $\sum \sigma : \text{Equiv.Perm } dB, X (\sigma p) (\sigma p) = \sum k : dB, \sum \sigma \in \text{Finset.univ.filter (fun } \sigma : \text{Equiv.Perm } dB \Rightarrow \sigma p = k$ ), X k k := by
    simp only [Finset.sum_filter]
    rw [Finset.sum_comm, Finset.sum_congr rfl]
    intro x a
    simp_all only [Finset.mem_univ, Finset.sum_ite_eq, reduceIte]
    simp_all [Finset.mul_sum]

/-
The sum formula for twirling: summing the conjugation entries over all  $(\sigma, f)$  pairs.
-/
private lemma twirling_sum_eq [Nonempty dB] (X : HermitianMat dB  $\mathbb{C}$ ) (p q : dB) :
   $\sum i : \text{Equiv.Perm } dB \times (dB \rightarrow \text{Bool}), (X.\text{conj } (\text{twirlingU } i.1 i.2 : \text{Matrix } dB dB \mathbb{C}))$ 
    if p = q then ((Fintype.card dB - 1).factorial *  $2^{\text{Fintype.card } dB}$  :  $\mathbb{N}$ ) *  $\sum$ 

```

```

    else 0 := by
      -- Rewrite the sum as a double sum over  $\sigma$  and  $f$  using Finset.sum_product'
      have h_double_sum :  $\sum i : \text{Equiv.Perm } \text{dB} \times (\text{dB} \rightarrow \text{Bool}),$ 
        ((conj (twirlingU i.1 i.2 : Matrix dB dB  $\mathbb{C}$ )) X) p q =
         $\sum \sigma : \text{Equiv.Perm } \text{dB}, \sum f : \text{dB} \rightarrow \text{Bool}, ((\text{if } f \text{ p then } (-$ 
1 :  $\mathbb{C}$ ) else 1) *
        (if f q then (-1 :  $\mathbb{C}$ ) else 1) * X ( $\sigma$  p) ( $\sigma$  q)) := by
      rw [← Finset.sum_product']
      refine Finset.sum_bij (fun i _ => (i.1, i.2)) ?_ ?_ ?_ ?_
      · simp
      · simp
      · simp
      · simp [twirlingU_conj_entry]
split_ifs with h
· simp_all [← Finset.mul_sum]
  have := sum_perm_diag_entry X q; simp_all [mul_assoc, mul_comm]
· rw [h_double_sum, Finset.sum_eq_zero]
  intro  $\sigma$  _
  rw [← Finset.sum_mul, sum_sign_prod p q]
  simp_all only [mul_ite, mul_neg, mul_one, ite_mul, neg_mul, one_mul, Fin
    CharP.cast_eq_zero, zero_mul]

/-
The identity for the twirling set, stated for  $\kappa = \text{Perm } \text{dB} \times (\text{dB} \rightarrow \text{Bool})$ .
-/
private lemma twirling_identity [Nonempty dB] (X : HermitianMat dB  $\mathbb{C}$ ) :
  (Fintype.card (Equiv.Perm dB  $\times$  (dB  $\rightarrow$  Bool))) :  $\mathbb{R}$ )-1 •
   $\sum i : \text{Equiv.Perm } \text{dB} \times (\text{dB} \rightarrow \text{Bool}), X.\text{conj } (\text{twirlingU } i.1 \ i.2 : \text{Matrix}$ 
    (X.trace / Fintype.card dB) • (1 : HermitianMat dB  $\mathbb{C}$ ) := by
  ext p q
  simp [Fintype.card_prod, Fintype.card_perm, Fintype.card_pi]
  ring_nf
  convert congr_arg ((2-1 ^ Fintype.card dB * (Fintype.card dB |> Nat.factor
    (twirling_sum_eq X p q) using 1
  · norm_num [Matrix.one_apply]
    convert Or.inl rfl
    induction (Finset.univ : Finset (Equiv.Perm dB  $\times$  (dB  $\rightarrow$  Bool))) using Fi
  · simp_all only [Finset.sum_empty, zero_apply]
  · rename_i a s a_1 a_2
    obtain ⟨fst, snd⟩ := a
    simp only [not_false_eq_true, Finset.sum_insert, *]
    rfl
  · norm_num [Matrix.one_apply]
    rw [show X.trace =  $\sum k, X \ k \ k$  from X.trace_eq_trace]
    rcases n : Fintype.card dB with (_ | n)
  · simp_all

```



```

    · simp_all [Nat.factorial_succ, mul_assoc, mul_comm, mul_left_comm]
      simp [Nat.factorial_ne_zero]

/-! ## Twirling Set

A twirling set for a finite-dimensional system `dB` is a set of unitary matrices
`{V_i}` on `dB` (indexed by some finite type `κ`) such that the average
`(1/|κ|) ∑_i V_i X V_i†` equals `Tr(X) · (1/dim(dB))` for all `X`.
When applied as `1_A ⊗ V_i` on a bipartite system `dA × dB`, this gives:
`(1/|κ|) ∑_i (1_A ⊗ V_i) ρ_AB (1_A ⊗ V_i)† = ρ_A ⊗ π_B`
where `π_B = 1/dim(dB)` is the maximally mixed state.

The standard construction uses the Heisenberg–Weyl (discrete Weyl) operators. -
/

/-- A twirling set for the system `dB` exists: there is a finite collection of unitaries
whose average conjugation action twirls any matrix to a multiple of the identity.
Specifically, `(1/|κ|) ∑_i V_i X V_i† = (Tr X / dim dB) · I` for all Hermitian `X` on `dB`.
The standard construction uses the discrete Heisenberg–Weyl group of order `|dB|^2`.
/
private lemma exists_twirling_unitaries [Nonempty dB] :
  ∃ (κ : Type) (α : Fintype κ) (α' : Nonempty κ) (V : κ → Matrix.unitaryGroup dB ℂ)
    ∀ (X : HermitianMat dB ℂ),
      (Fintype.card κ : ℝ)⁻¹ • ∑ i : κ, X.conj (V i : Matrix dB dB ℂ) =
        (X.trace / Fintype.card dB) • (1 : HermitianMat dB ℂ) := by
  use Shrink (Equiv.Perm dB × (dB → Bool)), inferInstance, inferInstance
  use fun i => twirlingU ((equivShrink _).symm i).1 ((equivShrink _).symm i).2
  intro X
  rw [Fintype.card_shrink]
  convert twirling_identity X using 2
  refine Finset.sum_bij (fun i _ => (equivShrink _).symm i) ?_ ?_ ?_ ?_
  · simp
  · simp
  · simp
  exact fun a b => ⟨_, Equiv.apply_symm_apply _ _⟩
  · simp

-- /-- Twirling on a bipartite system: applying `1_A ⊗ V_i` and averaging produces the
-- partial trace tensored with the maximally mixed state. -
/
-- theorem twirling_bipartite [Nonempty dB]
--   (κ : Type) [Fintype κ] (V : κ → Matrix.unitaryGroup dB ℂ)
--   (hV : ∀ (X : HermitianMat dB ℂ),
--     (Fintype.card κ : ℝ)⁻¹ • ∑ i : κ, X.conj (V i : Matrix dB dB ℂ) =
--       (X.trace / Fintype.card dB) • (1 : HermitianMat dB ℂ))

```

```

--      (A : HermitianMat (dA × dB) ℂ) :
--      (Fintype.card κ : ℝ)-1 • ∑ i : κ,
--      A.conj (Matrix.kroneckerMap (· * ·) (1 : Matrix dA dA ℂ) (V i : Ma
--      A.traceRight ⊗κ ((Fintype.card dB : ℝ)-1 • (1 : HermitianMat dB ℂ)
--      not needed...

/-! ## Tensor Invariance

`Q̃_α(ρ ⊗ τ || σ ⊗ τ) = Q̃_α(ρ || σ)` for any state `τ`.
This corresponds to equation (2.4) in the paper. -/

/-
The trace functional is multiplicative over tensor products:
`Q̃_α(ρ1 ⊗ ρ2 || σ1 ⊗ σ2) = Q̃_α(ρ1||σ1) · Q̃_α(ρ2||σ2)`.
-/
theorem sandwichedTraceFunctional_mul
  (ρ1 σ1 : MState dA) (ρ2 σ2 : MState dB) :
  Q̃_α(ρ1 ⊗ ρ2 || σ1 ⊗ σ2) = Q̃_α(ρ1||σ1) * Q̃_α(ρ2||σ2) := by
  exact sandwiched_term_product ρ1 σ1 ρ2 σ2 α ((1 - α) / (2 * α))

/-
The trace functional of a state with itself equals 1.
This follows from the calculation: `γ = (1-α)/(2α)` gives `2γ + 1 = 1/α`,
so `σγ · σ · σγ = σ(2γ+1) = σ(1/α)`, and `(σ(1/α))α = σ1`,
whose trace equals 1 since σ is a state.
-/
theorem sandwichedTraceFunctional_self (hα : 0 < α) (ρ : MState d) :
  Q̃_α(ρ||ρ) = 1 := by
  by_cases h : α = 1
  · subst h; simp [sandwichedTraceFunctional]
  · unfold sandwichedTraceFunctional
    have := ρ.pos
    have h_simp : (ρ.M.conj (ρ.M ^ ((1 - α) / (2 * α))).mat) =
      ρ.M ^ (1 + 2 * ((1 - α) / (2 * α))) := by
      rw [← conj_rpow]
      · rw [rpow_one]
      · exact le_of_lt this
      · exact div_ne_zero (sub_ne_zero_of_ne (Ne.symm h)) (mul_ne_zero two_
      · nlinarith [mul_div_cancel₀ (1 - α) (by positivity : (2 * α) ≠ 0)]
    have h_simp : (ρ.M ^ (1 + 2 * ((1 - α) / (2 * α)))) ^ α =
      ρ.M ^ ((1 + 2 * ((1 - α) / (2 * α))) * α) := by
      rw [← rpow_mul]
      exact le_of_lt this
    field_simp at *
    simp_all only [add_sub_cancel, one_div, rpow_one, MState.tr]

```

```

/-- The trace functional is invariant under tensoring with a fixed state.
This follows from multiplicativity (`sandwichedTraceFunctional_mul`) and
the self-trace-functional being 1 (`sandwichedTraceFunctional_self`). -
/
theorem sandwichedTraceFunctional_tensor_invariant (hα : 0 < α)
  (ρ σ : MState dA) (τ : MState dB) :
   $\tilde{Q}_\alpha(\rho \otimes^M \tau \| \sigma \otimes^M \tau) = \tilde{Q}_\alpha(\rho \| \sigma)$  := by
  rw [sandwichedTraceFunctional_mul, sandwichedTraceFunctional_self hα, mul_one]

/-! ## Twirling MState Helpers

Helper lemmas for constructing MStates via the twirling argument. -
/

/-- The MState obtained by conjugating a bipartite state by  $1_A \otimes V$  where  $V$  is a
unitary on the  $B$  system. This is  $(1_A \otimes V) \rho_{AB} (1_A \otimes V)^\dagger$ . -
/
def MState.conjTensorUnitary (ρ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ) :
  MState (dA × dB) :=
  ρ.U_conj ((1 : Matrix.unitaryGroup dA ℂ) ⊗u V)

/-- The twirled MState: averaging conjugation by  $1_A \otimes V_i$  over all elements of
the twirling set gives  $\rho_A \otimes \text{uniform}_B$ . We state the HermitianMat-
level
equality needed for the joint convexity argument. -/
theorem MState.conjTensorUnitary_M (ρ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ) :
  (ρ.conjTensorUnitary V).M = ρ.M.conj ((1 : Matrix.unitaryGroup dA ℂ) ⊗u V).val :
  rfl

/-- The trace functional is invariant under  $1_A \otimes V$  conjugation. -
/
theorem sandwichedTraceFunctional_conj_tensorUnitary
  (ρ σ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ) :
   $\tilde{Q}_\alpha(\rho.\text{conjTensorUnitary } V \| \sigma.\text{conjTensorUnitary } V) = \tilde{Q}_\alpha(\rho \| \sigma)$  := by
  exact sandwichedTraceFunctional_conj_unitary_MState _ ρ σ

section twirling

variable {dA dB : Type*}
variable [Fintype dA] [Fintype dB]
variable [DecidableEq dA] [DecidableEq dB]
open scoped InnerProductSpace RealInnerProductSpace HermitianMat Matrix

omit [DecidableEq dB] in
-- The  $((a_1, b_1), (a_2, b_2))$  entry of  $(1 \otimes V) M (1 \otimes V)^\dagger$ 
-- equals  $(V * \text{block}_{\{a_1, a_2\}} * V^\dagger)_{\{b_1, b_2\}}$ .

```

```

lemma conj_kron_one_entry (M : Matrix (dA × dB) (dA × dB) ℂ)
  (V : Matrix dB dB ℂ) (a₁ a₂ : dA) (b₁ b₂ : dB) :
  (Matrix.kroneckerMap (· * ·) (1 : Matrix dA dA ℂ) V * M *
    (Matrix.kroneckerMap (· * ·) (1 : Matrix dA dA ℂ) V).conjTranspose) (a
    (V * (Matrix.of fun b₁' b₂' => M (a₁, b₁') (a₂, b₂'))) * V.conjTranspose
  norm_num [Matrix.mul_apply, Matrix.adjugate_apply, Matrix.det_apply', Mat
  simp [Matrix.one_apply, Finset.sum_ite]
  apply Finset.sum_bij (fun x _ => x.2)
  · simp
  · simp
  · simp
  simp
  intro a b rfl
  left
  apply Finset.sum_bij (fun x _ => x.2)
  · simp
  · simp
  · simp
  simp +contextual

```

/-

For a Hermitian matrix, the twirling identity at the entry level.
 Extracts from hV the entry-level equation.

-/

```

lemma twirling_hermitian_entry
  (κ : Type) [Fintype κ] (V : κ → Matrix.unitaryGroup dB ℂ)
  (hV : ∀ (X : HermitianMat dB ℂ),
    (Fintype.card κ : ℝ)-1 • ∑ i : κ, X.conj (V i : Matrix dB dB ℂ) =
      (X.trace / Fintype.card dB) • (1 : HermitianMat dB ℂ))
  (X : HermitianMat dB ℂ) (b₁ b₂ : dB) :
  ∑ i : κ, ((V i).val * X.val * (V i).val.conjTranspose) b₁ b₂ =
    (X.val.trace / (Fintype.card dB : ℂ)) * (Fintype.card κ : ℂ) *
      (if b₁ = b₂ then 1 else 0) := by
  replace hV := congr_arg (fun s => s.val b₁ b₂) (hV X); simp_all [div_eq_i
  convert congr_arg (fun x : ℂ => x * Fintype.card κ) hV using 1 <=> ring_n
  · by_cases h : Fintype.card κ = 0 <=> simp_all [conj]
  · rw [Fintype.card_eq_zero_iff] at h
    simp_all only [Finset.univ_eq_empty, Finset.sum_empty]
  · classical induction (Finset.univ : Finset κ) using Finset.induction
    · simp_all [Matrix.mul_assoc]
    · simp_all [Matrix.mul_assoc]
    rfl
  · simp [Matrix.one_apply, mul_assoc, mul_comm]
  simp [Matrix.trace, trace]
  congr! 2
  exact Finset.sum_congr rfl fun _ _ => by simp [Complex.ext_iff]

```

```

/-
Extension of the twirling property from HermitianMat to general matrices.
-/
lemma twirling_general_matrix
  (κ : Type) [Fintype κ] (V : κ → Matrix.unitaryGroup dB ℂ)
  (hV : ∀ (X : HermitianMat dB ℂ),
    (Fintype.card κ : ℝ)-1 • ∑ i : κ, X.conj (V i : Matrix dB dB ℂ) =
      (X.trace / Fintype.card dB) • (1 : HermitianMat dB ℂ))
  (X : Matrix dB dB ℂ) (b₁ b₂ : dB) :
  ∑ i : κ, ((V i).val * X * (V i).val.conjTranspose) b₁ b₂ =
  (X.trace / (Fintype.card dB : ℂ)) * (Fintype.card κ : ℂ) *
    (if b₁ = b₂ then 1 else 0) := by
  -- Decompose X into Hermitian and anti-Hermitian parts.
  set X_herm : Matrix dB dB ℂ := (1 / 2 : ℂ) • (X + X.conjTranspose)
  set X_anti_herm : Matrix dB dB ℂ := (1 / (2 * Complex.I) : ℂ) • (X - X.conjTranspose)
  have h_decomp : X = X_herm + Complex.I • X_anti_herm := by
    ext i j; norm_num [X_herm, X_anti_herm]; ring_nf
    norm_num; ring
  -- Apply the twirling property to X_herm and X_anti_herm.
  have h_tw_h : ∑ i : κ, ((V i).val * X_herm * (V i).val.conjTranspose) b₁ b₂ =
    (X_herm.trace / (Fintype.card dB)) * (Fintype.card κ) * (if b₁ = b₂ then 1 else 0) :=
    convert twirling_hermitian_entry κ V hV ⟨X_herm, _⟩ b₁ b₂ using 1
    simp +zetaDelta at *
    ext i j; simp [Matrix.conjTranspose_apply]; ring
  have h_tw_a : ∑ i : κ, ((V i).val * X_anti_herm * (V i).val.conjTranspose) b₁ b₂ =
    (X_anti_herm.trace / (Fintype.card dB)) * (Fintype.card κ) * (if b₁ = b₂ then 1 else 0) :=
    convert twirling_hermitian_entry κ V hV ⟨X_anti_herm, ?_⟩ b₁ b₂ using 1
    ext i j; simp [X_anti_herm, Matrix.conjTranspose_apply]; ring
  rw [h_decomp]
  convert congr_arg₂ (· + ·) h_tw_h (congr_arg (fun x : ℂ => Complex.I * x) h_tw_a)
  <=> simp [mul_add, add_mul, mul_assoc, Finset.mul_sum, Finset.sum_add_distrib]
  ring_nf
  split_ifs <=> ring

/-- The MState obtained by conjugating a bipartite state by `1_A ⊗ V`. -
/
def MState.conjTensorUnitary' (ρ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ)
  MState (dA × dB) :=
  ρ.U_conj ((1 : Matrix.unitaryGroup dA ℂ) ⊗u V)

-- Entry-level form of the conjTensorUnitary.
lemma conjTensorUnitary'_entry (ρ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ)
  (a₁ a₂ : dA) (b₁ b₂ : dB) :
  (ρ.conjTensorUnitary' V).M.val (a₁, b₁) (a₂, b₂) =
  ((V : Matrix dB dB ℂ) * (Matrix.of fun b₁' b₂' => ρ.M.val (a₁, b₁') (a₂, b₂')) *
    (V : Matrix dB dB ℂ).conjTranspose) b₁ b₂ := by

```

```

apply conj_kron_one_entry

-- The RHS entry: (ρ.traceRight ⊗M uniform).M at ((a1,b1),(a2,b2)).
lemma prod_traceRight_uniform_entry [Nonempty dB] (ρ : MState (dA × dB))
  (a1 a2 : dA) (b1 b2 : dB) :
  (ρ.traceRight ⊗M MState.uniform).M.val (a1, b1) (a2, b2) =
  ρ.M.val.traceRight a1 a2 * ((Fintype.card dB : ℂ)-1 * if b1 = b2 then 1
  unfold MState.traceRight MState.uniform
  unfold MState.ofClassical
  unfold diagonal
  unfold MState.prod
  unfold kronecker
  simp [Matrix.kroneckerMap_apply]
  rw [Matrix.diagonal_apply]
  simp only [mul_ite, mul_zero]

theorem twirling_average_eq [Nonempty dB]
  (κ : Type) [Fintype κ] (V : κ → Matrix.unitaryGroup dB ℂ)
  (hV : ∀ (X : HermitianMat dB ℂ),
    (Fintype.card κ : ℝ)-1 • ∑ i : κ, X.conj (V i : Matrix dB dB ℂ) =
    (X.trace / Fintype.card dB) • (1 : HermitianMat dB ℂ))
  (ρ : MState (dA × dB)) :
  ∑ i : κ, ((Fintype.card κ : ℝ)-1 • (ρ.conjTensorUnitary' (V i)).M) =
  (ρ.traceRight ⊗M MState.uniform).M := by
  -- Apply the twirling hypothesis to each term in the sum.
  have h_sum : ∀ a1 a2 : dA, ∀ b1 b2 : dB, (∑ i, (1 / (Fintype.card κ : ℂ))
    (ρ.conjTensorUnitary' (V i)).M.val (a1, b1) (a2, b2)) =
    (ρ.M.val.traceRight a1 a2) * (1 / (Fintype.card dB : ℂ)) * (if b1 = b2
  intro a1 a2 b1 b2
  have h_sum : ∑ i : κ, ((V i : Matrix dB dB ℂ) * (Matrix.of fun b1' b2'
    ρ.M.val (a1, b1') (a2, b2')) * (V i : Matrix dB dB ℂ).conjTranspose
    (ρ.M.val.traceRight a1 a2) * (Fintype.card κ : ℂ) * (1 / (Fintype.c
    (if b1 = b2 then 1 else 0) := by
  convert twirling_general_matrix κ V hV
  (Matrix.of fun b1' b2' => ρ.M.val (a1, b1') (a2, b2')) b1 b2 using
  simp [Matrix.trace]
  ring_nf!
  convert congr_arg (fun x : ℂ => (1 / (Fintype.card κ : ℂ)) * x) h_sum u
  · norm_num [conjTensorUnitary'_entry]
  ring_nf
  rw [Finset.mul_sum]
  · norm_num [conjTensorUnitary'_entry]
  by_cases h : Fintype.card κ = 0 <=> simp_all [mul_assoc, mul_comm, mu
  specialize hV 1; norm_num at hV
  convert h_sum using 1
  constructor <=> intro h

```

```

· exact h_sum
· ext {a₁, b₁} {a₂, b₂}
  convert h a₁ a₂ b₁ b₂ using 1
  · classical induction (Finset.univ : Finset κ) using Finset.induction
  · simp_all
  · simp_all
    convert congr_arg₂ (· + ·) rfl <_> using 1
    simp [Algebra.smul_def]
  · convert prod_traceRight_uniform_entry ρ a₁ a₂ b₁ b₂ using 1
    ring

```

```
end twirling
```

```
/-! ## Monotonicity Under Partial Trace (α > 1)
```

The main intermediate result: for $\alpha > 1$, the trace functional $\tilde{Q}_\alpha$ is monotone under partial trace:

$\tilde{Q}_\alpha(\rho_{AB} \parallel \sigma_{AB}) \geq \tilde{Q}_\alpha(\rho_A \parallel \sigma_A)$.

The proof uses the twirling argument:

1. By unitary invariance, $\tilde{Q}_\alpha(\rho_{AB} \parallel \sigma_{AB}) = \tilde{Q}_\alpha(V_i \rho_{AB} V_i^\dagger \parallel V_i \sigma_{AB} V_i^\dagger)$ for e
2. Averaging: $\tilde{Q}_\alpha(\rho_{AB} \parallel \sigma_{AB}) = (1/|\kappa|) \sum_i \tilde{Q}_\alpha(V_i \rho_{AB} V_i^\dagger \parallel V_i \sigma_{AB} V_i^\dagger)$.
3. By joint convexity ($\alpha > 1$): $\geq \tilde{Q}_\alpha((1/|\kappa|) \sum_i V_i \rho_{AB} V_i^\dagger \parallel (1/|\kappa|) \sum_i V_i \sigma_{AB} V_i^\dagger)$.
4. By twirling: $= \tilde{Q}_\alpha(\rho_A \otimes \pi_B \parallel \sigma_A \otimes \pi_B)$.
5. By tensor invariance: $= \tilde{Q}_\alpha(\rho_A \parallel \sigma_A)$. -/

/-- If $\sigma.M.\text{ker} \leq \rho.M.\text{ker}$, then $(\sigma.\text{conj } B).\text{ker} \leq (\rho.\text{conj } B).\text{ker}$ for any matrix B . This follows from `ker_conj` (which expresses $(A.\text{conj } B).\text{ker}$ as a `comap`) and `Submodule.comap_mono`. -/

```

lemma ker_conj_le_of_ker_le {n : Type*} [Fintype n] [DecidableEq n]
  {A B : HermitianMat n ℂ} (hA : 0 ≤ A) (hB : 0 ≤ B) (h : A.ker ≤ B.ker)
  (C : Matrix n n ℂ) : (A.conj C).ker ≤ (B.conj C).ker := by
  rw [ker_conj hA, ker_conj hB]
  exact Submodule.comap_mono h

```

/-- Unitary conjugation preserves the kernel ordering between MStates.

If $\sigma.M.\text{ker} \leq \rho.M.\text{ker}$, then $(\sigma.\text{conjTensorUnitary } V).M.\text{ker} \leq (\rho.\text{conjTensorUnitary } V).M.\text{ker}$.

```

lemma MState.ker_conjTensorUnitary_le {dA dB : Type*} [Fintype dA] [Fintype dB]
  [DecidableEq dA] [DecidableEq dB]
  (ρ σ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ)
  (hker : σ.M.ker ≤ ρ.M.ker) :
  (σ.conjTensorUnitary V).M.ker ≤ (ρ.conjTensorUnitary V).M.ker := by
  simp only [MState.conjTensorUnitary_M]
  exact ker_conj_le_of_ker_le σ.nonneg ρ.nonneg hker _

```

```

/-- Monotonicity of the trace functional under partial trace for  $\alpha > 1$ .
Equation (2.8) of the paper (second line). -/
theorem sandwichedTraceFunctional_mono_traceRight [Nonempty dB]
  (hα : 1 < α) (ρ σ : MState (dA × dB)) (hker : σ.M.ker ≤ ρ.M.ker) :
     $\tilde{Q}_\alpha(\rho.\text{traceRight} \parallel \sigma.\text{traceRight}) \leq \tilde{Q}_\alpha(\rho \parallel \sigma)$  := by
  -- Obtain the twirling unitaries
  obtain (κ, hκ_fin, hκ_ne, V, hV) := exists_twirling_unitaries (dB := dB)
  letI : Fintype κ := hκ_fin
  letI : Nonempty κ := hκ_ne
  -- By unitary invariance,  $\tilde{Q}_\alpha(\rho \parallel \sigma) = \tilde{Q}_\alpha(V_i \rho V_i^\dagger \parallel V_i \sigma V_i^\dagger)$  for each
  have h_inv (i) :  $\tilde{Q}_\alpha(\rho.\text{conjTensorUnitary } (V \ i) \parallel \sigma.\text{conjTensorUnitary } (V \ i)) =$ 
    sandwichedTraceFunctional_conj_tensorUnitary ρ σ (V i)
  -- Step 2:  $\tilde{Q}_\alpha(\rho \parallel \sigma) = \sum_i (1/|\kappa|) * \tilde{Q}_\alpha(V_i \rho V_i^\dagger \parallel V_i \sigma V_i^\dagger)$ 
  have hcard_ne : (Fintype.card κ : ℝ) ≠ 0 :=
    Nat.cast_ne_zero.mpr Fintype.card_ne_zero
  have h_avg :  $\tilde{Q}_\alpha(\rho \parallel \sigma) = \sum i : \kappa, (Fintype.card \kappa : \mathbb{R})^{-1} * \tilde{Q}_\alpha(\rho.\text{conjTensorUnitary } (V \ i) \parallel \sigma.\text{conjTensorUnitary } (V \ i))$  := by
    simp only [h_inv, Finset.sum_const, Finset.card_univ, nsmul_eq_mul]
    field_simp
  -- Step 3: By joint convexity ( $\alpha > 1$ )
  have hw_sum :  $\sum i : \kappa, (Fintype.card \kappa : \mathbb{R})^{-1} = 1$  := by
    rw [Finset.sum_const, Finset.card_univ, nsmul_eq_mul]
    exact mul_inv_cancel₀ hcard_ne
  set ρ_mix := ρ.traceRight ⊗M MState.uniform (d := dB)
  set σ_mix := σ.traceRight ⊗M MState.uniform (d := dB)
  have hp_mix : ρ_mix.M =  $\sum i : \kappa, (Fintype.card \kappa : \mathbb{R})^{-1} \cdot (\rho.\text{conjTensorUnitary } (V \ i) \parallel \sigma.\text{conjTensorUnitary } (V \ i))$  := by
    (twirling_average_eq κ V hV ρ).symm
  have hσ_mix : σ_mix.M =  $\sum i : \kappa, (Fintype.card \kappa : \mathbb{R})^{-1} \cdot (\sigma.\text{conjTensorUnitary } (V \ i) \parallel \rho.\text{conjTensorUnitary } (V \ i))$  := by
    (twirling_average_eq κ V hV σ).symm
  have h_convex := sandwichedTraceFunctional_jointly_convex hα
    (fun _ : κ => (Fintype.card κ : ℝ)-1) (by intro; positivity) hw_sum
    (fun i => ρ.conjTensorUnitary (V i)) (fun i => σ.conjTensorUnitary (V i))
    ρ_mix σ_mix hp_mix hσ_mix
    (fun i => MState.ker_conjTensorUnitary_le ρ σ (V i) hker)
  -- Step 4 + 5:  $\tilde{Q}_\alpha(\rho_A \otimes \pi_B \parallel \sigma_A \otimes \pi_B) = \tilde{Q}_\alpha(\rho_A \parallel \sigma_A)$  by tensor invariance
  have h_tensor :  $\tilde{Q}_\alpha(\rho_{\text{mix}} \parallel \sigma_{\text{mix}}) = \tilde{Q}_\alpha(\rho.\text{traceRight} \parallel \sigma.\text{traceRight})$  :=
    sandwichedTraceFunctional_tensor_invariant (by linarith) ρ.traceRight σ.traceRight
  -- Combine
  calc  $\tilde{Q}_\alpha(\rho.\text{traceRight} \parallel \sigma.\text{traceRight})$ 
    =  $\tilde{Q}_\alpha(\rho_{\text{mix}} \parallel \sigma_{\text{mix}})$  := h_tensor.symm
    ≤  $\sum i : \kappa, (Fintype.card \kappa : \mathbb{R})^{-1} * \tilde{Q}_\alpha(\rho.\text{conjTensorUnitary } (V \ i) \parallel \sigma.\text{conjTensorUnitary } (V \ i))$  := h_convex
    =  $\tilde{Q}_\alpha(\rho \parallel \sigma)$  := h_avg.symm

```

/-! ## DPI for Sandwiched Rényi Divergence Under Partial Trace -
 /


```

/-- The "tensor product" of a vector v with basis vector e_b:
    (v ⊗ e_b)(a, b') = v(a) if b' = b, else 0 -/
private def vecTensorBasis (v : dA → ℂ) (b : dB) : (dA × dB) → ℂ :=
  fun ⟨a, b'⟩ => if b' = b then v a else 0

omit [DecidableEq dA] in
/-- Key identity: ⟨v, (Tr_B A)v⟩ = ∑_b ⟨v⊗e_b, A(v⊗e_b)⟩ -
/
private lemma inner_traceRight_eq_sum_inner_vecTensorBasis
  (A : Matrix (dA × dB) (dA × dB) ℂ) (v : dA → ℂ) :
  star v ∑_v A.traceRight *v v =
  ∑ b : dB, star (vecTensorBasis v b) ∑_v A *v (vecTensorBasis v b) := by
simp [Matrix.traceRight, Matrix.mulVec, dotProduct]
simp [vecTensorBasis, Fintype.sum_prod_type]
rw [Finset.sum_comm, Finset.sum_congr rfl]
simp [Finset.mul_sum _ _ _, mul_assoc, mul_comm]
intro x
rw [Finset.sum_comm]
congr
ext y
rw [Finset.sum_comm, Finset.sum_comm, Finset.sum_eq_single y]
· simp
· simp +contextual
· simp

omit [DecidableEq dA] in
/-- If A.mulVec(v⊗e_b) = 0 for all b, then (Tr_B A) *v v = 0 -
/
private lemma traceRight_mulVec_zero_of_vecTensorBasis_zero
  (A : Matrix (dA × dB) (dA × dB) ℂ) (v : dA → ℂ)
  (h : ∀ b : dB, A *v (vecTensorBasis v b) = 0) :
  A.traceRight *v v = 0 := by
ext i
simp_all [funext_iff, Matrix.mulVec, dotProduct]
convert Finset.sum_congr rfl fun j _ => h j i j using 1
any_goals exact Finset.univ
· unfold Matrix.traceRight vecTensorBasis; simp [Finset.sum_ite]
  simp [Finset.sum_mul, Finset.sum_sigma']
  apply Finset.sum_bij (fun x _ => ⟨x.2, x.1, x.2⟩)
  · simp
  · rintro ⟨fst, snd⟩ ha₁ ⟨fst_1, snd_1⟩ ha₂ ⟨rfl, ⟨rfl, right⟩⟩
    rfl
  · intro ⟨fst, ⟨fst_1, snd⟩⟩ a
    simp_all only [Finset.mem_sigma, Finset.mem_univ, Finset.mem_filter, true_and,
      heq_eq_eq, Prod.mk.injEq, exists_const, Sigma.exists, exists_eq_left, and_tr

```

```

      · simp
      · norm_num

/-- The kernel condition  $\sigma.M.\text{ker} \leq \rho.M.\text{ker}$  is preserved under partial tra
This follows because  $\text{supp}(\rho) \subseteq \text{supp}(\sigma)$  implies  $\text{supp}(\text{Tr}_B \rho) \subseteq \text{supp}(\text{Tr}_B \sigma)$ 
if  $v \in \text{supp}(\text{Tr}_B \rho)$ , then  $\langle v, (\text{Tr}_B \rho) v \rangle > 0$ , so for some basis vector  $e_b$ 
we have  $v \otimes e_b \in \text{supp}(\rho) \subseteq \text{supp}(\sigma)$ , hence  $\langle v, (\text{Tr}_B \sigma) v \rangle \geq \langle v \otimes e_b, \sigma (v \otimes e_b) \rangle$ 
/
theorem ker_le_traceRight { $\rho \ \sigma : \text{MState } (dA \times dB)$ }
  (hker :  $\sigma.M.\text{ker} \leq \rho.M.\text{ker}$ ) :
   $\sigma.\text{traceRight}.M.\text{ker} \leq \rho.\text{traceRight}.M.\text{ker} := \text{by}$ 
  simp only [MState.traceRight_M]
  intro v hv
  rw [mem_ker_iff_mulVec_zero] at hv
  have hv' :  $\sigma.M.\text{mat}.\text{traceRight} * v.v.\text{ofLp} = 0 := \text{by}$ 
    rwa [traceRight_mat] at hv
  have hin :  $\text{star } v.\text{ofLp} \sqsubseteq_v \sigma.M.\text{mat}.\text{traceRight} * v.v.\text{ofLp} = 0 := \text{by}$ 
    rw [hv']; simp [dotProduct]
  rw [inner_traceRight_eq_sum_inner_vecTensorBasis] at hin
  have h $\sigma$ _psd :  $\text{zero\_le\_iff.mp } \sigma.\text{nonneg}$ 
  have h_each_zero :  $\forall b : dB,$ 
     $\text{star } (\text{vecTensorBasis } v.\text{ofLp } b) \sqsubseteq_v \sigma.M.\text{mat} * v.v.\text{ofLp } b$ 
  have h_nonneg :  $\forall b, (0 : \mathbb{C}) \leq$ 
     $\text{star } (\text{vecTensorBasis } v.\text{ofLp } b) \sqsubseteq_v \sigma.M.\text{mat} * v.v.\text{ofLp } b$ 
  fun b => h $\sigma$ _psd.dotProduct_mulVec_nonneg _
  intro b
  exact Finset.sum_eq_zero_iff_of_nonneg (fun b _ => h_nonneg b) |>.mp hin
  have h $\sigma$ _zero :  $\forall b : dB, \sigma.M.\text{mat} * v.v.\text{ofLp } b = 0 :=$ 
    fun b => (h $\sigma$ _psd.dotProduct_mulVec_zero_iff _).mp (h_each_zero b)
  have h $\rho$ _zero :  $\forall b : dB, \rho.M.\text{mat} * v.v.\text{ofLp } b = 0 := \text{by}$ 
    intro b
    have hmem $\sigma$  : (WithLp.toLp 2 (vecTensorBasis v.ofLp b) : EuclideanSpace)
      rw [mem_ker_iff_mulVec_zero]; exact h $\sigma$ _zero b
    have hmem $\rho$  := hker hmem $\sigma$ 
    rwa [mem_ker_iff_mulVec_zero] at hmem $\rho$ 
    exact traceRight_mulVec_zero_of_vecTensorBasis_zero  $\rho.M.\text{mat} v.\text{ofLp } h_{\rho\_zero}$ 

/-- The sandwiched Rényi divergence is monotone under partial trace for  $\alpha$ 
This follows from monotonicity of the trace functional together with the fa
 $\tilde{D}_\alpha = \log(\tilde{Q}_\alpha) / (\alpha - 1)$  and both  $\log$  and  $1/(\alpha-1)$  are order-
preserving for  $\alpha > 1$ . -/
theorem sandwichedRenyiEntropy_mono_traceRight [Nonempty dB]
  (h $\alpha$  :  $1 < \alpha$ ) ( $\rho \ \sigma : \text{MState } (dA \times dB)$ )
  (hker :  $\sigma.M.\text{ker} \leq \rho.M.\text{ker}$ ) :
   $\tilde{D}_\alpha(\rho.\text{traceRight} || \sigma.\text{traceRight}) \leq \tilde{D}_\alpha(\rho || \sigma) := \text{by}$ 
  have h $\alpha_0$  :  $0 < \alpha := \text{by linarith}$ 

```

```

have hα₁ : α ≠ 1 := hα.ne'
have hker_tr := ker_le_traceRight hker
-- Rewrite both sides as log(Q) / (α - 1)
rw [sandwichedRelEntropy_eq_log_traceFunctional hα₀ hα₁ hker,
    sandwichedRelEntropy_eq_log_traceFunctional hα₀ hα₁ hker_tr]
apply ENNReal.ofReal_le_ofReal
apply div_le_div_of_nonneg_right _ (by linarith : 0 < α - 1).le
exact Real.log_le_log (sandwichedTraceFunctional_pos ρ.traceRight σ.traceRight hker)
    (sandwichedTraceFunctional_mono_traceRight hα ρ σ hker)

/-! ## DPI via Stinespring Dilation -/

/-
The sandwiched Rényi divergence is invariant under unitary conjugation.
-/
set_option maxHeartbeats 400000 in
theorem sandwichedRenyiEntropy_conj_unitary (hα : 0 < α) (ρ σ : MState d)
  (U : Matrix.unitaryGroup d ℂ) :
   $\tilde{D}_\alpha(\rho.U\_conj\ U || \sigma.U\_conj\ U) = \tilde{D}_\alpha(\rho || \sigma)$  := by
  -- Since unitary conjugation preserves the kernel, the condition  $\sigma.M.ker \leq \rho.M.ker$ 
  -- equivalent to  $(\sigma.U\_conj\ U).M.ker \leq (\rho.U\_conj\ U).M.ker$ .
  have h_kernel :  $\sigma.M.ker \leq \rho.M.ker \leftrightarrow (\sigma.U\_conj\ U).M.ker \leq (\rho.U\_conj\ U).M.ker$  := by
    have hk (A : HermitianMat d ℂ) : (A.conj U.val).ker = A.ker.map (U.val.toEuclideanLin)
    ext x
    simp [conj]
    constructor <=> intro hx
    all_goals generalize_proofs at *
    · use (U.val.conjTranspose.toEuclideanLin x)
      simp_all [ker, Matrix.toEuclideanLin]
      simp_all [lin, Matrix.toLpLin]
      have h_unitary : (U.val * U.val.conjTranspose) = 1 := by
        exact U.2.2
      generalize_proofs at *; (
        apply_fun (U.val.conjTranspose *v ·) at hx
        simp_all [Matrix.mul_assoc, Matrix.mulVec_mulVec]
        simp_all [← Matrix.mul_assoc, mul_eq_one_comm.mp h_unitary])
    · obtain ⟨y, hy, rfl⟩ := hx; simp_all [Matrix.toEuclideanLin, Matrix.mul_assoc]
      simp_all [ker, Matrix.toLpLin]
      simp_all [lin]
      simp_all [Matrix.toLpLin, Matrix.mulVec, funext_iff]
      simp_all [Matrix.mul_apply, dotProduct]
      -- Since U is unitary, we have  $\sum_{x_3} \langle U_{x_3\ x}, U_{x_3\ x_1} \rangle = \delta_{x\ x_1}$ .
      have h_unitary :  $\forall x\ x_1, \sum x_3, (\text{starRingEnd } \mathbb{C}) (U.val\ x_3\ x) * U.val\ x_3\ x_1 = \delta_{x\ x_1}$ 
        if  $x = x_1$  then 1 else 0 := by
        intro x x_1
        have this := congr_fun (congr_fun U.2.1 x) x_1

```

```

      simp_all [Matrix.mul_apply, Matrix.one_apply]
      simp_all [mul_assoc, Finset.sum_mul]
      intro x; rw [Finset.sum_comm]; simp_all [← Finset.mul_sum]
    simp [hk, MState.U_conj]
  constructor <=> intro h <=> simp_all [SetLike.le_def]
  · exact fun x hx => ⟨x, h hx, rfl⟩
  · intro x hx
    obtain ⟨y, hy, hy'⟩ := h x hx
    obtain {} : y = x := by
      apply_fun (U.val-1).mulVec at hy'
      simp_all [Matrix.mulVec_mulVec]
      exact PiLp.ext (congrFun hy')
    exact hy
  by_cases h : σ.M.ker ≤ ρ.M.ker <=> simp_all [SandwichedRelEntropy]
split_ifs <=> simp_all [MState.U_conj]
· congr 1
  rw [inner_sub_right, inner_sub_right]
  grind only [log_conj_unitary, inner_conj_unitary]
· ext1
  dsimp
  congr! 1
  convert congr_arg Real.log (sandwichedTraceFunctional_conj_unitary_MSta
/-
The sandwiched Rényi divergence is invariant under tensoring with a fixed p
`D̃_α(ρ ⊗ |ψ⟩⟨ψ| || σ ⊗ |ψ⟩⟨ψ|) = D̃_α(ρ || σ)` .
-/
theorem sandwichedRenyiEntropy_tensor_pure (hα : 0 < α) (ρ σ : MState d₁) (
  D̃_α(ρ ⊗M MState.pure ψ || σ ⊗M MState.pure ψ) = D̃_α(ρ || σ) := by
  simp [hα]

/-- The sandwiched Rényi divergence is invariant under SWAP. -
/
@[simp]
theorem sandwichedRenyiEntropy_SWAP (ρ σ : MState (dA × dB)) :
  D̃_α(ρ.SWAP || σ.SWAP) = D̃_α(ρ || σ) := by
  exact sandwichedRelEntropy_relabel ρ σ _

/-
Monotonicity of the sandwiched Rényi divergence under traceRight for `α > 1`
without the kernel condition. When the kernel condition fails, `D̃_α = τ` and
the inequality is trivial.
-/
theorem sandwichedRenyiEntropy_mono_traceRight' [Nonempty dB]
  (hα : 1 < α) (ρ σ : MState (dA × dB)) :
  D̃_α(ρ.traceRight || σ.traceRight) ≤ D̃_α(ρ || σ) := by

```

```

by_cases hker :  $\sigma.M.ker \leq \rho.M.ker$ 
· exact sandwichedRenyiEntropy_mono_traceRight h $\alpha$   $\rho$   $\sigma$  hker
· simp only [SandwichedRelRentropy, MState.traceRight_M]
  split
  next h => simp_all only [le_top]
  next h => simp_all only [not_lt, le_refl]

/-- Monotonicity of the sandwiched Rényi divergence under `traceLeft` for ` $\alpha > 1$ `.
Follows from `sandwichedRenyiEntropy_mono_traceRight` + SWAP invariance. -
/
theorem sandwichedRenyiEntropy_mono_traceLeft [Nonempty dA]
  (h $\alpha$  :  $1 < \alpha$ ) ( $\rho$   $\sigma$  : MState (dA  $\times$  dB)) :
   $\tilde{D}_\alpha(\rho.traceLeft || \sigma.traceLeft) \leq \tilde{D}_\alpha(\rho || \sigma)$  := by
  -- traceLeft = SWAP.traceRight, and SWAP preserves the SRD
  rw [← MState.traceRight_SWAP, ← MState.traceRight_SWAP]
  calc  $\tilde{D}_\alpha(\rho.SWAP.traceRight || \sigma.SWAP.traceRight)$ 
     $\leq \tilde{D}_\alpha(\rho.SWAP || \sigma.SWAP)$  :=
      sandwichedRenyiEntropy_mono_traceRight' h $\alpha$   $\rho.SWAP$   $\sigma.SWAP$ 
  _ =  $\tilde{D}_\alpha(\rho || \sigma)$  := sandwichedRenyiEntropy_SWAP  $\rho$   $\sigma$ 

/-- Helper: The Stinespring preparation `prep ∘ append` equals tensoring with a fixed
`append = ofEquiv (Equiv.prodPUnit d1).symm`.
TODO: PULLOUT to a more reasonable place. -/
theorem prep_append_eq_tensor_pure [Inhabited d2] ( $\rho$  : MState d1) :
  let  $\psi_0$  : Ket (d2  $\times$  d2) := Ket.basis default
  let  $\tau$  := MState.pure  $\psi_0$ 
  let zero_prep : CPTPMap Unit (d2  $\times$  d2) := CPTPMap.replacement  $\tau$ 
  let prep := (CPTPMap.id  $\otimes^{C P}$  zero_prep)
  let append : CPTPMap d1 (d1  $\times$  Unit) := CPTPMap.ofEquiv (Equiv.prodPUnit d1).symm
  (prep  $\circ_m$  append)  $\rho$  =  $\rho \otimes^M \tau$  := by
  apply MState.ext
  ext1
  funext {a1, b1} {a2, b2}
  have h := CPTPMap.prep_append_map_entry  $\rho.m$  a1 b1 a2 b2
  simp only [MState.prod, kronecker]
  exact h

/-- The Data Processing Inequality for the Sandwiched Rényi relative entropy ( $\alpha > 1$ )
Every CPTP map ` $\Phi$ ` satisfies ` $\tilde{D}_\alpha(\Phi\rho || \Phi\sigma) \leq \tilde{D}_\alpha(\rho || \sigma)$ `.

The proof uses the Stinespring representation (see `CPTPMap.exists_purify`):
every CPTP map can be written as ancilla preparation + unitary conjugation + partial trace
Since the sandwiched Rényi divergence is invariant under the first two operations
(by additivity and relabel invariance) and monotone under partial trace
(by `sandwichedRenyiEntropy_mono_traceRight`), the DPI follows. -
/

```

```

theorem sandwichedRenyiEntropy_DPI_gt_one (hα : 1 < α) (ρ σ : MState d₁) (Φ : CPTMap d₁ d₂) :
   $\tilde{D}_\alpha(\Phi \rho \parallel \Phi \sigma) \leq \tilde{D}_\alpha(\rho \parallel \sigma)$  := by
  have _ : Nonempty d₁ := ρ.nonempty
  have _ : Nonempty d₂ := (Φ ρ).nonempty
  haveI : Inhabited d₂ := Classical.inhabited_of_nonempty <_>
  let ψ₀ : Ket (d₂ × d₂) := Ket.basis default
  let τ := MState.pure ψ₀
  obtain (U, hU) := Φ.purify_IsUnitary
  -- Use the `zero_prep` / `prep` / `append` from `CPTMap.purify_trace`
  let zero_prep : CPTMap Unit (d₂ × d₂) := CPTMap.replacement τ
  let prep := ((CPTMap.id : CPTMap d₁ d₁) ⊗C P zero_prep)
  let append : CPTMap d₁ (d₁ × Unit) := CPTMap.ofEquiv (Equiv.prodPUnit d₁ Unit)
  calc  $\tilde{D}_\alpha(\Phi \rho \parallel \Phi \sigma)$ 
    =  $\tilde{D}_\alpha((\Phi.\text{purify } ((\text{prep} \circ_m \text{append}) \rho)).\text{traceLeft}.\text{traceLeft} \parallel$ 
       $(\Phi.\text{purify } ((\text{prep} \circ_m \text{append}) \sigma)).\text{traceLeft}.\text{traceLeft})$  := by
      have h_trace (ξ) : Φ ξ = (Φ.purify ((prep ◦m append) ξ)).traceLeft.
        simpα using congr($Φ.purify_trace ξ)
      rw [h_trace ρ, h_trace σ]
    =  $\tilde{D}_\alpha(((\rho \otimes^M \tau).U\_conj \ U).\text{traceLeft}.\text{traceLeft} \parallel$ 
       $((\sigma \otimes^M \tau).U\_conj \ U).\text{traceLeft}.\text{traceLeft})$  := by
      have h_app (ξ) : Φ.purify ξ = ξ.U_conj U := congr($hU ξ)
      rw [prep_append_eq_tensor_pure ρ, prep_append_eq_tensor_pure σ, h_app]
    ≤  $\tilde{D}_\alpha(((\rho \otimes^M \tau).U\_conj \ U).\text{traceLeft} \parallel ((\sigma \otimes^M \tau).U\_conj \ U).\text{traceLeft})$  :=
      sandwichedRenyiEntropy_mono_traceLeft hα ..
    ≤  $\tilde{D}_\alpha((\rho \otimes^M \tau).U\_conj \ U \parallel (\sigma \otimes^M \tau).U\_conj \ U)$  :=
      sandwichedRenyiEntropy_mono_traceLeft hα ..
    =  $\tilde{D}_\alpha(\rho \otimes^M \tau \parallel \sigma \otimes^M \tau)$  :=
      sandwichedRenyiEntropy_conj_unitary (by positivity) _ _ _
    =  $\tilde{D}_\alpha(\rho \parallel \sigma)$  :=
      sandwichedRenyiEntropy_tensor_pure (by positivity) ρ σ ψ₀

/-
The DPI for the sandwiched Rényi divergence at α = 1 (the quantum relative
entropy) follows from the α > 1 case by taking a limit, using the continuity of
the map α ↦  $\tilde{D}_\alpha(\rho \parallel \sigma)$  established in `sandwichedRelRentropy.continuousOn`.
-/
theorem sandwichedRenyiEntropy_DPI_eq_one (ρ σ : MState d₁) (Φ : CPTMap d₁ d₂) :
   $\tilde{D}_1(\Phi \rho \parallel \Phi \sigma) \leq \tilde{D}_1(\rho \parallel \sigma)$  := by
  -- Since α ↦  $\tilde{D}_\alpha(\rho \parallel \sigma)$  is continuous on (0, ∞), we can take the limit as α ↓ 1
  have h_cont :
    Filter.Tendsto (fun α : ℝ =>  $\tilde{D}_\alpha(\Phi \rho \parallel \Phi \sigma)$ ) (λ [>] 1) (λ (α : ℝ) =>  $\tilde{D}_\alpha(\rho \parallel \sigma)$ ) :=
    Filter.Tendsto (fun α : ℝ =>  $\tilde{D}_\alpha(\rho \parallel \sigma)$ ) (λ [>] 1) (λ (α : ℝ) =>  $\tilde{D}_\alpha(\rho \parallel \sigma)$ ) :=
    constructor
  · exact tendsto_nhdsWithin_of_tendsto_nhds (sandwichedRelRentropy.continuousOn
    ContinuousOn.continuousAt <| Ioi_mem_nhds zero_lt_one)
  · exact tendsto_nhdsWithin_of_tendsto_nhds (sandwichedRelRentropy.continuousOn

```

```

ContinuousOn.continuousAt <| Ioi_mem_nhds zero_lt_one)
exact le_of_tendsto_of_tendsto h_cont.1 h_cont.2 <| Filter.eventually_of_mem
  self_mem_nhdsWithin fun x hx => sandwichedRenyiEntropy_DPI_gt_one hx ρ σ Φ

/-- The Data Processing Inequality for the Sandwiched Renyi relative entropy.
Proved following the approach of Frank–Lieb and Leditzky–Rouzé–Datta. -
/
theorem sandwichedRenyiEntropy_DPI (hα : 1 ≤ α) (ρ σ : MState d₁) (Φ : CPTPMap d₁ d₂) :
  D_α(Φ ρ || Φ σ) ≤ D_α(ρ || σ) := by
  rcases hα.lt_or_eq with hα | rfl
  · exact sandwichedRenyiEntropy_DPI_gt_one hα ρ σ Φ
  · exact sandwichedRenyiEntropy_DPI_eq_one ρ σ Φ

```

1.111 QuantumInfo/Entropy/Relative.lean

```

public import QuantumInfo.Entropy.VonNeumann
public import Physlib.Meta.Sorry
open scoped Matrix ComplexOrder InnerProductSpace RealInnerProductSpace HermitianMat
@[sorryful]
omit [DecidableEq dA] in
open HermitianMat in
private lemma inner_kron_one_eq_inner_traceRight
  (A : HermitianMat dA ℂ) (M : HermitianMat (dA × dB) ℂ) :
  [A ⊗ₖ (1 : HermitianMat dB ℂ), M] = [A, M.traceRight] := by
  rw [inner_comm, inner_eq_re_trace, inner_eq_re_trace]
  exact (congrArg Complex.re (Matrix.trace_mul_kron_one_right M.mat A.mat)).trans <|
    simpa using congrArg Complex.re (Matrix.trace_mul_comm M.traceRight.mat A.mat)

omit [DecidableEq dB] in
open HermitianMat in
private lemma inner_one_kron_eq_inner_traceLeft
  (B : HermitianMat dB ℂ) (M : HermitianMat (dA × dB) ℂ) :
  [(1 : HermitianMat dA ℂ) ⊗ₖ B, M] = [B, M.traceLeft] := by
  rw [inner_comm, inner_eq_re_trace, inner_eq_re_trace]
  exact (congrArg Complex.re (Matrix.trace_mul_one_kron_right M.mat B.mat)).trans <|
    simpa using congrArg Complex.re (Matrix.trace_mul_comm M.traceLeft.mat B.mat)

private lemma fixed_support_kron_right (ρ : MState (dA × dB))
  {x : EuclideanSpace ℂ (dA × dB)} (hx : x ∈ ρ.M.support) :
  (ρ.traceRight.M.supportProj ⊗ₖ (1 : HermitianMat dB ℂ)).lin x = x := by
  let K : HermitianMat (dA × dB) ℂ := ρ.traceRight.M.kerProj ⊗ₖ (1 : HermitianMat dB ℂ)
  simpa [Matrix.toEuclideanLin,
    show K.mat.toEuclideanLin x = 0 by
      simpa [HermitianMat.lin] using

```

```

((HermitianMat.inner_zero_iff p.nonneg (by
  simp [K] using HermitianMat.kronecker_nonneg
  (by simp [HermitianMat.kerProj] using
    (HermitianMat.projector_nonneg (S := p.traceRight.M.ker)))
  (by rw [HermitianMat.zero_le_iff]; exact Matrix.PosSemidef.one)
  rw [HermitianMat.inner_comm, inner_kron_one_eq_inner_traceRight,
    HermitianMat.inner_comm]
  simp [K, MState.exp_val] using
    (p.traceRight.exp_val_eq_zero_iff
     (by simp [HermitianMat.kerProj] using
       (HermitianMat.projector_nonneg (S := p.traceRight.M.ker))))
  (by simp)) hx)] using
congrArg (fun T : HermitianMat (dA × dB) ℂ => T.mat.toEuclideanLin x)
(show K + p.traceRight.M.supportProj ⊗k (1 : HermitianMat dB ℂ) = 1 b
simp only [K, ← HermitianMat.add_kronecker,
  p.traceRight.M.kerProj_add_supportProj, HermitianMat.kronecker_on

private lemma fixed_support_kron_left (p : MState (dA × dB))
  {x : EuclideanSpace ℂ (dA × dB)} (hx : x ∈ p.M.support) :
  ((1 : HermitianMat dA ℂ) ⊗k p.traceLeft.M.supportProj).lin x = x := by
let K : HermitianMat (dA × dB) ℂ := (1 : HermitianMat dA ℂ) ⊗k p.traceLeft
let P : HermitianMat (dA × dB) ℂ := (1 : HermitianMat dB ℂ) ⊗k p.traceLeft
have hK_nonneg : 0 ≤ K := by
  dsimp [K]
  exact HermitianMat.kronecker_nonneg
  (by rw [HermitianMat.zero_le_iff]; exact Matrix.PosSemidef.one)
  (by simp [HermitianMat.kerProj] using
    (HermitianMat.projector_nonneg (S := p.traceLeft.M.ker)))
have hsum : K + P = 1 := by
  show K + P = 1
  simp only [K, P, ← HermitianMat.kronecker_add, p.traceLeft.M.kerProj_add,
    HermitianMat.kronecker_one_one]
simp [P, Matrix.toEuclideanLin,
  show K.mat.toEuclideanLin x = 0 by
  simp [HermitianMat.lin] using ((HermitianMat.inner_zero_iff p.nonneg
  rw [HermitianMat.inner_comm, inner_one_kron_eq_inner_traceLeft]
  rw [HermitianMat.inner_comm]
  simp [K, MState.exp_val] using
    (p.traceLeft.exp_val_eq_zero_iff
     (by simp [HermitianMat.kerProj] using
       (HermitianMat.projector_nonneg (S := p.traceLeft.M.ker))))).2
  (by simp)) hx)] using
congrArg (fun T : HermitianMat (dA × dB) ℂ => T.mat.toEuclideanLin x) h

have fixed_support_kron_prod : ∀ {x : EuclideanSpace ℂ (dA × dB)},
  x ∈ p.M.support →

```



```

      (p.traceRight.M.supportProj ⊗k p.traceLeft.M.supportProj).lin x = x := by
intro x hx
let A : Matrix (dA × dB) (dA × dB) ℂ :=
  Matrix.kroneckerMap (· * ·) p.traceRight.M.supportProj.mat (1 : Matrix dB dB ℂ)
let B : Matrix (dA × dB) (dA × dB) ℂ :=
  Matrix.kroneckerMap (· * ·) (1 : Matrix dA dA ℂ) p.traceLeft.M.supportProj.mat
have hxA' : A.toEuclideanLin x = x := by
  simpa [A, HermitianMat.lin, Matrix.toEuclideanLin] using fixed_support_kron_r
have hxB' : B.toEuclideanLin x = x := by
  simpa [B, HermitianMat.lin, Matrix.toEuclideanLin] using fixed_support_kron_le
have hmul : (A * B).toEuclideanLin x = x := by
  simpa [A, B, Matrix.toEuclideanLin, Matrix.mulVec_mulVec] using
    show A.toEuclideanLin (B.toEuclideanLin x) = x by rw [hxB', hxA']
  simpa [HermitianMat.lin, Matrix.toEuclideanLin, A, B, ← Matrix.mul_kronecker_mul]
have prod_marginals_ker_le : (p.traceRight ⊗M p.traceLeft).M.ker ≤ p.M.ker := by
  let P : HermitianMat (dA × dB) ℂ := p.traceRight.M.supportProj ⊗k p.traceLeft.M.
  have hP : p.M.support ≤ P.support := by
    intro x hx
    exact ⟨x, by simpa [P] using fixed_support_kron_prod hx⟩
  have hkerP : P.ker ≤ p.M.ker := by
    simpa [HermitianMat.support_orthogonal_eq_range] using Submodule.orthogonal_le
  exact (show (p.traceRight ⊗M p.traceLeft).M.ker ≤ P.ker by
    change LinearMap.ker
      ((Matrix.kroneckerMap (· * ·) p.traceRight.M.mat p.traceLeft.M.mat).toEuclideanLin)
      ≤ LinearMap.ker
      ((Matrix.kroneckerMap (· * ·) p.traceRight.M.supportProj.mat
        p.traceLeft.M.supportProj.mat).toEuclideanLin))
    exact ker_kron_le_of_le _ _ _ _
    (by
      simpa [HermitianMat.ker, HermitianMat.lin] using
        (show p.traceRight.M.ker ≤ p.traceRight.M.supportProj.ker by simp))
    (by
      simpa [HermitianMat.ker, HermitianMat.lin] using
        (show p.traceLeft.M.ker ≤ p.traceLeft.M.supportProj.ker by simp)))
  have right_mul_eq_of_fixed_support {Q pM : HermitianMat (dA × dB) ℂ}
    (hfix : ∀ x : EuclideanSpace ℂ (dA × dB), x ∈ pM.support → Q.lin x = x) :
    pM.mat * Q.mat = pM.mat := by
  have hleft : Q.mat * pM.mat = pM.mat := by
    rw [Matrix.ext_iff_mulVec]
    intro v
    have hv : WithLp.toLp 2 (pM.mat.mulVec v) ∈ pM.support :=
      Set.mem_range_self (WithLp.toLp 2 v)
    simpa [Matrix.mulVec_mulVec, HermitianMat.lin, Matrix.toEuclideanLin] using hfix
    simpa [Matrix.conjTranspose_mul, HermitianMat.conjTranspose_mat] using
      congrArg Matrix.conjTranspose hleft
  have inner_kron_support_right_eq (A : HermitianMat dA ℂ) :

```

```

    [p.M, A ⊗k p.traceLeft.M.supportProj] = [p.M, A ⊗k (1 : HermitianMat
let Q : HermitianMat (dA × dB) ℂ := (1 : HermitianMat dA ℂ) ⊗k p.traceL
have hQ : p.M.mat * Q.mat = p.M.mat := by
  apply right_mul_eq_of_fixed_support
  intro x hx
  simp [Q] using fixed_support_kron_left p hx
have hmat : Q.mat * (A ⊗k (1 : HermitianMat dB ℂ)).mat =
  (A ⊗k p.traceLeft.M.supportProj).mat := by
  simp [Q, HermitianMat.kronecker_mat, ← Matrix.mul_kronecker_mul]
rw [HermitianMat.inner_eq_re_trace, HermitianMat.inner_eq_re_trace]
apply congrArg Complex.re
rw [← hmat, ← Matrix.mul_assoc, hQ]
have inner_support_kron_left_eq (B : HermitianMat dB ℂ) :
  [p.M, p.traceRight.M.supportProj ⊗k B] = [p.M, (1 : HermitianMat dA ℂ)
let Q : HermitianMat (dA × dB) ℂ := p.traceRight.M.supportProj ⊗k (1 :
have hQ : p.M.mat * Q.mat = p.M.mat := by
  apply right_mul_eq_of_fixed_support
  intro x hx
  simp [Q] using fixed_support_kron_right p hx
have hmat : Q.mat * ((1 : HermitianMat dA ℂ) ⊗k B).mat =
  (p.traceRight.M.supportProj ⊗k B).mat := by
  simp [Q, HermitianMat.kronecker_mat, ← Matrix.mul_kronecker_mul]
rw [HermitianMat.inner_eq_re_trace, HermitianMat.inner_eq_re_trace]
apply congrArg Complex.re
rw [← hmat, ← Matrix.mul_assoc, hQ]
rw [qRelativeEnt_ker prod_marginals_ker_le, qMutualInfo,
  Svn_eq_neg_trace_log, Svn_eq_neg_trace_log, Svn_eq_neg_trace_log]
rw [show (p.traceRight ⊗M p.traceLeft).M.log =
  p.traceRight.M.log ⊗k p.traceLeft.M.supportProj +
  p.traceRight.M.supportProj ⊗k p.traceLeft.M.log by
  simp [MState.prod] using
    (HermitianMat.log_kron_with_proj (A := p.traceRight.M) (B := p.traceL
    inner_sub_right, inner_add_right)]
rw [show [p.M.log, p.M] = [p.M, p.M.log] by rw [HermitianMat.inner_comm]]
exact congrArg (fun x : ℝ => (x : EReal)) <| by
  rw [show [p.M, p.traceRight.M.log ⊗k p.traceLeft.M.supportProj] =
    [p.traceRight.M.log, p.traceRight.M] by
    calc
      _ = [p.M, p.traceRight.M.log ⊗k (1 : HermitianMat dB ℂ)] :=
        inner_kron_support_right_eq p.traceRight.M.log
      _ = [p.traceRight.M.log, p.traceRight.M] := by
        simp [HermitianMat.inner_comm] using
          inner_kron_one_eq_inner_traceRight p.traceRight.M.log p.M,
  show [p.M, p.traceRight.M.supportProj ⊗k p.traceLeft.M.log] =
    [p.traceLeft.M.log, p.traceLeft.M] by
    calc

```

```

_ = [p.M, (1 : HermitianMat dA ℂ) ⊗k p.traceLeft.M.log] :=
  inner_support_kron_left_eq p.traceLeft.M.log
_ = [p.traceLeft.M.log, p.traceLeft.M] := by
  simpa [HermitianMat.inner_comm] using
    inner_one_kron_eq_inner_traceLeft p.traceLeft.M.log p.M]
ring

```

1.112 QuantumInfo/Entropy/SSA.lean

```
public import QuantumInfo.Entropy.VonNeumann
```

1.113 QuantumInfo/Entropy/VonNeumann.lean

```

public import QuantumInfo.States.Pure.Braket
public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Channels.CPTP
public import QuantumInfo.Channels.Dual
public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.Channels.Unbundled

```

1.114 QuantumInfo/ForMathlib/ComplexLaplaceTransform.lean

```

/-
Copyright (c) 2025 Dennj Osele. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Dennj Osele
-/
module

public import Mathlib.Analysis.Complex.HasPrimitives
public import Mathlib.Analysis.Complex.RealDeriv
public import Mathlib.Analysis.SpecialFunctions.Log.Deriv
public import Mathlib.Data.Real.StarOrdered
public import Mathlib.MeasureTheory.Constructions.Pi
public import Mathlib.MeasureTheory.Constructions.BorelSpace.WithTop
public import Mathlib.MeasureTheory.Function.StronglyMeasurable.Basic
public import Mathlib.MeasureTheory.Integral.Bochner.Basic
public import Mathlib.MeasureTheory.Integral.Bochner.ContinuousLinearMap
public import Mathlib.MeasureTheory.Integral.Bochner.L1
public import Mathlib.MeasureTheory.Integral.Bochner.VitaliCaratheodory
public import Mathlib.MeasureTheory.Integral.DominatedConvergence
public import Mathlib.MeasureTheory.Measure.Prod

```

```

public import Mathlib.MeasureTheory.Measure.Haar.OfBasis
public import Mathlib.Order.CompletePartialOrder

@[expose] public section

noncomputable section

/-- The integrand of the complex Laplace transform of a possibly infinite-
valued energy function. -/
def ComplexLaplaceIntegrand {α : Type*} (E : α → WithTop ℝ) (z : ℂ) (x : α)
  if h : E x = ⊤ then 0 else Complex.exp (-z * (E x).untop h : ℂ)

/-- The complex Laplace transform of a possibly infinite-
valued energy function. -/
def ComplexLaplaceTransform {α : Type*} [MeasurableSpace α] [MeasureTheory.
  (E : α → WithTop ℝ) (z : ℂ) : ℂ :=
  ∫ x, ComplexLaplaceIntegrand E z x

/-- The complex convergence domain of a Laplace transform. -
/
def ComplexLaplaceConvergenceDomain {α : Type*} [MeasurableSpace α] [Measur
  (E : α → WithTop ℝ) : Set ℂ :=
  {z | MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplace

/-- A two-sided exponential envelope controlling the Laplace integrand near
/
private def ComplexLaplaceEnvelope {α : Type*} (E : α → WithTop ℝ) (z : ℂ)
  (x : α) : ℝ :=
  ||ComplexLaplaceIntegrand E (z - (δ : ℂ)) x|| +
  ||ComplexLaplaceIntegrand E (z + (δ : ℂ)) x||

private def ComplexLaplaceEndpointEnvelope {α : Type*} (E : α → WithTop ℝ)
  (x : α) : ℝ :=
  ||ComplexLaplaceIntegrand E z x|| + ||ComplexLaplaceIntegrand E w x||

private theorem norm_complexLaplaceIntegrand_le_envelope
  {α : Type*} {E : α → WithTop ℝ} {z w : ℂ} {δ : ℝ}
  (hw : w ∈ Metric.closedBall z δ) (x : α) :
  ||ComplexLaplaceIntegrand E w x|| ≤ ComplexLaplaceEnvelope E z δ x := by
  unfold ComplexLaplaceEnvelope ComplexLaplaceIntegrand
  by_cases h : E x = ⊤
  · simp [h]
  · simp only [h, dite_false]
    set e : ℝ := (E x).untop h
    have hdist : ||w - z|| ≤ δ := by
      simpa [Metric.mem_closedBall, dist_eq_norm, norm_sub_rev] using hw

```

```

have hre_abs : |w.re - z.re| ≤ δ := by
  exact (Complex.abs_re_le_norm (w - z)).trans (by simp [Complex.sub_re] using
  rw [Complex.norm_exp, Complex.norm_exp, Complex.norm_exp]
  rcases le_total 0 e with he | he
  · refine le_add_of_le_of_nonneg ?_ (Real.exp_pos _).le
    exact Real.exp_le_exp.mpr (by
      simp only [neg_mul, Complex.mul_re, Complex.neg_re, Complex.ofReal_re, Compl
      mul_zero, sub_zero, Complex.sub_re]
      nlinarith [abs_le.mp hre_abs |>.1])
  · refine le_add_of_nonneg_of_le (Real.exp_pos _).le ?_
    exact Real.exp_le_exp.mpr (by
      simp only [neg_mul, Complex.mul_re, Complex.neg_re, Complex.ofReal_re, Compl
      mul_zero, sub_zero, Complex.add_re]
      nlinarith [abs_le.mp hre_abs |>.2])

private theorem norm_complexLaplaceIntegrand_horizontal_le_endpointEnvelope
  {α : Type*} {E : α → WithTop ℝ} {a b : ℂ} {t : ℝ}
  (ht : t ∈ Set.uIcc a.re b.re) (x : α) :
  ||ComplexLaplaceIntegrand E (t + a.im * Complex.I) x|| ≤
  ComplexLaplaceEndpointEnvelope E a (b.re + a.im * Complex.I) x := by
  unfold ComplexLaplaceEndpointEnvelope ComplexLaplaceIntegrand
  by_cases h : E x = ⊤
  · simp [h]
  · simp only [h, dite_false]
    set e : ℝ := (E x).untop h
    rw [Complex.norm_exp, Complex.norm_exp, Complex.norm_exp]
    rcases Set.mem_uIcc.mp ht with ⟨hat, htb⟩ | ⟨hbt, hta⟩ <=> rcases le_total 0 e w
    · refine le_add_of_le_of_nonneg ?_ (Real.exp_pos _).le
      exact Real.exp_le_exp.mpr (by
        simp only [neg_mul, Complex.mul_re, Complex.neg_re, Complex.ofReal_re, Compl
        mul_zero, sub_zero, Complex.add_re, Complex.I_re]
        nlinarith)
    · refine le_add_of_nonneg_of_le (Real.exp_pos _).le ?_
      exact Real.exp_le_exp.mpr (by
        simp only [neg_mul, Complex.mul_re, Complex.neg_re, Complex.ofReal_re, Compl
        mul_zero, sub_zero, Complex.add_re, Complex.I_re]
        nlinarith)
    · refine le_add_of_nonneg_of_le (Real.exp_pos _).le ?_
      exact Real.exp_le_exp.mpr (by
        simp only [neg_mul, Complex.mul_re, Complex.neg_re, Complex.ofReal_re, Compl
        mul_zero, sub_zero, Complex.add_re, Complex.I_re]
        nlinarith)
    · refine le_add_of_le_of_nonneg ?_ (Real.exp_pos _).le
      exact Real.exp_le_exp.mpr (by
        simp only [neg_mul, Complex.mul_re, Complex.neg_re, Complex.ofReal_re, Compl
        mul_zero, sub_zero, Complex.add_re, Complex.I_re]

```

```

nlinarith)

private theorem norm_complexLaplaceIntegrand_vertical_le_endpointEnvelope
  {α : Type*} {E : α → WithTop ℝ} {a b : ℂ} {t : ℝ}
  (x : α) :
    ||ComplexLaplaceIntegrand E (b.re + t * Complex.I) x|| ≤
      ComplexLaplaceEndpointEnvelope E b (b.re + a.im * Complex.I) x := by
  unfold ComplexLaplaceEndpointEnvelope ComplexLaplaceIntegrand
  by_cases h : E x = ⊤
  · simp [h]
  · simp only [h, dite_false]
    rw [Complex.norm_exp, Complex.norm_exp, Complex.norm_exp]
    refine le_add_of_le_of_nonneg (le_of_eq ?_) (Real.exp_pos _).le
    congr 1
    simp only [neg_mul, Complex.mul_re, Complex.neg_re, Complex.ofReal_re,
      mul_zero, zero_mul, sub_zero, add_zero, Complex.add_re, Complex.I_re]

/-- For each configuration, the complex Laplace integrand is analytic in the
/
theorem analyticAt_complexLaplaceIntegrand
  {α : Type*} (E : α → WithTop ℝ) (x : α) (z : ℂ) :
    AnalyticAt ℂ (fun w : ℂ => ComplexLaplaceIntegrand E w x) z := by
  unfold ComplexLaplaceIntegrand
  split_ifs
  · fun_prop
  · fun_prop

theorem measurable_complexLaplaceIntegrand
  {α : Type*} [MeasurableSpace α] {E : α → WithTop ℝ} (hE : Measurable E)
  Measurable (ComplexLaplaceIntegrand E z) := by
  rw [show ComplexLaplaceIntegrand E z =
    fun x => if E x = ⊤ then 0 else Complex.exp (-z * ((E x).untopD 0 :
    funext x
    unfold ComplexLaplaceIntegrand
    by_cases hx : E x = ⊤
    · simp [hx]
    · simp only [hx, dite_false, ite_false]
    congr 2
    let y := E x
    have hy : y ≠ ⊤ := by simp [y] using hx
    change ((y.untopD hy : ℝ) : ℂ) = ((y.untopD 0 : ℝ) : ℂ)
    obtain ⟨e, he⟩ := WithTop.ne_top_iff_exists.mp hy
    have hUntop : y.untopD hy = e := by
      apply WithTop.coe_inj.mp
      rw [WithTop.coe_untop, ← he]
    have hUntopD : y.untopD 0 = e := by

```

```

      rw [← he]
      rfl
      rw [hUntop, hUntopD]]
    exact Measurable.ite (hE (measurableSet_singleton (τ : WithTop ℝ))) measurable_con
      (Complex.continuous_exp.measurable.comp (by fun_prop :
        Measurable fun x => -z * (((E x).untopD 0 : ℝ) : ℂ)))

/-- Interior convergence gives integrability throughout a neighborhood of the parame
/
theorem eventually_integrable_complexLaplaceIntegrand_of_mem_interior_convergenceDom
  {α : Type*} [MeasurableSpace α] [MeasureTheory.MeasureSpace α] {E : α → WithTop
    (hz : z ∈ interior (ComplexLaplaceConvergenceDomain E)) :
    ∀f w in nhds z,
      MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplaceIntegrand
        Filter.mem_of_superset (IsOpen.mem_nhds isOpen_interior hz) _root_.interior_subset

theorem continuousAt_complexLaplaceTransform_of_mem_interior_convergenceDomain
  {α : Type*} [MeasurableSpace α] [MeasureTheory.MeasureSpace α] {E : α → WithTop
    (hz : z ∈ interior (ComplexLaplaceConvergenceDomain E)) :
      ContinuousAt (ComplexLaplaceTransform E) z := by
    rcases Metric.isOpen_iff.mp isOpen_interior z hz with (ε, hε_pos, hε)
    have hint {w : ℂ} (hw : w ∈ Metric.ball z ε) :
      MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplaceIntegrand
        _root_.interior_subset (s := ComplexLaplaceConvergenceDomain E) (hε hw)
    have hbound_int : MeasureTheory.Integrable (μ := MeasureTheory.volume)
      (ComplexLaplaceEnvelope E z (ε / 2)) := by
      unfold ComplexLaplaceEnvelope
      simpa [Pi.add_apply] using (hint (by
        rw [Metric.mem_ball, dist_eq_norm]
        simpa [abs_of_pos hε_pos] using show ε / 2 < ε by linarith)).norm.add (hint (b
          rw [Metric.mem_ball, dist_eq_norm]
          simpa [abs_of_pos hε_pos] using show ε / 2 < ε by linarith)).norm
    simpa [ComplexLaplaceTransform] using
      MeasureTheory.tendsto_integral_filter_of_dominated_convergence
        (μ := MeasureTheory.volume) (l := nhds z)
        (F := fun w x => ComplexLaplaceIntegrand E w x)
        (f := fun x => ComplexLaplaceIntegrand E z x)
        (ComplexLaplaceEnvelope E z (ε / 2))
        ((eventually_integrable_complexLaplaceIntegrand_of_mem_interior_convergenceDom
          fun _ hw => hw.astronglyMeasurable)
        (Filter.Eventually.mono (Metric.ball_mem_nhds z (by positivity : 0 < ε / 2)) f
          Filter.Eventually.of_forall fun x => norm_complexLaplaceIntegrand_le_envelop
            (Metric.mem_closedBall.mpr (le_of_lt (Metric.mem_ball.mp hw))) x)
        hbound_int
        (Filter.Eventually.of_forall fun x => (analyticAt_complexLaplaceIntegrand E x

```

```

theorem continuousOn_complexLaplaceTransform_interior_convergenceDomain
  {α : Type*} [MeasurableSpace α] [MeasureTheory.MeasureSpace α] {E : α → ℝ}
  ContinuousOn (ComplexLaplaceTransform E) (interior (ComplexLaplaceConvergenceDomain E))
intro z hz
exact (continuousAt_complexLaplaceTransform_of_mem_interior_convergenceDomain E z hz)

private theorem integrable_uncurry_complexLaplaceIntegrand_horizontal
  {α : Type*} [MeasureTheory.MeasureSpace α]
  [MeasureTheory.SFinite (MeasureTheory.volume : MeasureTheory.Measure α)]
  {a b : ℂ}
  (hE : Measurable E)
  (ha : MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplaceIntegrand E (a.re + a.im * Complex.I)))
  (hb : MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplaceIntegrand E (b.re + b.im * Complex.I))) :
  MeasureTheory.Integrable
  (Function.uncurry fun t : ℝ => fun x : α =>
    ComplexLaplaceIntegrand E (t + a.im * Complex.I) x)
  ((MeasureTheory.volume.restrict (Set.uIoc a.re b.re)).prod MeasureTheory.volume) := by
  let μI := MeasureTheory.volume.restrict (Set.uIoc a.re b.re)
  haveI : MeasureTheory.IsFiniteMeasure μI := ⟨by
    rw [MeasureTheory.Measure.restrict_apply_univ]
    simp [Set.uIoc]⟩
  have hsm : MeasureTheory.StronglyMeasurable (Function.uncurry fun t : ℝ =>
    ComplexLaplaceIntegrand E (t + a.im * Complex.I) x) := by
    refine MeasureTheory.stronglyMeasurable_uncurry_of_continuous_of_stronglyMeasurable
    · intro x
      unfold ComplexLaplaceIntegrand
      split_ifs <=> fun_prop
    · intro t
      exact (measurable_complexLaplaceIntegrand hE (t + a.im * Complex.I)).
  have henv : MeasureTheory.Integrable
    (fun p : ℝ × α => ComplexLaplaceEndpointEnvelope E a (b.re + a.im * Complex.I) p) :=
    (μI.prod MeasureTheory.volume) :=
    (ha.norm.add hb.norm).comp_snd μI
  refine henv.mono' hsm.aestronglyMeasurable ?_
  simp only [Function.uncurry]
  rw [MeasureTheory.Measure.ae_prod_iff_ae_ae (measurableSet_le (Set.uIoc a.re b.re))]
  (show Measurable fun p : ℝ × α =>
    ||ComplexLaplaceIntegrand E (p.1 + a.im * Complex.I) p.2|| from hsm.measurable)
  (show Measurable fun p : ℝ × α =>
    ComplexLaplaceEndpointEnvelope E a (b.re + a.im * Complex.I) p.2 from
    ((measurable_complexLaplaceIntegrand hE a).norm.add
    (measurable_complexLaplaceIntegrand hE (b.re + a.im * Complex.I)))
  filter_upwards [MeasureTheory.ae_restrict_mem measurableSet_uIoc] with t
  exact Filter.Eventually.of_forall fun x =>
    norm_complexLaplaceIntegrand_horizontal_le_endpointEnvelope (Set.uIoc a.re b.re) x

```



```

private theorem integrable_uncurry_complexLaplaceIntegrand_vertical
  {α : Type*} [MeasureTheory.MeasureSpace α]
  [MeasureTheory.SFinite (MeasureTheory.volume : MeasureTheory.Measure α)] {E : α}
  {a b : ℂ}
  (hE : Measurable E)
  (hb : MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplaceIntegrand E (b.re + a.im * Complex.I)))
  (hba : MeasureTheory.Integrable (μ := MeasureTheory.volume)
    (ComplexLaplaceIntegrand E (b.re + a.im * Complex.I))) :
  MeasureTheory.Integrable
    (Function.uncurry fun t : ℝ => fun x : α =>
      ComplexLaplaceIntegrand E (b.re + t * Complex.I) x)
    ((MeasureTheory.volume.restrict (Set.uIoc a.im b.im)).prod MeasureTheory.volume)
let μI := MeasureTheory.volume.restrict (Set.uIoc a.im b.im)
have I : MeasureTheory.IsFiniteMeasure μI := ⟨by
  rw [MeasureTheory.Measure.restrict_apply_univ]
  simp [Set.uIoc]⟩
have hsm : MeasureTheory.StronglyMeasurable (Function.uncurry fun t : ℝ => fun x : α =>
  ComplexLaplaceIntegrand E (b.re + t * Complex.I) x) := by
  refine MeasureTheory.stronglyMeasurable_uncurry_of_continuous_of_stronglyMeasurable
  · intro x
    unfold ComplexLaplaceIntegrand
    split_ifs <;> fun_prop
  · intro t
    exact (measurable_complexLaplaceIntegrand hE (b.re + t * Complex.I)).stronglyMeasurable
refine ((hb.norm.add hba.norm).comp_snd μI).mono'
  hsm.astronglyMeasurable ?_
exact Filter.Eventually.of_forall fun p =>
  norm_complexLaplaceIntegrand_vertical_le_endpointEnvelope (a := a) (b := b) (t : ℝ)

private theorem wedgeIntegral_complexLaplaceTransform_eq_integral
  {α : Type*} [MeasureTheory.MeasureSpace α]
  [MeasureTheory.SFinite (MeasureTheory.volume : MeasureTheory.Measure α)] {E : α}
  {a b : ℂ}
  (hE : Measurable E)
  (hab : Complex.Rectangle a b ⊆ interior (ComplexLaplaceConvergenceDomain E)) :
  Complex.wedgeIntegral a b (ComplexLaplaceTransform E) =
    ∫ x, Complex.wedgeIntegral a b (fun w : ℂ => ComplexLaplaceIntegrand E w x) :=
  have hint {z : ℂ} (hz : z ∈ Complex.Rectangle a b) :
    MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplaceIntegrand E (z.re + z.im * Complex.I))
    _root_.interior_subset (s := ComplexLaplaceConvergenceDomain E) (hab hz)
  have ha : MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplaceIntegrand E (a.re + a.im * Complex.I))
    hint (by simp [Complex.Rectangle, Complex.mem_reProdIm])
  have hb : MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplaceIntegrand E (b.re + b.im * Complex.I))
    hint (by simp [Complex.Rectangle, Complex.mem_reProdIm])
  have hcorner : MeasureTheory.Integrable (μ := MeasureTheory.volume)

```

```

      (ComplexLaplaceIntegrand E (b.re + a.im * Complex.I)) :=
      hint (by simp [Complex.Rectangle, Complex.mem_reProdIm])
    have hH := MeasureTheory.intervalIntegral_integral_swap
      (μ := MeasureTheory.volume)
      (f := fun t x => ComplexLaplaceIntegrand E (t + a.im * Complex.I) x)
      (integrable_uncurry_complexLaplaceIntegrand_horizontal hE ha hcorner)
    have hV := MeasureTheory.intervalIntegral_integral_swap
      (μ := MeasureTheory.volume)
      (f := fun t x => ComplexLaplaceIntegrand E (b.re + t * Complex.I) x)
      (integrable_uncurry_complexLaplaceIntegrand_vertical hE hb hcorner)
    simp only [Complex.wedgeIntegral, ComplexLaplaceTransform]
    rw [hH, hV]
    rw [← MeasureTheory.interval_smul, ← MeasureTheory.interval_add]
    · rcases le_total a.re b.re with hab | hba
      · simp [intervalIntegral_integral_of_le hab, Function.uncurry, Set.uIo]
        using (integrable_uncurry_complexLaplaceIntegrand_horizontal hE ha)
      · simp [intervalIntegral_integral_of_ge hba, Function.uncurry, Set.uIo]
        using (integrable_uncurry_complexLaplaceIntegrand_horizontal hE ha)
    · refine (?! : MeasureTheory.Integrable _ MeasureTheory.volume).smul Comp
      rcases le_total a.im b.im with hab | hba
      · simp [intervalIntegral_integral_of_le hab, Function.uncurry, Set.uIo]
        using (integrable_uncurry_complexLaplaceIntegrand_vertical hE hb hc)
      · simp [intervalIntegral_integral_of_ge hba, Function.uncurry, Set.uIo]
        using (integrable_uncurry_complexLaplaceIntegrand_vertical hE hb hc)

/-- A complex Laplace transform is analytic on the interior of its convergence
/
theorem analyticAt_complexLaplaceTransform_of_mem_interior_convergenceDomain
  {α : Type*} [MeasureTheory.MeasureSpace α]
  [MeasureTheory.SFinite (MeasureTheory.volume : MeasureTheory.Measure α)]
  (hE : Measurable E)
  (hz : z ∈ interior (ComplexLaplaceConvergenceDomain E)) :
  AnalyticAt ℂ (ComplexLaplaceTransform E) z := by
  rcases Metric.isOpen_iff.mp isOpen_interior z hz with ⟨r, hr_pos, hr⟩
  have hcons : Complex.IsConservativeOn (ComplexLaplaceTransform E) (Metric
    intro a b hab
    rw [wedgeIntegral_complexLaplaceTransform_eq_integral hE (hab.trans hr)
      wedgeIntegral_complexLaplaceTransform_eq_integral hE (by
        simp [Complex.Rectangle, Set.uIcc_comm] using hab.trans hr)]
    · rw [← MeasureTheory.interval_neg]
      apply MeasureTheory.interval_congr_ae
      filter_upwards with x
      exact (DifferentiableOn.isConservativeOn
        (fun y _hy => (analyticAt_complexLaplaceIntegrand E x y).differenti
          a b (Set.subset_univ _))
      exact ((Complex.isConservativeOn_and_continuousOn_iff_isDifferentiableOn

```

1.115.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/BLOCKDIAGONAL.LEAN

```
(hcons, continuousOn_complexLaplaceTransform_interior_convergenceDomain.mono hr)
  (Metric.ball_mem_nhds z hr_pos)
```

end

1.115 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/BlockDiagonal

/-

Copyright (c) 2026 Hayata Yamasaki. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.

Authors: Kei Tsukamoto, Kento Mori, Hayata Yamasaki

-/

module

```
public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.LownerHeinzTheorem
public import Mathlib.Analysis.InnerProductSpace.Adjoint
public import Mathlib.Analysis.InnerProductSpace.PiL2
public import Mathlib.Topology.Algebra.Module.LinearMapPiProd
```

@[expose] public section

namespace JensenOperatorInequality

universe u

open LownerHeinzTheorem

variable { α : Type u}

variable [NormedAddCommGroup α] [InnerProductSpace $\mathbb{C}$ α] [CompleteSpace α]

variable [Nontrivial α]

/-- A two-fold Hilbert sum used for the block-operator proof of Jensen's inequality.

/

abbrev HSum (α : Type u) [NormedAddCommGroup α] [InnerProductSpace $\mathbb{C}$ α] : Type u :=
 PiLp 2 (fun _ : Fin 2 => α)

noncomputable def hsumEquiv (α : Type u) [NormedAddCommGroup α] [InnerProductSpace $\mathbb{C}$ α]
 HSum α $\approx_L$ [$\mathbb{C}$] (Fin 2 $\rightarrow$ α) :=

PiLp.continuousLinearEquiv (p := (2 : ENNReal)) (α := $\mathbb{C}$) (β := fun _ : Fin 2 => α)

noncomputable def hsumProj (α : Type u) [NormedAddCommGroup α] [InnerProductSpace $\mathbb{C}$ α]
 (i : Fin 2) : HSum α $\rightarrow_L$ [$\mathbb{C}$] α :=

(ContinuousLinearMap.proj (R := $\mathbb{C}$) (ϕ := fun _ : Fin 2 => α) i) $\circ_L$

```

(hsumEquiv []).toContinuousLinearMap

noncomputable def hsumIncl (α : Type u) [NormedAddCommGroup α] [InnerProductSpace α]
  (i : Fin 2) : α →L[ℂ] HSum α :=
  (hsumEquiv []).symm.toContinuousLinearMap ∘L
  (ContinuousLinearMap.single ℂ (fun _ : Fin 2 => α) i)

omit [CompleteSpace α] [Nontrivial α] in
@[simp] theorem hsumProj_hsumIncl_apply (i j : Fin 2) (x : α) :
  hsumProj α i (hsumIncl α j x) = if i = j then x else 0 := by
  fin_cases i <=> fin_cases j <=> simp [hsumProj, hsumIncl, hsumEquiv]

omit [CompleteSpace α] [Nontrivial α] in
@[simp] theorem inner_hsumIncl_hsumIncl (i j : Fin 2) (x y : α) :
  inner ℂ (hsumIncl α i x) (hsumIncl α j y) = if i = j then inner ℂ x y else 0 := by
  fin_cases i <=> fin_cases j <=> simp [hsumIncl, hsumEquiv, PiLp.inner_apply]

omit [Nontrivial α] in
@[simp] theorem hsumIncl_adjoint (i : Fin 2) :
  (hsumIncl α i).adjoint = hsumProj α i := by
  fin_cases i
  · ext x
    refine ext_inner_right ℂ fun y => ?_
    rw [ContinuousLinearMap.adjoint_inner_left]
    simp [hsumProj, hsumIncl, hsumEquiv, PiLp.inner_apply]
  · ext x
    refine ext_inner_right ℂ fun y => ?_
    rw [ContinuousLinearMap.adjoint_inner_left]
    simp [hsumProj, hsumIncl, hsumEquiv, PiLp.inner_apply]

omit [Nontrivial α] in
@[simp] theorem hsumProj_adjoint (i : Fin 2) :
  (hsumProj α i).adjoint = hsumIncl α i := by
  calc
    (hsumProj α i).adjoint = ((hsumIncl α i).adjoint).adjoint := by
      rw [hsumIncl_adjoint (α := α) i]
    _ = hsumIncl α i := ContinuousLinearMap.adjoint_adjoint _

omit [CompleteSpace α] [Nontrivial α] in
@[simp] theorem hsumIncl_proj_sum (z : HSum α) :
  hsumIncl α 0 (hsumProj α 0 z) + hsumIncl α 1 (hsumProj α 1 z) = z := by
  ext i
  fin_cases i <=> simp [hsumProj, hsumIncl, hsumEquiv]

/-- The block diagonal operator `diag(A, B)` on the two-fold Hilbert sum. -
/

```

1.115.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/BLOCKDIAGONAL.LEAN

```
noncomputable def blockDiagonal (A B : L  $\square$ ) : L (HSum  $\square$ ) :=
  hsumIncl  $\square$  0  $\circ$  L A  $\circ$  L hsumProj  $\square$  0 + hsumIncl  $\square$  1  $\circ$  L B  $\circ$  L hsumProj  $\square$  1
```

```
/-- A general `2 x 2` block operator on `HSum  $\square$ ` . -/
noncomputable def blockOp (A00 A01 A10 A11 : L  $\square$ ) : L (HSum  $\square$ ) :=
  hsumIncl  $\square$  0  $\circ$  L A00  $\circ$  L hsumProj  $\square$  0 +
  hsumIncl  $\square$  0  $\circ$  L A01  $\circ$  L hsumProj  $\square$  1 +
  hsumIncl  $\square$  1  $\circ$  L A10  $\circ$  L hsumProj  $\square$  0 +
  hsumIncl  $\square$  1  $\circ$  L A11  $\circ$  L hsumProj  $\square$  1
```

```
omit [CompleteSpace  $\square$ ] [Nontrivial  $\square$ ] in
theorem blockOp_ext {T S : L (HSum  $\square$ )}
  (h0 :  $\forall$  z : HSum  $\square$ , hsumProj  $\square$  0 (T z) = hsumProj  $\square$  0 (S z))
  (h1 :  $\forall$  z : HSum  $\square$ , hsumProj  $\square$  1 (T z) = hsumProj  $\square$  1 (S z)) :
  T = S := by
  ext z i
  fin_cases i
  · simp using h0 z
  · simp using h1 z
```

```
omit [Nontrivial  $\square$ ] in
@[simp] theorem blockDiagonal_star (A B : L  $\square$ ) :
  star (blockDiagonal ( $\square$  :=  $\square$ ) A B) = blockDiagonal ( $\square$  :=  $\square$ ) (star A) (star B) :=
  ext z i
  fin_cases i
  · simp [blockDiagonal, ContinuousLinearMap.star_eq_adjoint, ContinuousLinearMap.ad
  · simp [blockDiagonal, ContinuousLinearMap.star_eq_adjoint, ContinuousLinearMap.ad
```

```
noncomputable def blockDiagonalHom : (L  $\square$   $\times$  L  $\square$ )  $\rightarrow^{*a}[\mathbb{R}]$  L (HSum  $\square$ ) where
  toFun p := blockDiagonal ( $\square$  :=  $\square$ ) p.1 p.2
  map_one' := by
    ext z
    simp [blockDiagonal]
  map_mul' := by
    intro p q
    ext z i
    fin_cases i <=>
      simp [blockDiagonal, hsumProj, hsumIncl, hsumEquiv, ContinuousLinearMap.mul_de
  map_zero' := by
    ext z i
    fin_cases i <=> simp [blockDiagonal]
  map_add' := by
    intro p q
    ext z i
    fin_cases i <=>
      simp [blockDiagonal, hsumProj, hsumIncl, hsumEquiv, ContinuousLinearMap.add_ap
```

```

commutes' := by
  intro r
  ext z i
  fin_cases i <=> {
    simp [blockDiagonal, hsumProj, hsumIncl, hsumEquiv, Algebra.algebraMa
    rfl
  }
map_star' := by
  intro p
  simp

omit [Nontrivial []] in
@[simp] theorem blockDiagonalHom_apply (p : L [] × L []) :
  blockDiagonalHom ([] := []) p = blockDiagonal ([] := []) p.1 p.2 :=
  rfl

omit [CompleteSpace []] [Nontrivial []] in
@[simp] theorem hsumProj_blockDiagonal_zero (A B : L []) (z : HSum []) :
  hsumProj [] 0 (blockDiagonal A B z) = A (hsumProj [] 0 z) := by
  simp [blockDiagonal]

omit [CompleteSpace []] [Nontrivial []] in
@[simp] theorem hsumProj_blockDiagonal_one (A B : L []) (z : HSum []) :
  hsumProj [] 1 (blockDiagonal A B z) = B (hsumProj [] 1 z) := by
  simp [blockDiagonal]

omit [CompleteSpace []] [Nontrivial []] in
@[simp] theorem blockDiagonal_one :
  blockDiagonal ([] := []) (1 : L []) (1 : L []) = (1 : L (HSum [])) := by
  ext z i
  fin_cases i <=> simp [blockDiagonal]

omit [CompleteSpace []] [Nontrivial []] in
theorem blockDiagonal_nonneg {A B : L []} (hA : 0 ≤ A) (hB : 0 ≤ B) :
  0 ≤ blockDiagonal ([] := []) A B := by
  have hApos : A.IsPositive := (ContinuousLinearMap.nonneg_iff_isPositive A
  have hBpos : B.IsPositive := (ContinuousLinearMap.nonneg_iff_isPositive B
  refine (ContinuousLinearMap.nonneg_iff_isPositive _).mpr ?_
  rw [ContinuousLinearMap.isPositive_iff_complex]
  intro z
  have hAz := (ContinuousLinearMap.isPositive_iff_complex A).mp hApos (hsum
  have hBz := (ContinuousLinearMap.isPositive_iff_complex B).mp hBpos (hsum
  have hz0 :
    inner ℂ ((hsumIncl [] 0) (A (hsumProj [] 0 z))) z =
    inner ℂ (A (hsumProj [] 0 z)) (hsumProj [] 0 z) := by
  simp [hsumProj, hsumIncl, hsumEquiv, PiLp.inner_apply]

```

1.115.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/BLOCKDIAGONAL.LEAN

```

have hz1 :
  inner ℂ ((hsumIncl 1) (B (hsumProj 1 z))) z =
    inner ℂ (B (hsumProj 1 z)) (hsumProj 1 z) := by
      simp [hsumProj, hsumIncl, hsumEquiv, PiLp.inner_apply]
constructor
· dsimp [blockDiagonal]
  rw [inner_add_left, hz0, hz1]
  calc
    ↑(RCLike.re
      (inner ℂ (A (hsumProj 0 z)) (hsumProj 0 z) +
        inner ℂ (B (hsumProj 1 z)) (hsumProj 1 z))) =
      ↑(RCLike.re (inner ℂ (A (hsumProj 0 z)) (hsumProj 0 z)) +
        RCLike.re (inner ℂ (B (hsumProj 1 z)) (hsumProj 1 z))) := by
        simp
    _ = inner ℂ (A (hsumProj 0 z)) (hsumProj 0 z) +
      inner ℂ (B (hsumProj 1 z)) (hsumProj 1 z) := by
        have hsumre :
          (↑(RCLike.re (inner ℂ (A (hsumProj 0 z)) (hsumProj 0 z)) +
            RCLike.re (inner ℂ (B (hsumProj 1 z)) (hsumProj 1 z))) : ℂ) =
            ((RCLike.re (inner ℂ (A (hsumProj 0 z)) (hsumProj 0 z)) : ℂ) +
              (RCLike.re (inner ℂ (B (hsumProj 1 z)) (hsumProj 1 z)) : ℂ))
          simp
        rw [hsumre, hAz.1, hBz.1]
· dsimp [blockDiagonal]
  rw [inner_add_left, hz0, hz1]
  exact add_nonneg hAz.2 hBz.2

omit [CompleteSpace []] [Nontrivial []] in
@[simp] theorem hsumProj_blockOp_zero (A00 A01 A10 A11 : L []) (z : HSum []) :
  hsumProj 0 (blockOp (:=) A00 A01 A10 A11 z) =
  A00 (hsumProj 0 z) + A01 (hsumProj 1 z) := by
  simp [blockOp]

omit [CompleteSpace []] [Nontrivial []] in
@[simp] theorem hsumProj_blockOp_one (A00 A01 A10 A11 : L []) (z : HSum []) :
  hsumProj 1 (blockOp (:=) A00 A01 A10 A11 z) =
  A10 (hsumProj 0 z) + A11 (hsumProj 1 z) := by
  simp [blockOp]

omit [Nontrivial []] in
@[simp] theorem blockOp_star (A00 A01 A10 A11 : L []) :
  star (blockOp (:=) A00 A01 A10 A11) =
  blockOp (:=) (star A00) (star A10) (star A01) (star A11) := by
  ext z i
  fin_cases i
  · simp [blockOp, ContinuousLinearMap.star_eq_adjoint, ContinuousLinearMap.adjoint_

```

```

    abel
    · simp [blockOp, ContinuousLinearMap.star_eq_adjoint, ContinuousLinearMap
    abel
end JensenOperatorInequality

```

1.116 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/GeneralizedPerspectiveFunction

```

/-
Copyright (c) 2026 Hayata Yamasaki. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Kei Tsukamoto, Kento Mori, Hayata Yamasaki
-/
module

public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.JensenOperatorInequality
public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.LownerHeinzTheorem

@[expose] public section

namespace GeneralizedPerspectiveFunction

universe u

open LownerHeinzTheorem
open JensenOperatorInequality

section Convexity

variable {E F G : Type*}
variable [AddCommMonoid E] [Module ℝ E]
variable [AddCommMonoid F] [Module ℝ F]
variable [Preorder G] [AddCommMonoid G] [Module ℝ G]

/-- Joint convexity of a two-variable map on prescribed domains in each argument. -/
def JointlyConvexOn (s : Set E) (t : Set F) (Φ : E → F → G) : Prop :=
  ∀ A₁ A₂ : E B₁ B₂ : F θ : ℝ,
    A₁ ∈ s → A₂ ∈ s → B₁ ∈ t → B₂ ∈ t →
    0 ≤ θ → θ ≤ 1 →
    Φ ((1 - θ) • A₁ + θ • A₂) ((1 - θ) • B₁ + θ • B₂)
      ≤ (1 - θ) • Φ A₁ B₁ + θ • Φ A₂ B₂

/-- Joint convexity of a two-variable map without domain restrictions. -

```


1.116.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/GENERALIZEDPERSPECTIVEFUNCTION.LE

```

/
def JointlyConvex (Φ : E → F → G) : Prop :=
  JointlyConvexOn (Set.univ : Set E) (Set.univ : Set F) Φ

/-- Joint concavity of a two-variable map on prescribed domains in each argument. -
/
def JointlyConcaveOn (s : Set E) (t : Set F) (Φ : E → F → G) : Prop :=
  ∀ A₁ A₂ : E, B₁ B₂ : F, θ : ℝ,
    A₁ ∈ s → A₂ ∈ s → B₁ ∈ t → B₂ ∈ t →
    0 ≤ θ → θ ≤ 1 →
    (1 - θ) • Φ A₁ B₁ + θ • Φ A₂ B₂
    ≤ Φ ((1 - θ) • A₁ + θ • A₂) ((1 - θ) • B₁ + θ • B₂)

/-- Joint concavity of a two-variable map without domain restrictions. -
/
def JointlyConcave (Φ : E → F → G) : Prop :=
  JointlyConcaveOn (Set.univ : Set E) (Set.univ : Set F) Φ

end Convexity

section Definition

variable {α : Type u}
variable [NormedAddCommGroup α] [InnerProductSpace ℂ α] [CompleteSpace α]
variable [Nontrivial α]

/-- The operator `h(B)^(1/2)` defined by real continuous functional calculus. -
/
noncomputable def hSqrt (h : ℝ → ℝ) (B : L α) : L α :=
  cfcR (fun x : ℝ ↦ (h x) ^ ((1 : ℝ) / 2)) B

/-- The operator `h(B)^(-1/2)` defined by real continuous functional calculus. -
/
noncomputable def hInvSqrt (h : ℝ → ℝ) (B : L α) : L α :=
  cfcR (fun x : ℝ ↦ (h x) ^ ((-1 : ℝ) / 2)) B

/--
The generalized perspective function
`(fΔh)(A, B) = h(B)^(1/2) f(h(B)^(-1/2) A h(B)^(-1/2)) h(B)^(1/2)`.

This definition is intended to be used when `A` is Hermitian and `h(B)` is positive/
-/
noncomputable def GeneralizedPerspective (f h : ℝ → ℝ) (A B : L α) : L α :=
  hSqrt h B * cfcR f (hInvSqrt h B * A * hInvSqrt h B) * hSqrt h B

/-- Infix notation for generalized perspective: `(f Δ h) A B = GeneralizedPerspective

```

```

/
scoped infixl:70 " Δ " => GeneralizedPerspective

end Definition

section Theorem25Forward

variable {α : Type u}
variable [NormedAddCommGroup α] [InnerProductSpace ℂ α] [CompleteSpace α]
variable [Nontrivial α]

/-- Positive semidefinite operators. -/
def psdSet : Set (L α) :=
  {A | IsSelfAdjoint A ∧ spectrum ℝ A ⊆ Set.Ici (0 : ℝ)}

/-- Strictly positive operators. -/
def pdSet : Set (L α) :=
  {A | IsSelfAdjoint A ∧ spectrum ℝ A ⊆ Set.Ioi (0 : ℝ)}

private lemma spectrum_convexCombo_Ioi {A B : L α} {t : ℝ}
  (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B) (ht0 : 0 ≤ t) (ht1 : t ≤ 1)
  (As : spectrum ℝ A ⊆ Set.Ioi (0 : ℝ)) (Bs : spectrum ℝ B ⊆ Set.Ioi (0 : ℝ)) :
  spectrum ℝ ((1 - t) • A + t • B) ⊆ Set.Ioi (0 : ℝ) := by
  set C : L α := (1 - t) • A + t • B
  have hC : IsSelfAdjoint C := by
    simpa [C] using (IsSelfAdjoint.all (1 - t)).smul hA |>.add ((IsSelfAdjoint.all t).smul hB)
  have hApos : ∃ r > 0, algebraMap ℝ (L α) r ≤ A := by
    refine (CFC.exists_pos_algebraMap_le_iff (A := L α) (a := A) (ha := hA))
    intro x hx
    exact As hx
  have hBpos : ∃ r > 0, algebraMap ℝ (L α) r ≤ B := by
    refine (CFC.exists_pos_algebraMap_le_iff (A := L α) (a := B) (ha := hB))
    intro x hx
    exact Bs hx
  rcases hApos with ⟨rA, hrA, hrA_le⟩
  rcases hBpos with ⟨rB, hrB, hrB_le⟩
  set rC : ℝ := (1 - t) * rA + t * rB
  have hrC : 0 < rC := by
    by_cases h1t : (1 - t) = 0
    · have ht' : t = 1 := by linarith
      subst ht'
      simpa [rC] using hrB
    · have h1t_pos : 0 < 1 - t := lt_of_le_of_ne (sub_nonneg.mpr ht1) (Ne.symm h1t)
      simpa [rC] using
        add_pos_of_pos_of_nonneg (mul_pos h1t_pos hrA) (mul_nonneg ht0 (le_of_lt h1t_pos))
  have hrC_le : algebraMap ℝ (L α) rC ≤ C := by

```

1.116.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/GENERALIZEDPERSPECTIVEFUNCTION.LE

```

have hsum :
  (1 - t) • algebraMap ℝ (L []) rA + t • algebraMap ℝ (L []) rB ≤ C := by
  simpa [C] using
    add_le_add (smul_le_smul_of_nonneg_left hrA_le (sub_nonneg.mpr ht1))
      (smul_le_smul_of_nonneg_left hrB_le ht0)
have hLHS :
  (1 - t) • algebraMap ℝ (L []) rA + t • algebraMap ℝ (L []) rB =
    algebraMap ℝ (L []) rC := by
  simp [rC, Algebra.smul_def]
  simpa [hLHS] using hsum
intro x hx
simpa [C] using
  (CFC.exists_pos_algebraMap_le_iff (A := L []) (a := C) (ha := hC)).1 {rC, hrC, hrA}

omit [Nontrivial []] in
private lemma cfcR_sq_eq {g k : ℝ → ℝ} {A : L []}
  (hA : IsSelfAdjoint A)
  (hg : ContinuousOn g (spectrum ℝ A))
  (hk : ContinuousOn k (spectrum ℝ A))
  (hmul : ∀ x ∈ spectrum ℝ A, g x * k x = 1) :
  cfcR ([] := []) g A * cfcR ([] := []) k A = (1 : L []) := by
  rw [← cfc_mul (R := ℝ) (A := L []) (p := IsSelfAdjoint)
    (f := g) (g := k) (a := A) hg hk, ← cfc_const_one ℝ A]
  apply cfc_congr
  intro x hx
  simpa using hmul x hx

omit [Nontrivial []] in
private lemma cfcR_mul_eq {g k m : ℝ → ℝ} {A : L []}
  (hg : ContinuousOn g (spectrum ℝ A))
  (hk : ContinuousOn k (spectrum ℝ A))
  (hmul : ∀ x ∈ spectrum ℝ A, g x * k x = m x) :
  cfcR ([] := []) g A * cfcR ([] := []) k A = cfcR ([] := []) m A := by
  rw [← cfc_mul (R := ℝ) (A := L []) (p := IsSelfAdjoint)
    (f := g) (g := k) (a := A) hg hk]
  apply cfc_congr
  intro x hx
  simpa using hmul x hx

private lemma hpow_continuousOn
  (h : ℝ → ℝ) (p : ℝ)
  (hcont : ContinuousOn h (Set.Ioi (0 : ℝ)))
  (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
  ContinuousOn (fun x : ℝ ↦ (h x) ^ p) (Set.Ioi (0 : ℝ)) := by
  intro x hx
  have hg : ContinuousWithinAt (fun y : ℝ ↦ y ^ p) (Set.Ioi (0 : ℝ)) (h x) :=

```

```

      (Real.continuousAt_rpow_const (h x) p (Or.inl (ne_of_gt (hpos x hx))))).
    exact hg.comp (hcont x hx) (by
      intro y hy
      exact hpos y hy)

omit [Nontrivial  $\square$ ] in
private lemma hSqrt_selfAdjoint (h :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (B : L  $\square$ ) :
  IsSelfAdjoint (hSqrt ( $\square := \square$ ) h B) := by
  dsimp [hSqrt, cfcR]
  exact cfc_predicate _ _

omit [Nontrivial  $\square$ ] in
private lemma hInvSqrt_selfAdjoint (h :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (B : L  $\square$ ) :
  IsSelfAdjoint (hInvSqrt ( $\square := \square$ ) h B) := by
  dsimp [hInvSqrt, cfcR]
  exact cfc_predicate _ _

omit [Nontrivial  $\square$ ] in
private lemma hSqrt_mul_hInvSqrt_eq_one
  {h :  $\mathbb{R} \rightarrow \mathbb{R}$ } {B : L  $\square$ }
  (hB : IsSelfAdjoint B) (Bs : spectrum  $\mathbb{R}$  B  $\subseteq$  Set.Ioi (0 :  $\mathbb{R}$ ))
  (hcont : ContinuousOn h (Set.Ioi (0 :  $\mathbb{R}$ )))
  (hpos :  $\forall x \in \text{Set.Ioi } (0 : \mathbb{R}), 0 < h x$ ) :
  hSqrt ( $\square := \square$ ) h B * hInvSqrt ( $\square := \square$ ) h B = (1 : L  $\square$ ) := by
  have hsqrt :
    ContinuousOn (fun x :  $\mathbb{R} \mapsto (h x) ^ ((1 : \mathbb{R}) / 2)$ ) (spectrum  $\mathbb{R}$  B) :=
      (hpow_continuousOn h ((1 :  $\mathbb{R}) / 2$ ) hcont hpos).mono (by intro x hx; exa
  have hinv :
    ContinuousOn (fun x :  $\mathbb{R} \mapsto (h x) ^ ((-1 : \mathbb{R}) / 2)$ ) (spectrum  $\mathbb{R}$  B) :=
      (hpow_continuousOn h ((-1 :  $\mathbb{R}) / 2$ ) hcont hpos).mono (by intro x hx; ex
  have hmul :
     $\forall x \in \text{spectrum } \mathbb{R} B,$ 
     $(h x) ^ ((1 : \mathbb{R}) / 2) * (h x) ^ ((-1 : \mathbb{R}) / 2) = 1 := by$ 
    intro x hx
    have hxpos :  $0 < h x := hpos x (Bs hx)$ 
    rw [← Real.rpow_add hxpos]
    norm_num
  simp [hSqrt, hInvSqrt] using cfcR_sq_eq ( $\square := \square$ ) (A := B) hB hsqrt hinv

omit [Nontrivial  $\square$ ] in
private lemma hInvSqrt_mul_hSqrt_eq_one
  {h :  $\mathbb{R} \rightarrow \mathbb{R}$ } {B : L  $\square$ }
  (hB : IsSelfAdjoint B) (Bs : spectrum  $\mathbb{R}$  B  $\subseteq$  Set.Ioi (0 :  $\mathbb{R}$ ))
  (hcont : ContinuousOn h (Set.Ioi (0 :  $\mathbb{R}$ )))
  (hpos :  $\forall x \in \text{Set.Ioi } (0 : \mathbb{R}), 0 < h x$ ) :
  hInvSqrt ( $\square := \square$ ) h B * hSqrt ( $\square := \square$ ) h B = (1 : L  $\square$ ) := by

```

1.116.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/GENERALIZEDPERSPECTIVEFUNCTION.LE

```

have hsqrt :
  ContinuousOn (fun x : ℝ ↦ (h x) ^ ((1 : ℝ) / 2)) (spectrum ℝ B) :=
    (hpow_continuousOn h ((1 : ℝ) / 2) hcont hpos).mono (by intro x hx; exact Bs hx)
have hinv :
  ContinuousOn (fun x : ℝ ↦ (h x) ^ ((-1 : ℝ) / 2)) (spectrum ℝ B) :=
    (hpow_continuousOn h ((-1 : ℝ) / 2) hcont hpos).mono (by intro x hx; exact Bs hx)
have hmul :
  ∀ x ∈ spectrum ℝ B,
    (h x) ^ ((-1 : ℝ) / 2) * (h x) ^ ((1 : ℝ) / 2) = 1 := by
    intro x hx
    have hxpos : 0 < h x := hpos x (Bs hx)
    rw [← Real.rpow_add hxpos]
    norm_num
simpa [hSqrt, hInvSqrt] using cfcR_sq_eq (□ := □) (A := B) hB hinv hsqrt hmul

omit [Nontrivial □] in
private lemma hSqrt_mul_hSqrt_eq
  {h : ℝ → ℝ} {B : L □}
  (Bs : spectrum ℝ B ⊆ Set.Ioi (0 : ℝ))
  (hcont : ContinuousOn h (Set.Ioi (0 : ℝ)))
  (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
  hSqrt (□ := □) h B * hSqrt (□ := □) h B = cfcR (□ := □) h B := by
  have hsqrt :
    ContinuousOn (fun x : ℝ ↦ (h x) ^ ((1 : ℝ) / 2)) (spectrum ℝ B) :=
      (hpow_continuousOn h ((1 : ℝ) / 2) hcont hpos).mono (by intro x hx; exact Bs hx)
  have hmul :
    ∀ x ∈ spectrum ℝ B,
      (h x) ^ ((1 : ℝ) / 2) * (h x) ^ ((1 : ℝ) / 2) = h x := by
      intro x hx
      have hxpos : 0 < h x := hpos x (Bs hx)
      rw [← Real.rpow_add hxpos]
      norm_num
  simpa [hSqrt] using cfcR_mul_eq (□ := □) (A := B) hsqrt hsqrt hmul

omit [Nontrivial □] in
private lemma conj_le_conj {X Y T : L □} (hXY : X ≤ Y) (hT : IsSelfAdjoint T) :
  T * X * T ≤ T * Y * T := by
  have hnonneg : 0 ≤ Y - X := sub_nonneg.mpr hXY
  have hconj : 0 ≤ T * (Y - X) * T := by
    simpa using hT.conjugate_nonneg hnonneg
  have hsub : T * (Y - X) * T = T * Y * T - T * X * T := by
    simp [sub_eq_add_neg, mul_add, add_mul, mul_assoc]
    exact sub_nonneg.mp (by simpa [hsub] using hconj)

set_option maxHeartbeats 800000 in
-- The generalized-perspective normalization expands several nested CFC products.

```

```

private theorem theorem_2_5_forward_jointlyConvex0n_psd_pd_of_condV
  {f h :  $\mathbb{R} \rightarrow \mathbb{R}$ }
  (hcoreV : CondV ( $\square := \square$ ) f)
  (hconc : OperatorConcave0n ( $\square := \square$ ) (Set.Ioi ( $\theta : \mathbb{R}$ )) h)
  (hcont : Continuous0n h (Set.Ioi ( $\theta : \mathbb{R}$ )))
  (hpos :  $\forall x \in \text{Set.Ioi } (\theta : \mathbb{R}), \theta < h x$ ) :
  JointlyConvex0n (psdSet ( $\square := \square$ )) (pdSet ( $\square := \square$ )) (fun A B  $\mapsto$  (f  $\Delta$  h) A
intro A1 A2 B1 B2  $\theta$  hA1 hA2 hB1 hB2 h $\theta\theta$  h $\theta 1$ 
rcases hA1 with {hA1_sa, hA1_spec}
rcases hA2 with {hA2_sa, hA2_spec}
rcases hB1 with {hB1_sa, hB1_spec}
rcases hB2 with {hB2_sa, hB2_spec}
let A : L  $\square :=$  (1 -  $\theta$ ) • A1 +  $\theta$  • A2
let B : L  $\square :=$  (1 -  $\theta$ ) • B1 +  $\theta$  • B2
have hA1_nonneg : ( $\theta : \text{L } \square$ )  $\leq$  A1 := by
  refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R :=  $\mathbb{R}$ ) A1 (ha := h
intro x hx
  simp [Set.Ici] using hA1_spec hx
have hA2_nonneg : ( $\theta : \text{L } \square$ )  $\leq$  A2 := by
  refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R :=  $\mathbb{R}$ ) A2 (ha := h
intro x hx
  simp [Set.Ici] using hA2_spec hx
have hB_sa : IsSelfAdjoint B := by
  dsimp [B]
  simp using (IsSelfAdjoint.all (1 -  $\theta$ )).smul hB1_sa |>.add ((IsSelfAdjo
have hB_spec : spectrum  $\mathbb{R}$  B  $\subseteq$  Set.Ioi ( $\theta : \mathbb{R}$ ) :=
  spectrum_convexCombo_Ioi ( $\square := \square$ ) hB1_sa hB2_sa h $\theta\theta$  h $\theta 1$  hB1_spec hB2_sp
have hB_conc :
  (1 -  $\theta$ ) • cfcR ( $\square := \square$ ) h B1 +  $\theta$  • cfcR ( $\square := \square$ ) h B2  $\leq$  cfcR ( $\square := \square$ )
have hconc' := hconc
dsimp [OperatorConcave0n, OperatorConvex0n] at hconc'
have hneg :
  cfcR ( $\square := \square$ ) (fun x :  $\mathbb{R} \mapsto$  -h x) B  $\leq$ 
  (1 -  $\theta$ ) • cfcR ( $\square := \square$ ) (fun x :  $\mathbb{R} \mapsto$  -h x) B1 +
   $\theta$  • cfcR ( $\square := \square$ ) (fun x :  $\mathbb{R} \mapsto$  -h x) B2 := by
  simp [B] using
    hconc' (A := B1) (B := B2) (t :=  $\theta$ ) hB1_sa hB2_sa h $\theta\theta$  h $\theta 1$  hB1_spec
  simp [cfcR, cfc_neg, smul_neg, neg_add, add_comm, add_left_comm, add_a
    neg_le_neg hneg
let S : L  $\square :=$  hSqrt ( $\square := \square$ ) h B
let IR : L  $\square :=$  hInvSqrt ( $\square := \square$ ) h B
let S1 : L  $\square :=$  hSqrt ( $\square := \square$ ) h B1
let S2 : L  $\square :=$  hSqrt ( $\square := \square$ ) h B2
let IR1 : L  $\square :=$  hInvSqrt ( $\square := \square$ ) h B1
let IR2 : L  $\square :=$  hInvSqrt ( $\square := \square$ ) h B2
let T1 : L  $\square :=$  Real.sqrt (1 -  $\theta$ ) • (S1 * IR)

```

```

let T2 : L  $\square$  := Real.sqrt  $\theta$  • (S2 * IR)
let M1 : L  $\square$  := IR1 * A1 * IR1
let M2 : L  $\square$  := IR2 * A2 * IR2
have hS_sa : IsSelfAdjoint S := hSqrt_selfAdjoint ( $\square$  :=  $\square$ ) h B
have hIR_sa : IsSelfAdjoint IR := hInvSqrt_selfAdjoint ( $\square$  :=  $\square$ ) h B
have hS1_sa : IsSelfAdjoint S1 := hSqrt_selfAdjoint ( $\square$  :=  $\square$ ) h B1
have hS2_sa : IsSelfAdjoint S2 := hSqrt_selfAdjoint ( $\square$  :=  $\square$ ) h B2
have hIR1_sa : IsSelfAdjoint IR1 := hInvSqrt_selfAdjoint ( $\square$  :=  $\square$ ) h B1
have hIR2_sa : IsSelfAdjoint IR2 := hInvSqrt_selfAdjoint ( $\square$  :=  $\square$ ) h B2
have hSIR : S * IR = (1 : L  $\square$ ) :=
  hSqrt_mul_hInvSqrt_eq_one ( $\square$  :=  $\square$ ) hB_sa hB_spec hcont hpos
have hIRS : IR * S = (1 : L  $\square$ ) :=
  hInvSqrt_mul_hSqrt_eq_one ( $\square$  :=  $\square$ ) hB_sa hB_spec hcont hpos
have hS1IR1 : S1 * IR1 = (1 : L  $\square$ ) :=
  hSqrt_mul_hInvSqrt_eq_one ( $\square$  :=  $\square$ ) hB1_sa hB1_spec hcont hpos
have hIR1S1 : IR1 * S1 = (1 : L  $\square$ ) :=
  hInvSqrt_mul_hSqrt_eq_one ( $\square$  :=  $\square$ ) hB1_sa hB1_spec hcont hpos
have hS2IR2 : S2 * IR2 = (1 : L  $\square$ ) :=
  hSqrt_mul_hInvSqrt_eq_one ( $\square$  :=  $\square$ ) hB2_sa hB2_spec hcont hpos
have hIR2S2 : IR2 * S2 = (1 : L  $\square$ ) :=
  hInvSqrt_mul_hSqrt_eq_one ( $\square$  :=  $\square$ ) hB2_sa hB2_spec hcont hpos
have hM1_nonneg : (0 : L  $\square$ ) ≤ M1 := by
  dsimp [M1]
  simpa [mul_assoc] using hIR1_sa.conjugate_nonneg hA1_nonneg
have hM2_nonneg : (0 : L  $\square$ ) ≤ M2 := by
  dsimp [M2]
  simpa [mul_assoc] using hIR2_sa.conjugate_nonneg hA2_nonneg
have hM1_sa : IsSelfAdjoint M1 := IsSelfAdjoint.of_nonneg hM1_nonneg
have hM2_sa : IsSelfAdjoint M2 := IsSelfAdjoint.of_nonneg hM2_nonneg
have hM1_spec : spectrum  $\mathbb{R}$  M1 ⊆ Set.Ici (0 :  $\mathbb{R}$ ) := by
  intro x hx
  simpa [Set.Ici] using spectrum_nonneg_of_nonneg hM1_nonneg hx
have hM2_spec : spectrum  $\mathbb{R}$  M2 ⊆ Set.Ici (0 :  $\mathbb{R}$ ) := by
  intro x hx
  simpa [Set.Ici] using spectrum_nonneg_of_nonneg hM2_nonneg hx
have hT1 :
  star T1 * T1 = (1 -  $\theta$ ) • (IR * cfcR ( $\square$  :=  $\square$ ) h B1 * IR) := by
  calc
  star T1 * T1
    = (Real.sqrt (1 -  $\theta$ ) * Real.sqrt (1 -  $\theta$ )) • (IR * (S1 * S1) * IR) := by
      simp [T1, hS1_sa.star_eq, hIR_sa.star_eq, mul_assoc, smul_smul]
  _ = (1 -  $\theta$ ) • (IR * (S1 * S1) * IR) := by
      rw [Real.mul_self_sqrt (sub_nonneg.mpr h01)]
  _ = (1 -  $\theta$ ) • (IR * cfcR ( $\square$  :=  $\square$ ) h B1 * IR) := by
      rw [hSqrt_mul_hSqrt_eq ( $\square$  :=  $\square$ ) (B := B1) hB1_spec hcont hpos]
have hT2 :

```

```

    star T2 * T2 = θ • (IR * cfcR (□ := □) h B2 * IR) := by
  calc
    star T2 * T2
      = (Real.sqrt θ * Real.sqrt θ) • (IR * (S2 * S2) * IR) := by
        simp [T2, hS2_sa.star_eq, hIR_sa.star_eq, mul_assoc, smul_smu
      _ = θ • (IR * (S2 * S2) * IR) := by
        rw [Real.mul_self_sqrt hθ0]
      _ = θ • (IR * cfcR (□ := □) h B2 * IR) := by
        rw [hSqrt_mul_hSqrt_eq (□ := □) (B := B2) hB2_spec hcont hpos]
  have hTsum : star T1 * T1 + star T2 * T2 ≤ (1 : L □) := by
    rw [hT1, hT2]
  have hmid :
    (1 - θ) • (IR * cfcR (□ := □) h B1 * IR) + θ • (IR * cfcR (□ := □)
      ≤ IR * cfcR (□ := □) h B * IR := by
    calc
      (1 - θ) • (IR * cfcR (□ := □) h B1 * IR) + θ • (IR * cfcR (□ := □)
        = IR * ((1 - θ) • cfcR (□ := □) h B1 + θ • cfcR (□ := □) h B2)
          simp [mul_add, add_mul, mul_assoc]
      _ ≤ IR * cfcR (□ := □) h B * IR := conj_le_conj (□ := □) hB_conc hI
  have hunit : IR * cfcR (□ := □) h B * IR = (1 : L □) := by
    calc
      IR * cfcR (□ := □) h B * IR = IR * (S * S) * IR := by
        rw [hSqrt_mul_hSqrt_eq (□ := □) (B := B) hB_spec hcont hpos]
      _ = (IR * S) * (S * IR) := by simp [mul_assoc]
      _ = 1 := by simp [hIRS, hSIR]
  exact hmid.trans_eq hunit
  have hterm1 :
    star T1 * M1 * T1 = (1 - θ) • (IR * A1 * IR) := by
  calc
    star T1 * M1 * T1
      = (Real.sqrt (1 - θ) * Real.sqrt (1 - θ)) •
        (IR * (S1 * (IR1 * A1 * IR1) * S1) * IR) := by
        simp [T1, M1, hS1_sa.star_eq, hIR_sa.star_eq, mul_assoc, sm
      _ = (1 - θ) • (IR * (S1 * (IR1 * A1 * IR1) * S1) * IR) := by
        rw [Real.mul_self_sqrt (sub_nonneg.mpr hθ1)]
      _ = (1 - θ) • (IR * A1 * IR) := by
        rw [show IR * (S1 * (IR1 * A1 * IR1) * S1) * IR =
          IR * (((S1 * IR1) * A1) * (IR1 * S1)) * IR by simp [mul_ass
        rw [hS1IR1, hIR1S1]
        simp [mul_assoc]
  have hterm2 :
    star T2 * M2 * T2 = θ • (IR * A2 * IR) := by
  calc
    star T2 * M2 * T2
      = (Real.sqrt θ * Real.sqrt θ) •
        (IR * (S2 * (IR2 * A2 * IR2) * S2) * IR) := by

```


1.116.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/GENERALIZEDPERSPECTIVEFUNCTION.LE

```

      simp [T2, M2, hS2_sa.star_eq, hIR_sa.star_eq, mul_assoc, smul_smul]
_ = θ • (IR * (S2 * (IR2 * A2 * IR2) * S2) * IR) := by
  rw [Real.mul_self_sqrt hθ0]
_ = θ • (IR * A2 * IR) := by
  rw [show IR * (S2 * (IR2 * A2 * IR2) * S2) * IR =
        IR * ((S2 * IR2) * A2) * (IR2 * S2) * IR by simp [mul_assoc]]
  rw [hS2IR2, hIR2S2]
  simp [mul_assoc]
have hleft_inner :
  star T1 * M1 * T1 + star T2 * M2 * T2 = IR * A * IR := by
  rw [hterm1, hterm2]
  simp [A, mul_add, add_mul, mul_assoc]
have hcore :=
  hcoreV (A := M1) (B := M2) (X := T1) (Y := T2) hM1_sa hM2_sa hM1_spec hM2_spec h
have houter := conj_le_conj (□ := □) hcore hS_sa
rw [hleft_inner] at houter
have hright1 :
  S * (star T1 * cfcR (□ := □) f M1 * T1) * S =
  (1 - θ) • ((f Δ h) A1 B1) := by
  calc
  S * (star T1 * cfcR (□ := □) f M1 * T1) * S
    = (Real.sqrt (1 - θ) * Real.sqrt (1 - θ)) •
      (S * IR * (S1 * cfcR (□ := □) f M1 * S1) * IR * S) := by
        simp [T1, hS1_sa.star_eq, hIR_sa.star_eq, mul_assoc, smul_smul]
_ = (1 - θ) • (S * IR * (S1 * cfcR (□ := □) f M1 * S1) * IR * S) := by
  rw [Real.mul_self_sqrt (sub_nonneg.mpr hθ1)]
_ = (1 - θ) • (S1 * cfcR (□ := □) f M1 * S1) := by
  simp [mul_assoc, hSIR, hIRS]
_ = (1 - θ) • ((f Δ h) A1 B1) := by
  rfl
have hright2 :
  S * (star T2 * cfcR (□ := □) f M2 * T2) * S =
  θ • ((f Δ h) A2 B2) := by
  calc
  S * (star T2 * cfcR (□ := □) f M2 * T2) * S
    = (Real.sqrt θ * Real.sqrt θ) •
      (S * IR * (S2 * cfcR (□ := □) f M2 * S2) * IR * S) := by
        simp [T2, hS2_sa.star_eq, hIR_sa.star_eq, mul_assoc, smul_smul]
_ = θ • (S * IR * (S2 * cfcR (□ := □) f M2 * S2) * IR * S) := by
  rw [Real.mul_self_sqrt hθ0]
_ = θ • (S2 * cfcR (□ := □) f M2 * S2) := by
  simp [mul_assoc, hSIR, hIRS]
_ = θ • ((f Δ h) A2 B2) := by
  rfl
have hright :
  S * (star T1 * cfcR (□ := □) f M1 * T1 + star T2 * cfcR (□ := □) f M2 * T2) *

```

```

      (1 -  $\theta$ ) • ((f  $\Delta$  h) A1 B1) +  $\theta$  • ((f  $\Delta$  h) A2 B2) := by
    rw [mul_add, add_mul, hright1, hright2]
  have hleft :
    S * cfcR ( $\square := \square$ ) f (IR * A * IR) * S = (f  $\Delta$  h) A B := by
    rfl
  simp [hleft, hright] using houter

-- Restricted forward form of Theorem 2.5 on the positive cone.
theorem theorem_2_5_forward_jointlyConvexOn_psd_pd
  {f h :  $\mathbb{R} \rightarrow \mathbb{R}$ }
  (hf : CondIAll.{u} f)
  (hconc : OperatorConcaveOn ( $\square := \square$ ) (Set.Ioi ( $\theta : \mathbb{R}$ )) h)
  (hcont : ContinuousOn h (Set.Ioi ( $\theta : \mathbb{R}$ )))
  (hpos :  $\forall x \in \text{Set.Ioi } (\theta : \mathbb{R}), \theta < h\ x$ ) :
  JointlyConvexOn (psdSet ( $\square := \square$ )) (pdSet ( $\square := \square$ )) (fun A B  $\mapsto$  (f  $\Delta$  h) A B)
  have hcoreV : CondV ( $\square := \square$ ) f :=
    theorem_2_5_2_i_all_imp_v ( $\square := \square$ ) (f := f) hf
  exact theorem_2_5_forward_jointlyConvexOn_psd_pd_of_condV
    ( $\square := \square$ ) (f := f) (h := h) hcoreV hconc hcont hpos

-- Restricted localized forward form of Theorem 2.5 on the positive cone.
theorem theorem_2_5_forward_jointlyConvexOn_psd_pd_Ici
  {f h :  $\mathbb{R} \rightarrow \mathbb{R}$ }
  (hf : CondIciAll.{u} f)
  (hconc : OperatorConcaveOn ( $\square := \square$ ) (Set.Ioi ( $\theta : \mathbb{R}$ )) h)
  (hcont : ContinuousOn h (Set.Ioi ( $\theta : \mathbb{R}$ )))
  (hpos :  $\forall x \in \text{Set.Ioi } (\theta : \mathbb{R}), \theta < h\ x$ ) :
  JointlyConvexOn (psdSet ( $\square := \square$ )) (pdSet ( $\square := \square$ )) (fun A B  $\mapsto$  (f  $\Delta$  h) A B)
  have hcoreV : CondV ( $\square := \square$ ) f :=
    theorem_2_5_2_i_ici_all_imp_v ( $\square := \square$ ) (f := f) hf
  exact theorem_2_5_forward_jointlyConvexOn_psd_pd_of_condV
    ( $\square := \square$ ) (f := f) (h := h) hcoreV hconc hcont hpos

omit [Nontrivial  $\square$ ] in
private lemma generalizedPerspective_neg
  (f h :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (A B : L  $\square$ ) :
  ((fun x :  $\mathbb{R} \mapsto -f\ x$ )  $\Delta$  h) A B = -((f  $\Delta$  h) A B) := by
  simp [GeneralizedPerspective, cfcR, cfc_neg, mul_assoc]

omit [Nontrivial  $\square$ ] in
private lemma jointlyConvexOn_neg
  {s : Set (L  $\square$ )} {t : Set (L  $\square$ )} { $\Phi$  : L  $\square \rightarrow$  L  $\square \rightarrow$  L  $\square$ }
  (h $\Phi$  : JointlyConvexOn s t (fun A B  $\mapsto$  - $\Phi$  A B)) :
  JointlyConcaveOn s t  $\Phi$  := by
  intro A1 A2 B1 B2  $\theta$  hA1 hA2 hB1 hB2 h $\theta$ 0 h $\theta$ 1
  have hneg := h $\Phi$  hA1 hA2 hB1 hB2 h $\theta$ 0 h $\theta$ 1

```

1.116.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/GENERALIZEDPERSPECTIVEFUNCTION.LE

```

have hneg' :
  -Φ ((1 - θ) • A1 + θ • A2) ((1 - θ) • B1 + θ • B2) ≤
  -((1 - θ) • Φ A1 B1 + θ • Φ A2 B2) := by
  simpa [smul_neg, neg_add, add_comm, add_left_comm, add_assoc] using hneg
exact (neg_le_neg_iff.mp hneg')

/-- Restricted forward form of Corollary 2.6 on the positive cone. -
/
theorem theorem_2_6_forward_jointlyConcave0n_psd_pd
  {f h : ℝ → ℝ}
  (hfconc : OperatorConcaveAll.{u} f)
  (hfθ : 0 ≤ f θ)
  (hconc : OperatorConcave0n (□ := □) (Set.Ioi (0 : ℝ)) h)
  (hcont : Continuous0n h (Set.Ioi (0 : ℝ)))
  (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
  JointlyConcave0n (psdSet (□ := □)) (pdSet (□ := □)) (fun A B ↦ (f Δ h) A B) := b
have hfneg : CondIAll.{u} (fun x : ℝ ↦ -f x) := by
  refine {?, ?}
  · intro K _ _ _
    simpa [OperatorConcave] using (hfconc (K := K))
  · simpa using (neg_nonpos.mpr hfθ)
have hconv_neg :
  JointlyConvex0n (psdSet (□ := □)) (pdSet (□ := □))
  (fun A B ↦ -((f Δ h) A B)) := by
  simpa [generalizedPerspective_neg] using
    (theorem_2_5_forward_jointlyConvex0n_psd_pd
      (□ := □) (f := fun x : ℝ ↦ -f x) (h := h) hfneg hconc hcont hpos)
exact jointlyConvex0n_neg (□ := □) hconv_neg

/-- Restricted localized forward form of Corollary 2.6 on the positive cone. -
/
theorem theorem_2_6_forward_jointlyConcave0n_psd_pd_Ici
  {f h : ℝ → ℝ}
  (hfconc : OperatorConcave0nAll.{u} (Set.Ici (0 : ℝ)) f)
  (hfcont : Continuous0n f (Set.Ici (0 : ℝ)))
  (hfθ : 0 ≤ f θ)
  (hconc : OperatorConcave0n (□ := □) (Set.Ioi (0 : ℝ)) h)
  (hcont : Continuous0n h (Set.Ioi (0 : ℝ)))
  (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
  JointlyConcave0n (psdSet (□ := □)) (pdSet (□ := □)) (fun A B ↦ (f Δ h) A B) := b
have hfneg : CondIciAll.{u} (fun x : ℝ ↦ -f x) := by
  refine {?, ?, ?}
  · intro K _ _ _
    simpa [OperatorConcave0n, OperatorConvex0n] using (hfconc (K := K))
  · simpa using hfcont.neg
  · simpa using (neg_nonpos.mpr hfθ)

```

```

have hconv_neg :
  JointlyConvexOn (psdSet (α := β)) (pdSet (α := β))
    (fun A B ↦ -((f Δ h) A B)) := by
  simpa [generalizedPerspective_neg] using
    (theorem_2_5_forward_jointlyConvexOn_psd_pd_Ici
      (α := β) (f := fun x : ℝ ↦ -f x) (h := h) hfneg hconc hcont hpos)
  exact jointlyConvexOn_neg (α := β) hconv_neg

end Theorem25Forward

-- GeneralizedPerspectiveFunction Page 1 https://www.pnas.org/doi/10.1073/p
-- For any qudit α, any A ∈ Herm(α), any B ∈ Pd(α),
-- any real-valued continuous function f, any positive-valued continuous fu
-- (fΔh)(A,B) := h(B)^{1/2} f(h(B)^{-1/2} A h(B)^{-1/2}) h(B)^{1/2}

-- Theorem 2.5 https://www.pnas.org/doi/10.1073/pnas.1102518108
-- Suppose that f is a continuous function with f(0) ≤ 0, and h is a contin
-- If f is operator convex and h is operator concave,
-- then fΔh is jointly convex

end GeneralizedPerspectiveFunction

-- Corollary 2.6 https://www.pnas.org/doi/10.1073/pnas.1102518108
-- Suppose that f is a continuous function with f(0) ≥ 0, and h is a contin
-- If f is operator concave and h is operator concave,
-- then fΔh is jointly concave

```

1.117 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/Hilber

```

/-
Copyright (c) 2026 Hayata Yamasaki. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Kei Tsukamoto, Kento Mori, Hayata Yamasaki
-/
module

public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.LownerHein
public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.Generalize

public import Mathlib.Analysis.InnerProductSpace.PiL2
public import Mathlib.Analysis.InnerProductSpace.Trace
public import Mathlib.Analysis.Normed.Lp.PiLp
public import Mathlib.LinearAlgebra.Complex.FiniteDimensional
public import Mathlib.LinearAlgebra.Matrix.ToLin

```

1.117.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/HILBERTSCHMIDTOPERATORSPACE.LEAN

```
public import Mathlib.Topology.Algebra.Module.FiniteDimension
```

```
@[expose] public section
```

```
namespace HilbertSchmidtOperatorSpace
```

```
open LownerHeinzTheorem
```

```
universe u
```

```
noncomputable section
```

```
variable {α : Type u}
```

```
variable [NormedAddCommGroup α] [InnerProductSpace ℂ α] [CompleteSpace α]
```

```
variable [FiniteDimensional ℂ α]
```

```
/-- The canonical finite index used for Hilbert-Schmidt coordinates. -
```

```
/
```

```
abbrev HSIndex (α : Type u) [NormedAddCommGroup α] [InnerProductSpace ℂ α]
```

```
  [FiniteDimensional ℂ α] :=
```

```
  Fin (Module.finiteRank ℂ α)
```

```
/-- The standard orthonormal basis on a finite-dimensional Hilbert space. -
```

```
/
```

```
noncomputable abbrev hsOrthonormalBasis :
```

```
  OrthonormalBasis (HSIndex α) ℂ α :=
```

```
  stdOrthonormalBasis ℂ α
```

```
/-- Coordinate space for Hilbert-Schmidt operators. -/
```

```
abbrev HSCoords (α : Type u) [NormedAddCommGroup α] [InnerProductSpace ℂ α]
```

```
  [FiniteDimensional ℂ α] :=
```

```
  EuclideanSpace ℂ (HSIndex α × HSIndex α)
```

```
/-- The underlying coordinate function space for Hilbert-
```

```
Schmidt operators. -/
```

```
abbrev HSCoordFun (α : Type u) [NormedAddCommGroup α] [InnerProductSpace ℂ α]
```

```
  [FiniteDimensional ℂ α] :=
```

```
  HSIndex α × HSIndex α → ℂ
```

```
/-- The operator space  $\mathcal{L}(\alpha)$ , viewed later with the Hilbert-
```

```
Schmidt structure. -/
```

```
def HSOp (α : Type u) [NormedAddCommGroup α] [InnerProductSpace ℂ α] : Type u :=
```

```
  L α
```

```
instance : AddCommGroup (HSOp α) := by
```

```
  delta HSOp
```

```

infer_instance

set_option synthInstance.maxHeartbeats 200000 in
instance : Module  $\mathbb{C}$  (HSOp  $\alpha$ ) := by
  show Module  $\mathbb{C}$  ( $\alpha \rightarrow L[\mathbb{C}] \alpha$ )
  infer_instance

instance : Star (HSOp  $\alpha$ ) := by
  delta HSOp
  infer_instance

instance : Mul (HSOp  $\alpha$ ) := by
  delta HSOp
  infer_instance

instance : One (HSOp  $\alpha$ ) := by
  delta HSOp
  infer_instance

instance : Inhabited (HSOp  $\alpha$ ) := by
  delta HSOp
  infer_instance

instance : Zero (HSOp  $\alpha$ ) := by
  delta HSOp
  infer_instance

set_option synthInstance.maxHeartbeats 200000 in
instance : FiniteDimensional  $\mathbb{C}$  (HSOp  $\alpha$ ) := by
  show FiniteDimensional  $\mathbb{C}$  ( $\alpha \rightarrow L[\mathbb{C}] \alpha$ )
  infer_instance

noncomputable def hsCoordsLinearEquiv :
  HSCoords  $\alpha \approx_1[\mathbb{C}]$  (HSCoordFun  $\alpha$ ) := by
  simp [HSCoords, HSCoordFun] using
    (WithLp.linearEquiv (2 : ENNReal)  $\mathbb{C}$  (HSCoordFun  $\alpha$ ))

def matrixToFun :
  Matrix (HSIndex  $\alpha$ ) (HSIndex  $\alpha$ )  $\mathbb{C} \approx_1[\mathbb{C}]$  HSCoordFun  $\alpha$  where
  toFun M p := M p.1 p.2
  invFun f i j := f (i, j)
  map_add' M N := by
    ext (i, j)
    rfl
  map_smul' c M := by
    ext (i, j)

```

1.117.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/HILBERTSCHMIDTOPERATORSPACE. LEAN

```

    rfl
  left_inv M := by
    ext i j
    rfl
  right_inv f := by
    ext (i, j)
    rfl

noncomputable def matrixToCoords :
  Matrix (HSIndex  $\alpha$ ) (HSIndex  $\beta$ )  $\mathbb{C} \approx_1 [\mathbb{C}]$  HSCoords  $\alpha$  :=
  (matrixToFun ( $\alpha$  :=  $\alpha$ )).trans hsCoordsLinearEquiv.symm

/-- Forget continuity and identify continuous linear operators with linear endomorphisms -/
noncomputable def hsLinearMapEquiv :
  HSOp  $\alpha \approx_1 [\mathbb{C}]$  ( $\alpha \rightarrow_1 [\mathbb{C}] \alpha$ ) :=
  LinearMap.toContinuousLinearMap.symm

/-- Hilbert-Schmidt coordinates on  $\mathcal{L}(\alpha, \beta)$ . -/
noncomputable def toHSCoordsLinearEquiv :
  HSOp  $\alpha \approx_1 [\mathbb{C}]$  HSCoords  $\alpha$  :=
  (hsLinearMapEquiv ( $\alpha$  :=  $\alpha$ )).trans <|
  (LinearMap.toMatrix (hsOrthonormalBasis ( $\alpha$  :=  $\alpha$ )).toBasis
    (hsOrthonormalBasis ( $\beta$  :=  $\beta$ )).toBasis).trans <|
  matrixToCoords ( $\alpha$  :=  $\alpha$ )

omit [CompleteSpace  $\alpha$ ] in
@[simp] lemma toHSCoordsLinearEquiv_apply_apply
  (T : HSOp  $\alpha$ ) (i j : HSIndex  $\alpha$ ) :
  toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T (i, j) =
  LinearMap.toMatrix (hsOrthonormalBasis ( $\alpha$  :=  $\alpha$ )).toBasis
    (hsOrthonormalBasis ( $\beta$  :=  $\beta$ )).toBasis
    ((hsLinearMapEquiv ( $\alpha$  :=  $\alpha$ )) T) i j := by
change hsCoordsLinearEquiv (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T) (i, j) =
  LinearMap.toMatrix (hsOrthonormalBasis ( $\alpha$  :=  $\alpha$ )).toBasis
    (hsOrthonormalBasis ( $\beta$  :=  $\beta$ )).toBasis
    ((hsLinearMapEquiv ( $\alpha$  :=  $\alpha$ )) T) i j
let f : HSCoordFun  $\alpha$  := fun p =>
  LinearMap.toMatrix (hsOrthonormalBasis ( $\alpha$  :=  $\alpha$ )).toBasis
    (hsOrthonormalBasis ( $\beta$  :=  $\beta$ )).toBasis
    ((hsLinearMapEquiv ( $\alpha$  :=  $\alpha$ )) T) p.1 p.2
have h : hsCoordsLinearEquiv (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T) = f := by
  simp [toHSCoordsLinearEquiv, matrixToCoords, matrixToFun, f]
have hEval := congrArg (fun g : HSCoordFun  $\alpha$  => g (i, j)) h
simpa [f] using hEval

```

```

instance : NormedAddCommGroup (HSOp  $\alpha$ ) :=
  NormedAddCommGroup.induced (HSOp  $\alpha$ ) (HSCoords  $\alpha$ )
  (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ )) (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ )).injec

instance : NormedSpace  $\mathbb{C}$  (HSOp  $\alpha$ ) :=
  NormedSpace.induced  $\mathbb{C}$  (HSOp  $\alpha$ ) (HSCoords  $\alpha$ ) (toHSCoordsLinearEquiv ( $\alpha$  :=

instance : Inner  $\mathbb{C}$  (HSOp  $\alpha$ ) where
  inner T S := inner  $\mathbb{C}$  (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T) (toHSCoordsLinear

instance : InnerProductSpace  $\mathbb{C}$  (HSOp  $\alpha$ ) where
  norm_sq_eq_re_inner T := by
    change ||toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T|| ^ 2 =
      Complex.re (inner  $\mathbb{C}$  (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T)
        (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T))
  simp using
    (inner_self_eq_norm_sq ( $\alpha$  :=  $\mathbb{C}$ ) (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T)).s
  conj_inner_symm T S := by
    change star (inner  $\mathbb{C}$  (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) S)
      (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T)) =
      inner  $\mathbb{C}$  (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T) (toHSCoordsLinearEquiv ( $\alpha$ 
    simp [inner_conj_symm (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T)
      (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) S)]
  add_left T S R := by
    change inner  $\mathbb{C}$  (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) (T + S))
      (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) R) =
      inner  $\mathbb{C}$  (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T)
        (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) R) +
      inner  $\mathbb{C}$  (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) S)
        (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) R)
    simp [inner_add_left, map_add]
  smul_left T S z := by
    change inner  $\mathbb{C}$  (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) (z • T))
      (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) S) =
      star z * inner  $\mathbb{C}$  (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T)
        (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) S)
    simp [inner_smul_left, map_smul]

/-- Hilbert-Schmidt coordinates as a linear isometry. -/
noncomputable def toHSCoordsLinearIsometryEquiv :
  HSOp  $\alpha$   $\approx_{\text{li}}$  [ $\mathbb{C}$ ] HSCoords  $\alpha$  :=
  LinearEquiv.isometryOfInner (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ )) <| by
    intro T S
    change inner  $\mathbb{C}$  (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) T)
      (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) S) =
      inner  $\mathbb{C}$  T S

```


1.117.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/HILBERTSCHMIDTOPERATORSPACE.LEAN

```

    rfl

instance : CompleteSpace (HSOp  $\alpha$ ) :=
  let e : HSOp  $\alpha$   $\approx$  HSCoords  $\alpha$  :=
    (toHSCoordsLinearIsometryEquiv.toContinuousLinearEquiv.toLinearEquiv.toEquiv)
  (completeSpace_congr (e := e) <| by
    simp using
      (toHSCoordsLinearIsometryEquiv.toContinuousLinearEquiv.isUniformEmbedding)).2
  inferInstance

instance : ContinuousSMul  $\mathbb{C}$  (HSOp  $\alpha$ ) :=
  (toHSCoordsLinearIsometryEquiv ( $\alpha$  :=  $\alpha$ )).isometry.isUniformInducing.isInducing.com
    continuous_id fun {c x} => map_smul (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ )) c x

/-- Reinterpret an operator as an element of the Hilbert-
Schmidt operator space. -/
abbrev ofOp (T : L  $\alpha$ ) : HSOp  $\alpha$  := T

/-- Forget the Hilbert-Schmidt structure and recover the underlying operator. -
-/
abbrev toOp (T : HSOp  $\alpha$ ) : L  $\alpha$  := T

/-- Left multiplication on the Hilbert-Schmidt operator space. -
-/
noncomputable def leftMulHS (A : L  $\alpha$ ) : HSOp  $\alpha$   $\rightarrow$  L[ $\mathbb{C}$ ] HSOp  $\alpha$  :=
  LinearMap.toContinuousLinearMap
  { toFun := fun T => ofOp (A * toOp T)
    map_add' := fun T S => mul_add A (toOp T) (toOp S)
    map_smul' := fun z T => mul_smul_comm z A (toOp T) }

/-- Right multiplication on the Hilbert-Schmidt operator space. -
-/
noncomputable def rightMulHS (B : L  $\alpha$ ) : HSOp  $\alpha$   $\rightarrow$  L[ $\mathbb{C}$ ] HSOp  $\alpha$  :=
  LinearMap.toContinuousLinearMap
  { toFun := fun T => ofOp (toOp T * B)
    map_add' := fun T S => add_mul (toOp T) (toOp S) B
    map_smul' := fun z T => smul_mul_assoc z (toOp T) B }

omit [CompleteSpace  $\alpha$ ] in
@[simp] lemma leftMulHS_apply (A : L  $\alpha$ ) (T : HSOp  $\alpha$ ) :
  toOp (leftMulHS ( $\alpha$  :=  $\alpha$ ) A T) = A * toOp T := rfl

omit [CompleteSpace  $\alpha$ ] in
@[simp] lemma rightMulHS_apply (B : L  $\alpha$ ) (T : HSOp  $\alpha$ ) :
  toOp (rightMulHS ( $\alpha$  :=  $\alpha$ ) B T) = toOp T * B := rfl
```

```

omit [CompleteSpace  $\alpha$ ] in
@[simp] lemma leftMulHS_mul (A B : L  $\alpha$ ) :
  leftMulHS ( $\alpha$  :=  $\alpha$ ) (A * B) = leftMulHS ( $\alpha$  :=  $\alpha$ ) A * leftMulHS ( $\alpha$  :=  $\alpha$ ) B
ext T
simp [mul_assoc]

omit [CompleteSpace  $\alpha$ ] in
@[simp] lemma rightMulHS_mul (A B : L  $\alpha$ ) :
  rightMulHS ( $\alpha$  :=  $\alpha$ ) (A * B) = rightMulHS ( $\alpha$  :=  $\alpha$ ) B * rightMulHS ( $\alpha$  :=  $\alpha$ ) A
ext T
simp [mul_assoc]

omit [CompleteSpace  $\alpha$ ] in
@[simp] lemma leftMulHS_one :
  leftMulHS ( $\alpha$  :=  $\alpha$ ) (1 : L  $\alpha$ ) = (1 : L (HSOp  $\alpha$ )) := by
ext T
simp

omit [CompleteSpace  $\alpha$ ] in
@[simp] lemma rightMulHS_one :
  rightMulHS ( $\alpha$  :=  $\alpha$ ) (1 : L  $\alpha$ ) = (1 : L (HSOp  $\alpha$ )) := by
ext T
simp

omit [CompleteSpace  $\alpha$ ] in
lemma leftMulHS_rightMulHS_commute (A B : L  $\alpha$ ) :
  Commute (leftMulHS ( $\alpha$  :=  $\alpha$ ) A) (rightMulHS ( $\alpha$  :=  $\alpha$ ) B) := by
ext T
simp [mul_assoc]

omit [CompleteSpace  $\alpha$ ] in
private lemma hsInner_eq_pairSum (X Y : L  $\alpha$ ) :
  inner  $\mathbb{C}$  (ofOp X) (ofOp Y) =
     $\sum p : \text{HSIndex } \alpha \times \text{HSIndex } \alpha,$ 
      LinearMap.toMatrix (hsOrthonormalBasis ( $\alpha$  :=  $\alpha$ )).toBasis
        (hsOrthonormalBasis ( $\alpha$  :=  $\alpha$ )).toBasis Y.toLinearMap p.1 p.2 *
      star
        (LinearMap.toMatrix (hsOrthonormalBasis ( $\alpha$  :=  $\alpha$ )).toBasis
          (hsOrthonormalBasis ( $\alpha$  :=  $\alpha$ )).toBasis X.toLinearMap p.1 p.2) :=
have hXfun :
  ((toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) (ofOp X) : HSCoords  $\alpha$ ).ofLp : HSCoord  $\alpha$ )
  fun p =>
    LinearMap.toMatrix (hsOrthonormalBasis ( $\alpha$  :=  $\alpha$ )).toBasis
      (hsOrthonormalBasis ( $\alpha$  :=  $\alpha$ )).toBasis X.toLinearMap p.1 p.2 :=
funext p
change (toHSCoordsLinearEquiv ( $\alpha$  :=  $\alpha$ ) (ofOp X) :

```

1.117.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/HILBERTSCHMIDTOPERATORSPACE.LEAN

```

EuclideanSpace ℂ (HSIndex n × HSIndex n)) p = _
exact toHSCoordsLinearEquiv_apply_apply (n := n) (T := ofOp X) p.1 p.2
have hYfun :
  ((toHSCoordsLinearEquiv (n := n) (ofOp Y) : HSCoords n).ofLp : HSCoordFun n) =
    fun p =>
      LinearMap.toMatrix (hsOrthonormalBasis (n := n)).toBasis
        (hsOrthonormalBasis (n := n)).toBasis Y.toLinearMap p.1 p.2 := by
funext p
change (toHSCoordsLinearEquiv (n := n) (ofOp Y) :
  EuclideanSpace ℂ (HSIndex n × HSIndex n)) p = _
exact toHSCoordsLinearEquiv_apply_apply (n := n) (T := ofOp Y) p.1 p.2
change inner ℂ (toHSCoordsLinearEquiv (n := n) (ofOp X))
  (toHSCoordsLinearEquiv (n := n) (ofOp Y)) = _
rw [EuclideanSpace.inner_eq_star_dotProduct, hXfun, hYfun]
simp [dotProduct]

private lemma trace_star_mul_eq_pairSum (X Y : L n) :
  LinearMap.trace ℂ n ((star X * Y).toLinearMap) =
    ∑ p : HSIndex n × HSIndex n,
      LinearMap.toMatrix (hsOrthonormalBasis (n := n)).toBasis
        (hsOrthonormalBasis (n := n)).toBasis Y.toLinearMap p.1 p.2 *
      star
        (LinearMap.toMatrix (hsOrthonormalBasis (n := n)).toBasis
          (hsOrthonormalBasis (n := n)).toBasis X.toLinearMap p.1 p.2) := by
let b := hsOrthonormalBasis (n := n)
calc
  LinearMap.trace ℂ n ((star X * Y).toLinearMap)
    = ∑ i : HSIndex n, ∑ j : HSIndex n,
      LinearMap.toMatrix b.toBasis b.toBasis Y.toLinearMap j i *
      star (LinearMap.toMatrix b.toBasis b.toBasis X.toLinearMap j i) := by
  rw [LinearMap.trace_eq_sum_inner ((star X * Y).toLinearMap) b]
  apply Finset.sum_congr rfl
  intro i hi
  calc
    inner ℂ (b i) (((star X * Y).toLinearMap) (b i)) = inner ℂ (X (b i)) (
      simp [ContinuousLinearMap.star_eq_adjoint] using
        (ContinuousLinearMap.adjoint_inner_right (A := X) (x := b i) (y :=
_ = ∑ j : HSIndex n, inner ℂ (X (b i)) (b j) * inner ℂ (b j) (Y (b i))
      symm
      exact OrthonormalBasis.sum_inner_mul_inner b (X (b i)) (Y (b i))
_ = ∑ j : HSIndex n,
      LinearMap.toMatrix b.toBasis b.toBasis Y.toLinearMap j i *
      star (LinearMap.toMatrix b.toBasis b.toBasis X.toLinearMap j i
      apply Finset.sum_congr rfl
      intro j hj
      simp [LinearMap.toMatrix_apply, OrthonormalBasis.repr_apply_apply,

```

```

_ = ∑ p : HSIndex [] × HSIndex [],
  LinearMap.toMatrix b.toBasis b.toBasis Y.toLinearMap p.1 p.2 *
  star (LinearMap.toMatrix b.toBasis b.toBasis X.toLinearMap p.1
  rw [Finset.sum_comm]
  symm
  simp [Finset.univ_product_univ] using
    (Finset.sum_product' (s := (Finset.univ : Finset (HSIndex [])))
    (t := (Finset.univ : Finset (HSIndex [])))
    (f := fun j i =>
      LinearMap.toMatrix b.toBasis b.toBasis Y.toLinearMap j i *
      star (LinearMap.toMatrix b.toBasis b.toBasis X.toLinearMap
lemma hsInner_eq_trace (X Y : L []) :
  inner ℂ (ofOp X) (ofOp Y) = LinearMap.trace ℂ [] ((star X * Y).toLinearMap)
calc
  inner ℂ (ofOp X) (ofOp Y)
    = ∑ p : HSIndex [] × HSIndex [],
      LinearMap.toMatrix (hsOrthonormalBasis ([] := [])).toBasis
        (hsOrthonormalBasis ([] := [])).toBasis Y.toLinearMap p.1 p.2 *
      star
        (LinearMap.toMatrix (hsOrthonormalBasis ([] := [])).toBasis
          (hsOrthonormalBasis ([] := [])).toBasis X.toLinearMap p.1 p.2
      hsInner_eq_pairSum (X := X) (Y := Y)
  _ = LinearMap.trace ℂ [] ((star X * Y).toLinearMap) := by
    symm
    exact trace_star_mul_eq_pairSum (X := X) (Y := Y)

lemma re_hsInner_eq_traceRe (X Y : L []) :
  Complex.re (inner ℂ (ofOp X) (ofOp Y)) =
  Complex.re (LinearMap.trace ℂ [] ((star X * Y).toLinearMap)) := by
  simp using congrArg Complex.re (hsInner_eq_trace (X := X) (Y := Y))

@[simp] lemma leftMulHS_star (A : L []) :
  star (leftMulHS ([] := []) A) = leftMulHS ([] := []) (star A) := by
  rw [eq_comm, ContinuousLinearMap.star_eq_adjoint]
  refine (ContinuousLinearMap.eq_adjoint_iff
    (A := leftMulHS ([] := []) (star A)) (B := leftMulHS ([] := []) A)).2 ?_
  intro X Y
  change inner ℂ (ofOp ((star A) * toOp X)) (ofOp (toOp Y)) =
    inner ℂ (ofOp (toOp X)) (ofOp (A * toOp Y))
  rw [hsInner_eq_trace]
  rw [hsInner_eq_trace]
  simp [mul_assoc]

@[simp] lemma leftMulHS_real_smul_one (r : ℝ) :
  leftMulHS ([] := []) (r • (1 : L [])) = r • (1 : L (HSOp [])) := by

```

1.117.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/HILBERTSCHMIDTOPERATORSPACE.LEAN

```
ext T
change ofOp ((r • (1 : L [])) * toOp T) = ofOp (r • toOp T)
simp [Algebra.smul_def]
```

```
@[simp] lemma rightMulHS_real_smul_one (r : ℝ) :
  rightMulHS (· := ·) (r • (1 : L [])) = r • (1 : L (HSOp [])) := by
ext T
change ofOp (toOp T * (r • (1 : L []))) = ofOp (r • toOp T)
simp [Algebra.smul_def, Algebra.commutes (R := ℝ) (A := L []) r (toOp T)]
```

```
lemma leftMulHS_nonneg {A : L []} (hA0 : 0 ≤ A) :
  0 ≤ leftMulHS (· := ·) A := by
  have hApos : A.IsPositive := (ContinuousLinearMap.nonneg_iff_isPositive A).1 hA0
  have hA_sa : IsSelfAdjoint A := hApos.isSelfAdjoint
  have hleft_sa : IsSelfAdjoint (leftMulHS (· := ·) A) := by
    change star (leftMulHS (· := ·) A) = leftMulHS (· := ·) A
    simp [hA_sa.star_eq, leftMulHS_star (· := ·) A]
  refine (ContinuousLinearMap.nonneg_iff_isPositive _).2 ?_
  rw [ContinuousLinearMap.isPositive_iff_complex]
  intro X
  constructor
  · have hsymm :
      ((leftMulHS (· := ·) A : HSOp [] → L[ℂ] HSOp []) : HSOp [] →1 [ℂ] HSOp []).IsSymmetric
      (ContinuousLinearMap.isSelfAdjoint_iff_isSymmetric).mp hleft_sa
    simpa using LinearMap.IsSymmetric.coe_re_inner_apply_self hsymm X
  · let b := hsOrthonormalBasis (· := ·)
    have htrace :
      Complex.re (inner ℂ (leftMulHS (· := ·) A X) X) =
        ∑ i : HSIndex [], Complex.re (inner ℂ (A (toOp X (b i))) (toOp X (b i))) :=
      calc
        Complex.re (inner ℂ (leftMulHS (· := ·) A X) X)
        = Complex.re (LinearMap.trace ℂ
          ((star (A * toOp X) * toOp X).toLinearMap)) := by
          simpa [leftMulHS_apply] using
            re_hsInner_eq_traceRe (· := ·) (X := A * toOp X) (Y := toOp X)
        _ = ∑ i : HSIndex [],
          Complex.re (inner ℂ (b i)
            (((star (A * toOp X) * toOp X).toLinearMap) (b i))) := by
          rw [LinearMap.trace_eq_sum_inner ((star (A * toOp X) * toOp X).toLinearMap)]
          simp
        _ = ∑ i : HSIndex [],
          Complex.re (inner ℂ (A (toOp X (b i))) (toOp X (b i))) := by
          apply Finset.sum_congr rfl
          intro i hi
          exact congrArg Complex.re <| by
            simpa [ContinuousLinearMap.star_eq_adjoint, mul_assoc] using
```

```

      (ContinuousLinearMap.adjoint_inner_right
      (A := A * toOp X) (x := b i) (y := toOp X (b i)))
    have htrace' :
      RCLike.re (inner ℂ (leftMulHS (□ := □) A X) X) =
        ∑ i : HSIndex □, Complex.re (inner ℂ (A (toOp X (b i))) (toOp X (
        simp using htrace
      rw [htrace']
      exact Finset.sum_nonneg fun i _ => hApos.re_inner_nonneg_left (toOp X (

lemma leftMulHS_le_leftMulHS {A B : L □} (hAB : A ≤ B) :
  leftMulHS (□ := □) A ≤ leftMulHS (□ := □) B := by
  have hnonneg : 0 ≤ leftMulHS (□ := □) (B - A) :=
    leftMulHS_nonneg (□ := □) (sub_nonneg.mpr hAB)
  have hsub :
    leftMulHS (□ := □) B - leftMulHS (□ := □) A =
      leftMulHS (□ := □) (B - A) := by
    ext T; exact congrArg ofOp (sub_mul B A (toOp T)).symm
  exact sub_nonneg.mp (by simp [hsub] using hnonneg)

lemma leftMulHS_pdSet [ContinuousFunctionalCalculus ℝ (L □) IsSelfAdjoint]
  {A : L □} (hA : A ∈ GeneralizedPerspectiveFunction.pdSet (□ := □)) :
  leftMulHS (□ := □) A ∈ GeneralizedPerspectiveFunction.pdSet (□ := HSOp
rcases hA with {hA_sa, hA_spec}
have hleft_sa : IsSelfAdjoint (leftMulHS (□ := □) A) := by
  change star (leftMulHS (□ := □) A) = leftMulHS (□ := □) A
  simp [hA_sa.star_eq, leftMulHS_star (□ := □) A]
letI : Nontrivial (HSOp □) := by
  delta HSOp
  infer_instance
letI : Nontrivial (L (HSOp □)) := inferInstance
refine {?, ?_}
· exact hleft_sa
· rcases (CFC.exists_pos_algebraMap_le_iff (A := L □) (a := A) (ha := hA
  with {r, hr, hrA}
  refine (CFC.exists_pos_algebraMap_le_iff
    (A := L (HSOp □)) (a := leftMulHS (□ := □) A) (ha := hleft_sa)).1 ?_
  refine {r, hr, ?_}
  simp [Algebra.algebraMap_eq_smul_one, leftMulHS_real_smul_one (r := r)
    leftMulHS_le_leftMulHS (□ := □) hrA

@[simp] lemma rightMulHS_star (A : L □) :
  star (rightMulHS (□ := □) A) = rightMulHS (□ := □) (star A) := by
  rw [eq_comm, ContinuousLinearMap.star_eq_adjoint]
  refine (ContinuousLinearMap.eq_adjoint_iff
    (A := rightMulHS (□ := □) (star A)) (B := rightMulHS (□ := □) A)).2 ?_
  intro X Y

```

1.117.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/HILBERTSCHMIDTOPERATORSPACE.LEAN

```

change inner ℂ (ofOp (toOp X * star A)) (ofOp (toOp Y)) =
  inner ℂ (ofOp (toOp X)) (ofOp (toOp Y * A))
rw [hsInner_eq_trace]
rw [hsInner_eq_trace]
rw [star_mul, star_star, mul_assoc]
symm
simp [mul_assoc] using
  (LinearMap.trace_mul_cycle (R := ℂ) (M := []))
  (f := (star (toOp X)).toLinearMap) (g := (toOp Y).toLinearMap) (h := A.toLinearMap)

/-- Left multiplication as a real ``-algebra homomorphism on the Hilbert-
Schmidt operator space. -/
noncomputable def leftMulHSStarAlgHom : L [] →*a[ℝ] L (HSOp []) where
  toFun := leftMulHS ([] := [])
  map_one' := leftMulHS_one ([] := [])
  map_mul' := leftMulHS_mul ([] := [])
  map_zero' := by ext T; exact congrArg ofOp (zero_mul (toOp T))
  map_add' := fun A B => by ext T; exact congrArg ofOp (add_mul A B (toOp T))
  commutes' := by
    intro r
    simp only [Algebra.algebraMap_eq_smul_one]
    exact leftMulHS_real_smul_one ([] := []) r
  map_star' := by
    intro A
    simp [leftMulHS_star ([] := []) A]

/-- Right multiplication as a real ``-algebra homomorphism out of the opposite algebra. -/
noncomputable def rightMulHSStarAlgHom : (L [])mop →*a[ℝ] L (HSOp []) where
  toFun := fun A => rightMulHS ([] := []) (MulOpposite.unop A)
  map_one' := by simp [rightMulHS_one ([] := [])]
  map_mul' := by intro A B; ext T; simp [rightMulHS_apply, mul_assoc]
  map_zero' := by
    ext T
    change ofOp (toOp T * MulOpposite.unop (0 : (L [])mop)) = ofOp (0 : L [])
    congr 1; simp
  map_add' := fun A B => by
    ext T
    change ofOp (toOp T * MulOpposite.unop (A + B)) =
      ofOp (toOp T * MulOpposite.unop A + toOp T * MulOpposite.unop B)
    congr 1; simp [MulOpposite.unop_add, mul_add]
  commutes' := by
    intro r
    show rightMulHS ([] := []) (MulOpposite.unop (MulOpposite.op (r • (1 : L [])))) =
      r • (1 : L (HSOp []))
    simp [rightMulHS_real_smul_one ([] := []) r]

```

```

map_star' := by
  intro A
  simp [rightMulHS_star (□ := □) (MulOpposite.unop A)]

@[simp] theorem rightMulHSStarAlgHom_apply (A : (L □)mop) :
  rightMulHSStarAlgHom (□ := □) A = rightMulHS (□ := □) (MulOpposite.unop
  rfl

/-- The `*`-algebra hom sending `A` to `op (star A)`. On selfadjoint operat
/
noncomputable def opStarHSStarAlgHom : L □ →a [ℝ] (L □)mop where
  toFun := fun A => MulOpposite.op (star A)
  map_one' := by simp
  map_mul' := by
    intro A B
    simp [star_mul]
  map_zero' := by simp
  map_add' := by
    intro A B
    simp
  commutes' := by
    intro r
    apply congrArg MulOpposite.op
    simp [Algebra.algebraMap_eq_smul_one]
  map_star' := by
    intro A
    simp

private noncomputable def opStarHSAlgEquiv : L □ ≈a [ℝ] (L □)mop where
  toFun A := MulOpposite.op (star A)
  invFun A := star (MulOpposite.unop A)
  left_inv A := by simp
  right_inv A := by simp
  map_mul' A B := by simp [star_mul]
  map_add' A B := by simp
  commutes' r := by
    apply congrArg MulOpposite.op
    simp [Algebra.algebraMap_eq_smul_one]

omit [FiniteDimensional ℂ □] in
lemma op_isSelfAdjoint (A : L □) (hA : IsSelfAdjoint A) :
  IsSelfAdjoint (MulOpposite.op A : (L □)mop) := by
  exact congrArg MulOpposite.op hA.star_eq

private noncomputable def opStarHSLinearMap : L □ →1 [ℝ] (L □)mop where
  toFun := fun A => MulOpposite.op (star A)

```


1.117.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/HILBERTSCHMIDTOPERATORSPACE. LEAN

```

map_add' A B := by simp
map_smul' r A := by
  apply MulOpposite.unop_injective
  rw [MulOpposite.unop_op, MulOpposite.unop_smul, MulOpposite.unop_op]
  rw [Algebra.smul_def, star_mul]
  show star A * star ((algebraMap ℝ (L [])) r) = (algebraMap ℝ (L [])) r * star A
  have hstar : star ((algebraMap ℝ (L [])) r) = (algebraMap ℝ (L [])) r := by
    rw [Algebra.algebraMap_eq_smul_one, star_smul, star_one]
    simp
  rw [hstar]
  simpa using (Algebra.commutes (R := ℝ) (A := L []) r (star A)).symm

private noncomputable def leftMulHSLinearMap : L [] →1[ℝ] L (HSOp []) where
  toFun := leftMulHS (□ := □)
  map_add' A B := by ext T; exact congrArg ofOp (add_mul A B (toOp T))
  map_smul' r A := by
    ext T
    show ((r • A) * toOp T : L []) = r • (A * toOp T)
    rw [smul_mul_assoc]

private noncomputable def rightMulHSLinearMap : (L [])mop →1[ℝ] L (HSOp []) where
  toFun := fun A => rightMulHS (□ := □) (MulOpposite.unop A)
  map_add' A B := by
    ext T
    change ofOp (toOp T * MulOpposite.unop (A + B)) =
      ofOp (toOp T * MulOpposite.unop A + toOp T * MulOpposite.unop B)
    congr 1; simp [MulOpposite.unop_add, mul_add]
  map_smul' r A := by
    ext T
    change ofOp (toOp T * MulOpposite.unop (r • A)) =
      ofOp (r • (toOp T * MulOpposite.unop A))
    congr 1
    rw [MulOpposite.unop_smul, mul_smul_comm]

omit [FiniteDimensional ℂ []] in
lemma spectrum_op_eq [ContinuousFunctionalCalculus ℝ (L []) IsSelfAdjoint] [Nontrivial (L [])]
  (A : L []) (hA : IsSelfAdjoint A) :
  spectrum ℝ (MulOpposite.op A : (L [])mop) = spectrum ℝ A := by
  have hopA : IsSelfAdjoint (MulOpposite.op A : (L [])mop) := op_isSelfAdjoint (A :=
    simpa [opStarHSAAlgEquiv, hA.star_eq, hopA.star_eq] using
      (AlgEquiv.spectrum_eq (opStarHSAAlgEquiv (□ := □)) A))

lemma cfc_op_eq_op_cfc [ContinuousFunctionalCalculus ℝ (L []) IsSelfAdjoint]
  [ContinuousFunctionalCalculus ℝ ((L [])mop) IsSelfAdjoint] [Nontrivial (L [])]
  (f : ℝ → ℝ) (A : L []) (hA : IsSelfAdjoint A)
  (hf : ContinuousOn f (spectrum ℝ A)) :

```

```

    cfc (R := ℝ) (A := (L  $\square$ )mop) (p := IsSelfAdjoint) f (MulOpposite.op A)
      MulOpposite.op (cfcR f A) := by
let  $\varphi$  : L  $\square$   $\rightarrow$   $\star_a$ [ $\mathbb{R}$ ] (L  $\square$ )mop := opStarHSStarAlgHom ( $\square$  :=  $\square$ )
have h $\varphi$  : Continuous  $\varphi$  := by
  simp [opStarHSLinearMap] using
    (LinearMap.continuous_of_finiteDimensional (opStarHSLinearMap ( $\square$  :=  $\square$ )))
have hopA : IsSelfAdjoint (MulOpposite.op A : (L  $\square$ )mop) := op_isSelfAdjoint
have h $\varphi$ A : IsSelfAdjoint ( $\varphi$  A) := hA.map  $\varphi$ 
have hcfcA :
  IsSelfAdjoint (cfc (R := ℝ) (A := L  $\square$ ) (p := IsSelfAdjoint) f A) := b
  simp [IsSelfAdjoint.cfc (f := f) (a := A)]
have hopcfcA :
  IsSelfAdjoint
    (MulOpposite.op (cfc (R := ℝ) (A := L  $\square$ ) (p := IsSelfAdjoint) f A)
      op_isSelfAdjoint (A := cfc (R := ℝ) (A := L  $\square$ ) (p := IsSelfAdjoint) f A)
have hmap := StarAlgHom.map_cfc ( $\varphi$  :=  $\varphi$ ) (f := f) (a := A)
  (hf := hf) (h $\varphi$  := h $\varphi$ ) (ha := hA) (h $\varphi$ a := h $\varphi$ A)
simp [  $\varphi$ , opStarHSStarAlgHom, cfcR, hA.star_eq, hopA.star_eq, hcfcA.star_eq,
  hopcfcA.star_eq ] using hmap.symm

lemma leftMulHS_cfcR [ContinuousFunctionalCalculus  $\mathbb{R}$  (L  $\square$ ) IsSelfAdjoint] [
  (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (A : L  $\square$ ) (hA : IsSelfAdjoint A)
  (hf : ContinuousOn f (spectrum  $\mathbb{R}$  A)) :
  leftMulHS ( $\square$  :=  $\square$ ) (cfcR f A) =
    cfcR ( $\square$  := HSOp  $\square$ ) f (leftMulHS ( $\square$  :=  $\square$ ) A) := by
let  $\varphi$  : L  $\square$   $\rightarrow$   $\star_a$ [ $\mathbb{R}$ ] L (HSOp  $\square$ ) := leftMulHSStarAlgHom ( $\square$  :=  $\square$ )
have h $\varphi$  : Continuous  $\varphi$  := by
  simp [leftMulHSLinearMap] using
    (LinearMap.continuous_of_finiteDimensional (leftMulHSLinearMap ( $\square$  :=  $\square$ )))
have h $\varphi$ A : IsSelfAdjoint ( $\varphi$  A) := hA.map  $\varphi$ 
have hmap := StarAlgHom.map_cfc ( $\varphi$  :=  $\varphi$ ) (f := f) (a := A)
  (hf := hf) (h $\varphi$  := h $\varphi$ ) (ha := hA) (h $\varphi$ a := h $\varphi$ A)
simp [  $\varphi$ , cfcR ] using hmap

lemma rightMulHS_cfcR [ContinuousFunctionalCalculus  $\mathbb{R}$  (L  $\square$ ) IsSelfAdjoint] [
  ContinuousFunctionalCalculus  $\mathbb{R}$  ((L  $\square$ )mop) IsSelfAdjoint] [Nontrivial (
  (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (A : L  $\square$ ) (hA : IsSelfAdjoint A)
  (hf : ContinuousOn f (spectrum  $\mathbb{R}$  A)) :
  rightMulHS ( $\square$  :=  $\square$ ) (cfcR f A) =
    cfcR ( $\square$  := HSOp  $\square$ ) f (rightMulHS ( $\square$  :=  $\square$ ) A) := by
let  $\varphi$  : (L  $\square$ )mop  $\rightarrow$   $\star_a$ [ $\mathbb{R}$ ] L (HSOp  $\square$ ) := rightMulHSStarAlgHom ( $\square$  :=  $\square$ )
have h $\varphi$  : Continuous  $\varphi$  := by
  simp [rightMulHSLinearMap] using
    (LinearMap.continuous_of_finiteDimensional (rightMulHSLinearMap ( $\square$  :=  $\square$ )))
have hopA : IsSelfAdjoint (MulOpposite.op A : (L  $\square$ )mop) := op_isSelfAdjoint
have h $\varphi$ A : IsSelfAdjoint ( $\varphi$  (MulOpposite.op A)) := hopA.map  $\varphi$ 

```

1.118.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSENOPERATORINEQUALITY.LEAN

```

have hmap := StarAlgHom.map_cfc (φ := φ) (f := f) (a := MulOpposite.op A)
  (hf := by simpa [spectrum_op_eq (A := A) hA] using hf)
  (hφ := hφ) (ha := hopA) (hφa := hφA)
have hopcfc :
  cfc (R := ℝ) (A := (L ⊞)ᵐᵒᵖ) (p := IsSelfAdjoint) f (MulOpposite.op A) =
    MulOpposite.op (cfcR f A) :=
  cfc_op_eq_op_cfc (⊞ := ⊞) f A hA hf
simpa [φ, cfcR, rightMulHSStarAlgHom, hopcfc] using hmap

end

end HilbertSchmidtOperatorSpace

```

1.118 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperator

/-

Copyright (c) 2026 Hayata Yamasaki. All rights reserved.
 Released under Apache 2.0 license as described in the file LICENSE.
 Authors: Kei Tsukamoto, Kento Mori, Hayata Yamasaki

-/

module

public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.JensenOperatorInequality

@[expose] public section

namespace JensenOperatorInequality

universe u

open LownerHeinzTheorem

section Theorem252

variable {α : Type u}

variable [NormedAddCommGroup α] [InnerProductSpace ℂ α] [CompleteSpace α]

set_option synthInstance.maxHeartbeats 400000 in

-- IsStarNormal CFC is only a theorem in Mathlib; CStarAlgebra chain through WithLp
 noncomputable local instance : ContinuousFunctionalCalculus ℂ (L α × L α) IsStarNormal
 IsStarNormal.instContinuousFunctionalCalculus

set_option synthInstance.maxHeartbeats 400000 in

-- IsSelfAdjoint CFC for the product type, derived from IsStarNormal above.
 noncomputable local instance : ContinuousFunctionalCalculus ℝ (L α × L α) IsSelfAdjoint

```

    IsSelfAdjoint.instContinuousFunctionalCalculus
set_option synthInstance.maxHeartbeats 400000 in
-- CStarAlgebra → NonnegSpectrumClass chain through WithLp is too deep for
noncomputable local instance : NonnegSpectrumClass  $\mathbb{R}$  (L (HSum  $\square$ )) := inferI
set_option synthInstance.maxHeartbeats 400000 in
-- Module  $\mathbb{R}$  for L (HSum  $\square$ ) requires deep WithLp / CStarAlgebra chain.
noncomputable local instance : Module  $\mathbb{R}$  (L (HSum  $\square$ )) := inferInstance

omit  $\square$  [NormedAddCommGroup  $\square$ ] [InnerProductSpace  $\mathbb{C}$   $\square$ ] [CompleteSpace  $\square$ ] in
/--
Uniform version of Condition (iv), with the Hilbert space arbitrary in the
This is the theorem-level uniform counterpart to the operator-
level `...All` predicates.
-/
def CondIVAll (f :  $\mathbb{R}$  →  $\mathbb{R}$ ) : Prop :=
   $\forall$  {K : Type u}
    [NormedAddCommGroup K] [InnerProductSpace  $\mathbb{C}$  K] [CompleteSpace K]
    [Nontrivial K],
    CondIV ( $\square$  := K) f

omit [CompleteSpace  $\square$ ] in
/-- `L (HSum  $\square$ )` is nontrivial once `L  $\square$ ` is. -/
private theorem nontrivial_hsumL_wrap [Nontrivial  $\square$ ] : Nontrivial (L (HSum
  have h_not_sub :  $\neg$  Subsingleton  $\square$  := by
    intro hsub
    letI : Subsingleton  $\square$  := hsub
    letI : Subsingleton (L  $\square$ ) := by infer_instance
    exact (not_nontrivial_iff_subsingleton.mpr (by infer_instance))
      (inferInstance : Nontrivial (L  $\square$ ))
  have hH_nontriv : Nontrivial  $\square$  := (not_subsingleton_iff_nontrivial.mp h_n
  letI : Nontrivial  $\square$  := hH_nontriv
  rcases exists_pair_ne  $\square$  with  $\langle x, y, hxy \rangle$ 
  let w :  $\square$  := x - y
  have hw : w  $\neq$  0 := sub_ne_zero.mpr hxy
  have hdiag_ne_zero : (blockDiagonal ( $\square$  :=  $\square$ ) (1 : L  $\square$ ) 0 : L (HSum  $\square$ ))  $\neq$ 
    intro h0
    have hz :
      blockDiagonal ( $\square$  :=  $\square$ ) (1 : L  $\square$ ) 0 (hsumIncl  $\square$  0 w) = 0 := by
        simp [h0]
    have hw0 : w = 0 := by
      have hz0 := congrArg (fun z : HSum  $\square$  => hsumProj  $\square$  0 z) hz
      simp [blockDiagonal] using hz0
    exact hw hw0
  exact  $\langle 0, \text{blockDiagonal } (\square := \square) (1 : L \square) 0, \text{hdiag\_ne\_zero.symm} \rangle$ 

private lemma blockDiagonal_selfAdjoint_wrap {A B : L  $\square$ }

```

1.118.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN/SEMI-TRACE/SEMI-TRACE-INEQUALITY. LEAN

```

    (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B) :
      IsSelfAdjoint (blockDiagonal (□ := □) A B) := by
    change star (blockDiagonal (□ := □) A B) = blockDiagonal (□ := □) A B
    simp [blockDiagonal_star, hA.star_eq, hB.star_eq]

omit [CompleteSpace □] in
private lemma blockDiagonal_eq_blockOp_wrap (A B : L □) :
  blockDiagonal (□ := □) A B = blockOp (□ := □) A 0 0 B := by
  ext z i
  fin_cases i <=> simp [blockDiagonal, blockOp]

-- Multiplication of generic block operators is elaboration-
heavy even in the wrapper.
set_option maxHeartbeats 400000 in
-- The generic `blockOp` product expands into large block normal forms.
omit [CompleteSpace □] in
private lemma blockOp_mul_wrap (A00 A01 A10 A11 B00 B01 B10 B11 : L □) :
  blockOp (□ := □) A00 A01 A10 A11 * blockOp (□ := □) B00 B01 B10 B11 =
    blockOp (□ := □)
      (A00 * B00 + A01 * B10)
      (A00 * B01 + A01 * B11)
      (A10 * B00 + A11 * B10)
      (A10 * B01 + A11 * B11) := by
  refine blockOp_ext (□ := □) ?_ ?_
  · intro z
    simp [ContinuousLinearMap.mul_def, add_left_comm, add_comm]
  · intro z
    simp [ContinuousLinearMap.mul_def, add_left_comm, add_comm]

private lemma cfcR_blockDiagonal_wrap (f : ℝ → ℝ)
  (A B : L □) (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B)
  (hcont : ContinuousOn f (spectrum ℝ A ∪ spectrum ℝ B)) :
  cfcR (□ := HSum □) f (blockDiagonal (□ := □) A B) =
    blockDiagonal (□ := □) (cfcR (□ := □) f A) (cfcR (□ := □) f B) := by
  let φ : (L □ × L □) →*a[ℝ] L (HSum □) := blockDiagonalHom (□ := □)
  have hφ : Continuous φ := by
    change Continuous (fun p : L □ × L □ => blockDiagonal (□ := □) p.1 p.2)
    change Continuous (fun p : L □ × L □ =>
      hsumIncl □ 0 ◦L p.1 ◦L hsumProj □ 0 + hsumIncl □ 1 ◦L p.2 ◦L hsumProj □ 1)
    fun_prop
  have hpair : IsSelfAdjoint (A, B) := by
    change star (A, B) = (A, B)
    ext <=> simp [hA.star_eq, hB.star_eq]
  have hpair' : IsSelfAdjoint (φ (A, B)) := hpair.map φ
  have hmap := StarAlgHom.map_cfc (φ := φ) (f := f) (a := (A, B))
  (hf := by simpa [Prod.spectrum_eq] using hcont)

```

```

(hφ := hφ) (ha := hpair) (hφa := hpair')
have hprod :
  cfc (R := ℝ) (A := L [] × L []) (p := IsSelfAdjoint) f (A, B) =
    (cfcR (□ := □) f A, cfcR (□ := □) f B) := by
  simpa [cfcR] using
    (cfc_map_prod (R := ℝ) (S := ℝ)
      (A := L []) (B := L [])
      (pab := IsSelfAdjoint) (pa := IsSelfAdjoint) (pb := IsSelfAdjoint)
      f A B
      (hf := hcont)
      (hab := hpair) (ha := hA) (hb := hB))
calc
  cfcR (□ := HSum □) f (blockDiagonal (□ := □) A B)
    = cfc (R := ℝ) (A := L (HSum □)) (p := IsSelfAdjoint) f (φ (A, B))
      simp [cfcR, φ]
_ = φ (cfc (R := ℝ) (A := L [] × L []) (p := IsSelfAdjoint) f (A, B)) :=
  simpa using hmap.symm
_ = φ (cfcR (□ := □) f A, cfcR (□ := □) f B) := by
  rw [hprod]
_ = blockDiagonal (□ := □) (cfcR (□ := □) f A) (cfcR (□ := □) f B) := by
  simp [φ, blockDiagonalHom]

private lemma continuousOn_union_of_subset_Ici_wrap {f : ℝ → ℝ}
  (hcont : ContinuousOn f (Set.Ici (0 : ℝ))) {s t : Set ℝ}
  (hs : s ⊆ Set.Ici (0 : ℝ)) (ht : t ⊆ Set.Ici (0 : ℝ)) :
  ContinuousOn f (s ∪ t) := by
  refine hcont.mono ?_
  intro x hx
  rcases hx with hx | hx
  · exact hs hx
  · exact ht hx

private lemma spectrum_Ici_of_nonneg_wrap {A : L []} (hA0 : (0 : L []) ≤ A) :
  spectrum ℝ A ⊆ Set.Ici (0 : ℝ) := by
  exact
    (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) A
      (ha := IsSelfAdjoint.of_nonneg hA0)).1 hA0

variable [Nontrivial []]

private lemma spectrum_zero_subset_Ici_wrap :
  spectrum ℝ (0 : L []) ⊆ Set.Ici (0 : ℝ) := by
  intro x hx
  have hx0 : x = 0 := by
    simpa using hx
  simp [Set.Ici, hx0]

```

1.118.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN-~~SEMI~~WENOPEATORINEQUALITY.LEAN

```

omit [Nontrivial  $\alpha$ ] in
private lemma blockDiagonal_le_left_wrap {A0 A1 B0 B1 : L  $\alpha$ }
  (h : blockDiagonal ( $\alpha := \alpha$ ) A0 A1  $\leq$  blockDiagonal ( $\alpha := \alpha$ ) B0 B1) :
  A0  $\leq$  B0 := by
  have hnonneg : 0  $\leq$  blockDiagonal ( $\alpha := \alpha$ ) (B0 - A0) (B1 - A1) := by
    have hsub :
      blockDiagonal ( $\alpha := \alpha$ ) B0 B1 - blockDiagonal ( $\alpha := \alpha$ ) A0 A1 =
        blockDiagonal ( $\alpha := \alpha$ ) (B0 - A0) (B1 - A1) := by
      refine blockOp_ext ( $\alpha := \alpha$ ) ?_ ?_
      · intro z
        simp [sub_eq_add_neg]
      · intro z
        simp [sub_eq_add_neg]
    exact hsub ▸ sub_nonneg.mpr h
  have hpos :
    (blockDiagonal ( $\alpha := \alpha$ ) (B0 - A0) (B1 - A1)).IsPositive :=
      (ContinuousLinearMap.nonneg_iff_isPositive _).1 hnonneg
  have hleftPos : (B0 - A0).IsPositive := by
    rw [ContinuousLinearMap.isPositive_iff_complex]
    intro x
    have hx :=
      (ContinuousLinearMap.isPositive_iff_complex
        (blockDiagonal ( $\alpha := \alpha$ ) (B0 - A0) (B1 - A1))).1 hpos (hsumIncl  $\alpha$  0 x)
    simp [blockDiagonal, hsumProj, hsumIncl, hsumEquiv, PiLp.inner_apply] using hx
    exact sub_nonneg.mp ((ContinuousLinearMap.nonneg_iff_isPositive _).2 hleftPos)

-- Theorem 2.5.2 `(iv)  $\rightarrow$  (v)`.
set_option maxHeartbeats 3000000 in
-- Block-matrix normalization in this wrapper needs a larger local heartbeat budget.
theorem theorem_2_5_2_iv_imp_v {f :  $\mathbb{R} \rightarrow \mathbb{R}$ } (hiv : CondIVAll.{u} f)
  (hcont : ContinuousOn f Set.univ) :
  CondV ( $\alpha := \alpha$ ) f := by
  intro A B X Y hA hB hAs hBs hXY
  have hA0 : (0 : L  $\alpha$ )  $\leq$  A := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R :=  $\mathbb{R}$ ) A (ha := hA)).2 ?_
    intro x hx
    simp [Set.Ici] using hAs hx
  have hB0 : (0 : L  $\alpha$ )  $\leq$  B := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R :=  $\mathbb{R}$ ) B (ha := hB)).2 ?_
    intro x hx
    simp [Set.Ici] using hBs hx
  let Atilde : L (HSum  $\alpha$ ) := blockDiagonal ( $\alpha := \alpha$ ) A B
  let Xtilde : L (HSum  $\alpha$ ) := blockOp ( $\alpha := \alpha$ ) X 0 Y 0
  letI : Nontrivial (L (HSum  $\alpha$ )) := nontrivial_hsumL_wrap ( $\alpha := \alpha$ )
  have hAtilde_sa : IsSelfAdjoint Atilde := by

```

```

    simp [Atilde] using blockDiagonal_selfAdjoint_wrap (□ := □) hA hB
  have hAtilde0 : (0 : L (HSum □)) ≤ Atilde := by
    simp [Atilde] using blockDiagonal_nonneg (□ := □) hA0 hB0
  have hAtilde_spec : spectrum ℝ Atilde ≤ Set.Ici (0 : ℝ) := by
    exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) Atilde (ha :
  have hXtilde_star_mul :
    star Xtilde * Xtilde =
      blockDiagonal (□ := □) (star X * X + star Y * Y) 0 := by
    rw [show star Xtilde = blockOp (□ := □) (star X) (star Y) 0 0 by
      simp [Xtilde]]
    rw [show Xtilde = blockOp (□ := □) X 0 Y 0 by rfl]
    rw [blockOp_mul_wrap, blockDiagonal_eq_blockOp_wrap]
    simp
  have hXtilde_star_mul_le : star Xtilde * Xtilde ≤ (1 : L (HSum □)) := by
    have hblock_nonneg :
      (0 : L (HSum □)) ≤
        blockDiagonal (□ := □) (1 - (star X * X + star Y * Y)) (1 : L □)
    refine blockDiagonal_nonneg (□ := □) ?_ ?_
    · exact sub_nonneg.mpr hXY
    · simp
  have hsub :
    (1 : L (HSum □)) - blockDiagonal (□ := □) (star X * X + star Y * Y)
      blockDiagonal (□ := □) (1 - (star X * X + star Y * Y)) (1 : L □)
    refine blockOp_ext (□ := □) ?_ ?_
    · intro z
      simp [sub_eq_add_neg]
    · intro z
      simp [sub_eq_add_neg]
  have hblock :
    blockDiagonal (□ := □) (star X * X + star Y * Y) 0 ≤ (1 : L (HSum □)
    exact sub_nonneg.mp (by simp [hsub] using hblock_nonneg)
  simp [hXtilde_star_mul] using hblock
  have hXtilde_star_mul_nonneg : (0 : L (HSum □)) ≤ star Xtilde * Xtilde :=
    simp
  have hXtilde_norm : ||Xtilde|| ≤ 1 := by
    have hnormSq : ||star Xtilde * Xtilde|| ≤ 1 :=
      (CStarAlgebra.norm_le_one_iff_of_nonneg _ hXtilde_star_mul_nonneg).2
    have hnormSq' : ||Xtilde|| * ||Xtilde|| ≤ 1 := by
      simp [CStarRing.norm_star_mul_self] using hnormSq
    have hsq : ||Xtilde|| ^ 2 ≤ 1 := by
      simp [pow_two] using hnormSq'
    nlinarith [norm_nonneg Xtilde]
  have hiv_hsum : CondIV (□ := HSum □) f := @hiv (HSum □)
  have hcore := hiv_hsum (A := Atilde) (X := Xtilde) hAtilde_sa hAtilde_spe
  have hsum_sa : IsSelfAdjoint (star X * A * X + star Y * B * Y) := by
    change star (star X * A * X + star Y * B * Y) = star X * A * X + star Y

```


1.118.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN/OPERATORINEQUALITY.LEAN

```

simp [hA.star_eq, hB.star_eq, mul_assoc]
have hmul_block :
  star Xtilde * Atilde * Xtilde =
    blockDiagonal (□ := □) (star X * A * X + star Y * B * Y) 0 := by
  rw [show star Xtilde = blockOp (□ := □) (star X) (star Y) 0 0 by
    simp [Xtilde]]
  rw [show Atilde = blockOp (□ := □) A 0 0 B by
    simp [Atilde] using blockDiagonal_eq_blockOp_wrap (□ := □) A B]
  rw [show Xtilde = blockOp (□ := □) X 0 Y 0 by rfl]
  rw [blockOp_mul_wrap, blockOp_mul_wrap, blockDiagonal_eq_blockOp_wrap]
  congr 1 <=> simp [mul_assoc]
have hAtilde_cfc :
  cfcR (□ := HSum □) f Atilde =
    blockDiagonal (□ := □) (cfcR (□ := □) f A) (cfcR (□ := □) f B) := by
  simp [Atilde] using
    cfcR_blockDiagonal_wrap (□ := □) (f := f) A B hA hB
    (hcont.mono (by intro x hx; simp))
have hright_block :
  star Xtilde * cfcR (□ := HSum □) f Atilde * Xtilde =
    blockDiagonal (star X * cfcR (□ := □) f A * X + star Y * cfcR (□ := □) f B *
  rw [hAtilde_cfc]
  rw [show star Xtilde = blockOp (□ := □) (star X) (star Y) 0 0 by
    simp [Xtilde]]
  rw [show Xtilde = blockOp (□ := □) X 0 Y 0 by rfl]
  rw [blockDiagonal_eq_blockOp_wrap, blockOp_mul_wrap, blockOp_mul_wrap, blockDiag
  congr 1 <=> simp [mul_assoc]
have hleft_block :
  cfcR (□ := HSum □) f (star Xtilde * Atilde * Xtilde) =
    blockDiagonal (cfcR f (star X * A * X + star Y * B * Y)) (cfcR f 0) := by
  rw [hmul_block]
  simp using
    cfcR_blockDiagonal_wrap f (star X * A * X + star Y * B * Y) 0 hsum_sa (by simp
    (hcont.mono (by intro x hx; simp))
  rw [hleft_block, hright_block] at hcore
  exact blockDiagonal_le_left_wrap (□ := □) hcore

/-- Uniform consequence of Theorem 2.5.2: `(i) → (v)` via `(iv)` . -
/
theorem theorem_2_5_2_i_all_imp_v {f : ℝ → ℝ} (hf : CondIAll.{u} f) :
  CondV (□ := □) f := by
  have hconv : OperatorConvex (□ := □) f := by
    exact hf.1
  have hcont : ContinuousOn f Set.univ :=
    operatorConvex_continuousOn_univ (□ := □) hconv
  refine theorem_2_5_2_iv_imp_v (□ := □) ?_ hcont
  intro K _ _ _

```

```

exact theorem_2_5_2_i_all_imp_iv (□ := K) (f := f) hf

-- Uniform localized consequence of Theorem 2.5.2: `(i) → (v)` on `Set.Ici`
set_option maxHeartbeats 3000000 in
-- The localized wrapper repeats the same block-operator normalization as
-- `theorem_2_5_2_iv_imp_v`.
theorem theorem_2_5_2_i_ici_all_imp_v {f : ℝ → ℝ}
  (hf : CondIciAll.{u} f) :
    CondV (□ := □) f := by
  have hfIci := hf
  rcases hf with (_, hcontIci, _)
  intro A B X Y hA hB hAs hBs hXY
  have hA0 : (0 : L □) ≤ A := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) A (ha := hA))
  intro x hx
  simp [Set.Ici] using hAs hx
  have hB0 : (0 : L □) ≤ B := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B (ha := hB))
  intro x hx
  simp [Set.Ici] using hBs hx
  let Atilde : L (HSum □) := blockDiagonal (□ := □) A B
  let Xtilde : L (HSum □) := blockOp (□ := □) X 0 Y 0
  letI : Nontrivial (L (HSum □)) := nontrivial_hsumL_wrap (□ := □)
  have hAtilde_sa : IsSelfAdjoint Atilde := by
    simp [Atilde] using blockDiagonal_selfAdjoint_wrap (□ := □) hA hB
  have hAtilde0 : (0 : L (HSum □)) ≤ Atilde := by
    simp [Atilde] using blockDiagonal_nonneg (□ := □) hA0 hB0
  have hAtilde_spec : spectrum ℝ Atilde ≤ Set.Ici (0 : ℝ) :=
    spectrum_Ici_of_nonneg_wrap (□ := HSum □) hAtilde0
  have hXtilde_star_mul :
    star Xtilde * Xtilde =
      blockDiagonal (□ := □) (star X * X + star Y * Y) 0 := by
  rw [show star Xtilde = blockOp (□ := □) (star X) (star Y) 0 0 by
    simp [Xtilde]]
  rw [show Xtilde = blockOp (□ := □) X 0 Y 0 by rfl]
  rw [blockOp_mul_wrap, blockDiagonal_eq_blockOp_wrap]
  simp
  have hXtilde_star_mul_le : star Xtilde * Xtilde ≤ (1 : L (HSum □)) := by
  have hblock_nonneg :
    (0 : L (HSum □)) ≤
      blockDiagonal (□ := □) (1 - (star X * X + star Y * Y)) (1 : L □)
  refine blockDiagonal_nonneg (□ := □) ?_ ?_
  · exact sub_nonneg.mpr hXY
  · simp
  have hsub :
    (1 : L (HSum □)) - blockDiagonal (□ := □) (star X * X + star Y * Y)

```

1.118.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSENOPERATORINEQUALITY.LEAN

```

      blockDiagonal (□ := □) (1 - (star X * X + star Y * Y)) (1 : L □) := by
    refine blockOp_ext (□ := □) ?_ ?_
    · intro z
      simp [sub_eq_add_neg]
    · intro z
      simp [sub_eq_add_neg]
  have hblock :
    blockDiagonal (□ := □) (star X * X + star Y * Y) 0 ≤ (1 : L (HSum □)) := by
    exact sub_nonneg.mp (by simpa [hsub] using hblock_nonneg)
  simpa [hXtilde_star_mul] using hblock
  have hXtilde_star_mul_nonneg : (0 : L (HSum □)) ≤ star Xtilde * Xtilde := by
    simp
  have hXtilde_norm : ||Xtilde|| ≤ 1 := by
    have hnormSq : ||star Xtilde * Xtilde|| ≤ 1 :=
      (CStarAlgebra.norm_le_one_iff_of_nonneg _ hXtilde_star_mul_nonneg).2 hXtilde_s
    have hnormSq' : ||Xtilde|| * ||Xtilde|| ≤ 1 := by
      simpa [CStarRing.norm_star_mul_self] using hnormSq
    have hsq : ||Xtilde|| ^ 2 ≤ 1 := by
      simpa [pow_two] using hnormSq'
    nlinarith [norm_nonneg Xtilde]
  have hiv_hsum : CondIV (□ := HSum □) f :=
    theorem_2_5_2_i_ici_all_imp_iv (□ := HSum □) (f := f) hfIci
  have hcore := hiv_hsum (A := Atilde) (X := Xtilde) hAtilde_sa hAtilde_spec hXtilde
  have hsum_nonneg : (0 : L □) ≤ star X * A * X + star Y * B * Y := by
    have hXA : (0 : L □) ≤ star X * A * X := by
      simpa [mul_assoc] using star_left_conjugate_nonneg hA0 X
    have hYB : (0 : L □) ≤ star Y * B * Y := by
      simpa [mul_assoc] using star_left_conjugate_nonneg hB0 Y
    exact add_nonneg hXA hYB
  have hsum_sa : IsSelfAdjoint (star X * A * X + star Y * B * Y) :=
    IsSelfAdjoint.of_nonneg hsum_nonneg
  have hmul_block :
    star Xtilde * Atilde * Xtilde =
      blockDiagonal (□ := □) (star X * A * X + star Y * B * Y) 0 := by
    rw [show star Xtilde = blockOp (□ := □) (star X) (star Y) 0 0 by
      simp [Xtilde]]
    rw [show Atilde = blockOp (□ := □) A 0 0 B by
      simpa [Atilde] using blockDiagonal_eq_blockOp_wrap (□ := □) A B]
    rw [show Xtilde = blockOp (□ := □) X 0 Y 0 by rfl]
    rw [blockOp_mul_wrap, blockOp_mul_wrap, blockDiagonal_eq_blockOp_wrap]
    congr 1 <=> simp [mul_assoc]
  have hAtilde_cfc :
    cfcR (□ := HSum □) f Atilde =
      blockDiagonal (□ := □) (cfcR (□ := □) f A) (cfcR (□ := □) f B) := by
    simpa [Atilde] using
      cfcR_blockDiagonal_wrap (□ := □) (f := f) A B hA hB

```

```

      (continuousOn_union_of_subset_Ici_wrap (f := f) hcontIci hAs hBs)
have hright_block :
  star Xtilde * cfcR (□ := HSum □) f Atilde * Xtilde =
    blockDiagonal (□ := □)
      (star X * cfcR (□ := □) f A * X + star Y * cfcR (□ := □) f B * Y)
rw [hAtilde_cfc]
rw [show star Xtilde = blockOp (□ := □) (star X) (star Y) 0 0 by
  simp [Xtilde]]
rw [show Xtilde = blockOp (□ := □) X 0 Y 0 by rfl, blockDiagonal_eq_blockOp,
  blockOp_mul_wrap, blockOp_mul_wrap, blockDiagonal_eq_blockOp_wrap]
congr 1 <=> simp [mul_assoc]
have hsum_spec : spectrum ℝ (star X * A * X + star Y * B * Y) ⊆ Set.Ici (
  spectrum_Ici_of_nonneg_wrap (□ := □) hsum_nonneg)
have hleft_block :
  cfcR (□ := HSum □) f (star Xtilde * Atilde * Xtilde) =
    blockDiagonal (cfcR f (star X * A * X + star Y * B * Y)) (cfcR f 0)
rw [hmul_block]
simp using
  cfcR_blockDiagonal_wrap f (star X * A * X + star Y * B * Y) 0 hsum_spec
  (continuousOn_union_of_subset_Ici_wrap hcontIci hsum_spec spectrum_
rw [hleft_block, hright_block] at hcore
exact blockDiagonal_le_left_wrap (□ := □) hcore
end Theorem252

end JensenOperatorInequality

```

1.119 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/Jensen

/-

Copyright (c) 2026 Hayata Yamasaki. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Kei Tsukamoto, Kento Mori, Hayata Yamasaki

-/

module

```

public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.BlockDiagonal
public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.LownerHeinz
public import Mathlib.Analysis.CStarAlgebra.Unitary.Span
public import Mathlib.Algebra.Star.UnitaryStarAlgAut

```

@[expose] public section

namespace JensenOperatorInequality

1.119.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN/OPERATORINEQUALITYIIMPIV.LE

universe u

open LownerHeinzTheorem

section Theorem252

variable { α : Type u}

variable [NormedAddCommGroup α] [InnerProductSpace $\mathbb{C}$ α] [CompleteSpace α]

set_option synthInstance.maxHeartbeats 400000 in

-- IsStarNormal CFC is only a theorem in Mathlib; CStarAlgebra chain through WithLp
noncomputable local instance : ContinuousFunctionalCalculus $\mathbb{C}$ (L $\alpha \times$ L α) IsStarNormal

IsStarNormal.instContinuousFunctionalCalculus

set_option synthInstance.maxHeartbeats 400000 in

-- IsSelfAdjoint CFC for the product type, derived from IsStarNormal above.

noncomputable local instance : ContinuousFunctionalCalculus $\mathbb{R}$ (L $\alpha \times$ L α) IsSelfAdjoint

IsSelfAdjoint.instContinuousFunctionalCalculus

set_option synthInstance.maxHeartbeats 400000 in

-- CStarAlgebra $\rightarrow$ NonnegSpectrumClass chain through WithLp is too deep for default h

noncomputable local instance : NonnegSpectrumClass $\mathbb{R}$ (L (HSum α)) := inferInstance

/-- Condition (iv) in Theorem 2.5.2. -/

def CondIV (f : $\mathbb{R} \rightarrow \mathbb{R}$) : Prop :=

$\forall \alpha A X : L \alpha, \text{IsSelfAdjoint } A \rightarrow \text{spectrum } \mathbb{R} A \subseteq \text{Set.Ici } (0 : \mathbb{R}) \rightarrow \|X\| \leq 1 \rightarrow$
cfcR ($\alpha := \alpha$) f (star X * A * X) $\leq$ star X * cfcR ($\alpha := \alpha$) f A * X

/-- Condition (i) in Theorem 2.5.2 on the fixed Hilbert space ` α '. -

/

def CondI (f : $\mathbb{R} \rightarrow \mathbb{R}$) : Prop :=

OperatorConvex ($\alpha := \alpha$) f $\wedge$ f 0 $\leq$ 0

omit α [NormedAddCommGroup α] [InnerProductSpace $\mathbb{C}$ α] [CompleteSpace α] in

/--

Uniform version of Condition (i), packaged as `OperatorConvexAll` together with `f 0

-/

def CondIAll (f : $\mathbb{R} \rightarrow \mathbb{R}$) : Prop :=

OperatorConvexAll.{u} f $\wedge$
f 0 $\leq$ 0

omit α [NormedAddCommGroup α] [InnerProductSpace $\mathbb{C}$ α] [CompleteSpace α] in

/--

Uniform localized version of Condition (i), packaged as

`OperatorConvexOnAll (Set.Ici 0)` together with continuity and `f 0  $\leq$  0`.

-/

def CondIciAll (f : $\mathbb{R} \rightarrow \mathbb{R}$) : Prop :=

```

OperatorConvexOnAll.{u} (Set.Ici (0 : ℝ)) f ∧
ContinuousOn f (Set.Ici (0 : ℝ)) ∧
f 0 ≤ 0

/-- Condition (v) in Theorem 2.5.2. -/
def CondV (f : ℝ → ℝ) : Prop :=
  ∀ A B X Y : L [],
    IsSelfAdjoint A → IsSelfAdjoint B →
    spectrum ℝ A ⊆ Set.Ici (0 : ℝ) → spectrum ℝ B ⊆ Set.Ici (0 : ℝ) →
    star X * X + star Y * Y ≤ (1 : L []) →
    cfcR (· := ·) f (star X * A * X + star Y * B * Y) ≤
    star X * cfcR (· := ·) f A * X + star Y * cfcR (· := ·) f B * Y

/-- Selfadjoint `2 × 2` block built from a contraction candidate `X`. -
/
private noncomputable def blockSwap (X : L []) : L (HSum []) :=
  blockOp (· := ·) 0 (star X) X 0

private lemma blockSwap_star (X : L []) :
  star (blockSwap (· := ·) X) = blockSwap (· := ·) X := by
  simp [blockSwap, blockOp_star]

private lemma blockSwap_sq (X : L []) :
  blockSwap (· := ·) X * blockSwap (· := ·) X =
    blockDiagonal (· := ·) (star X * X) (X * star X) := by
  ext z i
  fin_cases i <|>
  simp [blockSwap, blockOp, blockDiagonal, ContinuousLinearMap.mul_def] <|>

omit [CompleteSpace []] in
private lemma blockDiagonal_eq_blockOp (A B : L []) :
  blockDiagonal (· := ·) A B = blockOp (· := ·) A 0 0 B := by
  ext z i
  fin_cases i <|> simp [blockDiagonal, blockOp]

set_option maxHeartbeats 400000 in
-- Multiplying two generic `blockOp` expressions is elaboration-
heavy.
omit [CompleteSpace []] in
private lemma blockOp_mul (A00 A01 A10 A11 B00 B01 B10 B11 : L []) :
  blockOp (· := ·) A00 A01 A10 A11 * blockOp (· := ·) B00 B01 B10 B11 =
    blockOp (· := ·)
      (A00 * B00 + A01 * B10)
      (A00 * B01 + A01 * B11)
      (A10 * B00 + A11 * B10)
      (A10 * B01 + A11 * B11) := by

```

1.119.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN/OPERATORINEQUALITYIIMPIV.LE

```

-- The extra heartbeat budget stays local to this normalization lemma.
refine blockOp_ext (□ := □) ?_ ?_
· intro z
  simp [ContinuousLinearMap.mul_def, add_left_comm, add_comm]
· intro z
  simp [ContinuousLinearMap.mul_def, add_left_comm, add_comm]

omit [CompleteSpace □] in
private lemma blockOp_add
  (A00 A01 A10 A11 B00 B01 B10 B11 : L □) :
  blockOp (□ := □) A00 A01 A10 A11 + blockOp (□ := □) B00 B01 B10 B11 =
  blockOp (□ := □) (A00 + B00) (A01 + B01) (A10 + B10) (A11 + B11) := by
  ext z i
  fin_cases i <=> simp [blockOp] <=> abel

set_option synthInstance.maxHeartbeats 100000 in
-- Scalar action on `blockOp` triggers expensive instance search for nested operator
private lemma blockOp_smulR
  (r : ℝ) (A00 A01 A10 A11 : L □) :
  r • blockOp (□ := □) A00 A01 A10 A11 =
  blockOp (□ := □) (r • A00) (r • A01) (r • A10) (r • A11) := by
  have coe_smul_hsum : ∀ (f : L (HSum □)) (x : HSum □), (r • f) x = r • (f x) := by
    intros; rfl
  have coe_smul_h : ∀ (f : L □) (x : □), (r • f) x = r • (f x) := by
    intros; rfl
  ext z i
  fin_cases i <=> {
    simp only [blockOp, ContinuousLinearMap.add_apply, ContinuousLinearMap.comp_apply,
      smul_add, coe_smul_hsum, coe_smul_h]
    simp [hsumProj, hsumIncl, hsumEquiv]
  }

private lemma blockSwap_add_I_smul_blockDiagonal (X R0 R1 : L □) :
  blockSwap (□ := □) X + Complex.I • blockDiagonal (□ := □) R0 R1 =
  blockOp (□ := □) (Complex.I • R0) (star X) X (Complex.I • R1) := by
  rw [blockDiagonal_eq_blockOp]
  ext z i
  fin_cases i <=> simp [blockSwap, blockOp] <=> abel

omit [CompleteSpace □] in
private lemma blockOp_mul_blockDiagonal_zero_right
  (P00 P01 P10 P11 A Q00 Q01 Q10 Q11 : L □) :
  blockOp (□ := □) P00 P01 P10 P11 * blockDiagonal (□ := □) 0 A *
  blockOp (□ := □) Q00 Q01 Q10 Q11 =
  blockOp (□ := □)
  (P01 * A * Q10)

```

```

(P01 * A * Q11)
(P11 * A * Q10)
(P11 * A * Q11) := by
rw [blockDiagonal_eq_blockOp, blockOp_mul, blockOp_mul]
simp [mul_assoc, add_comm]

set_option synthInstance.maxHeartbeats 400000 in
-- `StarAlgHom.map_cfc` needs `MulAction  $\mathbb{R}$  (L (HSum  $\square$ ))`; search is deep wi
private lemma cfcR_blockDiagonal (f :  $\mathbb{R} \rightarrow \mathbb{R}$ )
  (A B : L  $\square$ ) (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B)
  (hcont : ContinuousOn f (spectrum  $\mathbb{R}$  A  $\cup$  spectrum  $\mathbb{R}$  B)) :
  cfcR ( $\square :=$  HSum  $\square$ ) f (blockDiagonal ( $\square :=$   $\square$ ) A B) =
    blockDiagonal ( $\square :=$   $\square$ ) (cfcR ( $\square :=$   $\square$ ) f A) (cfcR ( $\square :=$   $\square$ ) f B) := by
let  $\phi$  : (L  $\square \times$  L  $\square$ )  $\rightarrow^{*a}[\mathbb{R}]$  L (HSum  $\square$ ) := blockDiagonalHom ( $\square :=$   $\square$ )
have h $\phi$  : Continuous  $\phi :=$  by
  change Continuous (fun p : L  $\square \times$  L  $\square \Rightarrow$  blockDiagonal ( $\square :=$   $\square$ ) p.1 p.2)
  change Continuous (fun p : L  $\square \times$  L  $\square \Rightarrow$ 
    hsumIncl  $\square$  0  $\circ$  L p.1  $\circ$  L hsumProj  $\square$  0 + hsumIncl  $\square$  1  $\circ$  L p.2  $\circ$  L hsumProj
  fun_prop
have hpair : IsSelfAdjoint (A, B) := by
  change star (A, B) = (A, B)
  ext <;> simp [hA.star_eq, hB.star_eq]
have hpair' : IsSelfAdjoint ( $\phi$  (A, B)) := hpair.map  $\phi$ 
have hmap := StarAlgHom.map_cfc ( $\phi :=$   $\phi$ ) (f := f) (a := (A, B))
  (hf := by simp [Prod.spectrum_eq] using hcont)
  (h $\phi :=$  h $\phi$ ) (ha := hpair) (h $\phi$ a := hpair')
have hprod :
  cfc (R :=  $\mathbb{R}$ ) (A := L  $\square \times$  L  $\square$ ) (p := IsSelfAdjoint) f (A, B) =
    (cfcR ( $\square :=$   $\square$ ) f A, cfcR ( $\square :=$   $\square$ ) f B) := by
simp [cfcR] using
  (cfc_map_prod (R :=  $\mathbb{R}$ ) (S :=  $\mathbb{R}$ )
    (A := L  $\square$ ) (B := L  $\square$ )
    (pab := IsSelfAdjoint) (pa := IsSelfAdjoint) (pb := IsSelfAdjoint)
    f A B
    (hf := hcont)
    (hab := hpair) (ha := hA) (hb := hB))
calc
  cfcR ( $\square :=$  HSum  $\square$ ) f (blockDiagonal ( $\square :=$   $\square$ ) A B)
    = cfc (R :=  $\mathbb{R}$ ) (A := L (HSum  $\square$ )) (p := IsSelfAdjoint) f ( $\phi$  (A, B))
      simp [cfcR,  $\phi$ ]
  _ =  $\phi$  (cfc (R :=  $\mathbb{R}$ ) (A := L  $\square \times$  L  $\square$ ) (p := IsSelfAdjoint) f (A, B)) :=
      simp using hmap.symm
  _ =  $\phi$  (cfcR ( $\square :=$   $\square$ ) f A, cfcR ( $\square :=$   $\square$ ) f B) := by
      rw [hprod]
  _ = blockDiagonal ( $\square :=$   $\square$ ) (cfcR ( $\square :=$   $\square$ ) f A) (cfcR ( $\square :=$   $\square$ ) f B) := b
      simp [ $\phi$ , blockDiagonalHom]

```


1.119.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSENOPERATORINEQUALITYIIMPIV.LE

-- Converting positivity on a block-diagonal operator to each diagonal block is expected

```
private lemma blockDiagonal_le_left {A0 A1 B0 B1 : L  $\square$ }
  (h : blockDiagonal ( $\square := \square$ ) A0 A1  $\leq$  blockDiagonal ( $\square := \square$ ) B0 B1) :
  A0  $\leq$  B0 := by
  have hnonneg : 0  $\leq$  blockDiagonal ( $\square := \square$ ) (B0 - A0) (B1 - A1) := by
    have hsub :
      blockDiagonal ( $\square := \square$ ) B0 B1 - blockDiagonal ( $\square := \square$ ) A0 A1 =
        blockDiagonal ( $\square := \square$ ) (B0 - A0) (B1 - A1) := by
      refine blockOp_ext ( $\square := \square$ ) ?_ ?_
      · intro z
        simp [sub_eq_add_neg]
      · intro z
        simp [sub_eq_add_neg]
    exact hsub ▸ sub_nonneg.mpr h
  have hpos :
    (blockDiagonal ( $\square := \square$ ) (B0 - A0) (B1 - A1)).IsPositive :=
      (ContinuousLinearMap.nonneg_iff_isPositive _).1 hnonneg
  have hleftPos : (B0 - A0).IsPositive := by
    rw [ContinuousLinearMap.isPositive_iff_complex]
    intro x
    have hx :=
      (ContinuousLinearMap.isPositive_iff_complex
        (blockDiagonal ( $\square := \square$ ) (B0 - A0) (B1 - A1))).1 hpos (hsumIncl  $\square$  0 x)
    simp [blockDiagonal, hsumProj, hsumIncl, hsumEquiv, PiLp.inner_apply] using hx
    exact (sub_nonneg.mp ((ContinuousLinearMap.nonneg_iff_isPositive _).2 hleftPos))

private lemma blockDiagonal_selfAdjoint {A B : L  $\square$ }
  (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B) :
  IsSelfAdjoint (blockDiagonal ( $\square := \square$ ) A B) := by
  change star (blockDiagonal ( $\square := \square$ ) A B) = blockDiagonal ( $\square := \square$ ) A B
  simp [blockDiagonal_star, hA.star_eq, hB.star_eq]

private lemma cfcR_zero (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :
  cfcR ( $\square := \square$ ) f (0 : L  $\square$ ) = algebraMap  $\mathbb{R}$  (L  $\square$ ) (f 0) := by
  change cfc (R :=  $\mathbb{R}$ ) (A := L  $\square$ ) (p := IsSelfAdjoint) f (0 : L  $\square$ ) =
    algebraMap  $\mathbb{R}$  (L  $\square$ ) (f 0)
  simp

private lemma cfcR_conj_unitary (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (hcont : ContinuousOn f Set.univ)
  (u : unitary (L  $\square$ )) (A : L  $\square$ ) (hA : IsSelfAdjoint A) :
  cfcR ( $\square := \square$ ) f (star u * A * u) = star u * cfcR ( $\square := \square$ ) f A * u := by
  let  $\phi$  : L  $\square \rightarrow_{*a} [\mathbb{R}]$  L  $\square$  := Unitary.conjStarAlgAut  $\mathbb{R}$  (L  $\square$ ) (star u)
  have h $\phi$  : Continuous  $\phi$  := by
    have h1 : Continuous (fun x : L  $\square \Rightarrow$  (star u : L  $\square$ ) * x * (u : L  $\square$ )) := by
      fun_prop
```

```

    have hEq : (fun x : L  $\square$  =>  $\phi$  x) = (fun x : L  $\square$  => (star u : L  $\square$ ) * x *
      funext x
      simp [ $\phi$ , Unitary.conjStarAlgAut_apply, mul_assoc]
    simpa [hEq] using h1
    have h $\phi$ A : IsSelfAdjoint ( $\phi$  A) := hA.map  $\phi$ 
    have hmap := StarAlgHom.map_cfc ( $\phi$  :=  $\phi$ ) (f := f) (a := A)
      (hf := hcont.mono (by intro x hx; simp))
      (h $\phi$  := h $\phi$ ) (ha := hA) (h $\phi$ a := h $\phi$ A)
    simpa [ $\phi$ , cfcR, Unitary.conjStarAlgAut_apply, mul_assoc] using hmap.symm

private lemma cfcR_conj_unitary_on (s : Set  $\mathbb{R}$ ) (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (hcont : Continuous
  {A : L  $\square$ } (hAs : spectrum  $\mathbb{R}$  A  $\subseteq$  s)
  (u : unitary (L  $\square$ )) (hA : IsSelfAdjoint A) :
  cfcR ( $\square$  :=  $\square$ ) f (star u * A * u) = star u * cfcR ( $\square$  :=  $\square$ ) f A * u := by
  let  $\phi$  : L  $\square$   $\rightarrow^{*a}[\mathbb{R}]$  L  $\square$  := Unitary.conjStarAlgAut  $\mathbb{R}$  (L  $\square$ ) (star u)
  have h $\phi$  : Continuous  $\phi$  := by
    have h1 : Continuous (fun x : L  $\square$  => (star u : L  $\square$ ) * x * (u : L  $\square$ )) :=
      fun_prop
    have hEq : (fun x : L  $\square$  =>  $\phi$  x) = (fun x : L  $\square$  => (star u : L  $\square$ ) * x *
      funext x
      simp [ $\phi$ , Unitary.conjStarAlgAut_apply, mul_assoc]
    simpa [hEq] using h1
    have h $\phi$ A : IsSelfAdjoint ( $\phi$  A) := hA.map  $\phi$ 
    have hmap := StarAlgHom.map_cfc ( $\phi$  :=  $\phi$ ) (f := f) (a := A)
      (hf := hcont.mono hAs)
      (h $\phi$  := h $\phi$ ) (ha := hA) (h $\phi$ a := h $\phi$ A)
    simpa [ $\phi$ , cfcR, Unitary.conjStarAlgAut_apply, mul_assoc] using hmap.symm

private lemma cfcR_real_sqrt_eq_sqrt {A : L  $\square$ } (hA : ( $\theta$  : L  $\square$ )  $\leq$  A) :
  cfcR ( $\square$  :=  $\square$ ) Real.sqrt A = CFC.sqrt A := by
  rw [CFC.sqrt_eq_real_sqrt A hA, cfc_n_eq_cfc (f := Real.sqrt) (a := A) (hf

omit [CompleteSpace  $\square$ ] in
private theorem nontrivial_hsumL [Nontrivial  $\square$ ] : Nontrivial (L (HSum  $\square$ )) :
  have h_not_sub :  $\neg$  Subsingleton  $\square$  := by
    intro hsub
    letI : Subsingleton  $\square$  := hsub
    letI : Subsingleton (L  $\square$ ) := by infer_instance
    exact (not_nontrivial_iff_subsingleton.mpr (by infer_instance))
      (inferInstance : Nontrivial (L  $\square$ ))
  have hH_nontriv : Nontrivial  $\square$  := (not_subsingleton_iff_nontrivial.mp h_n
  letI : Nontrivial  $\square$  := hH_nontriv
  rcases exists_pair_ne  $\square$  with  $\langle x, y, hxy \rangle$ 
  let w :  $\square$  := x - y
  have hw : w  $\neq$  0 := sub_ne_zero.mpr hxy
  have hdiag_ne_zero : (blockDiagonal ( $\square$  :=  $\square$ ) (1 : L  $\square$ ) 0 : L (HSum  $\square$ ))  $\neq$ 

```

1.119.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN/OPERATORINEQUALITYIIMPV.LE

```

intro h0
have hz :
  blockDiagonal (□ := □) (1 : L □) 0 (hsumIncl □ 0 w) = 0 := by
  exact congrArg (fun T : L (HSum □) => T (hsumIncl □ 0 w)) h0
have hw0 : w = 0 := by
  have hz0 := congrArg (fun z : HSum □ => hsumProj □ 0 z) hz
  simpa [blockDiagonal] using hz0
exact hw hw0
exact {0, blockDiagonal (□ := □) (1 : L □) 0, hdiag_ne_zero.symm)

set_option synthInstance.maxHeartbeats 100000 in
-- `CFC.sqrt` on block-diagonal operators triggers expensive instance search through
set_option linter.unusedSectionVars false in
set_option maxHeartbeats 400000 in
private lemma sqrt_blockDiagonal_of_nonneg
  [Nontrivial □]
  {A B : L □} (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B)
  (hA_nonneg : (0 : L □) ≤ A) (hB_nonneg : (0 : L □) ≤ B) :
  CFC.sqrt (blockDiagonal (□ := □) A B) =
    blockDiagonal (□ := □) (CFC.sqrt A) (CFC.sqrt B) := by
  letI : Algebra ℝ (L (HSum □)) := by infer_instance
  letI : Nontrivial (L (HSum □)) := nontrivial_hsumL (□ := □)
  have hdiag_nonneg : (0 : L (HSum □)) ≤ blockDiagonal (□ := □) A B :=
    blockDiagonal_nonneg (□ := □) hA_nonneg hB_nonneg
  rw [← cfcR_real_sqrt_eq_sqrt (□ := HSum □) hdiag_nonneg]
  rw [cfcR_blockDiagonal (□ := □) (f := Real.sqrt) (A := A) (B := B) hA hB]
  · rw [← cfcR_real_sqrt_eq_sqrt (□ := □) hA_nonneg, ← cfcR_real_sqrt_eq_sqrt (□ :=
  · simpa using
    (by cfc_cont_tac : ContinuousOn Real.sqrt (spectrum ℝ A ∪ spectrum ℝ B))

--BUG? In v4.28, this didn't require CompleteSpace □. Then it did.
-- The `NormedAlgebra ℝ (L □)` instances comes from a `CStarAlgebra (□ → L[ℂ] □)`, wh
-- `instCStarAlgebraContinuousLinearMapComplexIdOfCompleteSpace`, which requires Com
-- In principle this isn't necessary.
private lemma complex_I_smul_real_I_smul_invTwo (r : ℝ) (T : L □) :
  Complex.I • r • Complex.I • (2-1 : ℝ) • T =
    -((2-1 : ℝ) * r) • T := by
  ext x
  have hcomm : r • (Complex.I • ((2-1 : ℝ) • T x)) = Complex.I • (r • ((2-1 : ℝ) • T
    simpa using (smul_comm r (Complex.I : ℂ) ((2-1 : ℝ) • T x))
  calc
    Complex.I • r • Complex.I • (2-1 : ℝ) • T x
      = Complex.I • (r • (Complex.I • ((2-1 : ℝ) • T x))) := by
        rfl
    _ = Complex.I • (Complex.I • (r • ((2-1 : ℝ) • T x))) := by
      rw [hcomm]

```

```

_ = ((Complex.I : ℂ) * Complex.I) • (r • ((2-1 : ℝ) • T x)) := by
  rw [smul_smul]
_ = (-1 : ℂ) • (r • ((2-1 : ℝ) • T x)) := by
  norm_num
_ = (-1 : ℂ) • ((r * 2-1 : ℝ)) • T x := by
  rw [smul_smul]
_ = -((2-1 : ℝ) * r) • T x := by
  simp [neg_smul, mul_comm]

private lemma real_smul_complex_I_real_smul_complex_I_comm (s r : ℝ) (T : L
(s : ℝ) • Complex.I • r • Complex.I • T =
  Complex.I • r • Complex.I • (s : ℝ) • T := by
calc
(s : ℝ) • Complex.I • r • Complex.I • T
= Complex.I • ((s : ℝ) • (r • (Complex.I • T))) := by
  simp [smul_smul] using (smul_comm (s : ℝ) (Complex.I : ℂ) (r •
_ = Complex.I • (r • ((s : ℝ) • (Complex.I • T))) := by
  rw [smul_comm (s : ℝ) r (Complex.I • T)]
_ = Complex.I • (r • (Complex.I • ((s : ℝ) • T))) := by
  rw [smul_comm (s : ℝ) (Complex.I : ℂ) T]
_ = Complex.I • r • Complex.I • (s : ℝ) • T := by
  rfl

private lemma half_add_half_eq (T : L □) :
(2-1 : ℝ) • T + (2-1 : ℝ) • T = T := by
calc
(2-1 : ℝ) • T + (2-1 : ℝ) • T = (2-1 + 2-1 : ℝ) • T := by
  simp [add_smul]
_ = (1 : ℝ) • T := by norm_num
_ = T := by simp

private lemma half_mul_real_add_half_mul_real_eq (r : ℝ) (T : L □) :
((2-1 : ℝ) * r) • T + ((2-1 : ℝ) * r) • T = r • T := by
calc
((2-1 : ℝ) * r) • T + ((2-1 : ℝ) * r) • T =
  (((2-1 : ℝ) * r) + ((2-1 : ℝ) * r)) • T := by
  simp [add_smul]
_ = r • T := by ring_nf

private lemma rightEval_topLeft_scalar
(r : ℝ) (R0 X T : L □) :
(2-1 : ℝ) • (star X * (T * X)) +
  ((2-1 : ℝ) • (star X * (T * X)) +
    (-(2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0)) +
    -(2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0))) =
  star X * (T * X) + r • (R0 * R0) := by

```

1.119.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSENOPERATORINEQUALITYIMPIV.LE

```

have hP :
  (2-1 : ℝ) • (star X * (T * X)) +
  (2-1 : ℝ) • (star X * (T * X)) =
  star X * (T * X) := by
  simp using half_add_half_eq (□ := □) (star X * (T * X))
have hQhalf :
  -((2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0)) +
  -((2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0)) =
  r • (R0 * R0) := by
  have hterm :
    -((2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0)) =
    ((2-1 : ℝ) * r) • (R0 * R0) := by
  calc
    -((2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0))
    = -(Complex.I • r • Complex.I • (2-1 : ℝ) • (R0 * R0)) := by
      rw [real_smul_complex_I_real_smul_complex_I_comm
        (□ := □) (s := (2-1 : ℝ)) (r := r) (T := R0 * R0)]
    _ = ((2-1 : ℝ) * r) • (R0 * R0) := by
      rw [complex_I_smul_real_I_smul_invTwo (□ := □) (r := r) (T := R0 * R0)]
      simp
  calc
    -((2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0)) +
    -((2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0)) =
    ((2-1 : ℝ) * r) • (R0 * R0) + ((2-1 : ℝ) * r) • (R0 * R0) := by
      simp [hterm]
    _ = r • (R0 * R0) := half_mul_real_add_half_mul_real_eq (□ := □) r (R0 * R0)
  calc
    (2-1 : ℝ) • (star X * (T * X)) +
    ((2-1 : ℝ) • (star X * (T * X)) +
    -((2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0)) +
    -((2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0)))
    =
    ((2-1 : ℝ) • (star X * (T * X)) +
    (2-1 : ℝ) • (star X * (T * X))) +
    -((2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0)) +
    -((2-1 : ℝ) • Complex.I • r • Complex.I • (R0 * R0)) := by
      abel
    _ = star X * (T * X) + r • (R0 * R0) := by rw [hP, hQhalf]

private lemma rightEval_bottomRight_scalar
  (r : ℝ) (R1 X T : L □) :
  (2-1 * r) • (X * star X) +
  ((2-1 * r) • (X * star X) +
  ((2-1 : ℝ) • (R1 * (T * R1)) +
  (2-1 : ℝ) • (R1 * (T * R1)))) =
  (R1 * T * R1) + r • (X * star X) := by

```

```

have hS :
  (2-1 * r) • (X * star X) + (2-1 * r) • (X * star X) = r • (X * star X)
  simp using half_mul_real_add_half_mul_real_eq (□ := □) r (X * star X)
have hT :
  (2-1 : ℝ) • (R1 * (T * R1)) + (2-1 : ℝ) • (R1 * (T * R1)) =
  R1 * (T * R1) := by
  simp using half_add_half_eq (□ := □) (R1 * (T * R1))
calc
  (2-1 * r) • (X * star X) +
  ((2-1 * r) • (X * star X) +
   ((2-1 : ℝ) • (R1 * (T * R1)) +
    (2-1 : ℝ) • (R1 * (T * R1))))
  =
  ((2-1 * r) • (X * star X) + (2-1 * r) • (X * star X)) +
  ((2-1 : ℝ) • (R1 * (T * R1)) + (2-1 : ℝ) • (R1 * (T * R1))) := by
  abel
_ = r • (X * star X) + R1 * (T * R1) := by rw [hS, hT]
_ = (R1 * T * R1) + r • (X * star X) := by simp [mul_assoc, add_comm]

private lemma star_mul_le_one [Nontrivial □] (X : L □) (hX : ||X|| ≤ 1) :
  (star X * X : L □) ≤ 1 := by
  have h1 : star X * X ≤ algebraMap ℝ (L □) (||X|| ^ 2) := by
    simp [pow_two] using (CStarAlgebra.star_mul_le_algebraMap_norm_sq (a :
  have hsq : ||X|| ^ 2 ≤ 1 := by
    nlinarith [hX, norm_nonneg X]
  exact h1.trans (by simp [Algebra.algebraMap_eq_smul_one] using hsq)

private lemma mul_star_le_one [Nontrivial □] (X : L □) (hX : ||X|| ≤ 1) :
  (X * star X : L □) ≤ 1 := by
  have h1 : X * star X ≤ algebraMap ℝ (L □) (||X|| ^ 2) := by
    simp [pow_two] using (CStarAlgebra.star_mul_le_algebraMap_norm_sq (a :
  have hsq : ||X|| ^ 2 ≤ 1 := by
    nlinarith [hX, norm_nonneg X]
  exact h1.trans (by simp [Algebra.algebraMap_eq_smul_one] using hsq)

-- `simp` and normalization over block expressions are expensive here.
private lemma blockSwap_norm_le_one [Nontrivial □] (X : L □) (hX : ||X|| ≤ 1) :
  ||blockSwap (□ := □) X|| ≤ 1 := by
  have hSstar : star (blockSwap (□ := □) X) = blockSwap (□ := □) X :=
    blockSwap_star (□ := □) X
  have hSstarS :
    star (blockSwap (□ := □) X) * blockSwap (□ := □) X =
    blockDiagonal (□ := □) (star X * X) (X * star X) := by
    simp [hSstar] using blockSwap_sq (□ := □) X
  have hDiagLe :
    blockDiagonal (□ := □) (star X * X) (X * star X) ≤ (1 : L (HSum □)) :

```

1.119.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN/OPERATORINEQUALITYIIMPIV.LE

```

have hA : 0 ≤ (1 : L []) - star X * X := sub_nonneg.mpr (star_mul_le_one (□ := □))
have hB : 0 ≤ (1 : L []) - X * star X := sub_nonneg.mpr (mul_star_le_one (□ := □))
have hnonneg : 0 ≤ blockDiagonal (□ := □) (1 - star X * X) (1 - X * star X) :=
  blockDiagonal_nonneg (□ := □) hA hB
have hle :
  blockDiagonal (□ := □) (star X * X) (X * star X) ≤
    blockDiagonal (□ := □) (star X * X) (X * star X) +
      blockDiagonal (□ := □) (1 - star X * X) (1 - X * star X) :=
    le_add_of_nonneg_right hnonneg
have hsum :
  blockDiagonal (□ := □) (star X * X) (X * star X) +
    blockDiagonal (□ := □) (1 - star X * X) (1 - X * star X) =
      blockDiagonal (□ := □) (1 : L []) (1 : L []) := by
    refine blockOp_ext (□ := □) ?_ ?_
    · intro z
      simp [sub_eq_add_neg, add_left_comm, add_comm]
    · intro z
      simp [sub_eq_add_neg, add_left_comm, add_comm]
have hle' :
  blockDiagonal (□ := □) (star X * X) (X * star X) ≤
    blockDiagonal (□ := □) (1 : L []) (1 : L []) := by
    simpa [hsum] using hle
    simpa [blockDiagonal_one] using hle'
have hSstarSle :
  star (blockSwap (□ := □) X) * blockSwap (□ := □) X ≤ (1 : L (HSum [])) := by
    simpa [hSstarS] using hDiagLe
have hSstarSnonneg : 0 ≤ star (blockSwap (□ := □) X) * blockSwap (□ := □) X := by
  exact star_mul_self_nonneg (blockSwap (□ := □) X)
have hnormSq : ||star (blockSwap (□ := □) X) * blockSwap (□ := □) X|| ≤ 1 :=
  (CStarAlgebra.norm_le_one_iff_of_nonneg _ hSstarSnonneg).2 hSstarSle
have hnormSq' : ||blockSwap (□ := □) X|| * ||blockSwap (□ := □) X|| ≤ 1 := by
  simpa [CStarRing.norm_star_mul_self] using hnormSq
have hsq : ||blockSwap (□ := □) X|| ^ 2 ≤ 1 := by
  simpa [pow_two] using hnormSq'
have hnonneg : 0 ≤ ||blockSwap (□ := □) X|| := norm_nonneg _
nlinarith

private lemma continuousOn_union_of_subset_Ici {f : ℝ → ℝ}
  (hcont : ContinuousOn f (Set.Ici (0 : ℝ))) {s t : Set ℝ}
  (hs : s ⊆ Set.Ici (0 : ℝ)) (ht : t ⊆ Set.Ici (0 : ℝ)) :
  ContinuousOn f (s ∪ t) := by
  refine hcont.mono ?_
  intro x hx
  rcases hx with hx | hx
  · exact hs hx
  · exact ht hx

```

```

private lemma spectrum_Ici_of_nonneg {A : L  $\mathbb{R}$ } (hA0 : (0 : L  $\mathbb{R}$ )  $\leq$  A) :
  spectrum  $\mathbb{R}$  A  $\subseteq$  Set.Ici (0 :  $\mathbb{R}$ ) := by
  exact
    (StarOrderedRing.nonneg_iff_spectrum_nonneg (R :=  $\mathbb{R}$ ) A
     (ha := IsSelfAdjoint.of_nonneg hA0)).1 hA0

variable [Nontrivial  $\mathbb{R}$ ]

private lemma spectrum_zero_subset_Ici :
  spectrum  $\mathbb{R}$  (0 : L  $\mathbb{R}$ )  $\subseteq$  Set.Ici (0 :  $\mathbb{R}$ ) := by
  intro x hx
  have hx0 : x = 0 := by
    simpa using hx
  simp [Set.Ici, hx0]

--Theorem 2.5.2 `(i)  $\rightarrow$  (iv)`.

set_option maxHeartbeats 2000000 in
-- The localized proof duplicates the block-matrix normalization from the g
theorem theorem_2_5_2_i_ici_all_imp_iv {f :  $\mathbb{R} \rightarrow \mathbb{R}$ } (hf : CondIciAll.{u} f)
  CondIV ( $\mathbb{R}$  :=  $\mathbb{R}$ ) f := by
  rcases hf with (hconvAll, hcontIci, hf0)
  intro A X hA hAs hX
  have hconv : OperatorConvexOn ( $\mathbb{R}$  :=  $\mathbb{R}$ ) (Set.Ici (0 :  $\mathbb{R}$ )) f := hconvAll (K
  have hA0 : (0 : L  $\mathbb{R}$ )  $\leq$  A := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R :=  $\mathbb{R}$ ) A (ha := hA
    intro x hx
    simpa [Set.Ici] using hAs hX
  let S : L (HSum  $\mathbb{R}$ ) := blockSwap ( $\mathbb{R}$  :=  $\mathbb{R}$ ) X
  have hSsa : IsSelfAdjoint S := by
    change star S = S
    simpa [S] using blockSwap_star ( $\mathbb{R}$  :=  $\mathbb{R}$ ) X
  have hSnorm :  $\|S\| \leq 1$  := by
    simpa [S] using blockSwap_norm_le_one ( $\mathbb{R}$  :=  $\mathbb{R}$ ) X hX
  letI : Algebra  $\mathbb{R}$  (L (HSum  $\mathbb{R}$ )) := by
    infer_instance
  have hU_mem : S + Complex.I • CFC.sqrt (1 - S ^ 2)  $\in$  unitary (L (HSum  $\mathbb{R}$ ))
    exact IsSelfAdjoint.self_add_I_smul_cfcSqrt_sub_sq_mem_unitary S hSsa h
  let U : unitary (L (HSum  $\mathbb{R}$ )) :=
    (S + Complex.I • CFC.sqrt (1 - S ^ 2), hU_mem)
  let V : unitary (L (HSum  $\mathbb{R}$ )) := star U
  let Atilde : L (HSum  $\mathbb{R}$ ) := blockDiagonal ( $\mathbb{R}$  :=  $\mathbb{R}$ ) 0 A
  letI : Nontrivial (L (HSum  $\mathbb{R}$ )) := nontrivial_hsumL ( $\mathbb{R}$  :=  $\mathbb{R}$ )
  have hconv2 : OperatorConvexOn ( $\mathbb{R}$  := HSum  $\mathbb{R}$ ) (Set.Ici (0 :  $\mathbb{R}$ )) f :=

```


1.119.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN/OPERATORINEQUALITYIIMPIV.LE

```

hconvAll (K := HSum [])
have hR0nonneg : (0 : L []) ≤ 1 - star X * X := sub_nonneg.mpr (star_mul_le_one (0
have hR1nonneg : (0 : L []) ≤ 1 - X * star X := sub_nonneg.mpr (mul_star_le_one (0
let R0 : L [] := CFC.sqrt (1 - star X * X)
let R1 : L [] := CFC.sqrt (1 - X * star X)
have hR0sa : IsSelfAdjoint (1 - star X * X) := by
  change star (1 - star X * X) = 1 - star X * X
  simp
have hR1sa : IsSelfAdjoint (1 - X * star X) := by
  change star (1 - X * star X) = 1 - X * star X
  simp
have hSsq : S ^ 2 = blockDiagonal ([] := []) (star X * X) (X * star X) := by
  simpa [pow_two, S] using blockSwap_sq ([] := []) X
have hOneMinusSq :
  1 - S ^ 2 = blockDiagonal ([] := []) (1 - star X * X) (1 - X * star X) := by
  rw [hSsq]
  refine blockOp_ext ([] := []) ?_ ?_
  · intro z
    simp [sub_eq_add_neg, add_comm]
  · intro z
    simp [sub_eq_add_neg, add_comm]
have hOneMinusSqNonneg : (0 : L (HSum [])) ≤ 1 - S ^ 2 := by
  have hdiag : (0 : L (HSum [])) ≤
    blockDiagonal ([] := []) (1 - star X * X) (1 - X * star X) :=
    blockDiagonal_nonneg ([] := []) hR0nonneg hR1nonneg
  simpa [hOneMinusSq] using hdiag
have hRblock : CFC.sqrt (1 - S ^ 2) = blockDiagonal ([] := []) R0 R1 := by
  rw [hOneMinusSq]
  simp [R0, R1]
  simpa using
    (sqrt_blockDiagonal_of_nonneg ([] := []) (A := 1 - star X * X) (B := 1 - X * star X)
      hR0sa hR1sa hR0nonneg hR1nonneg)
have hR0self : IsSelfAdjoint R0 := by
  have h : IsSelfAdjoint (CFC.sqrt (1 - star X * X)) :=
    (CFC.sqrt_nonneg (1 - star X * X)).isSelfAdjoint
  simpa [R0] using h
have hR1self : IsSelfAdjoint R1 := by
  have h : IsSelfAdjoint (CFC.sqrt (1 - X * star X)) :=
    (CFC.sqrt_nonneg (1 - X * star X)).isSelfAdjoint
  simpa [R1] using h
have hU_block :
  (U : L (HSum [])) = blockOp ([] := []) (Complex.I • R0) (star X) X (Complex.I • R
  change S + Complex.I • CFC.sqrt (1 - S ^ 2) = _
  rw [hRblock]
  simpa [S] using blockSwap_add_I_smul_blockDiagonal ([] := []) X R0 R1
have hV_block :

```

```

      (V : L (HSum [])) = blockOp (□ := □) (-Complex.I • R0) (star X) X (-
Complex.I • R1) := by
change star (U : L (HSum [])) = _
rw [hU_block]
ext z i
fin_cases i <=>
  simp [blockOp_star, hR0self.star_eq, hR1self.star_eq]
have hB1_block :
  (star U : L (HSum [])) * Atilde * (U : L (HSum [])) =
    blockOp (□ := □)
      (star X * A * X)
      (star X * A * (Complex.I • R1))
      ((-Complex.I • R1) * A * X)
      ((-Complex.I • R1) * A * (Complex.I • R1)) := by
rw [show (star U : L (HSum [])) = (V : L (HSum [])) by rfl, hV_block, hU_block]
simp [Atilde, mul_assoc] using
  (blockOp_mul_blockDiagonal_zero_right (□ := □)
    (-Complex.I • R0) (star X) X (-Complex.I • R1) A
    (Complex.I • R0) (star X) X (Complex.I • R1))
have hB2_block :
  (star V : L (HSum [])) * Atilde * (V : L (HSum [])) =
    blockOp (□ := □)
      (star X * A * X)
      (star X * A * (-Complex.I • R1))
      ((Complex.I • R1) * A * X)
      ((Complex.I • R1) * A * (-Complex.I • R1)) := by
rw [show (star V : L (HSum [])) = (U : L (HSum [])) by simp [V], hU_block]
simp [Atilde, mul_assoc] using
  (blockOp_mul_blockDiagonal_zero_right (□ := □)
    (Complex.I • R0) (star X) X (Complex.I • R1) A
    (-Complex.I • R0) (star X) X (-Complex.I • R1))
have hmid_block :
  (1 / 2 : ℝ) • ((star U : L (HSum [])) * Atilde * (U : L (HSum [])) ) +
    (1 / 2 : ℝ) • ((star V : L (HSum [])) * Atilde * (V : L (HSum [])))
    blockDiagonal (□ := □) (star X * A * X) (R1 * A * R1) := by
rw [hB1_block, hB2_block]
rw [blockOp_smulR, blockOp_smulR, blockOp_add, blockDiagonal_eq_blockOp]
congr 1
· have hhalf : (2-1 + 2-1 : ℝ) = (1 : ℝ) := by norm_num
calc
  (1 / 2 : ℝ) • (star X * A * X) + (1 / 2 : ℝ) • (star X * A * X)
    = (2-1 + 2-1 : ℝ) • (star X * (A * X)) := by
      simp [add_smul, mul_assoc]
  _ = (1 : ℝ) • (star X * (A * X)) := by rw [hhalf]
  _ = star X * (A * X) := by simp
  _ = star X * A * X := by simp [mul_assoc]

```

1.119.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN/OPERATORINEQUALITYIIMPIV.LE

```

· simp [mul_assoc]
· simp [mul_assoc]
· have hhalf : (2-1 + 2-1 : ℝ) = (1 : ℝ) := by norm_num
calc
  (1 / 2 : ℝ) • (-Complex.I • R1 * A * (Complex.I • R1)) +
    (1 / 2 : ℝ) • (Complex.I • R1 * A * (-Complex.I • R1))
    = (2-1 + 2-1 : ℝ) • (R1 * (A * R1)) := by
      simp [Complex.I_mul_I, smul_smul, add_smul, mul_assoc]
  _ = (1 : ℝ) • (R1 * (A * R1)) := by rw [hhalf]
  _ = R1 * A * R1 := by simp [mul_assoc]
have hAtilde_sa : IsSelfAdjoint Atilde := by
  simpa [Atilde] using blockDiagonal_selfAdjoint (□ := □) (hA := by simp) hA
have hAtilde0 : (0 : L (HSum □)) ≤ Atilde := by
  simpa [Atilde] using blockDiagonal_nonneg (□ := □) (show (0 : L □) ≤ 0 by simp)
have hAtilde_spec : spectrum ℝ Atilde ⊆ Set.Ici (0 : ℝ) := spectrum_Ici_of_nonneg
have hB1_nonneg : (0 : L (HSum □)) ≤ (star U : L (HSum □)) * Atilde * (U : L (HSum □))
  simpa [mul_assoc] using star_left_conjugate_nonneg hAtilde0 (U : L (HSum □))
have hB2_nonneg : (0 : L (HSum □)) ≤ (star V : L (HSum □)) * Atilde * (V : L (HSum □))
  simpa [mul_assoc] using star_left_conjugate_nonneg hAtilde0 (V : L (HSum □))
have hB1_sa : IsSelfAdjoint ((star U : L (HSum □)) * Atilde * (U : L (HSum □))) :=
  IsSelfAdjoint.of_nonneg hB1_nonneg
have hB2_sa : IsSelfAdjoint ((star V : L (HSum □)) * Atilde * (V : L (HSum □))) :=
  IsSelfAdjoint.of_nonneg hB2_nonneg
have hB1_spec : spectrum ℝ (star U * Atilde * (U : L (HSum □))) ⊆ Set.Ici (0 : ℝ)
  spectrum_Ici_of_nonneg (□ := HSum □) hB1_nonneg
have hB2_spec : spectrum ℝ (star V * Atilde * (V : L (HSum □))) ⊆ Set.Ici (0 : ℝ)
  spectrum_Ici_of_nonneg (□ := HSum □) hB2_nonneg
have hmid_conv :
  cfcR (□ := HSum □) f
    ((1 / 2 : ℝ) • ((star U : L (HSum □)) * Atilde * (U : L (HSum □))) +
      (1 / 2 : ℝ) • ((star V : L (HSum □)) * Atilde * (V : L (HSum □)))) ≤
    ((1 / 2 : ℝ) • cfcR (□ := HSum □) f ((star U : L (HSum □)) * Atilde * (U : L (HSum □)))
      (1 / 2 : ℝ) • cfcR (□ := HSum □) f ((star V : L (HSum □)) * Atilde * (V : L (HSum □))))
have hhalf : (1 - (2-1 : ℝ)) = (2-1 : ℝ) := by norm_num
simpa [hhalf] using
  (hconv2
    (A := (star U : L (HSum □)) * Atilde * (U : L (HSum □)))
    (B := (star V : L (HSum □)) * Atilde * (V : L (HSum □)))
    (t := (1 / 2 : ℝ))
    hB1_sa hB2_sa (by positivity) (by norm_num) hB1_spec hB2_spec)
have hAtilde_cfc :
  cfcR (□ := HSum □) f Atilde =
    blockDiagonal (□ := □) (cfcR (□ := □) f 0) (cfcR (□ := □) f A) := by
  simpa [Atilde] using
    (cfcR_blockDiagonal (□ := □) (f := f) (A := 0) (B := A) (by simp) hA
      (continuousOn_union_of_subset_Ici (f := f) hcontIci

```

```

(s := spectrum ℝ (0 : L [])) (t := spectrum ℝ A)
spectrum_zero_subset_Ici hAs))
have hUcfc :
  cfcR (□ := HSum □) f ((star U : L (HSum □)) * Atilde * (U : L (HSum □)
    (star U : L (HSum □)) * cfcR (□ := HSum □) f Atilde * (U : L (HSum
  simp [mul_assoc] using
  cfcR_conj_unitary_on (□ := HSum □) (s := Set.Ici (0 : ℝ)) (f := f) hC
    hAtilde_spec U hAtilde_sa
have hVcfc :
  cfcR (□ := HSum □) f ((star V : L (HSum □)) * Atilde * (V : L (HSum □)
    (star V : L (HSum □)) * cfcR (□ := HSum □) f Atilde * (V : L (HSum
  simp [mul_assoc] using
  cfcR_conj_unitary_on (□ := HSum □) (s := Set.Ici (0 : ℝ)) (f := f) hC
    hAtilde_spec V hAtilde_sa
have hLeftEval :
  cfcR (□ := HSum □) f
    ((1 / 2 : ℝ) • ((star U : L (HSum □)) * Atilde * (U : L (HSum □))
      (1 / 2 : ℝ) • ((star V : L (HSum □)) * Atilde * (V : L (HSum □))
    blockDiagonal (cfcR f (star X * A * X)) (cfcR (□ := □) f (R1 * A *
  have hXAX_nonneg : (0 : L []) ≤ star X * A * X := by
    simp [mul_assoc] using star_left_conjugate_nonneg hA0 X
  have hR1AR1_nonneg : (0 : L []) ≤ R1 * A * R1 := by
    simp [hR1self.star_eq, mul_assoc] using star_right_conjugate_nonneg
  have hXAX_sa : IsSelfAdjoint (star X * A * X) := IsSelfAdjoint.of_nonneg
  have hR1AR1_sa : IsSelfAdjoint (R1 * A * R1) := IsSelfAdjoint.of_nonneg
  have hXAX_spec : spectrum ℝ (star X * A * X) ⊆ Set.Ici (0 : ℝ) :=
    spectrum_Ici_of_nonneg (□ := □) hXAX_nonneg
  have hR1AR1_spec : spectrum ℝ (R1 * A * R1) ⊆ Set.Ici (0 : ℝ) :=
    spectrum_Ici_of_nonneg (□ := □) hR1AR1_nonneg
rw [hmid_block]
refine cfcR_blockDiagonal (□ := □) (f := f) (A := star X * A * X) (B :=
  hXAX_sa hR1AR1_sa ?_
exact continuousOn_union_of_subset_Ici (f := f) hcontIci hXAX_spec hR1A
have hRightEval :
  ((1 / 2 : ℝ) • cfcR (□ := HSum □) f ((star U : L (HSum □)) * Atilde *
    (1 / 2 : ℝ) • cfcR (□ := HSum □) f ((star V : L (HSum □)) * Atilde
    blockDiagonal (□ := □)
      (star X * cfcR (□ := □) f A * X + (f 0) • (R0 * R0))
      ((R1 * cfcR (□ := □) f A * R1) + (f 0) • (X * star X)) := by
rw [hUcfc, hVcfc, hAtilde_cfc, cfcR_zero]
rw [hU_block, hV_block, blockDiagonal_eq_blockOp]
rw [blockOp_star, blockOp_star]
simp_rw [mul_assoc]
rw [blockOp_mul, blockOp_mul, blockOp_mul, blockOp_mul]
rw [blockOp_smulR, blockOp_smulR, blockOp_add, blockDiagonal_eq_blockOp]
have hTopLeft :

```

1.119.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN/OPERATORINEQUALITYIIMPIV.LE

```

(2-1 : ℝ) • (star X * (cfcR (□ := □) f A * X)) +
((2-1 : ℝ) • (star X * (cfcR (□ := □) f A * X)) +
(-(2-1 : ℝ) • Complex.I • f 0 • Complex.I • (R0 * R0)) +
(-(2-1 : ℝ) • Complex.I • f 0 • Complex.I • (R0 * R0))) =
star X * (cfcR (□ := □) f A * X) + (f 0) • (R0 * R0) := by
simpa using
rightEval_topLeft_scalar (□ := □) (r := f 0) (R0 := R0) (X := X)
(T := cfcR (□ := □) f A)
have hBottomRight :
(2-1 * f 0) • (X * star X) +
((2-1 * f 0) • (X * star X) +
((2-1 : ℝ) • (R1 * (cfcR (□ := □) f A * R1)) +
(2-1 : ℝ) • (R1 * (cfcR (□ := □) f A * R1)))) =
(R1 * cfcR (□ := □) f A * R1) + (f 0) • (X * star X) := by
simpa [mul_assoc] using
rightEval_bottomRight_scalar (□ := □) (r := f 0) (R1 := R1) (X := X)
(T := cfcR (□ := □) f A)
congr 1
· simpa [hR0self.star_eq, Algebra.algebraMap_eq_smul_one, mul_assoc,
add_assoc, add_left_comm, add_comm] using hTopLeft
· simp [Algebra.algebraMap_eq_smul_one]
abel
· simp [Algebra.algebraMap_eq_smul_one]
abel
· simpa [hR1self.star_eq, Algebra.algebraMap_eq_smul_one, Complex.I_mul_I, smul_
mul_assoc, add_assoc, add_left_comm, add_comm] using hBottomRight
have hcore :
blockDiagonal (□ := □) (cfcR (□ := □) f (star X * A * X)) (cfcR (□ := □) f (R1
blockDiagonal (□ := □)
(star X * cfcR (□ := □) f A * X + (f 0) • (R0 * R0))
((R1 * cfcR (□ := □) f A * R1) + (f 0) • (X * star X))) := by
rw [hLeftEval, hRightEval] at hmid_conv
exact hmid_conv
have hterm_nonpos : (f 0) • (R0 * R0) ≤ (0 : L □) := by
have hR0sq_nonneg : (0 : L □) ≤ R0 * R0 := by
simpa [hR0self.star_eq] using star_mul_self_nonneg R0
have hneg : (0 : L □) ≤ (- (f 0)) • (R0 * R0) := by
exact smul_nonneg (by linarith [hf0]) hR0sq_nonneg
exact (neg_nonneg.mp (by simpa [neg_smul] using hneg))
have htop :
cfcR (□ := □) f (star X * A * X) ≤ star X * cfcR (□ := □) f A * X + (f 0) • (R
exact blockDiagonal_le_left (□ := □) hcore
have hdrop :
star X * cfcR (□ := □) f A * X + (f 0) • (R0 * R0) ≤ star X * cfcR (□ := □) f
simpa [add_comm, add_left_comm, add_assoc] using
add_le_add_left hterm_nonpos (star X * cfcR (□ := □) f A * X)

```

```

exact htop.trans hdrop

set_option maxHeartbeats 2000000 in
-- The global proof repeats the same block-matrix normalization and unitary
theorem theorem_2_5_2_i_all_imp_iv {f : ℝ → ℝ} (hf : CondIAll.{u} f) :
  CondIV (□ := □) f := by
  rcases hf with ⟨hconvAll, hf0⟩
  intro A X hA hAs hX
  have hconv : OperatorConvex (□ := □) f := hconvAll (K := □)
  let S : L (HSum □) := blockSwap (□ := □) X
  have hSsa : IsSelfAdjoint S := by
    change star S = S
    simp [S] using blockSwap_star (□ := □) X
  have hSnorm : ||S|| ≤ 1 := by
    simp [S] using blockSwap_norm_le_one (□ := □) X hX
  let I : Algebra ℝ (L (HSum □)) := by
    infer_instance
  have hU_mem : S + Complex.I • CFC.sqrt (1 - S ^ 2) ∈ unitary (L (HSum □))
  exact IsSelfAdjoint.self_add_I_smul_cfcSqrt_sub_sq_mem_unitary S hSsa h
  let U : unitary (L (HSum □)) :=
    (S + Complex.I • CFC.sqrt (1 - S ^ 2), hU_mem)
  let V : unitary (L (HSum □)) := star U
  let Atilde : L (HSum □) := blockDiagonal (□ := □) 0 A
  let I : Nontrivial (L (HSum □)) := nontrivial_hsumL (□ := □)
  have hconv₂ : OperatorConvex (□ := HSum □) f := hconvAll (K := HSum □)
  have hcont₂ : ContinuousOn f Set.univ :=
    operatorConvex_continuousOn_univ (□ := HSum □) hconv₂
  have hR0nonneg : (0 : L □) ≤ 1 - star X * X := sub_nonneg.mpr (star_mul_1)
  have hR1nonneg : (0 : L □) ≤ 1 - X * star X := sub_nonneg.mpr (mul_star_1)
  let R0 : L □ := CFC.sqrt (1 - star X * X)
  let R1 : L □ := CFC.sqrt (1 - X * star X)
  have hR0sa : IsSelfAdjoint (1 - star X * X) := by
    change star (1 - star X * X) = 1 - star X * X
    simp
  have hR1sa : IsSelfAdjoint (1 - X * star X) := by
    change star (1 - X * star X) = 1 - X * star X
    simp
  have hSsq : S ^ 2 = blockDiagonal (□ := □) (star X * X) (X * star X) := b
    simp [pow_two, S] using blockSwap_sq (□ := □) X
  have hOneMinusSq :
    1 - S ^ 2 = blockDiagonal (□ := □) (1 - star X * X) (1 - X * star X)
    rw [hSsq]
    refine blockOp_ext (□ := □) ?_ ?_
    · intro z
      simp [sub_eq_add_neg, add_comm]
    · intro z

```

1.119.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSENOPERATORINEQUALITYIIMPIV.LE

```

    simp [sub_eq_add_neg, add_comm]
  have hOneMinusSqNonneg : (0 : L (HSum [])) ≤ 1 - S ^ 2 := by
    have hdiag : (0 : L (HSum [])) ≤
      blockDiagonal (□ := □) (1 - star X * X) (1 - X * star X) :=
      blockDiagonal_nonneg (□ := □) hR0nonneg hR1nonneg
    simp [hOneMinusSq] using hdiag
  have hRblock : CFC.sqrt (1 - S ^ 2) = blockDiagonal (□ := □) R0 R1 := by
    rw [hOneMinusSq]
    simp [R0, R1]
    simp using
      (sqrt_blockDiagonal_of_nonneg (□ := □) (A := 1 - star X * X) (B := 1 - X * star X)
        hR0sa hR1sa hR0nonneg hR1nonneg)
  have hR0self : IsSelfAdjoint R0 := by
    have h : IsSelfAdjoint (CFC.sqrt (1 - star X * X)) :=
      (CFC.sqrt_nonneg (1 - star X * X)).isSelfAdjoint
    simp [R0] using h
  have hR1self : IsSelfAdjoint R1 := by
    have h : IsSelfAdjoint (CFC.sqrt (1 - X * star X)) :=
      (CFC.sqrt_nonneg (1 - X * star X)).isSelfAdjoint
    simp [R1] using h
  have hU_block :
    (U : L (HSum [])) = blockOp (□ := □) (Complex.I • R0) (star X) X (Complex.I • R1)
    change S + Complex.I • CFC.sqrt (1 - S ^ 2) = _
    rw [hRblock]
    simp [S] using blockSwap_add_I_smul_blockDiagonal (□ := □) X R0 R1
  have hV_block :
    (V : L (HSum [])) = blockOp (□ := □) (-Complex.I • R0) (star X) X (-Complex.I • R1) := by
    change star (U : L (HSum [])) = _
    rw [hU_block]
    ext z i
    fin_cases i <|>
    simp [blockOp_star, hR0self.star_eq, hR1self.star_eq]
  have hB1_block :
    (star U : L (HSum [])) * Atilde * (U : L (HSum [])) =
      blockOp (□ := □)
        (star X * A * X)
        (star X * A * (Complex.I • R1))
        ((-Complex.I • R1) * A * X)
        ((-Complex.I • R1) * A * (Complex.I • R1)) := by
    rw [show (star U : L (HSum [])) = (V : L (HSum [])) by rfl, hV_block, hU_block]
    simp [Atilde, mul_assoc] using
      (blockOp_mul_blockDiagonal_zero_right (□ := □)
        (-Complex.I • R0) (star X) X (-Complex.I • R1) A
        (Complex.I • R0) (star X) X (Complex.I • R1))
  have hB2_block :

```

```

(star V : L (HSum [])) * Atilde * (V : L (HSum [])) =
  blockOp (□ := □)
    (star X * A * X)
    (star X * A * (-Complex.I • R1))
    ((Complex.I • R1) * A * X)
    ((Complex.I • R1) * A * (-Complex.I • R1)) := by
  rw [show (star V : L (HSum [])) = (U : L (HSum [])) by simp [V], hU_block]
  simp [Atilde, mul_assoc] using
    (blockOp_mul_blockDiagonal_zero_right (□ := □)
      (Complex.I • R0) (star X) X (Complex.I • R1) A
      (-Complex.I • R0) (star X) X (-Complex.I • R1))
  have hmid_block :
    (1 / 2 : ℝ) • ((star U : L (HSum [])) * Atilde * (U : L (HSum [])) ) +
      (1 / 2 : ℝ) • ((star V : L (HSum [])) * Atilde * (V : L (HSum [])))
    blockDiagonal (□ := □) (star X * A * X) (R1 * A * R1) := by
  rw [hB1_block, hB2_block]
  rw [blockOp_smulR, blockOp_smulR, blockOp_add, blockDiagonal_eq_blockOp]
  congr 1
  · have hhalf : (2-1 + 2-1 : ℝ) = (1 : ℝ) := by norm_num
    calc
      (1 / 2 : ℝ) • (star X * A * X) + (1 / 2 : ℝ) • (star X * A * X)
      = (2-1 + 2-1 : ℝ) • (star X * (A * X)) := by
        simp [add_smul, mul_assoc]
      _ = (1 : ℝ) • (star X * (A * X)) := by rw [hhalf]
      _ = star X * (A * X) := by simp
      _ = star X * A * X := by simp [mul_assoc]
  · simp [mul_assoc]
  · simp [mul_assoc]
  · have hhalf : (2-1 + 2-1 : ℝ) = (1 : ℝ) := by norm_num
    calc
      (1 / 2 : ℝ) • (-Complex.I • R1 * A * (Complex.I • R1)) +
        (1 / 2 : ℝ) • (Complex.I • R1 * A * (-Complex.I • R1))
      = (2-1 + 2-1 : ℝ) • (R1 * (A * R1)) := by
        simp [Complex.I_mul_I, smul_smul, add_smul, mul_assoc]
      _ = (1 : ℝ) • (R1 * (A * R1)) := by rw [hhalf]
      _ = R1 * A * R1 := by simp [mul_assoc]
  have hmid_conv :
    cfcR (□ := HSum []) f
      ((1 / 2 : ℝ) • ((star U : L (HSum [])) * Atilde * (U : L (HSum [])))
        (1 / 2 : ℝ) • ((star V : L (HSum [])) * Atilde * (V : L (HSum [])))
        ((1 / 2 : ℝ) • cfcR f ((star U : L (HSum [])) * Atilde * (U : L (HSum [])))
          (1 / 2 : ℝ) • cfcR f ((star V : L (HSum [])) * Atilde * (V : L (HSum []))))
  have hhalf : (1 - (2-1 : ℝ)) = (2-1 : ℝ) := by norm_num
  simp [hhalf] using
    (hconv₂
      (A := (star U : L (HSum [])) * Atilde * (U : L (HSum [])))

```


1.119.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN/OPERATORINEQUALITYIIMPIV.LE

```

      (B := (star V : L (HSum [])) * Atilde * (V : L (HSum [])))
      (t := (1 / 2 : ℝ))
      (by positivity) (by norm_num))
have hAtilde_sa : IsSelfAdjoint Atilde := by
  simpa [Atilde] using blockDiagonal_selfAdjoint (□ := □) (hA := by simp) hA
have hAtilde_cfc :
  cfcR (□ := HSum []) f Atilde =
    blockDiagonal (□ := □) (cfcR (□ := □) f 0) (cfcR (□ := □) f A) := by
  simpa [Atilde] using
    (cfcR_blockDiagonal (□ := □) (f := f) (A := 0) (B := A) (by simp) hA
    ((operatorConvex_continuousOn_univ (□ := □) hconv).mono (by intro x hx; simp
have hUcfc :
  cfcR (□ := HSum []) f ((star U : L (HSum [])) * Atilde * (U : L (HSum [])) ) =
    (star U : L (HSum [])) * cfcR (□ := HSum []) f Atilde * (U : L (HSum [])) := by
  simpa [mul_assoc] using cfcR_conj_unitary (□ := HSum []) f hcont₂ U Atilde hAtilde
have hVcfc :
  cfcR (□ := HSum []) f ((star V : L (HSum [])) * Atilde * (V : L (HSum [])) ) =
    (star V : L (HSum [])) * cfcR (□ := HSum []) f Atilde * (V : L (HSum [])) := by
  simpa [mul_assoc] using cfcR_conj_unitary (□ := HSum []) f hcont₂ V Atilde hAtilde
have hLeftEval :
  cfcR (□ := HSum []) f
    ((1 / 2 : ℝ) • ((star U : L (HSum [])) * Atilde * (U : L (HSum []))) +
    (1 / 2 : ℝ) • ((star V : L (HSum [])) * Atilde * (V : L (HSum [])))) =
    blockDiagonal (cfcR f (star X * A * X)) (cfcR f (R1 * A * R1)) := by
have hXAX_sa : IsSelfAdjoint (star X * A * X) := by
  change star (star X * A * X) = star X * A * X
  simp [hA.star_eq, mul_assoc]
have hR1AR1_sa : IsSelfAdjoint (R1 * A * R1) := by
  change star (R1 * A * R1) = R1 * A * R1
  simp [hR1self.star_eq, hA.star_eq, mul_assoc]
rw [hmid_block]
refine cfcR_blockDiagonal (f := f) (A := star X * A * X) (B := R1 * A * R1) hXAX
  · exact (operatorConvex_continuousOn_univ (□ := □) hconv).mono (by intro x hx; s
have hRightEval :
  ((1 / 2 : ℝ) • cfcR (□ := HSum []) f ((star U : L (HSum [])) * Atilde * (U : L (
  (1 / 2 : ℝ) • cfcR (□ := HSum []) f ((star V : L (HSum [])) * Atilde * (V : L
  blockDiagonal (□ := □)
    (star X * cfcR (□ := □) f A * X + (f 0) • (R0 * R0))
    ((R1 * cfcR (□ := □) f A * R1) + (f 0) • (X * star X)) := by
rw [hUcfc, hVcfc, hAtilde_cfc, cfcR_zero]
rw [hU_block, hV_block, blockDiagonal_eq_blockOp]
rw [blockOp_star, blockOp_star]
simp_rw [mul_assoc]
rw [blockOp_mul, blockOp_mul, blockOp_mul, blockOp_mul]
rw [blockOp_smulR, blockOp_smulR, blockOp_add, blockDiagonal_eq_blockOp]
have hTopLeft :

```

```

(2-1 : ℝ) • (star X * (cfcR (□ := □) f A * X)) +
((2-1 : ℝ) • (star X * (cfcR (□ := □) f A * X)) +
(-(2-1 : ℝ) • Complex.I • f 0 • Complex.I • (R0 * R0)) +
(-(2-1 : ℝ) • Complex.I • f 0 • Complex.I • (R0 * R0))) =
star X * (cfcR (□ := □) f A * X) + (f 0) • (R0 * R0) := by
simpa using
rightEval_topLeft_scalar (□ := □) (r := f 0) (R0 := R0) (X := X)
(T := cfcR (□ := □) f A)
have hBottomRight :
(2-1 * f 0) • (X * star X) +
((2-1 * f 0) • (X * star X) +
((2-1 : ℝ) • (R1 * (cfcR (□ := □) f A * R1)) +
(2-1 : ℝ) • (R1 * (cfcR (□ := □) f A * R1)))) =
(R1 * cfcR (□ := □) f A * R1) + (f 0) • (X * star X) := by
simpa [mul_assoc] using
rightEval_bottomRight_scalar (□ := □) (r := f 0) (R1 := R1) (X := X)
(T := cfcR (□ := □) f A)
congr 1
· simpa [hR0self.star_eq, Algebra.algebraMap_eq_smul_one, mul_assoc,
add_assoc, add_left_comm, add_comm] using hTopLeft
· simp [Algebra.algebraMap_eq_smul_one]
abel
· simp [Algebra.algebraMap_eq_smul_one]
abel
· simpa [hR1self.star_eq, Algebra.algebraMap_eq_smul_one, Complex.I_mul
mul_assoc, add_assoc, add_left_comm, add_comm] using hBottomRight
have hcore :
blockDiagonal (□ := □) (cfcR (□ := □) f (star X * A * X)) (cfcR (□ :=
blockDiagonal (□ := □)
(star X * cfcR (□ := □) f A * X + (f 0) • (R0 * R0))
((R1 * cfcR (□ := □) f A * R1) + (f 0) • (X * star X))) := by
rw [hLeftEval, hRightEval] at hmid_conv
exact hmid_conv
have hterm_nonpos : (f 0) • (R0 * R0) ≤ (0 : L □) := by
have hR0sq_nonneg : (0 : L □) ≤ R0 * R0 := by
simpa [hR0self.star_eq] using star_mul_self_nonneg R0
have hneg : (0 : L □) ≤ (- (f 0)) • (R0 * R0) := by
exact smul_nonneg (by linarith [hf0]) hR0sq_nonneg
exact (neg_nonneg.mp (by simpa [neg_smul] using hneg))
have htop :
cfcR (□ := □) f (star X * A * X) ≤ star X * cfcR (□ := □) f A * X + (
exact blockDiagonal_le_left (□ := □) hcore
have hdrop :
star X * cfcR (□ := □) f A * X + (f 0) • (R0 * R0) ≤ star X * cfcR (□
simpa [add_comm, add_left_comm, add_assoc] using
add_le_add_left hterm_nonpos (star X * cfcR f A * X)

```

1.120.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSENOPERATORINEQUALITYIVTOV.LEA

exact htop.trans hdrop

end Theorem252

end JensenOperatorInequality

1.120 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperato

/-

Copyright (c) 2026 Hayata Yamasaki. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Kei Tsukamoto, Kento Mori, Hayata Yamasaki

-/

module

public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.BlockDiagonal

public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.LownerHeinzTheorem

public import Mathlib.Analysis.CStarAlgebra.Unitary.Span

@[expose] public section

namespace JensenOperatorInequalityScratch

universe u

open LownerHeinzTheorem

open JensenOperatorInequality

section Theorem252

variable { α : Type u}

variable [NormedAddCommGroup α] [InnerProductSpace $\mathbb{C}$ α] [CompleteSpace α]

variable [Nontrivial α]

set_option synthInstance.maxHeartbeats 400000 in

noncomputable local instance : ContinuousFunctionalCalculus $\mathbb{C}$ (L α $\times$ L α) IsStarNorm

IsStarNormal.instContinuousFunctionalCalculus

set_option synthInstance.maxHeartbeats 400000 in

noncomputable local instance : ContinuousFunctionalCalculus $\mathbb{R}$ (L α $\times$ L α) IsSelfAdjo

IsSelfAdjoint.instContinuousFunctionalCalculus

set_option synthInstance.maxHeartbeats 400000 in

noncomputable local instance : NonnegSpectrumClass $\mathbb{R}$ (L (HSum α)) := inferInstance

/-- Local copy of condition (iv) for fast iteration on $\text{'(iv)} \rightarrow \text{'(v)}$. -

```

/
def CondIV (f : ℝ → ℝ) : Prop :=
  ∀ [A X : L [], IsSelfAdjoint A → spectrum ℝ A ⊆ Set.Ici (0 : ℝ) → ||X|| ≤ 1
    cfcR (· := ·) f (star X * A * X) ≤ star X * cfcR (· := ·) f A * X

omit [NormedAddCommGroup []] [InnerProductSpace ℂ []] [CompleteSpace []] [No
/--
Uniform version of Condition (iv), with the Hilbert space arbitrary in the
This scratch copy follows the same `...All` naming convention as the main J
-/
def CondIVAll (f : ℝ → ℝ) : Prop :=
  ∀ {K : Type u}
    [NormedAddCommGroup K] [InnerProductSpace ℂ K] [CompleteSpace K]
    [Nontrivial K],
    CondIV (· := K) f

/-- Local copy of condition (v) for fast iteration on `(iv) → (v)`. -
/
def CondV (f : ℝ → ℝ) : Prop :=
  ∀ [A B X Y : L []],
    IsSelfAdjoint A → IsSelfAdjoint B →
    spectrum ℝ A ⊆ Set.Ici (0 : ℝ) → spectrum ℝ B ⊆ Set.Ici (0 : ℝ) →
    star X * X + star Y * Y ≤ (1 : L []) →
    cfcR (· := ·) f (star X * A * X + star Y * B * Y) ≤
    star X * cfcR (· := ·) f A * X + star Y * cfcR (· := ·) f B * Y

/-- `L (HSum [])` is nontrivial once `L []` is. -/
private theorem nontrivial_hsumL : Nontrivial (L (HSum [])) := by
  have h_not_sub : ¬ Subsingleton [] := by
    intro hsub
    letI : Subsingleton [] := hsub
    letI : Subsingleton (L []) := by infer_instance
    exact (not_nontrivial_iff_subsingleton.mpr (by infer_instance))
      (inferInstance : Nontrivial (L []))
  have hH_nontriv : Nontrivial [] := (not_subsingleton_iff_nontrivial.mp h_not_sub)
  letI : Nontrivial [] := hH_nontriv
  rcases exists_pair_ne [] with ⟨x, y, hxy⟩
  let w : [] := x - y
  have hw : w ≠ 0 := sub_ne_zero.mpr hxy
  have hdiag_ne_zero : (blockDiagonal (· := ·) (1 : L []) 0 : L (HSum [])) ≠ 0
  intro h0
  have hz :
    blockDiagonal (· := ·) (1 : L []) 0 (hsumIncl [] 0 w) = 0 := by
    simp [h0]
  have hw0 : w = 0 := by
    have hz0 := congrArg (fun z : HSum [] => hsumProj [] 0 z) hz

```

1.120.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSENOPERATORINEQUALITYIVTOV.LEA

```

    simp [blockDiagonal] using hz0
    exact hw hw0
    exact {0, blockDiagonal (□ := □) (1 : L □) 0, hdiag_ne_zero.symm}

omit [Nontrivial □] in
private lemma blockDiagonal_selfAdjoint {A B : L □}
  (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B) :
  IsSelfAdjoint (blockDiagonal (□ := □) A B) := by
  change star (blockDiagonal (□ := □) A B) = blockDiagonal (□ := □) A B
  simp [blockDiagonal_star, hA.star_eq, hB.star_eq]

omit [Nontrivial □] in
private lemma blockDiagonal_eq_blockOp (A B : L □) :
  blockDiagonal (□ := □) A B = blockOp (□ := □) A 0 0 B := by
  ext z i
  fin_cases i <=> simp [blockDiagonal, blockOp]

-- Multiplication of generic block operators is elaboration-
heavy even in the scratch file.
omit [CompleteSpace □] [Nontrivial □] in
set_option maxHeartbeats 400000 in
-- The generic `blockOp` product expands into large block normal forms.
private lemma blockOp_mul (A00 A01 A10 A11 B00 B01 B10 B11 : L □) :
  blockOp (□ := □) A00 A01 A10 A11 * blockOp (□ := □) B00 B01 B10 B11 =
    blockOp (□ := □)
      (A00 * B00 + A01 * B10)
      (A00 * B01 + A01 * B11)
      (A10 * B00 + A11 * B10)
      (A10 * B01 + A11 * B11) := by
  refine blockOp_ext (□ := □) ?_ ?_
  · intro z
    simp [ContinuousLinearMap.mul_def, add_left_comm, add_comm]
  · intro z
    simp [ContinuousLinearMap.mul_def, add_left_comm, add_comm]

set_option synthInstance.maxHeartbeats 400000 in
set_option maxHeartbeats 800000 in
omit [Nontrivial □] in
private lemma cfcR_blockDiagonal (f : ℝ → ℝ)
  (A B : L □) (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B)
  (hcont : ContinuousOn f (spectrum ℝ A ∪ spectrum ℝ B)) :
  cfcR (□ := HSum □) f (blockDiagonal (□ := □) A B) =
    blockDiagonal (□ := □) (cfcR (□ := □) f A) (cfcR (□ := □) f B) := by
  let φ : (L □ × L □) →a [ℝ] L (HSum □) := blockDiagonalHom (□ := □)
  have hφ : Continuous φ := by
    change Continuous (fun p : L □ × L □ => blockDiagonal (□ := □) p.1 p.2)

```

```

change Continuous (fun p : L  $\square$   $\times$  L  $\square$  =>
  hsumIncl  $\square$  0  $\circ$  L p.1  $\circ$  L hsumProj  $\square$  0 + hsumIncl  $\square$  1  $\circ$  L p.2  $\circ$  L hsumProj)
fun_prop
have hpair : IsSelfAdjoint (A, B) := by
  change star (A, B) = (A, B)
  ext <;> simp [hA.star_eq, hB.star_eq]
have hpair' : IsSelfAdjoint ( $\phi$  (A, B)) := hpair.map  $\phi$ 
have hmap := StarAlgHom.map_cfc ( $\phi$  :=  $\phi$ ) (f := f) (a := (A, B))
  (hf := by simp [Prod.spectrum_eq] using hcont)
  (h $\phi$  := h $\phi$ ) (ha := hpair) (h $\phi$ a := hpair')
have hprod :
  cfc (R :=  $\mathbb{R}$ ) (A := L  $\square$   $\times$  L  $\square$ ) (p := IsSelfAdjoint) f (A, B) =
    (cfcR ( $\square$  :=  $\square$ ) f A, cfcR ( $\square$  :=  $\square$ ) f B) := by
  simp [cfcR] using
    (cfc_map_prod (R :=  $\mathbb{R}$ ) (S :=  $\mathbb{R}$ )
      (A := L  $\square$ ) (B := L  $\square$ )
      (pab := IsSelfAdjoint) (pa := IsSelfAdjoint) (pb := IsSelfAdjoint)
      f A B
      (hf := hcont)
      (hab := hpair) (ha := hA) (hb := hB))
calc
  cfcR ( $\square$  := HSum  $\square$ ) f (blockDiagonal ( $\square$  :=  $\square$ ) A B)
    = cfc (R :=  $\mathbb{R}$ ) (A := L (HSum  $\square$ )) (p := IsSelfAdjoint) f ( $\phi$  (A, B))
      simp [cfcR,  $\phi$ ]
_ =  $\phi$  (cfc (R :=  $\mathbb{R}$ ) (A := L  $\square$   $\times$  L  $\square$ ) (p := IsSelfAdjoint) f (A, B)) :=
  simp using hmap.symm
_ =  $\phi$  (cfcR ( $\square$  :=  $\square$ ) f A, cfcR ( $\square$  :=  $\square$ ) f B) := by
  rw [hprod]
_ = blockDiagonal ( $\square$  :=  $\square$ ) (cfcR ( $\square$  :=  $\square$ ) f A) (cfcR ( $\square$  :=  $\square$ ) f B) := by
  simp [ $\phi$ , blockDiagonalHom]

omit [Nontrivial  $\square$ ] in
private lemma blockDiagonal_le_left {A0 A1 B0 B1 : L  $\square$ }
  (h : blockDiagonal ( $\square$  :=  $\square$ ) A0 A1  $\leq$  blockDiagonal ( $\square$  :=  $\square$ ) B0 B1) :
  A0  $\leq$  B0 := by
  have hnonneg : 0  $\leq$  blockDiagonal ( $\square$  :=  $\square$ ) (B0 - A0) (B1 - A1) := by
    have hsub :
      blockDiagonal ( $\square$  :=  $\square$ ) B0 B1 - blockDiagonal ( $\square$  :=  $\square$ ) A0 A1 =
        blockDiagonal ( $\square$  :=  $\square$ ) (B0 - A0) (B1 - A1) := by
      refine blockOp_ext ( $\square$  :=  $\square$ ) ?_ ?_
      · intro z
        simp [sub_eq_add_neg]
      · intro z
        simp [sub_eq_add_neg]
    exact hsub  $\triangleright$  sub_nonneg.mpr h
  have hpos :

```

1.120.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/JENSEN/OPERATORINEQUALITYIVTOV.LEA

```

    (blockDiagonal (□ := □) (B0 - A0) (B1 - A1)).IsPositive :=
    (ContinuousLinearMap.nonneg_iff_isPositive _).1 hnonneg
have hleftPos : (B0 - A0).IsPositive := by
  rw [ContinuousLinearMap.isPositive_iff_complex]
  intro x
  have hx :=
    (ContinuousLinearMap.isPositive_iff_complex
      (blockDiagonal (□ := □) (B0 - A0) (B1 - A1))).1 hpos (hsumIncl □ 0 x)
  simp [blockDiagonal, hsumProj, hsumIncl, hsumEquiv, PiLp.inner_apply] using hx
  exact sub_nonneg.mp ((ContinuousLinearMap.nonneg_iff_isPositive _).2 hleftPos)

-- Scratch theorem for fast feedback while formalizing Theorem 2.5.2 `(iv) → (v)`.
-- This file intentionally avoids importing the heavy `(i) → (iv)` proof.
set_option synthInstance.maxHeartbeats 400000 in
set_option maxHeartbeats 3000000 in
-- The block-matrix reduction creates large normalization goals in this scratch file
theorem theorem_2_5_2_iv_imp_v {f : ℝ → ℝ} (hiv : CondIVAll.{u} f)
  (hcont : ContinuousOn f Set.univ) :
  CondV (□ := □) f := by
  intro A B X Y hA hB hAs hBs hXY
  have hA0 : (0 : L □) ≤ A := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) A (ha := hA)).2 ?_
    intro x hx
    simp [Set.Ici] using hAs hx
  have hB0 : (0 : L □) ≤ B := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B (ha := hB)).2 ?_
    intro x hx
    simp [Set.Ici] using hBs hx
  let Atilde : L (HSum □) := blockDiagonal (□ := □) A B
  let Xtilde : L (HSum □) := blockOp (□ := □) X 0 Y 0
  letI : Nontrivial (L (HSum □)) := nontrivial_hsumL (□ := □)
  have hAtilde_sa : IsSelfAdjoint Atilde := by
    simp [Atilde] using blockDiagonal_selfAdjoint (□ := □) hA hB
  have hAtilde0 : (0 : L (HSum □)) ≤ Atilde := by
    simp [Atilde] using blockDiagonal_nonneg (□ := □) hA0 hB0
  have hAtilde_spec : spectrum ℝ Atilde ⊆ Set.Ici (0 : ℝ) := by
    exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) Atilde (ha := hAtilde0))
  have hXtilde_star_mul :
    star Xtilde * Xtilde =
      blockDiagonal (□ := □) (star X * X + star Y * Y) 0 := by
  rw [show star Xtilde = blockOp (□ := □) (star X) (star Y) 0 0 by
    simp [Xtilde]]
  rw [show Xtilde = blockOp (□ := □) X 0 Y 0 by rfl]
  rw [blockOp_mul, blockDiagonal_eq_blockOp]
  simp
  have hXtilde_star_mul_le : star Xtilde * Xtilde ≤ (1 : L (HSum □)) := by

```

```

have hblock_nonneg :
  (0 : L (HSum [])) ≤
    blockDiagonal ([] := []) (1 - (star X * X + star Y * Y)) (1 : L [])
  refine blockDiagonal_nonneg ([] := []) ?_ ?_
  · exact sub_nonneg.mpr hXY
  · simp
have hsub :
  (1 : L (HSum [])) - blockDiagonal ([] := []) (star X * X + star Y * Y)
    blockDiagonal ([] := []) (1 - (star X * X + star Y * Y)) (1 : L [])
  refine blockOp_ext ([] := []) ?_ ?_
  · intro z
  · simp [sub_eq_add_neg]
  · intro z
  · simp [sub_eq_add_neg]
have hblock :
  blockDiagonal ([] := []) (star X * X + star Y * Y) 0 ≤ (1 : L (HSum []))
  exact sub_nonneg.mpr (by simpa [hsub] using hblock_nonneg)
  simpa [hXtilde_star_mul] using hblock
have hXtilde_star_mul_nonneg : (0 : L (HSum [])) ≤ star Xtilde * Xtilde :=
  simp
have hXtilde_norm : ||Xtilde|| ≤ 1 := by
  have hnormSq : ||star Xtilde * Xtilde|| ≤ 1 :=
    (CStarAlgebra.norm_le_one_iff_of_nonneg _ hXtilde_star_mul_nonneg).2
  have hnormSq' : ||Xtilde|| * ||Xtilde|| ≤ 1 := by
    simpa [CStarRing.norm_star_mul_self] using hnormSq
  have hsq : ||Xtilde|| ^ 2 ≤ 1 := by
    simpa [pow_two] using hnormSq'
  nlinarith [norm_nonneg Xtilde]
have hiv_hsum : CondIV ([] := HSum []) f := @hiv (HSum []) _ _ _
have hcore := hiv_hsum (A := Atilde) (X := Xtilde) hAtilde_sa hAtilde_spe
have hsum_sa : IsSelfAdjoint (star X * A * X + star Y * B * Y) := by
  change star (star X * A * X + star Y * B * Y) = star X * A * X + star Y * B * Y
  simp [hA.star_eq, hB.star_eq, mul_assoc]
have hmul_block :
  star Xtilde * Atilde * Xtilde =
    blockDiagonal ([] := []) (star X * A * X + star Y * B * Y) 0 := by
  rw [show star Xtilde = blockOp ([] := []) (star X) (star Y) 0 0 by
    simp [Xtilde]]
  rw [show Atilde = blockOp ([] := []) A 0 0 B by
    simpa [Atilde] using blockDiagonal_eq_blockOp ([] := []) A B]
  rw [show Xtilde = blockOp ([] := []) X 0 Y 0 by rfl]
  rw [blockOp_mul, blockOp_mul, blockDiagonal_eq_blockOp]
  congr 1 <=> simp [mul_assoc]
have hAtilde_cfc :
  cfcR ([] := HSum []) f Atilde =
    blockDiagonal (cfcR ([] := []) f A) (cfcR ([] := []) f B) := by

```


1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDOTRACE.LEAN

```

simp [Atilde] using
  cfcR_blockDiagonal (□ := □) (f := f) A B hA hB
  (hcont.mono (by intro x hx; simp))
have hright_block :
  star Xtilde * cfcR (□ := HSum □) f Atilde * Xtilde =
    blockDiagonal (□ := □) (star X * cfcR f A * X + star Y * cfcR f B * Y) 0 :=
rw [hAtilde_cfc]
rw [show star Xtilde = blockOp (□ := □) (star X) (star Y) 0 0 by
  simp [Xtilde]]
rw [show Xtilde = blockOp (□ := □) X 0 Y 0 by rfl]
rw [blockDiagonal_eq_blockOp, blockOp_mul, blockOp_mul, blockDiagonal_eq_blockOp]
congr 1 <=> simp [mul_assoc]
have hleft_block :
  cfcR (□ := HSum □) f (star Xtilde * Atilde * Xtilde) =
    blockDiagonal (cfcR f (star X * A * X + star Y * B * Y)) (cfcR f 0) := by
rw [hmul_block]
simp using
  cfcR_blockDiagonal (□ := □) (f := f) (star X * A * X + star Y * B * Y) 0 hsum_
  (hcont.mono (by intro x hx; simp))
rw [hleft_block, hright_block] at hcore
exact blockDiagonal_le_left (□ := □) hcore

```

end Theorem252

end JensenOperatorInequalityScratch

1.121 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LiebAndoTrace

/-

Copyright (c) 2026 Hayata Yamasaki. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Kei Tsukamoto, Kento Mori, Hayata Yamasaki

-/

module

```

public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.OperatorGeometricMe
public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.HilbertSchmidtOpera
public import Mathlib.Analysis.CStarAlgebra.Matrix
public import Mathlib.Analysis.InnerProductSpace.JointEigenspace
public import Mathlib.Analysis.Matrix.HermitianFunctionalCalculus
public import Mathlib.LinearAlgebra.Lagrange
public import Mathlib.LinearAlgebra.Trace

```

@[expose] public section

```

namespace LiebAndoTrace

universe u

open LowerHeinzTheorem
open GeneralizedPerspectiveFunction
open HilbertSchmidtOperatorSpace
open OperatorGeometricMean
open Module.End Polynomial

variable {α : Type u}
variable [NormedAddCommGroup α] [InnerProductSpace ℂ α] [CompleteSpace α]
variable [FiniteDimensional ℂ α] [Nontrivial α]

set_option synthInstance.maxHeartbeats 80000 in
noncomputable local instance : NonnegSpectrumClass ℝ ((L α)mop) := inferInst

set_option synthInstance.maxHeartbeats 80000 in
noncomputable local instance :
  IsometricContinuousFunctionalCalculus ℂ ((L α)mop) IsStarNormal := inferInst

set_option backward.isDefEq.respectTransparency false in
set_option synthInstance.maxHeartbeats 80000 in
noncomputable instance instCFCRealSelfAdjointMop :
  ContinuousFunctionalCalculus ℝ ((L α)mop) IsSelfAdjoint := inferInstance

/-- The real part of the finite-dimensional trace on bounded operators. -
/
noncomputable def traceRe (T : L α) : ℝ :=
  Complex.re (LinearMap.trace ℂ α T.toLinearMap)

/-- Trace functional appearing in Lieb's concavity theorem. -
/
noncomputable def liebTraceMap (s : ℝ) (K : L α) (A B : L α) : ℝ :=
  traceRe (α := α) (A ^ s * star K * B ^ (1 - s) * K)

/-- Trace functional appearing in Lieb's extension theorem. -
/
noncomputable def liebExtensionTraceMap (q p : ℝ) (K : L α) (A B : L α) : ℝ :=
  traceRe (α := α) (A ^ q * star K * B ^ p * K)

/-- Trace functional appearing in Corollary 1.3. -/
noncomputable def liebCorollaryTraceMap (q r : ℝ) (K : L α) (A B : L α) : ℝ :=
  traceRe (α := α) (A ^ q * star K * B ^ (1 - r) * K)

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBMANDOTTRACE.LEAN

-- Trace functional appearing in Ando's convexity theorem. -

/

```
noncomputable def andoTraceMap (q r : ℝ) (K : L ℝ) (A B : L ℝ) : ℝ :=
  traceRe (· := ·) (A ^ q * star K * B ^ (-r) * K)
```

omit [Nontrivial ·] in

private lemma rightMulHS_real_smul_one (r : ℝ) :

```
  rightMulHS (· := ·) (r • (1 : L ℝ)) = r • (1 : L (HSOp ℝ)) := by
  ext T
```

```
  change ofOp (toOp T * ((algebraMap ℝ (L ℝ)) r)) =
    r • ofOp (toOp T * (1 : L ℝ))
```

calc

```
  ofOp (toOp T * ((algebraMap ℝ (L ℝ)) r))
```

```
    = ofOp (((algebraMap ℝ (L ℝ)) r * toOp T) * (1 : L ℝ)) := by
```

```
      have hcomm := Algebra.commutates (R := ℝ) (A := L ℝ) r (toOp T)
```

```
      simpa [mul_assoc] using congrArg (fun X => X * (1 : L ℝ)) hcomm.symm
```

```
  _ = r • ofOp (toOp T) := by
```

```
    delta HSOp
```

```
    rfl
```

```
  _ = r • ofOp (toOp T * (1 : L ℝ)) := by simp
```

omit [Nontrivial ·] in

private lemma rightMulHS_nonneg {A : L ℝ} (hA0 : 0 ≤ A) :

```
  0 ≤ rightMulHS (· := ·) A := by
```

```
  let sqrtA : L ℝ := A ^ ((1 : ℝ) / 2)
```

```
  have hsqrt_sq_pow : sqrtA ^ (2 : ℕ) = A := by
```

calc

```
  sqrtA ^ (2 : ℕ) = sqrtA ^ (2 : ℝ) := by
```

```
    simpa using (CFC.rpow_natCast sqrtA 2).symm
```

```
  _ = A ^ (((1 : ℝ) / 2) * 2) := by
```

```
    simpa [sqrtA] using
```

```
      (CFC.rpow_rpow_of_exponent_nonneg A ((1 : ℝ) / 2) 2
```

```
        (by positivity) (by positivity) (ha := hA0))
```

```
  _ = A ^ (1 : ℝ) := by ring_nf
```

```
  _ = A := by simpa using CFC.rpow_one A
```

```
  have hsqrt_sq : sqrtA * sqrtA = A := by
```

```
    simpa [pow_two] using hsqrt_sq_pow
```

```
  have hsqrt_sa : IsSelfAdjoint sqrtA := IsSelfAdjoint.of_nonneg (by
```

```
    simp [sqrtA])
```

```
  let S : L (HSOp ℝ) := rightMulHS (· := ·) sqrtA
```

```
  have hSstar : star S = S := by
```

```
    change star (rightMulHS (· := ·) sqrtA) = rightMulHS (· := ·) sqrtA
```

```
    simp [hsqrt_sa.star_eq]
```

```
  have hSq : rightMulHS (· := ·) A = star S * S := by
```

calc

```
  rightMulHS (· := ·) A = rightMulHS (· := ·) (sqrtA * sqrtA) := by simp [hsqrt_
```

```

    _ = S * S := by simp [S]
    _ = star S * S := by simp [hSstar]
  simp [hSq]

omit [CompleteSpace  $\alpha$ ] [Nontrivial  $\alpha$ ] in
private lemma rightMulHS_le_rightMulHS {A B : L  $\alpha$ } (hAB : A ≤ B) :
  rightMulHS ( $\alpha$  :=  $\alpha$ ) A ≤ rightMulHS ( $\alpha$  :=  $\alpha$ ) B := by
  have hnonneg : 0 ≤ rightMulHS ( $\alpha$  :=  $\alpha$ ) (B - A) :=
    rightMulHS_nonneg ( $\alpha$  :=  $\alpha$ ) (sub_nonneg.mpr hAB)
  have hsub :
    rightMulHS ( $\alpha$  :=  $\alpha$ ) B - rightMulHS ( $\alpha$  :=  $\alpha$ ) A =
      rightMulHS ( $\alpha$  :=  $\alpha$ ) (B - A) := by
  ext T
  simp [sub_eq_add_neg] using (mul_add (toOp T) B (-A)).symm
  exact sub_nonneg.mp (by simp [hsub] using hnonneg)

private lemma rightMulHS_pdSet {A : L  $\alpha$ } (hA : A ∈ pdSet ( $\alpha$  :=  $\alpha$ )) :
  rightMulHS ( $\alpha$  :=  $\alpha$ ) A ∈ pdSet ( $\alpha$  := HSOp  $\alpha$ ) := by
  rcases hA with ⟨hA_sa, hA_spec⟩
  have hright_sa : IsSelfAdjoint (rightMulHS ( $\alpha$  :=  $\alpha$ ) A) := by
    change star (rightMulHS ( $\alpha$  :=  $\alpha$ ) A) = rightMulHS ( $\alpha$  :=  $\alpha$ ) A
    simp [hA_sa.star_eq]
  letI : Nontrivial (HSOp  $\alpha$ ) := by
    delta HSOp
    infer_instance
  letI : Nontrivial (L (HSOp  $\alpha$ )) := inferInstance
  refine ⟨hright_sa, ?_⟩
  rcases (CFC.exists_pos_algebraMap_le_iff (A := L  $\alpha$ ) (a := A) (ha := hA_sa)
    with ⟨r, hr, hrA⟩
  refine (CFC.exists_pos_algebraMap_le_iff
    (A := L (HSOp  $\alpha$ )) (a := rightMulHS ( $\alpha$  :=  $\alpha$ ) A) (ha := hright_sa)).1 ?_
  refine ⟨r, hr, ?_⟩
  simp [Algebra.algebraMap_eq_smul_one, rightMulHS_real_smul_one ( $\alpha$  :=  $\alpha$ )
    rightMulHS_le_rightMulHS ( $\alpha$  :=  $\alpha$ ) hrA]

private noncomputable def phiK (K : L  $\alpha$ ) (T : L (HSOp  $\alpha$ )) :  $\mathbb{R}$  :=
  Complex.re (inner  $\mathbb{C}$  (ofOp (star K)) (T (ofOp (star K))))

omit [Nontrivial  $\alpha$ ] in
private lemma phiK_nonneg (K : L  $\alpha$ ) {T : L (HSOp  $\alpha$ )} (hT : 0 ≤ T) :
  0 ≤ phiK ( $\alpha$  :=  $\alpha$ ) K T := by
  dsimp [phiK]
  have hpos : T.IsPositive := (ContinuousLinearMap.nonneg_iff_isPositive T)
  have hnonneg : 0 ≤ Complex.re (inner  $\mathbb{C}$  (T (ofOp (star K))) (ofOp (star K)))
    exact ((ContinuousLinearMap.isPositive_iff_complex T).1 hpos (ofOp (star K)))
  have hre :

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBMAN.DOTTRACE.LEAN

```

Complex.re (inner ℂ (ofOp (star K)) (T (ofOp (star K)))) =
  Complex.re (inner ℂ (T (ofOp (star K))) (ofOp (star K))) := by
  simpa using
    (inner_re_symm (· := ℂ) (x := ofOp (star K)) (y := T (ofOp (star K))))
rw [hre]
exact hnonneg

omit [Nontrivial ·] in
private lemma phiK_add (K : L ·) (T S : L (HSOp ·)) :
  phiK (· := ·) K (T + S) = phiK (· := ·) K T + phiK (· := ·) K S := by
  simp [phiK, inner_add_right, Complex.add_re]

omit [Nontrivial ·] in
private lemma phiK_smul (K : L ·) (r : ℝ) (T : L (HSOp ·)) :
  phiK (· := ·) K (r • T) = r * phiK (· := ·) K T := by
  rw [phiK]
  change Complex.re (inner ℂ (ofOp (star K)) (r • T (ofOp (star K)))) =
    r * Complex.re (inner ℂ (ofOp (star K)) (T (ofOp (star K))))
  rw [show inner ℂ (ofOp (star K)) (r • T (ofOp (star K))) =
    (r : ℂ) * inner ℂ (ofOp (star K)) (T (ofOp (star K))) by
    simpa using inner_smul_right (ofOp (star K)) (T (ofOp (star K))) (r : ℂ)]
  simp

omit [Nontrivial ·] in
private lemma phiK_mono (K : L ·) {T S : L (HSOp ·)} (hTS : T ≤ S) :
  phiK (· := ·) K T ≤ phiK (· := ·) K S := by
  have hnonneg :
    0 ≤ S + (-1 : ℝ) • T := by
    simpa [sub_eq_add_neg] using (sub_nonneg.mpr hTS)
  have hphi_nonneg := phiK_nonneg (· := ·) K hnonneg
  have hrewrite :
    phiK (· := ·) K (S + (-1 : ℝ) • T) =
      phiK (· := ·) K S - phiK (· := ·) K T := by
    rw [phiK_add, phiK_smul]
  ring
  linarith [hrewrite ▸ hphi_nonneg]

omit [CompleteSpace ·] [Nontrivial ·] in
private lemma leftMulHS_rankOne (A : L ·) (x y : ·) :
  leftMulHS (· := ·) A (ofOp (InnerProductSpace.rankOne ℂ x y)) =
    ofOp (InnerProductSpace.rankOne ℂ (A x) y) := by
  change (A * InnerProductSpace.rankOne ℂ x y) = InnerProductSpace.rankOne ℂ (A x) y
  simpa [leftMulHS_apply] using
    (InnerProductSpace.comp_rankOne (· := ℂ) (x := x) (y := y) (f := A))

omit [Nontrivial ·] in

```

```

private lemma rightMulHS_rankOne (B : L  $\alpha$ ) (x y :  $\alpha$ ) :
  rightMulHS ( $\alpha$  :=  $\alpha$ ) B (ofOp (InnerProductSpace.rankOne  $\mathbb{C}$  x y)) =
    ofOp (InnerProductSpace.rankOne  $\mathbb{C}$  x ((star B) y)) := by
  change (InnerProductSpace.rankOne  $\mathbb{C}$  x y * B) = InnerProductSpace.rankOne
  simpa [rightMulHS_apply, ContinuousLinearMap.star_eq_adjoint] using
    (InnerProductSpace.rankOne_comp ( $\alpha$  :=  $\mathbb{C}$ ) (x := x) (y := y) (f := B))

private lemma re_inner_nonneg_of_nonneg
  { $\alpha$  : Type*} [NormedAddCommGroup  $\alpha$ ] [InnerProductSpace  $\mathbb{C}$   $\alpha$ ]
  {T :  $\alpha$   $\rightarrow$  L[ $\mathbb{C}$ ]  $\alpha$ } (hT :  $0 \leq T$ ) :
   $\forall x : \alpha, 0 \leq \text{Complex.re (inner } \mathbb{C} x (T x))$  := by
  intro x
  have hpos : T.IsPositive := (ContinuousLinearMap.nonneg_iff_isPositive T)
  have hnonneg :  $0 \leq \text{Complex.re (inner } \mathbb{C} (T x) x)$  :=
    ((ContinuousLinearMap.isPositive_iff_complex T).1 hpos x).2
  have hre :
     $\text{Complex.re (inner } \mathbb{C} x (T x)) = \text{Complex.re (inner } \mathbb{C} (T x) x)$  := by
    simpa using (inner_re_symm ( $\alpha$  :=  $\mathbb{C}$ ) (x := x) (y := T x))
  rw [hre]
  exact hnonneg

private lemma aeval_apply_of_mem_eigenspace_realpoly
  { $\alpha$  : Type*} [NormedAddCommGroup  $\alpha$ ] [InnerProductSpace  $\mathbb{C}$   $\alpha$ ]
  {T :  $\alpha$   $\rightarrow$  L[ $\mathbb{C}$ ]  $\alpha$ } {r :  $\mathbb{R}$ } {x :  $\alpha$ }
  (hx : x  $\in$  eigenspace T.toLinearMap (r :  $\mathbb{C}$ )) (p :  $\mathbb{R}[X]$ ) :
  Polynomial.aeval T (p.map (algebraMap  $\mathbb{R}$   $\mathbb{C}$ )) x =
    ((p.map (algebraMap  $\mathbb{R}$   $\mathbb{C}$ )).eval (r :  $\mathbb{C}$ )) • x := by
  by_cases hx0 : x = 0
  · simp [hx0]
  have hmap :
    Polynomial.aeval T (p.map (algebraMap  $\mathbb{R}$   $\mathbb{C}$ )) x =
      Polynomial.aeval T.toLinearMap (p.map (algebraMap  $\mathbb{R}$   $\mathbb{C}$ )) x := by
    simpa using
      congrArg (fun F :  $\alpha \rightarrow_1 [\mathbb{C}] \alpha \Rightarrow F x$ )
        (Polynomial.map_aeval_eq_aeval_map
          (R :=  $\mathbb{C}$ ) (S :=  $\alpha \rightarrow_1 [\mathbb{C}] \alpha$ ) (T :=  $\mathbb{C}$ ) (U :=  $\alpha \rightarrow_1 [\mathbb{C}] \alpha$ )
            ( $\phi$  := RingHom.id  $\mathbb{C}$ ) ( $\psi$  := ContinuousLinearMap.toLinearMapRingHom)
            (h := by ext z; rfl) (p := p.map (algebraMap  $\mathbb{R}$   $\mathbb{C}$ )) (a := T))
  rw [hmap]
  simpa using
    (Module.End.aeval_apply_of_hasEigenvector
      (f := T.toLinearMap) (p := p.map (algebraMap  $\mathbb{R}$   $\mathbb{C}$ )) ( $\mu$  := (r :  $\mathbb{C}$ )) (x

-- The interpolation-based `cfcR`-on-eigenspace lemma is elaboration-
heavy.
set_option maxHeartbeats 400000 in

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDOTRACE.LEAN

```
private lemma cfcR_apply_of_mem_eigenspace_real
  {α : Type*} [NormedAddCommGroup α] [InnerProductSpace ℂ α] [CompleteSpace α]
  [FiniteDimensional ℂ α]
  [ContinuousFunctionalCalculus ℝ (L α) IsSelfAdjoint]
  (f : ℝ → ℝ) {T : L α} (hT : IsSelfAdjoint T) {r : ℝ} {x : α}
  (hx : x ∈ eigenspace T.toLinearMap (r : ℂ)) :
  cfcR (α := α) f T x = (f r : ℂ) • x := by
  haveI : IsScalarTower ℝ ℂ (L α) := RestrictScalars.isScalarTower ℝ ℂ (L α)
  classical
  by_cases hx0 : x = 0
  · simp [hx0]
  have hspecCfin : Set.Finite (spectrum ℂ T) := by
    change Set.Finite (spectrum ℂ ((Module.End.toContinuousLinearMap α) T.toLinearMap))
    simpa using Module.End.finite_spectrum (K := ℂ) (V := α) T.toLinearMap
  have hspecRfin : Set.Finite (spectrum ℝ T) := by
    rw [← spectrum.preimage_algebraMap ℂ]
    exact hspecCfin.preimage (FaithfulSMul.algebraMap_injective ℝ ℂ).injOn
  let s : Finset ℝ := hspecRfin.toFinset
  let q : ℝ[X] := Lagrange.interpolate s id fun y ↦ f y
  have hq_spec : (spectrum ℝ T).EqOn f q.eval := by
    intro y hy
    have hy' : y ∈ s := by
      simpa [s] using hy
    symm
    simpa [q] using
      (Lagrange.eval_interpolate_at_node
        (s := s) (v := id) (r := fun z ↦ f z) (i := y)
        (hvs := fun _ _ _ _ h => h) hy')
  have hcfc : cfcR (α := α) f T = cfcR (α := α) q.eval T := by
    simpa [cfcR] using (cfc_congr (a := T) (f := f) (g := q.eval) hq_spec)
  have hpoly : cfcR (α := α) q.eval T = Polynomial.aeval T q := by
    simpa [cfcR] using (cfc_polynomial (p := IsSelfAdjoint) (q := q) (a := T) hT)
  have hxv : Module.End.HasEigenvector T.toLinearMap (r : ℂ) x := ⟨hx, hx0⟩
  have hr_specC : (r : ℂ) ∈ spectrum ℂ T :=
    by
      change (r : ℂ) ∈ spectrum ℂ ((Module.End.toContinuousLinearMap α) T.toLinearMap)
      simpa using (Module.End.hasEigenvalue_of_hasEigenvector hxv).mem_spectrum
  have hr_spec : r ∈ spectrum ℝ T := spectrum.of_algebraMap_mem ℂ hr_specC
  calc
    cfcR (α := α) f T x = cfcR (α := α) q.eval T x := by rw [hcfc]
    _ = Polynomial.aeval T q x := by rw [hpoly]
    _ = Polynomial.aeval T (q.map (algebraMap ℝ ℂ)) x := by
      symm
      simp
    _ = ((q.map (algebraMap ℝ ℂ)).eval (r : ℂ)) • x := by
      simpa using aeval_apply_of_mem_eigenspace_realpoly hx q
```

```

_ = (f r : ℂ) • x := by
  congr 1
  rw [Polynomial.eval_map_algebraMap]
  calc
    Polynomial.aeval (algebraMap ℝ ℂ r) q = ((Polynomial.eval r q : ℝ)
      simp using
        (Polynomial.aeval_algebraMap_apply_eq_algebraMap_eval (A := ℂ))
    _ = (f r : ℂ) := by
      simp using congrArg (fun t : ℝ => (t : ℂ)) (hq_spec hr_spec).symm

-- This proof is isolated because the joint eigenspace decomposition is heavy.
set_option maxHeartbeats 800000 in
set_option backward.isDefEq.respectTransparency false in
private lemma hmiddle_leftMul_rightMul
  {s : ℝ} {A B : L []}
  (hA : A ∈ pdSet ([] := [])) (hB : B ∈ pdSet ([] := [])) :
  cfcR ([] := HSOp []) (fun x : ℝ ↦ x ^ s)
    (cfcR ([] := HSOp []) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS ([] := []))
      leftMulHS ([] := []) A *
      cfcR ([] := HSOp []) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS ([] := []))
        leftMulHS ([] := []) (A ^ s) * rightMulHS ([] := []) (B ^ (-
s))) := by
  rcases hA with ⟨hA_sa, hA_spec⟩
  rcases hB with ⟨hB_sa, hB_spec⟩
  have hA0 : 0 ≤ A := by
    exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) A (ha := hA_sa)
      (by intro x hx; exact (hA_spec hx).le))
  have hB0 : 0 ≤ B := by
    exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B (ha := hB_sa)
      (by intro x hx; exact (hB_spec hx).le))
  have hright_negHalf :
    cfcR ([] := HSOp []) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS ([] := []))
      rightMulHS ([] := []) (B ^ ((-1 : ℝ) / 2)) := by
    rw [show rightMulHS ([] := []) (B ^ ((-1 : ℝ) / 2)) =
      rightMulHS ([] := []) (cfcR ([] := []) (fun x : ℝ ↦ x ^ ((-
1 : ℝ) / 2)) B) by
      congr
      simp [cfcR] using
        (CFC.rpow_eq_cfc_real (A := L []) (a := B) (y := (-
1 : ℝ) / 2) (ha := hB0))]
    exact (rightMulHS_cfcR ([] := []) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) B hB_sa
      (by
        intro x hx
        exact (Real.continuousAt_rpow_const x ((-1 : ℝ) / 2)
          (Or.inl (ne_of_gt (hB_spec hx))))).continuousWithinAt)).symm

```


1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBMANDOTTRACE.LEAN

```

have hBunit : IsUnit B := by
  refine spectrum.isUnit_of_zero_notMem (R := ℝ) ?_
  intro h0
  exact (lt_irrefl (0 : ℝ)) (by simp [Set.Ioi] using hB_spec h0)
have hBnegOne :
  B ^ ((-1 : ℝ) / 2) * B ^ ((-1 : ℝ) / 2) = B ^ (-1 : ℝ) := by
  calc
    B ^ ((-1 : ℝ) / 2) * B ^ ((-1 : ℝ) / 2)
      = B ^ (((-1 : ℝ) / 2) + ((-1 : ℝ) / 2)) := by
        rw [← CFC.rpow_add hBunit]
    _ = B ^ (-1 : ℝ) := by ring_nf
have hmid_prod :
  cfcR (□ := HSOp □) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS (□ := □) B) *
    leftMulHS (□ := □) A *
    cfcR (□ := HSOp □) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS (□ := □) B)
    leftMulHS (□ := □) A * rightMulHS (□ := □) (B ^ (-
1 : ℝ)) := by
  rw [hright_negHalf]
  calc
    rightMulHS (□ := □) (B ^ ((-1 : ℝ) / 2)) *
      leftMulHS (□ := □) A *
      rightMulHS (□ := □) (B ^ ((-1 : ℝ) / 2)) =
      leftMulHS (□ := □) A *
      (rightMulHS (□ := □) (B ^ ((-1 : ℝ) / 2)) *
        rightMulHS (□ := □) (B ^ ((-1 : ℝ) / 2))) := by
        rw [← mul_assoc,
          (leftMulHS_rightMulHS_commute (□ := □) A (B ^ ((-
1 : ℝ) / 2))).eq.symm,
          mul_assoc]
    _ = leftMulHS (□ := □) A * rightMulHS (□ := □) (B ^ (-
1 : ℝ)) := by
      rw [← rightMulHS_mul (□ := □) (B ^ ((-1 : ℝ) / 2)) (B ^ ((-
1 : ℝ) / 2)), hBnegOne]
  rw [hmid_prod]
  let T0 : HSOp □ →1[ℂ] HSOp □ := (leftMulHS (□ := □) A).toLinearMap
  let T1 : HSOp □ →1[ℂ] HSOp □ := (rightMulHS (□ := □) (B ^ (-
1 : ℝ))).toLinearMap
  let lhs : L (HSOp □) :=
    cfcR (□ := HSOp □) (fun x : ℝ ↦ x ^ s)
    (leftMulHS (□ := □) A * rightMulHS (□ := □) (B ^ (-
1 : ℝ)))
  let rhs : L (HSOp □) :=
    leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B ^ (-
s))
  let D : L (HSOp □) := lhs - rhs
  have hleft_sa : IsSelfAdjoint (leftMulHS (□ := □) A) :=

```

```

    IsSelfAdjoint.of_nonneg (leftMulHS_nonneg (□ := □) hA0)
  have hT0_symm : T0.IsSymmetric := by
    simp [T0] using
      (ContinuousLinearMap.isSelfAdjoint_iff_isSymmetric.mp hleft_sa)
  have hBinv0 : 0 ≤ B ^ (-1 : ℝ) := by
    simp
  have hBinv_sa : IsSelfAdjoint (B ^ (-1 : ℝ)) := IsSelfAdjoint.of_nonneg h
  have hBinv_unit : IsUnit (B ^ (-1 : ℝ)) := by
    rcases hBunit with ⟨u, rfl⟩
    simp [CFC.rpow_neg_one_eq_inv u (by simp using hB0)]
  have hright_sa : IsSelfAdjoint (rightMulHS (□ := □) (B ^ (-
1 : ℝ))) :=
    IsSelfAdjoint.of_nonneg (rightMulHS_nonneg (□ := □) hBinv0)
  have hT1_symm : T1.IsSymmetric := by
    simp [T1] using
      (ContinuousLinearMap.isSelfAdjoint_iff_isSymmetric.mp hright_sa)
  have hcomm : Commute T0 T1 := by
    show T0 * T1 = T1 * T0
  ext x
  simp [T0, T1] using congrArg (fun F : L (HS0p □) => F x)
    (leftMulHS_rightMulHS_commute (□ := □) A (B ^ (-1 : ℝ))).eq
  have hleft_pow :
    cfcR (□ := HS0p □) (fun x : ℝ ↦ x ^ s) (leftMulHS (□ := □) A) =
      leftMulHS (□ := □) (A ^ s) := by
    rw [show leftMulHS (□ := □) (A ^ s) =
      leftMulHS (□ := □) (cfcR (□ := □) (fun x : ℝ ↦ x ^ s) A) by
      congr
    simp [cfcR] using
      (CFC.rpow_eq_cfc_real (A := L □) (a := A) (y := s) (ha := hA0))]
  exact (leftMulHS_cfcR (□ := □) (fun x : ℝ ↦ x ^ s) A hA_sa
    (by
      intro x hx
      exact (Real.continuousAt_rpow_const x s
        (0r.inl (ne_of_gt (hA_spec hx)))).continuousWithinAt)).symm
  have hright_pow :
    cfcR (□ := HS0p □) (fun x : ℝ ↦ x ^ s)
      (rightMulHS (□ := □) (B ^ (-1 : ℝ))) =
      rightMulHS (□ := □) (B ^ (-s)) := by
    rw [show rightMulHS (□ := □) (B ^ (-s)) =
      rightMulHS (□ := □) (cfcR (□ := □) (fun x : ℝ ↦ x ^ s) (B ^ (-
1 : ℝ))) by
      congr
      calc
        B ^ (-s) = (B ^ (-1 : ℝ)) ^ s := by
          symm
          simp using (CFC.rpow_rpow B (-1 : ℝ) s (by norm_num) (hBunit.isS

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDOTRACE.LEAN

```

_ = cfcR (□ := □) (fun x : ℝ ↦ x ^ s) (B ^ (-1 : ℝ)) := by
  simpa [cfcR] using
    (CFC.rpow_eq_cfc_real (A := L □) (a := B ^ (-
1 : ℝ)) (y := s) (ha := hBinv0)))
  exact (rightMulHS_cfcR (□ := □) (fun x : ℝ ↦ x ^ s) (B ^ (-
1 : ℝ)) hBinv_sa
    (by
      intro x hx
      have hx0 : x ≠ 0 := by
        intro hx0
        exact spectrum.zero_notMem (R := ℝ) hBinv_unit (by simpa [hx0] using hx)
        exact (Real.continuousAt_rpow_const x s (Or.inl hx0)).continuousWithinAt)).s
  have hprod0 :
    0 ≤ leftMulHS (□ := □) A * rightMulHS (□ := □) (B ^ (-
1 : ℝ)) := by
    exact (leftMulHS_rightMulHS_commute (□ := □) A (B ^ (-
1 : ℝ))).mul_nonneg
      (leftMulHS_nonneg (□ := □) hA0)
      (rightMulHS_nonneg (□ := □) hBinv0)
  have hprod_sa :
    IsSelfAdjoint
      (leftMulHS (□ := □) A * rightMulHS (□ := □) (B ^ (-
1 : ℝ))) :=
    IsSelfAdjoint.of_nonneg hprod0
  have htop :
    (□ α, □ β, eigenspace T0 α □ eigenspace T1 β) = τ := by
    exact LinearMap.IsSymmetric.iSup_iSup_eigenspace_inf_eigenspace_eq_top_of_commut
      hT0_symm hT1_symm hcomm
  have hjoint_ker :
    ∀ α β,
      eigenspace T0 α □ eigenspace T1 β ≤ LinearMap.ker D.toLinearMap := by
    intro α β x hx
    rcases hx with ⟨hx0, hx1⟩
    rw [LinearMap.mem_ker]
    by_cases hxzero : x = 0
    · simp [D, hxzero]
    have hxv0 : Module.End.HasEigenvector T0 α x := ⟨hx0, hxzero⟩
    have hxv1 : Module.End.HasEigenvector T1 β x := ⟨hx1, hxzero⟩
    have hαeq : α = (α.re : ℂ) := by
      exact (RCLike.conj_eq_iff_re.mp
        (hT0_symm.conj_eigenvalue_eq_self (Module.End.hasEigenvalue_of_hasEigenvecto
          ).symm
        ).symm
    have hβeq : β = (β.re : ℂ) := by
      exact (RCLike.conj_eq_iff_re.mp
        (hT1_symm.conj_eigenvalue_eq_self (Module.End.hasEigenvalue_of_hasEigenvecto
          ).symm

```

```

have hx0r : x ∈ eigenspace T0 (α.re : ℂ) := by
  rwa [hαeq] at hx0
have hx1r : x ∈ eigenspace T1 (β.re : ℂ) := by
  rwa [hβeq] at hx1
have hT0_nonneg_re :
  ∀ y : HSOp [], 0 ≤ Complex.re (inner ℂ y (T0 y)) := by
  intro y
  simp [T0] using
    re_inner_nonneg_of_nonneg
    (T := leftMulHS (□ := □) A)
    (leftMulHS_nonneg (□ := □) hA0) y
have hT1_nonneg_re :
  ∀ y : HSOp [], 0 ≤ Complex.re (inner ℂ y (T1 y)) := by
  intro y
  simp [T1] using
    re_inner_nonneg_of_nonneg
    (T := rightMulHS (□ := □) (B ^ (-1 : ℝ)))
    (rightMulHS_nonneg (□ := □) hBinv0) y
have hαnonneg : 0 ≤ α.re := by
  exact eigenvalue_nonneg_of_nonneg
    (Module.End.hasEigenvalue_of_hasEigenvector {hx0r, hxzero})
    hT0_nonneg_re
have hβnonneg : 0 ≤ β.re := by
  exact eigenvalue_nonneg_of_nonneg
    (Module.End.hasEigenvalue_of_hasEigenvector {hx1r, hxzero})
    hT1_nonneg_re
have hxprod :
  x ∈ eigenspace
    ((leftMulHS (□ := □) A * rightMulHS (□ := □) (B ^ (-
1 : ℝ))).toLinearMap)
    (((α.re * β.re : ℝ) : ℂ)) := by
  rw [Module.End.mem_eigenspace_iff]
have hx0apply : T0 x = (α.re : ℂ) • x := Module.End.mem_eigenspace_if
have hx1apply : T1 x = (β.re : ℂ) • x := Module.End.mem_eigenspace_if
have hx0apply' :
  leftMulHS (□ := □) A x = (α.re : ℂ) • x := by
  simp [T0] using hx0apply
have hx1apply' :
  rightMulHS (□ := □) (B ^ (-1 : ℝ)) x = (β.re : ℂ) • x := by
  simp [T1] using hx1apply
show leftMulHS (□ := □) A (rightMulHS (□ := □) (B ^ (-
1 : ℝ)) x) =
  (((α.re * β.re : ℝ) : ℂ)) • x
rw [hx1apply', ContinuousLinearMap.map_smul, hx0apply']
rw [smul_smul]
congr 1

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDOTRACE.LEAN

```

    simp [mul_comm]
  have hlhsx :
    lhs x = (((α.re * β.re : ℝ) ^ s : ℝ) : ℂ) • x := by
    simp [lhs] using
      cfcR_apply_of_mem_eigenspace_real
      (□ := HSOp □) (f := fun t : ℝ ↦ t ^ s) hprod_sa hxprod
  have hrhsx :
    rhs x = (((α.re ^ s) * (β.re ^ s) : ℝ) : ℂ) • x := by
    change (leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B ^ (-
s))) x =
      (((α.re ^ s) * (β.re ^ s) : ℝ) : ℂ) • x
  have hleftx :
    cfcR (□ := HSOp □) (fun t : ℝ ↦ t ^ s) (leftMulHS (□ := □) A) x =
      (((α.re ^ s : ℝ) : ℂ) • x) := by
    simp using
      cfcR_apply_of_mem_eigenspace_real
      (□ := HSOp □) (f := fun t : ℝ ↦ t ^ s) hleft_sa hx0r
  have hrightx :
    cfcR (□ := HSOp □) (fun t : ℝ ↦ t ^ s)
      (rightMulHS (□ := □) (B ^ (-1 : ℝ))) x =
      (((β.re ^ s : ℝ) : ℂ) • x) := by
    simp using
      cfcR_apply_of_mem_eigenspace_real
      (□ := HSOp □) (f := fun t : ℝ ↦ t ^ s) hright_sa hx1r
  rw [← hleft_pow, ← hright_pow, ContinuousLinearMap.mul_def]
  show cfcR (□ := HSOp □) (fun t : ℝ ↦ t ^ s) (leftMulHS (□ := □) A)
    (cfcR (□ := HSOp □) (fun t : ℝ ↦ t ^ s)
      (rightMulHS (□ := □) (B ^ (-1 : ℝ)))) x =
    (((α.re ^ s) * (β.re ^ s) : ℝ) : ℂ) • x
  rw [hrightx, ContinuousLinearMap.map_smul, hleftx]
  rw [smul_smul]
  congr 1
  simp [mul_comm]
  have hscal :
    (((α.re * β.re : ℝ) ^ s : ℝ) : ℂ) =
      (((α.re ^ s) * (β.re ^ s) : ℝ) : ℂ) := by
    exact congrArg (fun t : ℝ => (t : ℂ)) (Real.mul_rpow hannonneg hβnonneg)
  simp [D] using
    sub_eq_zero.mpr (hlhsx.trans (hscal ▸ hrhsx.symm))
  have hker_top : LinearMap.ker D.toLinearMap = T := by
    apply top_unique
    rw [← htop]
    refine iSup_le ?_
    intro α
    refine iSup_le ?_
    intro β

```

```

    exact hjoint_ker  $\alpha \beta$ 
  have hDzero :  $\bar{D} = 0 :=$  by
    ext x
    have hx :  $x \in \text{LinearMap.ker } D.\text{toLinearMap} :=$  by simp [hker_top]
    exact LinearMap.mem_ker.mp hx
  have hlhs_eq_rhs : lhs = rhs := sub_eq_zero.mp hDzero
  simpa [lhs, rhs] using hlhs_eq_rhs

-- The bridge lemma expands a large `HSOp`-valued generalized perspective t
set_option maxHeartbeats 800000 in
set_option backward.isDefEq.respectTransparency false in
private lemma phiK_operatorPowerMean_eq_liebTraceMap
  {s :  $\mathbb{R}$ } (K A B : L  $\square$ ) (hA : A  $\in$  pdSet ( $\square := \square$ )) (hB : B  $\in$  pdSet ( $\square := \square$ ))
  phiK ( $\square := \square$ ) K
  (operatorPowerMean ( $\square := \text{HSOp } \square$ ) s 1
    (leftMulHS ( $\square := \square$ ) A) (rightMulHS ( $\square := \square$ ) B)) =
    liebTraceMap ( $\square := \square$ ) s K A B := by
  rcases hA with {hA_sa, hA_spec}
  rcases hB with {hB_sa, hB_spec}
  have hA0 :  $0 \leq A :=$  by
    exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R :=  $\mathbb{R}$ ) A (ha := hA_
      (by intro x hx; exact (hA_spec hx).le)
  have hB0 :  $0 \leq B :=$  by
    exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R :=  $\mathbb{R}$ ) B (ha := hB_
      (by intro x hx; exact (hB_spec hx).le)
  have hright_half :
    cfcR ( $\square := \text{HSOp } \square$ ) (fun x :  $\mathbb{R} \mapsto x ^ ((1 : \mathbb{R}) / 2)$ ) (rightMulHS ( $\square :=$ 
      rightMulHS ( $\square := \square$ ) (B ^ ((1 :  $\mathbb{R}) / 2)$ ) := by
  rw [show rightMulHS ( $\square := \square$ ) (B ^ ((1 :  $\mathbb{R}) / 2)$ ) =
    rightMulHS ( $\square := \square$ ) (cfcR ( $\square := \square$ ) (fun x :  $\mathbb{R} \mapsto x ^ ((1 : \mathbb{R}) / 2)$ )
    congr
  simpa [cfcR] using
    (CFC.rpow_eq_cfc_real (A := L  $\square$ ) (a := B) (y := (1 :  $\mathbb{R}) / 2$ ) (ha :=
  exact (rightMulHS_cfcR ( $\square := \square$ ) (fun x :  $\mathbb{R} \mapsto x ^ ((1 : \mathbb{R}) / 2)$ ) B hB_sa
    (by
      intro x hx
      exact (Real.continuousAt_rpow_const x ((1 :  $\mathbb{R}) / 2$ )
        (Or.inr (by positivity))).continuousWithinAt)).symm
  have hright_negHalf :
    cfcR ( $\square := \text{HSOp } \square$ ) (fun x :  $\mathbb{R} \mapsto x ^ ((-1 : \mathbb{R}) / 2)$ ) (rightMulHS ( $\square :=$ 
      rightMulHS ( $\square := \square$ ) (B ^ ((-1 :  $\mathbb{R}) / 2)$ ) := by
  rw [show rightMulHS ( $\square := \square$ ) (B ^ ((-1 :  $\mathbb{R}) / 2)$ ) =
    rightMulHS ( $\square := \square$ ) (cfcR ( $\square := \square$ ) (fun x :  $\mathbb{R} \mapsto x ^ ((-1 : \mathbb{R}) / 2)$ ) B) by
  1 :  $\mathbb{R}) / 2$ ) B) by
    congr
  simpa [cfcR] using

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDOTRACE.LEAN

```

      (CFC.rpow_eq_cfc_real (A := L []) (a := B) (y := (-
1 : ℝ) / 2) (ha := hB0)))
    exact (rightMulHS_cfcR (□ := □) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) B hB_sa
      (by
        intro x hx
        exact (Real.continuousAt_rpow_const x ((-1 : ℝ) / 2)
          (Or.inl (ne_of_gt (hB_spec hx))))).continuousWithinAt)).symm
  have hBunit : IsUnit B := by
    refine spectrum.isUnit_of_zero_notMem (R := ℝ) ?_
    intro h0
    exact (lt_irrefl (0 : ℝ)) (by simpa [Set.Ioi] using hB_spec h0)
  have hBpow :
    B ^ ((1 : ℝ) / 2) * B ^ (-s) * B ^ ((1 : ℝ) / 2) = B ^ (1 - s) := by
    calc
      B ^ ((1 : ℝ) / 2) * B ^ (-s) * B ^ ((1 : ℝ) / 2)
        = B ^ (((1 : ℝ) / 2) + (-s)) * B ^ ((1 : ℝ) / 2) := by
          rw [← CFC.rpow_add hBunit]
      _ = B ^ (((1 : ℝ) / 2) + (-s)) + ((1 : ℝ) / 2) := by
          rw [← CFC.rpow_add hBunit]
      _ = B ^ (1 - s) := by ring_nf
  have hBnegOne :
    B ^ ((-1 : ℝ) / 2) * B ^ ((-1 : ℝ) / 2) = B ^ (-1 : ℝ) := by
    calc
      B ^ ((-1 : ℝ) / 2) * B ^ ((-1 : ℝ) / 2)
        = B ^ (((-1 : ℝ) / 2) + ((-1 : ℝ) / 2)) := by
          rw [← CFC.rpow_add hBunit]
      _ = B ^ (-1 : ℝ) := by ring_nf
  have hBinvHalf0 : 0 ≤ B ^ ((-1 : ℝ) / 2) := by
    simp
  have hBinv0 : 0 ≤ B ^ (-1 : ℝ) := by
    simp
  have hBinv_sa : IsSelfAdjoint (B ^ (-1 : ℝ)) := IsSelfAdjoint.of_nonneg hBinv0
  have hright_invHalf0 :
    0 ≤ rightMulHS (□ := □) (B ^ ((-1 : ℝ) / 2)) := by
    exact rightMulHS_nonneg (□ := □) hBinvHalf0
  have hmid_nonneg :
    0 ≤ cfcR (□ := HS0p □) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS (□ := □) B
      leftMulHS (□ := □) A *
      cfcR (□ := HS0p □) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS (□ := □) B
        rw [hright_negHalf]
      simpa [mul_assoc] using
        conjugate_nonneg_of_nonneg (leftMulHS_nonneg (□ := □) hA0) hright_invHalf0
  have hmid_prod :
    cfcR (□ := HS0p □) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS (□ := □) B) *
      leftMulHS (□ := □) A *
      cfcR (□ := HS0p □) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS (□ := □) B

```

```

      leftMulHS (□ := □) A * rightMulHS (□ := □) (B ^ (-
1 : ℝ)) := by
  rw [hright_negHalf]
  calc
    rightMulHS (□ := □) (B ^ ((-1 : ℝ) / 2)) *
      leftMulHS (□ := □) A *
        rightMulHS (□ := □) (B ^ ((-1 : ℝ) / 2))
    =
      leftMulHS (□ := □) A *
        (rightMulHS (□ := □) (B ^ ((-1 : ℝ) / 2)) *
          rightMulHS (□ := □) (B ^ ((-1 : ℝ) / 2))) := by
        rw [← mul_assoc,
          (leftMulHS_rightMulHS_commute (□ := □) A (B ^ ((-
1 : ℝ) / 2))).eq.symm,
          mul_assoc]
    = leftMulHS (□ := □) A * rightMulHS (□ := □) (B ^ (-
1 : ℝ)) := by
      rw [← rightMulHS_mul (□ := □) (B ^ ((-1 : ℝ) / 2)) (B ^ ((-
1 : ℝ) / 2)), hNegOne]
  have hmiddle :
    cfcR (□ := HSOp □) (fun x : ℝ ↦ x ^ s)
      (cfcR (□ := HSOp □) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS
        leftMulHS (□ := □) A *
          cfcR (□ := HSOp □) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS
            leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B ^ (-
s)) := by
      exact hmiddle_leftMul_rightMul (□ := □) (s := s) ⟨hA_sa, hA_spec⟩ ⟨hB_s
  have happly :
    operatorPowerMean (□ := HSOp □) s 1
      (leftMulHS (□ := □) A) (rightMulHS (□ := □) B) (ofOp (star K)) =
      ofOp (A ^ s * star K * B ^ (1 - s)) := by
  have hcomm_half :
    Commute (leftMulHS (□ := □) (A ^ s)) (rightMulHS (□ := □) (B ^ ((1
      leftMulHS_rightMulHS_commute (□ := □) (A ^ s) (B ^ ((1 : ℝ) / 2))
  have hright_pow :
    rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) *
      rightMulHS (□ := □) (B ^ (-s)) *
      rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) =
      rightMulHS (□ := □) (B ^ (1 - s)) := by
  calc
    rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) *
      rightMulHS (□ := □) (B ^ (-s)) *
      rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2))
    =
      rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2) * B ^ (-
s) * B ^ ((1 : ℝ) / 2)) := by

```


1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDOTRACE.LEAN

```

      simp [mul_assoc]
      _ = rightMulHS (□ := □) (B ^ (1 - s)) := by rw [hBpow]
    have hreorder :
      rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) *
        (leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B ^ (-
s))) *
      rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) =
      leftMulHS (□ := □) (A ^ s) *
        (rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) *
          rightMulHS (□ := □) (B ^ (-s)) *
            rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2))) := by
    calc
      rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) *
        (leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B ^ (-
s))) *
      rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2))
      =
      ((rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) *
        leftMulHS (□ := □) (A ^ s)) *
        rightMulHS (□ := □) (B ^ (-s))) *
        rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) := by
      simp [mul_assoc]
    =
    ((leftMulHS (□ := □) (A ^ s) *
      rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2))) *
      rightMulHS (□ := □) (B ^ (-s))) *
      rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) := by
      rw [hcomm_half.eq]
    =
    leftMulHS (□ := □) (A ^ s) *
      (rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) *
        rightMulHS (□ := □) (B ^ (-s)) *
          rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2))) := by
      simp [mul_assoc]
    calc
      operatorPowerMean (□ := HSOp □) s 1
      (leftMulHS (□ := □) A) (rightMulHS (□ := □) B) (ofOp (star K))
      =
      (rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) *
        (leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B ^ (-
s))) *
        rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2))) (ofOp (star K)) := by
      rw [OperatorGeometricMean.operatorPowerMean, GeneralizedPerspective,
        GeneralizedPerspectiveFunction.hSqrt, GeneralizedPerspectiveFunction.h
      simp only [Real.rpow_one]
      rw [hmiddle]

```

```

      rw [hright_half]
    - =
      (leftMulHS (□ := □) (A ^ s) *
        (rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)) *
          rightMulHS (□ := □) (B ^ (-s)) *
          rightMulHS (□ := □) (B ^ ((1 : ℝ) / 2)))) (ofOp (star K)) :
      rw [hreorder]
    - =
      (leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B ^ (1 - s)))
      rw [hright_pow]
    - = ofOp (A ^ s * star K * B ^ (1 - s)) := by
      simp [leftMulHS_apply, rightMulHS_apply, mul_assoc]
calc
  phiK (□ := □) K
  (operatorPowerMean (□ := HSOp □) s 1
    (leftMulHS (□ := □) A) (rightMulHS (□ := □) B))
  = Complex.re
    (inner ℂ (ofOp (star K))
      (ofOp (A ^ s * star K * B ^ (1 - s)))) := by
      simp [phiK, happly]
  - = traceRe (□ := □) (A ^ s * star K * B ^ (1 - s) * K) := by
      rw [traceRe]
      set X : L □ := A ^ s * star K * B ^ (1 - s)
      have htrace :=
        re_hsInner_eq_traceRe (□ := □) (X := star K) (Y := X)
      have htrace' :
        Complex.re (inner ℂ (ofOp (star K)) (ofOp X)) =
          Complex.re (LinearMap.trace ℂ □
            ((K * X).toLinearMap)) := by
            simpa [X, mul_assoc] using htrace
      have hcycle :
        Complex.re (LinearMap.trace ℂ □ ((K * X).toLinearMap)) =
          Complex.re (LinearMap.trace ℂ □ ((X * K).toLinearMap)) := b
      simpa using
        congrArg Complex.re
          (LinearMap.trace_mul_comm (R := ℂ) (M := □) K.toLinearMap X)
      simpa [X, mul_assoc] using htrace'.trans hcycle
  - = liebTraceMap (□ := □) s K A B := by
      rfl

omit [FiniteDimensional ℂ □] in
/-- Convex combinations preserve `pdSet` (strict positivity). -
/
lemma pdSet_convexCombo {A B : L □} {t : ℝ}
  (hA : A ∈ pdSet (□ := □)) (hB : B ∈ pdSet (□ := □))
  (ht0 : 0 ≤ t) (ht1 : t ≤ 1) :

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDOTRACE.LEAN

```

    ((1 - t) • A + t • B) ∈ pdSet (□ := □) := by
    rcases hA with ⟨hA_sa, hA_spec⟩
    rcases hB with ⟨hB_sa, hB_spec⟩
    set C : L □ := (1 - t) • A + t • B
    have hC : IsSelfAdjoint C := by
      simpa [C] using (IsSelfAdjoint.all (1 - t)).smul hA_sa |>.add ((IsSelfAdjoint.all
    have hApos : ∃ r > 0, algebraMap ℝ (L □) r ≤ A := by
      refine (CFC.exists_pos_algebraMap_le_iff (A := L □) (a := A) (ha := hA_sa)).2 ?_
      intro x hx
      exact hA_spec hx
    have hBpos : ∃ r > 0, algebraMap ℝ (L □) r ≤ B := by
      refine (CFC.exists_pos_algebraMap_le_iff (A := L □) (a := B) (ha := hB_sa)).2 ?_
      intro x hx
      exact hB_spec hx
    rcases hApos with ⟨rA, hrA, hrA_le⟩
    rcases hBpos with ⟨rB, hrB, hrB_le⟩
    set rC : ℝ := (1 - t) * rA + t * rB
    have hrC : 0 < rC := by
      by_cases h1t : (1 - t) = 0
      · have ht' : t = 1 := by linarith
        subst ht'
        simpa [rC] using hrB
      · have h1t_pos : 0 < 1 - t := lt_of_le_of_ne (sub_nonneg.mpr ht1) (Ne.symm h1t)
        simpa [rC] using
          add_pos_of_pos_of_nonneg (mul_pos h1t_pos hrA) (mul_nonneg ht0 (le_of_lt hrB))
    have hrC_le : algebraMap ℝ (L □) rC ≤ C := by
      have hsum :
        (1 - t) • algebraMap ℝ (L □) rA + t • algebraMap ℝ (L □) rB ≤ C := by
          simpa [C] using
            add_le_add (smul_le_smul_of_nonneg_left hrA_le (sub_nonneg.mpr ht1))
              (smul_le_smul_of_nonneg_left hrB_le ht0)
      have hLHS :
        (1 - t) • algebraMap ℝ (L □) rA + t • algebraMap ℝ (L □) rB =
          algebraMap ℝ (L □) rC := by
            simp [rC, Algebra.smul_def]
      simpa [hLHS] using hsum
    refine ⟨hC, ?_⟩
    intro x hx
    simpa [C] using
      (CFC.exists_pos_algebraMap_le_iff (A := L □) (a := C) (ha := hC)).1 ⟨rC, hrC, hr
  omit [Nontrivial □] in
  private lemma phiK_leftMul_rightMul_eq_traceRe (K C D : L □) :
    phiK (□ := □) K
      (leftMulHS (□ := □) C * rightMulHS (□ := □) D) =
      traceRe (□ := □) (C * star K * D * K) := by

```

```

calc
  phiK (⊠ := ⊠) K
    (leftMulHS (⊠ := ⊠) C * rightMulHS (⊠ := ⊠) D)
  = Complex.re
    (inner ℂ (ofOp (star K))
      ((leftMulHS (⊠ := ⊠) C * rightMulHS (⊠ := ⊠) D) (ofOp (star K))
        simp [phiK]
    _ = Complex.re
      (inner ℂ (ofOp (star K))
        (ofOp (C * star K * D))) := by
        simp [leftMulHS_apply, rightMulHS_apply, mul_assoc]
    _ = traceRe (⊠ := ⊠) (C * star K * D * K) := by
      rw [traceRe]
      set X : L ⊠ := C * star K * D
      have htrace :=
        re_hsInner_eq_traceRe (⊠ := ⊠) (X := star K) (Y := X)
      have htrace' :
        Complex.re (inner ℂ (ofOp (star K)) (ofOp X)) =
          Complex.re (LinearMap.trace ℂ ⊠ ((K * X).toLinearMap)) := b
      simp [X, mul_assoc] using htrace
      have hcycle :
        Complex.re (LinearMap.trace ℂ ⊠ ((K * X).toLinearMap)) =
          Complex.re (LinearMap.trace ℂ ⊠ ((X * K).toLinearMap)) := b
      simp using
        congrArg Complex.re
          (LinearMap.trace_mul_comm (R := ℂ) (M := ⊠) K.toLinearMap X
      simp [X, mul_assoc] using htrace'.trans hcycle

omit [FiniteDimensional ℂ ⊠] in
set_option maxHeartbeats 400000 in
private lemma pdSet_rpow_of_mem_Icc_zero_one
  {p : ℝ} (hp : p ∈ Set.Icc (0 : ℝ) 1) {A : L ⊠} (hA : A ∈ pdSet (⊠ := ⊠))
  A ^ p ∈ pdSet (⊠ := ⊠) := by
  rcases hA with ⟨hA_sa, hA_spec⟩
  have hA0 : 0 ≤ A := by
    exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) A (ha := hA_
      (by intro x hx; exact (hA_spec hx).le)
  have hApow0 : 0 ≤ A ^ p := by
    simp
  have hApow_sa : IsSelfAdjoint (A ^ p) := IsSelfAdjoint.of_nonneg hApow0
  rcases (CFC.exists_pos_algebraMap_le_iff (A := L ⊠) (a := A) (ha := hA_sa
    ⟨r, hr, hrA⟩)
  refine ⟨hApow_sa, ?_⟩
  have hr0 : 0 ≤ algebraMap ℝ (L ⊠) r := by
    simp [Algebra.algebraMap_eq_smul_one] using
      smul_nonneg hr.le (show (0 : L ⊠) ≤ 1 by simp)

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDOTRACE.LEAN

```

have hmono :=
  power_Icc_zero_one_operatorMonotoneOn_Ici (⊥ := ⊥) p hp
  (A := A) (B := algebraMap ℝ (L ⊥) r)
  hA0 hr0 hrA
  (by
    intro x hx
    have hx0 : 0 < x := by simpa [Set.Ioi] using hA_spec hx
    exact hx0.le)
  (by
    intro x hx
    exact spectrum_nonneg_of_nonneg hr0 hx)
have hApow :
  cfcR (⊥ := ⊥) (fun x : ℝ ↦ x ^ p) A = A ^ p := by
  simpa [cfcR, LownerHeinzCore.cfcR] using
    (CFC.rpow_eq_cfc_real (A := L ⊥) (a := A) (y := p) (ha := hA0)).symm
have hscalar :
  (algebraMap ℝ (L ⊥) r) ^ p = algebraMap ℝ (L ⊥) (r ^ p) := by
  rw [CFC.rpow_eq_cfc_real (A := L ⊥) (a := algebraMap ℝ (L ⊥) r) (y := p) (ha :=
    simp
  have hbound : algebraMap ℝ (L ⊥) (r ^ p) ≤ A ^ p := by
    simpa [hscalar, hApow] using hmono
  exact (CFC.exists_pos_algebraMap_le_iff (A := L ⊥) (a := A ^ p) (ha := hApow_sa)).
    (r ^ p, Real.rpow_pos_of_pos hr p, hbound)

omit [Nontrivial ⊥] in
set_option maxHeartbeats 400000 in
private lemma liebTraceMap_mono_right
  {s : ℝ} (hs : 1 - s ∈ Set.Icc (0 : ℝ) 1)
  (K A B1 B2 : L ⊥)
  (hA : A ∈ pdSet (⊥ := ⊥)) (hB1 : B1 ∈ pdSet (⊥ := ⊥)) (hB2 : B2 ∈ pdSet (⊥ := ⊥))
  (hB : B1 ≤ B2) :
  liebTraceMap (⊥ := ⊥) s K A B1 ≤ liebTraceMap (⊥ := ⊥) s K A B2 := by
  rcases hA with ⟨hA_sa, hA_spec⟩
  rcases hB1 with ⟨hB1_sa, hB1_spec⟩
  rcases hB2 with ⟨hB2_sa, hB2_spec⟩
  have hA0 : 0 ≤ A := by
    exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) A (ha := hA_sa)).2
    (by intro x hx; exact (hA_spec hx).le)
  have hB10 : 0 ≤ B1 := by
    exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B1 (ha := hB1_sa)).2
    (by intro x hx; exact (hB1_spec hx).le)
  have hB20 : 0 ≤ B2 := by
    exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B2 (ha := hB2_sa)).2
    (by intro x hx; exact (hB2_spec hx).le)
  have hcfc :=
    power_Icc_zero_one_operatorMonotoneOn_Ici (⊥ := ⊥) (1 - s) hs

```

```

(A := B₂) (B := B₁) hB₂₀ hB₁₀ hB
(by
  intro x hx
  have hx₀ : 0 < x := by simpa [Set.Ioi] using hB₂_spec hx
  exact hx₀.le)
(by
  intro x hx
  have hx₀ : 0 < x := by simpa [Set.Ioi] using hB₁_spec hx
  exact hx₀.le)
have hpow :
  B₁ ^ (1 - s) ≤ B₂ ^ (1 - s) := by
  simpa [cfcR, LownerHeinzCore.cfcR,
    CFC.rpow_eq_cfc_real (A := L []) (a := B₁) (y := 1 - s) (ha := hB₁₀),
    CFC.rpow_eq_cfc_real (A := L []) (a := B₂) (y := 1 - s) (ha := hB₂₀)]
have hApow₀ : 0 ≤ A ^ s := by
  simp
have hdiff₀ : 0 ≤ B₂ ^ (1 - s) - B₁ ^ (1 - s) := sub_nonneg.mpr hpow
have hprod₀ :
  0 ≤ leftMulHS (□ := □) (A ^ s) *
    rightMulHS (□ := □) (B₂ ^ (1 - s) - B₁ ^ (1 - s)) := by
  exact (leftMulHS_rightMulHS_commute (□ := □) (A ^ s) (B₂ ^ (1 - s) - B₁ ^ (1 - s))
    (leftMulHS_nonneg (□ := □) hApow₀)
    (rightMulHS_nonneg (□ := □) hdiff₀))
have hphi : 0 ≤
  phiK (□ := □) K
  (leftMulHS (□ := □) (A ^ s) *
    rightMulHS (□ := □) (B₂ ^ (1 - s) - B₁ ^ (1 - s))) :=
  phiK_nonneg (□ := □) K hprod₀
have hsplit :
  leftMulHS (□ := □) (A ^ s) *
    rightMulHS (□ := □) (B₂ ^ (1 - s) - B₁ ^ (1 - s)) =
  leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B₂ ^ (1 - s)) -
  leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B₁ ^ (1 - s)) :
  ext T
show (A ^ s * (toOp T * (B₂ ^ (1 - s) - B₁ ^ (1 - s)))) : L [] =
  A ^ s * (toOp T * B₂ ^ (1 - s)) - A ^ s * (toOp T * B₁ ^ (1 - s))
simp [mul_sub]
have hrewrite :
  phiK (□ := □) K
  (leftMulHS (□ := □) (A ^ s) *
    rightMulHS (□ := □) (B₂ ^ (1 - s) - B₁ ^ (1 - s))) =
  liebTraceMap (□ := □) s K A B₂ - liebTraceMap (□ := □) s K A B₁ :=
  rw [hsplit, sub_eq_add_neg, phiK_add]
  rw [show - (leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B₁ ^ (1 - s))
    (-1 : ℝ) • (leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B₁ ^ (1 - s)))
    phiK_smul]

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDTRACE.LEAN

```
simp [phiK_leftMul_rightMul_eq_traceRe, liebTraceMap, mul_assoc, sub_eq_add_neg]
linarith [hrewrite ▶ hphi]
```

```
omit [Nontrivial []] in
set_option maxHeartbeats 400000 in
private lemma liebTraceMap_antitone_right
  {s : ℝ} (hs : 1 - s ∈ Set.Icc (-1 : ℝ) 0)
  (K A B1 B2 : L [])
  (hA : A ∈ pdSet (· := ·)) (hB1 : B1 ∈ pdSet (· := ·)) (hB2 : B2 ∈ pdSet (· := ·))
  (hB : B1 ≤ B2) :
  liebTraceMap (· := ·) s K A B2 ≤ liebTraceMap (· := ·) s K A B1 := by
  rcases hA with ⟨hAsa, hAspec⟩
  rcases hB1 with ⟨hB1sa, hB1spec⟩
  rcases hB2 with ⟨hB2sa, hB2spec⟩
  have hA0 : 0 ≤ A := by
    exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) A (ha := hAsa)).2
    (by intro x hx; exact (hAspec hx).le)
  have hcfc :=
    power_Icc_neg_one_zero_neg_operatorMonotoneOn_Ioi (· := ·) (1 - s) hs
    (A := B2) (B := B1)
    (show (0 : L []) ≤ B2 by
      exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B2 (ha := hB2sa))
      (by intro x hx; exact (hB2spec hx).le))
    (show (0 : L []) ≤ B1 by
      exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B1 (ha := hB1sa))
      (by intro x hx; exact (hB1spec hx).le))
    hB
    (by
      intro x hx
      exact hB2spec hx)
    (by
      intro x hx
      exact hB1spec hx)
  have hpow :
    B2 ^ (1 - s) ≤ B1 ^ (1 - s) := by
  have hnegpow : -(B1 ^ (1 - s)) ≤ -(B2 ^ (1 - s)) := by
    simpa [cfcR, LowerHeinzCore.cfcR, cfc_neg,
      CFC.rpow_eq_cfc_real (A := L []) (a := B1) (y := 1 - s)
      (ha := (show 0 ≤ B1 by
        exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B1 (ha := hB1sa))
        (by intro x hx; exact (hB1spec hx).le))),
      CFC.rpow_eq_cfc_real (A := L []) (a := B2) (y := 1 - s)
      (ha := (show 0 ≤ B2 by
        exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B2 (ha := hB2sa))
        (by intro x hx; exact (hB2spec hx).le))))] using hcfc
    simpa using (neg_le_neg_iff.mp hnegpow)
```

```

have hApow0 : 0 ≤ A ^ s := by
  simp
have hdiff0 : 0 ≤ B1 ^ (1 - s) - B2 ^ (1 - s) := sub_nonneg.mpr hpow
have hprod0 :
  0 ≤ leftMulHS (□ := □) (A ^ s) *
    rightMulHS (□ := □) (B1 ^ (1 - s) - B2 ^ (1 - s)) := by
  exact (leftMulHS_rightMulHS_commute (□ := □) (A ^ s) (B1 ^ (1 - s) - B2 ^ (1 - s))
    (leftMulHS_nonneg (□ := □) hApow0)
    (rightMulHS_nonneg (□ := □) hdiff0))
have hphi : 0 ≤
  phiK (□ := □) K
    (leftMulHS (□ := □) (A ^ s) *
      rightMulHS (□ := □) (B1 ^ (1 - s) - B2 ^ (1 - s))) :=
  phiK_nonneg (□ := □) K hprod0
have hsplit :
  leftMulHS (□ := □) (A ^ s) *
    rightMulHS (□ := □) (B1 ^ (1 - s) - B2 ^ (1 - s)) =
  leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B1 ^ (1 - s)) -
  leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B2 ^ (1 - s)) :
  ext T
show (A ^ s * (toOp T * (B1 ^ (1 - s) - B2 ^ (1 - s)))) : L □ =
  A ^ s * (toOp T * B1 ^ (1 - s)) - A ^ s * (toOp T * B2 ^ (1 - s))
simp [mul_sub]
have hrewrite :
  phiK (□ := □) K
    (leftMulHS (□ := □) (A ^ s) *
      rightMulHS (□ := □) (B1 ^ (1 - s) - B2 ^ (1 - s))) =
  liebTraceMap (□ := □) s K A B1 - liebTraceMap (□ := □) s K A B2 :=
  rw [hsplit, sub_eq_add_neg, phiK_add]
  rw [show -(leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B2 ^ (1 - s))
    (-1 : ℝ) • (leftMulHS (□ := □) (A ^ s) * rightMulHS (□ := □) (B2 ^ (1 - s)))
    phiK_smul]
  simp [phiK_leftMul_rightMul_eq_traceRe, liebTraceMap, mul_assoc, sub_eq]
  linarith [hrewrite ▸ hphi]

private lemma phiK_weightedSum_operatorPowerMean_eq
  {s θ : ℝ} (K A1 A2 B1 B2 : L □)
  (hA1 : A1 ∈ pdSet (□ := □)) (hA2 : A2 ∈ pdSet (□ := □))
  (hB1 : B1 ∈ pdSet (□ := □)) (hB2 : B2 ∈ pdSet (□ := □)) :
  phiK (□ := □) K
    ((1 - θ) • operatorPowerMean (□ := HSOp □) s 1
      (leftMulHS (□ := □) A1) (rightMulHS (□ := □) B1) +
      θ • operatorPowerMean (□ := HSOp □) s 1
      (leftMulHS (□ := □) A2) (rightMulHS (□ := □) B2)) =
  (1 - θ) • liebTraceMap (□ := □) s K A1 B1 +
  θ • liebTraceMap (□ := □) s K A2 B2 := by

```


1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDOTRACE.LEAN

```

rw [phiK_add, phiK_smul, phiK_smul]
simp [smul_eq_mul] using
  show (1 - θ) * phiK (α := α) K
    (operatorPowerMean (α := HSOp α) s 1
      (leftMulHS (α := α) A₁) (rightMulHS (α := α) B₁)) +
    θ * phiK (α := α) K
      (operatorPowerMean (α := HSOp α) s 1
        (leftMulHS (α := α) A₂) (rightMulHS (α := α) B₂)) =
  (1 - θ) * liebTraceMap (α := α) s K A₁ B₁ +
  θ * liebTraceMap (α := α) s K A₂ B₂ by
  simp only [phiK_operatorPowerMean_eq_liebTraceMap (α := α) (s := s) K A₁ B₁ hA₁
    phiK_operatorPowerMean_eq_liebTraceMap (α := α) (s := s) K A₂ B₂ hA₂ hB₂]

-- The `HSOp`-valued `operatorPowerMean` terms are large enough that the skeleton it
theorem liebTrace_jointlyConcaveOn_pdSet
  {s : ℝ} (hs0 : 0 < s) (hs1 : s < 1) (K : L α) :
  JointlyConcaveOn (pdSet (α := α)) (pdSet (α := α))
    (liebTraceMap (α := α) s K) := by
  intro A₁ A₂ B₁ B₂ θ hA₁ hA₂ hB₁ hB₂ hθ0 hθ1
  have hleft_combo :
    (1 - θ) • leftMulHS (α := α) A₁ + θ • leftMulHS (α := α) A₂ =
    leftMulHS (α := α) ((1 - θ) • A₁ + θ • A₂) := by
  ext T
  show ((1 - θ) • (A₁ * toOp T) + θ • (A₂ * toOp T) : L α) =
    ((1 - θ) • A₁ + θ • A₂) * toOp T
  rw [add_mul, smul_mul_assoc, smul_mul_assoc]
  have hright_combo :
    (1 - θ) • rightMulHS (α := α) B₁ + θ • rightMulHS (α := α) B₂ =
    rightMulHS (α := α) ((1 - θ) • B₁ + θ • B₂) := by
  ext T
  show ((1 - θ) • (toOp T * B₁) + θ • (toOp T * B₂) : L α) =
    toOp T * ((1 - θ) • B₁ + θ • B₂)
  rw [mul_add, mul_smul_comm, mul_smul_comm]
  have hA_combo :
    ((1 - θ) • A₁ + θ • A₂) ∈ pdSet (α := α) := by
  exact pdSet_convexCombo (α := α) hA₁ hA₂ hθ0 hθ1
  have hB_combo :
    ((1 - θ) • B₁ + θ • B₂) ∈ pdSet (α := α) := by
  exact pdSet_convexCombo (α := α) hB₁ hB₂ hθ0 hθ1
  letI : Nontrivial (HSOp α) := by
  delta HSOp
  infer_instance
  letI : Nontrivial (L (HSOp α)) := inferInstance
  have hconc_hs :=
    operatorPowerMean_jointlyConcaveOn_pdSet
      (α := HSOp α) (α := s) (β := 1)

```

```

{le_of_lt hs0, hs1.le} {by norm_num, by norm_num}
(A1 := leftMulHS (□ := □) A1) (A2 := leftMulHS (□ := □) A2)
(B1 := rightMulHS (□ := □) B1) (B2 := rightMulHS (□ := □) B2)
(θ := θ)
(leftMulHS_pdSet (□ := □) hA1) (leftMulHS_pdSet (□ := □) hA2)
(rightMulHS_pdSet (□ := □) hB1) (rightMulHS_pdSet (□ := □) hB2)
h00 h01
have hconc :
  (1 - θ) • operatorPowerMean (□ := HSOp □) s 1
    (leftMulHS (□ := □) A1) (rightMulHS (□ := □) B1) +
  θ • operatorPowerMean (□ := HSOp □) s 1
    (leftMulHS (□ := □) A2) (rightMulHS (□ := □) B2) ≤
  operatorPowerMean (□ := HSOp □) s 1
    (leftMulHS (□ := □) ((1 - θ) • A1 + θ • A2))
    (rightMulHS (□ := □) ((1 - θ) • B1 + θ • B2)) := by
  simp [hleft_combo, hright_combo] using hconc_hs
have hphi_mono :
  phiK (□ := □) K
    ((1 - θ) • operatorPowerMean (□ := HSOp □) s 1
      (leftMulHS (□ := □) A1) (rightMulHS (□ := □) B1) +
      θ • operatorPowerMean (□ := HSOp □) s 1
      (leftMulHS (□ := □) A2) (rightMulHS (□ := □) B2)) ≤
  phiK (□ := □) K
    (operatorPowerMean (□ := HSOp □) s 1
      (leftMulHS (□ := □) ((1 - θ) • A1 + θ • A2))
      (rightMulHS (□ := □) ((1 - θ) • B1 + θ • B2))) := by
  exact phiK_mono (□ := □) K hconc
rw [phiK_weightedSum_operatorPowerMean_eq (□ := □) (s := s) (θ := θ) K A1
  hA1 hA2 hB1 hB2] at hphi_mono
rw [phiK_operatorPowerMean_eq_liebTraceMap (□ := □) (s := s) K
  ((1 - θ) • A1 + θ • A2) ((1 - θ) • B1 + θ • B2) hA_combo hB_combo] at
  simp [add_comm, add_left_comm, add_assoc] using hphi_mono

theorem liebTrace_jointlyConvex0n_pdSet
  {s : ℝ} (hs1 : 1 ≤ s) (hs2 : s ≤ 2) (K : L □) :
  JointlyConvex0n (pdSet (□ := □)) (pdSet (□ := □))
    (liebTraceMap (□ := □) s K) := by
  intro A1 A2 B1 B2 θ hA1 hA2 hB1 hB2 h00 h01
  have hleft_combo :
    (1 - θ) • leftMulHS (□ := □) A1 + θ • leftMulHS (□ := □) A2 =
    leftMulHS (□ := □) ((1 - θ) • A1 + θ • A2) := by
  ext T
  show ((1 - θ) • (A1 * toOp T) + θ • (A2 * toOp T) : L □) =
    ((1 - θ) • A1 + θ • A2) * toOp T
  rw [add_mul, smul_mul_assoc, smul_mul_assoc]
  have hright_combo :

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDOTRACE.LEAN

```

    (1 - θ) • rightMulHS (□ := □) B1 + θ • rightMulHS (□ := □) B2 =
      rightMulHS (□ := □) ((1 - θ) • B1 + θ • B2) := by
  ext T
  show ((1 - θ) • (toOp T * B1) + θ • (toOp T * B2) : L □) =
    toOp T * ((1 - θ) • B1 + θ • B2)
  rw [mul_add, mul_smul_comm, mul_smul_comm]
  have hA_combo :
    ((1 - θ) • A1 + θ • A2) ∈ pdSet (□ := □) := by
    exact pdSet_convexCombo (□ := □) hA1 hA2 hθ0 hθ1
  have hB_combo :
    ((1 - θ) • B1 + θ • B2) ∈ pdSet (□ := □) := by
    exact pdSet_convexCombo (□ := □) hB1 hB2 hθ0 hθ1
  letI : Nontrivial (HSOp □) := by
    delta HSOp
    infer_instance
  letI : Nontrivial (L (HSOp □)) := inferInstance
  have hconv_hs :=
    operatorPowerMean_jointlyConvexOn_pdSet
      (□ := HSOp □) (α := s) (β := 1)
      {hs1, hs2} {by norm_num, by norm_num}
      (A1 := leftMulHS (□ := □) A1) (A2 := leftMulHS (□ := □) A2)
      (B1 := rightMulHS (□ := □) B1) (B2 := rightMulHS (□ := □) B2)
      (θ := θ)
      (leftMulHS_pdSet (□ := □) hA1) (leftMulHS_pdSet (□ := □) hA2)
      (rightMulHS_pdSet (□ := □) hB1) (rightMulHS_pdSet (□ := □) hB2)
      hθ0 hθ1
  have hconv :
    operatorPowerMean (□ := HSOp □) s 1
      (leftMulHS (□ := □) ((1 - θ) • A1 + θ • A2))
      (rightMulHS (□ := □) ((1 - θ) • B1 + θ • B2)) ≤
      (1 - θ) • operatorPowerMean (□ := HSOp □) s 1
        (leftMulHS (□ := □) A1) (rightMulHS (□ := □) B1) +
        θ • operatorPowerMean (□ := HSOp □) s 1
          (leftMulHS (□ := □) A2) (rightMulHS (□ := □) B2) := by
    simpa [hleft_combo, hright_combo] using hconv_hs
  have hphi_mono :
    phiK (□ := □) K
      (operatorPowerMean (□ := HSOp □) s 1
        (leftMulHS (□ := □) ((1 - θ) • A1 + θ • A2))
        (rightMulHS (□ := □) ((1 - θ) • B1 + θ • B2))) ≤
    phiK (□ := □) K
      ((1 - θ) • operatorPowerMean (□ := HSOp □) s 1
        (leftMulHS (□ := □) A1) (rightMulHS (□ := □) B1) +
        θ • operatorPowerMean (□ := HSOp □) s 1
          (leftMulHS (□ := □) A2) (rightMulHS (□ := □) B2))) := by
    exact phiK_mono (□ := □) K hconv

```

```

rw [phiK_operatorPowerMean_eq_liebTraceMap (□ := □) (s := s) K
  ((1 - θ) • A1 + θ • A2) ((1 - θ) • B1 + θ • B2) hA_combo hB_combo] at
rw [phiK_weightedSum_operatorPowerMean_eq (□ := □) (s := s) (θ := θ) K A1
  hA1 hA2 hB1 hB2] at hphi_mono
simp [add_comm, add_left_comm, add_assoc] using hphi_mono

set_option maxHeartbeats 600000 in
theorem liebExtensionTrace_jointlyConcaveOn_pdSet
  {p q : ℝ} (hp : 0 < p) (hq : 0 < q) (hpq : p + q ≤ 1) (K : L □) :
  JointlyConcaveOn (pdSet (□ := □)) (pdSet (□ := □))
    (liebExtensionTraceMap (□ := □) q p K) := by
  have hq1 : q < 1 := by linarith
  let β : ℝ := p / (1 - q)
  have h1q : 0 < 1 - q := by linarith
  have hβ0 : 0 ≤ β := by
    dsimp [β]
    positivity
  have hβ1 : β ≤ 1 := by
    dsimp [β]
    field_simp [h1q.ne']
    linarith
  have hβ : β ∈ Set.Icc (0 : ℝ) 1 := ⟨hβ0, hβ1⟩
  have hβmul : β * (1 - q) = p := by
    dsimp [β]
    field_simp [h1q.ne']
  intro A1 A2 B1 B2 θ hA1 hA2 hB1 hB2 hθ0 hθ1
  have hA_combo :
    ((1 - θ) • A1 + θ • A2) ∈ pdSet (□ := □) := by
    exact pdSet_convexCombo (□ := □) hA1 hA2 hθ0 hθ1
  have hB_combo :
    ((1 - θ) • B1 + θ • B2) ∈ pdSet (□ := □) := by
    exact pdSet_convexCombo (□ := □) hB1 hB2 hθ0 hθ1
  have hB1β : B1 ^ β ∈ pdSet (□ := □) :=
    pdSet_rpow_of_mem_Icc_zero_one (□ := □) hβ hB1
  have hB2β : B2 ^ β ∈ pdSet (□ := □) :=
    pdSet_rpow_of_mem_Icc_zero_one (□ := □) hβ hB2
  have hB_comboβ :
    (((1 - θ) • B1 + θ • B2) ^ β) ∈ pdSet (□ := □) :=
    pdSet_rpow_of_mem_Icc_zero_one (□ := □) hβ hB_combo
  have hBpow_combo :
    ((1 - θ) • (B1 ^ β) + θ • (B2 ^ β)) ∈ pdSet (□ := □) := by
    exact pdSet_convexCombo (□ := □) hB1β hB2β hθ0 hθ1
  have hB1_mem := hB1
  have hB2_mem := hB2
  have hB_combo_mem := hB_combo
  rcases hB1 with ⟨hB1_sa, hB1_spec⟩

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDTRACE.LEAN

```

rcases hB2 with ⟨hB2_sa, hB2_spec⟩
rcases hB_combo with ⟨hB_combo_sa, hB_combo_spec⟩
have hB10 : 0 ≤ B1 := by
  exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B1 (ha := hB1_sa)).2
  (by intro x hx; exact (hB1_spec hx).le)
have hB20 : 0 ≤ B2 := by
  exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B2 (ha := hB2_sa)).2
  (by intro x hx; exact (hB2_spec hx).le)
have hBcombo0 : 0 ≤ ((1 - θ) • B1 + θ • B2) := by
  exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) ((1 - θ) • B1 + θ • B2)
    (ha := hB_combo_sa)).2
  (by intro x hx; exact (hB_combo_spec hx).le)
have hpow_conc :=
  power_Icc_zero_one_operatorConcaveOn_Ici (□ := □) β hβ
  (A := B1) (B := B2) (t := θ) hB1_sa hB2_sa h00 h01
  (by
    intro x hx
    have hx0 : 0 < x := by simpa [Set.Ioi] using hB1_spec hx
    exact hx0.le)
  (by
    intro x hx
    have hx0 : 0 < x := by simpa [Set.Ioi] using hB2_spec hx
    exact hx0.le)
have hBpow_le :
  (1 - θ) • (B1 ^ β) + θ • (B2 ^ β) ≤
  ((1 - θ) • B1 + θ • B2) ^ β := by
  have hBcombo0' : 0 ≤ θ • B2 + (1 - θ) • B1 := by
    simpa [add_comm, add_left_comm, add_assoc] using hBcombo0
  have hneg :
    -(((1 - θ) • B1 + θ • B2) ^ β) ≤
    -((1 - θ) • (B1 ^ β) + θ • (B2 ^ β)) := by
    simpa [cfcR, LowerHeinzCore.cfcR, cfc_neg, smul_neg, neg_add,
      add_comm, add_left_comm, add_assoc,
      CFC.rpow_eq_cfc_real (A := L □) (a := B1) (y := β) (ha := hB10),
      CFC.rpow_eq_cfc_real (A := L □) (a := B2) (y := β) (ha := hB20),
      CFC.rpow_eq_cfc_real (A := L □) (a := θ • B2 + (1 - θ) • B1) (y := β) (ha :=
      CFC.rpow_eq_cfc_real (A := L □) (a := ((1 - θ) • B1 + θ • B2)) (y := β) (ha :=
      using hpow_conc
    exact neg_le_neg_iff.mp hneg
have hsmono : 1 - q ∈ Set.Icc (0 : ℝ) 1 := by
  constructor <=> linarith
have hmono :
  liebTraceMap (□ := □) q K
  ((1 - θ) • A1 + θ • A2)
  ((1 - θ) • (B1 ^ β) + θ • (B2 ^ β)) ≤
  liebTraceMap (□ := □) q K

```

```

      ((1 - θ) • A1 + θ • A2)
      (((1 - θ) • B1 + θ • B2) ^ β) := by
exact liebTraceMap_mono_right (□ := □) (s := q) hsmono K
      ((1 - θ) • A1 + θ • A2)
      ((1 - θ) • (B1 ^ β) + θ • (B2 ^ β))
      (((1 - θ) • B1 + θ • B2) ^ β)
      hA_combo hBpow_combo hB_comboβ hBpow_le
have hconc :=
  liebTrace_jointlyConcaveOn_pdSet (□ := □) (s := q) hq hq1 K
  (A1 := A1) (A2 := A2) (B1 := B1 ^ β) (B2 := B2 ^ β)
  (θ := θ) hA1 hA2 hB1β hB2β hθ0 hθ1
have hpow_rewrite :
  ∀ {A B : L □}, B ∈ pdSet (□ := □) →
    liebTraceMap (□ := □) q K A (B ^ β) =
      liebExtensionTraceMap (□ := □) q p K A B := by
intro A B hB
rcases hB with ⟨hB_sa, hB_spec⟩
have hB0 : 0 ≤ B := by
  exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B (ha := h
    (by intro x hx; exact (hB_spec hx).le)
have hpow :
  (B ^ β) ^ (1 - q) = B ^ p := by
calc
  (B ^ β) ^ (1 - q) = B ^ (β * (1 - q)) := by
    simpa using
      (CFC.rpow_rpow_of_exponent_nonneg (A := L □) B β (1 - q) hβ0
        _ = B ^ p := by rw [hβmul]
    simp [liebTraceMap, liebExtensionTraceMap, hpow, mul_assoc]
simpa [hpow_rewrite hB1_mem, hpow_rewrite hB2_mem, hpow_rewrite hB_combo_
  (le_trans hconc hmono)

set_option maxHeartbeats 600000 in
theorem andoTrace_jointlyConvexOn_pdSet
  {q r : ℝ} (hq1 : 1 ≤ q) (hq2 : q ≤ 2) (hr0 : 0 ≤ r) (hr1 : r ≤ 1)
  (hqr : 1 ≤ q - r) (K : L □) :
  JointlyConvexOn (pdSet (□ := □)) (pdSet (□ := □))
  (andoTraceMap (□ := □) q r K) := by
by_cases hqeq : q = 1
· have hrz : r = 0 := by linarith
  subst hqeq
  subst hrz
  convert (liebTrace_jointlyConvexOn_pdSet (s := 1) (by norm_num) (by norm
    ext A B
  simp [andoTraceMap, liebTraceMap]
· have hqgt : 1 < q := lt_of_le_of_ne hq1 (Ne.symm hqeq)
  let β : ℝ := r / (q - 1)

```

1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDOTRACE.LEAN

```

have hqlpos :  $0 < q - 1$  := by linarith
have hβ0 :  $0 \leq \beta$  := by
  dsimp [β]
  positivity
have hβ1 :  $\beta \leq 1$  := by
  dsimp [β]
  field_simp [hqlpos.ne']
  linarith
have hβ :  $\beta \in \text{Set.Icc } (0 : \mathbb{R}) \ 1$  := ⟨hβ0, hβ1⟩
have hβmul :  $\beta * (1 - q) = -r$  := by
  dsimp [β]
  field_simp [hqlpos.ne']
  ring
intro A1 A2 B1 B2 θ hA1 hA2 hB1 hB2 hθ0 hθ1
have hA_combo :
   $((1 - \theta) \cdot A_1 + \theta \cdot A_2) \in \text{pdSet } (\square := \square) :=$  by
  exact pdSet_convexCombo ( $\square := \square$ ) hA1 hA2 hθ0 hθ1
have hB_combo :
   $((1 - \theta) \cdot B_1 + \theta \cdot B_2) \in \text{pdSet } (\square := \square) :=$  by
  exact pdSet_convexCombo ( $\square := \square$ ) hB1 hB2 hθ0 hθ1
have hB1β :  $B_1^\beta \in \text{pdSet } (\square := \square) :=$ 
  pdSet_rpow_of_mem_Icc_zero_one ( $\square := \square$ ) hβ hB1
have hB2β :  $B_2^\beta \in \text{pdSet } (\square := \square) :=$ 
  pdSet_rpow_of_mem_Icc_zero_one ( $\square := \square$ ) hβ hB2
have hB_comboβ :
   $((1 - \theta) \cdot B_1 + \theta \cdot B_2)^\beta \in \text{pdSet } (\square := \square) :=$ 
  pdSet_rpow_of_mem_Icc_zero_one ( $\square := \square$ ) hβ hB_combo
have hBpow_combo :
   $((1 - \theta) \cdot (B_1^\beta) + \theta \cdot (B_2^\beta)) \in \text{pdSet } (\square := \square) :=$  by
  exact pdSet_convexCombo ( $\square := \square$ ) hB1β hB2β hθ0 hθ1
have hB1_mem := hB1
have hB2_mem := hB2
have hB_combo_mem := hB_combo
rcases hB1 with ⟨hB1_sa, hB1_spec⟩
rcases hB2 with ⟨hB2_sa, hB2_spec⟩
rcases hB_combo with ⟨hB_combo_sa, hB_combo_spec⟩
have hB10 :  $0 \leq B_1$  := by
  exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B1 (ha := hB1_sa)).
    (by intro x hx; exact (hB1_spec hx).le)
have hB20 :  $0 \leq B_2$  := by
  exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B2 (ha := hB2_sa)).
    (by intro x hx; exact (hB2_spec hx).le)
have hBcombo0 :  $0 \leq ((1 - \theta) \cdot B_1 + \theta \cdot B_2)$  := by
  exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) ((1 - θ) · B1 + θ ·
    (ha := hB_combo_sa)).2
    (by intro x hx; exact (hB_combo_spec hx).le)

```

```

have hpow_conc :=
  power_Icc_zero_one_operatorConcaveOn_Ici (· := ·) β hβ
  (A := B1) (B := B2) (t := θ) hB1_sa hB2_sa hθ0 hθ1
  (by
    intro x hx
    have hx0 : 0 < x := by simp [Set.Ioi] using hB1_spec hx
    exact hx0.le)
  (by
    intro x hx
    have hx0 : 0 < x := by simp [Set.Ioi] using hB2_spec hx
    exact hx0.le)
have hBcombo0' : 0 ≤ θ • B2 + (1 - θ) • B1 := by
  simp [add_comm, add_left_comm, add_assoc] using hBcombo0
have hBpow_le :
  (1 - θ) • (B1 ^ β) + θ • (B2 ^ β) ≤
  ((1 - θ) • B1 + θ • B2) ^ β := by
  have hneg :
    -(((1 - θ) • B1 + θ • B2) ^ β) ≤
    -((1 - θ) • (B1 ^ β) + θ • (B2 ^ β)) := by
    simp [cfcR, LowerHeinzCore.cfcR, cfc_neg, smul_neg, neg_add,
      add_comm, add_left_comm, add_assoc,
      CFC.rpow_eq_cfc_real (A := L []) (a := B1) (y := β) (ha := hB1θ),
      CFC.rpow_eq_cfc_real (A := L []) (a := B2) (y := β) (ha := hB2θ),
      CFC.rpow_eq_cfc_real (A := L []) (a := θ • B2 + (1 - θ) • B1) (y :=
      CFC.rpow_eq_cfc_real (A := L []) (a := ((1 - θ) • B1 + θ • B2)) (y :=
      using hpow_conc
    exact neg_le_neg_iff.mp hneg
have hsanti : 1 - q ∈ Set.Icc (-1 : ℝ) 0 := by
  constructor <=> linarith
have hmono :
  liebTraceMap (· := ·) q K
  ((1 - θ) • A1 + θ • A2)
  (((1 - θ) • B1 + θ • B2) ^ β) ≤
  liebTraceMap (· := ·) q K
  ((1 - θ) • A1 + θ • A2)
  ((1 - θ) • (B1 ^ β) + θ • (B2 ^ β)) := by
  exact liebTraceMap_antitone_right (· := ·) (s := q) hsanti K
  ((1 - θ) • A1 + θ • A2)
  ((1 - θ) • (B1 ^ β) + θ • (B2 ^ β))
  (((1 - θ) • B1 + θ • B2) ^ β)
  hA_combo hBpow_combo hB_comboβ hBpow_le
have hconv :=
  liebTrace_jointlyConvexOn_pdSet (· := ·) (s := q) hq1 hq2 K
  (A1 := A1) (A2 := A2) (B1 := B1 ^ β) (B2 := B2 ^ β)
  (θ := θ) hA1 hA2 hB1β hB2β hθ0 hθ1
have hpow_rewrite :

```


1.121.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LIEBANDTRACE.LEAN

```

    ∀ {A B : L []}, B ∈ pdSet ([] := []) →
      liebTraceMap ([] := []) q K A (B ^ β) =
        andoTraceMap ([] := []) q r K A B := by
intro A B hB
rcases hB with ⟨hB_sa, hB_spec⟩
have hB0 : 0 ≤ B := by
  exact (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B (ha := hB_sa)).
    (by intro x hx; exact (hB_spec hx).le)
have hBunit : IsUnit B := by
  refine spectrum.isUnit_of_zero_notMem (R := ℝ) ?_
intro h0
  exact (lt_irrefl (0 : ℝ)) (by simp [Set.Ioi] using hB_spec h0)
have hpow :
  (B ^ β) ^ (1 - q) = B ^ (-r) := by
by_cases hβzero : β = 0
· have hrz : r = 0 := by
  rw [hβzero] at hβmul
  linarith
have hBzero : B ^ (0 : ℝ) = (1 : L []) := by
  simp using (CFC.rpow_zero (a := B))
calc
  (B ^ β) ^ (1 - q) = (B ^ (0 : ℝ)) ^ (1 - q) := by simp [hβzero]
  _ = (1 : L []) ^ (1 - q) := by rw [hBzero]
  _ = (1 : L []) := by simp
  _ = B ^ (0 : ℝ) := by rw [hBzero]
  _ = B ^ (-r) := by simp [hrz]
· calc
  (B ^ β) ^ (1 - q) = B ^ (β * (1 - q)) := by
    simp [mul_comm] using
      (CFC.rpow_rpow B β (1 - q) hβzero (hBunit.isStrictlyPositive hB0))
  _ = B ^ (-r) := by rw [hβmul]
simp [liebTraceMap, andoTraceMap, hpow, mul_assoc]
simp [hpow_rewrite hB1_mem, hpow_rewrite hB2_mem, hpow_rewrite hB_combo_mem] us
  (le_trans hmono hconv)

theorem liebCorollaryTrace_jointlyConvex0n_pdSet
  {q r : ℝ} (hr1 : 1 < r) (hrq : r ≤ q) (hq2 : q ≤ 2) (K : L []) :
  JointlyConvex0n (pdSet ([] := [])) (pdSet ([] := []))
  (liebCorollaryTraceMap ([] := []) q r K) := by
have hq1 : 1 ≤ q := by linarith
have hr0 : 0 ≤ r - 1 := by linarith
have hr1' : r - 1 ≤ 1 := by linarith
have hqr' : 1 ≤ q - (r - 1) := by linarith
convert
  (andoTrace_jointlyConvex0n_pdSet ([] := []) (q := q) (r := r - 1)
    hq1 hq2 hr0 hr1' hqr' K) using 1

```

1.122 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/Lowner

```

/-
Copyright (c) 2026 Hayata Yamasaki. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Kei Tsukamoto, Kento Mori, Hayata Yamasaki
-/

module

public import Mathlib.Analysis.CStarAlgebra.ContinuousFunctionalCalculus.Op
public import Mathlib.Analysis.SpecialFunctions.ContinuousFunctionalCalculus
public import Mathlib.Analysis.SpecialFunctions.ContinuousFunctionalCalculus
public import Mathlib.LinearAlgebra.Matrix.PosDef

@[expose] public section

/-!
## Löwner–Heinz Core / Wrapper

The Löwner–Heinz inequality states that for positive semidefinite operators  $A$  and  $B$ , and for  $0 \leq t \leq 1$ , the operator  $A^t B^{1-t}$  is also positive semidefinite. This section provides a formalization of this inequality in Lean, along with a core wrapper for the spectral order.

- `section Pure` defines the core wrapper for the spectral order.
- `section Spectrum` defines the spectral order class and its properties.
- `namespace LownerHeinzCore.Spectral` defines the spectral order class and its properties.

`spectralOrder` is a core wrapper for the spectral order.
`NonnegSpectrumClass` is a class for non-negative spectrum.
`infer_instance` is a function to infer the instance.

-/

namespace LownerHeinzCore

universe u v

open CFC

section Pure

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWER-RHEINZCORE.LEAN

```
variable {α : Type u}
variable [CStarAlgebra α] [PartialOrder α] [StarOrderedRing α]
variable [Nontrivial α]

noncomputable abbrev cfcR (f : ℝ → ℝ) (A : α) : α :=
  cfc (R := ℝ) (A := α) (p := IsSelfAdjoint) f A

/-- Fixed-space operator monotonicity on the ambient algebra `α`. -
/
def OperatorMonotone (f : ℝ → ℝ) : Prop :=
  ∀ A B : α, 0 ≤ A → 0 ≤ B → B ≤ A → cfcR f B ≤ cfcR f A

/-- Fixed-space operator monotonicity on `s` for the ambient algebra `α`. -
/
def OperatorMonotoneOn (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  ∀ A B : α,
    0 ≤ A → 0 ≤ B → B ≤ A →
    spectrum ℝ A ⊆ s → spectrum ℝ B ⊆ s →
    cfcR f B ≤ cfcR f A

/-- Fixed-space operator antitonicity on the ambient algebra `α`. -
/
def OperatorAntitone (f : ℝ → ℝ) : Prop :=
  ∀ A B : α, 0 ≤ A → 0 ≤ B → B ≤ A →
    cfcR f A ≤ cfcR f B

/-- Fixed-space operator antitonicity on `s` for the ambient algebra `α`. -
/
def OperatorAntitoneOn (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  ∀ A B : α,
    0 ≤ A → 0 ≤ B → B ≤ A →
    spectrum ℝ A ⊆ s → spectrum ℝ B ⊆ s →
    cfcR f A ≤ cfcR f B

/-- Fixed-space operator convexity on the ambient algebra `α`. -
/
def OperatorConvex (f : ℝ → ℝ) : Prop :=
  ∀ A B : α, t : ℝ, 0 ≤ t → t ≤ 1 →
    cfcR f ((1 - t) • A + t • B)
    ≤ (1 - t) • cfcR f A + t • cfcR f B

/-- Fixed-space operator convexity on `s` for the ambient algebra `α`. -
/
def OperatorConvexOn (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  ∀ A B : α, t : ℝ,
```

```

IsSelfAdjoint A → IsSelfAdjoint B →
0 ≤ t → t ≤ 1 →
spectrum ℝ A ⊆ s → spectrum ℝ B ⊆ s →
cfcR f ((1 - t) • A + t • B)
  ≤ (1 - t) • cfcR f A + t • cfcR f B

/-- Fixed-space operator concavity on the ambient algebra `α`. -
/
def OperatorConcave (f : ℝ → ℝ) : Prop :=
  OperatorConvex (α := α) (fun x => - f x)

/-- Fixed-space operator concavity on `s` for the ambient algebra `α`. -
/
def OperatorConcaveOn (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  OperatorConvexOn (α := α) (s : Set ℝ) (fun x => - f x)

omit α [CStarAlgebra α] [PartialOrder α] [StarOrderedRing α] [Nontrivial α]
/-- Uniform operator monotonicity over all ambient algebras in universe `u`
/
def OperatorMonotoneAll (f : ℝ → ℝ) : Prop :=
  ∀ {α : Type u} [CStarAlgebra α] [PartialOrder α] [StarOrderedRing α]
    [ContinuousFunctionalCalculus ℝ α IsSelfAdjoint] [Nontrivial α],
    OperatorMonotone (α := α) f

omit α [CStarAlgebra α] [PartialOrder α] [StarOrderedRing α] [Nontrivial α]
/-- Uniform operator monotonicity on `s` over all ambient algebras in universe `u`
/
def OperatorMonotoneOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  ∀ {α : Type u} [CStarAlgebra α] [PartialOrder α] [StarOrderedRing α]
    [ContinuousFunctionalCalculus ℝ α IsSelfAdjoint] [Nontrivial α],
    OperatorMonotoneOn (α := α) s f

omit α [CStarAlgebra α] [PartialOrder α] [StarOrderedRing α] [Nontrivial α]
/-- Uniform operator antitonicity over all ambient algebras in universe `u`
/
def OperatorAntitoneAll (f : ℝ → ℝ) : Prop :=
  ∀ {α : Type u} [CStarAlgebra α] [PartialOrder α] [StarOrderedRing α]
    [ContinuousFunctionalCalculus ℝ α IsSelfAdjoint] [Nontrivial α],
    OperatorAntitone (α := α) f

omit α [CStarAlgebra α] [PartialOrder α] [StarOrderedRing α] [Nontrivial α]
/-- Uniform operator antitonicity on `s` over all ambient algebras in universe `u`
/
def OperatorAntitoneOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  ∀ {α : Type u} [CStarAlgebra α] [PartialOrder α] [StarOrderedRing α]
    [ContinuousFunctionalCalculus ℝ α IsSelfAdjoint] [Nontrivial α],

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWERNERHEINZCORE.LEAN

```
OperatorAntitoneOn (α := β) s f

omit β [CStarAlgebra β] [PartialOrder β] [StarOrderedRing β] [Nontrivial β] in
/-- Uniform operator convexity over all ambient algebras in universe `u`. -
/
def OperatorConvexAll (f : ℝ → ℝ) : Prop :=
  ∀ {β : Type u} [CStarAlgebra β] [PartialOrder β] [StarOrderedRing β]
    [ContinuousFunctionalCalculus ℝ β IsSelfAdjoint] [Nontrivial β],
    OperatorConvex (α := β) f

omit β [CStarAlgebra β] [PartialOrder β] [StarOrderedRing β] [Nontrivial β] in
/-- Uniform operator convexity on `s` over all ambient algebras in universe `u`. -
/
def OperatorConvexOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  ∀ {β : Type u} [CStarAlgebra β] [PartialOrder β] [StarOrderedRing β]
    [ContinuousFunctionalCalculus ℝ β IsSelfAdjoint] [Nontrivial β],
    OperatorConvexOn (α := β) s f

omit β [CStarAlgebra β] [PartialOrder β] [StarOrderedRing β] [Nontrivial β] in
/-- Uniform operator concavity over all ambient algebras in universe `u`. -
/
def OperatorConcaveAll (f : ℝ → ℝ) : Prop :=
  ∀ {β : Type u} [CStarAlgebra β] [PartialOrder β] [StarOrderedRing β]
    [ContinuousFunctionalCalculus ℝ β IsSelfAdjoint] [Nontrivial β],
    OperatorConcave (α := β) f

omit β [CStarAlgebra β] [PartialOrder β] [StarOrderedRing β] [Nontrivial β] in
/-- Uniform operator concavity on `s` over all ambient algebras in universe `u`. -
/
def OperatorConcaveOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  ∀ {β : Type u} [CStarAlgebra β] [PartialOrder β] [StarOrderedRing β]
    [ContinuousFunctionalCalculus ℝ β IsSelfAdjoint] [Nontrivial β],
    OperatorConcaveOn (α := β) s f

end Pure

section Spectrum

variable {β : Type u}
variable [CStarAlgebra β] [PartialOrder β] [StarOrderedRing β]
variable [Nontrivial β]
variable [NonnegSpectrumClass ℝ β]

omit [Nontrivial (β)] [NonnegSpectrumClass ℝ β] in
lemma conjugate_isPositive {X T : β} (hX : 0 ≤ X) (hT : IsSelfAdjoint T) :
  0 ≤ T * X * T := by
```

```

simpa using hT.conjugate_nonneg hX

omit [Nontrivial  $\square$ ] [NonnegSpectrumClass  $\mathbb{R}$   $\square$ ] in
theorem one_div_operatorAntitoneOn_Ioi :
  OperatorAntitoneOn ( $\square := \square$ ) (Set.Ioi ( $0 : \mathbb{R}$ )) (fun x :  $\mathbb{R} \mapsto 1 / x$ ) := by
  dsimp [OperatorAntitoneOn]
  intro A B A_nonneg B_nonneg BA As Bs
  let f :  $\mathbb{R} \rightarrow \mathbb{R} := \text{fun } x \mapsto x$ 
  have hA_sa : IsSelfAdjoint A := IsSelfAdjoint.of_nonneg A_nonneg
  have hB_sa : IsSelfAdjoint B := IsSelfAdjoint.of_nonneg B_nonneg
  have hA_ne0 :  $\forall x \in \text{spectrum } \mathbb{R} A, f\ x \neq 0$  := by
    intro x hx
    exact ne_of_gt (As hx)
  have hB_ne0 :  $\forall x \in \text{spectrum } \mathbb{R} B, f\ x \neq 0$  := by
    intro x hx
    exact ne_of_gt (Bs hx)
  let uA : ( $\square$ )x :=
    cfcUnits (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint) f A hA_ne0 (ha := hA_sa)
  let uB : ( $\square$ )x :=
    cfcUnits (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint) f B hB_ne0 (ha := hB_sa)
  have huA_val : (uA :  $\square$ ) = A := by
    simp [uA, cfcUnits, f, cfc_id' (R :=  $\mathbb{R}$ ) (a := A) (ha := hA_sa)]
  have huB_val : (uB :  $\square$ ) = B := by
    simp [uB, cfcUnits, f, cfc_id' (R :=  $\mathbb{R}$ ) (a := B) (ha := hB_sa)]
  have huB_nonneg :  $0 \leq (uB : \square)$  := by
    simpa [huB_val] using B_nonneg
  have hub_le_hua : (uB :  $\square$ )  $\leq$  (uA :  $\square$ ) := by
    simpa [huA_val, huB_val] using BA
  have hinv : ( $\uparrow uA^{-1} : \square$ )  $\leq$  ( $\uparrow uB^{-1} : \square$ ) := by
    simpa using
      (CStarAlgebra.inv_le_inv (A :=  $\square$ ) (a := uB) (b := uA) huB_nonneg hub_
  -- convert the inverse inequality back to the desired `cfcR` inequality
  simpa [uA, uB, cfcUnits, cfcR, f, one_div] using hinv

private lemma spectrum_convexCombo_Ioi {A B :  $\square$ } {t :  $\mathbb{R}$ }
  (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B) (ht0 :  $0 \leq t$ ) (ht1 :  $t \leq 1$ )
  (As : spectrum  $\mathbb{R} A \subseteq \text{Set.Ioi } (0 : \mathbb{R})$ ) (Bs : spectrum  $\mathbb{R} B \subseteq \text{Set.Ioi } (0 : \mathbb{R})$ )
  (spect : spectrum  $\mathbb{R} ((1 - t) \cdot A + t \cdot B) \subseteq \text{Set.Ioi } (0 : \mathbb{R})$ ) := by
  set C :  $\square := (1 - t) \cdot A + t \cdot B$ 
  have hC : IsSelfAdjoint C := by
    simpa [C] using (IsSelfAdjoint.all (1 - t)).smul hA |>.add ((IsSelfAdjoint.all t).smul hB)
  have hApos :  $\exists r > 0, \text{algebraMap } \mathbb{R} (\square) r \leq A$  := by
    refine (CFC.exists_pos_algebraMap_le_iff (A :=  $\square$ ) (a := A) (ha := hA)).
    intro x hx
    exact As hx
  have hBpos :  $\exists r > 0, \text{algebraMap } \mathbb{R} (\square) r \leq B$  := by

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWER RHEINZCORE.LEAN

```

    refine (CFC.exists_pos_algebraMap_le_iff (A := []) (a := B) (ha := hB)).2 ?_
    intro x hx
    exact Bs hx
  rcases hApos with ⟨rA, hrA, hrA_le⟩
  rcases hBpos with ⟨rB, hrB, hrB_le⟩
  set rC : ℝ := (1 - t) * rA + t * rB
  have hrC : 0 < rC := by
    by_cases h1t : (1 - t) = 0
    · have ht' : t = 1 := by simpa [sub_eq_zero] using (sub_eq_zero.mp h1t).symm
      subst ht'
      simpa [rC, h1t] using hrB
    · simpa [rC] using add_pos_of_pos_of_nonneg (mul_pos (lt_of_le_of_ne' (sub_nonneg
have hrC_le : algebraMap ℝ ([]) rC ≤ C := by
  have hsum : (1 - t) • algebraMap ℝ ([]) rA + t • algebraMap ℝ ([]) rB ≤ C := by
    simpa [C] using add_le_add (smul_le_smul_of_nonneg_left hrA_le (sub_nonneg.mpr
have hLHS :
  (1 - t) • algebraMap ℝ ([]) rA + t • algebraMap ℝ ([]) rB =
    algebraMap ℝ ([]) rC := by
    simp [rC, Algebra.smul_def]
  simpa [hLHS] using hsum
intro x hx
  simpa [C] using (CFC.exists_pos_algebraMap_le_iff (A := []) (a := C) (ha := hC)).1

omit [Nontrivial ([])] in
omit [NonnegSpectrumClass ℝ []] in
private lemma posSemidef_block_one_inv {A : []} (hA : IsSelfAdjoint A)
  (As : spectrum ℝ A ⊆ Set.Ioi (0 : ℝ)) :
  Matrix.PosSemidef
    (!! [A, 1; 1, cfcR (fun x : ℝ ↦ x⁻¹) A] : Matrix (Fin 2) (Fin 2) []) := by
  -- Gram matrix construction using `A^{1/2}` and `A^{-1/2}`
  set sqrtA : [] := cfcR (fun x : ℝ ↦ x ^ ((1 : ℝ) / 2)) A
  set invSqrtA : [] := cfcR (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) A
  set v : Fin 2 → [] := fun i => if i = 0 then sqrtA else invSqrtA
  have posV : Matrix.PosSemidef (Matrix.vecMulVec v (star v)) := by
    simpa using (Matrix.posSemidef_vecMulVec_self_star (R := []) v)
  have hsqrtA : IsSelfAdjoint sqrtA := by
    dsimp [sqrtA, cfcR]
    exact cfc_predicate _
  have hinvSqrtA : IsSelfAdjoint invSqrtA := by
    dsimp [invSqrtA, cfcR]
    exact cfc_predicate _
  have hcont_sqrt : ContinuousOn (fun x : ℝ ↦ x ^ ((1 : ℝ) / 2)) (spectrum ℝ A) :=
    fun x hx => (Real.continuousAt_rpow_const x _ (0r.inl (ne_of_gt (As hx)))).conti
  have hcont_invSqrt : ContinuousOn (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (spectrum ℝ A)
    fun x hx => (Real.continuousAt_rpow_const x _ (0r.inl (ne_of_gt (As hx)))).conti
  have sqrtA_mul_invSqrtA : sqrtA * invSqrtA = (1 : []) := by

```

```

dsimp [sqrtA, invSqrtA, cfcR]
rw [← cfc_mul _ _ A hcont_sqrt hcont_invSqrt, ← cfc_const_one ℝ A]
apply cfc_congr
intro x hx
dsimp only
rw [← Real.rpow_add (As hx), show ((1 : ℝ) / 2 + (-1) / 2 : ℝ) = 0 from
have invSqrtA_mul_sqrtA : invSqrtA * sqrtA = (1 : ℝ) := by
dsimp [sqrtA, invSqrtA, cfcR]
rw [← cfc_mul _ _ A hcont_invSqrt hcont_sqrt, ← cfc_const_one ℝ A]
apply cfc_congr
intro x hx
dsimp only
rw [← Real.rpow_add (As hx), show ((-1 : ℝ) / 2 + (1 : ℝ) / 2 : ℝ) = 0
have invSqrtA_mul_invSqrtA : invSqrtA * invSqrtA = cfcR (fun x : ℝ ↦ x ^
1 : ℝ) A := by
dsimp [invSqrtA, cfcR]
rw [← cfc_mul _ _ A hcont_invSqrt hcont_invSqrt]
apply cfc_congr
intro x hx
dsimp only
rw [← Real.rpow_add (As hx), show ((-1 : ℝ) / 2 + (-1 : ℝ) / 2 : ℝ) = -
1 from by ring]
have sqrtA_mul_sqrtA : sqrtA * sqrtA = A := by
dsimp [sqrtA, cfcR]
rw [← cfc_mul _ _ A hcont_sqrt hcont_sqrt]
calc
  cfcR (fun x : ℝ ↦ x ^ ((1 : ℝ) / 2) * x ^ ((1 : ℝ) / 2)) A =
    cfcR (fun x : ℝ ↦ x) A := by
      apply cfc_congr
      intro x hx
      dsimp only
      rw [← Real.rpow_add (As hx), show ((1 : ℝ) / 2 + (1 : ℝ) / 2 :
_ = A := cfc_id' (R := ℝ) (a := A) (ha := hA)
have invA_eq : cfcR (fun x : ℝ ↦ x ^ (-1 : ℝ)) A = cfcR (fun x : ℝ ↦ x-1)
dsimp [cfcR]
apply cfc_congr
intro x hx
have hxne : x ≠ 0 := ne_of_gt (As hx)
simp [hxne] using (Real.rpow_neg_one x)
have hEq : Matrix.vecMulVec v (star v) = (!! [A, 1; 1, cfcR (fun x : ℝ ↦ x
Matrix (Fin 2) (Fin 2) (ℝ)) := by
ext i j
fin_cases i <=> fin_cases j <=>
  simp [Matrix.vecMulVec_apply, v, hsqrtA.star_eq, hinvSqrtA.star_eq, s
    sqrtA_mul_invSqrtA, invSqrtA_mul_sqrtA, invSqrtA_mul_invSqrtA, invA
simp [hEq] using posV

```


1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWENTRERHEINZCORE.LEAN

```

omit [Nontrivial (α)] in
omit [PartialOrder α] [StarOrderedRing α] [NonnegSpectrumClass ℝ α] in
private lemma schur_conj_eq_diagonal {C D invC : α} (hInvC_sa : IsSelfAdjoint invC)
  (invC_mul_C : invC * C = (1 : α)) (C_mul_invC : C * invC = (1 : α)) :
  star (![!(1 : α), -invC; 0, 1] : Matrix (Fin 2) (Fin 2) (α))
    * (![C, 1; 1, D] : Matrix (Fin 2) (Fin 2) (α))
    * (![!(1 : α), -invC; 0, 1] : Matrix (Fin 2) (Fin 2) (α))
  = Matrix.diagonal (fun i : Fin 2 => if i = 0 then C else D - invC) := by
set U : Matrix (Fin 2) (Fin 2) (α) := (![!(1 : α), -invC; 0, 1]
have hstarU : star U = (![!(1 : α), 0; -invC, 1] := by
  dsimp [U]
  ext i j
  fin_cases i <=> fin_cases j <=> simp [hInvC_sa.star_eq]
have hP :
  star U * (![C, 1; 1, D] : Matrix (Fin 2) (Fin 2) (α)) =
    (![C, 1; -invC * C + 1, -invC + D] := by
  simp [hstarU, U]
have hQ :
  star U * (![C, 1; 1, D] : Matrix (Fin 2) (Fin 2) (α)) * U =
    (![C, 0; 0, D - invC] := by
  have hstep :
    star U * (![C, 1; 1, D] : Matrix (Fin 2) (Fin 2) (α)) * U =
      (![C, 1; -invC * C + 1, -invC + D] : Matrix (Fin 2) (Fin 2) (α)) * U := by
    simpa [mul_assoc] using congrArg (fun X => X * U) hP
  dsimp [U] at hstep
  simp [hstep, C_mul_invC, invC_mul_C, sub_eq_add_neg, add_comm]
have hdiag :
  (Matrix.diagonal (fun i : Fin 2 => if i = 0 then C else D - invC)) =
    (![C, 0; 0, D - invC] : Matrix (Fin 2) (Fin 2) (α)) := by
  ext i j
  fin_cases i <=> fin_cases j <=> simp [Matrix.diagonal]
  simpa [hdiag] using hQ

theorem one_div_operatorConvexOn_Ioi :
  OperatorConvexOn (α := α) (Set.Ioi (0 : ℝ)) (fun x : ℝ => 1 / x) := by
  dsimp [OperatorConvexOn]
  intro A B t hA hB ht0 ht1 As Bs
  -- rewrite `1/x` as `x⁻¹`
  simp only [one_div]
  set C : α := (1 - t) • A + t • B
  have hC : IsSelfAdjoint C := by
    simpa [C] using (IsSelfAdjoint.all (1 - t)).smul hA |>.add ((IsSelfAdjoint.all t
  have specC : spectrum ℝ C ⊆ Set.Ioi (0 : ℝ) := by
    simpa [C] using
      spectrum_convexCombo_Ioi (A := A) (B := B) (t := t) hA hB ht0 ht1 As Bs

```

```

set invA :  $\square := \text{cfcR } (\text{fun } x : \mathbb{R} \mapsto x^{-1}) \text{ A}$ 
set invB :  $\square := \text{cfcR } (\text{fun } x : \mathbb{R} \mapsto x^{-1}) \text{ B}$ 
set invC :  $\square := \text{cfcR } (\text{fun } x : \mathbb{R} \mapsto x^{-1}) \text{ C}$ 
set D :  $\square := (1 - t) \cdot \text{invA} + t \cdot \text{invB}$ 
let M_A : Matrix (Fin 2) (Fin 2) ( $\square$ ) := !![A, 1; 1, invA]
let M_B : Matrix (Fin 2) (Fin 2) ( $\square$ ) := !![B, 1; 1, invB]
let M : Matrix (Fin 2) (Fin 2) ( $\square$ ) := (1 - t) • M_A + t • M_B
have posA : Matrix.PosSemidef M_A := by
  simp [M_A, invA] using posSemidef_block_one_inv (A := A) hA As
have posB : Matrix.PosSemidef M_B := by
  simp [M_B, invB] using posSemidef_block_one_inv (A := B) hB Bs
have posM : Matrix.PosSemidef M := by
  simp [M] using Matrix.PosSemidef.add
  (Matrix.PosSemidef.smul (x := M_A) (a := (1 - t)) posA (sub_nonneg.mp
    (Matrix.PosSemidef.smul (x := M_B) (a := t) posB ht0))
have hM : M = !![C, 1; 1, D] := by
  ext i j
  fin_cases i <=> fin_cases j
  · simp [M, M_A, M_B, C]
  · have h1 : (1 - t) • (1 :  $\square$ ) + t • (1 :  $\square$ ) = (1 :  $\square$ ) := by
      calc
        (1 - t) • (1 :  $\square$ ) + t • (1 :  $\square$ ) = ((1 - t) + t) • (1 :  $\square$ ) := by
          simp using (add_smul (1 - t) t (1 :  $\square$ )).symm
        _ = (1 :  $\square$ ) := by simp [sub_add_cancel]
      simp [M, M_A, M_B, h1]
  · have h1 : (1 - t) • (1 :  $\square$ ) + t • (1 :  $\square$ ) = (1 :  $\square$ ) := by
      calc
        (1 - t) • (1 :  $\square$ ) + t • (1 :  $\square$ ) = ((1 - t) + t) • (1 :  $\square$ ) := by
          simp using (add_smul (1 - t) t (1 :  $\square$ )).symm
        _ = (1 :  $\square$ ) := by simp [sub_add_cancel]
      simp [M, M_A, M_B, h1]
  · simp [M, M_A, M_B, D]
let U : Matrix (Fin 2) (Fin 2) ( $\square$ ) := !![(1 :  $\square$ ), -invC; 0, 1]
have hU : IsUnit U := by
  let V : Matrix (Fin 2) (Fin 2) ( $\square$ ) := !![(1 :  $\square$ ), invC; 0, 1]
  refine {<U, V, ?_, ?_>, rfl}
  · dsimp [U, V]
    simp [Matrix.one_fin_two]
  · dsimp [U, V]
    simp [Matrix.one_fin_two]
have hconj :
  star U * (!! [C, 1; 1, D] : Matrix (Fin 2) (Fin 2) ( $\square$ )) * U
  = Matrix.diagonal (fun i : Fin 2 => if i = 0 then C else D - invC)
have hInvC_sa : IsSelfAdjoint invC := by
  dsimp [invC, cfcR]
  exact cfc_predicate _ _

```

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWSETHREINZCORE.LEAN

```

have hcont_inv : ContinuousOn (fun x : ℝ ↦ x⁻¹) (spectrum ℝ C) :=
  fun x hx => (continuousAt_inv₀ (ne_of_gt (specC hx))).continuousWithinAt
have invC_mul_C : invC * C = (1 : ℝ) := by
  dsimp [invC, cfcR]
  have hmul :
    cfcR (fun x : ℝ ↦ x⁻¹) C * C =
      cfcR (fun x : ℝ ↦ x⁻¹ * x) C := by
    simp [cfc_id' (R := ℝ) (a := C) (ha := hC)] using
      (cfc_mul (fun x : ℝ ↦ x⁻¹) (fun x : ℝ ↦ x) C hcont_inv continuousOn_id).sy
  rw [hmul, ← cfc_const_one ℝ C]
  apply cfc_congr
  intro x hx
  have hxne : x ≠ 0 := ne_of_gt (specC hx)
  simp [hxne]
have C_mul_invC : C * invC = (1 : ℝ) := by
  dsimp [invC, cfcR]
  have hmul :
    C * cfcR (fun x : ℝ ↦ x⁻¹) C =
      cfcR (fun x : ℝ ↦ x * x⁻¹) C := by
    simp [cfc_id' (R := ℝ) (a := C) (ha := hC)] using
      (cfc_mul (fun x : ℝ ↦ x) (fun x : ℝ ↦ x⁻¹) C continuousOn_id hcont_inv).sy
  rw [hmul, ← cfc_const_one ℝ C]
  apply cfc_congr
  intro x hx
  have hxne : x ≠ 0 := ne_of_gt (specC hx)
  simp [hxne]
simp [U] using
  schur_conj_eq_diagonal (C := C) (D := D) (invC := invC)
  hInvC_sa invC_mul_C C_mul_invC
have posDiag :
  Matrix.PosSemidef (Matrix.diagonal (fun i : Fin 2 => if i = 0 then C else D -
have posConj : Matrix.PosSemidef (star U * M * U) := by
  simp [Matrix.star_eq_conjTranspose] using (posM.conjTranspose_mul_mul_same U)
have posConj' :
  Matrix.PosSemidef
    (star U * (!![[C, 1; 1, D] : Matrix (Fin 2) (Fin 2) ℝ]) * U) := by
  -- rewrite the middle block matrix as `M`
  rw [← hM]
  exact posConj
  -- rewrite the goal using the computed conjugation
  rw [← hconj]
  exact posConj'
have hinvc : invC ≤ D := by
have hDinvC : 0 ≤ D - invC := by
  simp using
    (Matrix.posSemidef_diagonal_iff (R := ℝ))

```

```

      (d := fun i : Fin 2 => if i = 0 then C else D - invC)).1 posDiag
    exact le_of_sub_nonneg hDinvC
  exact hinvC

omit [Nontrivial (□)] [NonnegSpectrumClass ℝ □] in
theorem one_div_add_t_operatorAntitoneOn_Ici : ∀ (t : ℝ), 0 < t →
  OperatorAntitoneOn (□ := □) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ 1 / (x + t)) :
  intro t ht
  dsimp [OperatorAntitoneOn]
  intro A B A_nonneg B_nonneg BA As Bs
  let f : ℝ → ℝ := fun x => x + t
  have hA_sa : IsSelfAdjoint A := IsSelfAdjoint.of_nonneg A_nonneg
  have hB_sa : IsSelfAdjoint B := IsSelfAdjoint.of_nonneg B_nonneg
  have hA_ne0 : ∀ x ∈ spectrum ℝ A, f x ≠ 0 := by
    intro x hx
    have hx0 : (0 : ℝ) ≤ x := by
      simpa [Set.Ici] using (As hx)
    exact ne_of_gt (add_pos_of_nonneg_of_pos hx0 ht)
  have hB_ne0 : ∀ x ∈ spectrum ℝ B, f x ≠ 0 := by
    intro x hx
    have hx0 : (0 : ℝ) ≤ x := by
      simpa [Set.Ici] using (Bs hx)
    exact ne_of_gt (add_pos_of_nonneg_of_pos hx0 ht)
  let uA : (□)x :=
    cfcUnits (R := ℝ) (A := □) (p := IsSelfAdjoint) f A hA_ne0 (ha := hA_sa)
  let uB : (□)x :=
    cfcUnits (R := ℝ) (A := □) (p := IsSelfAdjoint) f B hB_ne0 (ha := hB_sa)
  have huA_val : (uA : □) = A + algebraMap ℝ (□) t := by
    -- unfold `uA` to a `cfc` statement and use `cfc_add_const` + `cfc_id`
    simp [uA, cfcUnits, f]
    simpa [cfc_id' (R := ℝ) (a := A) (ha := hA_sa)] using
      (cfc_add_const (R := ℝ) (A := □) (p := IsSelfAdjoint) (r := t)
        (f := fun x : ℝ ↦ x) (a := A) (ha := hA_sa))
  have huB_val : (uB : □) = B + algebraMap ℝ (□) t := by
    -- unfold `uB` to a `cfc` statement and use `cfc_add_const` + `cfc_id`
    simp [uB, cfcUnits, f]
    simpa [cfc_id' (R := ℝ) (a := B) (ha := hB_sa)] using
      (cfc_add_const (R := ℝ) (A := □) (p := IsSelfAdjoint) (r := t)
        (f := fun x : ℝ ↦ x) (a := B) (ha := hB_sa))
  have huB_nonneg : 0 ≤ (uB : □) := by
    -- unfold `uB` and use pointwise nonnegativity on the spectrum
    simp only [uB, cfcUnits, f]
    refine cfc_nonneg (R := ℝ) (A := □) (p := IsSelfAdjoint) (a := B) ?_
    intro x hx
    have hx0 : (0 : ℝ) ≤ x := by
      simpa [Set.Ici] using (Bs hx)

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWE/RHEINZCORE.LEAN

```

    exact le_of_lt (add_pos_of_nonneg_of_pos hx0 ht)
  have hub_le_hua : (uB :  $\square$ )  $\leq$  (uA :  $\square$ ) := by
    simp [huA_val, huB_val, add_assoc, add_left_comm, add_comm] using add_le_add_r
  have hinv : ( $\uparrow$ uA $^{-1}$  :  $\square$ )  $\leq$  ( $\uparrow$ uB $^{-1}$  :  $\square$ ) := by
    simp using
      (CStarAlgebra.inv_le_inv (A :=  $\square$ ) (a := uB) (b := uA) huB_nonneg hub_le_hua)
  -- convert the inverse inequality back to the desired `cfcR` inequality
  simp [uA, uB, cfcUnits, cfcR, f, one_div] using hinv

```

-- Reduces to `one\_div\_operatorConvexOn\_Ioi` and is also elaboration-heavy.

```

theorem one_div_add_t_operatorConvexOn_Ici :  $\forall$  (t :  $\mathbb{R}$ ),  $0 < t \rightarrow$ 
  OperatorConvexOn ( $\square := \square$ ) (Set.Ici (0 :  $\mathbb{R}$ )) (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ ) := by
  /-
  It follows from one_div_operatorConvexOn_Ioi
  -/
  intro t ht
  dsimp [OperatorConvexOn]
  intro A B  $\theta$  hA hB h00 h01 As Bs
  -- rewrite `1 / (x + t)` as `(x + t) $^{-1}$ ` for `simp`/`cfc` lemmas
  simp only [one_div]
  -- Reduce to operator convexity of `x  $\mapsto$  x $^{-1}$ ` on `Ioi 0` by shifting by `t`.
  set C :  $\square := (1 - \theta) \cdot A + \theta \cdot B$ 
  have hC : IsSelfAdjoint C := by
    simp [C] using (IsSelfAdjoint.all (1 -  $\theta$ )).smul hA |>.add ((IsSelfAdjoint.all  $\theta$ 
  set shift :  $\mathbb{R} \mapsto \mathbb{R} := \text{fun } x \mapsto x + t$ 
  set T :  $\square := \text{algebraMap } \mathbb{R} (\square) t$ 
  have hT : IsSelfAdjoint T := by
    simp [T] using (IsSelfAdjoint.algebraMap (A :=  $\square$ ) (r := t)
      (hr := IsSelfAdjoint.all (t :  $\mathbb{R}$ )))
  have A_nonneg :  $0 \leq A :=$  by
    have h0 :  $0 \leq \text{cfcR } (\text{fun } x : \mathbb{R} \mapsto x) A :=$  by
      dsimp [cfcR]
      apply cfc_nonneg
      intro x hx
      simp [Set.Ici] using (As hx)
    simp [cfcR, cfc_id' (R :=  $\mathbb{R}$ ) (a := A) (ha := hA)] using h0
  have B_nonneg :  $0 \leq B :=$  by
    have h0 :  $0 \leq \text{cfcR } (\text{fun } x : \mathbb{R} \mapsto x) B :=$  by
      dsimp [cfcR]
      apply cfc_nonneg
      intro x hx
      simp [Set.Ici] using (Bs hx)
    simp [cfcR, cfc_id' (R :=  $\mathbb{R}$ ) (a := B) (ha := hB)] using h0
  have C_nonneg :  $0 \leq C :=$  by
    simp [C] using add_nonneg (smul_nonneg (sub_nonneg.mpr h01) A_nonneg) (smul_non

```

```

have hA_shift : cfcR shift A = A + T := by
  dsimp [cfcR, shift, T]
  simpa [cfc_id' (R := ℝ) (a := A) (ha := hA)] using
    (cfc_add_const (R := ℝ) (A := []) (p := IsSelfAdjoint) (r := t)
      (f := fun x : ℝ ↦ x) (a := A) (ha := hA))
have hB_shift : cfcR shift B = B + T := by
  dsimp [cfcR, shift, T]
  simpa [cfc_id' (R := ℝ) (a := B) (ha := hB)] using
    (cfc_add_const (R := ℝ) (A := []) (p := IsSelfAdjoint) (r := t)
      (f := fun x : ℝ ↦ x) (a := B) (ha := hB))
have hC_shift : cfcR shift C = C + T := by
  dsimp [cfcR, shift, T]
  simpa [cfc_id' (R := ℝ) (a := C) (ha := hC)] using
    (cfc_add_const (R := ℝ) (A := []) (p := IsSelfAdjoint) (r := t)
      (f := fun x : ℝ ↦ x) (a := C) (ha := hC))
set A1 : [] := A + T
set B1 : [] := B + T
set C1 : [] := (1 - θ) • A1 + θ • B1
have hA1_sa : IsSelfAdjoint A1 := by
  subst A1
  exact hA.add hT
have hB1_sa : IsSelfAdjoint B1 := by
  subst B1
  exact hB.add hT
have specA1 : spectrum ℝ A1 ⊆ Set.Ioi (0 : ℝ) := by
  intro x hx
  have hs : spectrum ℝ (cfc shift A) = shift '' spectrum ℝ A := by
    simpa [shift] using
      (cfc_map_spectrum (R := ℝ) (A := []) (p := IsSelfAdjoint) (a := A)
        (f := shift) (ha := hA))
  have hx' : x ∈ shift '' spectrum ℝ A := by
    have hx0 : x ∈ spectrum ℝ (cfc shift A) := by
      have hval : cfc shift A = A1 := by
        simpa [cfcR, shift, A1] using hA_shift
      simpa [hval] using hx
    simpa [hs] using hx0
  rcases hx' with ⟨y, hy, rfl⟩
  have hy0 : 0 ≤ y := by
    simpa [Set.Ici] using (As hy)
  simpa [Set.Ioi] using (add_pos_of_nonneg_of_pos hy0 ht)
have specB1 : spectrum ℝ B1 ⊆ Set.Ioi (0 : ℝ) := by
  intro x hx
  have hs : spectrum ℝ (cfc shift B) = shift '' spectrum ℝ B := by
    simpa [shift] using
      (cfc_map_spectrum (R := ℝ) (A := []) (p := IsSelfAdjoint) (a := B)
        (f := shift) (ha := hB))

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWENTROPY/LOWENTROPYCORE. LEAN

```

have hx' : x ∈ shift '' spectrum ℝ B := by
  have hx0 : x ∈ spectrum ℝ (cfc shift B) := by
    have hval : cfc shift B = B1 := by
      simp [cfcR, shift, B1] using hB_shift
    simp [hval] using hx
  simp [hs] using hx0
  rcases hx' with ⟨y, hy, rfl⟩
  have hy0 : 0 ≤ y := by
    simp [Set.Ici] using (Bs hy)
  simp [Set.Ioi] using (add_pos_of_nonneg_of_pos hy0 ht)
have hC1 : C1 = C + T := by
  subst A1 B1 C1
  simp [C, add_assoc, add_left_comm, add_comm, smul_add]
have hshift_ne0_A : ∀ x ∈ spectrum ℝ A, shift x ≠ 0 := by
  intro x hx
  have hx0 : 0 ≤ x := by
    simp [Set.Ici] using (As hx)
  exact ne_of_gt (by simp [shift] using (add_pos_of_nonneg_of_pos hx0 ht))
have hshift_ne0_B : ∀ x ∈ spectrum ℝ B, shift x ≠ 0 := by
  intro x hx
  have hx0 : 0 ≤ x := by
    simp [Set.Ici] using (Bs hx)
  exact ne_of_gt (by simp [shift] using (add_pos_of_nonneg_of_pos hx0 ht))
have hshift_ne0_C : ∀ x ∈ spectrum ℝ C, shift x ≠ 0 :=
  fun x hx ↦ ne_of_gt (by simp [shift] using (add_pos_of_nonneg_of_pos (spectrum_
have hA_inv : cfcR (fun x : ℝ ↦ (x + t)-1) A = Ring.inverse A1 := by
  have h' : cfc (fun x : ℝ ↦ (shift x)-1) A = Ring.inverse (cfc shift A) := by
    simp [shift] using (cfc_inv (R := ℝ) (A := []) (p := IsSelfAdjoint)
      (f := shift) (a := A) hshift_ne0_A (ha := hA))
  have hval : cfc shift A = A1 := by
    simp [cfcR, shift, A1] using hA_shift
  simp [cfcR, shift, hval] using h'
have hB_inv : cfcR (fun x : ℝ ↦ (x + t)-1) B = Ring.inverse B1 := by
  have h' : cfc (fun x : ℝ ↦ (shift x)-1) B = Ring.inverse (cfc shift B) := by
    simp [shift] using (cfc_inv (R := ℝ) (A := []) (p := IsSelfAdjoint)
      (f := shift) (a := B) hshift_ne0_B (ha := hB))
  have hval : cfc shift B = B1 := by
    simp [cfcR, shift, B1] using hB_shift
  simp [cfcR, shift, hval] using h'
have hC_inv : cfcR (fun x : ℝ ↦ (x + t)-1) C = Ring.inverse (C + T) := by
  have h' : cfc (fun x : ℝ ↦ (shift x)-1) C = Ring.inverse (cfc shift C) := by
    simp [shift] using (cfc_inv (R := ℝ) (A := []) (p := IsSelfAdjoint)
      (f := shift) (a := C) hshift_ne0_C (ha := hC))
  have hval : cfc shift C = C + T := by
    simp [cfcR, shift] using hC_shift
  simp [cfcR, shift, hval] using h'

```

```

have hA1_inv : cfcR (fun x :  $\mathbb{R} \rightarrow x^{-1}$ ) A1 = Ring.inverse A1 := by
  dsimp [cfcR]
  simpa [cfc_id' (R :=  $\mathbb{R}$ ) (a := A1) (ha := hA1_sa)] using
    (cfc_inv (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint) (f := fun x :  $\mathbb{R} \rightarrow x$ )
      (a := A1) (fun x hx  $\mapsto$  ne_of_gt (specA1 hx)) (ha := hA1_sa))
have hB1_inv : cfcR (fun x :  $\mathbb{R} \rightarrow x^{-1}$ ) B1 = Ring.inverse B1 := by
  dsimp [cfcR]
  simpa [cfc_id' (R :=  $\mathbb{R}$ ) (a := B1) (ha := hB1_sa)] using
    (cfc_inv (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint) (f := fun x :  $\mathbb{R} \rightarrow x$ )
      (a := B1) (fun x hx  $\mapsto$  ne_of_gt (specB1 hx)) (ha := hB1_sa))
have hA_eq : cfcR (fun x :  $\mathbb{R} \rightarrow (x + t)^{-1}$ ) A = cfcR (fun x :  $\mathbb{R} \rightarrow x^{-1}$ ) A1 :
  simp [hA_inv, hA1_inv]
have hB_eq : cfcR (fun x :  $\mathbb{R} \rightarrow (x + t)^{-1}$ ) B = cfcR (fun x :  $\mathbb{R} \rightarrow x^{-1}$ ) B1 :
  simp [hB_inv, hB1_inv]
have specC1 : spectrum  $\mathbb{R}$  C1  $\subseteq$  Set.Ioi (0 :  $\mathbb{R}$ ) := by
  intro x hx
  have hs : spectrum  $\mathbb{R}$  (cfc shift C) = shift '' spectrum  $\mathbb{R}$  C := by
    simpa [shift] using
      (cfc_map_spectrum (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint) (a := C)
        (f := shift) (ha := hC))
  have hx' : x  $\in$  shift '' spectrum  $\mathbb{R}$  C := by
    have hx0 : x  $\in$  spectrum  $\mathbb{R}$  (cfc shift C) := by
      have hval : cfc shift C = C + T := by
        simpa [cfcR, shift] using hC_shift
      have hval' : cfc shift C = C1 := by
        simpa [hC1] using hval
      simpa [hval'] using hx
    simpa [hs] using hx0
  rcases hx' with ⟨y, hy, rfl⟩
  have hy0 : 0  $\leq$  y := spectrum_nonneg_of_nonneg C_nonneg hy
  have : 0 < y + t := add_pos_of_nonneg_of_pos hy0 ht
  simpa [Set.Ioi] using this
have hC1_ne0 :  $\forall x \in \text{spectrum } \mathbb{R} \text{ C1}, (x : \mathbb{R}) \neq 0 := \text{fun } x \text{ hx} \mapsto \text{ne\_of\_gt } (s$ 
have hC1_sa : IsSelfAdjoint C1 := by
  simpa [hC1] using (hC.add hT)
have hC1_inv : cfcR (fun x :  $\mathbb{R} \rightarrow x^{-1}$ ) C1 = Ring.inverse C1 := by
  dsimp [cfcR]
  simpa [cfc_id' (R :=  $\mathbb{R}$ ) (a := C1) (ha := hC1_sa)] using
    (cfc_inv (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint) (f := fun x :  $\mathbb{R} \rightarrow x$ )
      (a := C1) hC1_ne0 (ha := hC1_sa))
have hC_eq : cfcR (fun x :  $\mathbb{R} \rightarrow (x + t)^{-1}$ ) C = cfcR (fun x :  $\mathbb{R} \rightarrow x^{-1}$ ) C1 :
  calc
    cfcR (fun x :  $\mathbb{R} \rightarrow (x + t)^{-1}$ ) C
      = Ring.inverse (C + T) := hC_inv
    _ = Ring.inverse C1 := by simp [hC1]
    _ = cfcR (fun x :  $\mathbb{R} \rightarrow x^{-1}$ ) C1 := by simpa using hC1_inv.symm

```


1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWENTROPY/REINZCORE.LEAN

```

have hconv :
  cfcR (fun x : ℝ ↦ x⁻¹) C1
    ≤ (1 - θ) • cfcR (fun x : ℝ ↦ x⁻¹) A1
      + θ • cfcR (fun x : ℝ ↦ x⁻¹) B1 := by
  simp [one_div] using
    (one_div_operatorConvexOn_Ioi (A := A1) (B := B1) (t := θ)
      hA1_sa hB1_sa hθ0 hθ1 specA1 specB1)
  -- conclude by rewriting everything to the shifted `1/x` convexity statement
  simp [C, hC_eq, hA_eq, hB_eq] using hconv

omit [Nontrivial (□)] [NonnegSpectrumClass ℝ □] in
theorem ratio_add_t_operatorMonotoneOn_Ici : ∀ (t : ℝ), 0 < t →
  OperatorMonotoneOn (□ := □) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ x / (x + t)) := by
  intro t ht
  dsimp [OperatorMonotoneOn]
  intro A B hA0 hB0 hBA hspA hspB
  let invfun : ℝ → ℝ := fun x : ℝ ↦ 1 / (x + t)
  have hmono_core :
    (1 : □) - t • cfcR invfun B ≤ (1 : □) - t • cfcR invfun A := by
    have h1 : t • cfcR invfun A ≤ t • cfcR invfun B :=
      smul_le_smul_of_nonneg_left
        ((one_div_add_t_operatorAntitoneOn_Ici t ht) hA0 hB0 hBA hspA hspB) (le_of_
          exact sub_le_sub_left h1 (1 : □))
  have hrepr (T : □) (hT0 : 0 ≤ T) (hspT : spectrum ℝ T ⊆ Set.Ici (0 : ℝ)) :
    cfcR (fun x : ℝ ↦ x / (x + t)) T = (1 : □) - t • cfcR invfun T := by
    have hT_sa : IsSelfAdjoint T := IsSelfAdjoint.of_nonneg hT0
    have hEqT :
      (spectrum ℝ T).EqOn (fun x : ℝ ↦ x / (x + t)) (fun x : ℝ ↦ 1 - t * invfun x)
    intro x hx
    have hx0 : (0 : ℝ) ≤ x := by
      simp [Set.Ici] using (hspT hx)
    simp [invfun]
    field_simp [ne_of_gt (add_pos_of_nonneg_of_pos hx0 ht)]
    ring
  have hT_ne0 : ∀ x ∈ spectrum ℝ T, x + t ≠ 0 := by
    intro x hx
    have hx0 : (0 : ℝ) ≤ x := by
      simp [Set.Ici] using (hspT hx)
    exact ne_of_gt (add_pos_of_nonneg_of_pos hx0 ht)
  have hT_cont : ContinuousOn invfun (spectrum ℝ T) := by
    simp [invfun, one_div] using (continuousOn_id.add continuousOn_const).inv0 hT
  dsimp [cfcR]
  have hcongr :
    cfcR (fun x : ℝ ↦ x / (x + t)) T =
      cfcR (fun x : ℝ ↦ 1 - t * invfun x) T :=
    cfc_congr hEqT

```

```

have hsub :
  cfcR (fun x : ℝ ↦ 1 - t * invfun x) T =
    cfcR (fun _ : ℝ ↦ (1 : ℝ)) T -
    cfcR (fun x : ℝ ↦ t * invfun x) T := by
  simp using
    (cfc_sub (R := ℝ) (A := []) (p := IsSelfAdjoint)
      (f := fun _ : ℝ ↦ (1 : ℝ)) (g := fun x : ℝ ↦ t * invfun x) (a :=
        (hf := continuousOn_const) (hg := continuousOn_const.mul hT_cont))
  have hone :
    cfcR (fun _ : ℝ ↦ (1 : ℝ)) T = (1 : []) := by
  simp using
    (cfc_const_one (R := ℝ) (A := []) (p := IsSelfAdjoint) (a := T) (ha
  have hmul :
    cfcR (fun x : ℝ ↦ t * invfun x) T =
      t • cfcR invfun T := by
  simp using
    (cfc_const_mul (R := ℝ) (A := []) (p := IsSelfAdjoint) (r := t) (f :
      (hf := hT_cont))
  calc
    cfcR (fun x : ℝ ↦ x / (x + t)) T =
      cfcR (fun x : ℝ ↦ 1 - t * invfun x) T := hcongr
  _ =
    cfcR (fun _ : ℝ ↦ (1 : ℝ)) T -
    cfcR (fun x : ℝ ↦ t * invfun x) T := hsub
  _ = (1 : []) - t • cfcR invfun T := by
  rw [hone, hmul]
  calc
    cfcR (fun x : ℝ ↦ x / (x + t)) B
      = (1 : []) - t • cfcR invfun B := by
      simp using hrepr B hB0 hspB
  _ ≤ (1 : []) - t • cfcR invfun A := hmono_core
  _ = cfcR (fun x : ℝ ↦ x / (x + t)) A := by
  simp using (hrepr A hA0 hspA).symm

theorem ratio_add_t_operatorConcaveOn_Ici : ∀ (t : ℝ), 0 < t →
  OperatorConcaveOn ([] := []) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ x / (x + t)) :=
  intro t ht
  dsimp [OperatorConcaveOn, OperatorConvexOn]
  intro A B u hA hB hu0 hu1 As Bs
  have hu0' : 0 ≤ (1 - u) := sub_nonneg.mpr hu1
  -- main input: operator convexity of `x ↦ 1 / (x + t)` on `Set.Ici 0`
  have hconv_inv :
    cfcR (fun x : ℝ ↦ 1 / (x + t)) ((1 - u) • A + u • B)
      ≤ (1 - u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
      + u • cfcR (fun x : ℝ ↦ 1 / (x + t)) B := by
  simp using (one_div_add_t_operatorConvexOn_Ici t ht) (A := A)

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWRHEINZCORE.LEAN

```

(B := B) (t := u) hA hB hu0 hu1 As Bs
-- rewrite `-(x/(x+t))` as `(-1) + t/(x+t)` under functional calculus
have hcalc (T :  $\square$ ) (hT : IsSelfAdjoint T) (Ts : spectrum  $\mathbb{R}$  T  $\subseteq$  Set.Ici (0 :  $\mathbb{R}$ ))
  cfcR (fun x :  $\mathbb{R} \mapsto -(x / (x + t))$ ) T
    = algebraMap  $\mathbb{R}$  ( $\square$ ) (-1 :  $\mathbb{R}$ )
      + t • cfcR (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ ) T := by
let invfun :  $\mathbb{R} \rightarrow \mathbb{R} := fun x \mapsto 1 / (x + t)$ 
have hne0 :  $\forall x \in \text{spectrum } \mathbb{R} T, x + t \neq 0 := by
  intro x hx
  have hx0 : (0 :  $\mathbb{R}$ )  $\leq$  x := by
    simpa [Set.Ici] using (Ts hx)
  exact ne_of_gt (add_pos_of_nonneg_of_pos hx0 ht)
have hcont : ContinuousOn invfun (spectrum  $\mathbb{R}$  T) := by
  simpa [invfun, one_div] using (continuousOn_id.add continuousOn_const).inv0
dsimp [cfcR]
have hcongr :
  cfcR (fun x :  $\mathbb{R} \mapsto -(x / (x + t))$ ) T
    = cfcR
      (fun x :  $\mathbb{R} \mapsto (-1 : \mathbb{R}) + t * \text{invfun } x$ ) T := by
apply cfc_congr
intro x hx
have hx0 : (0 :  $\mathbb{R}$ )  $\leq$  x := by
  simpa [Set.Ici] using (Ts hx)
have :
  - (x / (x + t)) = (-1 :  $\mathbb{R}$ ) + t * (1 / (x + t)) := by
    field_simp [ne_of_gt (add_pos_of_nonneg_of_pos hx0 ht)]
    ring_nf
  simpa [invfun] using this
calc
  cfcR (fun x :  $\mathbb{R} \mapsto -(x / (x + t))$ ) T
    = cfcR
      (fun x :  $\mathbb{R} \mapsto (-1 : \mathbb{R}) + t * \text{invfun } x$ ) T := hcongr
_ = algebraMap  $\mathbb{R}$  ( $\square$ ) (-1 :  $\mathbb{R}$ )
  + cfcR (fun x :  $\mathbb{R} \mapsto t * \text{invfun } x$ ) T := by
  simpa using
    (cfc_const_add (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint) (r := (-
1 :  $\mathbb{R}$ ))
      (f := fun x :  $\mathbb{R} \mapsto t * \text{invfun } x$ ) (a := T)
      (hf := continuousOn_const.mul hcont) (ha := hT))
_ = algebraMap  $\mathbb{R}$  ( $\square$ ) (-1 :  $\mathbb{R}$ )
  + t • cfcR invfun T := by
  simp [cfc_const_mul (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint) t invfun T
    (hf := hcont)]
_ = algebraMap  $\mathbb{R}$  ( $\square$ ) (-1 :  $\mathbb{R}$ )
  + t • cfcR (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ ) T := by
  simp [invfun]$ 
```

```

set AB :  $\square := (1 - u) \cdot A + u \cdot B$ 
have hAB : IsSelfAdjoint AB := by
  dsimp [AB]
  simpa using
    (IsSelfAdjoint.smul (by simp [IsSelfAdjoint]) hA).add
    (IsSelfAdjoint.smul (by simp [IsSelfAdjoint]) hB)
-- apply operator convexity of  $\lambda \mapsto \lambda/(x+\lambda)$ 
have hL :
  cfcR (fun x :  $\mathbb{R} \mapsto - (x / (x + t))$ ) AB
     $\leq (1 - u) \cdot \text{cfcR (fun x :  $\mathbb{R} \mapsto - (x / (x + t))$ ) A}$ 
      + u • cfcR (fun x :  $\mathbb{R} \mapsto - (x / (x + t))$ ) B := by
-- expand both sides using `hcalc`, then use `hconv_inv`
-- (filled in the next step)
set C :  $\square := \text{algebraMap } \mathbb{R} (\square) (-1 : \mathbb{R})$ 
have nonneg_of_spectrum_subset_Ici0 {T :  $\square$ } (hT : IsSelfAdjoint T)
  (Ts : spectrum  $\mathbb{R} T \subseteq \text{Set.Ici } (0 : \mathbb{R})$ ) :  $0 \leq T :=$  by
  have h' : algebraMap  $\mathbb{R} (\square) (0 : \mathbb{R}) \leq T :=$ 
    (algebraMap_le_iff_le_spectrum (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint)
      (r :=  $(0 : \mathbb{R})$ ) (a := T) (ha := hT)).2 (by
      intro x hx
      simpa [Set.Ici] using (Ts hx))
  simpa using h'
have hA0 :  $0 \leq A :=$ 
  nonneg_of_spectrum_subset_Ici0 (T := A) hA As
have hB0 :  $0 \leq B :=$ 
  nonneg_of_spectrum_subset_Ici0 (T := B) hB Bs
have hAB0 :  $0 \leq AB :=$  by
  dsimp [AB]
  exact add_nonneg (smul_nonneg hu0' hA0) (smul_nonneg hu0 hB0)
have ABs : spectrum  $\mathbb{R} AB \subseteq \text{Set.Ici } (0 : \mathbb{R}) :=$  by
  intro x hx
  have hx0 :  $(0 : \mathbb{R}) \leq x :=$ 
    spectrum_nonneg_of_nonneg ( $\square := \mathbb{R}$ ) (A :=  $\square$ ) (a := AB) hAB0 hx
  simpa [Set.Ici] using hx0
have hscale :
  t • cfcR (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ ) AB
     $\leq (t * (1 - u)) \cdot \text{cfcR (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ ) A}$ 
      + (t * u) • cfcR (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ ) B := by
  have hconv_inv_AB :
    cfcR (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ ) AB
       $\leq (1 - u) \cdot \text{cfcR (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ ) A}$ 
        + u • cfcR (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ ) B := by
    simpa [AB] using hconv_inv
  have hscale0 :
    t • cfcR (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ ) AB
       $\leq t \cdot$ 

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWERRHEINZCORE.LEAN

```

      ((1 - u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
      + u • cfcR (fun x : ℝ ↦ 1 / (x + t)) B) :=
    smul_le_smul_of_nonneg_left hconv_inv_AB (le_of_lt ht)
calc
  t • cfcR (fun x : ℝ ↦ 1 / (x + t)) AB
    ≤ t •
      ((1 - u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
      + u • cfcR (fun x : ℝ ↦ 1 / (x + t)) B) := hscale0
- =
  (t * (1 - u)) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
  + (t * u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) B := by
    simp [smul_add, smul_smul]
have hconst : (1 - u) • C + u • C = C := by
  simp [add_smul, sub_add_cancel] using (add_smul (1 - u) u C).symm
have hmain :
  C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) AB
    ≤ (1 - u) • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) A)
      + u • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) B) := by
have h' :
  C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) AB
    ≤ C
      + ((t * (1 - u)) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
      + (t * u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) B) := by
    exact add_le_add_right hscale C
have hR :
  C
    + ((t * (1 - u)) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
    + (t * u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) B)
    =
  (1 - u) • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) A)
    + u • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) B) := by
have hR' :
  (1 - u) • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) A)
    + u • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) B)
    =
  ((1 - u) • C + u • C)
    + ((t * (1 - u)) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
    + (t * u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) B) := by
calc
  (1 - u) • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) A)
    + u • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) B)
    =
  (1 - u) • C
    + (1 - u) • (t • cfcR (fun x : ℝ ↦ 1 / (x + t)) A)
    + (u • C + u • (t • cfcR (fun x : ℝ ↦ 1 / (x + t)) B)) := by
    simp [smul_add, add_assoc, add_left_comm, add_comm]

```

```

- =
  (1 - u) • C
  + (t * (1 - u)) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
  + (u • C + (t * u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) B)
  simp [smul_smul, mul_comm, add_assoc, add_left_comm, ad
- =
  ((1 - u) • C + u • C)
  + ((t * (1 - u)) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
  + (t * u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) B) := by
  abel
calc
C
+ ((t * (1 - u)) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
+ (t * u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) B)
=
((1 - u) • C + u • C)
+ ((t * (1 - u)) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
+ (t * u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) B) := by
simp [hconst, add_comm]
- =
(1 - u) • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) A)
+ u • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) B) := by
simp using hR'.symm
calc
C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) AB
≤
C
+ ((t * (1 - u)) • cfcR (fun x : ℝ ↦ 1 / (x + t)) A
+ (t * u) • cfcR (fun x : ℝ ↦ 1 / (x + t)) B) := h'
- =
(1 - u) • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) A)
+ u • (C + t • cfcR (fun x : ℝ ↦ 1 / (x + t)) B) := hR
dsimp [C] at hmain
rw [hcalc AB hAB ABs, hcalc A hA As, hcalc B hB Bs]
exact hmain
simp [AB] using hL

omit [Nontrivial (□)] in
theorem power_Icc_zero_one_operatorMonotoneOn_Ici : ∀ p ∈ Set.Icc (0 : ℝ) 1
  OperatorMonotoneOn (□ := □) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p) := by
  intro p hp
  dsimp [OperatorMonotoneOn]
  intro A B hA0 hB0 hBA hspA hspB
  have hA : cfcR (fun x : ℝ ↦ x ^ p) A = A ^ p := by
    simp [cfcR] using
      (CFC.rpow_eq_cfc_real (A := □) (a := A) (y := p) (ha := hA0)).symm

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWER/REHEINZCORE.LEAN

```

have hB : cfcR (fun x : ℝ ↦ x ^ p) B = B ^ p := by
  simpa [cfcR] using
    (CFC.rpow_eq_cfc_real (A := []) (a := B) (y := p) (ha := hB0)).symm
simpa [hA, hB] using (CFC.rpow_le_rpow (A := []) hp hBA)

```

```

omit [Nontrivial (□)] in
omit [StarOrderedRing □] in
private lemma cfc_n_rpowIntegrand_01_eq_smul_cfcR_ratio {q : NNReal} (hq : q ∈ Set.Ioo
  {t : ℝ} (htpos : 0 < t) (X : □) (hX0 : 0 ≤ X) :
  cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) X =
    (t ^ ((q : ℝ) - 1)) • cfcR (fun x : ℝ => x / (x + t)) X := by
have hq_real : ((q : ℝ) : ℝ) ∈ Set.Ioo (0 : ℝ) 1 := ((NNReal.coe_pos).2 hq.1, (NNR
let ratio : ℝ → ℝ := fun x => x / (x + t)
let r : ℝ := t ^ ((q : ℝ) - 1)
have hcont_ratio : ContinuousOn ratio (spectrum ℝ X) :=
  continuousOn_id.div (continuousOn_id.add continuousOn_const) (fun x hx ↦ ne_of_g
have hcfc_n : cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) X =
  cfcR (Real.rpowIntegrand_01 (q : ℝ) t) X := by
have hqs : quasispectrum ℝ X ⊆ Set.Ici (0 : ℝ) := by
  intro x hx
  have hx0 : (0 : ℝ) ≤ x := quasispectrum_nonneg_of_nonneg X hx0 x hx
  simpa [Set.Ici] using hx0
have hf : ContinuousOn (Real.rpowIntegrand_01 (q : ℝ) t) (quasispectrum ℝ X) :=
  (Real.continuousOn_rpowIntegrand_01_Ici hq_real htpos).mono hqs
simpa using
  (cfc_n_eq_cfc (R := ℝ) (A := []) (p := IsSelfAdjoint)
    (f := Real.rpowIntegrand_01 (q : ℝ) t) (a := X) (hf := hf) (hf0 := by simp))
have hEq :
  (spectrum ℝ X).EqOn (Real.rpowIntegrand_01 (q : ℝ) t) (fun x : ℝ ↦ r * ratio x)
  intro x hx
  simp [r, ratio, Real.rpowIntegrand_01_eq_pow_div hq_real (le_of_lt htpos) (spectr
    add_comm, mul_div_assoc]
have hcfc_congr :
  cfcR (Real.rpowIntegrand_01 (q : ℝ) t) X =
    cfcR (fun x : ℝ ↦ r * ratio x) X :=
  cfc_congr hEq
have hcfc_mul :
  cfcR (fun x : ℝ ↦ r * ratio x) X =
    r • cfcR ratio X := by
  simpa using
    (cfc_const_mul (R := ℝ) (A := []) (p := IsSelfAdjoint) (r := r) (f := ratio) (a
      (hf := hcont_ratio)))
have hmain :
  cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) X = r • cfcR ratio X := by
  calc
    cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) X =

```

```

      cfcR (Real.rpowIntegrand01 (q : ℝ) t) X := by
        simpa using hcfcn
    _ = cfcR (fun x : ℝ ↦ r * ratio x) X := hcfc_congr
    _ = r • cfcR ratio X := hcfc_mul
    _ = r • cfcR ratio X := by simp [cfcR]
  simpa [r, ratio] using hmain

private lemma cfcR_ratio_weighted_le {t : ℝ} (htpos : 0 < t) {A B : []} (hA0
  {a b : ℝ} (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) :
  a • cfcR (fun x : ℝ => x / (x + t)) A + b • cfcR (fun x : ℝ => x / (x +
    ≤ cfcR (fun x : ℝ => x / (x + t)) (a • A + b • B) := by
  have ha1 : a = 1 - b := by linarith [hab]
  have hb1 : b ≤ 1 := by linarith [ha, hab]
  have hspec (X : []) (hX0 : 0 ≤ X) : spectrum ℝ X ⊆ Set.Ici (0 : ℝ) := by
    intro x hx
    have hx0 : (0 : ℝ) ≤ x := spectrum_nonneg_of_nonneg hX0 hx
    simpa [Set.Ici] using hx0
  have h0p := ratio_add_t_operatorConcave0n_Ici ([] := []) t htpos
  have hneg :=
    (by
      dsimp [OperatorConcave0n, OperatorConvex0n] at h0p
      have := h0p (A := A) (B := B) (t := b) (IsSelfAdjoint.of_nonneg hA0)
      hb hb1 (hspec A hA0) (hspec B hB0)
      exact this)
  have hneg' :
    cfcR (fun x : ℝ ↦ - (x / (x + t))) ((1 - b) • A + b • B)
      ≤ (1 - b) • cfcR (fun x : ℝ ↦ - (x / (x + t))) A
      + b • cfcR (fun x : ℝ ↦ - (x / (x + t))) B := hneg
  have hneg'' :
    -cfcR (fun x : ℝ => x / (x + t)) ((1 - b) • A + b • B)
      ≤ -((1 - b) • cfcR (fun x : ℝ => x / (x + t)) A + b • cfcR (fun x :
  have h' :
    -cfcR (fun x : ℝ => x / (x + t)) ((1 - b) • A + b • B)
      ≤ -(b • cfcR (fun x : ℝ => x / (x + t)) B + (1 - b) • cfcR (fun x :
    simpa [cfcR, cfc_neg, smul_neg, neg_add] using hneg'
    simpa [add_comm, add_left_comm, add_assoc] using h'
    simpa [ha1, add_comm, add_left_comm, add_assoc] using neg_le_neg_iff.mp h

private lemma concave0n_cfcn_rpowIntegrand01 {q : NNReal} (hq : q ∈ Set.Ioo
  {t : ℝ} (htpos : 0 < t) :
  Concave0n ℝ (Set.Ici (0 : [])) (fun A : [] => cfcn (Real.rpowIntegrand01
  have hq_real : ((q : ℝ) : ℝ) ∈ Set.Ioo (0 : ℝ) 1 := by
    refine {?, ?}
    · exact (NNReal.coe_pos).2 hq.1
    · exact (NNReal.coe_lt_coe).2 hq.2
  refine {convex_Ici ([] := ℝ) (0 : []), ?}

```


1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWENTROPY/LOWENTROPYCORE.LEAN

```

intro A hA B hB a b ha hb hab
have hA0 : 0 ≤ A := by simpa [Set.Ici] using hA
have hB0 : 0 ≤ B := by simpa [Set.Ici] using hB
have hAB0 : 0 ≤ a • A + b • B := add_nonneg (smul_nonneg ha hA0) (smul_nonneg hb hB0)
let ratio : ℝ → ℝ := fun x => x / (x + 1)
let r : ℝ := t ^ ((q : ℝ) - 1)
have hr_nonneg : 0 ≤ r := Real.rpow_nonneg (le_of_lt htpos) _
have hrepr (X : ℝ) (hX0 : 0 ≤ X) :
  cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) X = r • cfcR ratio X := by
  simpa [r, ratio] using cfc_n_rpowIntegrand_01_eq_smul_cfcR_ratio (q := q) hq htpos
have hratio :
  a • cfcR ratio A + b • cfcR ratio B ≤ cfcR ratio (a • A + b • B) := by
  -- use the separate lemma to keep heartbeats per-declaration small
  -- (`ratio` is a local abbreviation here)
  simpa [ratio] using cfcR_ratio_weighted_le (t := t) htpos (A := A) (B := B) hA0 hB0
have hscaled : r • (a • cfcR ratio A + b • cfcR ratio B) ≤ r • cfcR ratio (a • A + b • B) :=
  smul_le_smul_of_nonneg_left hratio hr_nonneg
have hL :
  a • cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) A + b • cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) B
  = r • (a • cfcR ratio A + b • cfcR ratio B) := by
  simp [hrepr A hA0, hrepr B hB0, smul_add, smul_smul, mul_comm]
have hR :
  cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) (a • A + b • B) = r • cfcR ratio (a • A + b • B) := by
  simp [hrepr (a • A + b • B) hAB0]
calc
a • cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) A + b • cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) B
= r • (a • cfcR ratio A + b • cfcR ratio B) := hL
_ ≤ r • cfcR ratio (a • A + b • B) := hscaled
_ = cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) (a • A + b • B) := hR.symm

private lemma concave0n_nnrpow_Ioo {q : NNReal} (hq : q ∈ Set.Ioo (0 : NNReal) 1) :
  Concave0n ℝ (Set.Ici (0 : ℝ)) (fun A : ℝ => A ^ q) := by
  -- integral representation for `a ↦ a ^ q`
  obtain (μ, hμ) :=
    CFC.exists_measure_nnrpow_eq_integral_cfc_n_rpowIntegrand_01 (A := ℝ) hq
  let ν : MeasureTheory.Measure ℝ := μ.restrict (Set.Ioi (0 : ℝ))
  let F : ℝ → ℝ → ℝ := fun t A => cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) A
  have hF_int : ∀ A ∈ Set.Ici (0 : ℝ), MeasureTheory.Integrable (fun t => F t A) ν :=
    intro A hA
    simpa [F, ν, MeasureTheory.Integrable0n] using (hμ A hA).1
  have hF_conc :
    ∀ t ∂ν, Concave0n ℝ (Set.Ici (0 : ℝ)) (fun A : ℝ => F t A) := by
    filter_upwards [MeasureTheory.ae_restrict_mem measurableSet_Ioi] with t ht
    simpa [F] using (concave0n_cfc_n_rpowIntegrand_01 (q := q) hq ht)
  have hconc_int :
    Concave0n ℝ (Set.Ici (0 : ℝ)) (fun A : ℝ => ∫ t, F t A ∂ν) :=

```

```

    MeasureTheory.integral_concave0n_of_integrand_ae
      (μ := ν) (s := Set.Ici (0 : ℝ)) (f := fun t A => F t A)
      (convex_Ici (· := ℝ) (0 : ℝ)) hF_conc hF_int
-- identify the integral with `A ^ q` on `Ici 0`
refine hconc_int.congr ?_
intro A hA
-- `A ^ q` is the set integral of the integrand on `Ioi 0`
have hEq : A ^ q = ∫ t, F t A ∂ν := by
  simpa [F, ν] using (hμ A hA).2
simp [hEq]

private lemma concave0n_rpow_Ioo {p : ℝ} (hp : p ∈ Set.Ioo (0 : ℝ) 1) :
  Concave0n ℝ (Set.Ici (0 : ℝ)) (fun A : ℝ → A ^ p) := by
-- reduce to the `ℝ≥0` exponent case
let q : NNReal := ⟨p, le_of_lt hp.1⟩
have hq0 : (0 : NNReal) < q := by
  have : (0 : ℝ) < (q : ℝ) := by
    simpa [q] using hp.1
  exact (NNReal.coe_pos).1 this
have hq1 : q < (1 : NNReal) := by
  have : (q : ℝ) < (1 : ℝ) := by
    simpa [q] using hp.2
  exact (NNReal.coe_lt_coe).1 (by simpa using this)
have hq : q ∈ Set.Ioo (0 : NNReal) 1 := ⟨hq0, hq1⟩
-- main lemma: concavity for `a ↦ a ^ q`
have hconc : Concave0n ℝ (Set.Ici (0 : ℝ)) (fun A : ℝ → A ^ q) :=
  concave0n_nnrpow_Ioo hq
-- transport concavity from `A ^ q` to `A ^ p`
refine hconc.congr ?_
intro A hA
-- `A ^ q = A ^ (q : ℝ)`, and `(q : ℝ) = p`
simpa [q] using (CFC.nnrpow_eq_rpow (A := ℝ) (a := A) (x := q) hq0)

theorem power_Icc_zero_one_operatorConcave0n_Ici : ∀ p ∈ Set.Icc (0 : ℝ) 1,
  OperatorConcave0n (· := ℝ) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p) := by
  intro p hp
  by_cases hp0 : p = 0
  · subst hp0
    dsimp [OperatorConcave0n, OperatorConvex0n]
    intro A B u hA hB hu0 hu1 As Bs
    have hC : IsSelfAdjoint ((1 - u) • A + u • B) := by
      simpa using (IsSelfAdjoint.all (1 - u)).smul hA |>.add ((IsSelfAdjoint
    have hfun : (fun x : ℝ ↦ - (x ^ (0 : ℝ))) = (fun _ : ℝ ↦ (-
1 : ℝ)) := by
  funext x
  simp

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWERRHEINZCORE.LEAN

```

have hconst (T :  $\square$ ) (hT : IsSelfAdjoint T) :
  cfcR (fun _ :  $\mathbb{R} \mapsto (-1 : \mathbb{R})$ ) T = (-1 :  $\square$ ) := by
  simpa [cfcR] using
    (cfc_const (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint) (-
1 :  $\mathbb{R}$ ) T hT)
  rw [hfun]
  rw [hconst _ hC, hconst _ hA, hconst _ hB]
  have hR : (1 - u) • (-1 :  $\square$ ) + u • (-1 :  $\square$ ) = (-1 :  $\square$ ) := by
    calc
      (1 - u) • (-1 :  $\square$ ) + u • (-1 :  $\square$ ) = ((1 - u) + u) • (-
1 :  $\square$ ) := by
        simpa [add_smul] using (add_smul (1 - u) u (-1 :  $\square$ )).symm
        _ = (1 :  $\mathbb{R}$ ) • (-1 :  $\square$ ) := by simp
        _ = (-1 :  $\square$ ) := by simp
      simp
  by_cases hp1 : p = 1
  · subst hp1
    dsimp [OperatorConcaveOn, OperatorConvexOn]
    intro A B u hA hB hu0 hu1 As Bs
    have hC : IsSelfAdjoint ((1 - u) • A + u • B) := by
      simpa using (IsSelfAdjoint.all (1 - u)).smul hA |>.add ((IsSelfAdjoint.all u).
have hfun : (fun x :  $\mathbb{R} \mapsto - (x ^ (1 : \mathbb{R}))$ ) = (fun x :  $\mathbb{R} \mapsto -
x) := by
  funext x
  simp
  have hneg (T :  $\square$ ) (hT : IsSelfAdjoint T) :
    cfcR (fun x :  $\mathbb{R} \mapsto -x$ ) T = -T := by
      simpa [cfcR] using (cfc_neg_id (R :=  $\mathbb{R}$ ) (p := IsSelfAdjoint) (a := T) hT)
  rw [hfun]
  rw [hneg _ hC, hneg _ hA, hneg _ hB]
  -- both sides are `-(1-u)•A + u•B`
  simp [add_comm, sub_eq_add_neg]
  have hp01 : p ∈ Set.Ioo (0 :  $\mathbb{R}$ ) 1 := by
    refine {?, ?}
    · have : 0 ≤ p := hp.1
      exact lt_of_le_of_ne this (Ne.symm hp0)
    · have : p ≤ 1 := hp.2
      exact lt_of_le_of_ne this hp1
  dsimp [OperatorConcaveOn, OperatorConvexOn]
  intro A B u hA hB hu0 hu1 As Bs
  have hA0 : 0 ≤ A := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R :=  $\mathbb{R}$ ) A (ha := hA)).2 ?_
    intro x hx
    have : x ∈ Set.Ici (0 :  $\mathbb{R}$ ) := As hx
    simpa [Set.Ici] using this
  have hB0 : 0 ≤ B := by$ 
```

```

    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B (ha := hB)
    intro x hx
    have : x ∈ Set.Ici (0 : ℝ) := Bs hx
    simp [Set.Ici] using this
    have hu0' : 0 ≤ (1 - u) := sub_nonneg.mpr hu1
    have hC0 : 0 ≤ (1 - u) • A + u • B :=
      add_nonneg (smul_nonneg hu0' hA0) (smul_nonneg hu0 hB0)
    set C : [] := (1 - u) • A + u • B
    have hC_mem : C ∈ Set.Ici (0 : []) := by
      simp [C, Set.Ici] using hC0
    have hA_mem : A ∈ Set.Ici (0 : []) := by simp [Set.Ici] using hA0
    have hB_mem : B ∈ Set.Ici (0 : []) := by simp [Set.Ici] using hB0
    have hconcC : (1 - u) • (A ^ p) + u • (B ^ p) ≤ C ^ p := by
      have hab : (1 - u) + u = (1 : ℝ) := by ring
      simp [C] using (concaveOn_rpow_Ioo hp01).2 hA_mem hB_mem hu0' hu0 hab
    have hcalc (T : []) (hT0 : 0 ≤ T) :
      cfcR (fun x : ℝ ↦ x ^ p) T = T ^ p := by
      simp [cfcR] using
        (CFC.rpow_eq_cfc_real (A := []) (a := T) (y := p) (ha := hT0)).symm
    have hconcC' :
      (1 - u) • cfcR (fun x : ℝ ↦ x ^ p) A + u • cfcR (fun x : ℝ ↦ x ^ p) B
      ≤ cfcR (fun x : ℝ ↦ x ^ p) C := by
      simp [hcalc A hA0, hcalc B hB0, hcalc C hC0, C] using hconcC
    -- convert concavity into convexity of `x ↦ -x^p`
    simp [cfcR, cfc_neg, smul_neg, neg_add, add_assoc, add_left_comm, add_comm]

private lemma sq_mul_div_add (x t : ℝ) (hxt : x + t ≠ 0) :
  (x * x) / (x + t) = x - t + (t * t) / (x + t) := by
  field_simp [hxt]
  ring

private lemma convexOn_cfcR_one_div_add_t (t : ℝ) (htpos : 0 < t) :
  ConvexOn ℝ (Set.Ici (0 : [])) (fun X : [] ↦ cfcR (fun x : ℝ ↦ 1 / (x + t))
  have hs : Convex ℝ (Set.Ici (0 : [])) := convex_Ici (0 := ℝ) (0 : [])
  refine {hs, ?_}
  intro A hA B hB a b ha hb hab
  have ha1 : a = 1 - b := by linarith [hab]
  have hb1 : b ≤ 1 := by linarith [ha, hab]
  have hA0 : 0 ≤ A := by simp [Set.Ici] using hA
  have hB0 : 0 ≤ B := by simp [Set.Ici] using hB
  have hspec (X : []) (hX0 : 0 ≤ X) : spectrum ℝ X ⊆ Set.Ici (0 : ℝ) := by
    intro x hx
    have hx0 : (0 : ℝ) ≤ x := spectrum_nonneg_of_nonneg hX0 hx
    simp [Set.Ici] using hx0
  have hOp := one_div_add_t_operatorConvexOn_Ici (0 := []) t htpos
  dsimp [OperatorConvexOn] at hOp

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWER_RHEINZCORE.LEAN

```

simpa [one_div, ha1] using
  h0p (A := A) (B := B) (t := b) (IsSelfAdjoint.of_nonneg hA0) (IsSelfAdjoint.of_nonneg hB0)

omit [Nontrivial (□)] in
private lemma G_eq0n_rpowIntegrand01_mul {q : NNReal} (hq_real : (q : ℝ) ∈ Set.Ioo (0 : ℝ) (1 : ℝ)) :
  (t : ℝ) (htpos : 0 < t) :
  (Set.Ici (0 : ℝ)).Eq0n
    (fun X : ℝ → cfc_n (fun x : ℝ → x * Real.rpowIntegrand01 (q : ℝ) t x) X)
    (fun X : ℝ → (t ^ ((q : ℝ) - 1)) *
      (X - algebraMap ℝ (□) t + (t ^ (2 : ℕ)) * cfcR (fun x : ℝ → 1 / (x + t)) X))
intro X hX
have hX0 : 0 ≤ X := by simpa [Set.Ici] using hX
have hX_sa : IsSelfAdjoint X := IsSelfAdjoint.of_nonneg hX0
have hqs : quasispectrum ℝ X ⊆ Set.Ici (0 : ℝ) := by
  intro x hx
  have hx0 : (0 : ℝ) ≤ x := quasispectrum_nonneg_of_nonneg X hX0 x hx
  simpa [Set.Ici] using hx0
have hf_int : ContinuousOn (Real.rpowIntegrand01 (q : ℝ) t) (quasispectrum ℝ X) :=
  (Real.continuousOn_rpowIntegrand01_Ici hq_real htpos).mono hqs
have hf :
  ContinuousOn (fun x : ℝ → x * Real.rpowIntegrand01 (q : ℝ) t x) (quasispectrum ℝ X) :=
  continuousOn_id.mul hf_int
have hcfc_n :
  cfc_n (fun x : ℝ → x * Real.rpowIntegrand01 (q : ℝ) t x) X =
  cfcR
  (fun x : ℝ → x * Real.rpowIntegrand01 (q : ℝ) t x) X := by
  simpa using
    (cfc_n_eq_cfc (R := ℝ) (A := □) (p := IsSelfAdjoint)
      (f := fun x : ℝ → x * Real.rpowIntegrand01 (q : ℝ) t x) (a := X)
      (hf := hf) (hf0 := by simp))
have hEq :
  (spectrum ℝ X).Eq0n (fun x : ℝ → x * Real.rpowIntegrand01 (q : ℝ) t x)
    (fun x : ℝ → (t ^ ((q : ℝ) - 1)) * (x - t + (t ^ (2 : ℕ)) / (x + t))) := by
  intro x hx
  have hx0 : (0 : ℝ) ≤ x := spectrum_nonneg_of_nonneg hX0 hx
  have ht0 : (0 : ℝ) ≤ t := le_of_lt htpos
  have hxt : x + t ≠ 0 := ne_of_gt (add_pos_of_nonneg_of_pos hx0 htpos)
  have hdiv : (x * x) / (x + t) = x - t + (t * t) / (x + t) := sq_mul_div_add x t
  have hrepr0 :
    x * Real.rpowIntegrand01 (q : ℝ) t x = (t ^ ((q : ℝ) - 1)) * ((x * x) / (x + t))
  rw [Real.rpowIntegrand01_eq_pow_div hq_real ht0 hx0]
  rw [mul_div_assoc]
  have hnum : x * (t ^ ((q : ℝ) - 1) * x) = t ^ ((q : ℝ) - 1) * (x * x) := by ring
  rw [hnum, mul_div_assoc]
  simp [add_comm]
  simp [hrepr0, hdiv, pow_two, mul_comm]

```

```

have hcfc_congr :
  cfcR
    (fun x : ℝ → x * Real.rpowIntegrand_01 (q : ℝ) t x) X
  =
  cfcR
    (fun x : ℝ → (t ^ ((q : ℝ) - 1)) * (x - t + (t ^ (2 : ℕ)) / (x + t))) X
  cfc_congr hEq
have hne : ∀ x ∈ spectrum ℝ X, x + t ≠ 0 := by
  intro x hx
  have hx0 : (0 : ℝ) ≤ x := spectrum_nonneg_of_nonneg hx0 hx
  exact ne_of_gt (add_pos_of_nonneg_of_pos hx0 htpos)
have hcont_one_div : ContinuousOn (fun x : ℝ → 1 / (x + t)) (spectrum ℝ X) :=
  have hden : ContinuousOn (fun x : ℝ → x + t) (spectrum ℝ X) :=
    continuousOn_id.add continuousOn_const
  exact continuousOn_const.div hden hne
have hcont_inner :
  ContinuousOn (fun x : ℝ → x - t + (t ^ (2 : ℕ)) / (x + t)) (spectrum ℝ X) :=
  have hden : ContinuousOn (fun x : ℝ → x + t) (spectrum ℝ X) :=
    continuousOn_id.add continuousOn_const
  have hdiv : ContinuousOn (fun x : ℝ → (t ^ (2 : ℕ)) / (x + t)) (spectrum ℝ X) :=
    continuousOn_const.div hden hne
  have hsub : ContinuousOn (fun x : ℝ → x - t) (spectrum ℝ X) :=
    continuousOn_id.sub continuousOn_const
  exact hsub.add hdiv
have hcfc_scale :
  cfcR
    (fun x : ℝ → (t ^ ((q : ℝ) - 1)) * (x - t + (t ^ (2 : ℕ)) / (x + t))) X
  =
  (t ^ ((q : ℝ) - 1)) • cfcR
    (fun x : ℝ → x - t + (t ^ (2 : ℕ)) / (x + t)) X := by
  simpa using
    (cfc_const_mul (R := ℝ) (A := []) (p := IsSelfAdjoint) (r := (t ^ ((q : ℝ) - 1)) • cfcR (fun x : ℝ → x - t + (t ^ (2 : ℕ)) / (x + t)) X) (a := X) (hf := hcont_inner))
have hcfc_inner :
  cfcR
    (fun x : ℝ → x - t + (t ^ (2 : ℕ)) / (x + t)) X
  =
  X - algebraMap ℝ ([]) t + (t ^ (2 : ℕ)) • cfcR (fun x : ℝ → 1 / (x + t)) X
have hconst :
  cfcR (fun _ : ℝ → t) X =
  algebraMap ℝ ([]) t := by
  simpa using (cfc_const (R := ℝ) (A := []) (p := IsSelfAdjoint) (r := t) (a := X) (hf := hconst))
have hid :
  cfcR (fun x : ℝ → x) X = X := by
  simpa using (cfc_id' (R := ℝ) (A := []) (p := IsSelfAdjoint) (a := X) (hf := hid))

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWER RHEINZCORE. LEAN

```

have hpow :
  cfcR
    (fun x : ℝ → (t ^ (2 : ℕ)) / (x + t)) X
  =
    (t ^ (2 : ℕ)) • cfcR (fun x : ℝ → 1 / (x + t)) X := by
have :
  cfcR
    (fun x : ℝ → (t ^ (2 : ℕ)) / (x + t)) X
  =
    cfcR
      (fun x : ℝ → (t ^ (2 : ℕ)) * (1 / (x + t))) X := by
  refine cfc_congr ?_
  intro x hx
  simp [div_eq_mul_inv, mul_comm]
  rw [this]
  simp [cfcR] using
    (cfc_const_mul (R := ℝ) (A := []) (p := IsSelfAdjoint)
      (r := (t ^ (2 : ℕ))) (f := fun x : ℝ → 1 / (x + t)) (a := X) (hf := hcont_
calc
  cfcR
    (fun x : ℝ → x - t + (t ^ (2 : ℕ)) / (x + t)) X
  =
    cfcR (fun x : ℝ → x - t) X
    + cfcR (fun x : ℝ → (t ^ (2 : ℕ)) / (x + t)) X := by
  simp using
    (cfc_add (R := ℝ) (A := []) (p := IsSelfAdjoint)
      (f := fun x : ℝ → x - t) (g := fun x : ℝ → (t ^ (2 : ℕ)) / (x + t))
  =
  -
    (cfcR (fun x : ℝ → x) X
      - cfcR (fun _ : ℝ → t) X)
    + (t ^ (2 : ℕ)) • cfcR (fun x : ℝ → 1 / (x + t)) X := by
  simp [hpow] using
    (congrArg (fun z => z + cfcR
      (fun x : ℝ → (t ^ (2 : ℕ)) / (x + t)) X)
      (cfc_sub (R := ℝ) (A := []) (p := IsSelfAdjoint)
        (f := fun x : ℝ → x) (g := fun _ : ℝ → t) (a := X)))
  _ = X - algebraMap ℝ ([]) t + (t ^ (2 : ℕ)) • cfcR (fun x : ℝ → 1 / (x + t)) X
  simp [hid, hconst, sub_eq_add_neg, add_comm]
-- finish
calc
  cfc_n (fun x : ℝ → x * Real.rpowIntegrand_01 (q : ℝ) t x) X
  =
    cfcR
      (fun x : ℝ → x * Real.rpowIntegrand_01 (q : ℝ) t x) X := hcfc_n
  =
  -
    cfcR

```

```

      (fun x : ℝ → (t ^ ((q : ℝ) - 1)) * (x - t + (t ^ (2 : ℕ)) / (x + t))
    - =
      (t ^ ((q : ℝ) - 1)) • cfcR
      (fun x : ℝ → x - t + (t ^ (2 : ℕ)) / (x + t)) X := hcfc_scale
    - = (t ^ ((q : ℝ) - 1)) •
      (X - algebraMap ℝ ([]) t + (t ^ (2 : ℕ)) • cfcR (fun x : ℝ → 1 / (x
      simp [hcfc_inner, smul_add, smul_smul, mul_comm]

private lemma convex0n_G_rpowIntegrand01_mul {q : NNReal} (hq_real : (q : ℝ)
  (t : ℝ) (htpos : 0 < t) :
  Convex0n ℝ (Set.Ici (0 : []))
    (fun X : [] → cfc_n (fun x : ℝ → x * Real.rpowIntegrand01 (q : ℝ) t x))
  -- use `EqOn` to replace the integrand by a structured convex expression
  let r : ℝ := t ^ ((q : ℝ) - 1)
  have hr_nonneg : 0 ≤ r :=
    Real.rpow_nonneg (le_of_lt htpos) _
  have hs : Convex ℝ (Set.Ici (0 : [])) := convex_Ici ([] := ℝ) (0 : [])
  have h_aff : Convex0n ℝ (Set.Ici (0 : [])) (fun X : [] → X - algebraMap ℝ (
    have hid : Convex0n ℝ (Set.Ici (0 : [])) (fun X : [] → X) := by
      simpa using (convex0n_id ([] := ℝ) (s := Set.Ici (0 : [])) hs)
    have hconst : Convex0n ℝ (Set.Ici (0 : [])) (fun _ : [] → -
algebraMap ℝ ([]) t) :=
  convex0n_const (-algebraMap ℝ ([]) t) hs
  simpa [sub_eq_add_neg] using hid.add hconst
  have h_one_div : Convex0n ℝ (Set.Ici (0 : [])) (fun X : [] → cfcR (fun x :
    convex0n_cfcR_one_div_add_t t htpos
  have h_inner : Convex0n ℝ (Set.Ici (0 : []))
    (fun X : [] → X - algebraMap ℝ ([]) t + (t ^ (2 : ℕ)) • cfcR (fun x : ℝ →
  have hterm :
    Convex0n ℝ (Set.Ici (0 : []))
      (fun X : [] → (t ^ (2 : ℕ)) • cfcR (fun x : ℝ → 1 / (x + t)) X) :=
    (h_one_div.smul (sq_nonneg t))
  exact h_aff.add hterm
  have h_rhs :
    Convex0n ℝ (Set.Ici (0 : []))
      (fun X : [] → r •
        (X - algebraMap ℝ ([]) t + (t ^ (2 : ℕ)) • cfcR (fun x : ℝ → 1 / (
    h_inner.smul hr_nonneg
  -- transfer convexity back to the `cfc_n` expression
  refine h_rhs.congr ?_
  intro X hX
  have : (fun X : [] → cfc_n (fun x : ℝ → x * Real.rpowIntegrand01 (q : ℝ) t
    =
    (fun X : [] → r •
      (X - algebraMap ℝ ([]) t + (t ^ (2 : ℕ)) • cfcR (fun x : ℝ → 1 / (x
    simpa [r] using G_eq0n_rpowIntegrand01_mul hq_real t htpos hX

```


1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWERREINZCORE.LEAN

```

simpa using this.symm

omit [Nontrivial (□)] in
private lemma ae_cfc_n_mul_id_rpowIntegrand_01_restrict_Ioi {q : NNReal} (hq_real : (q : ℝ) > 0)
  (μ : MeasureTheory.Measure ℝ) (A : □) (hA0 : 0 ≤ A) :
  ∀m t ∂(μ.restrict (Set.Ioi (0 : ℝ))),
    cfc_n (fun x : ℝ → x * Real.rpowIntegrand_01 (q : ℝ) t x) A =
      A * cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) A := by
  filter_upwards [MeasureTheory.ae_restrict_mem measurableSet_Ioi] with t ht
  have hqs : quasispectrum ℝ A ⊆ Set.Ici (0 : ℝ) := by
    intro x hx
    have hx0 : (0 : ℝ) ≤ x := quasispectrum_nonneg_of_nonneg A hA0 x hx
    simpa [Set.Ici] using hx0
  have hg : ContinuousOn (Real.rpowIntegrand_01 (q : ℝ) t) (quasispectrum ℝ A) :=
    (Real.continuousOn_rpowIntegrand_01_Ici hq_real ht).mono hqs
  have hG_mul :
    cfc_n (fun x : ℝ → x * Real.rpowIntegrand_01 (q : ℝ) t x) A
    =
    cfc_n (fun x : ℝ → x) A * cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) A := by
  simpa using
    (cfc_n_mul (R := ℝ) (A := □) (p := IsSelfAdjoint)
      (f := fun x : ℝ → x) (g := Real.rpowIntegrand_01 (q : ℝ) t) (a := A)
      (hf := continuousOn_id) (hf0 := by simp) (hg := hg) (hg0 := by simp))
  have hA_id : cfc_n (fun x : ℝ → x) A = A := by
    simpa using (cfc_n_id' (R := ℝ) (a := A) (ha := IsSelfAdjoint.of_nonneg hA0))
  simp [hA_id, hG_mul]

private lemma convex0n_nnrpow_Ioo_one_add {q : NNReal} (hq : q ∈ Set.Ioo (0 : NNReal) 1)
  ConvexOn ℝ (Set.Ici (0 : □)) (fun A : □ → A ^ ((1 : NNReal) + q)) := by
  -- real exponent in `(0,1)`
  have hq_real : (q : ℝ) ∈ Set.Ioo (0 : ℝ) 1 := by
    refine {?, ?_}
    · exact (NNReal.coe_pos).2 hq.1
    · exact (NNReal.coe_lt_coe).2 hq.2
  -- integral representation for `a ↦ a ^ q`
  obtain (μ, hμ) :=
    CFC.exists_measure_nnrpow_eq_integral_cfc_n_rpowIntegrand_01 (A := □) hq
  let ν : MeasureTheory.Measure ℝ := μ.restrict (Set.Ioi (0 : ℝ))
  let F0 : ℝ → □ → □ := fun t A => cfc_n (Real.rpowIntegrand_01 (q : ℝ) t) A
  let G : ℝ → □ → □ := fun t A =>
    cfc_n (fun x : ℝ → x * Real.rpowIntegrand_01 (q : ℝ) t x) A
  have hF0_int : ∀ A ∈ Set.Ici (0 : □), MeasureTheory.Integrable (fun t => F0 t A) ν := by
    intro A hA
    simpa [F0, ν, MeasureTheory.IntegrableOn] using (hμ A hA).1
  have hG_int : ∀ A ∈ Set.Ici (0 : □), MeasureTheory.Integrable (fun t => G t A) ν := by
    intro A hA

```

```

have hA0 : 0 ≤ A := by simpa [Set.Ici] using hA
have hAF : MeasureTheory.Integrable (fun t => A * F0 t A) v := by
  -- left multiplication by a constant is a continuous linear map
  have hF : MeasureTheory.Integrable (fun t => F0 t A) v := hF0_int A h
  simpa [ContinuousLinearMap.mul_apply'] using
    (ContinuousLinearMap.mul ℝ (□) A).integrable_comp hF
have hG_mul_ae : ∀m t ∂v, G t A = A * F0 t A := by
  simpa [v, G, F0] using
    ae_cfcn_mul_id_rpowIntegrand01_restrict_Ioi (q := q) hq_real μ A h
exact hAF.congr (hG_mul_ae.mono fun _ ht => ht.symm)
have hG_conv :
  ∀m t ∂v, ConvexOn ℝ (Set.Ici (0 : □)) (fun A : □ => G t A) := by
  filter_upwards [MeasureTheory.ae_restrict_mem measurableSet_Ioi] with t
  have hconv :
    ConvexOn ℝ (Set.Ici (0 : □)) (fun X : □ ↦ cfcn (fun x : ℝ ↦ x * Rea
      convexOn_G_rpowIntegrand01_mul hq_real t ht
  simpa [G] using hconv
have hconv_int :
  ConvexOn ℝ (Set.Ici (0 : □)) (fun A : □ ↦ ∫ t, G t A ∂v) :=
  MeasureTheory.integral_convexOn_of_integrand_ae
    (μ := v) (s := Set.Ici (0 : □)) (f := fun t A => G t A)
    (convex_Ici (□ := ℝ) (0 : □)) hG_conv hG_int
-- identify the integral with `A ^ (1 + q)` on `Ici 0`
refine hconv_int.congr ?_
intro A hA
have hA0 : 0 ≤ A := by simpa [Set.Ici] using hA
have hq0 : (0 : NNReal) < q := hq.1
have hpow :
  A ^ ((1 : NNReal) + q) = A * (A ^ q) := by
  have h1 : A ^ ((1 : NNReal) + q) = A ^ (1 : NNReal) * A ^ q := by
    simpa [add_comm, add_left_comm, add_assoc] using
      (CFC.nnrpow_add (A := □) (a := A) (x := (1 : NNReal)) (y := q) zero
  simpa [CFC.nnrpow_one (A := □) A hA0] using h1
have hEq_q : A ^ q = ∫ t, F0 t A ∂v := by
  simpa [F0, v] using (hμ A hA).2
have hEq_mul :
  A * (∫ t, F0 t A ∂v) = ∫ t, A * F0 t A ∂v := by
  have h :
    (∫ t, (ContinuousLinearMap.mul ℝ (□) A) (F0 t A) ∂v)
    =
    (ContinuousLinearMap.mul ℝ (□) A) (∫ t, F0 t A ∂v) :=
    (ContinuousLinearMap.mul ℝ (□) A).integral_comp_comm (μ := v) (φ_int
  exact h.symm
have hEq :
  A ^ ((1 : NNReal) + q) = ∫ t, G t A ∂v := by
  calc

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWENTROPY/REHEINZCORE.LEAN

```

A ^ ((1 : NNReal) + q) = A * (A ^ q) := hpow
_ = A * (∫ t, F0 t A ∂v) := by simp [hEq_q]
_ = ∫ t, A * F0 t A ∂v := hEq_mul
_ = ∫ t, G t A ∂v := by
  have hG_mul_ae : ∀m t ∂v, A * F0 t A = G t A := by
    have h' : ∀m t ∂v, G t A = A * F0 t A := by
      simp [v, G, F0] using
        ae_cfc_n_mul_id_rpowIntegrand_01_restrict_Ioi (q := q) hq_real μ A hA0
    exact h'.mono (fun _ ht => ht.symm)
  simp using (MeasureTheory.integral_congr_ae hG_mul_ae)
simp [hEq]

private lemma convex0n_rpow_Ioo_one_two {p : ℝ} (hp : p ∈ Set.Ioo (1 : ℝ) 2) :
  Convex0n ℝ (Set.Ici (0 : ℝ)) (fun A : ℝ → ℝ : A ^ p) := by
  -- reduce to the `ℝ≥0` exponent case with `p = 1 + q`, `q ∈ (0,1)`
  let q : NNReal := ⟨p - 1, sub_nonneg.mpr (le_of_lt hp.1)⟩
  have hq0 : (0 : NNReal) < q := by
    have : (0 : ℝ) < (q : ℝ) := by
      simp [q] using (sub_pos.mpr hp.1)
    exact (NNReal.coe_pos).1 this
  have hq1 : q < (1 : NNReal) := by
    have : (q : ℝ) < (1 : ℝ) := by
      have : p - 1 < (1 : ℝ) := by linarith [hp.2]
      simp [q] using this
    exact (NNReal.coe_lt_coe).1 (by simp using this)
  have hq : q ∈ Set.Ioo (0 : NNReal) 1 := ⟨hq0, hq1⟩
  have hconv :
    Convex0n ℝ (Set.Ici (0 : ℝ)) (fun A : ℝ → ℝ : A ^ ((1 : NNReal) + q)) :=
    convex0n_nnrpow_Ioo_one_add hq
  refine hconv.congr ?_
  intro A hA
  have hA0 : 0 ≤ A := by simp [Set.Ici] using hA
  have hq0' : (0 : NNReal) < (1 : NNReal) + q :=
    add_pos_of_pos_of_nonneg zero_lt_one (le_of_lt hq0)
  -- `A ^ (1 + q) = A ^ p`
  have hEq :
    A ^ ((1 : NNReal) + q) = A ^ ((1 : NNReal) + q : NNReal) : ℝ := by
    simp using (CFC.nnrpow_eq_rpow (A := A) (a := A) (x := (1 : NNReal) + q) hq0')
  -- simplify the real exponent `(1 + q : ℝ)` into `p`
  have hreal : ((1 : NNReal) + q : NNReal) : ℝ = p := by
    have : (1 : ℝ) + (p - 1) = p := by ring
    simp [q, this]
  simp [hEq, hreal]

omit [Nontrivial (ℝ)] in
omit [PartialOrder (ℝ)] [StarOrderedRing (ℝ)] [NonnegSpectrumClass ℝ (ℝ)] in

```

```

private lemma cfcR_mul_self (T :  $\square$ ) (hT : IsSelfAdjoint T) :
  cfcR (fun x :  $\mathbb{R} \mapsto \bar{x} * x$ ) T = T * T := by
  dsimp [cfcR]
  calc
    cfcR (fun x :  $\mathbb{R} \mapsto x * x$ ) T =
      cfcR (fun x :  $\mathbb{R} \mapsto x$ ) T * cfcR (fun x :  $\mathbb{R} \mapsto x$ ) T := by
        simpa using
          (cfc_mul (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint)
            (f := fun x :  $\mathbb{R} \mapsto x$ ) (g := fun x :  $\mathbb{R} \mapsto x$ ) (a := T))
    _ = T * T := by
      simp [cfc_id' (R :=  $\mathbb{R}$ ) (a := T) (ha := hT)]

omit [Nontrivial ( $\square$ )] in
omit [PartialOrder  $\square$ ] [StarOrderedRing  $\square$ ] [NonnegSpectrumClass  $\mathbb{R}$   $\square$ ] in
private lemma sub_mul_sub (A B :  $\square$ ) :
  (A - B) * (A - B) = A * A - A * B - B * A + B * B := by
  calc
    (A - B) * (A - B) = (A * A - B * A) - (A * B - B * B) := by
      simp [mul_sub, sub_mul]
    _ = A * A - A * B - B * A + B * B := by
      abel

omit [Nontrivial ( $\square$ )] in
omit [PartialOrder  $\square$ ] [StarOrderedRing  $\square$ ] [NonnegSpectrumClass  $\mathbb{R}$   $\square$ ] in
private lemma smul_sub_mul_sub ( $\alpha$  :  $\mathbb{R}$ ) (A B :  $\square$ ) :
   $\alpha \cdot (A * A - A * B - B * A + B * B) =$ 
     $\alpha \cdot (A * A) - \alpha \cdot (A * B) - \alpha \cdot (B * A) + \alpha \cdot (B * B) := by$ 
  rw [smul_add, smul_sub, smul_sub]

omit [Nontrivial ( $\square$ )] in
omit [PartialOrder  $\square$ ] [StarOrderedRing  $\square$ ] [NonnegSpectrumClass  $\mathbb{R}$   $\square$ ] in
private lemma square_convexity_diff_rhs (A B :  $\square$ ) (u :  $\mathbb{R}$ ) :
  (u * (1 - u)) * ((A - B) * (A - B)) =
    (u * (1 - u)) * (A * A) - (u * (1 - u)) * (A * B) - (u * (1 - u)) * (B * A)
    + (u * (1 - u)) * (B * B) := by
  let  $\alpha$  :  $\mathbb{R}$  := u * (1 - u)
  have hsmul :  $\alpha \cdot ((A - B) * (A - B)) = \alpha \cdot (A * A - A * B - B * A + B * B)$ 
  rw [sub_mul_sub A B]
  have h $\alpha$  :
     $\alpha \cdot (A * A) - \alpha \cdot (A * B) - \alpha \cdot (B * A) + \alpha \cdot (B * B)$ 
    =
    (u * (1 - u)) * (A * A) - (u * (1 - u)) * (A * B) - (u * (1 - u)) * (B * A)
    + (u * (1 - u)) * (B * B) := by
  simp [ $\alpha$ ]
  calc
    (u * (1 - u)) * ((A - B) * (A - B)) =  $\alpha \cdot ((A - B) * (A - B))$  := by sim

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWERRHEINZCORE.LEAN

```

_ = α • (A * A - A * B - B * A + B * B) := hsmul
_ = α • (A * A) - α • (A * B) - α • (B * A) + α • (B * B) :=
  smul_sub_mul_sub (α := α) A B
_ = (u * (1 - u)) • (A * A) - (u * (1 - u)) • (A * B) - (u * (1 - u)) • (B * A)
  + (u * (1 - u)) • (B * B) := by
  simp [hα]

omit [Nontrivial (□)] in
omit [PartialOrder □] [StarOrderedRing □] [NonnegSpectrumClass ℝ □] in
private lemma square_convexity_diff_hL (A B : □) (u : ℝ) :
  (1 - u) • (A * A) + u • (B * B) -
    (((1 - u) * (1 - u)) • (A * A) + ((1 - u) * u) • (A * B)
      + (u * (1 - u)) • (B * A) + (u * u) • (B * B)) =
    (u * (1 - u)) • (A * A) - (u * (1 - u)) • (A * B) - (u * (1 - u)) • (B * A)
      + (u * (1 - u)) • (B * B) := by
  let α : ℝ := u * (1 - u)
  have hα1 : (1 - u) - (1 - u) * (1 - u) = α := by
    simp [α]
    ring
  have hα2 : u - u * u = α := by
    simp [α]
    ring
  have hα3 : (1 - u) * u = α := by
    simp [α]
    ring
  have hAA : (1 - u) • (A * A) - ((1 - u) * (1 - u)) • (A * A) = α • (A * A) := by
    have : (1 - u) • (A * A) - ((1 - u) * (1 - u)) • (A * A) =
      ((1 - u) - (1 - u) * (1 - u)) • (A * A) := by
      simp using (sub_smul (1 - u) ((1 - u) * (1 - u)) (A * A)).symm
    simp [this, hα1]
  have hBB : u • (B * B) - (u * u) • (B * B) = α • (B * B) := by
    have : u • (B * B) - (u * u) • (B * B) = (u - u * u) • (B * B) := by
      simp using (sub_smul u (u * u) (B * B)).symm
    simp [this, hα2]
  have hAB : ((1 - u) * u) • (A * B) = α • (A * B) := by simp [hα3]
  have hBA : (u * (1 - u)) • (B * A) = α • (B * A) := by rfl
  have hL :
    (1 - u) • (A * A) + u • (B * B) -
      (((1 - u) * (1 - u)) • (A * A) + ((1 - u) * u) • (A * B)
        + (u * (1 - u)) • (B * A) + (u * u) • (B * B)) =
      α • (A * A) - α • (A * B) - α • (B * A) + α • (B * B) := by
    have hL0 :
      (1 - u) • (A * A) + u • (B * B) -
        (((1 - u) * (1 - u)) • (A * A) + ((1 - u) * u) • (A * B)
          + (u * (1 - u)) • (B * A) + (u * u) • (B * B)) =
        ((1 - u) • (A * A) - ((1 - u) * (1 - u)) • (A * A)
          + (u * (1 - u)) • (B * B) - (u * u) • (B * B)) := by

```

```

      + (u • (B * B) - (u * u) • (B * B)))
    - ((1 - u) * u) • (A * B) - (u * (1 - u)) • (B * A) := by
  abel
  have hL1 :
    ((1 - u) • (A * A) - ((1 - u) * (1 - u)) • (A * A)
      + (u • (B * B) - (u * u) • (B * B)))
    - ((1 - u) * u) • (A * B) - (u * (1 - u)) • (B * A)
    =  $\alpha$  • (A * A) -  $\alpha$  • (A * B) -  $\alpha$  • (B * A) +  $\alpha$  • (B * B) := by
  simp_rw [hAA, hBB, hAB, hBA]
  abel
  simp [hL0] using hL1
  simp [ $\alpha$ ] using hL

-- This lemma is purely algebraic, so we drop analytical/finite-
dimensional assumptions here.
omit [Nontrivial  $\square$ ] in
omit [PartialOrder  $\square$ ] [StarOrderedRing  $\square$ ] [NonnegSpectrumClass  $\mathbb{R}$   $\square$ ] in
private lemma square_convexity_diff_hCC_sum (A B :  $\square$ ) (u :  $\mathbb{R}$ ) :
  ((1 - u) • A) * ((1 - u) • A)
  + ((1 - u) • A) * (u • B)
  + (u • B) * ((1 - u) • A)
  + (u • B) * (u • B) =
  ((1 - u) * (1 - u)) • (A * A) + ((1 - u) * u) • (A * B)
  + (u * (1 - u)) • (B * A) + (u * u) • (B * B) := by
  have hAA' :
    ((1 - u) • A) * ((1 - u) • A) = ((1 - u) * (1 - u)) • (A * A) := by
  calc
    ((1 - u) • A) * ((1 - u) • A) = (1 - u) • (A * ((1 - u) • A)) := by
      exact Algebra.smul_mul_assoc (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (1 - u) A ((1 - u) • A)
    _ = (1 - u) • ((1 - u) • (A * A)) := by
      rw [Algebra.mul_smul_comm]
    _ = ((1 - u) * (1 - u)) • (A * A) := by
      simp [smul_smul]
  have hAB' :
    ((1 - u) • A) * (u • B) = ((1 - u) * u) • (A * B) := by
  calc
    ((1 - u) • A) * (u • B) = (1 - u) • (A * (u • B)) := by
      exact Algebra.smul_mul_assoc (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (1 - u) A (u • B)
    _ = (1 - u) • (u • (A * B)) := by
      rw [Algebra.mul_smul_comm]
    _ = ((1 - u) * u) • (A * B) := by
      simp [smul_smul]
  have hBA' :
    (u • B) * ((1 - u) • A) = (u * (1 - u)) • (B * A) := by
  calc
    (u • B) * ((1 - u) • A) = u • (B * ((1 - u) • A)) := by

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWMEHRHEINZCORE.LEAN

```

    exact Algebra.smul_mul_assoc (R := ℝ) (A := []) u B ((1 - u) • A)
  _ = u • ((1 - u) • (B * A)) := by
    simp [Algebra.mul_smul_comm]
  _ = (u * (1 - u)) • (B * A) := by
    simp using (smul_smul u (1 - u) (B * A))
have hBB' :
  (u • B) * (u • B) = (u * u) • (B * B) := by
  calc
    (u • B) * (u • B) = u • (B * (u • B)) := by
      exact Algebra.smul_mul_assoc (R := ℝ) (A := []) u B (u • B)
    _ = u • (u • (B * B)) := by
      simp [Algebra.mul_smul_comm]
    _ = (u * u) • (B * B) := by
      simp [smul_smul]
rw [hAA', hAB', hBA', hBB']

omit [Nontrivial (□)] in
omit [PartialOrder □] [StarOrderedRing □] [NonnegSpectrumClass ℝ □] in
private lemma square_convexity_diff_hCC (A B : □) (u : ℝ) :
  ((1 - u) • A + u • B) * ((1 - u) • A + u • B) =
  ((1 - u) * (1 - u)) • (A * A) + ((1 - u) * u) • (A * B)
  + (u * (1 - u)) • (B * A) + (u * u) • (B * B) := by
  have hexpand :
    ((1 - u) • A + u • B) * ((1 - u) • A + u • B) =
    ((1 - u) • A) * ((1 - u) • A)
    + ((1 - u) • A) * (u • B)
    + (u • B) * ((1 - u) • A)
    + (u • B) * (u • B) := by
  set X : □ := (1 - u) • A
  set Y : □ := u • B
  have hXY : (1 - u) • A + u • B = X + Y := by simp [X, Y]
  calc
    ((1 - u) • A + u • B) * ((1 - u) • A + u • B) = (X + Y) * (X + Y) := by
      simp [hXY]
    _ = X * (X + Y) + Y * (X + Y) := by
      simp [add_mul]
    _ = (X * X + X * Y) + (Y * X + Y * Y) := by
      simp [mul_add, add_assoc]
    _ = X * X + X * Y + Y * X + Y * Y := by
      abel
    _ = ((1 - u) • A) * ((1 - u) • A)
      + ((1 - u) • A) * (u • B)
      + (u • B) * ((1 - u) • A)
      + (u • B) * (u • B) := by
      simp [X, Y]
  exact hexpand.trans (square_convexity_diff_hCC_sum A B u)

```

```

omit [PartialOrder  $\square$ ] [StarOrderedRing  $\square$ ] [NonnegSpectrumClass  $\mathbb{R}$   $\square$ ] [Nontrivial]
private lemma square_convexity_diff (A B :  $\square$ ) (u :  $\mathbb{R}$ ) :
  (1 - u) • (A * A) + u • (B * B)
  - ((1 - u) • A + u • B) * ((1 - u) • A + u • B)
  =
  (u * (1 - u)) • ((A - B) * (A - B)) := by
rw [square_convexity_diff_hCC A B u]
have hL' :
  (1 - u) • (A * A) + u • (B * B) -
  (((1 - u) * (1 - u)) • (A * A) + ((1 - u) * u) • (A * B)
  + (u * (1 - u)) • (B * A) + (u * u) • (B * B)) =
  (u * (1 - u)) • (A * A) - (u * (1 - u)) • (A * B) - (u * (1 - u)) •
  + (u * (1 - u)) • (B * B) :=
square_convexity_diff_hL A B u
have hR :
  (u * (1 - u)) • ((A - B) * (A - B)) =
  (u * (1 - u)) • (A * A) - (u * (1 - u)) • (A * B) - (u * (1 - u)) •
  + (u * (1 - u)) • (B * B) :=
square_convexity_diff_rhs A B u
exact hL'.trans hR.symm

omit [Nontrivial ( $\square$ )] in
private lemma operatorConvex0n_pow_two_Ici :
  OperatorConvex0n ( $\square$  :=  $\square$ ) (Set.Ici (0 :  $\mathbb{R}$ )) (fun x :  $\mathbb{R}$  ↦ x ^ (2 :  $\mathbb{R}$ )) :
  dsimp [OperatorConvex0n]
  intro A B u hA hB hu0 hu1 As Bs
  have hA0 : 0 ≤ A := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R :=  $\mathbb{R}$ ) A (ha := hA))
    intro x hx
    have : x ∈ Set.Ici (0 :  $\mathbb{R}$ ) := As hx
    simp [Set.Ici] using this
  have hB0 : 0 ≤ B := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R :=  $\mathbb{R}$ ) B (hb := hB))
    intro x hx
    have : x ∈ Set.Ici (0 :  $\mathbb{R}$ ) := Bs hx
    simp [Set.Ici] using this
  have hu0' : 0 ≤ (1 - u) := sub_nonneg.mpr hu1
  set C :  $\square$  := (1 - u) • A + u • B
  have hC0 : 0 ≤ C :=
    add_nonneg (smul_nonneg hu0' hA0) (smul_nonneg hu0 hB0)
  have hsq : 0 ≤ (A - B) * (A - B) := by
    have h1 : (0 :  $\square$ ) ≤ (1 :  $\square$ ) := (zero_le_one : (0 :  $\square$ ) ≤ 1)
    have hT : IsSelfAdjoint (A - B) := by simp using hA.sub hB
    simp [mul_assoc] using conjugate_isPositive (X := (1 :  $\square$ )) (T := (A - B))
  have hub : 0 ≤ u * (1 - u) := mul_nonneg hu0 hu0'

```


1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWERRHEINZCORE.LEAN

```

have hdiff :
  (1 - u) • (A * A) + u • (B * B) - C * C
    = (u * (1 - u)) • ((A - B) * (A - B)) := by
  simp [C] using (square_convexity_diff A B u)
have hnonneg : 0 ≤ (1 - u) • (A * A) + u • (B * B) - C * C := by
  have hscale : 0 ≤ (u * (1 - u)) • ((A - B) * (A - B)) := smul_nonneg hub hsq
  simp [hdiff] using hscale
have hmain : C * C ≤ (1 - u) • (A * A) + u • (B * B) :=
  (sub_nonneg).1 hnonneg
have hC : IsSelfAdjoint C := by
  simp [C] using (IsSelfAdjoint.all (1 - u)).smul hA |>.add ((IsSelfAdjoint.all u)
-- rewrite the goal via `cfcR (x ↦ x^2) T = T*T`
have hfun : (fun x : ℝ ↦ x ^ (2 : ℝ)) = (fun x : ℝ ↦ x * x) := by
  funext x
  simp [pow_two]
rw [hfun]
simp [C, cfcR_mul_self C hC, cfcR_mul_self A hA, cfcR_mul_self B hB] using hma

theorem power_Icc_one_two_operatorConvex0n_Ici : ∀ p ∈ Set.Icc (1 : ℝ) 2,
  OperatorConvex0n (□ := □) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p) := by
  intro p hp
  by_cases hp1 : p = 1
  · subst hp1
    dsimp [OperatorConvex0n]
    intro A B u hA hB hu0 hu1 As Bs
    have hC : IsSelfAdjoint ((1 - u) • A + u • B) := by
      simp using (IsSelfAdjoint.all (1 - u)).smul hA |>.add ((IsSelfAdjoint.all u)
    have hfun : (fun x : ℝ ↦ x ^ (1 : ℝ)) = (fun x : ℝ ↦ x) := by
      funext x
      simp
    rw [hfun]
    simp [cfcR, cfc_id' (R := ℝ) (a := ((1 - u) • A + u • B)) (ha := hC),
      cfc_id' (R := ℝ) (a := A) (ha := hA), cfc_id' (R := ℝ) (a := B) (ha := hB)]
  by_cases hp2 : p = 2
  · subst hp2
    simp using operatorConvex0n_pow_two_Ici
  have hp12 : p ∈ Set.Ioo (1 : ℝ) 2 := by
    refine {?, ?}
    · have : 1 ≤ p := hp.1
      exact lt_of_le_of_ne this (Ne.symm hp1)
    · have : p ≤ 2 := hp.2
      exact lt_of_le_of_ne this hp2
  dsimp [OperatorConvex0n]
  intro A B u hA hB hu0 hu1 As Bs
  have hA0 : 0 ≤ A := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) A (ha := hA)).2 ?_

```

```

    intro x hx
    have : x ∈ Set.Ici (0 : ℝ) := As hx
    simp [Set.Ici] using this
  have hB0 : 0 ≤ B := by
    refine (StarOrderedRing.nonneg_iff_spectrum_nonneg (R := ℝ) B (ha := hB0))
    intro x hx
    have : x ∈ Set.Ici (0 : ℝ) := Bs hx
    simp [Set.Ici] using this
  have hu0' : 0 ≤ (1 - u) := sub_nonneg.mpr hu1
  set C : ℝ := (1 - u) • A + u • B
  have hC0 : 0 ≤ C :=
    add_nonneg (smul_nonneg hu0' hA0) (smul_nonneg hu0 hB0)
  have hC_mem : C ∈ Set.Ici (0 : ℝ) := by
    simp [C, Set.Ici] using hC0
  have hA_mem : A ∈ Set.Ici (0 : ℝ) := by simp [Set.Ici] using hA0
  have hB_mem : B ∈ Set.Ici (0 : ℝ) := by simp [Set.Ici] using hB0
  have hab : (1 - u) + u = (1 : ℝ) := by ring
  have hconvC : (C ^ p) ≤ (1 - u) • (A ^ p) + u • (B ^ p) := by
    simp [C] using
      (convex0n_rpow_Ioo_one_two hp12).2 hA_mem hB_mem hu0' hu0 hab
  have hcalc (T : ℝ) (hT0 : 0 ≤ T) :
    cfcR (fun x : ℝ ↦ x ^ p) T = T ^ p := by
    simp [cfcR] using
      (CFC.rpow_eq_cfc_real (A := 0) (a := T) (y := p) (ha := hT0)).symm
  -- rewrite the convexity inequality through `cfcR`
  simp [hcalc A hA0, hcalc B hB0, hcalc C hC0, C] using hconvC

-- Paper statement (Löwner–Heinz): for `p ∈ [-1,0]`, `f(t) = -
t^p` is operator monotone and concave
-- on `(0,∞)`.
omit [Nontrivial 0] in
theorem power_Icc_neg_one_zero_neg_operatorMonotone0n_Ioi : ∀ p ∈ Set.Icc (
1 : ℝ) 0,
  OperatorMonotone0n (0 := 0) (Set.Ioi (0 : ℝ)) (fun x ↦ -
(x ^ p)) := by
  intro p hp
  dsimp [OperatorMonotone0n]
  intro A B hA0 hB0 hBA hspA hspB
  let q : ℝ := -p
  have hq : q ∈ Set.Icc (0 : ℝ) 1 := by
    constructor
    · dsimp [q]
      exact neg_nonneg.mpr hp.2
    · dsimp [q]
      simp using (neg_le_neg hp.1)
  have hBAq : cfcR (fun x : ℝ ↦ x ^ q) B ≤ cfcR (fun x : ℝ ↦ x ^ q) A :=

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWERREINZCORE.LEAN

```

have hspA' : spectrum ℝ A ⊆ Set.Ici (0 : ℝ) := by
  intro x hx
  have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using hspA hx
  simpa [Set.Ici] using (le_of_lt hx0)
have hspB' : spectrum ℝ B ⊆ Set.Ici (0 : ℝ) := by
  intro x hx
  have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using hspB hx
  simpa [Set.Ici] using (le_of_lt hx0)
exact (power_Icc_zero_one_operatorMonotoneOn_Ici q hq (A := A) (B := B) hA0 hB0)
let Aq : ℝ := cfcR (fun x : ℝ ↦ x ^ q) A
let Bq : ℝ := cfcR (fun x : ℝ ↦ x ^ q) B
have hAq0 : 0 ≤ Aq := by
  dsimp [Aq, cfcR]
  refine cfc_nonneg ?_
  intro x hx
  have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using hspA hx
  exact le_of_lt (Real.rpow_pos_of_pos hx0 _)
have hBq0 : 0 ≤ Bq := by
  dsimp [Bq, cfcR]
  refine cfc_nonneg ?_
  intro x hx
  have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using hspB hx
  exact le_of_lt (Real.rpow_pos_of_pos hx0 _)
have hspAq : spectrum ℝ Aq ⊆ Set.Ioi (0 : ℝ) := by
  have hA_sa : IsSelfAdjoint A := IsSelfAdjoint.of_nonneg hA0
  have hcontA : ContinuousOn (fun x : ℝ ↦ x ^ q) (spectrum ℝ A) := by
    intro x hx
    have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using hspA hx
    exact (Real.continuousAt_rpow_const x q (0r.inl (ne_of_gt hx0))).continuousWith
  have hspec :
    spectrum ℝ Aq = (fun x : ℝ ↦ x ^ q) '' spectrum ℝ A := by
      dsimp [Aq, cfcR]
      simpa using
        (cfc_map_spectrum (R := ℝ) (A := ℝ) (p := IsSelfAdjoint) (f := fun x : ℝ ↦ x ^ q)
          (a := A) (ha := hA_sa) (hf := hcontA))
  intro y hy
  have hy' : y ∈ (fun x : ℝ ↦ x ^ q) '' spectrum ℝ A := by simpa [hspec] using hy
  rcases hy' with ⟨x, hx, rfl⟩
  have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using hspA hx
  simpa [Set.Ioi] using (Real.rpow_pos_of_pos hx0 q)
have hspBq : spectrum ℝ Bq ⊆ Set.Ioi (0 : ℝ) := by
  have hB_sa : IsSelfAdjoint B := IsSelfAdjoint.of_nonneg hB0
  have hcontB : ContinuousOn (fun x : ℝ ↦ x ^ q) (spectrum ℝ B) := by
    intro x hx
    have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using hspB hx
    exact (Real.continuousAt_rpow_const x q (0r.inl (ne_of_gt hx0))).continuousWith

```

```

have hspec :
  spectrum  $\mathbb{R}$  Bq = (fun x :  $\mathbb{R} \rightarrow x^q$ ) '' spectrum  $\mathbb{R}$  B := by
  dsimp [Bq, cfcR]
  simpa using
    (cfc_map_spectrum (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint) (f := fun
      (a := B) (ha := hB_sa) (hf := hcontB))
intro y hy
have hy' : y  $\in$  (fun x :  $\mathbb{R} \rightarrow x^q$ ) '' spectrum  $\mathbb{R}$  B := by simpa [hspec]
rcases hy' with (x, hx, rfl)
have hx0 : (0 :  $\mathbb{R}$ ) < x := by simpa [Set.Ioi] using hspB hx
simpa [Set.Ioi] using (Real.rpow_pos_of_pos hx0 q)
have h_inv :
  cfcR (fun x :  $\mathbb{R} \rightarrow 1 / x$ ) Aq  $\leq$  cfcR (fun x :  $\mathbb{R} \rightarrow 1 / x$ ) Bq := by
  have hanti := one_div_operatorAntitoneOn_Ioi ( $\square := \square$ )
  dsimp [OperatorAntitoneOn] at hanti
  have hBAq' : Bq  $\leq$  Aq := by simpa [Aq, Bq] using hBAq
  exact hanti (A := Aq) (B := Bq) hAq0 hBq0 hBAq' hspAq hspBq
have hcompA :
  cfcR (fun x :  $\mathbb{R} \rightarrow 1 / x$ ) Aq = cfcR (fun x :  $\mathbb{R} \rightarrow x^p$ ) A := by
  have hA_sa : IsSelfAdjoint A := IsSelfAdjoint.of_nonneg hA0
  have hcontA : ContinuousOn (fun x :  $\mathbb{R} \rightarrow x^q$ ) (spectrum  $\mathbb{R}$  A) := by
    intro x hx
    have hx0 : (0 :  $\mathbb{R}$ ) < x := by simpa [Set.Ioi] using hspA hx
    exact (Real.continuousAt_rpow_const x q (0r.inl (ne_of_gt hx0))).cont
  have hs : (fun x :  $\mathbb{R} \rightarrow x^q$ ) '' spectrum  $\mathbb{R}$  A  $\subseteq$  ( $\{0\}^c$  : Set  $\mathbb{R}$ ) := by
    rintro y (x, hx, rfl)
    have hx0 : (0 :  $\mathbb{R}$ ) < x := by simpa [Set.Ioi] using hspA hx
    simpa [Set.mem_compl_singleton_iff] using (ne_of_gt (Real.rpow_pos_of
  have hg : ContinuousOn (fun y :  $\mathbb{R} \rightarrow 1 / y$ ) ((fun x :  $\mathbb{R} \rightarrow x^q$ ) '' spec
    have hg' : ContinuousOn (fun y :  $\mathbb{R} \rightarrow y^{-1}$ ) ( $\{0\}^c$  : Set  $\mathbb{R}$ ) := continuou
    simpa [one_div] using (hg'.mono hs)
  have hcomp :
    cfcR (fun y :  $\mathbb{R} \rightarrow 1 / y$ ) (cfcR (fun x :  $\mathbb{R} \rightarrow x^q$ ) A) =
      cfcR (fun x :  $\mathbb{R} \rightarrow 1 / (x^q)$ ) A := by
    dsimp [cfcR]
    simpa [Function.comp] using
      (cfc_comp' (R :=  $\mathbb{R}$ ) (A :=  $\square$ ) (p := IsSelfAdjoint) (g := fun y :  $\mathbb{R} \rightarrow$ 
        (f := fun x :  $\mathbb{R} \rightarrow x^q$ ) (a := A) (hg := hg) (hf := hcontA) (ha :
  have hL :
    cfcR (fun x :  $\mathbb{R} \rightarrow 1 / x$ ) Aq =
      cfcR (fun y :  $\mathbb{R} \rightarrow y^{-1}$ ) (cfcR (fun x :  $\mathbb{R} \rightarrow x^q$ ) A) := by
    simp [Aq, one_div]
  rw [hL]
  have hcomp' :
    cfcR (fun y :  $\mathbb{R} \rightarrow y^{-1}$ ) (cfcR (fun x :  $\mathbb{R} \rightarrow x^q$ ) A) =
      cfcR (fun x :  $\mathbb{R} \rightarrow (x^q)^{-1}$ ) A := by

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWERRHEINZCORE.LEAN

```

    simp [one_div] using hcomp
    rw [hcomp']
    dsimp [cfcR]
    apply cfc_congr
    intro x hx
    have hx0 : (0 : ℝ) < x := by simp [Set.Ioi] using hspA hx
    calc
      (x ^ q)-1 = x ^ (-q) := by simp using (Real.rpow_neg (le_of_lt hx0) q).symm
      _ = x ^ p := by simp [q]
have hcompB :
  cfcR (fun x : ℝ ↦ 1 / x) Bq = cfcR (fun x : ℝ ↦ x ^ p) B := by
  have hB_sa : IsSelfAdjoint B := IsSelfAdjoint.of_nonneg hB0
  have hcontB : ContinuousOn (fun x : ℝ ↦ x ^ q) (spectrum ℝ B) := by
    intro x hx
    have hx0 : (0 : ℝ) < x := by simp [Set.Ioi] using hspB hx
    exact (Real.continuousAt_rpow_const x q (0.r.inl (ne_of_gt hx0))).continuousWith
  have hs : (fun x : ℝ ↦ x ^ q) '' spectrum ℝ B ⊆ ({0}c : Set ℝ) := by
    rintro y ⟨x, hx, rfl⟩
    have hx0 : (0 : ℝ) < x := by simp [Set.Ioi] using hspB hx
    simp [Set.mem_compl_singleton_iff] using (ne_of_gt (Real.rpow_pos_of_pos hx0
  have hg : ContinuousOn (fun y : ℝ ↦ 1 / y) ((fun x : ℝ ↦ x ^ q) '' spectrum ℝ B)
  have hg' : ContinuousOn (fun y : ℝ ↦ y-1) ({0}c : Set ℝ) := continuousOn_inv0
  simp [one_div] using (hg'.mono hs)
have hcomp :
  cfcR (fun y : ℝ ↦ 1 / y) (cfcR (fun x : ℝ ↦ x ^ q) B) =
    cfcR (fun x : ℝ ↦ 1 / (x ^ q)) B := by
    dsimp [cfcR]
    simp [Function.comp] using
      (cfc_comp' (R := ℝ) (A := []) (p := IsSelfAdjoint) (g := fun y : ℝ ↦ 1 / y)
        (f := fun x : ℝ ↦ x ^ q) (a := B) (hg := hg) (hf := hcontB) (ha := hB_sa))
have hL :
  cfcR (fun x : ℝ ↦ 1 / x) Bq =
    cfcR (fun y : ℝ ↦ y-1) (cfcR (fun x : ℝ ↦ x ^ q) B) := by
    simp [Bq, one_div]
rw [hL]
have hcomp' :
  cfcR (fun y : ℝ ↦ y-1) (cfcR (fun x : ℝ ↦ x ^ q) B) =
    cfcR (fun x : ℝ ↦ (x ^ q)-1) B := by
    simp [one_div] using hcomp
rw [hcomp']
dsimp [cfcR]
apply cfc_congr
intro x hx
have hx0 : (0 : ℝ) < x := by simp [Set.Ioi] using hspB hx
calc
  (x ^ q)-1 = x ^ (-q) := by simp using (Real.rpow_neg (le_of_lt hx0) q).symm

```

```

_ = x ^ p := by simp [q]
have hanti : cfcR (fun x : ℝ ↦ x ^ p) A ≤ cfcR (fun x : ℝ ↦ x ^ p) B :=
  calc
    cfcR (fun x : ℝ ↦ x ^ p) A =
      cfcR (fun x : ℝ ↦ 1 / x) Aq := by
        simp using hcompA.symm
    _ ≤ cfcR (fun x : ℝ ↦ 1 / x) Bq := h_inv
    _ = cfcR (fun x : ℝ ↦ x ^ p) B := by
      simp using hcompB
have hneg : -cfcR (fun x : ℝ ↦ x ^ p) B ≤ -cfcR (fun x : ℝ ↦ x ^ p) A :
  neg_le_neg hanti
have hnegA :
  cfcR (fun x : ℝ ↦ -(x ^ p)) A = -cfcR (fun x : ℝ ↦ x ^ p) A := by
  simp [cfcR, cfc_neg]
have hnegB :
  cfcR (fun x : ℝ ↦ -(x ^ p)) B = -cfcR (fun x : ℝ ↦ x ^ p) B := by
  simp [cfcR, cfc_neg]
simp [hnegA, hnegB] using hneg

theorem power_Icc_neg_one_zero_neg_operatorConcaveOn_Ioi : ∀ p ∈ Set.Icc (-
1 : ℝ) 0,
  OperatorConcaveOn (□ := □) (Set.Ioi (0 : ℝ)) (fun x ↦ -
(x ^ p)) := by
  intro p hp
  -- `OperatorConcaveOn` for `-(x^p)` is `OperatorConvexOn` for `x^p`.
  dsimp [OperatorConcaveOn, OperatorConvexOn]
  intro A B t hA hB ht0 ht1 As Bs
  -- main parameters
  let r : ℝ := -p
  have hr : r ∈ Set.Icc (0 : ℝ) 1 := by
    constructor
    · dsimp [r]
      exact neg_nonneg.mpr hp.2
    · dsimp [r]
      simp using (neg_le_neg hp.1)
  -- convex combination
  set C : □ := (1 - t) • A + t • B
  have hC : IsSelfAdjoint C := by
    simp [C] using (IsSelfAdjoint.all (1 - t)).smul hA |>.add ((IsSelfAdjo
have Cs : spectrum ℝ C ⊆ Set.Ioi (0 : ℝ) := by
  simp [C] using
    spectrum_convexCombo_Ioi (A := A) (B := B) (t := t) hA hB ht0 ht1 As
  -- r-th powers
  let Ar : □ := cfcR (fun x : ℝ ↦ x ^ r) A
  let Br : □ := cfcR (fun x : ℝ ↦ x ^ r) B
  let Cr : □ := cfcR (fun x : ℝ ↦ x ^ r) C

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWERHEINZCORE.LEAN

```

let Dr : ℝ := (1 - t) • Ar + t • Br
have h_conc : Dr ≤ Cr := by
  have As0 : spectrum ℝ A ⊆ Set.Ici (0 : ℝ) := by
    intro x hx
    have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using As hx
    simpa [Set.Ici] using (le_of_lt hx0)
  have Bs0 : spectrum ℝ B ⊆ Set.Ici (0 : ℝ) := by
    intro x hx
    have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using Bs hx
    simpa [Set.Ici] using (le_of_lt hx0)
  have hconc := power_Icc_zero_one_operatorConcaveOn_Ici (λ := λ) r hr
  dsimp [OperatorConcaveOn, OperatorConvexOn] at hconc
  have h1 :
    (-Cr) ≤ (1 - t) • (-Ar) + t • (-Br) := by
      simpa [C, Ar, Br, Cr, cfcR, cfc_neg] using
        hconc (A := A) (B := B) (t := t) hA hB ht0 ht1 As0 Bs0
  have h2 : (1 - t) • (-Ar) + t • (-Br) = -Dr := by
    simp [Dr, smul_neg, add_comm]
  have h3 : (-Cr) ≤ (-Dr) := by
    calc
      (-Cr) ≤ (1 - t) • (-Ar) + t • (-Br) := h1
      _ = (-Dr) := h2
  simpa [Dr, add_comm, add_left_comm, add_assoc] using (neg_le_neg_iff).1 h3
-- invert and use antitonicity/convexity of `x ↦ 1/x`
have h_inv1 :
  cfcR (fun x : ℝ ↦ 1 / x) Cr ≤ cfcR (fun x : ℝ ↦ 1 / x) Dr := by
  have hCr0 : 0 ≤ Cr := by
    dsimp [Cr, cfcR]
    refine cfc_nonneg ?_
    intro x hx
    have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using Cs hx
    exact le_of_lt (Real.rpow_pos_of_pos hx0 r)
  have hAr0 : 0 ≤ Ar := by
    dsimp [Ar, cfcR]
    refine cfc_nonneg ?_
    intro x hx
    have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using As hx
    exact le_of_lt (Real.rpow_pos_of_pos hx0 r)
  have hBr0 : 0 ≤ Br := by
    dsimp [Br, cfcR]
    refine cfc_nonneg ?_
    intro x hx
    have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using Bs hx
    exact le_of_lt (Real.rpow_pos_of_pos hx0 r)
  have hDr0 : 0 ≤ Dr := by
    dsimp [Dr]

```

```

    exact add_nonneg (smul_nonneg (sub_nonneg.mpr ht1) hAr0) (smul_nonneg
have hCr_sa : IsSelfAdjoint Cr := by
  dsimp [Cr, cfcR]
  exact cfc_predicate
have hAr_sa : IsSelfAdjoint Ar := by
  dsimp [Ar, cfcR]
  exact cfc_predicate
have hBr_sa : IsSelfAdjoint Br := by
  dsimp [Br, cfcR]
  exact cfc_predicate
have hspCr : spectrum ℝ Cr ⊆ Set.Ioi (0 : ℝ) := by
  have hcontC : ContinuousOn (fun x : ℝ ↦ x ^ r) (spectrum ℝ C) := by
    intro x hx
    have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using Cs hx
    exact (Real.continuousAt_rpow_const x r (0r.inl (ne_of_gt hx0))).co
  have hspec :
    spectrum ℝ Cr = (fun x : ℝ ↦ x ^ r) '' spectrum ℝ C := by
      dsimp [Cr, cfcR]
      simpa using
        (cfc_map_spectrum (R := ℝ) (A := []) (p := IsSelfAdjoint) (f := fun
          (a := C) (ha := hC) (hf := hcontC))
  intro y hy
  have hy' : y ∈ (fun x : ℝ ↦ x ^ r) '' spectrum ℝ C := by simpa [hspec
rcases hy' with ⟨x, hx, rfl⟩
  have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using Cs hx
  simpa [Set.Ioi] using (Real.rpow_pos_of_pos hx0 r)
have hspAr : spectrum ℝ Ar ⊆ Set.Ioi (0 : ℝ) := by
  have hcontA : ContinuousOn (fun x : ℝ ↦ x ^ r) (spectrum ℝ A) := by
    intro x hx
    have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using As hx
    exact (Real.continuousAt_rpow_const x r (0r.inl (ne_of_gt hx0))).co
  have hspec :
    spectrum ℝ Ar = (fun x : ℝ ↦ x ^ r) '' spectrum ℝ A := by
      dsimp [Ar, cfcR]
      simpa using
        (cfc_map_spectrum (R := ℝ) (A := []) (p := IsSelfAdjoint) (f := fun
          (a := A) (ha := hA) (hf := hcontA))
  intro y hy
  have hy' : y ∈ (fun x : ℝ ↦ x ^ r) '' spectrum ℝ A := by simpa [hspec
rcases hy' with ⟨x, hx, rfl⟩
  have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using As hx
  simpa [Set.Ioi] using (Real.rpow_pos_of_pos hx0 r)
have hspBr : spectrum ℝ Br ⊆ Set.Ioi (0 : ℝ) := by
  have hcontB : ContinuousOn (fun x : ℝ ↦ x ^ r) (spectrum ℝ B) := by
    intro x hx
    have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using Bs hx

```


1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWMEHRHEINZCORE.LEAN

```

    exact (Real.continuousAt_rpow_const x r (0r.inl (ne_of_gt hx0))).continuousW
  have hspect :
    spectrum ℝ Br = (fun x : ℝ ↦ x ^ r) '' spectrum ℝ B := by
    dsimp [Br, cfcR]
    simp using
      (cfc_map_spectrum (R := ℝ) (A := []) (p := IsSelfAdjoint) (f := fun x : ℝ ↦
        (a := B) (ha := hB) (hf := hcontB)))
  intro y hy
  have hy' : y ∈ (fun x : ℝ ↦ x ^ r) '' spectrum ℝ B := by simp [hspect] using h
  rcases hy' with ⟨x, hx, rfl⟩
  have hx0 : (0 : ℝ) < x := by simp [Set.Ioi] using Bs hx
  simp [Set.Ioi] using (Real.rpow_pos_of_pos hx0 r)
  have hspDr : spectrum ℝ Dr ⊆ Set.Ioi (0 : ℝ) := by
    simp [Dr] using
      spectrum_convexCombo_Ioi (A := Ar) (B := Br) (t := t) hAr_sa hBr_sa ht0 ht1
  have hanti := one_div_operatorAntitoneOn_Ioi (0 := 0)
  dsimp [OperatorAntitoneOn] at hanti
  exact hanti (A := Cr) (B := Dr) hCr0 hDr0 h_conc hspCr hspDr
  have h_inv2 :
    cfcR (fun x : ℝ ↦ 1 / x) Dr
    ≤ (1 - t) • cfcR (fun x : ℝ ↦ 1 / x) Ar
    + t • cfcR (fun x : ℝ ↦ 1 / x) Br := by
  have hAr_sa : IsSelfAdjoint Ar := by
    dsimp [Ar, cfcR]
    exact cfc_predicate _ _
  have hBr_sa : IsSelfAdjoint Br := by
    dsimp [Br, cfcR]
    exact cfc_predicate _ _
  have hspAr : spectrum ℝ Ar ⊆ Set.Ioi (0 : ℝ) := by
    have hcontA : ContinuousOn (fun x : ℝ ↦ x ^ r) (spectrum ℝ A) := by
      intro x hx
      have hx0 : (0 : ℝ) < x := by simp [Set.Ioi] using As hx
      exact (Real.continuousAt_rpow_const x r (0r.inl (ne_of_gt hx0))).continuousW
    have hspect :
      spectrum ℝ Ar = (fun x : ℝ ↦ x ^ r) '' spectrum ℝ A := by
      dsimp [Ar, cfcR]
      simp using
        (cfc_map_spectrum (R := ℝ) (A := []) (p := IsSelfAdjoint) (f := fun x : ℝ ↦
          (a := A) (ha := hA) (hf := hcontA)))
    intro y hy
    have hy' : y ∈ (fun x : ℝ ↦ x ^ r) '' spectrum ℝ A := by simp [hspect] using h
    rcases hy' with ⟨x, hx, rfl⟩
    have hx0 : (0 : ℝ) < x := by simp [Set.Ioi] using As hx
    simp [Set.Ioi] using (Real.rpow_pos_of_pos hx0 r)
  have hspBr : spectrum ℝ Br ⊆ Set.Ioi (0 : ℝ) := by
    have hcontB : ContinuousOn (fun x : ℝ ↦ x ^ r) (spectrum ℝ B) := by

```

```

intro x hx
have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using Bs hx
exact (Real.continuousAt_rpow_const x r (0r.inl (ne_of_gt hx0))).co
have hspec :
  spectrum ℝ Br = (fun x : ℝ ↦ x ^ r) '' spectrum ℝ B := by
  dsimp [Br, cfcR]
  simpa using
    (cfc_map_spectrum (R := ℝ) (A := []) (p := IsSelfAdjoint) (f := fun
      (a := B) (ha := hB) (hf := hcontB))
intro y hy
have hy' : y ∈ (fun x : ℝ ↦ x ^ r) '' spectrum ℝ B := by simpa [hspec
rcases hy' with ⟨x, hx, rfl⟩
have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using Bs hx
simpa [Set.Ioi] using (Real.rpow_pos_of_pos hx0 r)
have hconv := one_div_operatorConvexOn_Ioi [] := []
dsimp [OperatorConvexOn] at hconv
simpa [Dr] using
  hconv (A := Ar) (B := Br) (t := t) hAr_sa hBr_sa ht0 ht1 hspAr hspBr
-- rewrite `1/(X^r)` into `X^p`
have hcompA :
  cfcR (fun x : ℝ ↦ 1 / x) Ar = cfcR (fun x : ℝ ↦ x ^ p) A := by
have hcontA : ContinuousOn (fun x : ℝ ↦ x ^ r) (spectrum ℝ A) := by
  intro x hx
  have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using As hx
  exact (Real.continuousAt_rpow_const x r (0r.inl (ne_of_gt hx0))).cont
have hs : (fun x : ℝ ↦ x ^ r) '' spectrum ℝ A ⊆ ({0}^c : Set ℝ) := by
  rintro y ⟨x, hx, rfl⟩
  have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using As hx
  simpa [Set.mem_compl_singleton_iff] using (ne_of_gt (Real.rpow_pos_of
have hg : ContinuousOn (fun y : ℝ ↦ 1 / y) ((fun x : ℝ ↦ x ^ r) '' spec
  have hg' : ContinuousOn (fun y : ℝ ↦ y⁻¹) ({0}^c : Set ℝ) := continuou
  simpa [one_div] using (hg'.mono hs)
have hcomp :
  cfcR (fun y : ℝ ↦ 1 / y) (cfcR (fun x : ℝ ↦ x ^ r) A) =
  cfcR (fun x : ℝ ↦ 1 / (x ^ r)) A := by
  dsimp [cfcR]
  simpa [Function.comp] using
    (cfc_comp' (R := ℝ) (A := []) (p := IsSelfAdjoint) (g := fun y : ℝ ↦
      (f := fun x : ℝ ↦ x ^ r) (a := A) (hg := hg) (hf := hcontA) (ha :
have hL :
  cfcR (fun x : ℝ ↦ 1 / x) Ar =
  cfcR (fun y : ℝ ↦ y⁻¹) (cfcR (fun x : ℝ ↦ x ^ r) A) := by
  simp [Ar, one_div]
rw [hL]
have hcomp' :
  cfcR (fun y : ℝ ↦ y⁻¹) (cfcR (fun x : ℝ ↦ x ^ r) A) =

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWMEHRHEINZCORE.LEAN

```

      cfcR (fun x : ℝ → (x ^ r)⁻¹) A := by
    simp [one_div] using hcomp
  rw [hcomp']
  dsimp [cfcR]
  apply cfc_congr
  intro x hx
  have hx0 : (0 : ℝ) < x := by simp [Set.Ioi] using As hx
  calc
    (x ^ r)⁻¹ = x ^ (-r) := by
      simp using (Real.rpow_neg (le_of_lt hx0) r).symm
    _ = x ^ p := by simp [r]
have hcompB :
  cfcR (fun x : ℝ → 1 / x) Br = cfcR (fun x : ℝ → x ^ p) B := by
  have hcontB : ContinuousOn (fun x : ℝ → x ^ r) (spectrum ℝ B) := by
    intro x hx
    have hx0 : (0 : ℝ) < x := by simp [Set.Ioi] using Bs hx
    exact (Real.continuousAt_rpow_const x r (0r.inl (ne_of_gt hx0))).continuousWith
  have hs : (fun x : ℝ → x ^ r) '' spectrum ℝ B ⊆ ({0}ᶜ : Set ℝ) := by
    rintro y ⟨x, hx, rfl⟩
    have hx0 : (0 : ℝ) < x := by simp [Set.Ioi] using Bs hx
    simp [Set.mem_compl_singleton_iff] using (ne_of_gt (Real.rpow_pos_of_pos hx0
  have hg : ContinuousOn (fun y : ℝ → 1 / y) ((fun x : ℝ → x ^ r) '' spectrum ℝ B)
  have hg' : ContinuousOn (fun y : ℝ → y⁻¹) ({0}ᶜ : Set ℝ) := continuousOn_inv0
  simp [one_div] using (hg'.mono hs)
have hcomp :
  cfcR (fun y : ℝ → 1 / y) (cfcR (fun x : ℝ → x ^ r) B) =
    cfcR (fun x : ℝ → 1 / (x ^ r)) B := by
  dsimp [cfcR]
  simp [Function.comp] using
    (cfc_comp' (R := ℝ) (A := []) (p := IsSelfAdjoint) (g := fun y : ℝ → 1 / y)
      (f := fun x : ℝ → x ^ r) (a := B) (hg := hg) (hf := hcontB) (ha := hB)).sy
have hL :
  cfcR (fun x : ℝ → 1 / x) Br =
    cfcR (fun y : ℝ → y⁻¹) (cfcR (fun x : ℝ → x ^ r) B) := by
  simp [Br, one_div]
  rw [hL]
have hcomp' :
  cfcR (fun y : ℝ → y⁻¹) (cfcR (fun x : ℝ → x ^ r) B) =
    cfcR (fun x : ℝ → (x ^ r)⁻¹) B := by
  simp [one_div] using hcomp
  rw [hcomp']
  dsimp [cfcR]
  apply cfc_congr
  intro x hx
  have hx0 : (0 : ℝ) < x := by simp [Set.Ioi] using Bs hx
  calc

```

```

(x ^ r)-1 = x ^ (-r) := by
  simpa using (Real.rpow_neg (le_of_lt hx0) r).symm
_ = x ^ p := by simp [r]
have hcompC :
  cfcR (fun x : ℝ ↦ 1 / x) Cr = cfcR (fun x : ℝ ↦ x ^ p) C := by
have hcontC : ContinuousOn (fun x : ℝ ↦ x ^ r) (spectrum ℝ C) := by
  intro x hx
  have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using Cs hx
  exact (Real.continuousAt_rpow_const x r (0r.inl (ne_of_gt hx0))).cont
have hs : (fun x : ℝ ↦ x ^ r) '' spectrum ℝ C ⊆ ({0}c : Set ℝ) := by
  rintro y ⟨x, hx, rfl⟩
  have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using Cs hx
  simpa [Set.mem_compl_singleton_iff] using (ne_of_gt (Real.rpow_pos_of
have hg : ContinuousOn (fun y : ℝ ↦ 1 / y) ((fun x : ℝ ↦ x ^ r) '' spec
have hg' : ContinuousOn (fun y : ℝ ↦ y-1) ({0}c : Set ℝ) := continuous
  simpa [one_div] using (hg'.mono hs)
have hcomp :
  cfcR (fun y : ℝ ↦ 1 / y) (cfcR (fun x : ℝ ↦ x ^ r) C) =
  cfcR (fun x : ℝ ↦ 1 / (x ^ r)) C := by
  dsimp [cfcR]
  simpa [Function.comp] using
    (cfc_comp' (R := ℝ) (A := []) (p := IsSelfAdjoint) (g := fun y : ℝ ↦
      (f := fun x : ℝ ↦ x ^ r) (a := C) (hg := hg) (hf := hcontC) (ha :
have hL :
  cfcR (fun x : ℝ ↦ 1 / x) Cr =
  cfcR (fun y : ℝ ↦ y-1) (cfcR (fun x : ℝ ↦ x ^ r) C) := by
  simp [Cr, one_div]
rw [hL]
have hcomp' :
  cfcR (fun y : ℝ ↦ y-1) (cfcR (fun x : ℝ ↦ x ^ r) C) =
  cfcR (fun x : ℝ ↦ (x ^ r)-1) C := by
  simpa [one_div] using hcomp
rw [hcomp']
dsimp [cfcR]
apply cfc_congr
intro x hx
have hx0 : (0 : ℝ) < x := by simpa [Set.Ioi] using Cs hx
calc
  (x ^ r)-1 = x ^ (-r) := by
    simpa using (Real.rpow_neg (le_of_lt hx0) r).symm
  _ = x ^ p := by simp [r]
-- finish
have hmain :
  cfcR (fun x : ℝ ↦ x ^ p) C
  ≤ (1 - t) • cfcR (fun x : ℝ ↦ x ^ p) A
  + t • cfcR (fun x : ℝ ↦ x ^ p) B := by

```

1.122.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWNERHEINZCORE.LEAN

```
have hchain :
  cfcR (fun x : ℝ → 1 / x) Cr
    ≤ (1 - t) • cfcR (fun x : ℝ → 1 / x) Ar
      + t • cfcR (fun x : ℝ → 1 / x) Br :=
  le_trans h_inv1 h_inv2
-- convert via `hcomp*`
have hcompA' : cfcR (fun x : ℝ → x⁻¹) Ar = cfcR (fun x : ℝ → x ^ p) A := by
  simpa [one_div] using hcompA
have hcompB' : cfcR (fun x : ℝ → x⁻¹) Br = cfcR (fun x : ℝ → x ^ p) B := by
  simpa [one_div] using hcompB
have hcompC' : cfcR (fun x : ℝ → x⁻¹) Cr = cfcR (fun x : ℝ → x ^ p) C := by
  simpa [one_div] using hcompC
have hchain' :
  cfcR (fun x : ℝ → x⁻¹) Cr
    ≤ (1 - t) • cfcR (fun x : ℝ → x⁻¹) Ar
      + t • cfcR (fun x : ℝ → x⁻¹) Br := by
  simpa [one_div] using hchain
  simpa [hcompA', hcompB', hcompC'] using hchain'
simpa [C] using hmain

end Spectrum

namespace Spectral

variable {α : Type u} [CStarAlgebra α]
variable [Nontrivial α]

section

-- Confine `spectralOrder` to this wrapper namespace.
-- We use local instances so the order change does not leak outside.
noncomputable local instance : PartialOrder α := CStarAlgebra.spectralOrder α
noncomputable local instance : StarOrderedRing α := CStarAlgebra.spectralOrderedRing α
noncomputable local instance : NonnegSpectrumClass ℝ α := inferInstance

-- Wrappers: expose the main theorems under spectral order without duplicating proof
omit [Nontrivial α] in
theorem one_div_operatorAntitoneOn_Ioi :
  OperatorAntitoneOn (α := α) (Set.Ioi (0 : ℝ)) (fun x : ℝ → 1 / x) := by
  simpa using (LownerHeinzCore.one_div_operatorAntitoneOn_Ioi (α := α))

theorem one_div_operatorConvexOn_Ioi :
  OperatorConvexOn (α := α) (Set.Ioi (0 : ℝ)) (fun x : ℝ → 1 / x) := by
  simpa using (LownerHeinzCore.one_div_operatorConvexOn_Ioi (α := α))
```

```

omit [Nontrivial  $\square$ ] in
theorem one_div_add_t_operatorAntitone0n_Ici :  $\forall (t : \mathbb{R}), 0 < t \rightarrow$ 
  OperatorAntitone0n ( $\square := \square$ ) (Set.Ici ( $0 : \mathbb{R}$ )) (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ )
  intro t ht
  simpa using (LowerHeinzCore.one_div_add_t_operatorAntitone0n_Ici ( $\square := \square$ ))

theorem one_div_add_t_operatorConvex0n_Ici :  $\forall (t : \mathbb{R}), 0 < t \rightarrow$ 
  OperatorConvex0n ( $\square := \square$ ) (Set.Ici ( $0 : \mathbb{R}$ )) (fun x :  $\mathbb{R} \mapsto 1 / (x + t)$ ) :
  intro t ht
  simpa using (LowerHeinzCore.one_div_add_t_operatorConvex0n_Ici ( $\square := \square$ ))

omit [Nontrivial  $\square$ ] in
theorem ratio_add_t_operatorMonotone0n_Ici :  $\forall (t : \mathbb{R}), 0 < t \rightarrow$ 
  OperatorMonotone0n ( $\square := \square$ ) (Set.Ici ( $0 : \mathbb{R}$ )) (fun x :  $\mathbb{R} \mapsto x / (x + t)$ )
  intro t ht
  simpa using (LowerHeinzCore.ratio_add_t_operatorMonotone0n_Ici ( $\square := \square$ ))

theorem ratio_add_t_operatorConcave0n_Ici :  $\forall (t : \mathbb{R}), 0 < t \rightarrow$ 
  OperatorConcave0n ( $\square := \square$ ) (Set.Ici ( $0 : \mathbb{R}$ )) (fun x :  $\mathbb{R} \mapsto x / (x + t)$ )
  intro t ht
  simpa using (LowerHeinzCore.ratio_add_t_operatorConcave0n_Ici ( $\square := \square$ ))

omit [Nontrivial  $\square$ ] in
theorem power_Icc_zero_one_operatorMonotone0n_Ici :  $\forall p \in \text{Set.Icc } (0 : \mathbb{R}) 1$ 
  OperatorMonotone0n ( $\square := \square$ ) (Set.Ici ( $0 : \mathbb{R}$ )) (fun x  $\mapsto x^p$ ) := by
  intro p hp
  simpa using (LowerHeinzCore.power_Icc_zero_one_operatorMonotone0n_Ici ( $\square$ ))

theorem power_Icc_zero_one_operatorConcave0n_Ici :  $\forall p \in \text{Set.Icc } (0 : \mathbb{R}) 1$ ,
  OperatorConcave0n ( $\square := \square$ ) (Set.Ici ( $0 : \mathbb{R}$ )) (fun x  $\mapsto x^p$ ) := by
  intro p hp
  simpa using (LowerHeinzCore.power_Icc_zero_one_operatorConcave0n_Ici ( $\square$ ))

theorem power_Icc_one_two_operatorConvex0n_Ici :  $\forall p \in \text{Set.Icc } (1 : \mathbb{R}) 2$ ,
  OperatorConvex0n ( $\square := \square$ ) (Set.Ici ( $0 : \mathbb{R}$ )) (fun x  $\mapsto x^p$ ) := by
  intro p hp
  simpa using (LowerHeinzCore.power_Icc_one_two_operatorConvex0n_Ici ( $\square := \square$ ))

omit [Nontrivial  $\square$ ] in
theorem power_Icc_neg_one_zero_neg_operatorMonotone0n_Ioi :  $\forall p \in \text{Set.Icc } (1 : \mathbb{R}) 0$ ,
  OperatorMonotone0n ( $\square := \square$ ) (Set.Ioi ( $0 : \mathbb{R}$ )) (fun x  $\mapsto -$ 
  (x ^ p)) := by
  intro p hp
  simpa using (LowerHeinzCore.power_Icc_neg_one_zero_neg_operatorMonotone0n_Ioi ( $\square := \square$ ))

```

1.123.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWNERHEINZTHEOREM.LEAN

```
theorem power_Icc_neg_one_zero_neg_operatorConcave0n_Ioi :  $\forall p \in \text{Set.Icc} (-1 : \mathbb{R}) \ 0,$ 
```

```
  OperatorConcave0n ( $\square := \square$ ) (Set.Ioi ( $0 : \mathbb{R}$ )) (fun x  $\mapsto -$ 
(x ^ p)) := by
```

```
  intro p hp
```

```
  simp using (LownerHeinzCore.power_Icc_neg_one_zero_neg_operatorConcave0n_Ioi ( $\square :$ 
```

```
end
```

```
end Spectral
```

```
end LownerHeinzCore
```

1.123 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LownerHeinzTheorem

```
/-
```

Copyright (c) 2026 Hayata Yamasaki. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Kei Tsukamoto, Kento Mori, Hayata Yamasaki

```
-/
```

```
module
```

```
public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.LownerHeinzCore
```

```
public import Mathlib.Analysis.CStarAlgebra.ContinuousLinearMap
```

```
public import Mathlib.Analysis.Convex.Continuous
```

```
public import Mathlib.Analysis.InnerProductSpace.StarOrder
```

```
@[expose] public section
```

```
/-!
```

```
## Wrapper  $\square$  B( $\square$ )  $\square$ 
```

```
 $\square\square\square\square\square$  `LownerHeinzCore`  $\square\square\square\square$  `L  $\square := \square \rightarrow L[\mathbb{C}]$   $\square$   $\square\square\square\square\square\square\square\square$ 
```

```
** $\square\square\square\square\square\square\square\square\square$  wrapper**  $\square\square$ 
```

```
-  $\square\square$  `simp using`  $\square\square$  Core  $\square\square\square\square\square\square\square\square\square\square\square\square$ 
```

```
- `B( $\square$ )`  $\square\square\square\square$  Loewner order  $\square$  ecosystem  $\square\square\square\square$  `spectralOrder`  $\square\square\square\square\square$ 
```

```
   $\square$  `spectralOrder`  $\square\square\square\square\square$  `LownerHeinzCore.Spectral`  $\square\square\square$ 
```

```
-/
```

```
namespace LownerHeinzTheorem
```

```
universe u v
```

open CFC

```
abbrev L (α : Type u) [NormedAddCommGroup α] [InnerProductSpace ℂ α] : Type
  α → L[ℂ] α
```

```
variable {α : Type u} [NormedAddCommGroup α] [InnerProductSpace ℂ α] [Comple
variable [Nontrivial α]
```

```
noncomputable instance instNontrivialL : Nontrivial (L α) := inferInstance
```

```
set_option synthInstance.maxHeartbeats 40000 in
noncomputable local instance : NonnegSpectrumClass ℝ (L α) := inferInstance
```

```
set_option synthInstance.maxHeartbeats 80000 in
noncomputable instance instCFCRealSelfAdjoint :
  ContinuousFunctionalCalculus ℝ (L α) IsSelfAdjoint := inferInstance
```

```
noncomputable abbrev cfcR (f : ℝ → ℝ) (A : L α) : L α :=
  LownerHeinzCore.cfcR (α := L α) f A
```

```
/-- Fixed-space operator monotonicity on the Hilbert space `α`. -
/
```

```
def OperatorMonotone (f : ℝ → ℝ) : Prop :=
  LownerHeinzCore.OperatorMonotone (α := L α) f
```

```
/-- Fixed-space operator monotonicity on `s` for the Hilbert space `α`. -
/
```

```
def OperatorMonotoneOn (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  LownerHeinzCore.OperatorMonotoneOn (α := L α) s f
```

```
/-- Fixed-space operator antitonicity on the Hilbert space `α`. -
/
```

```
def OperatorAntitone (f : ℝ → ℝ) : Prop :=
  LownerHeinzCore.OperatorAntitone (α := L α) f
```

```
/-- Fixed-space operator antitonicity on `s` for the Hilbert space `α`. -
/
```

```
def OperatorAntitoneOn (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  LownerHeinzCore.OperatorAntitoneOn (α := L α) s f
```

```
/-- Fixed-space operator convexity on the Hilbert space `α`. -
/
```

```
def OperatorConvex (f : ℝ → ℝ) : Prop :=
  LownerHeinzCore.OperatorConvex (α := L α) f
```


1.123.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWNERHEINZTHEOREM.LEAN

/-- Fixed-space operator convexity on `s` for the Hilbert space `H`. -
/

def OperatorConvexOn (s : Set $\mathbb{R}$) (f : $\mathbb{R} \rightarrow \mathbb{R}$) : Prop :=
 LownerHeinzCore.OperatorConvexOn ($\lambda := L \lambda$) s f

/-- Fixed-space operator concavity on the Hilbert space `H`. -
/

def OperatorConcave (f : $\mathbb{R} \rightarrow \mathbb{R}$) : Prop :=
 LownerHeinzCore.OperatorConcave ($\lambda := L \lambda$) f

/-- Fixed-space operator concavity on `s` for the Hilbert space `H`. -
/

def OperatorConcaveOn (s : Set $\mathbb{R}$) (f : $\mathbb{R} \rightarrow \mathbb{R}$) : Prop :=
 LownerHeinzCore.OperatorConcaveOn ($\lambda := L \lambda$) s f

omit λ [NormedAddCommGroup λ] [InnerProductSpace $\mathbb{C} \lambda$] [CompleteSpace λ] [Nontrivial λ]
/-- Uniform operator monotonicity over all Hilbert spaces in universe `u`. -
/

def OperatorMonotoneAll (f : $\mathbb{R} \rightarrow \mathbb{R}$) : Prop :=
 $\forall \{K : \text{Type } u\}$
 [NormedAddCommGroup K] [InnerProductSpace $\mathbb{C} K$] [CompleteSpace K]
 [Nontrivial K],
 OperatorMonotone ($\lambda := K$) f

omit λ [NormedAddCommGroup λ] [InnerProductSpace $\mathbb{C} \lambda$] [CompleteSpace λ] [Nontrivial λ]
/-- Uniform operator monotonicity on `s` over all Hilbert spaces in universe `u`. -
/

def OperatorMonotoneOnAll (s : Set $\mathbb{R}$) (f : $\mathbb{R} \rightarrow \mathbb{R}$) : Prop :=
 $\forall \{K : \text{Type } u\}$
 [NormedAddCommGroup K] [InnerProductSpace $\mathbb{C} K$] [CompleteSpace K]
 [Nontrivial K],
 OperatorMonotoneOn ($\lambda := K$) s f

omit λ [NormedAddCommGroup λ] [InnerProductSpace $\mathbb{C} \lambda$] [CompleteSpace λ] [Nontrivial λ]
/-- Uniform operator antitonicity over all Hilbert spaces in universe `u`. -
/

def OperatorAntitoneAll (f : $\mathbb{R} \rightarrow \mathbb{R}$) : Prop :=
 $\forall \{K : \text{Type } u\}$
 [NormedAddCommGroup K] [InnerProductSpace $\mathbb{C} K$] [CompleteSpace K]
 [Nontrivial K],
 OperatorAntitone ($\lambda := K$) f

omit λ [NormedAddCommGroup λ] [InnerProductSpace $\mathbb{C} \lambda$] [CompleteSpace λ] [Nontrivial λ]
/-- Uniform operator antitonicity on `s` over all Hilbert spaces in universe `u`. -
/

def OperatorAntitoneOnAll (s : Set $\mathbb{R}$) (f : $\mathbb{R} \rightarrow \mathbb{R}$) : Prop :=

```

  ∀ {K : Type u}
    [NormedAddCommGroup K] [InnerProductSpace ℂ K] [CompleteSpace K]
    [Nontrivial K],
    OperatorAntitoneOn (· := K) s f

omit [] [NormedAddCommGroup []] [InnerProductSpace ℂ []] [CompleteSpace []] [No
/-- Uniform operator convexity over all Hilbert spaces in universe `u`. -
/
def OperatorConvexAll (f : ℝ → ℝ) : Prop :=
  ∀ {K : Type u}
    [NormedAddCommGroup K] [InnerProductSpace ℂ K] [CompleteSpace K]
    [Nontrivial K],
    OperatorConvex (· := K) f

omit [] [NormedAddCommGroup []] [InnerProductSpace ℂ []] [CompleteSpace []] [No
/-- Uniform operator convexity on `s` over all Hilbert spaces in universe `
/
def OperatorConvexOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  ∀ {K : Type u}
    [NormedAddCommGroup K] [InnerProductSpace ℂ K] [CompleteSpace K]
    [Nontrivial K],
    OperatorConvexOn (· := K) s f

omit [] [NormedAddCommGroup []] [InnerProductSpace ℂ []] [CompleteSpace []] [No
/-- Uniform operator concavity over all Hilbert spaces in universe `u`. -
/
def OperatorConcaveAll (f : ℝ → ℝ) : Prop :=
  ∀ {K : Type u}
    [NormedAddCommGroup K] [InnerProductSpace ℂ K] [CompleteSpace K]
    [Nontrivial K],
    OperatorConcave (· := K) f

omit [] [NormedAddCommGroup []] [InnerProductSpace ℂ []] [CompleteSpace []] [No
/-- Uniform operator concavity on `s` over all Hilbert spaces in universe `
/
def OperatorConcaveOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop :=
  ∀ {K : Type u}
    [NormedAddCommGroup K] [InnerProductSpace ℂ K] [CompleteSpace K]
    [Nontrivial K],
    OperatorConcaveOn (· := K) s f

theorem operatorConvex_convexOn_univ {f : ℝ → ℝ} (hf : OperatorConvex (· :=
  ConvexOn ℝ Set.univ f := by
  refine ⟨convex_univ, ?_⟩
  intro x _ y _ a b ha hb hab
  have hb1 : b ≤ 1 := by

```

1.123.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/LOWNERHEINZTHEOREM.LEAN

```

linarith
have halg_combo (u v r s : ℝ) :
  u • (algebraMap ℝ (L []) r) + v • (algebraMap ℝ (L []) s) =
    algebraMap ℝ (L []) (u * r + v * s) := by
calc
  u • (algebraMap ℝ (L []) r) + v • (algebraMap ℝ (L []) s) =
    (algebraMap ℝ (L []) u) * (algebraMap ℝ (L []) r) +
    (algebraMap ℝ (L []) v) * (algebraMap ℝ (L []) s) := by
    rw [Algebra.smul_def, Algebra.smul_def]
  _ = algebraMap ℝ (L []) (u * r) + algebraMap ℝ (L []) (v * s) := by
    simp
  _ = algebraMap ℝ (L []) (u * r + v * s) := by
    simp
have hop :
  cfcR (□ := □) f
    ((1 - b) • (algebraMap ℝ (L []) x) + b • (algebraMap ℝ (L []) y)) ≤
    (1 - b) • cfcR (□ := □) f (algebraMap ℝ (L []) x) +
    b • cfcR (□ := □) f (algebraMap ℝ (L []) y) :=
  hf (A := algebraMap ℝ (L []) x) (B := algebraMap ℝ (L []) y) (t := b) hb hb1
have hx :
  cfcR (□ := □) f (algebraMap ℝ (L []) x) = algebraMap ℝ (L []) (f x) := by
  simp [cfcR, LownerHeinzCore.cfcR]
have hy :
  cfcR (□ := □) f (algebraMap ℝ (L []) y) = algebraMap ℝ (L []) (f y) := by
  simp [cfcR, LownerHeinzCore.cfcR]
have hxy :
  cfcR (□ := □) f (algebraMap ℝ (L []) ((1 - b) * x + b * y)) =
    algebraMap ℝ (L []) (f ((1 - b) * x + b * y)) := by
  simp [cfcR, LownerHeinzCore.cfcR] using
    cfc_algebraMap (A := L []) ((1 - b) * x + b * y) f
  rw [halg_combo (u := 1 - b) (v := b) (r := x) (s := y), hxy, hx, hy,
    halg_combo (u := 1 - b) (v := b) (r := f x) (s := f y)] at hop
have hscalar :
  algebraMap ℝ (L []) (f ((1 - b) * x + b * y)) ≤
    algebraMap ℝ (L []) ((1 - b) * f x + b * f y) := by
  simp [Algebra.smul_def] using hop
have hspec_le :
  ∀ z ∈ spectrum ℝ (algebraMap ℝ (L []) (f ((1 - b) * x + b * y))),
    z ≤ (1 - b) * f x + b * f y := by
  simp using
    (le_algebraMap_iff_spectrum_le
      (R := ℝ) (A := L [])
      (a := algebraMap ℝ (L []) (f ((1 - b) * x + b * y)))
      (r := (1 - b) * f x + b * f y)
      (ha := cfc_predicate_algebraMap (A := L []) (f ((1 - b) * x + b * y))))).1 hsc
have ha' : a = 1 - b := by

```

```

linarith
have hscalar' : f ((1 - b) * x + b * y) ≤ (1 - b) * f x + b * f y := by
  obtain ⟨z, hz⟩ :=
    ContinuousFunctionalCalculus.spectrum_nonempty (R := ℝ)
      (algebraMap ℝ (L []) (f ((1 - b) * x + b * y)))
      (cfc_predicate_algebraMap (A := L []) (f ((1 - b) * x + b * y)))
  have hz_eq : z = f ((1 - b) * x + b * y) := by
    have hz_single :=
      CFC.spectrum_algebraMap_subset (R := ℝ) (A := L []) (f ((1 - b) * x
    simp using hz_single
  simp [hz_eq] using hspec_le z hz
  simp [smul_eq_mul, ha'] using hscalar'

theorem operatorConvex_continuousOn_univ {f : ℝ → ℝ} (hf : OperatorConvex (
  ContinuousOn f Set.univ :=
  ConvexOn.continuousOn isOpen_univ (operatorConvex_convexOn_univ ([] := []))

theorem operatorConvex_continuousOn_spectrum_union {f : ℝ → ℝ}
  (hf : OperatorConvex ([] := []) f) (A B : L []) :
  ContinuousOn f (spectrum ℝ A ∪ spectrum ℝ B) :=
  (operatorConvex_continuousOn_univ ([] := []) hf).mono (by intro x hx; simp)

omit [Nontrivial []] in
theorem one_div_operatorAntitoneOn_Ioi :
  OperatorAntitoneOn ([] := []) (Set.Ioi (0 : ℝ)) (fun x : ℝ ↦ 1 / x) := by
  simp using (LowerHeinzCore.one_div_operatorAntitoneOn_Ioi ([] := L []))

theorem one_div_operatorConvexOn_Ioi :
  OperatorConvexOn ([] := []) (Set.Ioi (0 : ℝ)) (fun x : ℝ ↦ 1 / x) := by
  simp using (LowerHeinzCore.one_div_operatorConvexOn_Ioi ([] := L []))

omit [Nontrivial []] in
theorem one_div_add_t_operatorAntitoneOn_Ici : ∀ (t : ℝ), 0 < t →
  OperatorAntitoneOn ([] := []) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ 1 / (x + t))
  intro t ht
  simp using (LowerHeinzCore.one_div_add_t_operatorAntitoneOn_Ici ([] := L

theorem one_div_add_t_operatorConvexOn_Ici : ∀ (t : ℝ), 0 < t →
  OperatorConvexOn ([] := []) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ 1 / (x + t)) :
  intro t ht
  simp using (LowerHeinzCore.one_div_add_t_operatorConvexOn_Ici ([] := L []

omit [Nontrivial []] in
theorem ratio_add_t_operatorMonotoneOn_Ici : ∀ (t : ℝ), 0 < t →
  OperatorMonotoneOn ([] := []) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ x / (x + t))

```

1.124.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/OPERATORGEOMETRICMEAN.LEAN

```

    intro t ht
    simp using (LowerHeinzCore.ratio_add_t_operatorMonotone0n_Ici (□ := L □) t ht)

theorem ratio_add_t_operatorConcave0n_Ici : ∀ (t : ℝ), 0 < t →
  OperatorConcave0n (□ := □) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ x / (x + t)) := by
  intro t ht
  simp using (LowerHeinzCore.ratio_add_t_operatorConcave0n_Ici (□ := L □) t ht)

omit [Nontrivial □] in
theorem power_Icc_zero_one_operatorMonotone0n_Ici : ∀ p ∈ Set.Icc (0 : ℝ) 1,
  OperatorMonotone0n (□ := □) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p) := by
  intro p hp
  simp using (LowerHeinzCore.power_Icc_zero_one_operatorMonotone0n_Ici (□ := L □) p)

theorem power_Icc_zero_one_operatorConcave0n_Ici : ∀ p ∈ Set.Icc (0 : ℝ) 1,
  OperatorConcave0n (□ := □) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p) := by
  intro p hp
  simp using (LowerHeinzCore.power_Icc_zero_one_operatorConcave0n_Ici (□ := L □) p)

theorem power_Icc_one_two_operatorConvex0n_Ici : ∀ p ∈ Set.Icc (1 : ℝ) 2,
  OperatorConvex0n (□ := □) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p) := by
  intro p hp
  simp using (LowerHeinzCore.power_Icc_one_two_operatorConvex0n_Ici (□ := L □) p)

omit [Nontrivial □] in
theorem power_Icc_neg_one_zero_neg_operatorMonotone0n_Ioi : ∀ p ∈ Set.Icc (-
1 : ℝ) 0,
  OperatorMonotone0n (□ := □) (Set.Ioi (0 : ℝ)) (fun x ↦ -
(x ^ p)) := by
  intro p hp
  simp using (LowerHeinzCore.power_Icc_neg_one_zero_neg_operatorMonotone0n_Ioi (□ := L □) p)

theorem power_Icc_neg_one_zero_neg_operatorConcave0n_Ioi : ∀ p ∈ Set.Icc (-
1 : ℝ) 0,
  OperatorConcave0n (□ := □) (Set.Ioi (0 : ℝ)) (fun x ↦ -
(x ^ p)) := by
  intro p hp
  simp using (LowerHeinzCore.power_Icc_neg_one_zero_neg_operatorConcave0n_Ioi (□ := L □) p)

end LowerHeinzTheorem

```

1.124 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/OperatorGeome

/-

```

Copyright (c) 2026 Hayata Yamasaki. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Kei Tsukamoto, Kento Mori, Hayata Yamasaki
-/
module

public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.Generalized

@[expose] public section

namespace OperatorGeometricMean

universe u

open LowerHeinzTheorem
open GeneralizedPerspectiveFunction

variable {α : Type u}
variable [NormedAddCommGroup α] [InnerProductSpace ℂ α] [CompleteSpace α]
variable [Nontrivial α]

/-- The operator  $(\alpha, \beta)$ -power mean, realized as a generalized perspective /
/
noncomputable def operatorPowerMean (α β : ℝ) (A B : L α) : L α :=
  GeneralizedPerspective (fun x : ℝ ↦ x ^ α) (fun x : ℝ ↦ x ^ β) A B

private lemma rpow_continuousOn_Ici (p : ℝ) (hp : 0 ≤ p) :
  ContinuousOn (fun x : ℝ ↦ x ^ p) (Set.Ici (0 : ℝ)) := by
  intro x hx
  exact (Real.continuousAt_rpow_const x p (0r.inr hp)).continuousWithinAt

omit [Nontrivial α] in
private lemma operatorConcaveOn_Ioi_of_Ici {f : ℝ → ℝ}
  (h : OperatorConcaveOn (α := α) (Set.Ici (0 : ℝ)) f) :
  OperatorConcaveOn (α := α) (Set.Ioi (0 : ℝ)) f := by
  intro A B t hA hB ht0 ht1 hAs hBs
  exact h hA hB ht0 ht1 (Set.Subset.trans hAs Set.Ioi_subset_Ici_self)
  (Set.Subset.trans hBs Set.Ioi_subset_Ici_self)

omit [Nontrivial α] in
private lemma pdSet_subset_psdSet : pdSet (α := α) ⊆ psdSet (α := α) := by
  intro A hA
  rcases hA with ⟨hA_sa, hA_spec⟩
  exact ⟨hA_sa, Set.Subset.trans hA_spec Set.Ioi_subset_Ici_self⟩

private lemma rpow_pos_on_Ioi (p : ℝ) :
```

1.124.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/OPERATORGEOMETRICMEAN.LEAN

```

    ∀ x ∈ Set.Ioi (0 : ℝ), 0 < x ^ p := by
    intro x hx
    exact Real.rpow_pos_of_pos hx p

```

/--

Theorem 1.1, concave range: the operator (α, β) -power mean is jointly concave on strictly positive operators for $0 \leq \alpha, \beta \leq 1$.

-/

theorem operatorPowerMean_jointlyConcave0n_pdSet

```

    {α β : ℝ}
    (hα : α ∈ Set.Icc (0 : ℝ) 1)
    (hβ : β ∈ Set.Icc (0 : ℝ) 1) :
    JointlyConcave0n (pdSet (· := ·)) (pdSet (· := ·))
    (operatorPowerMean (· := ·) α β) := by
    intro A1 A2 B1 B2 θ hA1 hA2 hB1 hB2 hθ0 hθ1
    have hconc :=
    theorem_2_6_forward_jointlyConcave0n_psd_pd_Ici
    (· := ·)
    (f := fun x : ℝ ↦ x ^ α)
    (h := fun x : ℝ ↦ x ^ β)
    (hfconc := by
    intro K _ _ _
    exact power_Icc_zero_one_operatorConcave0n_Ici (· := K) α hα)
    (hfcont := rpow_continuous0n_Ici α hα.1)
    (hfθ := by
    exact Real.rpow_nonneg (show (0 : ℝ) ≤ 0 by simp) α)
    (hconc := by
    exact operatorConcave0n_Ioi_of_Ici (· := ·)
    (power_Icc_zero_one_operatorConcave0n_Ici (· := ·) β hβ))
    (hcont := by
    intro x hx
    exact (Real.continuousAt_rpow_const x β (0r.inl (ne_of_gt hx))).continuousWi
    (hpos := rpow_pos_on_Ioi β)
    simp [operatorPowerMean] using
    hconc (A1 := A1) (A2 := A2) (B1 := B1) (B2 := B2) (θ := θ)
    (pdSet_subset_psdSet (· := ·) hA1) (pdSet_subset_psdSet (· := ·) hA2)
    hB1 hB2 hθ0 hθ1

```

/--

Theorem 1.1, convex range: the operator (α, β) -power mean is jointly convex on strictly positive operators for $1 \leq \alpha \leq 2$ and $0 \leq \beta \leq 1$.

-/

theorem operatorPowerMean_jointlyConvex0n_pdSet

```

    {α β : ℝ}
    (hα : α ∈ Set.Icc (1 : ℝ) 2)
    (hβ : β ∈ Set.Icc (0 : ℝ) 1) :

```

```

JointlyConvexOn (pdSet (□ := □)) (pdSet (□ := □))
  (operatorPowerMean (□ := □) α β) := by
intro A₁ A₂ B₁ B₂ θ hA₁ hA₂ hB₁ hB₂ hθ₀ hθ₁
have hconv :=
  theorem_2_5_forward_jointlyConvexOn_psd_pd_Ici
    (□ := □)
    (f := fun x : ℝ ↦ x ^ α)
    (h := fun x : ℝ ↦ x ^ β)
    (hf := by
      refine {?, ?, ?}
      · intro K _ _ _
        exact power_Icc_one_two_operatorConvexOn_Ici (□ := K) α hα
      · exact rpow_continuousOn_Ici α (by linarith [hα.1])
      · have hα₀ : α ≠ 0 := by linarith [hα.1]
        simp [Real.zero_rpow hα₀]
    )
  (hconc := by
    exact operatorConcaveOn_Ioi_of_Ici (□ := □)
      (power_Icc_zero_one_operatorConcaveOn_Ici (□ := □) β hβ))
  (hcont := by
    intro x hx
    exact (Real.continuousAt_rpow_const x β (0r.inl (ne_of_gt hx))).con
  )
  (hpos := rpow_pos_on_Ioi β)
simp [operatorPowerMean] using
  hconv (A₁ := A₁) (A₂ := A₂) (B₁ := B₁) (B₂ := B₂) (θ := θ)
  (pdSet_subset_psdSet (□ := □) hA₁) (pdSet_subset_psdSet (□ := □) hA₂)
  hB₁ hB₂ hθ₀ hθ₁

end OperatorGeometricMean

```

1.125 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/PORTIN

When porting the code from <https://github.com/Hayata-Yamasaki-Group/lean-quantum> to Lean-QuantumInfo, besides moving the code to a different directory (and thus editing the import files), a few changes were made:

- * Some of the `Nontrivial` and `CompleteSpace` instance variables were moved or to prevent the linter warnings about unneeded variables. This was done in
 - BlockDiagonal.lean
 - GeneralizedPerspectiveFunction.lean
 - HilbertSchmidtOperatorSpace.lean
 - JensenOperatorInequality.lean
 - JensenOperatorInequalityIImpIV.lean
 - JensenOperatorInequalityIVtoV.lean

- LiebAndoTrace.Lean
- * Code was moved to the "module system", adding ``module`/`public import`/`@[expose]` of each file.
- * Some whitespace was modified to make lines fit without the longLine linter complaint

The project this code is graciously borrowed from is the work of the following:

Kazumi Kasaura (OMRON SINIC X, RIKEN)
 Kei Tsukamoto (The University of Tokyo)
 Kento Mori (MOLSIIS)
 Lisa Mizuno (The University of Tokyo)
 Takahiro Namatame (Kyoto University)
 Yuta Oriike (CyberAgent)
 Masaya Taniguchi (RIKEN)
 Sho Sonoda (RIKEN, CyberAgent)
 Hayata Yamasaki (The University of Tokyo)

1.126 QuantumInfo/ForMathlib/HermitianMat.lean

```
public import QuantumInfo.ForMathlib.HermitianMat.LiebConcavity
public import QuantumInfo.ForMathlib.HermitianMat.Peierls
```

1.127 QuantumInfo/ForMathlib/HermitianMat/Basic.lean

```
intro h rfl; grind
```

1.128 QuantumInfo/ForMathlib/HermitianMat/CFC.lean

```
theorem cfc_monoOn_pos_of_monoOn_posDef {d : Type*} [Fintype d] [DecidableEq d]
  MonotoneOn (HermitianMat.cfc · f) { A : HermitianMat d ℂ | A.mat.PosDef } := by
  exact False.elim hf_is_operator_convex
```

1.129 QuantumInfo/ForMathlib/HermitianMat/CompoundMatrix.lean

```
/-
Copyright (c) 2026 Dennj Osele. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Dennj Osele
-/
module
```

```

public import QuantumInfo.ForMathlib.Majorization
public import QuantumInfo.ForMathlib.HermitianMat.Order

/-! # Compound matrices on `HermitianMat`

This file lifts the compound-matrix construction from
`QuantumInfo.ForMathlib.Majorization` to `HermitianMat`. The compound matrix
a Hermitian matrix is again Hermitian, and inherits the natural spectral and
positivity properties of the original.
-/

@[expose] public section

open scoped AllOrdered

namespace HermitianMat

variable {d : Type*} [Fintype d] [DecidableEq d]

/-- The `k`-th compound matrix of a Hermitian matrix, packaged as a
`HermitianMat`. -/
noncomputable def compoundHermitian (A : HermitianMat d ℂ) (k : ℕ) :
  HermitianMat {S : Finset d // S.card = k} ℂ :=
  (compoundMatrix A.mat k, (compoundMatrix_conjTranspose A.mat k).symm.trans
    congrArg (compoundMatrix · k) A.H)

/-- The eigenvalues of `compoundHermitian A k` are the products of eigenvalues
of `A` over `k`-subsets, up to an index permutation. -/
lemma compoundHermitian_eigenvalues (A : HermitianMat d ℂ) (k : ℕ) :
  ∃ σ : {S : Finset d // S.card = k} ≈ {S : Finset d // S.card = k},
    (compoundHermitian A k).H.eigenvalues ∘ σ =
      fun S => ∏ i : Fin k, A.H.eigenvalues (S.1.orderEmbOffFin S.2 i) :=
    (compoundHermitian A k).H.eigenvalues_eq_of_unitary_similarity_diagonal
      (U := compoundMatrix (Matrix.IsHermitian.eigenvectorUnitary A.H) k)
      (compoundMatrix_unitary _ (by simp [Matrix.unitaryGroup]) _)
      (by simp [compoundHermitian, compoundMatrix_mul, compoundMatrix_diagonal
        Matrix.star_eq_conjTranspose, ← compoundMatrix_conjTranspose] using
        congrArg (fun x => compoundMatrix x k) (Matrix.IsHermitian.spectral_t

/-- `compoundHermitian` preserves positive semidefiniteness. -
/
lemma compoundHermitian_nonneg (A : HermitianMat d ℂ) (hA : 0 ≤ A) (k : ℕ)
  0 ≤ compoundHermitian A k := by
  rw [HermitianMat.zero_le_iff, (compoundHermitian A k).H.posSemidef_iff_ei
  obtain ⟨σ, hσ⟩ := compoundHermitian_eigenvalues A k

```

```

intro S
rw [show (compoundHermitian A k).H.eigenvalues S = _ by
  simp [Function.comp_def] using congrFun hσ (σ.symm S)]
exact Finset.prod_nonneg fun _ _ => A.eigenvalues_nonneg hA _

/-- `compoundHermitian` distributes over `HermitianMat.conj`: the compound of
`A.conj B` equals the conjugation of `compoundHermitian A k` by `compoundMatrix B k`
/
lemma compoundHermitian_conj (A : HermitianMat d ℂ) (B : Matrix d d ℂ) (k : ℕ) :
  compoundHermitian (A.conj B) k =
    (compoundHermitian A k).conj (compoundMatrix B k) :=
  Subtype.ext <| by simp [compoundHermitian, HermitianMat.conj_apply_mat, compoundMa
    ← compoundMatrix_conjTranspose, Matrix.mul_assoc]

end HermitianMat

```

1.130 QuantumInfo/ForMathlib/HermitianMat/Jordan.lean

```

scoped instance : NonAssocCommRing (HermitianMat d ℝ) where
  mul_comm := HermitianMat.symmMul_comm

```

1.131 QuantumInfo/ForMathlib/HermitianMat/LiebConcavity.lean

```

/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under MIT license as described in the file LICENSE.
Authors: Alex Meiburg
-/
module

public import QuantumInfo.ForMathlib.HermitianMat.Rpow
public import QuantumInfo.ForMathlib.HermitianMat.Schatten
public import QuantumInfo.ForMathlib.HayataGroup.TraceInequality.LiebAndoTrace
public import Mathlib

@[expose] public section

/-! ## Main result for DPI

We derive the concavity of the trace functional  $\sigma \mapsto \text{Tr}[(\sigma^s H \sigma^s)^p]$  from
the Lieb–Ando trace inequalities proved in `LiebAndoTrace.lean`.
-/

```

```

variable {d : Type*} [Fintype d] [DecidableEq d]

namespace HermitianMatBridge

/- Bridge lemmas: HermitianMat  $\leftrightarrow$  L (EuclideanSpace  $\mathbb{C}$  d)

We use `Matrix.toEuclideanCLM` (a  $\approx^*_a[\mathbb{C}]$ ) to bridge between `Matrix d d  $\mathbb{C}$ `
and bounded operators on `EuclideanSpace  $\mathbb{C}$  d`. This allows us to apply
the Lieb–Ando trace inequalities proved in `LiebAndoTrace.lean` to
`HermitianMat` trace functionals.
-/

open LiebAndoTrace GeneralizedPerspectiveFunction

/-- Abbreviation for the star algebra isomorphism. -/
noncomputable abbrev  $\Phi$  : Matrix d d  $\mathbb{C} \approx^*_a[\mathbb{C}]$  (EuclideanSpace  $\mathbb{C}$  d  $\rightarrow$  L[ $\mathbb{C}$ ] EuclideanSpace  $\mathbb{C}$  d)
  Matrix.toEuclideanCLM (n := d) ( $\square$  :=  $\mathbb{C}$ )

/-- ` $\Phi$ ` is continuous (as a linear map between finite-dimensional spaces).
-/
lemma  $\Phi$ _continuous : Continuous ( $\uparrow\Phi$  : Matrix d d  $\mathbb{C} \rightarrow$  _) :=
  ( $\Phi$  (d := d)).toAlgEquiv.toLinearEquiv.toLinearMap.continuous_of_finiteDim

/-- ` $\Phi$ ` maps Hermitian matrices to self-adjoint operators. -
-/
lemma  $\Phi$ _isSelfAdjoint (A : HermitianMat d  $\mathbb{C}$ ) :
  IsSelfAdjoint ( $\Phi$  A.mat) := by
  rw [isSelfAdjoint_iff,  $\leftarrow$  map_star ( $\Phi$  (d := d))]
  congr 1; exact A.conjTranspose_mat

/-
` $\Phi$ ` preserves nonneg: PSD HermitianMat maps to nonneg operators.
-/
lemma  $\Phi$ _nonneg (A : HermitianMat d  $\mathbb{C}$ ) (hA :  $0 \leq A$ ) :
  ( $0$  : EuclideanSpace  $\mathbb{C}$  d  $\rightarrow$  L[ $\mathbb{C}$ ] EuclideanSpace  $\mathbb{C}$  d)  $\leq \Phi$  A.mat := by
  refine' { .. }
  · convert  $\Phi$ _isSelfAdjoint A using 1
    simp [IsSelfAdjoint, LinearMap.IsSymmetric]
    simp [ContinuousLinearMap.ext_iff, ContinuousLinearMap.star_eq_adjoint]
    grind only [IsSelfAdjoint.adjoint_eq,  $\Phi$ _isSelfAdjoint, ContinuousLinearMap.star_eq_adjoint]
  · intro x
    have h_inner :  $\forall x : \text{EuclideanSpace } \mathbb{C} \text{ d}, 0 \leq \text{Complex.re } (\text{inner } \mathbb{C} \text{ x } (\Phi \text{ A.mat x}))$ 
    intro x
    have h_inner :  $0 \leq \text{Complex.re } (\sum i, \sum j, \text{star } (x \ i) * \text{A.mat } i \ j * x \ j)$ 
    have := hA.2
    specialize this (Finsupp.equivFunOnFinite.symm x.ofLp); simp_all [Finsupp.equivFunOnFinite.symm]

```

1.131.

QUANTUMINFO/FORMATHLIB/HERMITIANMAT/LIEBCONCAVITY.LEAN 431

```
      simp_all [Complex.le_def]
      convert h_inner using 1
      simp [inner, mul_comm]
      simp [Matrix.mulVec, dotProduct, mul_comm, Finset.sum_add_distrib]
      simp [mul_add, mul_sub, Finset.mul_sum _ _ _, Finset.sum_add_distrib, Finset.s
      simp [mul_left_comm]
      ring
      convert h_inner x using 1
      simp [ContinuousLinearMap.reApplyInnerSelf]
      rw [← inner_conj_symm, Complex.conj_re]

open ComplexOrder in
/-- `Φ` maps PosDef HermitianMat to pdSet. -/
lemma Φ_mem_pdSet [Nonempty d] (A : HermitianMat d ℂ) (hA : A.mat.PosDef) :
  Φ A.mat ∈ pdSet (□ := EuclideanSpace ℂ d) := by
  have h_spectrum : spectrum ℝ (Φ A.mat) = spectrum ℝ A.mat := by
    ext x
    simp [spectrum.mem_iff]
    rw [show (algebraMap ℝ _) x = Φ (algebraMap ℝ _ x) from ?_,
        show (algebraMap ℝ (Matrix d d ℂ)) x = x • 1 from ?_]
    · simp [← map_sub, -map_smul]
    · simp [Algebra.smul_def]
    · ext
      simp [Φ]
      simp [Algebra.algebraMap_eq_smul_one, Matrix.mulVec, dotProduct]
      simp [Matrix.one_apply, Finset.sum_ite_eq]
      rfl
  refine' { Φ_isSelfAdjoint A, h_spectrum.symm ▸ _ }
  exact HermitianMat.Matrix.PosDef.spectrum_subset_Ioi hA

set_option backward.isDefEq.respectTransparency false in
/-- `Φ` commutes with CFC for Hermitian matrices. -/
lemma Φ_cfc (A : HermitianMat d ℂ) (f : ℝ → ℝ) :
  Φ (cfc f A.mat) = cfc f (Φ A.mat) := by
  exact StarAlgHomClass.map_cfc Φ f A.mat (hφ := Φ_continuous)
  (ha := A.H.isSelfAdjoint)

set_option backward.isDefEq.respectTransparency false in
/-- `Φ` commutes with rpow for PSD matrices. -/
lemma Φ_rpow (A : HermitianMat d ℂ) (hA : 0 ≤ A) (r : ℝ) :
  Φ (A ^ r).mat = (Φ A.mat) ^ r := by
  rw [HermitianMat.rpow_eq_cfc, HermitianMat.mat_cfc]
  rw [Φ_cfc, CFC.rpow_eq_cfc_real (ha := Φ_nonneg A hA)]

set_option maxHeartbeats 800000 in
/-- General trace bridge: the operator trace of Φ(M) equals the matrix trace of M,
```

```

for any matrix M (not just Hermitian). -/
lemma trace_Φ_eq (M : Matrix d d ℂ) :
  (LinearMap.trace ℂ (EuclideanSpace ℂ d)) (Φ M).toLinearMap = M.trace :=
  rw [LinearMap.trace_eq_matrix_trace ℂ (EuclideanSpace.basisFun d ℂ).toBas
  congr 1
  ext i j
  simp [Φ, Matrix.toEuclideanCLM, EuclideanSpace.basisFun]

/-- `traceRe(Φ(M)) = re(Tr[M])` for any matrix M. -/
lemma traceRe_Φ_general (M : Matrix d d ℂ) :
  traceRe (Φ M) = Complex.re M.trace := by
  simp [traceRe, trace_Φ_eq]

end HermitianMatBridge

namespace HermitianMat

open LiebAndoTrace GeneralizedPerspectiveFunction ComplexOrder

set_option maxHeartbeats 400000 in
/-- The PSD cone is convex. -/
private lemma psd_convex : Convex ℝ {σ : HermitianMat d ℂ | 0 ≤ σ} := by
  intro σ₁ hσ₁ σ₂ hσ₂ a b ha hb _
  simp only [Set.mem_setOf_eq] at *
  exact add_nonneg (smul_nonneg ha hσ₁) (smul_nonneg hb hσ₂)

/-- The trace of rpow applied to a congruence is continuous in the base mat
/
private lemma trace_conj_rpow_continuous {s p : ℝ} (hs : 0 ≤ s) (hp : 0 ≤ p)
  (H : HermitianMat d ℂ) :
  Continuous (fun σ : HermitianMat d ℂ ↦
    ((H.conj (σ ^ s).mat) ^ p).trace) := by
  have h_rpow_cont : Continuous (fun σ : HermitianMat d ℂ => σ ^ s) :=
    rpow_const_continuous hs
  have h_conj_cont : Continuous (fun σ : HermitianMat d ℂ => (σ ^ s).mat) :
    Continuous.subtype_val h_rpow_cont
  have h_trace_cont : Continuous (fun σ : HermitianMat d ℂ => σ.trace) := b
    simp [HermitianMat.trace]; fun_prop
  have h_comp_cont : Continuous (fun σ : Matrix d d ℂ => ((conj σ H) ^ p).t
    have h_conj_cont : Continuous (fun σ : Matrix d d ℂ => conj σ H) :=
      continuous_conj H
    exact h_trace_cont.comp (rpow_const_continuous hp |>.comp h_conj_cont)
  exact h_comp_cont.comp h_conj_cont

/-! ### Density and continuity lemmas for PD/PSD extension -
/

```

1.131.

QUANTUMINFO/FORMATHLIB/HERMITIANMAT/LIEBCONCAVITY.LEAN433

```

private lemma psd_add_eps_posdef [Nonempty d] (σ : HermitianMat d ℂ) (hσ : 0 ≤ σ)
  (ε : ℝ) (hε : 0 < ε) : (σ + ε • (1 : HermitianMat d ℂ)).mat.PosDef := by
  refine' { _, _ }
  · exact H (σ + ε • 1)
  · intro x hx_ne_zero
    have h_pos : 0 < ∑ i, ∑ j, star (x i) * (σ.mat i j + ε * (if i = j then 1 else 0)) := by
      have h_pos : 0 ≤ ∑ i, ∑ j, star (x i) * σ i j * x j := by
        have := hσ.2
        simp [Finsupp.sum_fintype, Finset.sum_mul _ _ _] using this x
        simp_all [mul_add, add_mul, Finset.sum_add_distrib]
        refine' add_pos_of_nonneg_of_pos h_pos _
        simp_all [mul_comm, mul_left_comm, Complex.mul_conj, Complex.normSq_eq_norm_sq]
        contrapose! hx_ne_zero
        ext i
        simp only [Finsupp.coe_zero, Pi.zero_apply]
        exact not_not.mp fun hi => hx_ne_zero <| lt_of_lt_of_le (by positivity) <|
          Finset.single_le_sum (fun i _ => by positivity) <| Finset.mem_univ i
      simp [Finsupp.sum]
      convert h_pos using 1
      rw [Finset.sum_subset (Finset.subset_univ x.support)]
      · refine Finset.sum_congr rfl fun i hi => ?_
        exact Finset.sum_subset (Finset.subset_univ _) fun j hj₁ hj₂ => by aesop
      · aesop

/-- σ + εI → σ as ε → 0+. -/
private lemma tendsto_add_eps (σ : HermitianMat d ℂ) :
  Filter.Tendsto (fun ε : ℝ => σ + ε • (1 : HermitianMat d ℂ))
    (nhdsWithin 0 (Set.Ioi 0)) (nhds σ) := by
  exact tendsto_nhdsWithin_of_tendsto_nhds
    (Continuous.tendsto' (by continuity) _ _ (by simp))

/-! ### Helper lemmas for the core concavity proof -/

set_option maxHeartbeats 800000 in
/-- **AB/BA trace identity for rpow**: `Tr[(C^*C)^p] = Tr[(CC^*)^p]` for any square
  /
private lemma trace_rpow_conjTranspose_mul_comm [Nonempty d]
  (C : Matrix d d ℂ) (p : ℝ) :
  let M₁ : HermitianMat d ℂ := ⟨_, Matrix.isHermitian_conjTranspose_mul_self C⟩
  let M₂ : HermitianMat d ℂ := ⟨_, Matrix.isHermitian_mul_conjTranspose_self C⟩
  (M₁ ^ p).trace = (M₂ ^ p).trace := by
  intro M₁ M₂
  rw [trace_rpow_eq_sum M₁ p, trace_rpow_eq_sum M₂ p]
  have hcharpoly : M₁.mat.charpoly = M₂.mat.charpoly :=
    Matrix.charpoly_mul_comm C.conjTranspose C

```

```

rw [M1.H.charpoly_eq, M2.H.charpoly_eq] at hcharpoly
have hmultiset : Finset.univ.val.map (fun i => (M1.H.eigenvalues i : ℂ))
  Finset.univ.val.map (fun i => (M2.H.eigenvalues i : ℂ))
  have h1 := Polynomial.roots_multiset_prod_X_sub_C
    (Finset.univ.val.map (fun i => (M1.H.eigenvalues i : ℂ)))
  have h2 := Polynomial.roots_multiset_prod_X_sub_C
    (Finset.univ.val.map (fun i => (M2.H.eigenvalues i : ℂ)))
  simp only [Multiset.map_map] at h1 h2
  rw [← h1, ← h2]; congr 1
have hmap : Finset.univ.val.map (fun i => M1.H.eigenvalues i ^ p) =
  Finset.univ.val.map (fun i => M2.H.eigenvalues i ^ p) := by
  have := congr_arg (Multiset.map (fun x : ℂ => x.re ^ p)) hmultiset
  simp [Multiset.map_map, Function.comp, Complex.ofReal_re] at this
  exact this
simp using congr_arg Multiset.sum hmap

/-! ### Core concavity on positive definite matrices -/

section VariationalAndBridge
open InnerProductSpace

/-
Variational lower bound from trace Young inequality:
`Tr[X^p] ≥ p · [X, Z^r] - (p-1) · Tr[Z]` where r = (p-1)/p.
Proof: Young says [X, Z^r] ≤ Tr[X^p]/p + Tr[Z]/q (with q=p/(p-1)),
so p·[X, Z^r] ≤ Tr[X^p] + (p-1)·Tr[Z].
-/
private lemma variational_lower_bound
  (X Z : HermitianMat d ℂ) (hX : 0 ≤ X) (hZ : 0 ≤ Z)
  {p : ℝ} (hp : 1 < p) :
  p * [X, Z ^ ((p-1)/p)]_ℝ - (p - 1) * Z.trace ≤ (X ^ p).trace := by
  have := @HermitianMat.trace_young d _ _ X (Z ^ ((p - 1) / p)) hX (?) p (
    · -- Using the fact that $Z$ is positive semi-definite, we can simplify t
  have hZ_pow : ((Z ^ ((p - 1) / p)) ^ (p / (p - 1))) = Z := by
    rw [← HermitianMat.rpow_mul]
    · field_simp
    rw [div_self (by linarith), HermitianMat.rpow_one]
    · exact hZ
  simp_all
  field_simp at this
  exact this
  · exact rpow_nonneg hZ
  · grind

/-

```


1.131.

QUANTUMINFO/FORMATHLIB/HERMITIANMAT/LIEBCONCAVITY.LEAN435

At the optimizer $Z = X^p$, the variational bound is tight.

```

-/
private lemma variational_eq_optimizer
  (X : HermitianMat d ℂ) (hX : 0 ≤ X)
  {p : ℝ} (hp : 1 < p) :
  p * ⌊X, (X ^ p) ^ ((p-1)/p)⌋_ℝ - (p - 1) * (X ^ p).trace = (X ^ p).trace := by
  -- (X ^ p) ^ ((p - 1) / p) = X ^ (p * ((p - 1) / p)) = X ^ (p - 1)
  have h_exp : (X ^ p) ^ ((p - 1) / p) = X ^ (p - 1) := by
    rw [← rpow_mul hX, mul_div_cancel₀ _ (by positivity)]
  have h_inner : ⌊X, X ^ (p - 1)⌋_ℝ = (X ^ p).trace := by
    have h_inner : ⌊X, X ^ (p - 1)⌋_ℝ = (X * (X ^ (p - 1))).mat.trace.re := by
      exact Real.ext_cauchy rfl
    convert h_inner using 1
  have h_exp : (X ^ p).mat = X.mat * (X ^ (p - 1)).mat := by
    convert mat_rpow_add hX _
    rotate_left
    rotate_left
    exacts [1, by linarith, by ring, by simp]
  exact h_exp ▸ rfl
  rw [h_exp, h_inner]; ring

```

/-

Joint concavity of the Lieb extension trace map on HermitianMat.

This bridges `liebExtensionTrace\_jointlyConcaveOn\_pdSet` to HermitianMat.

-/

```

set_option maxHeartbeats 1600000 in
set_option backward.isDefEq.respectTransparency false in
private lemma liebExtension_bridge [Nonempty d]
  {q r : ℝ} (hq : 0 < q) (hr : 0 < r) (hqr : q + r ≤ 1)
  (K : HermitianMat d ℂ)
  (σ₁ σ₂ Z₁ Z₂ : HermitianMat d ℂ)
  (hσ₁ : σ₁.mat.PosDef) (hσ₂ : σ₂.mat.PosDef)
  (hZ₁ : Z₁.mat.PosDef) (hZ₂ : Z₂.mat.PosDef)
  (θ : ℝ) (hθ₀ : 0 ≤ θ) (hθ₁ : θ ≤ 1) :
  (1 - θ) * ⌊(σ₁ ^ q).conj K, Z₁ ^ r⌋_ℝ + θ * ⌊(σ₂ ^ q).conj K, Z₂ ^ r⌋_ℝ ≤
  ⌊((1 - θ) • σ₁ + θ • σ₂) ^ q⌋.conj K, ((1 - θ) • Z₁ + θ • Z₂) ^ r⌋_ℝ := by
  open HermitianMatBridge GeneralizedPerspectiveFunction in
  -- Rewrite the inequality using the joint concavity result.
  have h_joint_concave :=
    LiebAndoTrace.liebExtensionTrace_jointlyConcaveOn_pdSet hr hq (by linarith) (Φ K)
  have h_rewrite : ∀ σ Z : HermitianMat d ℂ, 0 ≤ σ → 0 ≤ Z →
    ⌊(σ ^ q).conj K, Z ^ r⌋_ℝ = liebExtensionTraceMap q r (Φ K.mat) (Φ σ.mat) (Φ Z.mat)
  intros σ Z hσ hZ
  have h_inner : ⌊(σ ^ q).conj K, Z ^ r⌋_ℝ = ((σ ^ q).mat * K * (Z ^ r).mat * K).trace
  rw [inner_eq_re_trace]
  simp [Matrix.mul_assoc, Matrix.trace_mul_comm K.mat]

```

```

convert h_inner using 1
rw [← traceRe_Φ_general]
simp [liebExtensionTraceMap, Φ_rpow, hσ, hZ]
rw [show star (Φ K.mat) = Φ K.mat from ?_]
have h_rewrite : IsSelfAdjoint (Φ K.mat) := by
  exact Φ_isSelfAdjoint K
exact h_rewrite
convert h_joint_concave (Φ_mem_pdSet σ1 hσ1) (Φ_mem_pdSet σ2 hσ2)
(Φ_mem_pdSet Z1 hZ1) (Φ_mem_pdSet Z2 hZ2) hθ0 hθ1 using 1
· rw [h_rewrite σ1 Z1 (by
  constructor
  · simp [Matrix.IsHermitian]
  · intro x; have := hσ1.2
    simp_all
    exact if hx : x = 0 then by simp [hx] else le_of_lt (this hx))
    (by finiteness), h_rewrite σ2 Z2 (by finiteness) (by finiteness)]
norm_num [Algebra.smul_def]
· convert h_rewrite ((1 - θ) • σ1 + θ • σ2) ((1 - θ) • Z1 + θ • Z2) _ _ u
· congr! 2
· ext; simp [Φ]
  simp [Matrix.mulVec, dotProduct, Finset.mul_sum, mul_assoc]
· ext; simp [Φ]
  simp [Matrix.mulVec, dotProduct, Finset.mul_sum]
  simp only [mul_assoc]
· nontriviality
  have h_pos_def : ∀ (A : HermitianMat d ℂ), A.mat.PosDef → 0 ≤ A := by
    intro A hA
    have := hA.2
    constructor
    · simp [Matrix.IsHermitian]
    · intro x; by_cases hx : x = 0 <=> simp_all [Matrix.PosDef]
      exact le_of_lt (hA.2 hx)
  positivity [sub_nonneg.2 hθ1]
· have : 0 ≤ 1 - θ := by linarith
  positivity

/-
**AB/BA rewrite**: `Tr[(H.conj (σ^s))^p] = Tr[((σ^{2s}).conj (H^{1/2}))^p]`
-/
private lemma trace_conj_rpow_eq_conj_sqrt [Nonempty d]
(σ H : HermitianMat d ℂ) (hσ : 0 ≤ σ) (hH : 0 ≤ H) (s p : ℝ) (hs : 0 <
((H.conj (σ ^ s).mat) ^ p).trace =
(((σ ^ (2 * s)).conj (H ^ (1/2 : ℝ)).mat) ^ p).trace := by
norm_num [conj_apply_mat, Matrix.mul_assoc]
have h_exp : (σ ^ (2 * s)).mat = (σ ^ s).mat * (σ ^ s).mat := by
  convert mat_rpow_add hσ _ using 1 <=> ring_nf

```

1.131.

QUANTUMINFO/FORMATHLIB/HERMITIANMAT/LIEBCONCAVITY.LEAN 437

```

positivity
have h_exp' : (H ^ (1 / 2 : ℝ)).mat * (H ^ (1 / 2 : ℝ)).mat = H.mat := by
  apply HermitianMat.pow_half_mul hH
-- Apply the lemma that states the equality of the traces of the conjugates.
have := trace_rpow_conjTranspose_mul_comm ((σ ^ s).mat * (H ^ (1 / 2 : ℝ)).mat) p
convert this.symm using 3 <=> simp [mul_assoc]
· ext; simp [← mul_assoc]
  simp [conj_apply]
  simp_all [mul_assoc]
· ext; simp [← mul_assoc]
  simp [conj, h_exp]
  simp [mul_assoc]

/-
Extension of liebExtension_bridge from PD to PSD Z inputs via continuity.
-/
private lemma liebExtension_bridge_psd [Nonempty d]
  {q r : ℝ} (hq : 0 < q) (hr : 0 < r) (hqr : q + r ≤ 1)
  (K σ₁ σ₂ Z₁ Z₂ : HermitianMat d ℂ)
  (hσ₁ : σ₁.mat.PosDef) (hσ₂ : σ₂.mat.PosDef)
  (hZ₁ : 0 ≤ Z₁) (hZ₂ : 0 ≤ Z₂)
  (θ : ℝ) (hθ₀ : 0 ≤ θ) (hθ₁ : θ ≤ 1) :
  (1 - θ) * [ (σ₁ ^ q).conj K, Z₁ ^ r ]_ℝ + θ * [ (σ₂ ^ q).conj K, Z₂ ^ r ]_ℝ ≤
  [ ((1 - θ) * σ₁ + θ * σ₂) ^ q ].conj K, ((1 - θ) * Z₁ + θ * Z₂) ^ r ]_ℝ := by
  open scoped Topology in
  have h_cont : ∀ (ε : ℝ), 0 < ε → (1 - θ) * [ (σ₁ ^ q).conj K, (Z₁ + ε • 1) ^ r ]_ℝ +
    θ * [ (σ₂ ^ q).conj K, (Z₂ + ε • 1) ^ r ]_ℝ ≤ [ ((1 - θ) * σ₁ + θ * σ₂) ^ q ].conj K,
    ((1 - θ) * (Z₁ + ε • 1) + θ * (Z₂ + ε • 1)) ^ r ]_ℝ := by
    intro ε hε_pos
    exact liebExtension_bridge hq hr hqr K σ₁ σ₂ (Z₁ + ε • 1) (Z₂ + ε • 1) hσ₁ hσ₂
      (psd_add_eps_posdef Z₁ hZ₁ ε hε_pos) (psd_add_eps_posdef Z₂ hZ₂ ε hε_pos) θ hθ
  -- Apply the continuity results to take the limit as ε approaches 0.
  have h_lim :
    Filter.Tendsto (fun ε : ℝ => [ (σ₁ ^ q).conj K, (Z₁ + ε • 1) ^ r ]_ℝ) (Filter.> 0)
      (Filter.> 0) &
    Filter.Tendsto (fun ε : ℝ => [ (σ₂ ^ q).conj K, (Z₂ + ε • 1) ^ r ]_ℝ) (Filter.> 0)
      (Filter.> 0) := by
    constructor <=> refine' Filter.Tendsto.mono_left _ nhdsWithin_le_nhds
  · have h_cont : Continuous (fun ε : ℝ => (Z₁ + ε • 1) ^ r) := by
      have h_cont : Continuous (fun ε : ℝ => (Z₁ + ε • 1)) := by
        fun_prop
      exact (HermitianMat.rpow_const_continuous (show 0 ≤ r by positivity)).comp h
    convert Filter.Tendsto.inner tendsto_const_nhds (h_cont.tendsto 0) using 2
    norm_num
  · have h_inner_cont : Continuous (fun ε : ℝ => (Z₂ + ε • 1) ^ r) := by
      have h_cont : Continuous (fun ε : HermitianMat d ℂ => ε ^ r) := by

```

```

        apply_rules [HermitianMat.rpow_const_continuous]
        positivity
        fun_prop (disch := solve_by_elim)
        convert Filter.Tendsto.inner tendsto_const_nhds (h_inner_cont.tendsto
refine le_of_tendsto_of_tendsto
  ((tendsto_const_nhds.mul h_lim.1).add (tendsto_const_nhds.mul h_lim.2))
  (Filter.eventually_of_mem self_mem_nhdsWithin h_cont)
refine Filter.Tendsto.inner tendsto_const_nhds ?_
refine (rpow_const_continuous (by positivity) |> Continuous.continuousAt
  h.tendsto.comp (show Filter.Tendsto (fun  $\varepsilon$  :  $\mathbb{R}$  =>  $(1 - \theta) \cdot (Z_1 + \varepsilon \cdot 1$ 
  (nhdsWithin  $\theta$  (Set.Ioi  $\theta$ )) (nhds  $((1 - \theta) \cdot Z_1 + \theta \cdot Z_2)$ ) from ?_))
refine' tendsto_nhdsWithin_of_tendsto_nhds _
refine' Continuous.tendsto' _ _ _ _ <=> norm_num
fun_prop

set_option maxHeartbeats 1600000 in
/-- Core concavity inequality on positive definite matrices. -
/
private lemma trace_conj_rpow_concave_pd [Nonempty d] { $\alpha$  :  $\mathbb{R}$ } (h $\alpha$  :  $1 < \alpha$ )
  (H : HermitianMat d  $\mathbb{C}$ ) (hH :  $0 \leq H$ )
  ( $\sigma_1$   $\sigma_2$  : HermitianMat d  $\mathbb{C}$ ) (h $\sigma_1$  :  $\sigma_1$ .mat.PosDef) (h $\sigma_2$  :  $\sigma_2$ .mat.PosDef)
  (a b :  $\mathbb{R}$ ) (ha :  $0 \leq a$ ) (hb :  $0 \leq b$ ) (hab :  $a + b = 1$ ) :
  let s := ( $\alpha - 1$ ) / (2 *  $\alpha$ )
  let p :=  $\alpha$  / ( $\alpha - 1$ )
  a * ((H.conj ( $\sigma_1$  ^ s).mat) ^ p).trace + b * ((H.conj ( $\sigma_2$  ^ s).mat) ^ p)
  ((H.conj ((a *  $\sigma_1$  + b *  $\sigma_2$ ) ^ s).mat) ^ p).trace := by
intro s p
-- Key derived parameters
have h $\alpha$ _pos :  $0 < \alpha$  := by linarith
have h $\alpha$ 1_pos :  $0 < \alpha - 1$  := by linarith
have h $\alpha$ _ne :  $\alpha \neq 0$  := ne_of_gt h $\alpha$ _pos
have h $\alpha$ 1_ne :  $\alpha - 1 \neq 0$  := ne_of_gt h $\alpha$ 1_pos
have h $s$ _pos :  $0 < s$  := by show  $0 < (\alpha - 1) / (2 * \alpha)$ ; positivity
have h $p$ _gt1 :  $1 < p$  := by
  show  $1 < \alpha / (\alpha - 1)$ ; rw [lt_div_iff0 h $\alpha$ 1_pos]; linarith
have h $p$ _pos :  $0 < p$  := by linarith
-- The exponents for the bridge
set q := ( $\alpha - 1$ ) /  $\alpha$  with q_def
set r := 1 /  $\alpha$  with r_def
have h $q$ _pos :  $0 < q$  := by simp only [q_def]; positivity
have h $r$ _pos :  $0 < r$  := by simp only [r_def]; positivity
have h $q$ r :  $q + r \leq 1$  := by
  simp only [q_def, r_def]; rw [← add_div, sub_add_cancel, div_self h $\alpha$ _ne]
have h2s_eq_q :  $2 * s = q$  := by
  show  $2 * ((\alpha - 1) / (2 * \alpha)) = (\alpha - 1) / \alpha$ ; field_simp
have h $r$ _eq :  $r = (p - 1) / p$  := by

```

1.131.

QUANTUMINFO/FORMATHLIB/HERMITIANMAT/LIEBCONCAVITY.LEAN439

```

    show 1 /  $\alpha$  = ( $\alpha$  / ( $\alpha$  - 1) - 1) / ( $\alpha$  / ( $\alpha$  - 1)); field_simp; ring
-- K =  $H^{\{1/2\}}$ 
set K := H ^ (1/2 :  $\mathbb{R}$ ) with K_def
have hK :  $0 \leq K$  := rpow_nonneg hH
-- PSD facts for  $\sigma_i$ 
have h $\sigma_1$ _psd :  $0 \leq \sigma_1$  := HermitianMat.zero_le_iff.mpr h $\sigma_1$ .posSemidef
have h $\sigma_2$ _psd :  $0 \leq \sigma_2$  := HermitianMat.zero_le_iff.mpr h $\sigma_2$ .posSemidef
have h $\sigma_{mix}$ _psd :  $0 \leq a \cdot \sigma_1 + b \cdot \sigma_2$  :=
  add_nonneg (smul_nonneg ha h $\sigma_1$ _psd) (smul_nonneg hb h $\sigma_2$ _psd)
--  $X_i = (\sigma_i \wedge q)$ .conj K
set X $_1$  := ( $\sigma_1 \wedge q$ ).conj K.mat with X $_1$ _def
set X $_2$  := ( $\sigma_2 \wedge q$ ).conj K.mat with X $_2$ _def
set X $_{mix}$  := (( $a \cdot \sigma_1 + b \cdot \sigma_2$ )  $\wedge q$ ).conj K.mat with X $_{mix}$ _def
have hX $_1$  :  $0 \leq X_1$  := conj_nonneg _ (rpw_nonneg h $\sigma_1$ _psd)
have hX $_2$  :  $0 \leq X_2$  := conj_nonneg _ (rpw_nonneg h $\sigma_2$ _psd)
have hX $_{mix}$  :  $0 \leq X_{mix}$  := conj_nonneg _ (rpw_nonneg h $\sigma_{mix}$ _psd)
--  $Z_i = X_i \wedge p$ 
set Z $_1$  := X $_1 \wedge p$  with Z $_1$ _def
set Z $_2$  := X $_2 \wedge p$  with Z $_2$ _def
have hZ $_1$  :  $0 \leq Z_1$  := rpow_nonneg hX $_1$ 
have hZ $_2$  :  $0 \leq Z_2$  := rpow_nonneg hX $_2$ 
have hZ $_{mix}$  :  $0 \leq a \cdot Z_1 + b \cdot Z_2$  :=
  add_nonneg (smul_nonneg ha hZ $_1$ ) (smul_nonneg hb hZ $_2$ )
-- Step 1: Rewrite using AB/BA identity
have rewrite $_1$  : ((H.conj ( $\sigma_1 \wedge s$ ).mat)  $\wedge p$ ).trace = (Z $_1$ ).trace := by
  rw [trace_conj_rpow_eq_conj_sqrt  $\sigma_1$  H h $\sigma_1$ _psd hH s p hs_pos, h2s_eq_q]
have rewrite $_2$  : ((H.conj ( $\sigma_2 \wedge s$ ).mat)  $\wedge p$ ).trace = (Z $_2$ ).trace := by
  rw [trace_conj_rpow_eq_conj_sqrt  $\sigma_2$  H h $\sigma_2$ _psd hH s p hs_pos, h2s_eq_q]
have rewrite $_{mix}$  : ((H.conj (( $a \cdot \sigma_1 + b \cdot \sigma_2$ )  $\wedge s$ ).mat)  $\wedge p$ ).trace = (X $_{mix} \wedge p$ ).
  rw [trace_conj_rpow_eq_conj_sqrt ( $a \cdot \sigma_1 + b \cdot \sigma_2$ ) H h $\sigma_{mix}$ _psd hH s p hs_pos, h
  rw [rewrite $_1$ , rewrite $_2$ , rewrite $_{mix}$ ]
-- Step 2a: Use variational_eq_optimizer
have var_opt $_1$  := variational_eq_optimizer X $_1$  hX $_1$  hp_gt1
have var_opt $_2$  := variational_eq_optimizer X $_2$  hX $_2$  hp_gt1
rw [← hr_eq] at var_opt $_1$  var_opt $_2$ 
-- Step 2b: Rewrite LHS
rw [show Z $_1$ .trace = p *  $\square X_1$ , Z $_1 \wedge r_{\square \mathbb{R}}$  - (p - 1) * Z $_1$ .trace from var_opt $_1$ .symm,
  show Z $_2$ .trace = p *  $\square X_2$ , Z $_2 \wedge r_{\square \mathbb{R}}$  - (p - 1) * Z $_2$ .trace from var_opt $_2$ .symm]
-- Goal:  $a \cdot (p \cdot \square X_1, Z_1 \wedge r_{\square \mathbb{R}} - (p-1) \cdot Z_1.trace) + b \cdot (p \cdot \square X_2, Z_2 \wedge r_{\square \mathbb{R}} -$ 
(p-1) * Z $_2$ .trace)  $\leq (X_{mix} \wedge p).trace$ 
-- Step 2c-f: Chain inequality
calc a * (p *  $\square X_1$ , Z $_1 \wedge r_{\square \mathbb{R}}$  - (p - 1) * Z $_1$ .trace) +
  b * (p *  $\square X_2$ , Z $_2 \wedge r_{\square \mathbb{R}}$  - (p - 1) * Z $_2$ .trace)
  = p * (a *  $\square X_1$ , Z $_1 \wedge r_{\square \mathbb{R}}$  + b *  $\square X_2$ , Z $_2 \wedge r_{\square \mathbb{R}}$ ) -
    (p - 1) * (a * Z $_1$ .trace + b * Z $_2$ .trace) := by ring
  _  $\leq$  p *  $\square X_{mix}$ , (a * Z $_1$  + b * Z $_2$ )  $\wedge r_{\square \mathbb{R}}$  -

```

```

    (p - 1) * (a • Z1 + b • Z2).trace := by
    have bridge := liebExtension_bridge_psd hq_pos hr_pos hqr K
      σ1 σ2 Z1 Z2 hσ1 hσ2 hZ1 hZ2 b hb (by linarith)
    rw [show (1 : ℝ) - b = a from by linarith] at bridge
    have trace_lin : (a • Z1 + b • Z2).trace = a * Z1.trace + b * Z2.trace
    rw [trace_add, trace_smul, trace_smul]
    rw [trace_lin]
    linarith [mul_le_mul_of_nonneg_left bridge hp_pos.le]
  _ ≤ (Xmix ^ p).trace := by
    rw [hr_eq]
    exact variational_lower_bound Xmix (a • Z1 + b • Z2) hXmix hZmix
end VariationalAndBridge

/-
**Concavity of the trace functional for DPI**: For  $\alpha > 1$ ,  $H \geq 0$ , the map
 $\sigma \mapsto \text{Tr}[(\sigma^s H \sigma^s)^p]$  is concave on PSD matrices,
where  $s = (\alpha - 1)/(2\alpha)$  and  $p = \alpha/(\alpha - 1)$ .
-/
theorem trace_conj_rpow_concave {α : ℝ} (hα : 1 < α)
  (H : HermitianMat d ℂ) (hH : 0 ≤ H) :
  ConcaveOn ℝ {σ : HermitianMat d ℂ | 0 ≤ σ}
    (fun σ ↦ ((H.conj (σ ^ ((α - 1) / (2 * α))).mat) ^ (α / (α - 1))).trace)
  refine' {psd_convex, fun σ1 hσ1 σ2 hσ2 a b ha hb hab => _}
  by_cases hd : Nonempty d
  · simp only [Set.mem_setOf_eq, smul_eq_mul] at *
    open scoped Topology in
    refine' le_of_tendsto_of_tendsto (b := [][>] (0 : ℝ))
      (f := fun ε ↦ a * ((H.conj ((σ1 + ε • 1) ^ ((α - 1) / (2 * α))).mat)
        b * ((H.conj ((σ2 + ε • 1) ^ ((α - 1) / (2 * α))).mat) ^ (α / (α - 1)
        (g := fun ε ↦ ((H.conj ((a • (σ1 + ε • 1) + b • (σ2 + ε • 1)) ^ ((α - 1) / (α - 1))).trace)
      ?_ ?_
    · have hcont : Continuous (fun σ : HermitianMat d ℂ ↦ ((H.conj (σ ^ ((α - 1) / (α - 1))).trace) :=
      trace_conj_rpow_continuous
        (div_nonneg (sub_nonneg.2 hα.le) (by positivity))
        (div_nonneg (by positivity) (by linarith)) H
      exact Filter.Tendsto.add (tendsto_const_nhds.mul (hcont.continuousAt.
        (tendsto_add_eps _))) (tendsto_const_nhds.mul (hcont.continuousAt.
        (tendsto_add_eps _))) |> fun h => h.trans (by simp)
    · have hcont : Continuous (fun σ : HermitianMat d ℂ ↦ ((H.conj (σ ^ ((α - 1) / (α - 1))).trace) :=
      trace_conj_rpow_continuous (div_nonneg (sub_nonneg.2 hα.le) (by pos
        (div_nonneg (by positivity) (by linarith)) H
      refine hcont.continuousAt.tendsto.comp (show Filter.Tendsto

```

```

      (fun ε ↦ a • (σ1 + ε • 1) + b • (σ2 + ε • 1)) (□[>] 0) (□ (a • σ1 + b • σ2))
    apply tendsto_nhdsWithin_of_tendsto_nhds
    exact Continuous.tendsto' (by fun_prop) _ _ (by simp)
  · filter_upwards [self_mem_nhdsWithin] with ε hε
    refine trace_conj_rpow_concave_pd hα H hH (σ1 + ε • 1) (σ2 + ε • 1) ?_ ?_ a b
  · exact psd_add_eps_posdef σ1 hσ1 ε hε
  · exact psd_add_eps_posdef σ2 hσ2 ε hε
  · simp_all [HermitianMat.trace]

end HermitianMat

```

1.132 QuantumInfo/ForMathlib/HermitianMat/Order.lean

```

/-- The self-Kronecker map `A ↦ A ⊗k A` is monotone on nonnegative Hermitian matrices /
theorem kronecker_self_mono (hA : 0 ≤ A) (hB : 0 ≤ B) (hAB : A ≤ B) :
  A ⊗k A ≤ B ⊗k B := by
  rw [← sub_nonneg]
  have hAC : A ⊗k B + -(A ⊗k A) = A ⊗k (B - A) := by
    rw [show -(A ⊗k A) = A ⊗k (-A) by
      symm
      ext1
      simp using (Matrix.kronecker_smul (-1 : ℂ) A.mat A.mat)]
    simp [sub_eq_add_neg] using
      (HermitianMat.kronecker_add (A := A) (B := B) (C := -
A)).symm
  have hEq : B ⊗k B - A ⊗k A = A ⊗k (B - A) + (B - A) ⊗k B := by
    calc
      B ⊗k B - A ⊗k A = (A + (B - A)) ⊗k B - A ⊗k A := by
        rw [show A + (B - A) = B by abel]
      _ = (A ⊗k B + (B - A) ⊗k B) - A ⊗k A := by rw [HermitianMat.add_kronecker]
      _ = (A ⊗k B + -(A ⊗k A)) + (B - A) ⊗k B := by abel
      _ = A ⊗k (B - A) + (B - A) ⊗k B := by rw [hAC]
  simp [hEq] using add_nonneg
  (HermitianMat.kronecker_nonneg hA (sub_nonneg.mpr hAB))
  (HermitianMat.kronecker_nonneg (sub_nonneg.mpr hAB) hB)

/-- If a Hermitian matrix is bounded by `M * I`, then all its eigenvalues are at most M /
theorem le_smul_one_imp_eigenvalues_le [DecidableEq n] (A : HermitianMat n ℂ) (M : ℝ)
  (h : A ≤ M • (1 : HermitianMat n ℂ)) (i : n) :
  A.H.eigenvalues i ≤ M := by
  let v : n → ℂ := (A.H.eigenvectorBasis i).ofLp
  have hv : star v □v v = (1 : ℂ) := by

```

```

    rw [show v = (A.H.eigenvectorBasis i).ofLp from rfl]
    rw [dotProduct_comm, ← EuclideanSpace.inner_eq_star_dotProduct]
    simp [A.H.eigenvectorBasis.orthonormal.1 i]
    have hquad := (le_iff_mulVec_le_mulVec A (M • (1 : HermitianMat n ℂ))).mp
    rw [show A.mat.mulVec v = (A.H.eigenvalues i : ℂ) • v from by
        simp [v] using A.H.mulVec_eigenvectorBasis i] at hquad
    rw [dotProduct_smul, hv] at hquad
    change (A.H.eigenvalues i : ℂ) • 1 ≤
        star v ▯v ((M : ℂ) • (1 : Matrix n n ℂ)) *v v at hquad
    have hquadC : (A.H.eigenvalues i : ℂ) ≤ (M : ℂ) := by
        have hright : star v ▯v ((M : ℂ) • (1 : Matrix n n ℂ)) *v v = (M : ℂ) :
            simp [Matrix.smul_mulVec, hv] using (Algebra.algebraMap_eq_smul_one
            simp [Matrix.smul_mulVec, hv] using hquad.trans_eq hright
        exact_mod_cast hquadC

open MatrixOrder in
/-- If all eigenvalues of a Hermitian matrix are at most `M`, then it is bo
/
theorem eigenvalues_le_imp_le_smul_one [DecidableEq n] (A : HermitianMat n
    (h : ∀ i, A.H.eigenvalues i ≤ M) :
    A ≤ M • (1 : HermitianMat n ℂ) := by
    simp [HermitianMat.mat_one] using
        (Matrix.PosSemidef.le_smul_one_of_eigenvalues_iff A.H M).mp h

```

1.133 QuantumInfo/ForMathlib/HermitianMat/Peierls.lean

```

/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
module

public import QuantumInfo.ForMathlib.HermitianMat.Sqrt
public import QuantumInfo.ForMathlib.HermitianMat.LiebConcavity

@[expose] public section

noncomputable section

variable {d : Type*}
variable [Fintype d] [DecidableEq d]
variable {▯ : Type*} [RCLike ▯]

```



```

open HermitianMat
open scoped InnerProductSpace RealInnerProductSpace Topology

namespace HermitianMat

/--
The trace of cfc(g, A) is invariant under unitary conjugation of A.
Follows from `cfc_conj_unitary` and `trace_conj_unitary`.
-/
lemma trace_cfc_conj_unitary (A : HermitianMat d ℂ) (g : ℝ → ℝ) (U : [d]) :
  ((A.conj U.val).cfc g).trace = (A.cfc g).trace := by
  rw [cfc_conj_unitary, trace_conj_unitary]

/--
Peierls inequality: for a convex function g, the sum of g applied to the
diagonal entries of a Hermitian matrix is at most the trace of g(A).
This follows from Jensen's inequality applied to the spectral decomposition.
-/
theorem peierls_inequality (A : HermitianMat d ℂ) (g : ℝ → ℝ) (hg : ConvexOn ℝ Set.univ) :
  ∑ i, g ((A.mat i i).re) ≤ (A.cfc g).trace := by
  -- By the properties of the trace and the convexity of $g$, we have:
  have h_trace_le : ∑ i, g ((A.mat i i).re) ≤ ∑ j, g (A.H.eigenvalues j) *
    ∑ i, ||(A.H.eigenvectorUnitary.val i j)||^2 := by
    -- By the spectral theorem, we can write A as ∑_i λ_i u_i u_i^*,
    -- where λ_i are the eigenvalues and u_i are the corresponding eigenvectors.
    have h_spectral : ∀ i, (A.mat i i).re = ∑ j, A.H.eigenvalues j *
      ||(A.H.eigenvectorUnitary.val i j)||^2 := by
      intro i
      have h_sum : (A.mat i i).re = ∑ j, (A.H.eigenvectorBasis j i) *
        star (A.H.eigenvectorBasis j i) * A.H.eigenvalues j := by
        have this := congr_fun (congr_fun A.H.spectral_theorem i) i
        simp_all [Matrix.mul_apply, Matrix.diagonal]
        simp [Complex.ext_iff, mul_comm, mul_left_comm]
        exact Finset.sum_congr rfl fun _ => by ring
      simp_all [Complex.ext_iff, mul_comm]
      simp [Complex.normSq, Complex.sq_norm]
    have h_jensen : ∀ i, g ((A.mat i i).re) ≤ ∑ j, ||(A.H.eigenvectorUnitary.val i j)||^2 *
      g (A.H.eigenvalues j) := by
      intro i
      have h_convex_comb : ∑ j, ||(A.H.eigenvectorUnitary.val i j)||^2 = 1 := by
        have := congr_fun (congr_fun A.H.eigenvectorUnitary.2.2 i) i
        simp_all [Matrix.mul_apply, Complex.mul_conj, Complex.normSq_eq_norm_sq]
        exact_mod_cast this
      convert hg.map_sum_le _ _ _ <=> simp_all [mul_comm]
    convert Finset.sum_le_sum fun i _ => h_jensen i using 1

```

```

    rw [Finset.sum_comm, Finset.sum_congr rfl]; intros; rw [Finset.mul_sum]
    have h_unitary :  $\forall (j : d), \sum i, \|(A.H.eigenvectorUnitary.val i j)\|^2 = 1$ 
    exact fun j => Matrix.unitaryGroup_row_norm (H A).eigenvectorUnitary j
    simp_all [trace_cfc_eq]

theorem peierls_inequality_ici (A : HermitianMat d  $\mathbb{C}$ ) (g :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (hg : Convex
(hA :  $0 \leq A$ ) :
   $\sum i, g ((A.mat i i).re) \leq (A.cfc g).trace :=$  by
  -- By the properties of the trace and the convexity of  $g$ , we have:
  have h_trace_le :  $\sum i, g ((A.mat i i).re) \leq \sum j, g (A.H.eigenvalues j) *$ 
   $\sum i, \|(A.H.eigenvectorUnitary.val i j)\|^2 :=$  by
  -- By the spectral theorem, we can write  $A$  as  $\sum_i \lambda_i u_i u_i^*$ ,
  -- where  $\lambda_i$  are the eigenvalues and  $u_i$  are the corresponding eigenvectors
  have h_spectral :  $\forall i, (A.mat i i).re = \sum j, A.H.eigenvalues j *$ 
   $\|(A.H.eigenvectorUnitary.val i j)\|^2 :=$  by
  intro i
  have h_sum :  $(A.mat i i).re = \sum j, (A.H.eigenvectorBasis j i) *$ 
   $star (A.H.eigenvectorBasis j i) * A.H.eigenvalues j :=$  by
  have := A.H.spectral_theorem
  replace this := congr_fun (congr_fun this i) i; simp_all [Matrix.mul_sum]
  simp [Complex.ext_iff, mul_comm, mul_left_comm]
  exact Finset.sum_congr rfl fun _ => by ring
  simp_all [Complex.ext_iff, mul_comm]
  simp [Complex.normSq, Complex.sq_norm]
  have h_jensen :  $\forall i, g ((A.mat i i).re) \leq \sum j, \|(A.H.eigenvectorUnitary$ 
   $g (A.H.eigenvalues j) :=$  by
  intro i
  have h_convex_comb :  $\sum j, \|(A.H.eigenvectorUnitary.val i j)\|^2 = 1 :=$ 
  replace this := congr_fun (congr_fun A.H.eigenvectorUnitary.2.2 i)
  simp_all [Matrix.mul_apply, Complex.mul_conj, Complex.normSq_eq_normSq]
  exact_mod_cast this
  convert hg.map_sum_le _ _ _
  · simp_all [mul_comm]
  · simp
  · simp
  · simp
  intro i
  exact A.eigenvalues_nonneg hA i
  convert Finset.sum_le_sum fun i _ => h_jensen i using 1
  rw [Finset.sum_comm, Finset.sum_congr rfl]; intros; rw [Finset.mul_sum]
  have h_unitary :  $\forall (j : d), \sum i, \|(A.H.eigenvectorUnitary.val i j)\|^2 = 1$ 
  exact fun j => Matrix.unitaryGroup_row_norm (H A).eigenvectorUnitary j
  simp_all [trace_cfc_eq]

/--
Joint convexity of the trace functional: for a convex function  $g$ ,

```

the map $A \mapsto \text{tr}(g(A))$ is convex on the space of Hermitian matrices.

-/

```

theorem trace_function_convex_univ (g : ℝ → ℝ) (hg : ConvexOn ℝ Set.univ g) :
  ConvexOn ℝ Set.univ (fun A : HermitianMat d ℂ => (A.cfc g).trace) := by
  refine {convex_univ, ?_}
  intro A _ B _ a b ha hb hab
  -- Let  $C = aA + bB$ .
  set C : HermitianMat d ℂ := a • A + b • B
  -- By the properties of the trace and the convexity of  $g$ , we have:
  have h_trace : (C.cfc g).trace = ∑ i, g (C.H.eigenvalues i) := by
    exact trace_cfc_eq C g
  have h_sum : ∑ i, g (C.H.eigenvalues i) ≤
    a * ∑ i, g ((A.conj (star C.H.eigenvectorUnitary.val)).mat i i).re +
    b * ∑ i, g ((B.conj (star C.H.eigenvectorUnitary.val)).mat i i).re |> Complex.re
  have h_sum : ∀ i, g (C.H.eigenvalues i) ≤
    a * g ((A.conj (star C.H.eigenvectorUnitary.val)).mat i i).re +
    b * g ((B.conj (star C.H.eigenvectorUnitary.val)).mat i i).re := by
    intro i
    have h_eigenvalue : C.H.eigenvalues i =
      a * ((A.conj (star C.H.eigenvectorUnitary.val)).mat i i).re +
      b * ((B.conj (star C.H.eigenvectorUnitary.val)).mat i i).re := by
      have h_eigenvalue : (C.conj (star C.H.eigenvectorUnitary.val)).mat i i =
        a * (A.conj (star C.H.eigenvectorUnitary.val)).mat i i +
        b * (B.conj (star C.H.eigenvectorUnitary.val)).mat i i := by
        simp +zetaDelta at *
        simp [conj]
        exact Complex.ext rfl rfl
      have h_eigenvalue : (C.conj (star C.H.eigenvectorUnitary.val)) =
        (diagonal ℂ C.H.eigenvalues).conj 1 := by
        convert congr_arg (conj (star C.H.eigenvectorUnitary.val) ·) (eq_conj_diag)
        simp [conj_conj]
        simp_all [conj]
        convert congr_arg Complex.re < (diagonal ℂ _) i i = _ > using 1
        · exact Eq.symm (by erw [show (diagonal ℂ _ : HermitianMat d ℂ) i i =
          (C.H.eigenvalues i : ℂ) by exact if_pos rfl]; norm_cast)
        · norm_num [Complex.ext_iff]
      rw [h_eigenvalue]
      exact hg.2 trivial trivial ha hb hab
    simp only [Finset.mul_sum, Finset.sum_add_distrib] using Finset.sum_le_sum fun
  have hAtr : ∑ i, g ((A.conj (star C.H.eigenvectorUnitary.val)).mat i i).re ≤ (A.cfc
    convert peierls_inequality _ _ hg using 1
    convert trace_cfc_conj_unitary _ _ _ using 1
    rotate_right
    exact C.H.eigenvectorUnitary
    simp [conj_conj]
  have hBtr : ∑ i, g ((B.conj (star C.H.eigenvectorUnitary.val)).mat i i).re ≤ (B.cfc

```

```

convert peierls_inequality _ _ hg using 1
convert trace_cfc_conj_unitary _ _ _
rotate_right
exact C.H.eigenvectorUnitary
simp [conj_conj]
simp only [h_trace] using h_sum.trans
  (add_le_add (mul_le_mul_of_nonneg_left hAttr ha) (mul_le_mul_of_nonneg_l

open ComplexOrder in
/--
Convexity of trace functions: if `g` is convex on  $\mathbb{R}_+$ , then  $A \mapsto \text{Tr}[g(A)]$ 
convex on PSD matrices. -/
theorem trace_function_convex_ici {g :  $\mathbb{R} \rightarrow \mathbb{R}$ } (hg : ConvexOn  $\mathbb{R}$  (Set.Ici 0)
  ConvexOn  $\mathbb{R}$  {A : HermitianMat d  $\mathbb{C}$  |  $0 \leq A$ } (fun A => (A.cfc g).trace) :=
  refine (convex_Ici 0, ?_)
  intro A hA B hB a b ha hb hab
  -- Let  $C = aA + bB$ .
  set C : HermitianMat d  $\mathbb{C}$  := a • A + b • B
  -- By the properties of the trace and the convexity of  $g$ , we have:
  have h_trace : (C.cfc g).trace =  $\sum i, g (C.H.eigenvalues i) :=$  by
    exact trace_cfc_eq C g
  have h_sum :  $\sum i, g (C.H.eigenvalues i) \leq$ 
    a *  $\sum i, g ((A.conj (star C.H.eigenvectorUnitary.val)).mat i i).re +$ 
    b *  $\sum i, g ((B.conj (star C.H.eigenvectorUnitary.val)).mat i i).re :=$ 
  have h_sum :  $\forall i, g (C.H.eigenvalues i) \leq$ 
    a * g ((A.conj (star C.H.eigenvectorUnitary.val)).mat i i).re +
    b * g ((B.conj (star C.H.eigenvectorUnitary.val)).mat i i).re := by
    intro i
    have h_eigenvalue : C.H.eigenvalues i =
      a * ((A.conj (star C.H.eigenvectorUnitary.val)).mat i i).re +
      b * ((B.conj (star C.H.eigenvectorUnitary.val)).mat i i).re := by
    have h_eigenvalue : (C.conj (star C.H.eigenvectorUnitary.val)).mat
      a * (A.conj (star C.H.eigenvectorUnitary.val)).mat i i +
      b * (B.conj (star C.H.eigenvectorUnitary.val)).mat i i := by
      simp +zetaDelta only [mat_add, mat_smul, map_add, mat_apply]
      simp only [conj, AddMonoidHom.coe_mk, ZeroHom.coe_mk, mat_smul, A
        Algebra.smul_mul_assoc]
      rfl
    have h_eig2 : (C.conj (star C.H.eigenvectorUnitary.val)) =
      (diagonal  $\mathbb{C}$  C.H.eigenvalues).conj 1 := by
      convert congr_arg (conj (star C.H.eigenvectorUnitary.val) ·) (eq_
        simp [conj_conj]
      simp_all [conj]
    convert congr_arg Complex.re h_eigenvalue using 1
    · exact Eq.symm (by erw [show (diagonal  $\mathbb{C}$  _ : HermitianMat d  $\mathbb{C}$ ) i i
      (C.H.eigenvalues i :  $\mathbb{C}$ ) by exact if_pos rfl]; norm_cast)

```

```

    · norm_num [Complex.ext_iff]
  rw [h_eigenvalue]
  refine hg.2 ?_ ?_ ha hb hab
  · simp
    exact (Complex.le_def.mp (((zero_le_iff.mp (conj_nonneg _ hA)).diag_nonneg (
  · simp
    exact (Complex.le_def.mp (((zero_le_iff.mp (conj_nonneg _ hB)).diag_nonneg (
  simp only [Finset.mul_sum, Finset.sum_add_distrib] using Finset.sum_le_sum fun
-- With convexity of g, we have  $\sum_i g(A_{ii}) \leq \text{Tr}(g(A))$  and  $\sum_i g(B_{ii}) \leq \text{Tr}(g(B))$ 
have hAtr :  $\sum i, g ((A.conj (star C.H.eigenvectorUnitary.val)).mat i i).re \leq (A.cfc$ 
have hA' :  $0 \leq A.conj (star C.H.eigenvectorUnitary.val) := A.conj\_nonneg \_ hA$ 
calc  $\sum i, g ((A.conj (star C.H.eigenvectorUnitary.val)).mat i i |> \text{Complex.re})$ 
   $\leq ((A.conj (star C.H.eigenvectorUnitary.val)).cfc g).trace :=$ 
    peierls_inequality_ici _ _ hg hA'
  _ = (A.cfc g).trace :=
    trace_cfc_conj_unitary A g (star C.H.eigenvectorUnitary.val, by
      rw [Matrix.mem_unitaryGroup_iff, star_star]; exact C.H.eigenvectorUnitary
have hBtr :  $\sum i, g ((B.conj (star C.H.eigenvectorUnitary.val)).mat i i).re \leq (B.cfc$ 
have hB' :  $0 \leq B.conj (star C.H.eigenvectorUnitary.val) := B.conj\_nonneg \_ hB$ 
calc  $\sum i, g ((B.conj (star C.H.eigenvectorUnitary.val)).mat i i |> \text{Complex.re})$ 
   $\leq ((B.conj (star C.H.eigenvectorUnitary.val)).cfc g).trace :=$ 
    peierls_inequality_ici _ _ hg hB'
  _ = (B.cfc g).trace :=
    trace_cfc_conj_unitary B g (star C.H.eigenvectorUnitary.val, by
      rw [Matrix.mem_unitaryGroup_iff, star_star]; exact C.H.eigenvectorUnitary
simp only [h_trace] using h_sum.trans
  (add_le_add (mul_le_mul_of_nonneg_left hAtr ha) (mul_le_mul_of_nonneg_left hBtr
-- /-- Strict convexity of trace functions: if `g` is strictly convex on  $\mathbb{R}_+$ , then
-- `A ↦ Tr[g(A)]` is strictly convex on PSD matrices. -/
-- theorem trace_function_strictConvex {g :  $\mathbb{R} \rightarrow \mathbb{R}$ } (hg : StrictConvexOn  $\mathbb{R}$  (Set.Ici 0
--   (hg_cont : Continuous g) :
--     StrictConvexOn  $\mathbb{R}$  {A : HermitianMat d  $\mathbb{C}$  |  $0 \leq A$ }
--       (fun A => (A.cfc g).trace) := by
--   not needed right now

```

1.134 QuantumInfo/ForMathlib/HermitianMat/Proj.lean

```

public import Mathlib.Analysis.InnerProductSpace.Positive
theorem projector_nonneg :  $0 \leq \text{projector } S$  := by
  rw [zero_le_iff]
  unfold projector
  let P := S.subtypeL.comp S.orthogonalProjection
  have hP : P.toLinearMap.IsSymmetricProjection := by

```

```

    simp [Submodule.starProjection, P] using
      (Submodule.isSymmetricProjection_starProjection (U := S))
    exact LinearMap.posSemidef_toMatrix_iff _ |>.2
      ((LinearMap.IsIdempotentElem.isPositive_iff_isSymmetric hP.1).2 hP.2)

@[simp]
theorem projector_ker : (projector S).ker = S⊥ := by
  ext v
  change (Matrix.toEuclideanLin
    (LinearMap.toMatrix (PiLp.basisFun 2 ⊔ n) (PiLp.basisFun 2 ⊔ n)
      (S.subtypeL.comp S.orthogonalProjection)) v = 0 ↔ v ∈ S⊥)
  rw [show Matrix.toEuclideanLin = Matrix.toLpLin (2 : ENNReal) (2 : ENNReal)
    Matrix.toLpLin_eq_toLin, Matrix.toLin_toMatrix]
  exact Submodule.starProjection_apply_eq_zero_iff (K := S)

@[simp]
theorem supportProj_ker : A.supportProj.ker = A.ker := by
  rw [supportProj, projector_ker, support_orthogonal_eq_range]

@[simp]
theorem kerProj_ker : A.kerProj.ker = A.support := by
  rw [kerProj, projector_ker, ker_orthogonal_eq_support]

theorem posPart_eq_zero_iff : A+ = 0 ↔ A ≤ 0 := by
  refine {fun h => by simp [h] using posPart_le (A := A), fun hA => ?_}
  have hnegPart : (-A)- = A+ := by
    rw [negPart_eq_cfc_ite, posPart_eq_cfc_ite]
    nth_rw 1 [← cfc_id A]
    rw [← cfc_neg, ← cfc_comp]
    congr! 2 with x; simp
  have h0 : ⊥-A, A+⊥ = 0 := by
    simp [hnegPart] using (inner_negPart_zero_iff (A := -
A)).2 (by simp using hA)
  have hA_eq : -A = A- - A+ := by
    conv_lhs => rw [show A = A+ - A- from (posPart_add_negPart A).symm]
    abel
  have hself : ⊥A+, A+⊥ = 0 := by
    rw [hA_eq, HermitianMat.inner_sub_right, HermitianMat.inner_comm A- A+,
      posPart_inner_negPart_zero, zero_sub, neg_eq_zero] at h0
    exact h0
  exact inner_self_eq_zero.mp hself

```

1.135 QuantumInfo/ForMathlib/HermitianMat/Rpow.lean

```
public import QuantumInfo.ForMathlib.HermitianMat.CompoundMatrix
/-! # Matrix Powers With Real Exponents
This file defines real powers of finite-dimensional Hermitian matrices by continuous
functional calculus:
```

```
`A ^ r = A.cfc (fun x => x ^ r)`.
```

The scalar function is ``Real.rpow``, so this is a total operation on Hermitian matrices. For positive semidefinite matrices it has the expected spectral meaning. For negative exponents on singular matrices it follows the ``Real.rpow`` convention, so zero eigenvalues remain zero; use positive-definiteness hypotheses for inverse laws.

Main results:

```
* algebraic and spectral lemmas for real powers;
* continuity in the exponent and in the matrix argument;
* the Loewner-Heinz monotonicity theorem for  $0 < q \leq 1$ ;
* Lieb-Thirring and Rotfel'd trace inequalities for  $0 < p \leq 1$ .
/-- Real powers of Hermitian matrices via continuous functional calculus.
This is the total functional-calculus definition `A.cfc (fun x => x ^ r)`. For positive
semidefinite `A`, this is the usual spectral power. For negative exponents on singular
matrices, zero eigenvalues stay zero because this uses `Real.rpow`. -
/
/-- For positive semidefinite `A`,  $(A^2)^{(p/2)} = A^p$  in functional calculus.
```

The nonnegative exponent assumption is needed for continuity at zero. For positive definite matrices, the same identity should hold for all real p . -

```

(A : HermitianMat d) (hA : 0 ≤ A) (p : ℝ) (hp : 0 ≤ p) :
have h_sqrt : ∀ (f g : ℝ → ℝ), Continuous f → Continuous g → ∀ (A : HermitianMat d)
· exact continuous_id.rpow_const fun x => 0r.inr <| div_nonneg hp (by norm_num)
have h_cont_tq : ContinuousOn (fun t : ℝ => t ^ q) (Set.Icc 0 T) := by
  exact continuousOn_id.rpow_const fun _ => 0r.inr hq
have h_cont_const :
  ContinuousOn (fun t : ℝ => (1 + t)-1 • (1 : HermitianMat d ℂ)) (Set.Icc 0 T) :
  exact ((continuousOn_const.add continuousOn_id).inv₀
    (fun t ht => by
      change (1 : ℝ) + t ≠ 0
      linarith [ht.1])).smul continuousOn_const
have h_integrable (C : HermitianMat d ℂ) (hC : C.mat.PosDef) :
  Integrable
    (fun t => t ^ q • ((1 + t)-1 • (1 : HermitianMat d ℂ) - (C + t • 1)-1))
    (volume.restrict (Set.Ioc 0 T)) := by
have h_const :
  IntervalIntegrable
```

```

      (fun t : ℝ => (1 + t)-1 • (1 : HermitianMat d ℂ)) volume 0 T :=
    h_cont_const.intervalIntegrable_of_Icc hT.le
  have h_diff :
    IntervalIntegrable
      (fun t : ℝ => (1 + t)-1 • (1 : HermitianMat d ℂ) - (C + t • 1)-1)
      volume 0 T :=
    h_const.sub (integrable_inv_shift hC T hT.le)
  exact (h_diff.continuousOn_smul (by simp [Set.uIcc_of_le hT.le] using
    · exact h_integrable A hA
    · exact h_integrable B hB
  /- The resolvent constant is positive for `0 < q < 1`. -
  /
  /- Loewner-Heinz for positive definite lower endpoint.

```

This is the positive-definite core used to prove the positive-semidefinite statement by adding $\varepsilon \cdot 1$ and taking a limit. -/
 /-! ## Araki-Lieb-Thirring Inequality

The next section proves the trace inequality
 $\text{Tr}[(B \wedge r) (A \wedge r) (B \wedge r)] \leq \text{Tr}[(B A B) \wedge r]$ for $0 < r \leq 1$.
 The proof uses compound matrices to convert the multiplicative singular-value estimates into a weak log-majorization argument.
 -/

```

open ComplexOrder MatrixOrder
open scoped AllOrdered

```

```

private lemma compoundHermitian_rpow (A : HermitianMat d ℂ) (hA : 0 ≤ A)
  (k : ℕ) (r : ℝ) :
  compoundHermitian (A ^ r) k = (compoundHermitian A k) ^ r := by
  obtain ⟨U, a, ha, hAeq⟩ :
    ∃ U : Matrix d d, ∃ a : d → ℝ, (∀ i, 0 ≤ a i) ∧ A = conj U.val (diagonal ℂ
      rw [zero_le_iff] at hA
    exact ⟨_, _, hA.eigenvalues_nonneg, eq_conj_diagonal A⟩
  have hdiag_rpow :
    compoundHermitian ((diagonal ℂ a) ^ r) k =
      (compoundHermitian (diagonal ℂ a) k) ^ r := by
    have hdiag (b : d → ℝ) :
      compoundHermitian (diagonal ℂ b) k =
        diagonal ℂ
          (fun S : {S : Finset d // S.card = k} =>
            ∏ i : Fin k, b (S.1.orderEmb0Fin S.2 i)) := by
    ext1
    simp [compoundHermitian, compoundMatrix_diagonal]

```



```

    rw [rpow_diagonal, hdiag, hdiag, rpow_diagonal]
  ext S T
  by_cases h : S = T
  · subst h
    simp [Real.finset_prod_rpow _ _ (fun i _ => ha _)]
  · rw [HermitianMat.diagonal_mat, HermitianMat.diagonal_mat]
    simp [Matrix.diagonal, h]
  rw [hAeq, rpow_conj_unitary, compoundHermitian_conj, compoundHermitian_conj, hdiag]
  exact (rpow_conj_unitary ((diagonal ℂ a).compoundHermitian k) (compoundUnitary U k)

private lemma conj_rpow_le_one_of_conj_le_one_posDef
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : B.mat.PosDef)
  {r : ℝ} (hr0 : 0 < r) (hr1 : r ≤ 1)
  (hAB : A.conj B.mat ≤ 1) :
  (A ^ r).conj (B ^ r).mat ≤ 1 := by
  have hA_le : A ≤ B ^ (-2 : ℝ) := by
  have hconj := HermitianMat.conj_mono (M := (B ^ (-1 : ℝ)).mat) hAB
  have hleft : (A.conj B.mat).conj (B ^ (-1 : ℝ)).mat = A := by
    rw [HermitianMat.conj_conj]
  have hmul : (B ^ (-1 : ℝ)).mat * B.mat = 1 := by
    simpa using rpow_neg_mul_rpow_self hB 1
  exact (HermitianMat.conj_one (A := A)).symm ▸ by simp [hmul]
  have hright : (1 : HermitianMat d ℂ).conj (B ^ (-1 : ℝ)).mat = B ^ (-
2 : ℝ) := by
  ext1
  rw [HermitianMat.conj_apply_mat]
  simp only [HermitianMat.conjTranspose_mat]
  rw [show (B ^ (-2 : ℝ)).mat =
    (B ^ (-1 : ℝ)).mat * (B ^ (-1 : ℝ)).mat by
    rw [← HermitianMat.mat_rpow_add (zero_le_iff.mpr hB.posSemidef)] <=> norm_nu
  simp
  simpa [hleft, hright] using hconj
  have hconj := HermitianMat.conj_mono (M := (B ^ r).mat)
  (rpow_le_rpow_of_le hA hA_le hr0 hr1)
  have hright : ((B ^ (-2 : ℝ)) ^ r).conj (B ^ r).mat = 1 := by
  ext1
  rw [HermitianMat.conj_apply_mat]
  simp only [HermitianMat.conjTranspose_mat]
  have h1 : ((B ^ (-2 : ℝ)) ^ r) = B ^ (-2 * r) := by
    rw [← HermitianMat.rpow_mul (zero_le_iff.mpr hB.posSemidef)]
  rw [h1]
  calc
    (B ^ r).mat * (B ^ (-2 * r)).mat * (B ^ r).mat = (B ^ r).mat * (B ^ (-
r)).mat := by
    rw [Matrix.mul_assoc, ← HermitianMat.mat_rpow_add (zero_le_iff.mpr hB.posSem
  · ring_nf

```

```

    · linarith [hr0.ne']
  _ = 1 := by
    have hInv : (B ^ (-r : ℝ)).mat = (B ^ r).mat-1 := by
      simpa [HermitianMat.mat_inv] using congrArg HermitianMat.mat (rpo
      rw [hInv]
      exact Matrix.mul_nonsing_inv _ (((B ^ r).mat).isUnit_iff_isUnit_det
    exact le_trans hconj hright.le

private lemma conj_smul_right (A B : HermitianMat d ℂ) (c : ℝ) :
  A.conj (↑(c • B).mat) = c ^ 2 • A.conj B.mat := by
  ext1
  rw [HermitianMat.conj_apply_mat]
  simp [HermitianMat.mat_smul, Matrix.conjTranspose_smul, sq, smul_smul]

private lemma conjTranspose_half_mul_eq_conj
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A) :
  ((A ^ (1/2 : ℝ)).mat * B.mat).conjTranspose * ((A ^ (1/2 : ℝ)).mat * B.
  = (A.conj B.mat).mat := by
  have := HermitianMat.pow_half_mul hA
  simp only [Matrix.conjTranspose_mul, HermitianMat.conjTranspose_mat, Matr
  simpa [HermitianMat.conj_apply_mat, Matrix.mul_assoc] using congrArg (fun

private lemma top_singular_le_of_self_mul_le_smul_one
  {e : Type*} [Fintype e] [DecidableEq e] (X : Matrix e e ℂ)
  {α : ℝ} (_ : 0 ≤ α) (hX : X.conjTranspose * X ≤ α • (1 : Matrix e e ℂ))
  (hcard : 0 < Fintype.card e) :
  singularValuesSorted X {0, hcard} ≤ Real.sqrt α := by
  letI : Nonempty e := Fintype.card_pos_iff.mp hcard
  let hne : (Finset.univ : Finset e).Nonempty := by
    simp
  rw [singularValuesSorted_zero_eq_sup X hcard]
  refine Finset.sup'_le _ _ ?_
  intro i hi
  unfold singularValues
  refine Real.sqrt_le_sqrt ?_
  exact (Matrix.PosSemidef.le_smul_one_of_eigenvalues_iff
    (Matrix.isHermitian_mul_conjTranspose_self X.conjTranspose) α).mpr (by

private lemma compound_top_singular_le_posDef
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : B.mat.PosDef)
  {r : ℝ} (hr0 : 0 < r) (hr1 : r ≤ 1)
  (k : ℕ) (hk : k ≤ Fintype.card d) :
  singularValuesSorted (compoundMatrix ((A ^ (r / 2 : ℝ)).mat * (B ^ r).m
  singularValuesSorted (compoundMatrix ((A ^ (1 / 2 : ℝ)).mat * B.mat)
  let Ak : HermitianMat {S : Finset d // S.card = k} ℂ := compoundHermitian
  let Bk : HermitianMat {S : Finset d // S.card = k} ℂ := compoundHermitian

```

```

let Mk : Matrix {S : Finset d // S.card = k} {S : Finset d // S.card = k} ℂ :=
  compoundMatrix ((A ^ (1 / 2 : ℝ)).mat * B.mat) k
let Nk : Matrix {S : Finset d // S.card = k} {S : Finset d // S.card = k} ℂ :=
  compoundMatrix ((A ^ (r / 2 : ℝ)).mat * (B ^ r).mat) k
let c : ℝ := singularValuesSorted Mk (compoundZero k hk)
have hAk : 0 ≤ Ak := compoundHermitian_nonneg A hA k
have hBk : Bk.mat.PosDef := by
  rw [(compoundHermitian B k).H.posDef_iff_eigenvalues_pos]
  obtain ⟨σ, hσ⟩ := compoundHermitian_eigenvalues B k
  intro S
  rw [show (compoundHermitian B k).H.eigenvalues S =
    [] i : Fin k, B.H.eigenvalues ((σ.symm S).1.orderEmbOffFin (σ.symm S).2 i) by
    simp [Function.comp_def] using congrFun hσ (σ.symm S)]
  exact Finset.prod_pos (fun i _ => Matrix.PosDef.eigenvalues_pos hB _)
have hc_nonneg : 0 ≤ c := singularValuesSorted_nonneg Mk (compoundZero k hk)
have hMk_conj : Mk.conjTranspose * Mk = (Ak.conj Bk.mat).mat := by
  dsimp [Mk, Ak, Bk]
  rw [compoundMatrix_mul]
  change ((compoundHermitian (A ^ (1 / 2 : ℝ)) k).mat *
    (compoundHermitian B k).mat).conjTranspose *
    ((compoundHermitian (A ^ (1 / 2 : ℝ)) k).mat *
    (compoundHermitian B k).mat) =
    (((compoundHermitian A k).conj (compoundHermitian B k).mat).mat
  rw [compoundHermitian_rpow A hA k (1 / 2 : ℝ)]
  exact conjTranspose_half_mul_eq_conj (A := compoundHermitian A k)
    (B := compoundHermitian B k) hAk
have hNk_conj : Nk.conjTranspose * Nk = ((Ak ^ r).conj (Bk ^ r).mat).mat := by
  dsimp [Nk, Ak, Bk]
  rw [compoundMatrix_mul]
  change ((compoundHermitian (A ^ (r / 2 : ℝ)) k).mat *
    (compoundHermitian (B ^ r) k).mat).conjTranspose *
    ((compoundHermitian (A ^ (r / 2 : ℝ)) k).mat *
    (compoundHermitian (B ^ r) k).mat) =
    (((compoundHermitian A k) ^ r).conj ((compoundHermitian B k) ^ r).mat).mat
  rw [compoundHermitian_rpow A hA k (r / 2 : ℝ),
    compoundHermitian_rpow B (zero_le_iff.mpr hB.posSemidef) k r]
  rw [show (compoundHermitian A k) ^ (r / 2 : ℝ) = ((compoundHermitian A k) ^ r) ^
    change Ak ^ (r / 2 : ℝ) = (Ak ^ r) ^ (1 / 2 : ℝ)
    rw [← HermitianMat.rpow_mul hAk]; ring_nf]
  exact conjTranspose_half_mul_eq_conj
    (A := (compoundHermitian A k) ^ r) (B := (compoundHermitian B k) ^ r)
    (rpow_nonneg hAk)
have hMk_le : Ak.conj Bk.mat ≤ c ^ 2 • 1 := by
  change (Ak.conj Bk.mat).mat ≤ (c ^ 2 • (1 : HermitianMat {S : Finset d // S.card
  have hmat : Mk.conjTranspose * Mk ≤
    c ^ 2 • (1 : Matrix {S : Finset d // S.card = k} {S : Finset d // S.card = k

```

```

by
  simpa using
    (Matrix.PosSemidef.le_smul_one_of_eigenvalues_iff
     (Matrix.isHermitian_mul_conjTranspose_self Mk.conjTranspose) (c
       fun i => by
         dsimp [c]
         exact eigenvalue_le_singularValuesSorted_sq Mk (compound_ca
    simpa [HermitianMat.conj_apply_mat, hMk_conj] using hmat
by_cases hc_zero : c = 0
· have hMk_eq_zero : Ak.conj Bk.mat = 0 := by
  apply le_antisymm
  · have := hMk_le; rw [hc_zero] at this; simpa using this
  · exact HermitianMat.conj_nonneg (M := Bk.mat) hAk
have hAk_eq_zero : Ak = 0 := by
  have hconj := congrArg (fun H : HermitianMat _  $\mathbb{C}$  => H.conj (Bk ^ (-
1 :  $\mathbb{R}$ )).mat) hMk_eq_zero
have hleft : (Ak.conj Bk.mat).conj (Bk ^ (-1 :  $\mathbb{R}$ )).mat = Ak := by
  rw [HermitianMat.conj_conj]
  have : (Bk ^ (-1 :  $\mathbb{R}$ )).mat * Bk.mat = 1 := by
    simpa using rpow_neg_mul_rpow_self hBk 1
    simp [this, HermitianMat.conj_one (A := Ak)]
  simpa [hleft] using hconj
have hNk_zero : Nk.conjTranspose * Nk = 0 := by
  rw [hNk_conj, hAk_eq_zero]
  simp [rpow_eq_cfc, cfc_apply_zero, Real.zero_rpow hr0.ne']
have htop : singularValuesSorted Nk (compoundZero k hk)  $\leq$  0 := by
  simpa using
    top_singular_le_of_self_mul_le_smul_one Nk (show 0  $\leq$  (0 :  $\mathbb{R}$ ) by non
      (by simp [hNk_zero])
      (compound_card_pos k hk)
  change singularValuesSorted Nk (compoundZero k hk)  $\leq$  c ^ r
  rw [hc_zero, Real.zero_rpow hr0.ne']
  exact htop
· have hc_pos : 0 < c := lt_of_le_of_ne hc_nonneg (Ne.symm hc_zero)
have hscaled : Ak.conj ((c-1) • Bk).mat  $\leq$  1 := by
  have hmul := smul_le_smul_of_nonneg_left hMk_le (show 0  $\leq$  c-1 ^ 2 by
    rw [conj_smul_right])
  have hs : (c ^ 2)-1 * c ^ 2 = 1 := by field_simp [pow_two, hc_zero]
  simpa [smul_smul, hs] using hmul
have hpow := conj_rpow_le_one_of_conj_le_one_posDef hAk
  (by simpa [HermitianMat.mat_smul] using hBk.smul (inv_pos.mpr hc_pos)
    hr0 hr1 hscaled)
have hpow' : ((c-1) ^ r) ^ 2 • ((Ak ^ r).conj (Bk ^ r).mat)  $\leq$  1 := by
  have hBk_scale : ((c-1) • Bk) ^ r = (c-1) ^ r • (Bk ^ r) := by
    rw [show (c-1) • Bk = Bk.cfc (fun x => c-1 * x) from
      (HermitianMat.cfc_const_mul_id (A := Bk) (r := c-1)).symm]

```

```

    rw [HermitianMat.rpow_eq_cfc, ← HermitianMat.cfc_comp]
    calc
      Bk.cfc (((fun x => x ^ r) : ℝ → ℝ) ∘ fun x => c-1 * x)
      = Bk.cfc (fun x => (c-1) ^ r * x ^ r) := by
        apply HermitianMat.cfc_congr_of_nonneg (zero_le_iff.mpr hBk.posSemid)
        intro x hx
        rw [Function.comp_apply, Real.mul_rpow (inv_nonneg.mpr hc_nonneg) hx]
      _ = (c-1) ^ r • (Bk ^ r) := by
        rw [HermitianMat.cfc_const_mul, HermitianMat.rpow_eq_cfc]
    rwa [hBk_scale, conj_smul_right] at hpow
    have hNk_le : ((Ak ^ r).conj (Bk ^ r).mat) ≤ c ^ (2 * r) • 1 := by
    have hmul := smul_le_smul_of_nonneg_left hpow' (show 0 ≤ c ^ (2 * r) by positivity)
    have hs : c ^ (2 * r) * ((c-1) ^ r) ^ 2 = 1 := by
      have hcr_pos : 0 < c ^ r := Real.rpow_pos_of_pos hc_pos r
      have hpow : c ^ (2 * r) = (c ^ r) ^ 2 := by
        calc
          c ^ (2 * r) = c ^ (r + r) := by ring_nf
          _ = c ^ r * c ^ r := by rw [Real.rpow_add hc_pos]
          _ = (c ^ r) ^ 2 := by ring
      have hinv : (c-1) ^ r = (c ^ r)-1 := by
        rw [Real.inv_rpow hc_nonneg]
    calc
      c ^ (2 * r) * ((c-1) ^ r) ^ 2 = (c ^ r) ^ 2 * ((c ^ r)-1) ^ 2 := by
        rw [hpow, hinv]
      _ = 1 := by
        field_simp [pow_two, hcr_pos.ne']
    rwa [smul_smul, hs, one_smul] at hmul
    have htop : singularValuesSorted Nk (compoundZero k hk) ≤ Real.sqrt (c ^ (2 * r))
    top_singular_le_of_self_mul_le_smul_one Nk
    (by positivity)
    (hNk_conj ▸ hNk_le)
    (compound_card_pos k hk)
    have hsqrt : Real.sqrt (c ^ (2 * r)) = c ^ r := by
      have hpow : c ^ (2 * r) = (c ^ r) ^ 2 := by
        rw [mul_comm, Real.rpow_mul hc_nonneg]
      norm_num
    rw [hpow, Real.sqrt_sq_eq_abs, abs_of_nonneg]
    exact Real.rpow_nonneg hc_nonneg r
    exact htop.trans (by rw [hsqrt])

```

/-- Expresses a conjugated trace power as a sum of singular values.

This is private infrastructure for the Lieb-Thirring inequality. -
/

```

private lemma trace_conj_rpow_eq_sum_singularValuesSorted
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (α : ℝ) :

```

```

((A.conj B.mat) ^ α).trace =
  ∑ i : Fin (Fintype.card d),
    singularValuesSorted ((A ^ (1 / 2 : ℝ)).mat * B.mat) i ^ (2 * α) :=
let M : Matrix d d ℂ := (A ^ (1 / 2 : ℝ)).mat * B.mat
let H : HermitianMat d ℂ :=
  (M.conjTranspose * M, by
    simp using Matrix.isHermitian_mul_conjTranspose_self M.conjTranspose)
have hH : H = A.conj B.mat := by
  ext1; dsimp [H, M]
exact conjTranspose_half_mul_eq_conj (A := A) (B := B) hA
calc
  ((A.conj B.mat) ^ α).trace = ∑ i, (H.H.eigenvalues i) ^ α := by
    simp [hH] using trace_rpow_eq_sum H α
  _ = ∑ i : Fin (Fintype.card d), singularValuesSorted M i ^ (2 * α) := b
  rw [← sum_singularValues_rpow_eq_sum_sorted M (2 * α)]
  refine Finset.sum_congr rfl fun i _ => ?_
  unfold singularValues
  have h_nn := Matrix.eigenvalues_conjTranspose_mul_self_nonneg M i
  rw [show H.H.eigenvalues i = Real.sqrt (H.H.eigenvalues i) ^ 2 from
    (Real.sq_sqrt h_nn).symm, ← Real.rpow_natCast _ 2,
    ← Real.rpow_mul (Real.sqrt_nonneg _)]
  push_cast
  congr 1
  simp [H, Matrix.conjTranspose_conjTranspose]

private lemma lieb_thirring_le_one_posDef
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : B.mat.PosDef)
  {r : ℝ} (hr0 : 0 < r) (hr1 : r ≤ 1) :
  ((A ^ r).conj (B ^ r).mat).trace ≤ ((A.conj B.mat) ^ r).trace := by
  classical
  let M : Matrix d d ℂ := (A ^ (1 / 2 : ℝ)).mat * B.mat
  let N : Matrix d d ℂ := (A ^ (r / 2 : ℝ)).mat * (B ^ r).mat
  have hlogmaj :
    ∀ (k : ℕ) (_ : k ≤ Fintype.card d),
      [] i : Fin k, singularValuesSorted N {i.val, by omega} ≤
      [] i : Fin k, (singularValuesSorted M {i.val, by omega}) ^ r := by
  intro k hk
  calc
    [] i : Fin k, singularValuesSorted N {i.val, by omega}
    = singularValuesSorted (compoundMatrix N k) (compoundZero k hk) :=
      simp [N] using prod_singularValuesSorted_eq_compoundSV N k hk
    _ ≤ singularValuesSorted (compoundMatrix M k) (compoundZero k hk) ^ r
      simp [M, N] using compound_top_singular_le_posDef hA hB hr0 hr1
    _ = [] i : Fin k, (singularValuesSorted M {i.val, by omega}) ^ r := by
      have hprod := congrArg (· ^ r) (prod_singularValuesSorted_eq_co
        have hrpow := (Real.finset_prod_rpow (s := Finset.univ)

```

```

      (f := fun i : Fin k => singularValuesSorted M {i.val, by omega}))
      (fun i _ => singularValuesSorted_nonneg M _) r)
    simp using hprod.trans hrpow.symm
  have hsum :
     $\sum i : \text{Fin} (\text{Fintype.card } d), \text{singularValuesSorted } N i ^ 2 \leq$ 
     $\sum i : \text{Fin} (\text{Fintype.card } d), (\text{singularValuesSorted } M i ^ r) ^ 2 := \text{by}$ 
    simp using weak_log_maj_sum_rpow_le
    (n := Fintype.card d)
    (x := singularValuesSorted N)
    (y := fun i : Fin (Fintype.card d) => singularValuesSorted M i ^ r)
    (r := (2 : ℝ))
    (fun i => singularValuesSorted_nonneg N i)
    (fun i => Real.rpow_nonneg (singularValuesSorted_nonneg M i) r)
    (singularValuesSorted_antitone N)
    (rpow_antitone_of_nonneg_antitone
      (singularValuesSorted_antitone M)
      (singularValuesSorted_nonneg M)
      hr0)
    hlogmaj
    (by norm_num)
  have hleft :
    ((A ^ r).conj (B ^ r).mat).trace =
       $\sum i : \text{Fin} (\text{Fintype.card } d), \text{singularValuesSorted } N i ^ 2 := \text{by}$ 
    have := trace_conj_rpow_eq_sum_singularValuesSorted (A := A ^ r) (B := B ^ r) (h
    rw [show ((A ^ r) ^ (1 / 2 : ℝ)) = A ^ (r / 2 : ℝ) from by
      rw [← HermitianMat.rpow_mul hA]; ring_nf] at this
    simp [N] using this
  have hright :
    ((A.conj B.mat) ^ r).trace =
       $\sum i : \text{Fin} (\text{Fintype.card } d), (\text{singularValuesSorted } M i ^ r) ^ 2 := \text{by}$ 
    calc ((A.conj B.mat) ^ r).trace
      =  $\sum i : \text{Fin} (\text{Fintype.card } d), \text{singularValuesSorted } M i ^ (2 * r) := \text{by}$ 
      simp [M] using trace_conj_rpow_eq_sum_singularValuesSorted (A := A) (B
      _ =  $\sum i, (\text{singularValuesSorted } M i ^ r) ^ 2 := \text{Finset.sum\_congr rfl fun i _ =>}$ 
      have hn := singularValuesSorted_nonneg M i
      rw [← Real.rpow_natCast _ 2, ← Real.rpow_mul hn]; push_cast; ring_nf
    simp [hleft, hright] using hsum

/-- Lieb-Thirring type inequality for positive semidefinite Hermitian matrices.

For  $\theta < r \leq 1$ ,
 $\text{Tr}[(B ^ r) (A ^ r) (B ^ r)] \leq \text{Tr}[(B A B) ^ r]$ , written using
`HermitianMat.conj`. -/
{r : ℝ} (hr0 : 0 < r) (hr1 : r ≤ 1) :
  set Bε : ℝ → HermitianMat d ℂ := fun ε => B + ε • 1
  have hPos : ∀ ε > 0, (Bε ε).mat.PosDef := by

```

```

intro ε hε
simp [Bε, add_comm, add_left_comm, add_assoc] using
  ((Matrix.PosDef.one).smul hε).add_posSemidef (HermitianMat.zero_le_if
-- Helper to avoid expensive isDefEq: prove f(0) = target explicitly, the
have tendsto_helper {f : ℝ → HermitianMat d ℂ} (hf : Continuous f)
  {target : HermitianMat d ℂ} (h0 : f 0 = target) :
  Filter.Tendsto (fun ε => (f ε).trace) (nhdsWithin 0 (Set.Ioi 0)) (nhd
rw [← h0]
simp only [HermitianMat.trace_eq_re_trace]
exact (RCLike.continuous_re.comp (HermitianMat.continuous_mat.comp hf)).
have hleft_tendsto :
  Filter.Tendsto
    (fun ε => ((A ^ r).conj ((Bε ε) ^ r).mat).trace)
    (nhdsWithin 0 (Set.Ioi 0))
    (nhds (((A ^ r).conj (B ^ r).mat).trace)) :=
  tendsto_helper
    ((HermitianMat.continuous_conj (p := A ^ r)).comp
      (HermitianMat.continuous_mat.comp ((rpow_const_continuous hr0.le).c
    (by simp [Bε]))
have hright_tendsto :
  Filter.Tendsto
    (fun ε => ((A.conj (Bε ε).mat) ^ r).trace)
    (nhdsWithin 0 (Set.Ioi 0))
    (nhds (((A.conj B.mat) ^ r).trace)) :=
  tendsto_helper
    ((rpow_const_continuous hr0.le).comp
      ((HermitianMat.continuous_conj (p := A)).comp
        (HermitianMat.continuous_mat.comp (by fun_prop))))
    (by simp [Bε]))
exact le_of_tendsto_of_tendsto hleft_tendsto hright_tendsto
  (Filter.eventually_of_mem self_mem_nhdsWithin fun ε hε => by
    simp [Bε] using lieb_thirring_le_one_posDef hA (hPos ε hε) hr0 hr1)
/-! ## Rotfel'd Trace Subadditivity
For positive semidefinite `A`, `B` and `0 < p <= 1`, Rotfel'd's inequality
`Tr[(A + B) ^ p] <= Tr[A ^ p] + Tr[B ^ p]`.
The proof here uses the same resolvent-integral representation as Loewner-
Heinz, plus
monotonicity of the trace after inserting square roots. A stronger version
as a majorization theorem.
open ComplexOrder MatrixOrder MeasureTheory Filter

private lemma trace_conj_mono_sqrt {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (h
  {t : ℝ} (ht : 0 < t) :
  (((A + B + t • 1)⁻¹).conj A.sqrt.mat).trace ≤ (((A + t • 1)⁻¹).conj A.s
  have hInv : (A + B + t • 1)⁻¹ ≤ (A + t • 1)⁻¹ := by
    apply inv_antitone

```



```

    · simpa [add_comm, add_left_comm, add_assoc] using
      ((Matrix.PosDef.one).smul ht).add_posSemidef (HermitianMat.zero_le_iff.mp hA)
    · simpa [add_assoc, add_left_comm, add_comm] using add_le_add_right hB (A + t •
have hTrace := HermitianMat.trace_nonneg (show
  0 ≤ ((A + t • 1)-1).conj A.sqrt.mat - ((A + B + t • 1)-1).conj A.sqrt.mat by
  simp [sub_nonneg] using HermitianMat.conj_mono (M := A.sqrt.mat) hInv)
simpa [HermitianMat.trace_sub] using hTrace

private lemma trace_conj_sqrt_eq_trace_mul_left {A X : HermitianMat d ℂ} (hA : 0 ≤ A
  (((X).conj A.sqrt.mat).trace : ℂ) = (A.mat * X.mat).trace := by
change algebraMap ℝ ℂ ((X).conj A.sqrt.mat).trace) = (A.mat * X.mat).trace
simp [HermitianMat.trace_eq_trace, HermitianMat.conj_apply_mat, HermitianMat.conjT
rw [Matrix.trace_mul_cycle, HermitianMat.sqrt_sq hA]

private lemma scalarRpowApprox_eq_mul_integral_sub_integral {x q T : ℝ}
  (hx : 0 < x) (hq : 0 < q) (hT : 0 < T) :
  scalarRpowApprox q T x =
    x * (∫ t in (0)..T, t ^ (q - 1) / (x + t)) -
      (∫ t in (0)..T, t ^ (q - 1) / (1 + t)) := by
have : ∀ t ∈ Set.Ioc (0 : ℝ) T,
  t ^ q * (1 / (1 + t) - 1 / (x + t)) =
    x * (t ^ (q - 1) / (x + t)) - (t ^ (q - 1) / (1 + t)) := by
intro t ht
rw [Real.rpow_sub_one ht.1.ne']
grind
have hInt (y : ℝ) (hy : 0 < y) :
  Integrable (fun t => t ^ (q - 1) / (y + t)) (volume.restrict (Set.Ioc 0 T)) :=
  apply Integrable.mono' (g := fun t => t ^ (q - 1) / y)
  · exact (intervalIntegral.intervalIntegrable_rpow' (by linarith)).1.div_const _
  · apply Measurable.aestronglyMeasurable
    fun_prop
  · filter_upwards [ae_restrict_mem measurableSet_Ioc] with t ht
    rw [Real.norm_of_nonneg (div_nonneg (Real.rpow_nonneg ht.1.le _) (by linarith
      exact div_le_div_of_nonneg_left (Real.rpow_nonneg ht.1.le _) (by linarith [hy]
        (by linarith [ht.1]))
    rw [HermitianMat.scalarRpowApprox, intervalIntegral.integral_of_le hT.le,
      intervalIntegral.integral_of_le hT.le, intervalIntegral.integral_of_le hT.le]
    rw [← MeasureTheory.integral_const_mul, ← MeasureTheory.integral_sub]
    · exact setIntegral_congr_fun measurableSet_Ioc this
    · exact (hInt x hx).const_mul _
    · simp using hInt 1 zero_lt_one

private lemma tendsto_mul_intervalIntegral_rpow_div_add {x q : ℝ} (hx : 0 < x)
  (hq : 0 < q) (hq1 : q < 1) :
  Filter.Tendsto
    (fun T => x * (∫ t in (0)..T, t ^ (q - 1) / (x + t)))

```

```

    Filter.atTop
    (nhds (rpowConst q * x ^ q)) := by
have hEq :  $\forall^f T$  in Filter.atTop,
  x * ( $\int t$  in (0)..T,  $t^{(q-1)/(x+t)}$ ) =
    scalarRpowApprox q T x + ( $\int t$  in (0)..T,  $t^{(q-1)/(1+t)}$ ) :=
  filter_upwards [Filter.eventually_gt_atTop (0 :  $\mathbb{R}$ )] with T hT
  rw [scalarRpowApprox_eq_mul_integral_sub_integral hx hq hT]
  ring
  rw [Filter.tendsto_congr' hEq]
  convert (scalarRpowApprox_tendsto hx hq hq1).add
  (show Filter.Tendsto
    (fun T =>  $\int t$  in (0)..T,  $t^{(q-1)/(1+t)}$ )
    Filter.atTop
    (nhds (rpowConst q)) from by
    apply MeasureTheory.intervalIntegral_tendsto_integral_Ioi
    · exact rpowConst_integrableOn hq hq1
    · exact Filter.tendsto_id) using 1
  ring_nf

private lemma intervalIntegrable_weighted_div_nonneg {x p T :  $\mathbb{R}$ }
  (hx : 0 ≤ x) (hp : 0 < p) (hT : 0 < T) :
  IntervalIntegrable (fun t =>  $t^{(p-1)} * (x / (x+t))$ ) volume 0 T :=
  rw [intervalIntegrable_iff_integrableOn_Ioc_of_le hT.le]
  apply Integrable.mono' (g := fun t =>  $t^{(p-1)}$ )
  · exact (intervalIntegral.intervalIntegrable_rpow' (by linarith : -
1 < p - 1)).1
  |>.mono_set (Set.Ioc_subset_Ioc le_refl le_refl)
  · apply Measurable.astronglyMeasurable
  fun_prop
  · filter_upwards [ae_restrict_mem measurableSet_Ioc] with t ht
  rw [Real.norm_of_nonneg (mul_nonneg (Real.rpow_nonneg ht.1.le _)
    (div_nonneg hx (by linarith [ht.1]))))]
  simp using mul_le_mul_of_nonneg_left
  ((div_le_one (by linarith [ht.1])).2 (by linarith [ht.1]))
  (Real.rpow_nonneg ht.1.le _)

private lemma tendsto_intervalIntegral_weighted_div_nonneg {x p :  $\mathbb{R}$ }
  (hx : 0 ≤ x) (hp : 0 < p) (hp1 : p < 1) :
  Filter.Tendsto
    (fun T =>  $\int t$  in (0)..T,  $t^{(p-1)} * (x / (x+t))$ )
    Filter.atTop
    (nhds (rpowConst p * x ^ p)) := by
  rcases eq_or_lt_of_le hx with rfl | hx'
  · have hEq :  $\forall^f T$  in Filter.atTop,
    ( $\int t$  in (0)..T,  $t^{(p-1)} * (0 / (0+t))$ ) = 0 := by

```

```

    filter_upwards [Filter.eventually_gt_atTop (0 : ℝ)] with T hT
    apply intervalIntegral.integral_zero_ae
    filter_upwards with t ht
    simp
    rw [Filter.tendsto_congr' hEq]
    simp [Real.zero_rpow hp.ne']
    · have hEq :  $\forall^f T$  in Filter.atTop,
      ( $\int t$  in (0)..T,  $t^{(p-1)} * (x / (x + t))$ ) =
       $x * (\int t$  in (0)..T,  $t^{(p-1)} / (x + t)$ ) := by
      filter_upwards [Filter.eventually_gt_atTop (0 : ℝ)] with T hT
      rw [intervalIntegral.integral_of_le hT.le, intervalIntegral.integral_of_le hT]
      rw [← MeasureTheory.integral_const_mul]
      refine setIntegral_congr_fun measurableSet_Ioc ?_
      intro t ht
      field_simp [hx'.ne', ht.1.ne']
    rw [Filter.tendsto_congr' hEq]
    exact tendsto_mul_intervalIntegral_rpow_div_add hx' hp hp1

private lemma trace_mul_inv_shift_add_le {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB :
{t : ℝ} (ht : 0 < t) :
  (((A + B).mat * (A + B + t • 1).mat⁻¹).trace : ℂ) ≤
  ((A.mat * (A + t • 1).mat⁻¹).trace : ℂ) + ((B.mat * (B + t • 1).mat⁻¹).trace : ℂ)
rw [show (((A + B).mat * (A + B + t • 1).mat⁻¹).trace : ℂ) =
  ((A.mat * (A + B + t • 1).mat⁻¹).trace : ℂ) + ((B.mat * (A + B + t • 1).mat⁻¹).trace : ℂ)
  by simp [HermitianMat.mat_add, Matrix.add_mul, Matrix.trace_add]]
apply add_le_add
· change ((A.mat * ((A + B + t • 1)⁻¹).mat).trace : ℂ) ≤
  ((A.mat * ((A + t • 1)⁻¹).mat).trace : ℂ)
  rw [← trace_conj_sqrt_eq_trace_mul_left hA (X := (A + B + t • 1)⁻¹),
    ← trace_conj_sqrt_eq_trace_mul_left hA (X := (A + t • 1)⁻¹)]
  exact_mod_cast trace_conj_mono_sqrt hA hB ht
· change ((B.mat * ((A + B + t • 1)⁻¹).mat).trace : ℂ) ≤
  ((B.mat * ((B + t • 1)⁻¹).mat).trace : ℂ)
  rw [← trace_conj_sqrt_eq_trace_mul_left hB (X := (A + B + t • 1)⁻¹),
    ← trace_conj_sqrt_eq_trace_mul_left hB (X := (B + t • 1)⁻¹)]
  exact_mod_cast by simp [add_comm, add_left_comm, add_assoc] using
    trace_conj_mono_sqrt hB hA ht

private lemma trace_mul_inv_shift_eq_sum_div {A : HermitianMat d ℂ} (hA : 0 ≤ A) {t
(ht : 0 < t) :
  (((A.mat * (A + t • 1).mat⁻¹).trace : ℂ)).re =
   $\sum i, A.H.eigenvalues\ i / (A.H.eigenvalues\ i + t)$  := by
change (((A.mat * ((A + t • 1)⁻¹).mat).trace : ℂ)).re =
rw [show (A + t • 1)⁻¹ = A.cfc (fun u => (u + t)⁻¹) from by
  rw [show A + t • 1 = A.cfc (fun u => u + t) from by
    rw [show (fun u => u + t) = (fun u => u) + fun _ => t from rfl, cfc_add]]

```

```

    simp]
  apply inv_cfc_eq_cfc_inv
  intro i
  exact ne_of_gt (add_pos_of_nonneg_of_pos (by simp using hA.eigenvalues
calc
  (((A.mat * (A.cfc fun u => (u + t)-1).mat).trace : ℂ)).re
    = ∑ i, A.H.eigenvalues i *
      ((A.H.eigenvalues i + t) / Complex.normSq (↑(A.H.eigenvalues i)
        simp using congrArg Complex.re (HermitianMat.trace_mul_cfc A
_ = ∑ i, A.H.eigenvalues i / (A.H.eigenvalues i + t) := by
  refine Finset.sum_congr rfl ?_
  intro i hi
  have hpos : 0 < A.H.eigenvalues i + t := by
    exact add_pos_of_nonneg_of_pos (by simp using (zero_le_iff.mp
  rw [show ((↑(A.H.eigenvalues i) + ↑t : ℂ)) = ↑(A.H.eigenvalues i
    Complex.normSq_ofReal]
  field_simp [hpos.ne', div_eq_mul_inv]

private lemma intervalIntegrable_weighted_trace_mul_inv_shift {A : Hermitia
  (hA : 0 ≤ A) {p T : ℝ} (hp : 0 < p) (hT : 0 < T) :
  IntervalIntegrable
    (fun t => t ^ (p - 1) * (((A.mat * (A + t • 1).mat-1).trace : ℂ)).re)
    volume 0 T := by
  classical
  let g : ℝ → ℝ := fun t =>
    ∑ i, t ^ (p - 1) * (A.H.eigenvalues i / (A.H.eigenvalues i + t))
  have hg : IntervalIntegrable g volume 0 T := by
    unfold g
    classical
    induction (Finset.univ : Finset d) using Finset.induction_on with
    | empty =>
      simp
    | @insert i s hi hs =>
      simp [Finset.sum_insert hi] using
        (intervalIntegrable_weighted_div_nonneg
          (x := A.H.eigenvalues i) (by simp using (zero_le_iff.mp hA).ei
            hp hT).add hs
  refine hg.congr ?_
  intro t ht
  have htIoc : t ∈ Set.Ioc (0 : ℝ) T := by
    simp [Set.uIoc_of_le hT.le] using ht
  unfold g
  change ∑ i, t ^ (p - 1) * (A.H.eigenvalues i / (A.H.eigenvalues i + t)) =
    t ^ (p - 1) * (((A.mat * (A + t • 1).mat-1).trace : ℂ)).re
  rw [trace_mul_inv_shift_eq_sum_div hA htIoc.1]
  simp [Finset.mul_sum]

```

```

private lemma tendsto_intervalIntegral_weighted_trace_mul_inv_shift {A : HermitianMa
  (hA : 0 ≤ A) (p : ℝ) (hp : 0 < p) (hp1 : p < 1) :
  Filter.Tendsto
    (fun T => ∫ t in (0)..T, t ^ (p - 1) * (((A.mat * (A + t • 1).mat⁻¹).trace : ℂ)
      Filter.atTop
        (nhds (rpowConst p * (A ^ p).trace))) := by
classical
have hEq : ∀f T in Filter.atTop,
  (∫ t in (0)..T, t ^ (p - 1) * (((A.mat * (A + t • 1).mat⁻¹).trace : ℂ)).re) =
    ∑ i, ∫ t in (0)..T, t ^ (p - 1) * (A.H.eigenvalues i / (A.H.eigenvalues i +
  filter_upwards [Filter.eventually_gt_atTop (0 : ℝ)] with T hT
  calc
    (∫ t in (0)..T, t ^ (p - 1) * (((A.mat * (A + t • 1).mat⁻¹).trace : ℂ)).re)
      = ∫ t in (0)..T, ∑ i, t ^ (p - 1) * (A.H.eigenvalues i / (A.H.eigenvalues i
        apply intervalIntegral.integral_congr_ae
        filter_upwards with t hT
        rw [trace_mul_inv_shift_eq_sum_div hA (by
          have hTloc : t ∈ Set.Ioc (0 : ℝ) T := by
            simpa [Set.uIoc_of_le hT.le] using ht
            exact hTloc.1)]
          simp [Finset.mul_sum]
        _ = ∑ i, ∫ t in (0)..T, t ^ (p - 1) * (A.H.eigenvalues i / (A.H.eigenvalues i
          rw [intervalIntegral.integral_finset_sum]
          intro i hi
          exact intervalIntegrable_weighted_div_nonneg
            (x := A.H.eigenvalues i) (by simpa using (zero_le_iff.mp hA).eigenvalu
            hp hT
        rw [Filter.tendsto_congr' hEq]
        simpa [trace_rpow_eq_sum, Finset.mul_sum] using
          (tendsto_finset_sum Finset.univ fun i _ =>
            tendsto_intervalIntegral_weighted_div_nonneg
              (x := A.H.eigenvalues i) (by simpa using (zero_le_iff.mp hA).eigenvalues_non
              hp hp1)

/-- Rotfel'd trace subadditivity for positive semidefinite Hermitian matrices.

For `0 < p <= 1`, `Tr[(A + B) ^ p] <= Tr[A ^ p] + Tr[B ^ p]`. -
/
by_cases hp_eq : p = 1; · subst hp_eq; simp
have hp1' : p < 1 := lt_of_le_of_ne hp1 hp_eq
suffices hmono :
  (fun T => ∫ t in (0)..T, t ^ (p - 1) * (((A + B).mat * (A + B + t • 1).mat⁻¹)
    ≤f[Filter.atTop]
  (fun T => (∫ t in (0)..T, t ^ (p - 1) * (((A.mat * (A + t • 1).mat⁻¹).trace :
    (∫ t in (0)..T, t ^ (p - 1) * (((B.mat * (B + t • 1).mat⁻¹).trace : ℂ)).re

```

```

exact le_of_mul_le_mul_left
  (by
    simpa [mul_add] using le_of_tendsto_of_tendsto
      (tendsto_intervalIntegral_weighted_trace_mul_inv_shift (add_nonneg
        ((tendsto_intervalIntegral_weighted_trace_mul_inv_shift hA p hp h
          (tendsto_intervalIntegral_weighted_trace_mul_inv_shift hB p hp
            hmono)
          (rpowConst_pos hp hp1'))
      filter_upwards [Filter.eventually_gt_atTop (0 : ℝ)] with T hT
    rw [← intervalIntegral.integral_add
      (intervalIntegrable_weighted_trace_mul_inv_shift hA hp hT)
      (intervalIntegrable_weighted_trace_mul_inv_shift hB hp hT)]
    refine intervalIntegral.integral_mono_on hT.le
      (intervalIntegrable_weighted_trace_mul_inv_shift (add_nonneg hA hB) hp
        ((intervalIntegrable_weighted_trace_mul_inv_shift hA hp hT).add
          (intervalIntegrable_weighted_trace_mul_inv_shift hB hp hT)) ?_
    intro t ht
    rcases eq_or_lt_of_le ht.1 with rfl | ht0
  · simp [Real.zero_rpow (sub_ne_zero.mpr hp_eq)]
  · simpa [mul_add] using mul_le_mul_of_nonneg_left
      (show (((A + B).mat * (A + B + t • 1).mat-1).trace : ℂ)).re ≤
        (((A.mat * (A + t • 1).mat-1).trace : ℂ)).re +
          (((B.mat * (B + t • 1).mat-1).trace : ℂ)).re from by
        simpa [Complex.add_re] using (Complex.le_def.mp (trace_mul_inv_shift
          (Real.rpow_nonneg ht0.le _))

```

1.136 QuantumInfo/ForMathlib/HermitianMat/Schatten.lean

```

convert congr_arg _ (A.cfc_sq_rpow_eq_cfc_rpow hA p hp.le) using 1
intro i
  simpa only [one_div, HermitianMat.conjTranspose_mat, HermitianMat.conjTranspose_mat, Matrix.conjTranspose_mul] using
    ((A ^ (1 / 2 : ℝ)).mat * B.mat).eigenvalues_conjTranspose_mul_sel
  simp only [Matrix.trace, Matrix.IsHermitian.cfc, one_div, Matrix.conjTranspose_mat, Matrix.conjTranspose_mat, Complex.coe_algebraMap, Unitary.conjTranspose_mat, Matrix.diag_apply, Complex.re_sum, ge_iff_le]
  simp only [one_div, Matrix.conjTranspose_mul, HermitianMat.conjTranspose_mat, HermitianMat.conjTranspose_mat, HermitianMat.conjTranspose_mat] at h_eig
  simp_all only
  simp only [Matrix.diagonal, Function.comp_apply, one_div, Matrix.conjTranspose_mat, HermitianMat.conjTranspose_mat, Matrix.mul_apply, Matrix.IsHermitian, Matrix.of_apply, mul_ite, mul_zero, Finset.sum_ite_eq', Finset.mem_univ, Matrix.star_apply, RCLike.star_def, Complex.re_sum, Complex.mul_re, Complex.ofReal_im, sub_zero, Complex.conj_re, Complex.mul_im, zero_add]

```

```

    mul_neg, sub_neg_eq_add, h_conj]
  field_simp

```

1.137 QuantumInfo/ForMathlib/HermitianMat/Unitary.lean

```

namespace HermitianMat

```

```

variable {α : Type*} [RCLike α] {n : Type*} [Fintype n] [DecidableEq n]
variable (A B : HermitianMat n α) (U : Matrix.unitaryGroup n α)

```

```

@[simp]
theorem trace_conj_unitary : (conj U.val A).trace = A.trace := by
  simp [Matrix.trace_mul_cycle, conj, ← Matrix.star_eq_conjTranspose, trace]

```

```

@[simp]
theorem le_conj_unitary : A.conj U.val ≤ B.conj U ↔ A ≤ B := by
  rw [← sub_nonneg, ← sub_nonneg (b := A), ← map_sub]
  constructor
  · intro h
    simp [HermitianMat.conj_conj] using conj_nonneg (star U).val h
  · exact fun h ↦ conj_nonneg U.val h

```

```

open RealInnerProductSpace in

```

```

@[simp]
theorem inner_conj_unitary : [A.conj U.val, B.conj U.val] = [A, B] := by
  dsimp [conj]
  simp only [inner_eq_re_trace, mat_mk]
  rw [← mul_assoc, ← mul_assoc, mul_assoc _ _ U.val]
  rw [Matrix.trace_mul_cycle, ← mul_assoc, ← mul_assoc _ _ A.mat]
  simp [← Matrix.star_eq_conjTranspose]

```

```

/--

```

The eigenvalues of a Hermitian matrix conjugated by a unitary matrix are the same as the eigenvalues of the original matrix.

```

-/

```

```

@[simp]
theorem eigenvalues_conj : (A.conj U.val).H.eigenvalues = A.H.eigenvalues := by
  rw [Matrix.IsHermitian.eigenvalues_eq_eigenvalues_iff]
  change (U.val * A.mat * star U.val).charpoly = _
  rw [Matrix.charpoly_mul_comm, ← mul_assoc, U.2.1, one_mul]

```

```

end HermitianMat

```

1.138 QuantumInfo/ForMathlib/Majorization.lean

```

/-- The compound matrix of a unitary matrix is unitary. -
/
lemma compoundMatrix_unitary (U : Matrix d d ℂ)
  (hU : U ∈ Matrix.unitaryGroup d ℂ) (k : ℕ) :
  compoundMatrix U k ∈ Matrix.unitaryGroup {S : Finset d // S.card = k} ℂ
rw [Matrix.mem_unitaryGroup_iff', Matrix.star_eq_conjTranspose,
  ← compoundMatrix_conjTranspose, ← compoundMatrix_mul,
  show UH * U = 1 by simpa using Matrix.mem_unitaryGroup_iff'.mp hU]
simpa using compoundMatrix_diagonal (1 : d → ℂ) k

/-- The `k`-th compound matrix bundled as a unitary. -/
noncomputable def compoundUnitary (U : Matrix.unitaryGroup d ℂ) (k : ℕ) :
  Matrix.unitaryGroup {S : Finset d // S.card = k} ℂ :=
  ⟨compoundMatrix U.val k, compoundMatrix_unitary U.val U.property k⟩

omit [DecidableEq d] in
/-- The index set of the `k`-th compound matrix is nonempty when `k ≤ card`
/
lemma compound_card_pos (k : ℕ) (hk : k ≤ Fintype.card d) :
  0 < Fintype.card {S : Finset d // S.card = k} := by
  classical
  obtain ⟨S, _, hScard⟩ := Finset.exists_subset_card_eq (s := (Finset.univ
    exact Fintype.card_pos_iff.mpr ⟨⟨S, hScard⟩⟩))

/-- The canonical zero index in `Fin (Fintype.card {S : Finset d // S.card
witnessed by `compound_card_pos`. -/
def compoundZero (k : ℕ) (hk : k ≤ Fintype.card d) :
  Fin (Fintype.card {S : Finset d // S.card = k}) :=
  ⟨0, compound_card_pos k hk⟩

· exact compoundMatrix_unitary _ (by simp [Matrix.unitaryGroup]) _
· exact compoundMatrix_unitary _ (by simp [Matrix.unitaryGroup]) _

```

1.139 QuantumInfo/ForMathlib/Matrix.lean

```

open scoped ComplexOrder MatrixOrder

```

```

omit [DecidableEq n] in
/-- The block Gram matrix `[YHY, YHX], [XHY, XHX]]` is positive semidefinite
/
theorem fromBlocks_gram_posSemidef {m n k : Type*} [Fintype m] [Fintype n]
  (X : Matrix k n ℂ) (Y : Matrix k m ℂ) :

```



```

    (fromBlocks (YH * Y) (YH * X) (XH * Y) (XH * X)).PosSemidef := by
  convert posSemidef_conjTranspose_mul_self
    (fromBlocks Y X (0 : Matrix k m ℂ) (0 : Matrix k n ℂ)) using 1
  rw [fromBlocks_conjTranspose, fromBlocks_multiply]
  simp

  · rintro rfl; exact absurd (hA.diag_pos (i := Classical.arbitrary n)) (by simp)

```

1.140 QuantumInfo/ForMathlib/MatrixNorm/TraceNorm.lean

```

public import QuantumInfo.ForMathlib.Majorization
public import QuantumInfo.ForMathlib.HermitianMat.Unitary
theorem traceNorm_neg (A : Matrix m n R) : traceNorm (-A) = traceNorm A := by
/-- The trace norm is invariant under left multiplication by an isometry. -
/
/-- The trace norm is invariant under right multiplication by the adjoint of an isometry. -
/
have hsqrt : CFC.sqrt (u * (AH * A) * uH) = u * CFC.sqrt (AH * A) * uH := by
  have h_conj (B : Matrix m m R) (hB : 0 ≤ B) : 0 ≤ u * B * uH := by
    rw [Matrix.nonneg_iff_posSemidef] at hB
    exact hB.mul_mul_conjTranspose_same u
  apply (CFC.sqrt_eq_iff _ (h_conj _ hA) (h_conj _ (CFC.sqrt_nonneg _))).mpr
  rw [Matrix.mul_assoc, ← Matrix.mul_assoc uH, ← Matrix.mul_assoc uH]
  simp [show uH * u = 1 from hu1]
  rw [← Matrix.mul_assoc, Matrix.mul_assoc u, CFC.sqrt_mul_sqrt_self _ hA]
  rw [hsqrt, Matrix.trace_mul_comm, ← Matrix.mul_assoc]
private theorem traceNorm_isometry_conj {A : Matrix n n R} {u : Matrix m n R}
/-- The trace norm is invariant under unitary conjugation. -
/
have hu := (Matrix.mem_unitaryGroup_iff_isometry U.val).mp U.2
/-- For Hermitian matrices, the trace norm is the sum of absolute eigenvalues.

This is Proposition 9.1.1 in Wilde. -/
  · rw [cfcn_eq_cfc (by fun_prop) (by simp)]
/-- The trace norm is nonnegative. Property 9.1.1 in Wilde. -
/
/-- The trace norm is zero iff the matrix is zero. -/
/-- The trace norm is homogeneous under scalar multiplication. Property 9.1.2 in Wilde. -
/
section complexTraceNorm

variable [DecidableEq n]

omit [Fintype m] [DecidableEq n] in

```

```

private lemma inner_A_mulVec_eq (A : Matrix n n ℂ) (v w : n → ℂ) :
  inner ℂ (WithLp.toLp 2 (A.mulVec v)) (WithLp.toLp 2 (A.mulVec w)) =
    star v ⌊v ((AH * A).mulVec w) := by
  rw [EuclideanSpace.inner_eq_star_dotProduct, dotProduct_comm, Matrix.star
    Matrix.dotProduct_mulVec, Matrix.vecMul_vecMul, Matrix.dotProduct_mulVec]

/-- Singular value decomposition for square complex matrices, with singular
square roots of the eigenvalues of `AH * A`. -/
theorem exists_svd_sqrt_eigenvalues (A : Matrix n n ℂ) :
  let hH : (AH * A).IsHermitian := by
    simp using (Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose)
  ∃ V W : Matrix.unitaryGroup n ℂ,
    A = V.val * Matrix.diagonal (fun i => (Real.sqrt (hH.eigenvalues i) : ℂ))
  let hH : (AH * A).IsHermitian := by
    simp using (Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose)
  let s : n → ℂ := fun i => Real.sqrt (hH.eigenvalues i)
  have hs_ne {i : n} (hi : hH.eigenvalues i ≠ 0) : s i ≠ 0 := by
    dsimp [s]
    exact_mod_cast Real.sqrt_ne_zero'.2
    (lt_of_le_of_ne (Matrix.eigenvalues_conjTranspose_mul_self_nonneg A i)
  let u : n → EuclideanSpace ℂ n := fun i =>
    if hi : hH.eigenvalues i ≠ 0 then
      ((s i)-1 • WithLp.toLp 2 (A.mulVec (hH.eigenvectorBasis i).ofLp))
    else 0
  have hu : Orthonormal ℂ ({i | hH.eigenvalues i ≠ 0}.restrict u) := by
    rw [orthonormal_iff_ite]
    intro i j
    dsimp [u, s]
    have hi' : hH.eigenvalues i.1 ≠ 0 := i.2
    have hj' : hH.eigenvalues j.1 ≠ 0 := j.2
    simp only [hi', hj', not_false_eq_true, if_true]
    rw [inner_smul_left, inner_smul_right, inner_A_mulVec_eq, hH.mulVec_eig]
    by_cases hij : i.1 = j.1
    · cases Subtype.ext hij
      simp [dotProduct_comm, ← EuclideanSpace.inner_eq_star_dotProduct, mul_comm]
      field_simp [show (Real.sqrt (hH.eigenvalues i.1) : ℂ) ≠ 0 by simp [s]
      exact_mod_cast (Real.sq_sqrt (Matrix.eigenvalues_conjTranspose_mul_self_nonneg A i))]
    · simp [hij, dotProduct_comm, ← EuclideanSpace.inner_eq_star_dotProduct, mul_comm]
      orthonormal_iff_ite.mp hH.eigenvectorBasis.orthonormal, mul_comm]
      using (show i ≠ j from fun h => hij (congrArg Subtype.val h))
  obtain (b, hb) :=
    Orthonormal.exists_orthonormalBasis_extension_of_card_eq
      (⌊ := ℂ) (E := EuclideanSpace ℂ n) (ι := n)
      (by simp [finrank_euclideanSpace]) (v := u)
      (s := {i | hH.eigenvalues i ≠ 0}) hu
  let V : Matrix.unitaryGroup n ℂ := ⟨Matrix.of (fun i j ↦ b j i), by

```

```

simp only [Matrix.mem_unitaryGroup_iff]
ext i j
  simp [inner] using b.sum_inner_mul_inner (EuclideanSpace.single i 1) (Euclidean
let W : Matrix.unitaryGroup n ℂ := hH.eigenvectorUnitary
have hAW : A * W.val = V.val * Matrix.diagonal s := by
  ext i j
  have hleft : (A * W.val) i j = A.mulVec (hH.eigenvectorBasis j).ofLp i := by
    simp [Matrix.mul_apply, Matrix.mulVec, dotProduct, W, Matrix.IsHermitian.eigen
  by_cases hj : hH.eigenvalues j = 0
  · have hzero : A.mulVec (hH.eigenvectorBasis j).ofLp = 0 := by
      apply (WithLp.toLp_injective (p := 2))
      exact inner_self_eq_zero.mp (by
        rw [inner_A_mulVec_eq]
        rw [hH.mulVec_eigenvectorBasis j, hj]
        simp)
      rw [hleft, congrFun hzero i]
      simp [Matrix.mul_apply, Matrix.diagonal, V, s, hj]
  · have hbji : b j i = (s j)-1 * A.mulVec (hH.eigenvectorBasis j).ofLp i := by
      simp [u, hj] using congrArg (fun x : EuclideanSpace ℂ n => x.ofLp i) (hb j
      have hs_mul : s j * b j i = A.mulVec (hH.eigenvectorBasis j).ofLp i := by
        rw [hbji]; field_simp [hs_ne hj]
      rw [hleft, ← hs_mul]; simp [Matrix.mul_apply, Matrix.diagonal, V, s, mul_comm]
refine ⟨V, W, ?_⟩
simp [W, Matrix.IsHermitian.eigenvectorUnitary, Matrix.mul_assoc] using
  congrArg (fun X => X * W.valH) hAW

open scoped MatrixOrder in
private lemma traceNorm_eq_sum_sqrt_eigenvalues (A : Matrix n n ℂ) :
  let hH : (AH * A).IsHermitian := by
    simp using (Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose)
  A.traceNorm = ∑ i, Real.sqrt (hH.eigenvalues i) := by
  intro hH
  unfold Matrix.traceNorm
  rw [CFC.sqrt_eq_real_sqrt (AH * A)
    (Matrix.nonneg_iff_posSemidef.mpr A.posSemidef_conjTranspose_mul_self),
    cfc_n_eq_cfc, Matrix.IsHermitian.cfc_eq hH, Matrix.IsHermitian.cfc]
  simp [Matrix.trace_mul_comm, Matrix.mul_assoc]

omit [DecidableEq n] in
/-- The trace norm of a square complex matrix is the sum of its singular values. -
/
theorem traceNorm_eq_sum_singularValues [DecidableEq n] (A : Matrix n n ℂ) :
  A.traceNorm = ∑ i, singularValues A i := by
  let hH : (AH * A).IsHermitian := by
    simp using (Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose)
  rw [traceNorm_eq_sum_sqrt_eigenvalues A]

```

```

    refine Finset.sum_congr rfl ?_
    intro i hi
    simp [singularValues]

omit [DecidableEq n] in
/-- The trace norm of a square complex matrix is the sum of its sorted singular values. -/
theorem traceNorm_eq_sum_singularValuesSorted [DecidableEq n] (A : Matrix n n ℂ) :
  A.traceNorm = ∑ i : Fin (Fintype.card n), singularValuesSorted A i := by
  rw [traceNorm_eq_sum_singularValues]
  simp using (sum_singularValues_rpow_eq_sum_sorted A (1 : ℝ))
section
open scoped Matrix.Norms.L2Operator

omit [DecidableEq n] in
/-- Every singular value is bounded by the operator norm. -/
theorem singularValues_le_opNorm [DecidableEq n] (A : Matrix n n ℂ) (i : n) :
  singularValues A i ≤ ||A|| := by
  letI : Nonempty n := ⟨i⟩
  let hH : (AH * A).IsHermitian := by
    simp using (Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose)
  have hmem : hH.eigenvalues i ∈ spectrum ℝ (AH * A) := by
    rw [hH.spectrum_real_eq_range_eigenvalues]
    exact ⟨i, rfl⟩
  have hsq : singularValues A i * singularValues A i ≤ ||A|| * ||A|| := by
    have hsv_sq : singularValues A i * singularValues A i = hH.eigenvalues
      dsimp [singularValues]
      simp [pow_two] using (Real.sq_sqrt (Matrix.eigenvalues_conjTranspose_mul_self A))
    rw [hsv_sq]
    calc
      hH.eigenvalues i ≤ ||AH * A|| := by
        simp [Real.norm_eq_abs,
          abs_of_nonneg (Matrix.eigenvalues_conjTranspose_mul_self_nonneg A),
          spectrum.norm_le_norm_of_mem hmem]
      _ = ||A|| * ||A|| := Matrix.l2_opNorm_conjTranspose_mul_self A
    exact (sq_le_sq (singularValues_nonneg A i) (norm_nonneg A)).mp (by simp)

omit [DecidableEq n] in
/-- The trace norm is bounded by the operator norm on the left times the trace norm on the right. -/
theorem traceNorm_mul_le_opNorm_traceNorm [DecidableEq n] (A B : Matrix n n ℂ) :
  (A * B).traceNorm ≤ ||A|| * B.traceNorm := by
  classical
  by_cases h : IsEmpty n
  · letI := h

```

```

simp [Subsingleton.elim A 0, Subsingleton.elim B 0]
· letI : Nonempty n := not_isEmpty_iff.mp h
have hcard : 0 < Fintype.card n := Fintype.card_pos_iff.mpr <Nonempty n>
have htop : singularValuesSorted A (0, hcard) ≤ ||A|| := by
  rw [singularValuesSorted_zero_eq_sup A hcard]
  rw [Finset.sup'_le_iff]
  exact fun i _ => singularValues_le_opNorm A i
have hA_bound : ∀ i : Fin (Fintype.card n), singularValuesSorted A i ≤ ||A|| := by
  intro i
  exact ((singularValuesSorted_antitone A) (Fin.zero_le i)).trans htop
calc
(A * B).traceNorm = ∑ i : Fin (Fintype.card n), singularValuesSorted (A * B) i
  rw [traceNorm_eq_sum_singularValuesSorted]
_ ≤ ∑ i : Fin (Fintype.card n), singularValuesSorted A i * singularValuesSorted B i
  simp using (sum_rpow_singularValues_mul_le A B (by positivity : 0 < (1 : ℝ)))
_ ≤ ∑ i : Fin (Fintype.card n), ||A|| * singularValuesSorted B i := by
  refine Finset.sum_le_sum ?_
  intro i hi
  exact mul_le_mul_of_nonneg_right (hA_bound i) (singularValuesSorted_nonneg B i)
_ = ||A|| * ∑ i : Fin (Fintype.card n), singularValuesSorted B i := by
  rw [Finset.mul_sum]
_ = ||A|| * B.traceNorm := by
  rw [traceNorm_eq_sum_singularValuesSorted]

omit [DecidableEq n] in
/-- The trace norm is invariant under conjugate transpose. -
/
theorem traceNorm_conjTranspose (A : Matrix n n ℂ) :
  AH.traceNorm = A.traceNorm := by
  classical
  letI : DecidableEq n := Classical.decEq n
  have hH : (AH * A).IsHermitian := Matrix.isHermitian_conjTranspose_mul_self A
  obtain ⟨V, W, hA⟩ := Matrix.exists_svd_sqrt_eigenvalues A
  set D : Matrix n n ℂ :=
    Matrix.diagonal (fun i => (Real.sqrt (hA.eigenvalues i) : ℂ))
  have hDH : DH = D := by simp [D, Matrix.diagonal_conjTranspose]
  calc
  AH.traceNorm = (W.val * D * V.valH).traceNorm := by
    rw [hA, Matrix.conjTranspose_mul, Matrix.conjTranspose_mul,
      Matrix.conjTranspose_conjTranspose, hDH, ← Matrix.mul_assoc]
  _ = D.traceNorm := traceNorm_isometry_conj
    ((Matrix.mem_unitaryGroup_iff_isometry W.val).mp W.prop).1
    ((Matrix.mem_unitaryGroup_iff_isometry V.val).mp V.prop).1
  _ = (V.val * D * W.valH).traceNorm := (traceNorm_isometry_conj
    ((Matrix.mem_unitaryGroup_iff_isometry V.val).mp V.prop).1
    ((Matrix.mem_unitaryGroup_iff_isometry W.val).mp W.prop).1).symm

```

```

    _ = A.traceNorm := by rw [hA]

omit [Fintype m] [RCLike R] [DecidableEq n] in
/-- The trace norm is bounded by trace norm on the left times operator norm
/
theorem traceNorm_mul_le_traceNorm_opNorm [DecidableEq n] (A B : Matrix n n ℝ) :
  (A * B).traceNorm ≤ A.traceNorm * ||B|| := by
  calc
    (A * B).traceNorm = ((A * B)H).traceNorm := by rw [traceNorm_conjTranspose]
    _ ≤ A.traceNorm * ||B|| := by
      simpa [Matrix.conjTranspose_mul, Matrix.l2_opNorm_conjTranspose,
        traceNorm_conjTranspose, mul_comm] using
        Matrix.traceNorm_mul_le_opNorm_traceNorm BH AH

omit [Fintype m] [RCLike R] [DecidableEq n] in
/-- Multiplication on both sides by contractions does not increase trace norm
/
theorem traceNorm_sandwich_le [DecidableEq n] {S M T : Matrix n n ℂ} (hS :
  (hT : ||T|| ≤ 1) : (S * M * T).traceNorm ≤ M.traceNorm :=
  calc (S * M * T).traceNorm
    ≤ (M * T).traceNorm := by
      exact le_trans
        (by simpa [Matrix.mul_assoc] using Matrix.traceNorm_mul_le_opNorm
          (by simpa using mul_le_mul_of_nonneg_right hS (Matrix.traceNorm_n
            _ ≤ M.traceNorm := by
              exact le_trans (traceNorm_mul_le_traceNorm_opNorm M T)
                (by simpa using mul_le_mul_of_nonneg_left hT (Matrix.traceNorm_n

end

/-- The absolute value of the trace is bounded by the trace norm. -
/
theorem abs_trace_le_traceNorm (A : Matrix n n ℂ) :
  ||A.trace|| ≤ A.traceNorm := by
  let hH : (AH * A).IsHermitian := by
    simpa using (Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose)
  obtain ⟨V, W, hA⟩ := exists_svd_sqrt_eigenvalues A
  set D : Matrix n n ℂ := Matrix.diagonal (fun i => (Real.sqrt (hH.eigenval
  set C : Matrix.unitaryGroup n ℂ := star W * V
  calc
    ||A.trace|| = ||(C.val * D).trace|| := by
      congr 1
      rw [hA]
      change (V.val * D * W.valH).trace = (W.valH * V.val * D).trace
      rw [Matrix.trace_mul_comm, Matrix.mul_assoc]
    _ ≤ ∑ i, ||(C.val * D) i i|| := by

```

```

    simp [Matrix.trace] using norm_sum_le (s := Finset.univ) (f := fun i => (C.val i i) * Real.sqrt (hH.eigenvalues i)) := by
    _ = ∑ i, ||C.val i i|| * Real.sqrt (hH.eigenvalues i) := by
    simp [D, Matrix.mul_apply, Matrix.diagonal, Real.norm_eq_abs, abs_of_nonneg]
    _ ≤ ∑ i, Real.sqrt (hH.eigenvalues i) := by
    exact Finset.sum_le_sum (fun i _ => by
      simp using mul_le_mul_of_nonneg_right
      (entry_norm_bound_of_unitary C.property i i)
      (Real.sqrt_nonneg _))
    _ = A.traceNorm := by
    simp [hH] using (traceNorm_eq_sum_sqrt_eigenvalues A).symm

end complexTraceNorm

/-- For square complex matrices, the trace norm is the maximum of `re (Tr[U * A])`
over unitaries `U`. -/
theorem traceNorm_eq_max_re_tr_U (A : Matrix n n ℂ) :
  IsGreatest {x : ℝ | ∃ U : unitaryGroup n ℂ, Complex.re ((U.val * A).trace) = x}
  classical
  let hH : (AH * A).IsHermitian := by
    simp using (Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose)
  obtain ⟨V, W, hA⟩ :
    ∃ V W : Matrix.unitaryGroup n ℂ,
      A = V.val * Matrix.diagonal (fun i => (Real.sqrt (hH.eigenvalues i) : ℂ)) *
    simp [hH] using exists_svd_sqrt_eigenvalues A
  have htraceNorm : A.traceNorm = ∑ i, Real.sqrt (hH.eigenvalues i) := by
    simp [hH] using traceNorm_eq_sum_sqrt_eigenvalues A
  set D : Matrix n n ℂ := Matrix.diagonal (fun i => (Real.sqrt (hH.eigenvalues i) : ℂ))
  have hVu : V.valH * V.val = 1 := (Matrix.mem_unitaryGroup_iff_isometry V.val).mp V
  have hWu : W.valH * W.val = 1 := (Matrix.mem_unitaryGroup_iff_isometry W.val).mp W
  refine ⟨(W * star V, ?_), ?_⟩
  · calc Complex.re ((W * star V).val * A).trace)
    = Complex.re (D.trace) := by
      rw [hA]; congr 1
      change (W.val * V.valH * (V.val * D * W.valH)).trace = D.trace
      simp [Matrix.mul_assoc, hVu, Matrix.trace_mul_comm, hWu]
    _ = A.traceNorm := by simp [D, Matrix.trace, htraceNorm]
  · rintro _ ⟨U, rfl⟩
    set C : Matrix.unitaryGroup n ℂ := star W * U * V
    rw [show Complex.re ((U.val * A).trace) =
      ∑ i, Real.sqrt (hH.eigenvalues i) * Complex.re (C.val i i) by
      conv_lhs => rw [hA]
    have h1 : (U.val * (V.val * D * W.valH)).trace = (C.val * D).trace := by
      change _ = (W.valH * U.val * V.val * D).trace
      rw [show (U.val * (V.val * D * W.valH)).trace =
        (((U.val * V.val) * D) * W.valH).trace by simp [Matrix.mul_assoc],
        Matrix.trace_mul_comm _ W.valH]

```

```

      simp [Matrix.mul_assoc]
    rw [h1]
    simp [D, Matrix.trace, Matrix.mul_apply, Matrix.diagonal, Complex.mul,
    htraceNorm]
  have hdiag_le : ∀ i, Complex.re (C.val i i) ≤ 1 := fun i =>
    (Complex.re_le_norm _).trans (by
      have hsq : ‖C.val i i‖ ^ 2 ≤ 1 := by
        linarith [(Finset.single_le_sum (f := fun j => ‖C.val i j‖ ^ 2)
          (fun j _ => by positivity) (Finset.mem_univ i)).trans_eq
          (Matrix.unitary_row_sum_norm_sq C.val (Matrix.mem_unitaryGroup_
            nlinarith [norm_nonneg (C.val i i), hsq])
        exact Finset.sum_le_sum fun i _ => by
          nlinarith [hdiag_le i, Real.sqrt_nonneg (hH.eigenvalues i)]

/-- The trace norm satisfies the triangle inequality for square complex mat
/
theorem traceNorm_add_le (A B : Matrix n n ℂ) : (A + B).traceNorm ≤ A.traceNorm + B.traceNorm
  obtain ⟨Uab, h1⟩ := (traceNorm_eq_max_re_tr_U (A + B)).left
  rw [Matrix.mul_add, Matrix.trace_add, Complex.add_re] at h1
  obtain h2 := (traceNorm_eq_max_re_tr_U A).right
  obtain h3 := (traceNorm_eq_max_re_tr_U B).right
  simp only [upperBounds, Set.mem_setOf_eq] at h2 h3
  calc
    _ = RCLike.re ((Uab.1 * A).trace) + RCLike.re ((Uab.1 * B).trace) := h1
    _ ≤ traceNorm A + RCLike.re ((Uab.1 * B).trace) := by
      simp [add_comm] using add_le_add_right
      (h2 (a := RCLike.re ((Uab.1 * A).trace)) ⟨Uab, rfl⟩)
      (RCLike.re ((Uab.1 * B).trace))
    _ ≤ _ := by
      simp [add_comm] using add_le_add_left
      (h3 (a := RCLike.re ((Uab.1 * B).trace)) ⟨Uab, rfl⟩) (traceNorm A)

/-- A positive semidefinite matrix has trace norm equal to its trace. -
/
theorem PosSemidef.traceNorm_eq_trace {A : Matrix m m ℝ} (hA : A.PosSemidef) :
  A.traceNorm = A.trace := by
  /- The trace norm is convex. Property 9.1.5 in Wilde. -/
theorem traceNorm_convex (M N : Matrix n n ℂ) (l : ℝ) (hl : 0 ≤ l ∧ l ≤ 1) :
  ((l : ℂ) • M + ((1 - l) : ℂ) • N).traceNorm ≤ l * M.traceNorm + (1 -
  l) * N.traceNorm := by
  refine (traceNorm_add_le _ _).trans ?_
  nth_rw 1 [← Complex.ofReal_one]
  simp_rw [← Complex.ofReal_sub, Complex.norm_real]
  simp [Real.norm_eq_abs, abs_of_nonneg (hl.1), abs_of_nonneg (sub_nonneg.m

```


1.141 QuantumInfo/ForMathlib/SionMinimax.lean

```
intro; grind [inf_le_iff, le_trans]
grind
```

1.142 QuantumInfo/ForMathlib/Unitary.lean

```
{ σ : Equiv.Perm (Fin n) //
  (hT.conj_unitary_IsSymmetric U).eigenvalues hn = hT.eigenvalues hn ∘ σ } := by
set hS := hT.conj_unitary_IsSymmetric U
suffices heq : hS.eigenvalues hn = hT.eigenvalues hn from ⟨1, by simp [heq]⟩
apply List.ofFn_injective
apply List.Perm.eq_of_sortedGE
  (hS.eigenvalues_antitone hn).sortedGE_ofFn (hT.eigenvalues_antitone hn).sortedGE
rw [← Multiset.coe_eq_coe, ← Fin.univ_val_map, ← Fin.univ_val_map]
refine Multiset.ext.mpr fun a => ?_
have h (R : E →1 [0] E) (hR : R.IsSymmetric) :
  (Finset.univ.val.map (hR.eigenvalues hn)).count a
  = Module.finrank [0] (eigenspace R (a : [0])) := by
  rw [Multiset.count_map, ← hR.card_filter_eigenvalues_eq hn ↑a, Finset.card_def]
  congr 1; ext; simp [eq_comm]
rw [h _ hS, h _ hT, (conj_unitary_eigenspace_equiv T U ↑a).finrank_eq]
```

1.143 QuantumInfo/Measurements/POVM.lean

```
public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Channels.CPTP
public import QuantumInfo.Channels.Dual
public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.Channels.Unbundled
theorem measureForget_eq_kraus (Λ : POVM X d) :
  ) := by
  apply CPTPMap.funext
  intro ρ
  apply MState.ext_m
  rw [CPTPMap.mat_coe_eq_apply_mat (Λ := Λ.measureForget) (ρ := ρ)]
  ext i j
  change (Matrix.traceRight (Λ.measurementMap.map ρ.m)) i j =
    (MatrixMap.of_kraus (fun i => ((Λ.mats i) ^ (1 / 2 : ℝ)).mat)
      (fun i => ((Λ.mats i) ^ (1 / 2 : ℝ)).mat) ρ.m) i j
  rw [measurementMap_apply_matrix]
  simp [Matrix.traceRight, MatrixMap.of_kraus]
  simp_rw [Matrix.sum_apply]
```

```

refine Finset.sum_congr rfl fun x _ => ?_
rw [Finset.sum_eq_single x]
· simp [Matrix.single]
· intro y _ hyx; simp [Matrix.single, hyx]
· simp

```

1.144 QuantumInfo/Operators/Unitary.lean

```
public import QuantumInfo.States.Mixed.MState
```

1.145 QuantumInfo/Regularized.lean

```

public import Mathlib.Topology.Order.LiminfLimsup
private def realNegOrderIso :  $\mathbb{R} \approx_o \mathbb{R}^{o d}$  where
  toEquiv := Equiv.neg  $\mathbb{R}$ 
  map_rel_iff' := by
    intro a b
    change  $(-b : \mathbb{R}) \leq -a \leftrightarrow a \leq b$ 
    simpa using neg_le_neg_iff

private theorem limsup_eq_neg_liminf_neg {fn :  $\mathbb{N} \rightarrow \mathbb{R}$ } {_lb _ub :  $\mathbb{R}$ }Expand c
  (hl :  $\forall n, \_lb \leq fn\ n$ ) (hu :  $\forall n, fn\ n \leq \_ub$ ) :
  Filter.atTop.limsup fn = -Filter.atTop.liminf (fun n => -
fn n) := by
  have hneg : -Filter.atTop.limsup fn = Filter.atTop.liminf (fun n => -
fn n) := by
    have hdual := OrderIso.limsup_apply (f := Filter.atTop) (u := fn) realN
    (hu := Filter.isBoundedUnder_of_eventually_le (f := Filter.atTop) (u
(Filter.Eventually.of_forall hu))
    (hu_co := Filter.isCoboundedUnder_le_of_le Filter.atTop hl)
    (hgu := Filter.isBoundedUnder_of_eventually_le ( $\alpha := \mathbb{R}^{o d}$ ) (f := Filte
(u := fun n => (-fn n :  $\mathbb{R}^{o d}$ )) (Filter.Eventually.of_forall fun n =>
(hgu_co := Filter.isCoboundedUnder_le_of_le ( $\alpha := \mathbb{R}^{o d}$ ) Filter.atTop
(f := fun n => (-fn n :  $\mathbb{R}^{o d}$ )) (x := (-_lb :  $\mathbb{R}^{o d}$ )) fun n => neg_le_n
simpa [Filter.limsup, Filter.liminf, Filter.limsSup, Filter.limsInf, re
congrArg OrderDual.ofDual hdual
linarith

have liminf_neg : Filter.liminf fn Filter.atTop = -(Filter.limsup (-
fn) Filter.atTop) := by
  simp [Filter.limsup_eq, Filter.liminf_eq, Real.sInf_def]
  exact Real.ext_cauchy (congrArg Real.cauchy liminf_neg)
  unfold InfRegularized SupRegularized

```

```

exact limsup_eq_neg_liminf_neg h1 hu
have h2 : Filter.Tendsto fn .atTop (nhds hf.lim) := by
  refine h1.congr' ?_
  filter_upwards [Filter.eventually_ne_atTop 0] with n hn
  have : (n : ℝ) ≠ 0 := Nat.cast_ne_zero.mpr hn
  field_simp
exact h2.liminf_eq.symm

```

1.146 QuantumInfo/ResourceTheory/FreeState.lean

```

public import QuantumInfo.Entropy.VonNeumann
public import QuantumInfo.Entropy.SSA
public import QuantumInfo.Entropy.Relative
public import QuantumInfo.Entropy.DPI
public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Channels.CPTP
public import QuantumInfo.Channels.Dual
public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.Channels.Unbundled
/-- A `ResourcePretheory` is a family of Hilbert spaces closed under tensor products
instance of `Fintype` and `DecidableEq` for each. It forms a pre-
structure then on which to
  discuss resource theories. For instance, to talk about "two-
party scenarios", we could write
  `ResourcePretheory (ℕ × ℕ)`, with `H (a,b) := (Fin a) × (Fin b)`.

```

1.147 QuantumInfo/ResourceTheory/HypothesisTesting.lean

```

public import QuantumInfo.Entropy.VonNeumann
public import QuantumInfo.Entropy.SSA
public import QuantumInfo.Entropy.Relative
public import QuantumInfo.Entropy.DPI
public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Channels.CPTP
public import QuantumInfo.Channels.Dual
public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.Channels.Unbundled
public import QuantumInfo.Measurements.POVm
  exact sandwichedRenyiEntropy_DPI hα.le ρ σ Φ
simp only [← Subtype.coe_lt_coe, Prob.coe_zero] at hb_pos
· simp only [← Subtype.coe_lt_coe, Prob.coe_zero]; positivity

```

1.148 QuantumInfo/ResourceTheory/ResourceTheory.lean

```
public import QuantumInfo.ResourceTheory.FreeState
```

1.149 QuantumInfo/ResourceTheory/SteinsLemma.lean

```
public import QuantumInfo.ResourceTheory.FreeState
public import QuantumInfo.ResourceTheory.HypothesisTesting
public import QuantumInfo.Channels.Pinching
open NNReal ComplexOrder Topology
open ResourcePretheory FreeStateTheory UnitalPretheory UnitalFreeStateTheory
open scoped ENNReal Prob RealInnerProductSpace InnerProductSpace
```

1.150 QuantumInfo/States/Ensemble.lean

```
public import QuantumInfo.States.Mixed.MState
public import Physlib.Meta.Sorry
open scoped RealInnerProductSpace InnerProductSpace
theorem MState.exp_val_pure_eq_one_iff {d : Type*} [Fintype d] [DecidableEq d]
  (p : MState d) (ψ : Ket d) :
  have hpure_inner_prob :  $\langle \text{MState.pure } \psi, \text{MState.pure } \psi \rangle_{\text{Prob}} = 1 :=$ 
    Subtype.ext (by rw [MState.pure_inner]; simp [Braket.dot_self_eq_one])
  have hpure_inner :  $\langle \text{MState.pure } \psi \rangle.M, (\text{MState.pure } \psi).M \rangle = 1 :=$  by
    simp [MState.inner_def] using congrArg (fun p : Prob => (p : ℝ)) hpure_inner_prob
  constructor
  · intro h
    have hp_le :  $\langle p.M, p.M \rangle \leq 1 :=$  by
      have := HermitianMat.inner_le_mul_trace p.nonneg p.nonneg; simp [p.trace]
    have hinner :  $\langle p.M, (\text{MState.pure } \psi).M \rangle = 1 :=$  by simp [MState.exp_val]
    have hsq :  $\langle p.M - (\text{MState.pure } \psi).M, p.M - (\text{MState.pure } \psi).M \rangle =$ 
       $\langle p.M, p.M \rangle - 2 * \langle p.M, (\text{MState.pure } \psi).M \rangle + \langle (\text{MState.pure } \psi).M, (\text{MState.pure } \psi).M \rangle$ 
      simp only [HermitianMat.inner_def, IsMaximalSelfAdjoint.RCLike_selfadjoint, MState.mat_M, RCLike.re_to_complex]
      simp [Matrix.mul_sub, Matrix.sub_mul, Matrix.trace_sub, Matrix.trace_mul]
    exact MState.ext (eq_of_sub_eq_zero (inner_self_eq_zero.mp (le_antisymm (by rw [hsq]; linarith) (p.M - (MState.pure ψ).M).inner_self_nonneg)))
  · rintro rfl; simp [MState.exp_val] using hpure_inner
```

1.151 QuantumInfo/States/Entanglement.lean

```
public import QuantumInfo.States.Pure.Braket
public import QuantumInfo.States.Ensemble
```

```

public import QuantumInfo.Entropy.VonNeumann
public import QuantumInfo.Entropy.SSA
public import QuantumInfo.Entropy.Relative
public import QuantumInfo.Entropy.DPI

```

1.152 QuantumInfo/States/Mixed/Fidelity.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
module

public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Channels.CPTP
public import QuantumInfo.Channels.Dual
public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.Channels.Unbundled
public import Physlib.Meta.Sorry
@[expose] public section

noncomputable section

open BigOperators
open ComplexConjugate
open Kronecker
open scoped Matrix ComplexOrder RealInnerProductSpace InnerProductSpace

variable {d d₂ : Type*} [Fintype d] [DecidableEq d] [Fintype d₂] (p σ : MState d)

namespace MState

/-- The fidelity of two quantum states. This is the quantum version of the Bhattacharya
    coefficient. -/
def fidelity (p σ : MState d) : ℝ :=
  (σ.M.conj p.M.sqrt.mat).sqrt.trace

theorem fidelity_ge_zero : 0 ≤ fidelity p σ := by
  apply HermitianMat.trace_nonneg
  apply HermitianMat.sqrt_nonneg

theorem fidelity_le_one : fidelity p σ ≤ 1 := by
  unfold fidelity

```

```

rw [HermitianMat.sqrt_eq_cfc_rpow_half, ← HermitianMat.rpow_eq_cfc]
calc ((σ.M.conj p.M.sqrt.mat) ^ (1/2 : ℝ)).trace
  ≤ ((σ.M ^ (2/2 : ℝ)).trace ^ (1/2 : ℝ) *
      (p.M.sqrt ^ (2 : ℝ)).trace ^ (1/2 : ℝ)) ^ (2 * (1/2 : ℝ)) :=
    HermitianMat.trace_rpow_conj_le σ.nonneg (HermitianMat.sqrt_nonneg
      (by norm_num) (by norm_num) (by norm_num) (by norm_num))
_ = 1 := by
  have h1 : (σ.M ^ (2/2 : ℝ)).trace = 1 := by
    rw [show (2:ℝ)/2 = 1 from by norm_num, HermitianMat.rpow_one]; ex
  have h2 : (p.M.sqrt ^ (2 : ℝ)).trace = 1 := by
    rw [show p.M.sqrt = p.M ^ (1/2 : ℝ) from by
        rw [HermitianMat.sqrt_eq_cfc_rpow_half, ← HermitianMat.rpow_eq_cfc]
        ← HermitianMat.rpow_mul p.nonneg,
        show (1:ℝ)/2 * 2 = 1 from by norm_num, HermitianMat.rpow_one];
    simp [h2]

/-- The fidelity, as a `Prob` probability with value between 0 and 1. -
/
def fidelity_prob : Prob :=
  {fidelity p σ, {fidelity_ge_zero p σ, fidelity_le_one p σ}}

/-- A state has perfect fidelity with itself. -/
theorem fidelity_self_eq_one : fidelity p p = 1 := by
  simp only [fidelity, HermitianMat.sqrt_eq_cfc_rpow_half]
  conv =>
    enter [1, 1, 1, 2]
    rw [← HermitianMat.cfc_id p.M]
  rw [HermitianMat.cfc_conj, ← HermitianMat.cfc_comp_apply]
  convert p.tr using 2
  convert p.M.cfc_id using 1
  apply HermitianMat.cfc_congr_of_nonneg p.nonneg
  intro x hx
  simp only [one_div, Pi.mul_apply, id_eq, Pi.pow_apply]
  rw [← Real.rpow_two, Real.rpow_inv_rpow hx (by norm_num), ← sq, ← Real.rpow_inv]
  exact Real.rpow_rpow_inv hx (by norm_num)

/-- Fidelity can be rewritten as the trace norm of the product of square roots. -
/
theorem fidelity_eq_traceNorm_sqrt_mul_sqrt (p σ : MState d) :
  fidelity p σ = (σ.M.sqrt.mat * p.M.sqrt.mat).traceNorm := by
  open MatrixOrder in
  rw [fidelity, HermitianMat.sqrt_eq_cfc_rpow_half, HermitianMat.trace_eq_traceNorm,
      Matrix.traceNorm, CFC.sqrt_eq_rpow]
  change RCLike.re (((σ.M.conj p.M.sqrt.mat) ^ (1 / 2 : ℝ)).mat.trace) = _
  rw [show ((σ.M.conj p.M.sqrt.mat) ^ (1 / 2 : ℝ)).mat =
      ((σ.M.conj p.M.sqrt.mat).mat) ^ (1 / 2 : ℝ) by

```

```

    rw [HermitianMat.rpow_eq_cfc, HermitianMat.mat_cfc, CFC.rpow_eq_cfc_real (ha :=
    simp [HermitianMat.conj_apply_mat, Matrix.mul_assoc, (HermitianMat.sqrt_sq σ.nonne

/-- The fidelity is 1 if and only if the two states are the same. -
/
theorem fidelity_eq_one_iff_self : fidelity ρ σ = 1 ↔ ρ = σ := by
  refine (fun h => ?_, fun h => h ▸ fidelity_self_eq_one ρ)
  set A : Matrix d d ℂ := ρ.M.sqrt.mat
  set B : Matrix d d ℂ := σ.M.sqrt.mat
  have hAh : AH = A := by simp [A]
  have hBh : BH = B := by simp [B]
  have hAeq : ρ.m = AH * A := by simpa [hAh] using (HermitianMat.sqrt_sq ρ.nonneg).s
  have hBeq : σ.m = BH * B := by simpa [hBh] using (HermitianMat.sqrt_sq σ.nonneg).s
  obtain (U, hU) := (Matrix.traceNorm_eq_max_re_tr_U (B * A)).left
  have hUB : (U.1 * B)H * (U.1 * B) = BH * B := by
    rw [Matrix.conjTranspose_mul, Matrix.mul_assoc]
    simp [show (U.1)H = star U.1 from rfl, ← Matrix.mul_assoc, U.2.1]
  set z : ℂ := (U.1 * (B * A)).trace
  have hzre : z.re = 1 := hU.trans ((fidelity_eq_traceNorm_sqrt_mul_sqrt ρ σ).symm.t
  have hzconj : z + conj z = 2 := by rw [Complex.add_conj, hzre]; push_cast; ring
  have hz1 : (AH * (U.1 * B)).trace = z := by
    show _ = (U.1 * (B * A)).trace
    rw [hAh, ← Matrix.mul_assoc, Matrix.trace_mul_cycle, Matrix.trace_mul_comm]
  have hz2 : ((U.1 * B)H * A).trace = conj z := by
    rw [show ((U.1 * B)H * A) = (AH * (U.1 * B))H from by
      simp [Matrix.conjTranspose_mul, hAh, hBh]]
    simpa [hz1] using Matrix.trace_conjTranspose (AH * (U.1 * B))
  have hAU : A = U.1 * B := by
    refine sub_eq_zero.mp <| Matrix.trace_conjTranspose_mul_self_eq_zero_iff.mp ?_
    have h_expand : ((A - U.1 * B)H * (A - U.1 * B)).trace =
      (AH * A).trace - (AH * (U.1 * B)).trace - ((U.1 * B)H * A).trace
      + ((U.1 * B)H * (U.1 * B)).trace := by
      simp [sub_eq_add_neg, Matrix.conjTranspose_mul, Matrix.mul_add, Matrix.add_mul
        Matrix.trace_add, Matrix.trace_neg, add_assoc, add_left_comm, add_comm]
    rw [h_expand, hz1, hz2, ← hAeq, ρ.tr', hUB, ← hBeq, σ.tr']
    linear_combination -hzconj
  exact MState.ext_m <| by rw [hAeq, hAU, hUB, ← hBeq]

/-- The fidelity is a symmetric quantity. -/
theorem fidelity_symm : fidelity ρ σ = fidelity σ ρ := by
  simp only [fidelity]
  have expand : ∀ (a b : MState d), (a.M.conj b.M.sqrt.mat).mat =
    (b.M.sqrt.mat * a.M.sqrt.mat) * (a.M.sqrt.mat * b.M.sqrt.mat) := fun a b => by
    simp [HermitianMat.conj_apply_mat, b.M.sqrt.conjTranspose_mat,
      (HermitianMat.sqrt_sq a.nonneg).symm, Matrix.mul_assoc]
  have h_eig := ((σ.M.conj ρ.M.sqrt.mat).H.eigenvalues_eq_eigenvalues_iff

```

```

      (ρ.M.conj σ.M.sqrt.mat).H).mpr (by rw [expand σ ρ, expand ρ σ, Matrix.c
show ((σ.M.conj ρ.M.sqrt.mat).cfc Real.sqrt).trace = ((ρ.M.conj σ.M.sqrt.
rw [HermitianMat.trace_cfc_eq, HermitianMat.trace_cfc_eq, h_eig]

/-- The fidelity cannot decrease under the application of a channel. -
/
@[sorryful]
theorem fidelity_channel_nondecreasing [DecidableEq d₂] (Λ : CPTMap d d₂)
  sorry

--TODO: Real.arccos ∘ fidelity forms a metric (triangle inequality), the Fu
--Matches with classical (squared) Bhattacharyya coefficient
--Invariance under unitaries
--Uhlmann's theorem

end MState

```

1.153 QuantumInfo/States/Mixed/MState.lean

```

public import QuantumInfo.ForMathlib.ContinuousLinearMap
public import QuantumInfo.ForMathlib.ComplexLaplaceTransform
public import QuantumInfo.ForMathlib.ContinuousSup
public import QuantumInfo.ForMathlib.Filter
public import QuantumInfo.ForMathlib.HermitianMat
public import QuantumInfo.ForMathlib.Isometry
public import QuantumInfo.ForMathlib.LinearEquiv
public import QuantumInfo.ForMathlib.MatrixNorm.TraceNorm
public import QuantumInfo.ForMathlib.Matrix
public import QuantumInfo.ForMathlib.Minimax
public import QuantumInfo.ForMathlib.Misc
public import QuantumInfo.ForMathlib.Unitary
public import QuantumInfo.States.Pure.Braket
theorem pure_inner : ∀ pure ψ, pure φ → Prob = ||Braket.dot ψ φ||^2 := by
  simp [MState.inner_def, HermitianMat.inner_def, pure, Matrix.vecMulVec_mu
    Braket.dot_eq_dotProduct, Matrix.trace_smul]
  rw [show ((ψ : d → ℂ) → (φ : Bra d) : d → ℂ) =
    conj (((ψ : Bra d) : d → ℂ) → (φ : d → ℂ)) from by
    change (ψ : d → ℂ) → star (φ : d → ℂ) =
    conj (((ψ : Bra d) : d → ℂ) → (φ : d → ℂ))
  rw [Matrix.dotProduct_star (ψ : d → ℂ) (φ : d → ℂ)]
  congr 1
  simp [Bra.eq_conj] using dotProduct_comm (φ : d → ℂ) (star (ψ : d → ℂ))
  simp [Complex.normSq_apply] using
    Complex.normSq_eq_norm_sq (((ψ : Bra d) : d → ℂ) → (φ : d → ℂ))

```



```

_ = p.m.trace := p.psd.traceNorm_eq_trace
show Continuous (fun p : MState d => p.M.mat)

```

1.154 QuantumInfo/States/Mixed/TraceDistance.lean

```

public import QuantumInfo.States.Mixed.MState
public import QuantumInfo.ForMathlib.ContinuousLinearMap
public import QuantumInfo.ForMathlib.ComplexLaplaceTransform
public import QuantumInfo.ForMathlib.ContinuousSup
public import QuantumInfo.ForMathlib.Filter
public import QuantumInfo.ForMathlib.HermitianMat
public import QuantumInfo.ForMathlib.Isometry
public import QuantumInfo.ForMathlib.LinearEquiv
public import QuantumInfo.ForMathlib.MatrixNorm.TraceNorm
public import QuantumInfo.ForMathlib.Matrix
public import QuantumInfo.ForMathlib.Minimax
public import QuantumInfo.ForMathlib.Misc
public import QuantumInfo.ForMathlib.Unitary
theorem le_one : TrDistance  $\rho$   $\sigma \leq 1$  := by
  have htri := Matrix.traceNorm_add_le  $\rho.m$  ( $-\sigma.m$ )
  simp [TrDistance, sub_eq_add_neg, Matrix.traceNorm_neg,
     $\rho.traceNorm\_eq\_1$ ,  $\sigma.traceNorm\_eq\_1$ ] at htri
  linarith
  rw [← Matrix.traceNorm_neg, neg_sub]

```

1.155 QuantumInfo/States/Pure/BargmannInvariant.lean

```

/-
Copyright (c) 2026 Anand Nambakam. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Anand Nambakam
-/
module

public import QuantumInfo.States.Pure.Braket

/-!
# Bargmann Invariant and Geometric Phase

The Bargmann invariant for three quantum states is the cyclic product
of inner products  $\langle \psi_1 | \psi_2 \rangle \cdot \langle \psi_2 | \psi_3 \rangle \cdot \langle \psi_3 | \psi_1 \rangle$ . Its argument is
the geometric (Pancharatnam-Berry) phase accumulated around the
geodesic triangle in projective Hilbert space.

```

```

## Important definitions
* `bargmannInvariantThree`: the 3-vertex Bargmann invariant  $\Delta_3$ 
* `bargmannPhaseThree`: the geometric phase  $\arg(\Delta_3)$ 

## Important results
* `bargmannInvariantThree_degenerate`: identical states give  $\Delta_3 = 1$ 
* `bargmannPhaseThree_degenerate`: identical states give phase  $= 0$ 
* `bargmannInvariantThree_reverse`: reversing conjugates  $\Delta_3$ 
* `bargmannPhaseThree_reverse`: reversing negates the phase (mod  $2\pi$ )
* `bargmannInvariantThree_cyclic`: cyclic permutation preserves  $\Delta_3$ 
* `bargmannPhaseThree_cyclic`: cyclic permutation preserves the phase
* `norm_bargmannInvariantThree_le_one`:  $\|\Delta_3\| \leq 1$  (via `Braket.norm_dot_1`)

## References
* [V. Bargmann, *Note on Wigner's theorem on symmetry operations*,
  J. Math. Phys. 5, 862–868 (1964)][bargmann1964]
* [S. Pancharatnam, *Generalized theory of interference, and its
  applications*, Proc. Indian Acad. Sci. A 44, 247–262 (1956)][pancharatna]
-/

open Braket Complex

variable {d : Type*} [Fintype d] [DecidableEq d]

noncomputable section

/-- The three-vertex Bargmann invariant:  $\langle \psi_1 | \psi_2 \rangle \cdot \langle \psi_2 | \psi_3 \rangle \cdot \langle \psi_3 | \psi_1 \rangle$ .
This is a gauge-invariant complex number whose argument is the
geometric phase of the geodesic triangle. -/
def bargmannInvariantThree (ψ₁ ψ₂ ψ₃ : Ket d) : ℂ :=
  ⟨ψ₁ || ψ₂⟩ * ⟨ψ₂ || ψ₃⟩ * ⟨ψ₃ || ψ₁⟩

/-- The geometric (Pancharatnam-Berry) phase of three states. -
/
def bargmannPhaseThree (ψ₁ ψ₂ ψ₃ : Ket d) : ℝ :=
  Complex.arg (bargmannInvariantThree ψ₁ ψ₂ ψ₃)

/-! ## Degenerate triangles -/

omit [DecidableEq d] in
/-- The Bargmann invariant of three identical states is 1. -
/
@[simp]
lemma bargmannInvariantThree_degenerate (ψ : Ket d) :
  bargmannInvariantThree ψ ψ ψ = 1 := by

```

```

    unfold bargmannInvariantThree
    simp [Braket.dot_self_eq_one]

omit [DecidableEq d] in
/-- The geometric phase of three identical states is 0. -
/
lemma bargmannPhaseThree_degenerate ( $\psi$  : Ket d) :
  bargmannPhaseThree  $\psi$   $\psi$   $\psi$  = 0 := by
  unfold bargmannPhaseThree; simp [Complex.arg_one]

/-! ## Conjugacy -/

omit [DecidableEq d] in
/-- Reversing the cyclic order conjugates the invariant. -
/
lemma bargmannInvariantThree_reverse ( $\psi_1$   $\psi_2$   $\psi_3$  : Ket d) :
  bargmannInvariantThree  $\psi_3$   $\psi_2$   $\psi_1$  = starRingEnd  $\mathbb{C}$  (bargmannInvariantThree  $\psi_1$   $\psi_2$   $\psi_3$ )
  unfold bargmannInvariantThree
  conv_lhs =>
    rw [Braket.dot_swap_conj  $\psi_2$   $\psi_3$ , Braket.dot_swap_conj  $\psi_1$   $\psi_2$ , Braket.dot_swap_conj
      ← map_mul, ← map_mul]
  congr 1; ring

omit [DecidableEq d] in
/-- Reversing the cyclic order negates the geometric phase (mod  $2\pi$ ). -
/
lemma bargmannPhaseThree_reverse ( $\psi_1$   $\psi_2$   $\psi_3$  : Ket d) :
  (bargmannPhaseThree  $\psi_3$   $\psi_2$   $\psi_1$  : Real.Angle) =
  -(bargmannPhaseThree  $\psi_1$   $\psi_2$   $\psi_3$  : Real.Angle) := by
  unfold bargmannPhaseThree; rw [bargmannInvariantThree_reverse]
  exact Complex.arg_conj_coe_angle (bargmannInvariantThree  $\psi_1$   $\psi_2$   $\psi_3$ )

/-! ## Cyclic symmetry -/

omit [DecidableEq d] in
/-- Cyclic permutation of the three states preserves the Bargmann invariant. -
/
lemma bargmannInvariantThree_cyclic ( $\psi_1$   $\psi_2$   $\psi_3$  : Ket d) :
  bargmannInvariantThree  $\psi_2$   $\psi_3$   $\psi_1$  = bargmannInvariantThree  $\psi_1$   $\psi_2$   $\psi_3$  := by
  unfold bargmannInvariantThree; ring

omit [DecidableEq d] in
/-- Cyclic permutation preserves the geometric phase. -/
lemma bargmannPhaseThree_cyclic ( $\psi_1$   $\psi_2$   $\psi_3$  : Ket d) :
  bargmannPhaseThree  $\psi_2$   $\psi_3$   $\psi_1$  = bargmannPhaseThree  $\psi_1$   $\psi_2$   $\psi_3$  := by
  unfold bargmannPhaseThree; rw [bargmannInvariantThree_cyclic]

```

```

/-! ## Norm bounds -/

omit [DecidableEq d] in
/-- The Bargmann invariant has norm at most 1: each inner product factor
    is bounded by Cauchy-Schwarz on unit vectors. -/
lemma norm_bargmannInvariantThree_le_one (ψ₁ ψ₂ ψ₃ : Ket d) :
  ||bargmannInvariantThree ψ₁ ψ₂ ψ₃|| ≤ 1 := by
  unfold bargmannInvariantThree
  calc ||ψ₁||ψ₂|| * ||ψ₂||ψ₃|| * ||ψ₃||ψ₁||
    = ||ψ₁||ψ₂|| * ||ψ₂||ψ₃|| * ||ψ₃||ψ₁|| := by rw [norm_mul, norm_mul]
  _ ≤ 1 * 1 * 1 := by
    gcongr <=> exact Braket.norm_dot_le_one _ _
  _ = 1 := by ring

```

1.156 QuantumInfo/States/Pure/BlochSphere.lean

```

/-
Copyright (c) 2026 Anand Nambakam. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Anand Nambakam
-/
module

public import QuantumInfo.States.Pure.BargmannInvariant
public import Mathlib.LinearAlgebra.CrossProduct
public import Mathlib.Analysis.InnerProductSpace.PiL2

/-!
# Bloch Sphere

The Bloch sphere is the unit sphere  $S^2$  in  $\mathbb{R}^3$ , used to represent
pure states of a two-level quantum system (qubit). Each point on the
sphere corresponds to a qubit state up to global phase.

This file defines the Bloch sphere type and its angular parameterization,
then builds the solid angle and dot product API on sphere points.

## Important definitions
* `BlochSphere`: the unit sphere `Metric.sphere 0 1` in `EuclideanSpace ℝ`
* `blochPoint`: point on the Bloch sphere from polar angle  $\alpha$  and azimuthal
* `solidAngle`: solid angle of a geodesic triangle via Van Vleck formula

```

```

## Important results
* `blochPoint_norm`: the Bloch vector has unit norm
* `dot_blochPoint`: dot product of Bloch vectors in terms of angle differences

## References
* [S. Pancharatnam, *Generalized theory of interference, and its
  applications*, Proc. Indian Acad. Sci. A 44, 247–262 (1956)][pancharatnam1956]
* [M. V. Berry, *Quantal phase factors accompanying adiabatic changes*,
  Proc. R. Soc. London A 392, 45–57 (1984)][berry1984]
-/

open Complex Matrix

noncomputable section

/-- The Bloch sphere: unit sphere in `EuclideanSpace ℝ (Fin 3)` . -
/
abbrev BlochSphere := Metric.sphere (0 : EuclideanSpace ℝ (Fin 3)) 1

namespace BlochSphere

/-- The raw Bloch vector for angles  $(\alpha, \theta)$ . Internal helper for `blochPoint`. -
/
private def blochVecRaw  $(\alpha \theta : \mathbb{R}) : \text{Fin } 3 \rightarrow \mathbb{R} :=$ 
  ![Real.sin  $\alpha$  * Real.cos  $\theta$ , Real.sin  $\alpha$  * Real.sin  $\theta$ , Real.cos  $\alpha$ ]

/-- The Bloch vector has unit norm:  $\sin^2\alpha(\cos^2\theta + \sin^2\theta) + \cos^2\alpha = 1$ . -
/
private lemma blochVecRaw_norm  $(\alpha \theta : \mathbb{R}) :$ 
  Real.sqrt ((blochVecRaw  $\alpha \theta$  0) ^ 2 + (blochVecRaw  $\alpha \theta$  1) ^ 2 +
    (blochVecRaw  $\alpha \theta$  2) ^ 2) = 1 := by
  have : (blochVecRaw  $\alpha \theta$  0) ^ 2 + (blochVecRaw  $\alpha \theta$  1) ^ 2 +
    (blochVecRaw  $\alpha \theta$  2) ^ 2 = 1 := by
    simp [blochVecRaw, Fin.sum_univ_three]
    have h1 := Real.sin_sq_add_cos_sq  $\alpha$ 
    have h2 := Real.sin_sq_add_cos_sq  $\theta$ 
    nlinarith [sq_nonneg (Real.sin  $\alpha$  * Real.cos  $\theta$ ),
      sq_nonneg (Real.sin  $\alpha$  * Real.sin  $\theta$ ), sq_nonneg (Real.cos  $\alpha$ ),
      sq_abs (Real.sin  $\alpha$ ), sq_abs (Real.cos  $\theta$ )]
  rw [this, Real.sqrt_one]

/-- A point on the Bloch sphere parameterized by polar angle ` $\alpha$ ` and
  azimuthal angle ` $\theta$ `. -/
def blochPoint  $(\alpha \theta : \mathbb{R}) : \text{BlochSphere} :=$ 
  (⟨(WithLp.equiv 2 _).symm (blochVecRaw  $\alpha \theta$ ), by
    rw [Metric.mem_sphere, dist_comm, EuclideanSpace.dist_eq]
```

```

simp [EuclideanSpace.norm_eq, Fin.sum_univ_three, sub_zero, blochVecRaw
      Real.sqrt_one])

/-- The underlying vector of a `blochPoint`. -/
lemma blochPoint_val (α θ : ℝ) :
  (blochPoint α θ : EuclideanSpace ℝ (Fin 3)) =
    (WithLp.equiv 2 _).symm (blochVecRaw α θ) := rfl

/-! ## Solid angle via Van Vleck formula -/

/-- Solid angle of a geodesic triangle on the Bloch sphere, computed via
    the Van Vleck formula using `Complex.arg` for full-quadrant support.

     $\Omega = 2 \cdot \arg(\text{den} + \text{num} \cdot i)$  where
     $\text{den} = 1 + n_1 \cdot n_2 + n_2 \cdot n_3 + n_3 \cdot n_1$  and  $\text{num} = n_1 \cdot (n_2 \times n_3)$ . -
/
def solidAngle (p1 p2 p3 : BlochSphere) : ℝ :=
  let n1 := (p1 : EuclideanSpace ℝ (Fin 3))
  let n2 := (p2 : EuclideanSpace ℝ (Fin 3))
  let n3 := (p3 : EuclideanSpace ℝ (Fin 3))
  let num := n1  $\square_v$  n2  $\times_3$  n3
  let den := 1 + n1  $\square_v$  n2 + n2  $\square_v$  n3 + n3  $\square_v$  n1
  2 * Complex.arg ((den : ℝ) + (num : ℝ) * Complex.I)

/-! ## Dot product of Bloch vectors -/

/-- The dot product of two Bloch points expressed in angle differences. -
/
lemma dot_blochPoint (α1 θ1 α2 θ2 : ℝ) :
  (blochPoint α1 θ1 : EuclideanSpace ℝ (Fin 3))  $\square_v$ 
  (blochPoint α2 θ2 : EuclideanSpace ℝ (Fin 3)) =
    Real.sin α1 * Real.sin α2 * Real.cos (θ2 - θ1) +
    Real.cos α1 * Real.cos α2 := by
  simp [blochPoint_val, dotProduct, blochVecRaw, Fin.sum_univ_three, WithLp
        rw [show Real.cos (θ2 - θ1) = Real.cos θ1 * Real.cos θ2 +
            Real.sin θ1 * Real.sin θ2 from by rw [Real.cos_sub]; ring]
  ring

end BlochSphere

```

1.157 QuantumInfo/States/Pure/Braket.lean

```

public import QuantumInfo.ForMathlib.ContinuousLinearMap
public import QuantumInfo.ForMathlib.ComplexLaplaceTransform

```

```

public import QuantumInfo.ForMathlib.ContinuousSup
public import QuantumInfo.ForMathlib.Filter
public import QuantumInfo.ForMathlib.HermitianMat
public import QuantumInfo.ForMathlib.Isometry
public import QuantumInfo.ForMathlib.LinearEquiv
public import QuantumInfo.ForMathlib.MatrixNorm.TraceNorm
public import QuantumInfo.ForMathlib.Matrix
public import QuantumInfo.ForMathlib.Minimax
public import QuantumInfo.ForMathlib.Misc
public import QuantumInfo.ForMathlib.Unitary
/-- Swapping the arguments conjugates the bra-ket product:
     $\langle \psi_2 | \psi_1 \rangle = \text{conj}(\langle \psi_1 | \psi_2 \rangle)$ . -/
lemma Braket.dot_swap_conj (ψ₁ ψ₂ : Ket d) :
   $\langle \psi_2 | \psi_1 \rangle = \text{starRingEnd } \mathbb{C} \langle \psi_1 | \psi_2 \rangle$  := by
  simp +decide [Braket.dot]
  ac_rfl

/-! ## Norm bounds -/

section norm_bounds
open Braket

variable {d : Type*} [Fintype d]

private def ketToEuclidean (ψ : Ket d) : EuclideanSpace  $\mathbb{C}$  d :=
  (WithLp.equiv 2 _).symm ψ.vec

private lemma ketToEuclidean_norm (ψ : Ket d) :  $\| \text{ketToEuclidean } \psi \| = 1$  := by
  rw [EuclideanSpace.norm_eq]; convert Real.sqrt_one
  simp only [ketToEuclidean]; convert ψ.normalized'

private lemma dot_eq_euclidean_inner (ψ₁ ψ₂ : Ket d) :
   $\langle \psi_1 | \psi_2 \rangle = @\text{inner } \mathbb{C} (\text{EuclideanSpace } \mathbb{C} d) \_$ 
    (ketToEuclidean ψ₁) (ketToEuclidean ψ₂) := by
  unfold dot ketToEuclidean
  simp only [Bra.eq_conj, PiLp.inner_apply]
  congr 1; ext x
  rw [RCLike.inner_apply']
  simp [WithLp.equiv, Ket.apply]

/-- The bra-ket product of normalized states has norm at most 1 (Cauchy-
Schwarz). -/
lemma Braket.norm_dot_le_one (ψ₁ ψ₂ : Ket d) :  $\| \langle \psi_1 | \psi_2 \rangle \| \leq 1$  := by
  rw [dot_eq_euclidean_inner]
  calc  $\| @\text{inner } \mathbb{C} (\text{EuclideanSpace } \mathbb{C} d) \_ (\text{ketToEuclidean } \psi_1) (\text{ketToEuclidean } \psi_2) \|$ 

```

```

    ≤ ||ketToEuclidean  $\psi_1$ || * ||ketToEuclidean  $\psi_2$ || := norm_inner_le_norm _ _
    _ = 1 := by rw [ketToEuclidean_norm, ketToEuclidean_norm, mul_one]

end norm_bounds

```

1.158 QuantumInfo/States/Pure/Qubit.lean

```

public import QuantumInfo.Channels.Bundled
public import QuantumInfo.Channels.CPTP
public import QuantumInfo.Channels.Dual
public import QuantumInfo.Channels.MatrixMap
public import QuantumInfo.Channels.Unbundled
public import Physlib.Meta.TODO.Basic
TODO "Improve the module doc-string of the `Qubit` file, to explain the
      current implementation."

```

1.159 docs/WildeTODO.md

```

□ `Bra` and `Ket` in `QuantumInfo.States.Pure.Braket`.
□ `Ket.IsEntangled` in `QuantumInfo.States.Pure.Braket`.
□ `MState.spectrum` is a `Distribution`, in `QuantumInfo.States.Mixed.MState`.
□ `MState` in `QuantumInfo.States.Mixed.MState`.
□ `MState.uniform` in `QuantumInfo.States.Mixed.MState`.
□ `MState.instMixable` in `QuantumInfo.States.Mixed.MState`.
□ `MState.purity` in `QuantumInfo.States.Mixed.MState`.
□ `MState.pure_iff_purity_one` in `QuantumInfo.States.Mixed.MState`.
□ `POVM` in `QuantumInfo.Measurements.POVM`.
□ `MState.prod` in `QuantumInfo.States.Mixed.MState` constructs product states.
□ `MState.IsSeparable` in `QuantumInfo.States.Mixed.MState`.
□ `MState.IsEntangled` in `QuantumInfo.States.Mixed.MState`.

```

1.160 docs/references.bib

```

@Book{
  hall2013quantum,
  author    = "Hall, Brian C.",
  title     = "{Quantum Theory for Mathematicians}",
  publisher = "Springer",
  year      = "2013"
}

```



```

@Article{      robertson1929uncertainty,
  author       = "Robertson, H. P.",
  title        = "{The Uncertainty Principle}",
  doi          = "10.1103/PhysRev.34.163",
  journal      = "Phys. Rev.",
  volume       = "34",
  pages        = "163--164",
  year         = "1929"
}

@Article{      schrodinger1930heisenberg,
  author       = "Schr{\o}dinger, Erwin",
  title        = "{Zum Heisenbergschen Unscharfeprinzip}",
  journal      = "Sitzungsberichte der Preussischen Akademie der Wissenschaften,
                Physikalisch-mathematische Klasse",
  pages        = "296--303",
  year         = "1930"
}

```

1.161 scripts/MetaPrograms/TODO_to_yaml.lean

```

| QuantumInfo
| QuantumInfo => "Quantum Information"
| QuantumInfo => "□"
  PhyslibCategory.QuantumInfo,
else if n.toString.startsWith "Physlib.QuantumInfo" then
  PhyslibCategory.QuantumInfo
let env ← importModules (loadExts := true) #[`Physlib, `QuantumInfo] {} 0

```

1.162 scripts/MetaPrograms/sorry_lint.lean

```

let env ← importModules (loadExts := true) #[`Physlib, `QuantumInfo] {} 0

```

1.163 scripts/MetaPrograms/spellingWords.txt

```

succSuccAbove
predPredAbove
funPredPredAbove

```

1.164 scripts/lint_all.lean

```
println! sorryPseudoCheck.stderr
```